Spring Boot

Beyond the Basics: Production Services

Container internals, JPA and Hibernate in depth, Spring Security with JWT and OAuth2, microservices and resilience, testing, observability, Docker and Kafka, with diagrams, complete code and production incidents shown from symptom to diagnosis, fix and prevention.

Bean Lifecycle · AOP Proxies · Auto-configuration · Hibernate · N+1 · Transactions
Security Filters · JWT · OAuth2 · Gateway · Resilience4j · Testcontainers · Kafka

```
$ curl -H 'Authorization: Bearer eyJhbGci...' :8080/api/orders/42
  DispatcherServlet -> SecurityFilterChain
  JwtAuthenticationFilter  OK  sub=maruthi
  OrderService.find() @Transactional(readOnly)
  HikariPool-1 - connection acquired 0.4 ms
  200 OK  12 ms
```


Maruthi Sreeram

Prepared for Java full stack interview preparation

Maruthi Sreeram

Version basis

Targets Spring Boot 3.x on JDK 21. Spring artefacts could not be downloaded on the build machine; Spring code and output are labelled typical where not run; plain-JDK simulations were run
Prepared for Maruthi Sreeram

How to use this guide

- **Who this is for.** A Java full stack developer who must explain, and diagnose, what the operating system does beneath a running service: process and thread management, memory, files, I/O, containers and synchronisation.
- **Every chapter follows the same steps:** (1) the concept and the problem it solves, (2) the mechanism, shown with diagrams, (3) a worked trace or calculation, (4) a hands-on experiment with real commands, C or Java code, (5) **real-world incidents**, each presented as symptoms, diagnosis, root cause, fix and prevention, and (6) the questions interviewers ask, with answers.
- **How to study.** Read each diagram before the prose and narrate it aloud. Reproduce every experiment on a Linux machine or virtual machine (never on a production host). Treat each incident as an exercise: cover the diagnosis, decide which command to run first, then compare with the text.
- **Running the code.** Save a Java listing as `Main.java` and run `java Main.java`; compile C with `gcc -O0 -o demo demo.c`. Terminal sessions begin with `$`. Output shown was produced on the build machine unless labelled typical.
- **Scope.** The focus is Linux, the platform on which most Java services run, together with the JVM and container behaviour that interacts with it. Windows and macOS are mentioned only where the difference matters.

Contents

Part 1: Container and Web Internals 4

1.1 The Spring Container Internals 4

1.2 AOP and Proxies 16

1.3 Auto-Configuration and Configuration in Depth 26

1.4 The Spring MVC Request Lifecycle 37

1.5 HTTP Clients: RestClient, RestTemplate, WebClient and HTTP Interfaces 55

Part 2: Data Access in Depth 63

2.1 JPA and Hibernate Architecture 63

2.2 Mapping in Depth 76

2.3 Fetching and the N+1 Family 86

2.4 Transactions in Spring 95

2.5 Performance of the Data Layer 108

2.6 Query Techniques and Spring Data Internals 119

Part 3: Spring Security 128

3.1 Security Architecture: Filters, Authentication and Password Storage 128

3.2 Authorisation: Request Rules, Method Security and Ownership 144

3.3 Web Attacks and Defences 156

3.4 Sessions and Tokens: Choosing an Authentication State Model 168

3.5 JSON Web Tokens in Depth 171

3.6 OAuth 2.0 and OpenID Connect 190

3.7 Securing Microservices and Testing Security 199

Part 4: Microservices and Resilience 206

4.1 Microservice Architecture 206

4.2 Synchronous Communication 218

sreeram

4.3 Service Discovery, Configuration and Gateways . . . 229
4.4 Resilience: Timeouts, Retries, Circuit Breakers and Bulkheads . . . 242
4.5 Distributed Data and Transactions . . . 256
4.6 Distributed Tracing and Correlated Logging . . . 272
4.7 API Design for Services . . . 279

Part 5: Testing, Observability and Performance . . . 286

5.1 Testing Strategy and Unit Tests . . . 286
5.2 Spring Slice Tests and Context Caching . . . 296
5.3 Integration Testing with Testcontainers . . . 305
5.4 Actuator and Micrometer . . . 315
5.5 Logging in Spring Boot Applications . . . 328
5.6 Performance Tuning . . . 336

Part 6: Messaging, Delivery and Interview Preparation . . . 350

6.1 Messaging with Spring and Kafka . . . 350
6.2 RabbitMQ, Scheduling, Async Work and Events . . . 368
6.3 Caching with Redis in Spring . . . 375
6.4 Packaging and Containers . . . 381
6.5 Kubernetes for Spring Developers . . . 388
6.6 CI/CD, Maven and Release Strategies . . . 395
6.7 Interview Question Bank . . . 401
6.8 System Design Scenarios for Spring Services . . . 407
6.9 Cheat Sheets . . . 412
6.10 Six-Week Study Plan . . . 417
6.11 Putting it Together: One Request through a Spring Boot Microservice . . . 419

sreeram

PART 1

Container and Web Internals

This Part opens the framework: the container that builds beans, the proxies that add behaviour, the configuration machinery, and the web stack that serves requests.

In this part

1.1 The Spring Container Internals
1.2 AOP and Proxies
1.3 Auto-Configuration and Configuration in Depth
1.4 The Spring MVC Request Lifecycle
1.5 HTTP Clients: RestClient, RestTemplate, WebClient and HTTP Interfaces

1.1 The Spring Container Internals

The introductory material describes dependency injection as a service: classes declare what they need and the container supplies it. This chapter examines the machinery behind that service. Every production defect that mentions a "circular reference", a "bean not found", an "ambiguous dependency" or "a configuration that is ignored" is a defect in how bean definitions are registered, ordered, instantiated or wrapped. A developer who can name the classes involved and the sequence of calls can read a stack trace as a trace of a known algorithm instead of an opaque failure.

The container performs three distinct jobs, and keeping them apart clarifies most behaviour. First, it **collects metadata**: it reads annotations, Java configuration and imports, and converts them into `BeanDefinition` objects. Second, it **post-processes metadata**: components that implement `BeanFactoryPostProcessor` may rewrite the definitions before any application object exists. Third, it **creates and decorates objects**: the container instantiates beans, injects dependencies, runs callbacks, and allows `BeanPostProcessor` implementations to replace the raw object with a wrapper (a proxy). Each job has its own extension point, and a defect almost always belongs to exactly one of them.

1.1.1 Concept and mechanism

BeanDefinition, BeanFactory and ApplicationContext

A `BeanDefinition` is a recipe: the bean class or factory method, scope, lazy flag, constructor arguments, property values, init and destroy method names, dependency names (`@DependsOn`), and autowire candidate flags. The concrete implementations are `GenericBeanDefinition`, `RootBeanDefinition` (the merged, runtime view used for creation), `AnnotatedGenericBeanDefinition` and `ScannedGenericBeanDefinition` (produced by component scanning, carrying annotation metadata read from bytecode without loading the class).

`BeanFactory` is the minimal contract (`getBean`, `containsBean`, `isSingleton`, `getType`). The implementation that does the work is `DefaultListableBeanFactory`, which is also a `BeanDefinitionRegistry`. `ApplicationContext` extends the factory contract with message resolution (`MessageSource`), event publication (`ApplicationEventPublisher`), resource loading (`ResourcePatternResolver`), and an `Environment`. Concrete contexts do not inherit from the factory; they delegate to an internal `DefaultListableBeanFactory`. In a servlet web application Spring Boot creates an `AnnotationConfigServletWebServerApplicationContext`.

Sheerfin

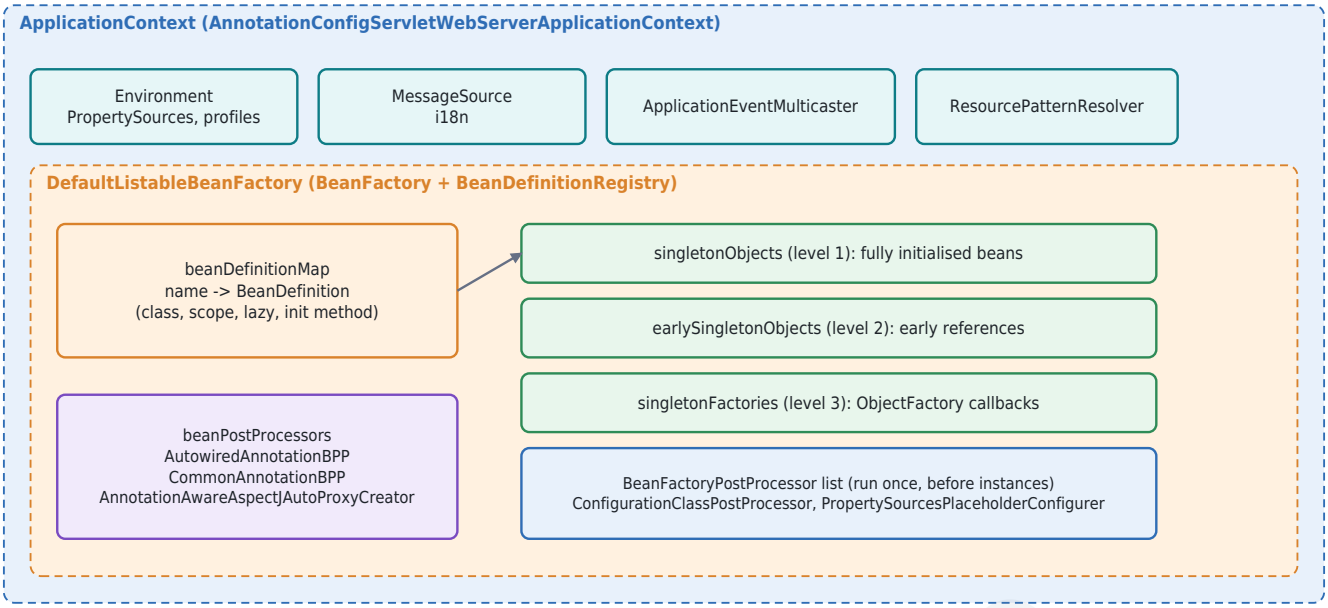


Figure 1.1. The application context wraps one DefaultListableBeanFactory; the factory stores BeanDefinitions and four caches of singleton state, and the post-processor lists extend it at two points.

A practical consequence: `getBean()` on a context is a method of the delegated factory, and the first call for a singleton name creates the object, while later calls return the cached instance. Beans created eagerly at start-up are produced by `preInstantiateSingletons()`, which iterates over all definitions that are non-lazy singletons.

The refresh() algorithm

`SpringApplication.run()` creates the context and calls `AbstractApplicationContext.refresh()`. This method is a template with a fixed order of steps, and the order explains which extension point can see which objects.

Order	Method	Effect
1	`prepareRefresh`	Initialises property sources, validates required properties, creates the early event set.
2	`obtainFreshBeanFactory`	Returns the DefaultListableBeanFactory (for a GenericApplicationContext it already exists).
3	`prepareBeanFactory`	Registers environment beans, Aware processors (ApplicationContextAwareProcessor), resolvable dependencies such as BeanFactory and ApplicationContext.
4	`postProcessBeanFactory`	Subclass hook; web contexts register request and session scopes here.
5	`invokeBeanFactoryPostProcessors`	Runs BeanDefinitionRegistryPostProcessor, then BeanFactoryPostProcessor. ConfigurationClassPostProcessor parses @Configuration, @ComponentScan, @Import and @Bean here.
6	`registerBeanPostProcessors`	Instantiates all BeanPostProcessor beans and sorts them (PriorityOrdered, Ordered, rest).
7	`initMessageSource`, `initApplicationEventMulticaster`	Creates the infrastructure beans.
8	`onRefresh`	Subclass hook; ServletWebServerApplicationContext creates the embedded Tomcat here (not yet started accepting traffic).
9	`registerListeners`	Registers ApplicationListener beans with the multicaster.
10	`finishBeanFactoryInitialization`	Freezes configuration and calls preInstantiateSingletons(): all non-lazy singletons are created.
11	`finishRefresh`	Starts Lifecycle beans (SmartLifecycle, including the web server) and publishes ContextRefreshedEvent.

Steps of refresh() in AbstractApplicationContext.

sreeram

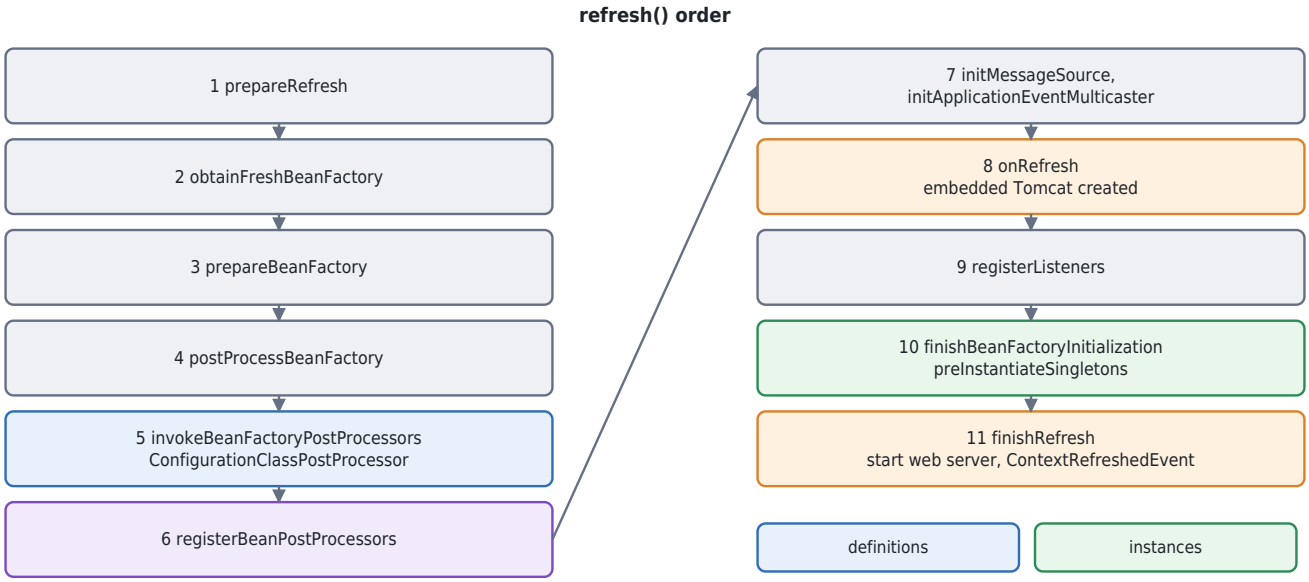


Figure 1.2. The sequence of refresh() separates definition-time work (steps 3 to 5) from instance-time work (steps 6 to 11); beans created in step 6 are the only ones that exist before the post-processors have all been registered.

Step 5 is the origin of most "why was my annotation ignored" questions. `ConfigurationClassPostProcessor` is a `BeanDefinitionRegistryPostProcessor`: it reads every configuration candidate, recursively processes `@Import`, `@ImportResource` and `@ComponentScan`, evaluates `@Conditional` annotations, and registers the resulting `BeanDefinition` objects through `ConfigurationClassBeanDefinitionReader`. Auto-configuration enters at this point through `AutoConfigurationImportSelector`, a `DeferredImportSelector` that is processed after all user configuration so that `@ConditionalOnMissingBean` can observe user-defined beans.

BeanFactoryPostProcessor and BeanPostProcessor

The two interfaces have similar names and fundamentally different targets.

Aspect	BeanFactoryPostProcessor	BeanPostProcessor
Operates on	BeanDefinition objects (metadata)	Bean instances
Timing	Before any ordinary bean is instantiated (step 5)	Around initialisation of every bean created afterwards (step 10)
Typical use	PropertySourcesPlaceholderConfigurer resolves ${...} placeholders; ConfigurationClassPostProcessor registers definitions	AutowiredAnnotationBeanPostProcessor (@Autowired, @Value), CommonAnnotationBeanPostProcessor (@PostConstruct, @PreDestroy, @Resource), AnnotationAwareAspectJAutoProxyCreator (AOP proxies)
Declaration advice	Declare the @Bean method static	Declare the @Bean method static
Danger	Requesting an ordinary bean early forces its creation before the other post-processors exist	The beans a BeanPostProcessor depends on are created early and are not eligible for processing

Comparison of the two post-processor families.

When a `@Bean` method for a post-processor is non-static, the container must instantiate the enclosing configuration class to call it, and that class is then created too early to be processed by later post-processors. The container logs `Bean 'x' of type [...] is not eligible for getting processed by all BeanPostProcessors` at INFO level. The corrective action is `static`.

The complete bean lifecycle

Instance creation is implemented in `AbstractAutowireCapableBeanFactory.doCreateBean()`. The phases, in the order executed for a singleton declared with a constructor and field injection, are:

1. `InstantiationAwareBeanPostProcessor.postProcessBeforeInstantiation` may return a replacement object and short-circuit everything below.

2. Instance creation: `createBeanInstance` selects a constructor (`AutowiredAnnotationBeanPostProcessor.determineCandidateConstructors`), a factory method, or an instance supplier.
3. `MergedBeanDefinitionPostProcessor.postProcessMergedBeanDefinition` collects injection metadata and lifecycle method metadata.
4. An early-reference factory is registered when the bean is a singleton in creation and circular references are allowed (see the section on circular dependencies).
5. `populateBean`: `postProcessAfterInstantiation`, then `postProcessProperties`, where `@Autowired` and `@Value` fields and setters are injected.
6. `initializeBean`: Aware callbacks (`BeanNameAware`, `BeanClassLoaderAware`, `BeanFactoryAware`), then `postProcessBeforeInitialization` of every `BeanPostProcessor` (this includes `ApplicationContextAware` and `EnvironmentAware`, applied by `ApplicationContextAwareProcessor`, and `@PostConstruct`, applied by `CommonAnnotationBeanPostProcessor`), then `InitializingBean.afterPropertiesSet`, then the custom init method (`@Bean(initMethod = ...)`), then `postProcessAfterInitialization` of every `BeanPostProcessor`. AOP proxies are created in this last step, which is why the bean visible to other beans can be a different object from the one the constructor produced.
7. After all singletons exist: `SmartInitializingSingleton.afterSingletonsInstantiated`.
8. On `close()`: `@PreDestroy`, `DisposableBean.destroy`, the custom destroy method. For `@Bean` methods the destroy method is inferred when a public no-argument `close` or `shutdown` method exists.

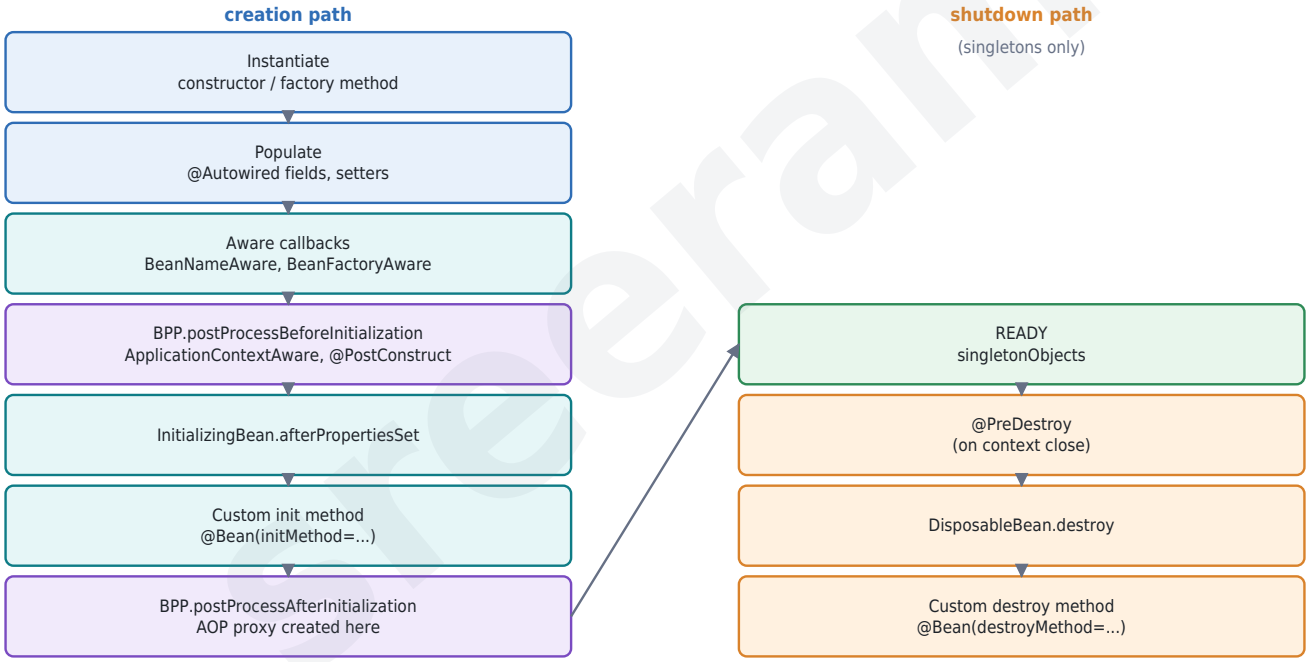


Figure 1.3. The lifecycle is a fixed pipeline; proxy creation happens in the last initialisation step, after @PostConstruct has already run on the raw object.

GOTCHA

Prototype-scoped beans are created, injected and then abandoned: the container holds no reference and never invokes destruction callbacks for them. Resources held by a prototype bean must be released by the code that requested the bean.

Scopes and scoped proxies

The default scope is `singleton`: one instance per container (not per class loader or JVM). `prototype` creates a new instance for each injection point or `getBean` call. Web-aware scopes are `request`, `session`, `application` and `websocket`. Custom scopes implement `org.springframework.beans.factory.config.Scope`.

A singleton that needs a shorter-lived bean has a structural problem: the singleton is constructed once, so a plain injection captures exactly one instance. Two correct solutions exist. The first is a **scoped proxy**: the container injects a singleton proxy that, on every method call, resolves the real target from the current scope (`@RequestScope` is shorthand for `@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)`). The second is **on-demand lookup** with `ObjectProvider<T>`, which calls `getObject()` when needed.

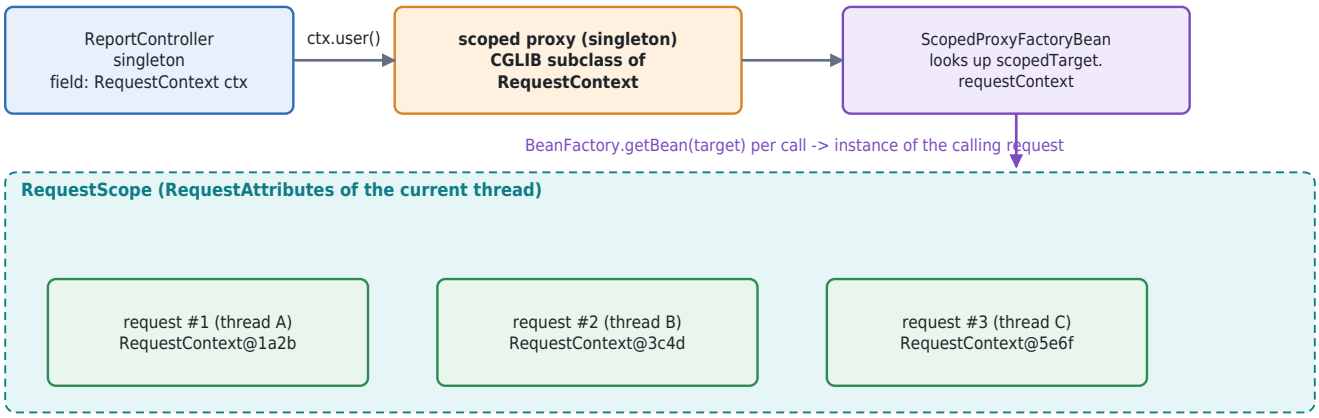


Figure 1.4. A request-scoped bean injected into a singleton is replaced by a proxy that fetches the target from the current request on every call.

Circular dependencies and the three-level cache

A cycle exists when bean A needs B and B needs A. With **constructor injection** no solution exists in principle: neither object can be constructed before the other. Spring throws `BeanCurrentlyInCreationException`. With **field or setter injection** the objects can be created first and wired afterwards, and `DefaultSingletonBeanRegistry` supports this using three maps:

- `singletonObjects` (level 1): completely initialised singletons.
- `earlySingletonObjects` (level 2): early references that were already handed out to another bean.
- `singletonFactories` (level 3): `ObjectFactory` callbacks that produce an early reference on demand.

When A is instantiated, a factory for A's raw instance is placed in level 3 before A's dependencies are populated. When B requires A, the lookup finds A in creation, calls the level-3 factory, moves the result to level 2, and injects it. The factory indirection exists so that `getEarlyBeanReference` can apply `SmartInstantiationAwareBeanPostProcessor` logic, for example to create the AOP proxy early so that B receives the same proxy that other beans will later receive.

Since Spring Boot 2.6 circular references are prohibited by default (`spring.main.allow-circular-references=false`), because a cycle indicates a design problem and because early references interact poorly with post-processors that do not support early exposure. A bean wrapped by `@Async` after an early reference was handed out produces the failure "has been injected into other beans in its raw version as part of a circular reference, but has eventually been wrapped".

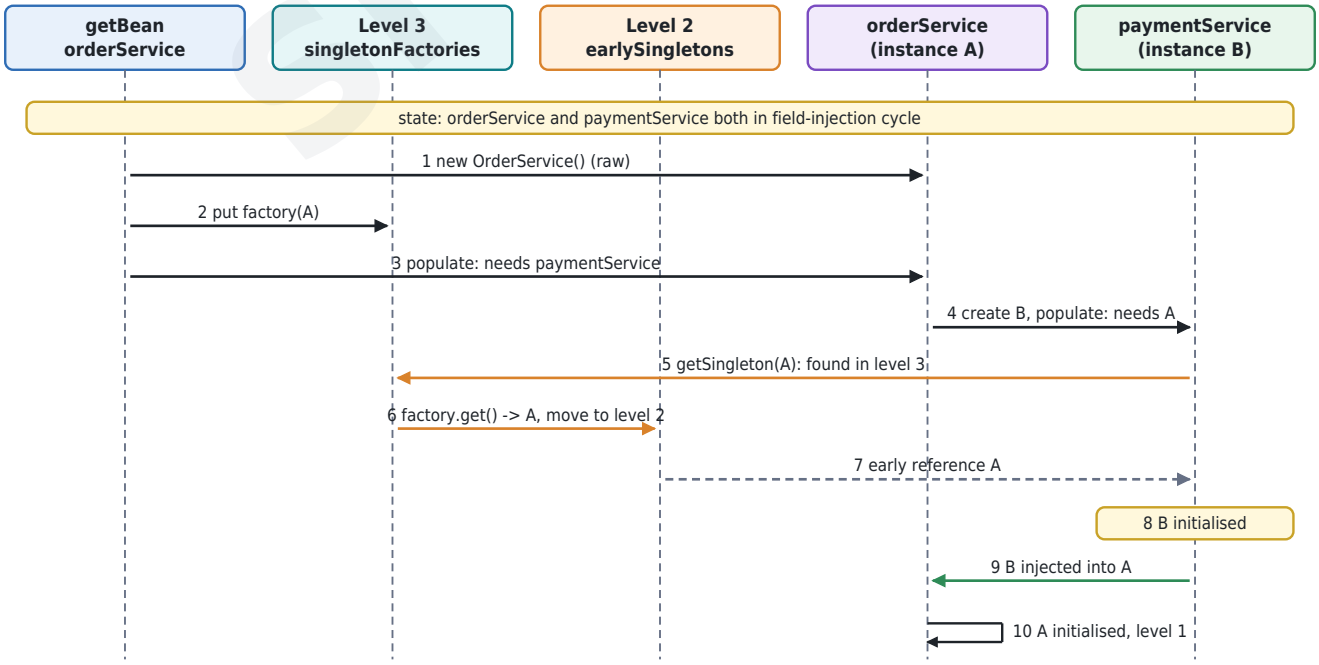


Figure 1.5. The sequence for field-injected beans A and B: A's raw instance is exposed through level 3, B receives it, and A is completed afterwards.

@Configuration proxying, lite mode and conditional registration

A class annotated `@Configuration` is, by default, subclassed at start-up with CGLIB (`ConfigurationClassEnhancer`). Calls to one `@Bean` method from another are intercepted by `BeanMethodInterceptor`, which returns the container's singleton instead of executing the method body again. This is **full mode** (`proxyBeanMethods = true`). In **lite mode** (`@Configuration(proxyBeanMethods = false)`, or `@Bean` methods declared in a `@Component`) the class is not subclassed and inter-bean method calls are ordinary Java calls that create a fresh object each time. Spring Boot's own auto-configuration uses `proxyBeanMethods = false` for faster start-up; the safe style is to pass dependencies as method parameters, which works in both modes.

`@Conditional` registers or skips a definition by evaluating a `Condition.matches(ConditionContext, AnnotatedTypeMetadata)` call during configuration parsing. `@Profile`, `@ConditionalOnProperty` and `@ConditionalOnMissingBean` are all implementations of this single mechanism (the Boot conditions are examined in the chapter on auto-configuration).

Events, listeners and ordering

The context is an `ApplicationEventPublisher`. Any object can be published (since Spring 4.2); listeners are `ApplicationListener<E>` beans or methods annotated `@EventListener`. The default `SimpleApplicationEventMulticaster` invokes listeners **synchronously on the publishing thread**, in the order given by `@Order`. A listener that throws propagates the exception to the publisher unless an error handler is configured. Asynchronous delivery requires `@Async` on the listener method (which makes it a proxy case, see the next chapter) or an executor on the multicaster. Transaction-bound delivery with `@TransactionalEventListener` is covered in the chapter on transactions.

Spring Boot publishes a sequence of its own events during start-up: `ApplicationStartingEvent`, `ApplicationEnvironmentPreparedEvent`, `ApplicationContextInitializedEvent`, `ApplicationPreparedEvent`, `ContextRefreshedEvent`, `ApplicationStartedEvent` (context refreshed, runners not yet called), `AvailabilityChangeEvent` (liveness correct), `ApplicationReadyEvent` (after all `CommandLineRunner` and `ApplicationRunner` beans have run; readiness accepting traffic), and `ApplicationFailedEvent` on a start-up error.

Ordering rules appear in several places and are often confused. `@Order`, `Ordered` and `@Priority` sort collections injected as `List<T>`, listeners, filters registered as beans, and post-processors; they do **not** control the order in which beans are created. Creation order follows dependencies and `@DependsOn`.

1.1.2 Worked example

The first listing is a complete plain-JDK container, about a hundred lines, that implements the algorithm described above: bean definitions, constructor instantiation by reflection, field injection of an `@Inject` annotation, a `BeanPostProcessor` list, `@PostConstruct`, and the three-level cache for a deliberate cycle. It is not Spring code, but each method corresponds to a Spring method named in the comments.

MiniContainer.java JAVA

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

public class MiniContainer {

    @Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD) @interface Inject {}
    @Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) @interface PostConstruct {}

    interface BeanPostProcessor {
        default Object before(Object bean, String name) { return bean; }
        default Object after(Object bean, String name) { return bean; }
    }

    static class OrderService {
        @Inject PaymentService payments;
        OrderService() { System.out.println("  [instantiate] OrderService"); }
        @PostConstruct void init() { System.out.println("  [@PostConstruct] OrderService"); }
        String place() { return "order placed, " + payments.charge(); }
    }
    static class PaymentService {
        @Inject OrderService orders;   // circular reference
        PaymentService() { System.out.println("  [instantiate] PaymentService"); }
        @PostConstruct void init() { System.out.println("  [@PostConstruct] PaymentService"); }
        String charge() { return "payment charged"; }
```

MiniContainer.java (continued) JAVA

```
    }

    private final Map<String, Class<?>> definitions = new LinkedHashMap<>();
    private final Map<String, Object> singletonObjects = new HashMap<>();       // level 1
    private final Map<String, Object> earlySingletonObjects = new HashMap<>();  // level 2
    private final Map<String, java.util.function.Supplier<Object>> singletonFactories
        = new HashMap<>();                                                       // level 3
    private final Set<String> inCreation = new LinkedHashSet<>();
    private final List<BeanPostProcessor> processors = new ArrayList<>();

    void register(String name, Class<?> type) { definitions.put(name, type); }

    Object getBean(String name) {
        Object o = getSingleton(name);
        return o != null ? o : create(name);
    }

    private Object getSingleton(String name) {       // DefaultSingletonBeanRegistry.getSingleton
        Object o = singletonObjects.get(name);
        if (o == null && inCreation.contains(name)) {
            o = earlySingletonObjects.get(name);
            if (o == null && singletonFactories.containsKey(name)) {
                o = singletonFactories.remove(name).get();      // promote level 3 -> level 2
                earlySingletonObjects.put(name, o);
                System.out.println("  [cache] early reference for '" + name
                    + "' promoted from level 3 to level 2");
            }
        }
        return o;
    }

    private Object create(String name) {             // doCreateBean in the Spring factory
        System.out.println("create '" + name + "'");
        inCreation.add(name);
        try {
            Class<?> type = definitions.get(name);
            Object raw = type.getDeclaredConstructor().newInstance();
            singletonFactories.put(name, () -> raw);            // addSingletonFactory
            for (Field f : type.getDeclaredFields()) {          // populateBean
                if (f.isAnnotationPresent(Inject.class)) {
                    String dep = definitions.entrySet().stream()
                        .filter(e -> f.getType().isAssignableFrom(e.getValue()))
                        .findFirst().orElseThrow().getKey();
                    System.out.println("  [inject] " + name + "." + f.getName()
                        + " <- '" + dep + "'");
                    f.setAccessible(true);
                    f.set(raw, getBean(dep));
                }
            }
            Object bean = raw;                                  // initializeBean
            for (BeanPostProcessor p : processors) bean = p.before(bean, name);
            for (Method m : type.getDeclaredMethods())
                if (m.isAnnotationPresent(PostConstruct.class)) {
                    m.setAccessible(true);
                    m.invoke(raw);
                }
            for (BeanPostProcessor p : processors) bean = p.after(bean, name);
            singletonObjects.put(name, bean);
            singletonFactories.remove(name);
            earlySingletonObjects.remove(name);
            return bean;
        } catch (ReflectiveOperationException e) {
            throw new IllegalStateException(e);
        } finally {
            inCreation.remove(name);
        }
    }

    public static void main(String[] args) {
        MiniContainer ctx = new MiniContainer();
        ctx.processors.add(new BeanPostProcessor() {
            public Object before(Object b, String n) {
                System.out.println("  [BPP.before] " + n);
                return b;
            }
```

MiniContainer.java (continued) JAVA

```
            public Object after(Object b, String n) {
                System.out.println("  [BPP.after]  " + n);
                return b;
            }
        });
        ctx.register("orderService", OrderService.class);
        ctx.register("paymentService", PaymentService.class);
        OrderService os = (OrderService) ctx.getBean("orderService");
        System.out.println(os.place());
        System.out.println("same instance: " + (os.payments.orders == os));
    }
}
```

The second listing is the Spring Boot equivalent used to observe every callback of one bean in a real container, together with a `BeanPostProcessor` and a `BeanFactoryPostProcessor`.

LifecycleProbe.java JAVA

```
package com.example.library.container;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.BeanNameAware;
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.beans.factory.config.ConfigurableListableBeanFactory;
import org.springframework.beans.factory.config.BeanFactoryPostProcessor;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ApplicationContextAware;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration(proxyBeanMethods = false)
public class LifecycleProbe {

    @Bean
    static BeanFactoryPostProcessor definitionLogger() {           // static: runs very early
        return (ConfigurableListableBeanFactory bf) ->
            System.out.println("BFPP   definitions=" + bf.getBeanDefinitionCount()
                + " probe.scope=" + bf.getBeanDefinition("probe").getScope());
    }

    @Bean
    static BeanPostProcessor stepLogger() {
        return new BeanPostProcessor() {
            @Override
            public Object postProcessBeforeInitialization(Object bean, String name)
                    throws BeansException {
                if (name.equals("probe")) System.out.println("BPP    before-init");
                return bean;
            }
            @Override
            public Object postProcessAfterInitialization(Object bean, String name)
                    throws BeansException {
                if (name.equals("probe")) System.out.println("BPP    after-init");
                return bean;
            }
        };
    }

    @Bean(initMethod = "customInit", destroyMethod = "customDestroy")
    Probe probe() {
        return new Probe();
    }

    public static class Probe implements BeanNameAware, ApplicationContextAware,
                                         InitializingBean, DisposableBean {
        public Probe() { System.out.println("1 constructor"); }
        @Override public void setBeanName(String n) { System.out.println("2 BeanNameAware " + n); }
        @Override public void setApplicationContext(ApplicationContext c) {
            System.out.println("3 ApplicationContextAware"); }
        @PostConstruct void postConstruct() { System.out.println("4 @PostConstruct"); }
```

LifecycleProbe.java (continued) JAVA

```
        @Override public void afterPropertiesSet() { System.out.println("5 afterPropertiesSet"); }
        void customInit() { System.out.println("6 initMethod"); }
        @PreDestroy void preDestroy() { System.out.println("7 @PreDestroy"); }
        @Override public void destroy() { System.out.println("8 DisposableBean.destroy"); }
        void customDestroy() { System.out.println("9 destroyMethod"); }
    }
}
```

1.1.3 Hands-on

The plain-JDK container was compiled and run on the build machine (JDK 21, `java MiniContainer.java`). The output demonstrates the three-level cache: `paymentService` is created while `orderService` is still in creation, receives the early reference, and is completed first.

Output of the mini container (real run) TERMINAL

```
$ java MiniContainer.java
create 'orderService'
  [instantiate] OrderService
  [inject] orderService.payments <- 'paymentService'
create 'paymentService'
  [instantiate] PaymentService
  [inject] paymentService.orders <- 'orderService'
  [cache] early reference for 'orderService' promoted from level 3 to level 2
  [BPP.before] paymentService
  [@PostConstruct] PaymentService
  [BPP.after]  paymentService
  [BPP.before] orderService
  [@PostConstruct] OrderService
  [BPP.after]  orderService
order placed, payment charged
same instance: true
```

What to notice: the `@PostConstruct` of `paymentService` runs **before** that of `orderService`, although `orderService` was requested first. A bean is initialised when its own population is complete, which for the inner bean of a cycle occurs earlier. Code in `@PostConstruct` must therefore not assume that the beans it depends on in a cycle have finished initialising.

Running `LifecycleProbe` in a Boot application produces the following sequence; the output is typical and was not run on the build machine, since no Spring artifacts are available there.

typical output (not run on the build machine) TERMINAL

```
BFPP   definitions=24 probe.scope=singleton
1 constructor
2 BeanNameAware probe
3 ApplicationContextAware
BPP    before-init
4 @PostConstruct
5 afterPropertiesSet
6 initMethod
BPP    after-init
... application started ...
7 @PreDestroy
8 DisposableBean.destroy
9 destroyMethod
```

The line order of 3, `BPP before-init` and 4 reflects that `ApplicationContextAwareProcessor` and `CommonAnnotationBeanPostProcessor` are themselves `BeanPostProcessor` instances that run ahead of the user-defined one by order. The exact count of definitions depends on the auto-configuration present. To inspect definitions of a running application, `GET /actuator/beans` (when the `beans` endpoint is exposed) lists every bean, its scope, its type and its dependencies.

1.1.4 Real-world problems and solutions

Problem 1: application fails at start-up with a circular dependency

Scenario. A service `InvoiceService` was extended to send notifications. `NotificationService` already depended on `InvoiceService` to load invoice data. After the change the application no longer starts.

Symptoms. The log ends with:

```
***************************
APPLICATION FAILED TO START
***************************

Description:

The dependencies of some of the beans in the application context form a cycle:

┌──────┐
|  invoiceService defined in file [.../InvoiceService.class]
↑     ↓
|  notificationService defined in file [.../NotificationService.class]
└──────┘

Action:

Relying upon circular references is discouraged and they are prohibited by default.
Update your application to remove the dependency cycle between beans. As a last resort,
it may be possible to break the cycle automatically by setting
spring.main.allow-circular-references to true.
```

Diagnosis. The failure analyser (`BeanCurrentlyInCreationFailureAnalyzer`) prints the cycle and each class. The cycle can also be confirmed with the Actuator `beans` endpoint on a variant that runs with `spring.main.allow-circular-references=true`, which lists the `dependencies` array of both beans. Constructor injection on at least one side confirms that the cycle is not resolvable even with the flag enabled.

Root cause. Both services require each other during construction. The flag change in Spring Boot 2.6 turned a previously tolerated design fault into a start-up failure.

Fix. Break the cycle by design, in this order of preference: (1) extract the shared logic into a third bean, `InvoiceQueries`, that both services use; (2) invert the dependency with an event, `InvoiceIssuedEvent`, published by `InvoiceService` and consumed by a listener in `NotificationService`; (3) as a last resort, mark one injection point `@Lazy`, which injects a proxy that resolves the target on first use.

NotificationService.java JAVA

```java
@Service
public class NotificationService {

    private final InvoiceService invoices;

    public NotificationService(@Lazy InvoiceService invoices) {   // proxy resolves later
        this.invoices = invoices;
    }
}
```

Prevention. Keep the property at its default of `false`. Review dependency direction (controllers to services to repositories) and fail the build on cycles with an architecture test.

Problem 2: the wrong implementation is injected

Scenario. Two beans implement `PaymentGateway`: `StripeGateway` and `SandboxGateway`. A new `CheckoutService` receives `SandboxGateway` in production.

Symptoms. Either start-up fails with `NoUniqueBeanDefinitionException: expected single matching bean but found 2`, or, worse, start-up succeeds and the wrong implementation is used.

Diagnosis. `GET /actuator/beans` filtered for the interface type shows both candidates. The resolution algorithm in `DefaultListableBeanFactory.determineAutowireCandidate` applies, in order: a `@Primary` candidate, the candidate with the highest `@Priority`, and finally a match of the injection point's parameter or field **name** against a bean name. The silent selection happened through the last rule: the constructor parameter was named `sandboxGateway`.

Root cause. Name-based fallback matches hide ambiguity. A refactoring that renames the parameter changes behaviour.

Fix. State the intent explicitly with `@Qualifier("stripeGateway")` at the injection point, or mark the production bean `@Primary`. For default-with-override designs, Spring Framework 6.2 offers `@Fallback`. Use `@Profile("!prod")` to keep the sandbox bean out of production.

Prevention. Inject `List<PaymentGateway>` or `Map<String, PaymentGateway>` when selection is a runtime decision, and avoid several beans of one type without a qualifier.

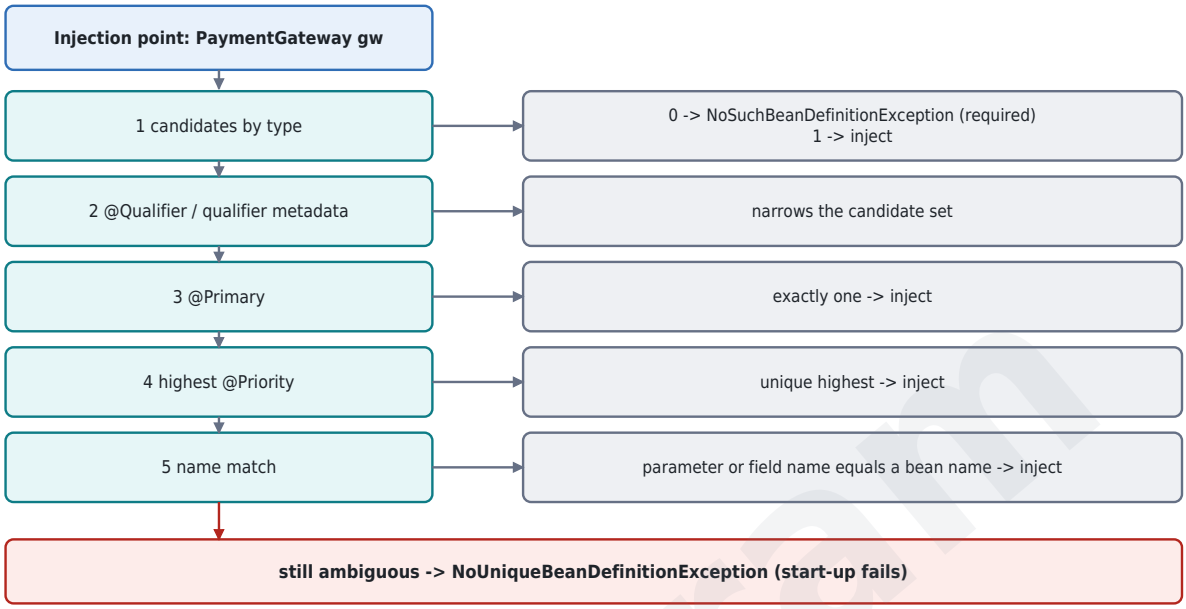


Figure 1.6. Autowire candidate selection: qualifier first, then @Primary, then @Priority, and only then a name match; failure produces NoUniqueBeanDefinitionException.

Problem 3: start-up takes more than a minute

Scenario. A service with 400 beans needs 70 seconds to become ready, which makes rolling deployments slow and probe configuration fragile.

Symptoms. Log timestamps show a long gap between "Starting" and "Started". Container start-up is fast on a developer laptop and slow in a CPU-limited pod.

Diagnosis. (1) Set `logging.level.org.springframework.boot.autoconfigure=DEBUG` or run with `--debug` to confirm the number of auto-configurations that matched. (2) Attach a `BufferingApplicationStartup(2048)` through `SpringApplication.setApplicationStartup(...)` and expose the Actuator `startup` endpoint (`GET /actuator/startup`), which returns timed steps such as `spring.beans.instantiate` with the bean name. (3) Sort by duration to find the slowest beans. Typical findings are beans that open a remote connection in `@PostConstruct`, Hibernate schema validation, a Flyway migration, and a very wide `@ComponentScan`.

Root cause. Eager instantiation of all singletons does expensive work serially on the main thread. A CPU limit of 0.5 core further lengthens the class-loading and reflection work.

Fix. Move remote work out of `@PostConstruct` (initialise lazily or on `ApplicationReadyEvent`), narrow `@ComponentScan`, and consider `spring.main.lazy-initialization=true`. Lazy initialisation defers failures from start-up to the first request and delays the first request's latency, so it is applied selectively with `@Lazy` on heavy beans rather than globally in production. For the larger options (class data sharing, AOT, native image) see the chapter on performance tuning.

Prevention. Record start-up time in CI from the `startup` endpoint and fail on regressions.

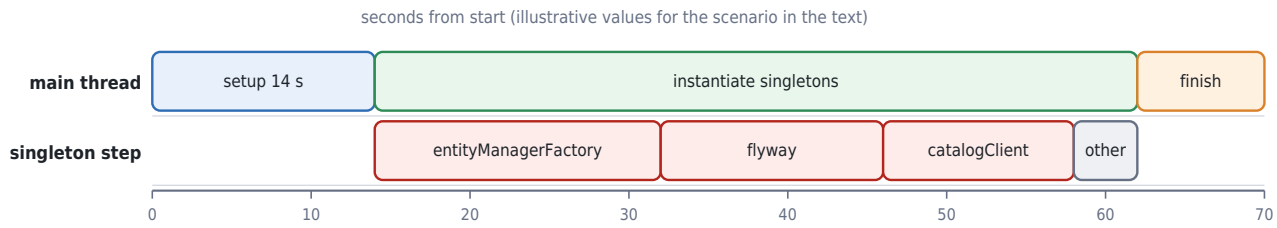


Figure 1.7. A start-up timeline broken down per step identifies the dominant phase; in this illustrative case three beans that perform remote or schema work dominate the total.

1.1.5 Questions to expect

1. **What is the difference between BeanFactory and ApplicationContext?** `BeanFactory` is the basic container contract (definition registry, `getBean`, lazy creation). `ApplicationContext` adds eager singleton creation, events, message sources, resource loading, environment and automatic post-processor detection. In practice the application always uses an `ApplicationContext`, which delegates to a `DefaultListableBeanFactory`.
2. **What happens inside refresh()?** The definitions are loaded and post-processed (`BeanFactoryPostProcessor`, `ConfigurationClassPostProcessor`), `BeanPostProcessor` beans are registered, the web server is created in `onRefresh`, non-lazy singletons are created in `finishBeanFactoryInitialization`, and `finishRefresh` starts lifecycle beans and publishes `ContextRefreshedEvent`.
3. **Explain BeanFactoryPostProcessor versus BeanPostProcessor.** The first modifies metadata before instances exist; the second wraps or modifies instances around initialisation. AOP proxies come from a `BeanPostProcessor`.
4. **In which order are callbacks invoked on a bean?** Constructor, injection, Aware callbacks, `@PostConstruct`, `afterPropertiesSet`, custom init method, `postProcessAfterInitialization` (proxying); on shutdown `@PreDestroy`, `destroy`, custom destroy method.
5. **How does Spring resolve circular dependencies?** For singletons with setter or field injection, through the three-level cache: an early reference is exposed through an `ObjectFactory` before population. Constructor cycles cannot be resolved. Since Boot 2.6 the feature is off by default.
6. **Why does a request-scoped bean in a singleton need a proxy?** The singleton is constructed once; the proxy resolves the correct target per call from the scope.
7. **What is the difference between full and lite `@Configuration`?** Full mode subclasses the class with CGLIB so that inter-`@Bean` calls return the singleton; lite mode (`proxyBeanMethods = false`) performs plain calls and is faster.
8. **Are Spring events asynchronous?** No, by default they run synchronously on the publisher's thread, including participation in the publisher's transaction.
9. **Does `@Order` determine bean creation order?** No. It orders collections and listeners; creation order follows dependencies and `@DependsOn`.

Check yourself

1. In `MiniContainer`, why does the `@PostConstruct` of `paymentService` run before that of `orderService`?
2. A `@Bean` method returning a `BeanPostProcessor` is not static. What is logged and what is the consequence?
3. A singleton declares `@Autowired private ShoppingCart cart;` where `ShoppingCart` is `@Scope("prototype")`. How many carts exist after 100 requests?
4. What is wrong with this configuration? `@Configuration(proxyBeanMethods = false) class C { @Bean A a() { return new A(b()); } @Bean B b() { return new B(); } }`
5. Which step of `refresh()` creates the embedded Tomcat instance, and which step starts accepting traffic?

Answers

1. The inner bean of the cycle completes population first because its dependency (the outer bean) is available as an early reference without being initialised.
2. An INFO message states that the bean is not eligible for processing by all `BeanPostProcessor` instances; the enclosing configuration class was created too early. Declare the method `static`.
3. One. The prototype is injected once into the singleton. Use `ObjectProvider<ShoppingCart>` to obtain a new one per use.
4. In lite mode `b()` is a plain call, so `A` receives a different `B` instance from the bean registered by `b()`. Take `B` as a parameter of `a(B b)`.

Sriram

5. `onRefresh` creates it; `finishRefresh` starts it through the `WebServerStartStopLifecycle` after all singletons exist.

KEY POINTS

- Metadata (`BeanDefinition`), definition post-processing (`BeanFactoryPostProcessor`) and instance decoration (`BeanPostProcessor`) are separate stages of `refresh()`.
- Proxies are created in `postProcessAfterInitialization`; the exposed bean may differ from the constructed object.
- Circular references work only for singletons with field or setter injection through the three-level cache, and are disabled by default since Boot 2.6.
- Full `@Configuration` mode uses CGLIB to preserve singleton semantics between `@Bean` calls; lite mode does not.
- Events are synchronous by default; `@Order` sorts collections, not creation.
- Ambiguity is resolved by qualifier, `@Primary`, `@Priority`, then parameter name; state intent explicitly.

1.2 AOP and Proxies

Transactions, asynchronous execution, caching, retries, method security, metrics and logging share one property: each is behaviour that must run around a method call without being part of the method's business logic. Writing that behaviour inside every service method duplicates code and couples the domain to infrastructure. Aspect-oriented programming (AOP) separates it: the behaviour is declared once and applied to many methods by a rule. In Spring, the mechanism is almost always a **proxy**, an object that stands in front of the real bean and intercepts calls. Understanding proxies explains, in a single model, why `@Transactional`, `@Async`, `@Cacheable`, `@Retryable` and `@PreAuthorize` all fail in exactly the same situations.

1.2.1 Concept and mechanism

Vocabulary

Term	Meaning
`Aspect`	A module that holds cross-cutting logic, declared with @Aspect on a Spring bean.
`Join point`	A point in program execution. Spring AOP supports only **method execution** on Spring beans.
`Pointcut`	A predicate that selects join points, written in AspectJ pointcut syntax (execution, within, @annotation, args, bean).
`Advice`	The action taken at a join point: @Before, @AfterReturning, @AfterThrowing, @After, @Around.
`Advisor`	Spring's pairing of one Advice and one Pointcut. Every @Transactional or @Async facility is an advisor.
`Target`	The real bean being advised.
`Proxy`	The object created by the framework that applies the advisors and delegates to the target.

AOP terms as used by Spring.

Spring AOP is not AspectJ. AspectJ modifies bytecode at compile time or load time and can advise field access, constructors and calls made from `this`. Spring AOP only builds runtime proxies. It borrows the AspectJ **annotation and pointcut language** (through the `aspectjweaver` library) but not its weaving. This choice explains every limitation listed later.

How a proxy gets into the application

The proxy is created by a `BeanPostProcessor`. When `spring-boot-starter-aop` is present, `AopAutoConfiguration` enables `AnnotationAwareAspectJAutoProxyCreator`. In `postProcessAfterInitialization` it collects every advisor in the context (beans annotated `@Aspect`, plus infrastructure advisors such as `BeanFactoryTransactionAttributeSourceAdvisor`), tests each advisor's pointcut against the bean's class, and, if any match, wraps the bean with `ProxyFactory`. The context then stores the **proxy** in `singletonObjects`, so every consumer that injects the type receives the proxy and the raw object remains reachable only from inside the proxy.

Two proxy technologies exist.

Aspect	JDK dynamic proxy (JdkDynamicAopProxy)	CGLIB proxy (CglibAopProxy)
Requires	The target implements at least one interface	A non-final class with a visible constructor

Sheeran

Aspect	JDK dynamic proxy (JdkDynamicAopProxy)	CGLIB proxy (CglibAopProxy)
Mechanism	java.lang.reflect.Proxy generates a class implementing the interfaces; InvocationHandler.invoke dispatches	A generated **subclass** of the target overrides methods and calls the interceptors; the instance is created without invoking the constructor (Objenesis)
Cast to implementation class	Not possible (a ClassCastException appears)	Possible, because the proxy is a subclass
Final class or method	Not relevant	Final classes cannot be proxied; final methods are not intercepted
Default in Spring Boot	No	Yes: spring.aop.proxy-target-class=true

JDK dynamic proxy versus CGLIB proxy.

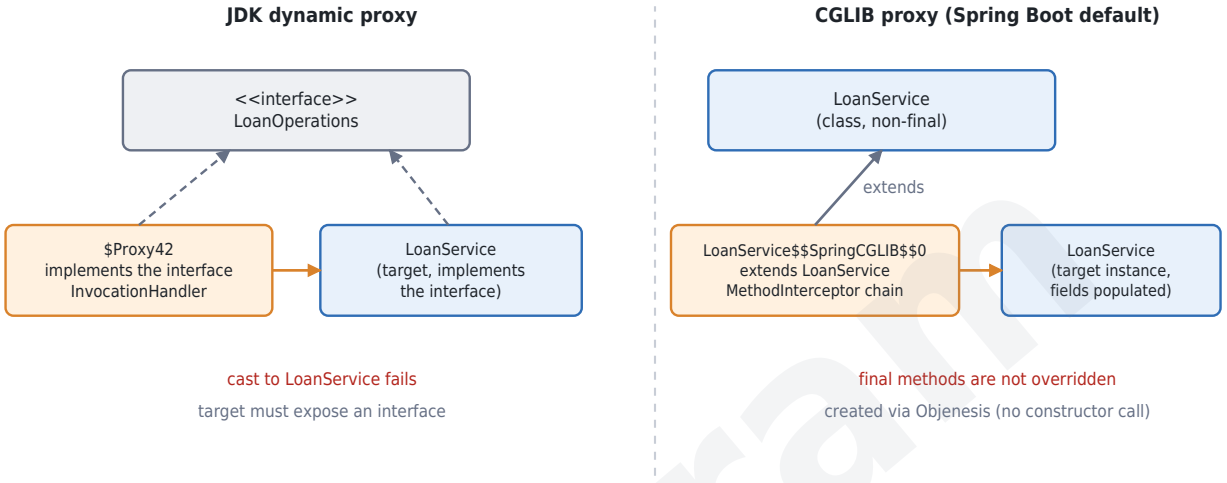


Figure 1.8. A JDK proxy implements the interface and wraps the target; a CGLIB proxy extends the target class. Both hold a reference to the real object and a chain of advisors.

Spring Boot defaults to class-based (CGLIB) proxies even for beans with interfaces, because injection by implementation class then works. A plain Spring Framework application created without Boot defaults to JDK proxies when interfaces exist.

Advice types and the interceptor chain

All advice is converted into `MethodInterceptor` objects. When a method is called on the proxy, the proxy builds a `ReflectiveMethodInvocation` that holds the ordered list of interceptors for that method and calls `proceed()`. Each interceptor either calls `proceed()` to continue (the next interceptor, or finally the target method through reflection) or returns early. This recursion is why advice nests like parentheses: the first interceptor is outermost.

Annotation	Semantics
`@Before`	Runs before the method. Cannot prevent execution except by throwing an exception.
`@AfterReturning`	Runs after a normal return; can bind the return value.
`@AfterThrowing`	Runs after an exception; can bind the exception. The exception continues to propagate.
`@After`	Runs in all cases (finally semantics).
`@Around`	Receives a ProceedingJoinPoint, controls whether and when proceed() is called, and may change arguments and result. All others can be expressed through it.

Advice types.

Use the weakest advice that suffices. `@Around` is powerful and has one recurring defect: forgetting to return the value of `proceed()`, which silently makes every advised method return `null`.

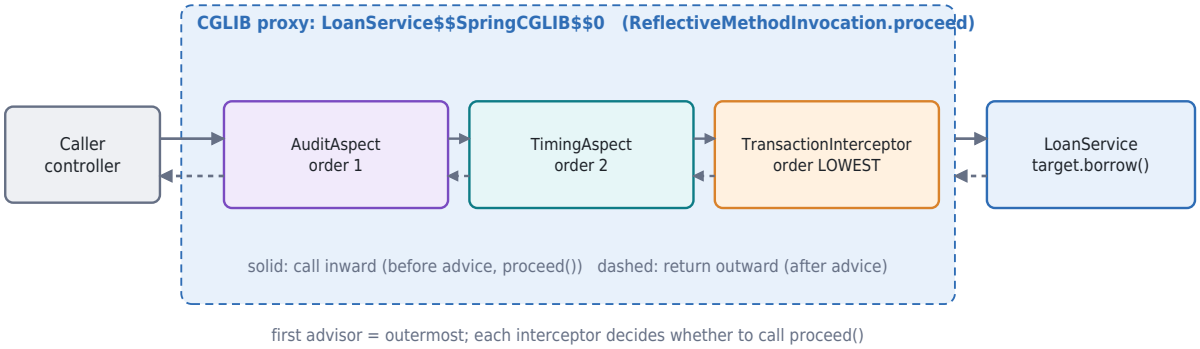


Figure 1.9. A call enters the proxy, passes through the interceptor chain in order, reaches the target, and unwinds in reverse; the first advisor is the outermost.

Pointcut expressions

The most used designators are:

Designator	Example and meaning
`execution`	`execution(public * com.example.library..*Service.*(..))`: public methods of classes named *Service in the package and subpackages
`within`	`within(com.example.library.web..*)`: all methods of types in the package
`@annotation`	`@annotation(com.example.library.Timed)`: methods carrying the annotation
`@within`	`@within(org.springframework.stereotype.Service)`: methods of classes carrying the annotation
`args`	`args(String, ..)`: methods whose first argument is a String
`bean`	`bean(*Repository)`: bean name pattern (Spring specific)

Pointcut designators (from the Spring Framework reference).

Pointcuts can be named with `@Pointcut` methods and combined with `&&`, `||` and `!`. Evaluation happens twice: statically when the post-processor decides whether to wrap the class, and, for `args` and `@annotation` binding, dynamically at call time. Pointcuts that cover many beans slow down start-up, because each pointcut is tested against each bean class.

Transactions, asynchronous execution, caching and retry are proxies

The annotations used in the data and service layers are implemented as advisors that follow the same chain model.

Annotation	Interceptor	Enabled by
`@Transactional`	TransactionInterceptor with the TransactionAttributeSource	@EnableTransactionManagement (Boot: TransactionAutoConfiguration)
`@Async`	AnnotationAsyncExecutionInterceptor submits the call to an AsyncTaskExecutor	@EnableAsync, via AsyncAnnotationBeanPostProcessor
`@Cacheable, @CacheEvict, @CachePut`	CacheInterceptor	@EnableCaching
`@Retryable`	RetryOperationsInterceptor (spring-retry library; Framework 7 integrates retry into the core)	@EnableRetry
`@PreAuthorize`	AuthorizationManagerBeforeMethodInterceptor	@EnableMethodSecurity

Built-in facilities implemented with proxies.

Because they share one mechanism, they share every limitation, and they stack. A method that carries both `@Transactional` and `@Cacheable` passes through two interceptors whose relative order is controlled by the `order` attribute of the `@Enable...` annotations (both default to lowest precedence, which means the order between them is not guaranteed unless stated).

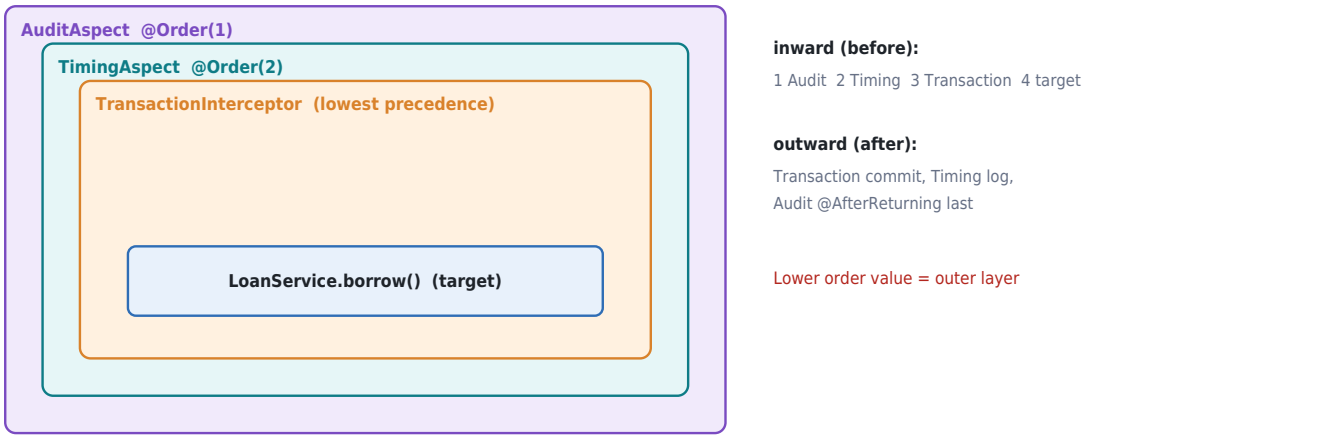


Figure 1.10. Lower order values are outer interceptors; here the audit aspect (order 1) wraps the timing aspect (order 2), which wraps the transaction interceptor, which wraps the target.

> **NOTE**
> Advice methods inside **one** aspect class are ordered since Spring Framework 5.2.7 as Around, Before, After, AfterReturning, AfterThrowing on the way in. Between **different** aspects, the `@Order` value (or the `Ordered` interface) decides; lower numbers have higher precedence. Without an order the sequence is undefined.

Limitations of proxy-based AOP

1. **Self-invocation.** A call to `this.method()` inside the target bypasses the proxy, because `this` is the raw object. No advice runs.
2. **Final classes and methods** cannot be overridden by a CGLIB subclass. A final method invoked on the proxy runs on the proxy instance, whose fields were never initialised (the instance was created without the constructor), which produces `NullPointerException` or silently incorrect values.
3. **Private methods** are never intercepted. Since Spring Framework 6.0, class-based proxies also advise protected and package-visible methods for transactions and similar annotations; private methods remain unsupported.
4. **Objects not managed by Spring** (created with `new`) have no proxy.
5. **Early use.** A bean used during initialisation of a `BeanPostProcessor` is created before the auto-proxy creator exists and is therefore not advised.

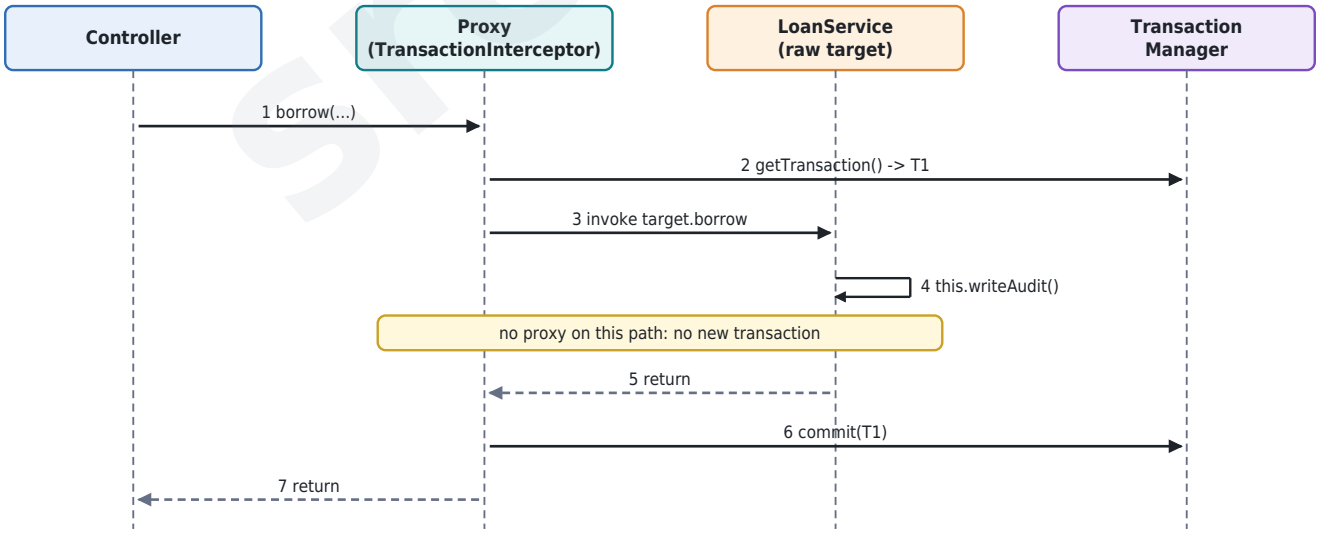


Figure 1.11. The external call reaches the proxy and the transaction starts; the internal call from issue() to archive() stays inside the target object and never touches the proxy.

1.2.2 Worked example

The scenario is a library service whose borrow operation must be audited, timed, and transactional. The code uses Spring Boot 3.x with `spring-boot-starter-aop`. Each file is shown in full.

Timed.java JAVA

```java
package com.example.library.aop;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Timed {
    /** Name of the metric or log label; defaults to the method name. */
    String value() default "";
}
```

TimingAspect.java JAVA

```java
package com.example.library.aop;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Pointcut;
import org.aspectj.lang.reflect.MethodSignature;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Component
@Order(2)
public class TimingAspect {

    private static final Logger log = LoggerFactory.getLogger(TimingAspect.class);

    @Pointcut("within(com.example.library..*Service)")
    void serviceLayer() { }

    @Around("serviceLayer() && @annotation(timed)")
    public Object time(ProceedingJoinPoint pjp, Timed timed) throws Throwable {
        MethodSignature sig = (MethodSignature) pjp.getSignature();
        String label = timed.value().isEmpty() ? sig.getName() : timed.value();
        long start = System.nanoTime();
        try {
            return pjp.proceed();                       // the return value must be passed on
        } finally {
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.info("{} took {} ms on thread {}", label, ms, Thread.currentThread().getName());
        }
    }
}
```

AuditAspect.java JAVA

```java
package com.example.library.aop;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.AfterThrowing;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Component
@Order(1)                                       // outermost: wraps TimingAspect
public class AuditAspect {
```

AuditAspect.java (continued) JAVA

```
    private static final Logger log = LoggerFactory.getLogger(AuditAspect.class);

    @AfterReturning(pointcut = "@annotation(com.example.library.aop.Timed)", returning = "result")
    public void ok(JoinPoint jp, Object result) {
        log.info("audit ok: {} -> {}", jp.getSignature().toShortString(), result);
    }

    @AfterThrowing(pointcut = "@annotation(com.example.library.aop.Timed)", throwing = "ex")
    public void failed(JoinPoint jp, Exception ex) {
        log.warn("audit failed: {} -> {}", jp.getSignature().toShortString(), ex.toString());
    }
}
```

LoanService.java JAVA

```
package com.example.library.loan;

import com.example.library.aop.Timed;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Service
public class LoanService {

    private final LoanRepository loans;

    public LoanService(LoanRepository loans) {
        this.loans = loans;
    }

    @Timed("loan.borrow")
    @Transactional
    public Loan borrow(long bookId, String member) {
        Loan loan = loans.save(new Loan(bookId, member));
        writeAudit(loan);                          // BUG: self-invocation, see the problems section
        return loan;
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void writeAudit(Loan loan) {
        loans.saveAudit(loan.id(), "BORROWED");
    }
}
```

What to notice: `borrow` is advised by three interceptors (audit, timing, transaction) because all three pointcuts match; the call to `writeAudit` inside the same class is not advised at all, so `REQUIRES_NEW` has no effect (see Problem 1).

1.2.3 Hands-on

The self-invocation defect and the chain order can be reproduced with plain JDK dynamic proxies, which are the same technology as `JdkDynamicAopProxy`. The program below places an advice that stands for `TransactionInterceptor` around a service whose `transfer` method calls its own `audit` method.

SelfInvocation.java JAVA

```
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.util.List;

public class SelfInvocation {

    interface AccountService {
        void transfer(String from, String to);
        void audit(String message);
    }

    static class AccountServiceImpl implements AccountService {
        @Override public void transfer(String from, String to) {
            System.out.println("    transfer " + from + " -> " + to);
            audit("transfer done");           // this.audit(...): bypasses the proxy
```

sreeram

SelfInvocation.java (continued) JAVA

```
        }
        @Override public void audit(String message) {
            System.out.println("    audit: " + message);
        }
    }

    static class TxHandler implements InvocationHandler {
        private final Object target;
        private final String order;
        TxHandler(Object target, String order) { this.target = target; this.order = order; }
        @Override public Object invoke(Object proxy, Method m, Object[] args) throws Throwable {
            System.out.println("  [" + order + "] before " + m.getName());
            try {
                return m.invoke(target, args);
            } finally {
                System.out.println("  [" + order + "] after  " + m.getName());
            }
        }
    }

    static AccountService wrap(AccountService target, String order) {
        return (AccountService) Proxy.newProxyInstance(
            AccountService.class.getClassLoader(), new Class<?>[] {AccountService.class},
            new TxHandler(target, order));
    }

    public static void main(String[] args) {
        AccountService target = new AccountServiceImpl();

        System.out.println("1. call through one proxy:");
        AccountService proxy = wrap(target, "tx");
        proxy.transfer("A", "B");

        System.out.println("2. chain of two proxies (outer = first advice):");
        AccountService chained = wrap(wrap(target, "inner"), "outer");
        chained.audit("direct call");

        System.out.println("3. proxy class facts:");
        System.out.println("  class = " + proxy.getClass().getName());
        System.out.println("  is subclass of impl? " + (proxy instanceof AccountServiceImpl));
        System.out.println("  interfaces = " + List.of(proxy.getClass().getInterfaces()));
    }
}
```

Real run on the build machine (JDK 21) TERMINAL

```
$ java SelfInvocation.java
1. call through one proxy:
  [tx] before transfer
    transfer A -> B
    audit: transfer done
  [tx] after  transfer
2. chain of two proxies (outer = first advice):
  [outer] before audit
  [inner] before audit
    audit: direct call
  [inner] after  audit
  [outer] after  audit
3. proxy class facts:
  class = $Proxy0
  is subclass of impl? false
  interfaces = [interface SelfInvocation$AccountService]
```

Section 1 shows the defect: `[tx] before` appears once, for `transfer`, and nothing appears around the nested `audit`. Section 2 shows the nesting order of a chain: before-advice in order, after-advice in reverse. Section 3 shows why a JDK proxy cannot be cast to the implementation class.

The final-method limitation of class-based proxies is reproduced by writing the subclass by hand, which is what CGLIB generates at runtime.

FinalMethodProxy.java JAVA

```java
public class FinalMethodProxy {

    static class ReportService {
        private final String owner;
        ReportService(String owner) { this.owner = owner; }
        public String open()  { return "open by " + owner; }          // overridable
        public final String close() { return "close by " + owner; }   // final: cannot be overridden
    }

    /** A CGLIB proxy amounts to a subclass that forwards overridable methods to the target. */
    static class ReportServiceProxy extends ReportService {
        private final ReportService target;
        ReportServiceProxy(ReportService target) {
            super(null);                        // proxy instance has no state of its own
            this.target = target;
        }
        @Override public String open() {
            System.out.println("  [advice] before open");
            return target.open();
        }
    }

    public static void main(String[] args) {
        ReportService real = new ReportService("alice");
        ReportService proxy = new ReportServiceProxy(real);
        System.out.println("open():  " + proxy.open());
        System.out.println("close(): " + proxy.close());
    }
}
```

Real run on the build machine (JDK 21) TERMINAL

```
$ java FinalMethodProxy.java
  [advice] before open
open():  open by alice
close(): close by null
```

`close()` executes on the proxy instance, whose field `owner` is `null`, and no advice runs. The same effect in Spring appears as a `NullPointerException` in a final method that uses an injected field.

Running the library application with the aspects above produces the following output for one borrow request. The output is typical and was not run on the build machine.

typical output (not run on the build machine) TERMINAL

```
INFO  c.e.library.aop.TimingAspect : loan.borrow took 14 ms on thread http-nio-8080-exec-2
INFO  c.e.library.aop.AuditAspect  : audit ok: LoanService.borrow(..) -> Loan[id=17, ...]
```

The audit line follows the timing line because `AuditAspect` (order 1) is outermost: its after-returning advice executes after the timing advice has already finished. To confirm that a bean is proxied at runtime, evaluate `AopUtils.isAopProxy(bean)` or print `bean.getClass()`, which shows a name of the form `LoanService$$SpringCGLIB$$0`.

1.2.4 Real-world problems and solutions

Problem 1: @Transactional ignored on an internal call

Scenario. `borrow` must always write an audit row, even when the surrounding business transaction rolls back. The audit method was declared `REQUIRES_NEW` so that it commits independently. In production a failed borrow leaves no audit row.

Symptoms. The audit table lacks rows for failed borrows. Unit tests that call `writeAudit` directly pass. SQL logs show only one transaction per request.

Diagnosis. (1) Enable `logging.level.org.springframework.transaction.interceptor=TRACE`; the log shows `Getting transaction for [LoanService.borrow]` and no line for `writeAudit`. (2) Enable `logging.level.org.springframework.jdbc.datasource.DataSourceTransactionManager=DEBUG` (or `JpaTransactionManager`) and count `Creating new transaction` lines: one appears where two were expected. (3) Print `AopUtils.isAopProxy(this)` inside `borrow`: it returns `false`, because `this` is the raw object.

Root cause. `borrow` calls `writeAudit` through `this`. The proxy, which holds the `TransactionInterceptor`, is bypassed (see the self-invocation limitation).

Fix. Move `writeAudit` to a separate bean so that the call crosses a proxy boundary.

AuditWriter.java JAVA

```java
@Service
public class AuditWriter {

    private final LoanRepository loans;

    public AuditWriter(LoanRepository loans) {
        this.loans = loans;
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void write(long loanId, String event) {
        loans.saveAudit(loanId, event);
    }
}
```

`LoanService` then injects `AuditWriter` and calls `audit.write(loan.id(), "BORROWED")`. Alternatives, in decreasing order of preference: inject the bean into itself with `@Lazy` (`@Autowired @Lazy LoanService self;`), which keeps the code in one class but obscures the design; use `AopContext.currentProxy()` with `@EnableAspectJAutoProxy(exposeProxy = true)`, which couples the code to Spring AOP; or switch to AspectJ weaving mode (`@EnableTransactionManagement(mode = AdviceMode.ASPECTJ)`), which requires a weaver at compile or load time.

Prevention. Keep annotated methods that are called only internally to a minimum, and write a slice or integration test that asserts the number of transactions (`TransactionSynchronizationManager.isActualTransactionActive()`) or the persisted audit row.

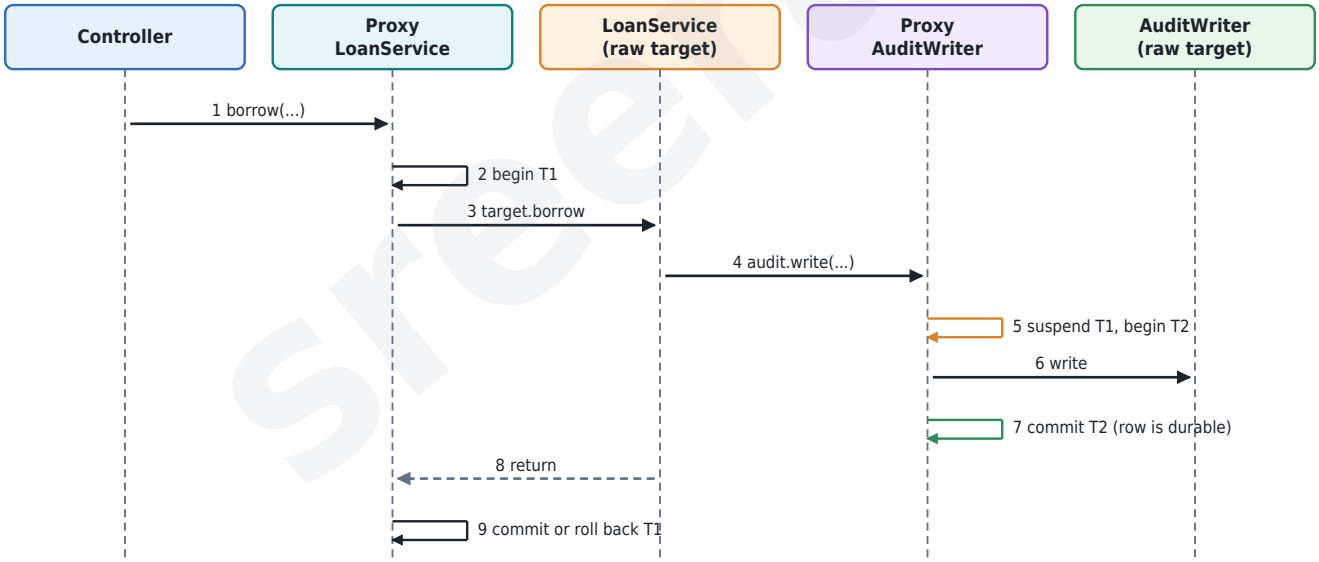


Figure 1.12. After the fix the second call crosses the AuditWriter proxy and therefore starts its own REQUIRES_NEW transaction on a second connection.

Problem 2: @Async method runs on the request thread

Scenario. `ReportService.generate()` was annotated `@Async` to free the request thread, yet the endpoint still takes eight seconds.

Symptoms. The log line printed inside the method shows `http-nio-8080-exec-3` instead of `task-1`. Response time equals the processing time.

Diagnosis. Check the thread name in a log statement inside the method. Check that `@EnableAsync` is present on a configuration class, that the method is called from **another** bean, and that the method is not `private` or `final`. Print `Thread.currentThread().getName()` at both the call site and inside the method.

Root cause. Either `@EnableAsync` is missing (no `AsyncAnnotationBeanPostProcessor`, so no proxy), or the method is called from another method of the same class (self-invocation).

Fix. Add `@EnableAsync` to a `@Configuration` class, call the method from a different bean, and declare an executor explicitly so that the pool is bounded:

AsyncConfig.java JAVA

```java
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "reportExecutor")
    ThreadPoolTaskExecutor reportExecutor() {
        ThreadPoolTaskExecutor ex = new ThreadPoolTaskExecutor();
        ex.setCorePoolSize(4);
        ex.setMaxPoolSize(8);
        ex.setQueueCapacity(50);
        ex.setThreadNamePrefix("report-");
        ex.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
        return ex;
    }
}
```

The annotated method is then `@Async("reportExecutor")` and returns `void` or `CompletableFuture<T>`. An exception thrown from a `void` async method is not visible to the caller and goes to the `AsyncUncaughtExceptionHandler`; a `CompletableFuture` return type propagates it through the future.

Prevention. Log the thread name in a start-up test of each async method. Name and bound the executor explicitly instead of relying on implicit defaults.

Problem 3: NullPointerException in a final method of a proxied bean

Scenario. After adding `@Transactional` to a class, a helper method `String describe()` that returns a value computed from an injected field starts to throw `NullPointerException` in production only through the controller.

Symptoms. The stack trace points into the method body, at an injected field. The same method works in a unit test constructed with `new`.

Diagnosis. The method is declared `final` (or the class is a Kotlin class without the `kotlin-spring` plugin, where classes are final by default). Inspect `bean.getClass()` at the injection site: the name contains `$$SpringCGLIB$$`, which confirms a class-based proxy. The stack trace shows no `CglibAopProxy` frame for the failing call.

Root cause. The CGLIB subclass cannot override a final method, so the call executes on the proxy instance, whose fields were never populated.

Fix. Remove `final` from methods of proxied classes, or extract the helper into a non-proxied utility.

Prevention. Avoid `final` on public methods of Spring beans that carry advice; configure the `kotlin-spring` compiler plugin for Kotlin.

1.2.5 Questions to expect

1. **How does Spring implement `@Transactional`?** A `BeanPostProcessor` (the auto-proxy creator) wraps beans that match the `BeanFactoryTransactionAttributeSourceAdvisor` pointcut. The proxy's `TransactionInterceptor` obtains a transaction from the `PlatformTransactionManager` before invoking the target, and commits or rolls back afterwards.
2. **JDK proxy versus CGLIB?** A JDK proxy implements the target's interfaces through `java.lang.reflect.Proxy`; CGLIB generates a subclass. Spring Boot defaults to CGLIB (`spring.aop.proxy-target-class=true`).
3. **Why does `@Transactional` not work when a method calls another method of the same class?** The internal call uses `this`, which is the raw target, so the proxy is bypassed. Call through another bean, or restructure.
4. **Which methods can be advised?** Public methods on Spring-managed beans always; protected and package-visible methods with class-based proxies since Spring 6.0; never private, static or final.
5. **Spring AOP versus AspectJ?** Spring AOP uses runtime proxies and supports method execution on Spring beans. AspectJ weaves bytecode and supports field access, constructors, and self-invocation.
6. **How is the order of two aspects defined?** By `@Order` or `Ordered`; the lower value is the outer interceptor. Without an order the sequence is undefined.
7. **What is the difference between `@Before` and `@Around`?** `@Around` controls whether the method proceeds and can transform arguments and result; `@Before` cannot stop execution except by throwing.

sreeram

8. **Why can an advised bean not be created with `new`?** Only container-managed beans pass through the post-processor, so only they are proxied.
9. **What does `@Async` return and where do exceptions go?** `void` or `Future`/`CompletableFuture`. Exceptions in void methods go to the `AsyncUncaughtExceptionHandler`.

Check yourself

1. A method of a class marked `@Service` is annotated `@Cacheable` and is invoked from another method of the same class. Is the result cached on the second call?
2. What does this advice return for every advised method? `@Around("execution(* *(..))") Object a(ProceedingJoinPoint p) throws Throwable { p.proceed(); return null; }`
3. In `SelfInvocation`, what would `chained.transfer("A", "B")` print for the nested `audit` call?
4. Aspect X has `@Order(5)`, aspect Y has `@Order(1)`. Which `@Before` advice runs first and which `@After` advice runs first?
5. Why can `proxy instanceof AccountServiceImpl` be false while the proxy is fully usable as `AccountService`?

Answers

1. No. The internal call bypasses the proxy, so `CacheInterceptor` does not run.
2. `null`. The result of `proceed()` is discarded; for primitive return types this also causes a failure.
3. Nothing around the nested `audit`: the call goes through `this` inside the implementation, so neither `[inner]` nor `[outer]` lines appear for it.
4. `Y` runs its `@Before` first (outermost). The `@After` of `X` runs first on the way out, because unwinding is in reverse.
5. The JDK proxy class implements the interfaces and does not extend the implementation class.

KEY POINTS

- Spring AOP is proxy-based: a `BeanPostProcessor` replaces the bean with a JDK proxy (interfaces) or a CGLIB subclass (Boot default).
- `@Transactional`, `@Async`, `@Cacheable`, `@Retryable` and method security are advisors; they all fail on self-invocation, on private, static and final methods, and on objects created with `new`.
- Interceptors nest like parentheses; `@Order` (lower is outer) decides between aspects.
- Fix self-invocation by calling across a bean boundary; the proxy must be on the call path.
- Verify with `AopUtils.isAopProxy`, the class name containing `$$SpringCGLIB$$`, and TRACE logging of the interceptor packages.

1.3 Auto-Configuration and Configuration in Depth

Spring Boot reduces a hundred lines of infrastructure wiring to the presence of a dependency on the classpath. The reduction is not magic: it is a list of candidate configuration classes, each guarded by conditions that inspect the classpath, the existing beans and the environment. A developer who understands the list, the conditions and the binding of properties can answer the two questions that dominate production work: "why did Boot create this bean?" and "why did Boot ignore my setting?".

1.3.1 Concept and mechanism

Anatomy of @SpringBootApplication

`@SpringBootApplication` is a composed annotation with three responsibilities.

- `@SpringBootConfiguration` (a specialisation of `@Configuration`) marks the class as a source of bean definitions and lets tests locate it.
- `@ComponentScan` scans the package of the annotated class and its subpackages. It carries two exclude filters, `TypeExcludeFilter` (used by test slices) and `AutoConfigurationExcludeFilter` (prevents auto-configuration classes from being picked up by scanning).
- `@EnableAutoConfiguration` is itself composed of `@AutoConfigurationPackage`, which records the base package for later use (for example by Spring Data to find entities and repositories), and `@Import(AutoConfigurationImportSelector.class)`.

The consequence of the first two points is a classical defect: a class located **outside** the package tree of the main class is not scanned, so its beans do not exist. Setting `scanBasePackages` or moving the main class to the root package corrects it.

From the imports file to bean definitions

`AutoConfigurationImportSelector` is a `DeferredImportSelector`: it is processed after all user `@Configuration` classes, which guarantees that `@ConditionalOnMissingBean` can see user-defined beans. It reads the file `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` from every jar on the classpath, one fully qualified class name per line. The `spring-boot-autoconfigure` jar alone lists more than a hundred classes. (Before Boot 2.7 the list was registered in `spring.factories` under the key `EnableAutoConfiguration`; Boot 3.0 removed that mechanism for auto-configuration.)

The selector applies three filters before any class is parsed:

1. Names listed in `exclude` or `excludeName` of `@EnableAutoConfiguration`, or in the property `spring.autoconfigure.exclude`, are removed.
2. `AutoConfigurationImportFilter` implementations (`OnClassCondition`, `OnBeanCondition`, `OnWebApplicationCondition`) evaluate the cheap conditions using pre-computed metadata in `META-INF/spring-autoconfigure-metadata.properties`, so that classes whose required library is absent are dropped without being loaded.
3. The survivors are sorted by `@AutoConfigureOrder`, `@AutoConfigureBefore`/`@AutoConfigureAfter` (or the `before` and `after` attributes of `@AutoConfiguration`). Ordering only affects the **registration order of bean definitions**; instantiation order still follows dependencies.

The remaining classes are parsed like ordinary configuration classes, and each `@Conditional` on a class or `@Bean` method is evaluated at parse or registration time.

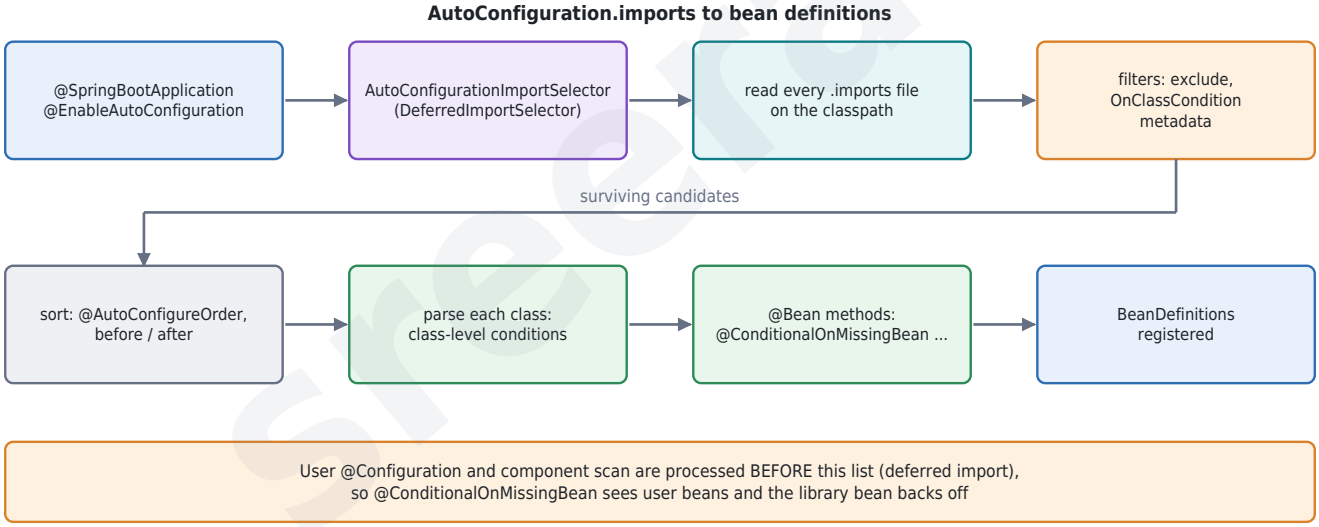


Figure 1.13. The selector filters candidates from every .imports file, orders them after user configuration, then each class and each @Bean method passes through its own conditions.

The condition annotations

Annotation	Evaluates
`@ConditionalOnClass / @ConditionalOnMissingClass`	Whether a class is present on the classpath. On a @Bean method use the name attribute (strings), because a class literal would be loaded while reading the annotation.
`@ConditionalOnBean / @ConditionalOnMissingBean`	Whether a bean of a type, name or annotation exists. Intended for auto-configuration classes, which run after user beans.
`@ConditionalOnProperty`	A property by prefix, name, havingValue and matchIfMissing. Without havingValue any value other than false matches.
`@ConditionalOnWebApplication`	Servlet, reactive or any web application type.
`@ConditionalOnResource`	Presence of a resource such as a file on the classpath.
`@ConditionalOnExpression`	A SpEL expression. Referencing beans in it forces early initialisation.
`@ConditionalOnSingleCandidate`	Exactly one bean, or a primary one, of a type.
`@Profile`	Active profile expression (implemented as a Condition).

Frequently used conditions (Spring Boot 3.x).

Two design rules follow from the mechanism. First, every bean definition in an auto-configuration should carry `@ConditionalOnMissingBean`, so that an application can replace it by declaring its own bean of the same type; the library's bean then **backs off**. Second, give the `@Bean` method the most specific return type available: bean conditions inspect the declared return type before the method runs.

Property binding and relaxed binding

Typed configuration is bound by `ConfigurationPropertiesBindingPostProcessor` using the `Binder`. A class annotated `@ConfigurationProperties("audit")` is registered either with `@EnableConfigurationProperties(AuditProperties.class)` or through `@ConfigurationPropertiesScan`. Binding sources are all `PropertySource` objects of the `Environment`. Java records and classes with a single parameterised constructor use **constructor binding** automatically in Boot 3; mutable beans use setters. `@Validated` activates Jakarta Bean Validation on the bound object, so an invalid value fails at start-up instead of at first use.

Relaxed binding means a property can be written in several spellings for the same canonical key. The canonical form used for lookup and in documentation is lower-case kebab-case separated by dots (`audit.slow-threshold`).

Source	Spelling	Rule
`.properties and .yml`	`audit.slow-threshold`	Kebab case; recommended.
`.properties and .yml`	`audit.slowThreshold, audit.slow_threshold`	Camel case and underscore notation are also accepted.
`Environment variable`	`AUDIT_SLOWTHRESHOLD`	Replace dots with underscores, remove dashes, upper case.
`System property`	`-Daudit.slowThreshold=2s`	Camel or kebab case.
`Lists in env vars`	`MY_LIST_0=a MY_LIST_1=b`	Index suffix with underscores.

Spellings of one key (Spring Boot reference, externalized configuration).

The mapping function and the precedence of sources are demonstrated by a plain-JDK program that follows the rules above (the canonicalisation rules and the order of the property sources are from the reference documentation; the program only reproduces them).

RelaxedBinding.java JAVA

```java
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

public class RelaxedBinding {

    /** Canonical form: lower-case, kebab-case, dot-separated (as in the Spring Boot reference). */
    static String canonical(String name) {
        StringBuilder sb = new StringBuilder();
        for (char c : name.toCharArray()) {
            if (c == '_') sb.append('-');                  // underscore notation
            else if (Character.isUpperCase(c)) sb.append('-').append(Character.toLowerCase(c));
            else sb.append(c);
```

RelaxedBinding.java (continued) JAVA

```
        }
        return sb.toString();
    }

    /** Environment variable form: dots to underscores, dashes removed, upper case. */
    static String toEnvVar(String canonical) {
        return canonical.replace('.', '_').replace("-", "").toUpperCase(Locale.ROOT);
    }

    record Source(String name, Map<String, String> values) {}

    static String resolve(String key, List<Source> sourcesByPrecedence) {
        for (Source s : sourcesByPrecedence) {
            String v = s.values().get(key);
            if (v != null) {
                System.out.printf("  %-24s -> %-8s (from %s)%n", key, v, s.name());
                return v;
            }
        }
        System.out.printf("  %-24s -> <unbound>%n", key);
        return null;
    }

    public static void main(String[] args) {
        System.out.println("Spellings of one property and its canonical form:");
        for (String s : List.of("app.max-retries", "app.maxRetries", "app.max_retries"))
            System.out.printf("  %-18s -> %s%n", s, canonical(s));
        System.out.println("Environment variable for app.max-retries: "
            + toEnvVar("app.max-retries"));

        Map<String, String> cmd = new LinkedHashMap<>(Map.of("app.timeout", "1s"));
        Map<String, String> env = new LinkedHashMap<>(
            Map.of("app.timeout", "2s", "app.pool-size", "20"));
        Map<String, String> profileYaml = new LinkedHashMap<>(
            Map.of("app.timeout", "3s", "app.region", "eu"));
        Map<String, String> baseYaml = new LinkedHashMap<>(
            Map.of("app.timeout", "9s", "app.region", "us",
            "app.pool-size", "5", "app.name", "orders"));
        List<Source> sources = List.of(new Source("command line", cmd),
            new Source("environment", env), new Source("application-prod.yml", profileYaml),
            new Source("application.yml", baseYaml));
        System.out.println("Resolution, highest precedence first:");
        for (String k : List.of("app.timeout", "app.pool-size", "app.region", "app.name",
                                "app.missing"))
            resolve(k, sources);
    }
}
```

Real run on the build machine (JDK 21) TERMINAL

```
$ java RelaxedBinding.java
Spellings of one property and its canonical form:
  app.max-retries    -> app.max-retries
  app.maxRetries     -> app.max-retries
  app.max_retries    -> app.max-retries
Environment variable for app.max-retries: APP_MAXRETRIES
Resolution, highest precedence first:
  app.timeout              -> 1s       (from command line)
  app.pool-size            -> 20       (from environment)
  app.region               -> eu       (from application-prod.yml)
  app.name                 -> orders   (from application.yml)
  app.missing              -> <unbound>
```

Externalised configuration order

Spring Boot merges many property sources and the **last source wins**. The reference lists them from lowest to highest priority. The sources that matter in everyday work are, in ascending order:

1. Defaults set with `SpringApplication.setDefaultProperties`.
2. `@PropertySource` annotations (loaded late: too late for `logging.*` and `spring.main.*`).
3. Config data: `application.properties` and `application.yml`, then profile-specific `application-{profile}` files. Within the config data, files outside the jar override files inside the jar, and profile-specific files override non-specific

ones. The default search locations, in ascending order, are `classpath:/`, `classpath:/config/`, `file:./`, `file:./config/` and `file:./config/*/`.

4. OS environment variables.
5. Java system properties (`-D`).
6. `SPRING_APPLICATION_JSON` / `spring.application.json`.
7. Command-line arguments (`--server.port=9090`).
8. In tests: the `properties` attribute of `@SpringBootTest`, `@DynamicPropertySource`, and `@TestPropertySource`.

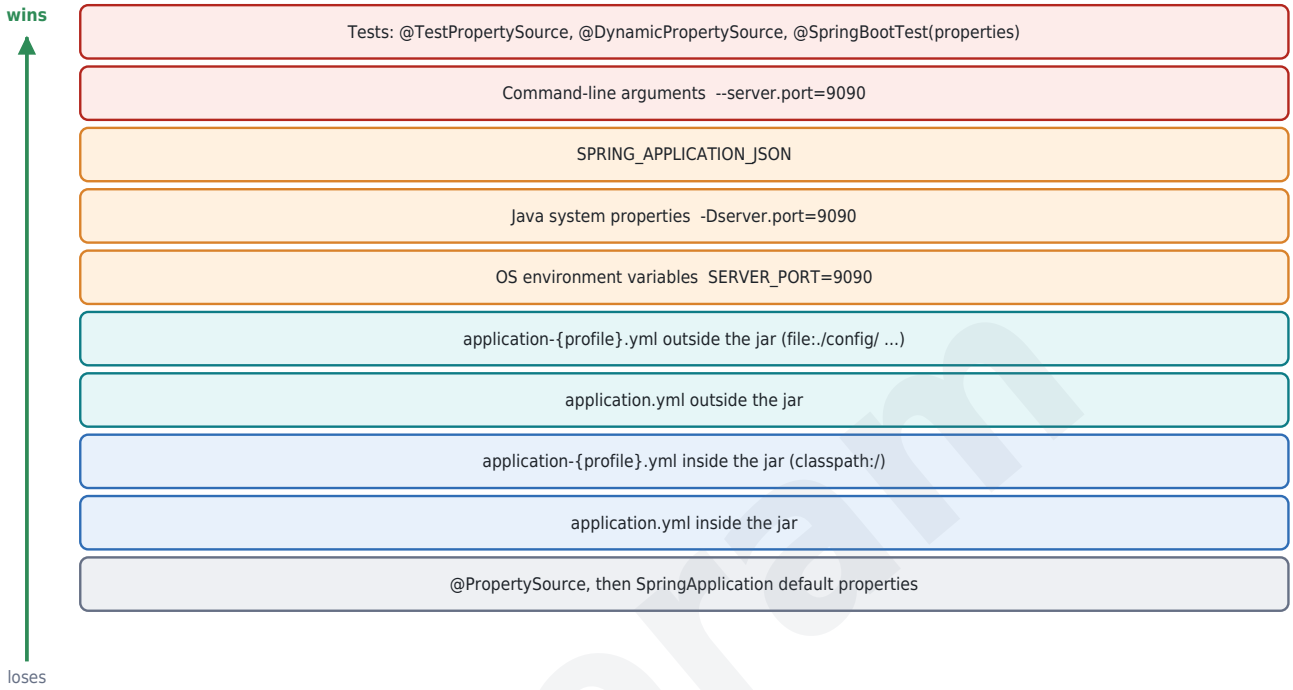


Figure 1.14. The same key set in several places resolves to the topmost source; the file layer inside config data has its own internal order.

`spring.config.location` **replaces** the default search locations, while `spring.config.additional-location` adds locations that take precedence over the defaults. `spring.config.import` pulls in further files or sources (`optional:file:./local.yml`, `configtree:/run/secrets/`, `env:MY_JSON`) and the imported values override those of the importing file. Profiles are activated with `spring.profiles.active`, grouped with `spring.profiles.group.prod=proddb,prodmq`, and used inside a multi-document YAML file with `spring.config.activate.on-profile`. The legacy `bootstrap` context is not used in Boot 3 unless the `spring-cloud-starter-bootstrap` dependency is added.

Secrets

Secrets must not be committed to the repository or baked into the image. Acceptable sources, in increasing order of maturity: environment variables injected by the platform; files mounted from a secret store and read with `spring.config.import=optional:configtree:/run/secrets/` (each file name becomes a key and the file content the value, which suits Kubernetes secrets); and an external secret manager reached through a Spring Cloud integration. The Actuator `env` and `configprops` endpoints mask values by default since Boot 3.0 (`management.endpoint.env.show-values` defaults to `never`), and must still be protected, because key names and origins are exposed.

1.3.2 Worked example

The worked example is a custom starter, `audit-spring-boot-starter`, that provides an `AuditLogger` bean. It consists of two modules, following the naming convention in the reference: an `audit-spring-boot-autoconfigure` module containing code and configuration, and an empty `audit-spring-boot-starter` module that depends on it. (The name of a third-party module must not begin with `spring-boot`.)

```
audit-spring-boot-autoconfigure/
  src/main/java/com/example/audit/
    AuditLogger.java
    AuditProperties.java
    AuditAutoConfiguration.java
  src/main/resources/META-INF/spring/
    org.springframework.boot.autoconfigure.AutoConfiguration.imports
audit-spring-boot-starter/
  pom.xml              (depends on audit-spring-boot-autoconfigure and spring-boot-starter)
```

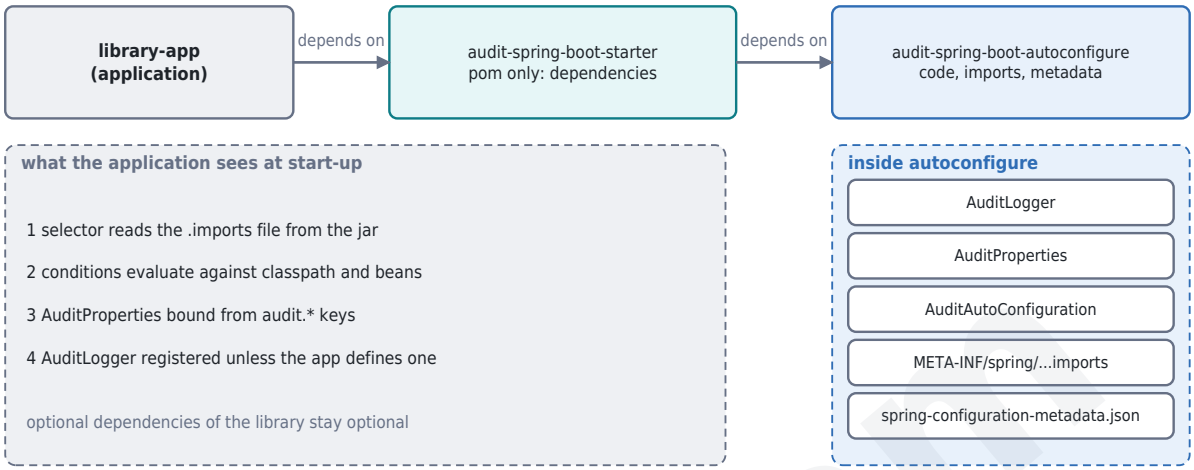


Figure 1.15. The starter module only carries dependencies; the autoconfigure module carries the code, the imports file and the metadata that the application consumes.

AuditProperties.java JAVA

```java
package com.example.audit;

import jakarta.validation.constraints.NotBlank;
import java.time.Duration;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.context.properties.bind.DefaultValue;
import org.springframework.validation.annotation.Validated;

/**
 * Configuration of the audit starter.
 *
 * @param enabled whether audit logging is active
 * @param prefix text placed in front of every audit line
 * @param slowThreshold operations slower than this are logged at WARN
 */
@ConfigurationProperties(prefix = "audit")
@Validated
public record AuditProperties(
        @DefaultValue("true") boolean enabled,
        @DefaultValue("AUDIT") @NotBlank String prefix,
        @DefaultValue("500ms") Duration slowThreshold) {
}
```

AuditLogger.java JAVA

```java
package com.example.audit;

import java.time.Duration;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class AuditLogger {

    private static final Logger log = LoggerFactory.getLogger(AuditLogger.class);

    private final String prefix;
    private final Duration slowThreshold;

    public AuditLogger(String prefix, Duration slowThreshold) {
        this.prefix = prefix;
```

AuditLogger.java (continued) JAVA

```
        this.slowThreshold = slowThreshold;
    }

    public String prefix() {
        return prefix;
    }

    public void record(String event, Duration took) {
        if (took.compareTo(slowThreshold) > 0) {
            log.warn("{} slow event={} took={}ms", prefix, event, took.toMillis());
        } else {
            log.info("{} event={} took={}ms", prefix, event, took.toMillis());
        }
    }
}
```

AuditAutoConfiguration.java JAVA

```
package com.example.audit;

import org.springframework.boot.autoconfigure.AutoConfiguration;
import org.springframework.boot.autoconfigure.condition.ConditionalOnClass;
import org.springframework.boot.autoconfigure.condition.ConditionalOnMissingBean;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;

@AutoConfiguration
@ConditionalOnClass(AuditLogger.class)
@ConditionalOnProperty(prefix = "audit", name = "enabled", havingValue = "true",
        matchIfMissing = true)
@EnableConfigurationProperties(AuditProperties.class)
public class AuditAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean            // backs off when the application defines its own
    AuditLogger auditLogger(AuditProperties properties) {
        return new AuditLogger(properties.prefix(), properties.slowThreshold());
    }
}
```

META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

```
com.example.audit.AuditAutoConfiguration
```

The auto-configuration is verified without starting the application by `ApplicationContextRunner` from `spring-boot-test`. Each test builds a minimal context containing only the auto-configuration under test.

AuditAutoConfigurationTest.java JAVA

```
package com.example.audit;

import static org.assertj.core.api.Assertions.assertThat;

import java.time.Duration;
import org.junit.jupiter.api.Test;
import org.springframework.boot.autoconfigure.AutoConfigurations;
import org.springframework.boot.test.context.runner.ApplicationContextRunner;

class AuditAutoConfigurationTest {

    private final ApplicationContextRunner runner = new ApplicationContextRunner()
            .withConfiguration(AutoConfigurations.of(AuditAutoConfiguration.class));

    @Test
    void createsLoggerWithDefaults() {
        runner.run(ctx -> assertThat(ctx).hasSingleBean(AuditLogger.class));
    }

    @Test
    void bindsProperties() {
        runner.withPropertyValues("audit.prefix=SEC", "audit.slow-threshold=2s")
              .run(ctx -> assertThat(ctx.getBean(AuditLogger.class).prefix()).isEqualTo("SEC"));
    }
```

sreram

AuditAutoConfigurationTest.java (continued) JAVA

```
    @Test
    void backsOffWhenApplicationDefinesBean() {
        runner.withBean(AuditLogger.class, () -> new AuditLogger("CUSTOM", Duration.ZERO))
              .run(ctx -> assertThat(ctx.getBean(AuditLogger.class).prefix()).isEqualTo("CUSTOM"));
    }

    @Test
    void disabledByProperty() {
        runner.withPropertyValues("audit.enabled=false")
              .run(ctx -> assertThat(ctx).doesNotHaveBean(AuditLogger.class));
    }

    @Test
    void blankPrefixFailsStartup() {
        runner.withPropertyValues("audit.prefix= ")
              .run(ctx -> assertThat(ctx).hasFailed());
    }
}
```

What to notice: no test starts a web server or scans packages; the five tests cover each decision point of the auto-configuration (default creation, binding, back-off, property switch, validation).

The mechanism that decides each of these outcomes can be modelled in plain Java. The following program keeps user beans first, then evaluates an ordered list of candidates with conditions, and prints a report in the style of the Boot condition report. The candidates use a real class-presence check (`Class.forName`), a property check, and a missing-bean check.

MiniAutoConfig.java JAVA

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Predicate;
import java.util.function.Supplier;

public class MiniAutoConfig {

    record Ctx(Map<String, Object> beans, Map<String, String> props) {
        boolean hasBeanOfType(Class<?> t) {
            return beans.values().stream().anyMatch(t::isInstance);
        }
    }

    record Condition(String description, Predicate<Ctx> test) {}

    record Candidate(String beanName, List<Condition> conditions, Supplier<Object> factory) {}

    static Condition onClass(String className) {
        return new Condition("@ConditionalOnClass(" + className + ")", c -> {
            try {
                Class.forName(className);
                return true;
            } catch (ClassNotFoundException e) {
                return false;
            }
        });
    }
    static Condition onProperty(String key, String expected) {
        return new Condition("@ConditionalOnProperty(" + key + "=" + expected + ")",
            c -> expected.equals(c.props().get(key)));
    }
    static Condition onMissingBean(Class<?> type) {
        return new Condition("@ConditionalOnMissingBean(" + type.getSimpleName() + ")",
            c -> !c.hasBeanOfType(type));
    }

    interface Clock2 { String now(); }
    record SystemClock2() implements Clock2 { public String now() { return "system"; } }
    record FixedClock2() implements Clock2 { public String now() { return "fixed"; } }
    record MetricsRegistry(String backend) {}

    public static void main(String[] args) {
```

sreeram

MiniAutoConfig.java (continued) JAVA

```
        Ctx ctx = new Ctx(new LinkedHashMap<>(), new LinkedHashMap<>());
        ctx.props().put("app.metrics.enabled", "true");

        // 1. user configuration is processed first
        ctx.beans().put("userClock", new FixedClock2());

        // 2. auto-configuration candidates (the order of the imports file), evaluated afterwards
        List<Candidate> imports = List.of(
            new Candidate("systemClock", List.of(onMissingBean(Clock2.class)), SystemClock2::new),
            new Candidate("prometheusRegistry", List.of(
                onClass("io.prometheus.client.CollectorRegistry"),
                onProperty("app.metrics.enabled", "true")),
                () -> new MetricsRegistry("prometheus")),
            new Candidate("simpleRegistry", List.of(
                onClass("java.util.concurrent.ConcurrentHashMap"),
                onProperty("app.metrics.enabled", "true"),
                onMissingBean(MetricsRegistry.class)), () -> new MetricsRegistry("simple")),
            new Candidate("debugEndpoint", List.of(onProperty("app.debug", "true")),
                () -> "endpoint"));

        List<String> positive = new ArrayList<>();
        List<String> negative = new ArrayList<>();
        for (Candidate c : imports) {
            Condition failed = c.conditions().stream()
                .filter(k -> !k.test().test(ctx)).findFirst().orElse(null);
            if (failed == null) {
                ctx.beans().put(c.beanName(), c.factory().get());
                positive.add(c.beanName());
            } else {
                negative.add(c.beanName() + ": did not match, " + failed.description());
            }
        }
        System.out.println("Positive matches: " + positive);
        System.out.println("Negative matches:");
        negative.forEach(n -> System.out.println("  " + n));
        System.out.println("Beans: " + ctx.beans());
    }
}
```

Real run on the build machine (JDK 21) TERMINAL

```
$ java MiniAutoConfig.java
Positive matches: [simpleRegistry]
Negative matches:
  systemClock: did not match, @ConditionalOnMissingBean(Clock2)
  prometheusRegistry: did not match, @ConditionalOnClass(io.prometheus.client.CollectorRegistry)
  debugEndpoint: did not match, @ConditionalOnProperty(app.debug=true)
Beans: {userClock=FixedClock2[], simpleRegistry=MetricsRegistry[backend=simple]}
```

The result mirrors the real behaviour: the user's clock suppresses `systemClock`; the first registry candidate fails on a missing class; the second matches because the first did not register a bean (`@ConditionalOnMissingBean` is order-sensitive).

1.3.3 Hands-on

An application that includes the starter and sets `audit.prefix=SEC` in `application.yml` can be asked to explain itself. Starting it with `--debug` (or `debug=true`) prints the condition evaluation report. The output below is the typical shape of the report for the audit starter (not run on the build machine).

typical output (not run on the build machine) TERMINAL

```
$ java -jar library-app.jar --debug
============================
CONDITIONS EVALUATION REPORT
============================

Positive matches:
-----------------

   AuditAutoConfiguration matched:
      - @ConditionalOnClass found required class 'com.example.audit.AuditLogger' (OnClassCondition)
      - @ConditionalOnProperty (audit.enabled=true) matched (OnPropertyCondition)
```

typical output (not run on the build machine) (continued) TERMINAL

```
   AuditAutoConfiguration#auditLogger matched:
      - @ConditionalOnMissingBean (types: com.example.audit.AuditLogger; SearchStrategy: all)
        did not find any beans (OnBeanCondition)

Negative matches:
-----------------

   ActiveMQAutoConfiguration:
      Did not match:
         - @ConditionalOnClass did not find required class 'jakarta.jms.ConnectionFactory'
```

In a running application the same information is available over HTTP when the endpoints are exposed (`management.endpoints.web.exposure.include=conditions,configprops,env`):

typical commands (not run on the build machine) TERMINAL

```
$ curl -s localhost:8080/actuator/conditions | jq '.contexts[].positiveMatches | keys[]' | head -3
$ curl -s localhost:8080/actuator/configprops/audit
$ curl -s localhost:8080/actuator/env/audit.prefix
```

`/actuator/env/audit.prefix` lists every property source that defines the key in precedence order, with the origin (file and line) of the winning value, which settles most disputes about "which file won".

1.3.4 Real-world problems and solutions

Problem 1: a property is set but the value is not applied

Scenario. `audit.slowThreshold: 2s` was added to `application.yml` and the service still warns about 600 ms operations.

Symptoms. The bean shows the default of 500 ms; there is no error, because an unbound but spelled-differently property is simply an unknown key.

Diagnosis. (1) `GET /actuator/configprops/audit` shows the bound values and confirms the effective `slowThreshold`. (2) `GET /actuator/env/audit.slow-threshold` lists the property sources that define the canonical key; an empty result means the key never reached the `Environment`. (3) `logging.level.org.springframework.boot.context.config=TRACE` prints every config file candidate that was loaded or skipped. Typical causes found this way: the file sits in `src/test/resources` or is not copied to the jar; it is named `application.yaml` and `application.yml` at once; a `spring.config.location` entry replaced the default locations; a profile-specific file is not active; the YAML indent places the key under another prefix.

When a value is found but invalid, the failure analyser prints a structured explanation (typical text):

```
APPLICATION FAILED TO START

Description:

Failed to bind properties under 'audit' to com.example.audit.AuditProperties:

    Property: audit.slow-threshold
    Value: "2 secs"
    Origin: class path resource [application.yml] - 4:19
    Reason: failed to convert java.lang.String to java.time.Duration

Action:

Update your application's configuration
```

Root cause. In the scenario the key was written under the wrong parent (`audit.slow-threshold` placed under `spring:`), so it was a different key.

Fix. Correct the nesting. Use the Duration formats `2s`, `500ms`, `PT2S` or the ISO-8601 form; `2 secs` is invalid.

Prevention. Add `spring-boot-configuration-processor` to the autoconfigure module as an annotation processor so that IDEs flag unknown keys; use `@Validated` for constraints; and in tests assert the bound object.

Problem 2: an auto-configured bean is missing or has been replaced

Scenario. After a team declared a `DataSource` bean for a second database, the application's HikariCP settings under `spring.datasource.hikari.*` stopped having any effect, and the Actuator `metrics` no longer showed the `hikaricp.*` series.

Symptoms. Properties are ignored without a message; the pool shows default values.

Diagnosis. Run with `--debug` and search for `DataSourceAutoConfiguration` in the negative matches: `@ConditionalOnMissingBean (types: javax.sql.DataSource; SearchStrategy: all) found beans 'secondaryDataSource'`. This line is the definitive explanation: the application bean made the auto-configuration back off. `GET /actuator/beans` confirms that only the user-defined `DataSource` exists.

Root cause. Auto-configuration is a default, not a mandate. The user-defined bean replaced the auto-configured one, and the user-defined bean never read the `spring.datasource.*` properties.

Fix. Create each data source explicitly and bind the properties explicitly:

DataSourceConfig.java JAVA

```java
@Configuration(proxyBeanMethods = false)
public class DataSourceConfig {

    @Bean
    @Primary
    @ConfigurationProperties("app.datasource.primary")
    HikariDataSource primaryDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).build();
    }

    @Bean
    @ConfigurationProperties("app.datasource.secondary")
    HikariDataSource secondaryDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).build();
    }
}
```

Prevention. Treat any user-declared bean of an infrastructure type (`DataSource`, `ObjectMapper`, `RestTemplate`, `TransactionManager`) as a decision to own that type's configuration, and keep the condition report in the diagnostic toolkit.

Problem 3: the wrong value wins in production

Scenario. `server.port` is 8080 in `application.yml` and 8443 in `application-prod.yml`. The production container listens on 9000.

Symptoms. The port differs from both files.

Diagnosis. `GET /actuator/env/server.port` (or the `env` endpoint output) lists sources in precedence order: `systemEnvironment: SERVER_PORT=9000`, then `applicationConfig: [classpath:/application-prod.yml]`, then the base file. The environment variable defined in the deployment manifest wins over both files.

Root cause. The precedence rules were not applied: OS environment variables override config data. The manifest had been copied from another service.

Fix. Remove the stale variable, or make the intent explicit. Keep a single authoritative place per setting.

Prevention. Document the intended source of each deployment-specific setting and print the winning origin during start-up checks. Do not rely on overriding the same key in many layers.

1.3.5 Questions to expect

1. **What does `@SpringBootApplication` consist of?** `@SpringBootConfiguration`, `@EnableAutoConfiguration` and `@ComponentScan` with two filters.
2. **How does auto-configuration find its classes?** `AutoConfigurationImportSelector` reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` from all jars (`spring.factories` before Boot 2.7), filters by `OnClassCondition` metadata, orders the rest and imports them as deferred configuration.
3. **How can an application override an auto-configured bean?** Declare a bean of the same type; the auto-configured bean's `@ConditionalOnMissingBean` condition fails and it backs off.

sreeram

4. **How is the reason for a missing bean found?** Start with `--debug` or call `/actuator/conditions` and search the negative matches.
5. **What is relaxed binding?** Several spellings of a key map to one canonical kebab-case name; environment variables use upper case with underscores and no dashes.
6. **What is the order of property sources?** Defaults, `@PropertySource`, config data (jar, then outside the jar, with profile files after non-profile files), environment variables, system properties, JSON, command line, test sources; the last wins.
7. `@Value` **versus** `@ConfigurationProperties`**?** `@Value` injects a single expression, matches relaxed forms only when the canonical kebab-case name is used in the placeholder, and offers no validation or metadata; `@ConfigurationProperties` binds a typed, validated structure with relaxed rules and IDE metadata.
8. **How is a custom starter built?** An autoconfigure module with `@AutoConfiguration` classes, an `.imports` file, `@ConfigurationProperties` and conditions, plus a starter module that only declares dependencies.
9. **How are secrets supplied safely?** Environment variables, mounted secret files imported with `configtree:`, or a secret manager; never in the repository or image.

Check yourself

1. A bean class lives in `com.example.shared`, the main class in `com.example.library`. Is the bean created?
2. In `MiniAutoConfig`, what happens if `simpleRegistry` is listed before `prometheusRegistry` and the Micrometer class is on the classpath?
3. Which wins: `--server.port=1` or `SERVER_PORT=2`?
4. `audit.slow-threshold=2 secs`: what happens at start-up?
5. Why must the `@Bean` method of an auto-configuration declare a specific return type?

Answers

1. No. Component scanning covers only the main class's package and below; use `scanBasePackages` or `@Import`.
2. `simpleRegistry` registers first. `prometheusRegistry` has no `@ConditionalOnMissingBean`, so it registers as well and two `MetricsRegistry` beans exist. Order matters only for conditions that inspect beans.
3. The command-line argument; it has a higher priority than environment variables.
4. Binding fails and the application stops with a `BindException` explanation naming property, value, origin and reason.
5. Bean conditions inspect the declared return type before the method is invoked.

KEY POINTS

- Auto-configuration is a list in `.imports` files whose entries are filtered and guarded by conditions; user beans are processed first and make library beans back off.
- `--debug` and `/actuator/conditions` explain every decision; `/actuator/configprops` and `/actuator/env/<key>` explain every value and its origin.
- Relaxed binding accepts several spellings; kebab-case is canonical, environment variables drop dashes.
- The last property source wins: file, environment, system property, JSON, command line, test sources.
- A starter is an autoconfigure module plus an empty dependency module; test it with `ApplicationContextRunner`.
- Secrets belong in the platform, never in the repository.

1.4 The Spring MVC Request Lifecycle

A REST endpoint that appears to consist of one annotated method is, at run time, a pipeline of at least a dozen components: a socket acceptor, a worker thread, servlet filters, the `DispatcherServlet`, a handler mapping, interceptors, argument resolvers, the controller, a return-value handler, an HTTP message converter, and an exception resolver. Every observable symptom of a web defect (a 415 response, a request that hangs, a missing CORS header, a lazy-loading exception during JSON output) belongs to exactly one stage of this pipeline. This chapter follows one request through the stages in order, names the classes involved, and then applies the model to four kinds of defects.

1.4.1 Concept and mechanism

From the socket to the DispatcherServlet

Spring Boot embeds Apache Tomcat (through `TomcatServletWebServerFactory`). The default connector is `Http11NioProtocol`. Its structure determines how many requests the service can process at once.

- An **acceptor** thread accepts new TCP connections and hands them to a **poller**, which watches the open sockets with a selector (NIO). Idle keep-alive connections cost a socket and a small buffer, not a thread.
- When a request is readable, the poller assigns a **worker thread** from the executor pool. The worker parses the HTTP request line and headers, runs the servlet filter chain and the `DispatcherServlet`, writes the response, and returns to the pool.
- The pool is bounded: `server.tomcat.threads.max` (default 200) and `server.tomcat.threads.min-spare` (default 10, threads kept ready).
- `server.tomcat.max-connections` (default 8192) limits connections that Tomcat accepts and watches. When that limit is reached the operating system queues further connections up to `server.tomcat.accept-count` (default 100); beyond it the connection attempt is refused.

The central fact is that with a blocking controller **one worker thread is occupied for the full duration of a request**, including time spent waiting for a database, a downstream service or a lock. Throughput is therefore bounded by `threads.max / mean request duration`.

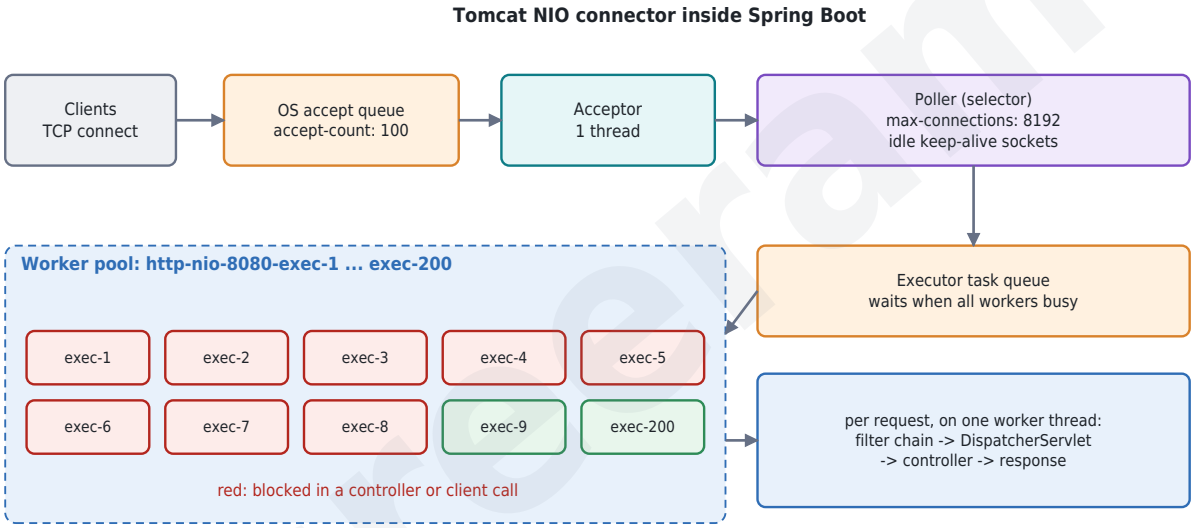


Figure 1.16. The acceptor and poller are cheap; the worker pool is the bottleneck, and requests that find every worker busy wait in the executor queue and then in the accept queue.

DispatcherServlet.doDispatch

Tomcat calls the servlet `service()` method, which `FrameworkServlet` routes to `processRequest()` and then to `DispatcherServlet.doService()` and `doDispatch()`. This method is the front controller of Spring MVC. Its steps, in order:

1. `getHandler()` iterates over the `HandlerMapping` beans in order (`RequestMappingHandlerMapping` for `@RequestMapping` methods, then `RouterFunctionMapping`, `BeanNameUrlHandlerMapping`, the static resource mapping) and returns a `HandlerExecutionChain`: the handler (a `HandlerMethod`) plus the applicable `HandlerInterceptor` list. When none matches, the result is a 404, or `NoHandlerFoundException` when configured.
2. `getHandlerAdapter()` selects the adapter that can invoke the handler (`RequestMappingHandlerAdapter` for `HandlerMethod`).
3. `HandlerInterceptor.preHandle()` is called in registration order. A `false` return stops the chain and triggers `afterCompletion` for the interceptors that already ran.
4. `RequestMappingHandlerAdapter.handle()` creates an `InvocableHandlerMethod`. Each parameter is filled by a `HandlerMethodArgumentResolver`: `@PathVariable`, `@RequestParam`, `@RequestHeader`, `@ModelAttribute` (form binding with validation), and `@RequestBody` (`RequestResponseBodyMethodProcessor`, which reads the body through an `HttpMessageConverter` and applies `@Valid`).
5. The controller method runs.

6. A `HandlerMethodReturnValueHandler` processes the result. For `@ResponseBody` and `ResponseEntity` it selects an `HttpMessageConverter` by content negotiation and writes the body; for a view name it prepares a `ModelAndView`.
7. `postHandle()` is called in **reverse** order, then the result is rendered.
8. If any step threw an exception, `processHandlerException()` asks the `HandlerExceptionResolver` composite: `ExceptionHandlerExceptionResolver` (methods annotated `@ExceptionHandler`, local to the controller or global through `@ControllerAdvice`), `ResponseStatusExceptionResolver` (`@ResponseStatus`, `ResponseStatusException`), and `DefaultHandlerExceptionResolver` (standard Spring exceptions mapped to 4xx and 5xx).
9. `afterCompletion()` is called in reverse order for every interceptor whose `preHandle` returned true, in all cases, including exceptions.

An exception that no resolver handles is rethrown to the container, which forwards to `/error`, served by `BasicErrorController`.

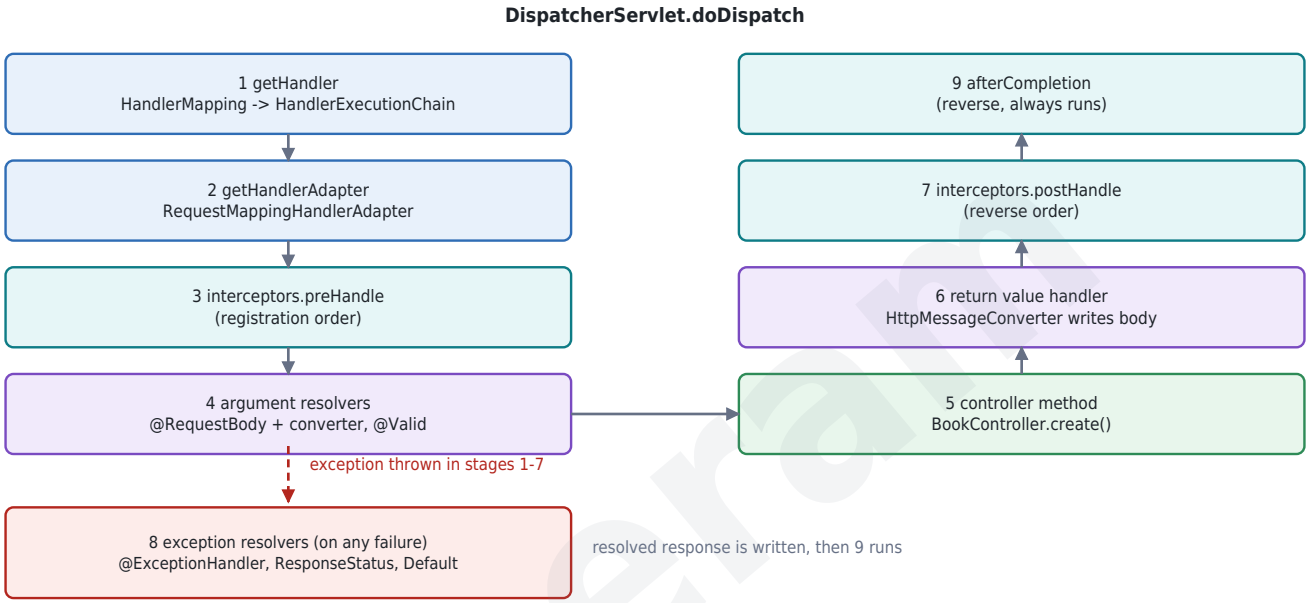


Figure 1.17. The numbered stages of doDispatch; the controller is step 5 of 9, and the failure classes of the next sections map to stages 1, 4, 6 and 8.

Filters versus interceptors

Aspect	Servlet Filter	HandlerInterceptor
Defined by	Servlet specification; managed by the container	Spring MVC
Position	Before the DispatcherServlet; also sees static resources, /error and requests with no handler	Inside the DispatcherServlet, after the handler is chosen
Can wrap request or response	Yes (HttpServletRequestWrapper), so it can replace the body or headers	No; it can only read and set attributes and headers
Knows the controller method	No	Yes (the handler argument is a HandlerMethod)
Exceptions	Must handle its own; @ExceptionHandler does not see them	Exceptions reach exception resolvers (preHandle) and afterCompletion
Typical use	Security (Spring Security is a filter chain), request ID and MDC, compression, CORS	Per-handler timing, audit, locale, authorisation checks that need handler annotations

Servlet filter versus Spring MVC interceptor.

A filter registered as a Spring bean is added to the servlet chain automatically (by `ServletContextInitializerBeans`); its order is controlled with `@Order` or `FilterRegistrationBean.setOrder`. The registered filters execute in that order before the `DispatcherServlet`; Spring Security's `DelegatingFilterProxy` participates in the same chain.

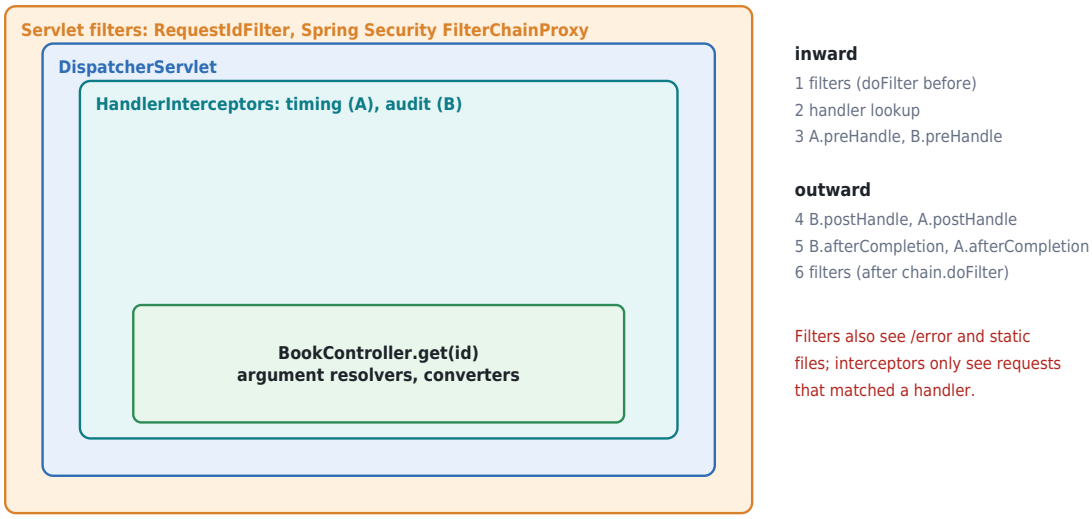


Figure 1.18. Filters wrap the whole servlet call; interceptors run only for requests that reached a handler, and the postHandle and afterCompletion callbacks unwind in reverse order.

Message conversion and content negotiation

The step that selects the representation is content negotiation. For a `@RequestBody` parameter, the converter is chosen by the request `Content-Type` against `consumes`; for the response, by the request `Accept` header, the `produces` condition of the mapping, and the converters registered. Boot registers, among others, `ByteArrayHttpMessageConverter`, `StringHttpMessageConverter` (UTF-8), `ResourceHttpMessageConverter`, and `MappingJackson2HttpMessageConverter` when Jackson is on the classpath. The order matters: a method returning `String` is written by the string converter as `text/plain`, not as a JSON string.

Status	Exception	Typical cause
400	HttpMessageNotReadableException	Malformed JSON, wrong type for a field, missing body when required
400	MethodArgumentNotValidException	A @Valid body failed Bean Validation (@NotBlank, @Min ...)
415	HttpMediaTypeNotSupportedException	The Content-Type of the request is not consumed by any converter or by the `consumes` condition (for example form data posted to a JSON-only endpoint)
406	HttpMediaTypeNotAcceptableException	The Accept header lists only types the endpoint cannot produce (Accept: application/xml with Jackson only)
405	HttpRequestMethodNotSupportedException	The path matches, the HTTP method does not

Message-conversion failures and their status codes.

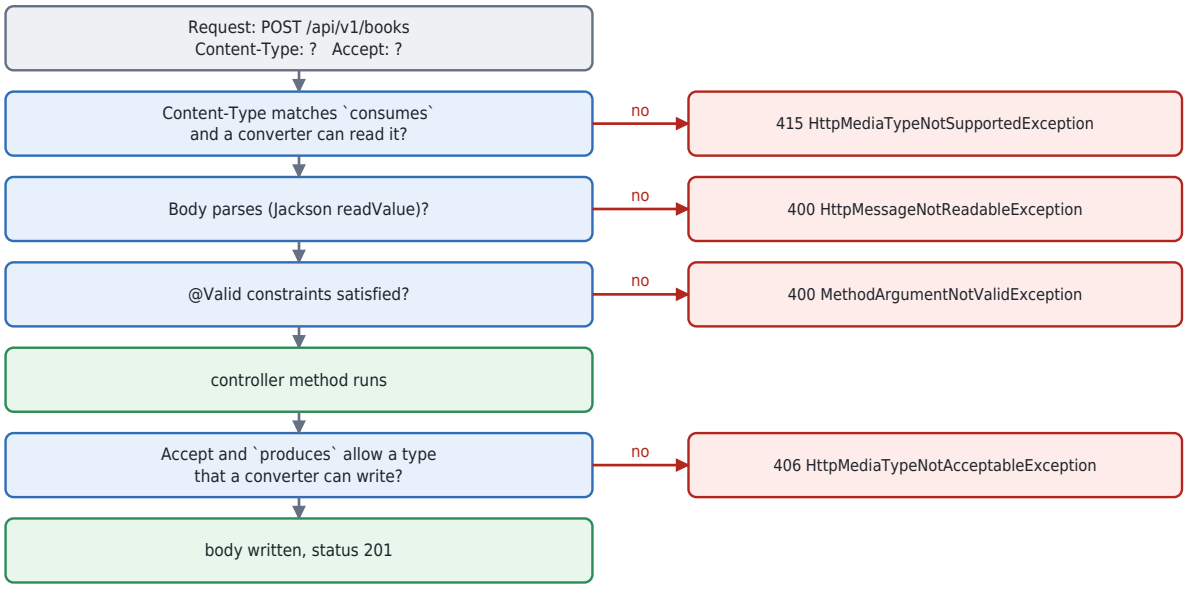


Figure 1.19. Content negotiation: the Content-Type header selects the reading converter (415 when absent), the Accept header and `produces` select the writing converter (406 when absent).

Jackson's behaviour in Boot differs from a bare `ObjectMapper`: Boot builds the mapper with `Jackson2ObjectMapperBuilder` and disables `FAIL_ON_UNKNOWN_PROPERTIES` and `WRITE_DATES_AS_TIMESTAMPS`, registers the `jackson-datatype-jsr310` module so `java.time` types serialise as ISO-8601 strings, and honours the `spring.jackson.*` properties. Declaring a custom `ObjectMapper` bean makes the auto-configured one back off, and the application then loses these defaults; customise with a `Jackson2ObjectMapperBuilderCustomizer` bean instead.

Asynchronous request processing

A servlet container can release the worker thread while a result is pending: `request.startAsync()` detaches the request and the thread returns to the pool; when the result is ready, the container performs an **async dispatch** that passes through the `DispatcherServlet` again and writes the response. Spring MVC exposes this through controller return types:

- `Callable<T>`: Spring runs the callable on a `TaskExecutor` (in Boot, `applicationTaskExecutor`).
- `DeferredResult<T>` and `CompletableFuture<T>`: the application completes the result from any thread (a message listener, a client callback).
- `StreamingResponseBody` and `SseEmitter`: streaming output and server-sent events.

`spring.mvc.async.request-timeout` bounds the wait; an expired request yields a 503 via `AsyncRequestTimeoutException`. Asynchronous MVC does not make the downstream call non-blocking: a `Callable` that blocks on a database still occupies a thread, on the task executor instead of a Tomcat worker. The gain is real only when the work is fed by an event rather than computed by a blocked thread.

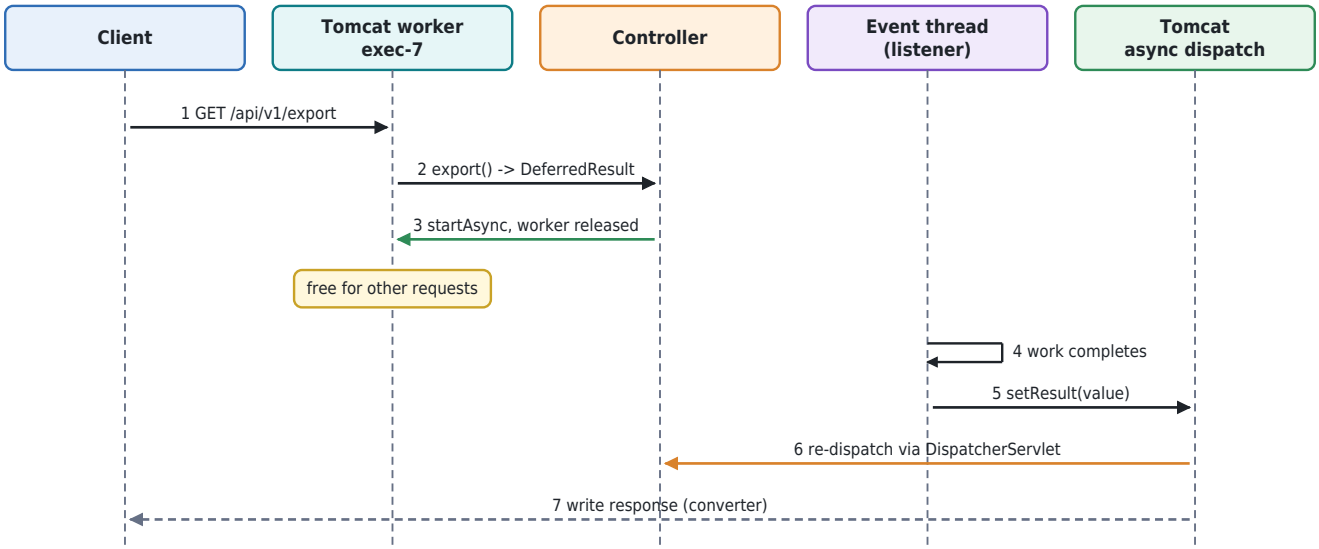


Figure 1.20. With DeferredResult the Tomcat worker is released at step 3 and a second dispatch writes the response at step 7.

Virtual threads in Spring Boot 3.2 and later

Since Spring Boot 3.2 on Java 21, setting `spring.threads.virtual.enabled=true` configures Tomcat (and Jetty) to run each request on a **virtual thread** and switches the `applicationTaskExecutor` and scheduler to virtual-thread implementations. The blocking style of the controller code is unchanged; a blocked virtual thread unmounts from its carrier thread, so the limit of 200 workers no longer caps the number of concurrent requests that are waiting. Consequences that must be understood:

- The bottleneck moves downstream. A database pool of 10 connections still allows only 10 concurrent queries; thousands of waiting requests simply queue on the pool (and need a timeout, see the clients chapter).
- On JDK 21, blocking inside a `synchronized` block **pins** the virtual thread to its carrier, which reintroduces thread starvation; JDK 24 (JEP 491) removed the pinning for `synchronized`. Native calls and some file operations still pin.
- Code that relies on thread-pool bounds for load limiting (for example one pool per downstream) must introduce explicit limits such as semaphores or bulkheads (see the chapter on resilience).
- `ThreadLocal` values are per virtual thread and are not shared across requests; avoid caching large objects in thread locals.

A plain-JDK measurement that mirrors the effect is presented in the hands-on section.

Other mechanisms in brief

File upload. `MultipartFile` parameters are filled by `StandardServletMultipartResolver`. Limits are `spring.servlet.multipart.max-file-size` (default 1MB) and `max-request-size` (default 10MB); exceeding either gives a 413-type failure (`MaxUploadSizeExceededException`, usually surfaced as 413 by Boot's error handling). For large files, stream with `file.getInputStream()` to the destination and never call `getBytes()`.

CORS. Cross-origin requests that are not "simple" are preceded by a preflight `OPTIONS` request with `Origin`, `Access-Control-Request-Method` and `Access-Control-Request-Headers`. Spring MVC answers it in the handler mapping (via `CorsFilter` or the `CorsProcessor` of `AbstractHandlerMapping`) according to `@CrossOrigin` or `addCorsMappings`, **without invoking the controller**. When Spring Security is present the preflight must also pass the security filter chain, which requires `http.cors(Customizer.withDefaults())` so that the CORS configuration is applied before authentication.

HTTP caching. A `ResponseEntity` with an `eTag(...)` or `lastModified(...)` header makes the framework answer a matching conditional `GET` (`If-None-Match`) with `304 Not Modified` and no body. `ShallowEtagHeaderFilter` computes the ETag from the generated body for any endpoint; it saves bandwidth, not server computation, because the body is still produced. `Cache-Control` is set with `CacheControl.maxAge(...)`.

Pagination and sorting. Spring Data's `PageableHandlerMethodArgumentResolver` converts `?page=0&size=20&sort=title,asc` into a `Pageable`. The default maximum page size is 2000 (`spring.data.web.pageable.max-page-size`); public APIs should lower it. Sort property names are passed to the data layer, so they must be restricted to a whitelist of allowed properties. Return DTOs, not entities.

API versioning. In Spring Framework 6.2 and Boot 3.x there is no first-class versioning mechanism; the practical strategies are a path prefix (`/api/v1/books`), a custom header (`X-API-Version`), and media-type versioning with the `produces` condition (`application/vnd.library.v2+json`). First-class version support (a `version` attribute on mappings and `spring.mvc.apiversion.*` properties) arrived with Spring Framework 7 and Boot 4. Whatever the strategy, a change that removes or renames a field requires a new version; adding optional fields does not.

Error format. `ProblemDetail` implements RFC 9457 (which obsoletes RFC 7807) with the members `type`, `title`, `status`, `detail`, `instance` and extension members, served as `application/problem+json`. Extending `ResponseEntityExceptionHandler` converts all standard MVC exceptions to this format. If the application declares no such advice, `spring.mvc.problemdetails.enabled=true` enables Boot's built-in equivalent.

1.4.2 Worked example

The example is the REST API of a library: a paged book list with sorting, a single book with an ETag, creation with validation, a uniform error format, a request-ID filter, a timing interceptor and a CORS policy. The code is Spring Boot 3.x with `spring-boot-starter-web`, `spring-boot-starter-validation` and Spring Data Commons (for `Pageable`).

BookDto.java JAVA

```java
package com.example.library.book;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record BookDto(long id, String title, String author, int year, long version) {

    public record CreateRequest(
            @NotBlank @Size(max = 200) String title,
            @NotBlank String author,
            @Min(1450) @Max(2100) int year) {
    }
}
```

BookService.java JAVA

```java
package com.example.library.book;

import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageImpl;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;

@Service
public class BookService {

    private static final List<String> SORTABLE = List.of("title", "author", "year");

    private final Map<Long, BookDto> store = new ConcurrentHashMap<>();
    private final AtomicLong ids = new AtomicLong();

    public Page<BookDto> page(Pageable pageable) {
        Comparator<BookDto> order = Comparator.comparing(BookDto::title);
        Sort.Order requested = pageable.getSort().stream().findFirst().orElse(null);
        if (requested != null) {
            if (!SORTABLE.contains(requested.getProperty())) {
                throw new IllegalArgumentException("Unsupported sort property: "
                        + requested.getProperty());
            }
            order = switch (requested.getProperty()) {
                case "author" -> Comparator.comparing(BookDto::author);
                case "year" -> Comparator.comparingInt(BookDto::year);
                default -> Comparator.comparing(BookDto::title);
            };
            if (requested.isDescending()) {
                order = order.reversed();
```

BookService.java (continued) JAVA

```
            }
        }
        List<BookDto> all = store.values().stream().sorted(order).toList();
        int from = (int) Math.min(pageable.getOffset(), all.size());
        int to = Math.min(from + pageable.getPageSize(), all.size());
        return new PageImpl<>(all.subList(from, to), pageable, all.size());
    }

    public BookDto find(long id) {
        BookDto book = store.get(id);
        if (book == null) {
            throw new BookNotFoundException(id);
        }
        return book;
    }

    public BookDto create(BookDto.CreateRequest request) {
        long id = ids.incrementAndGet();
        BookDto book = new BookDto(id, request.title(), request.author(), request.year(), 1);
        store.put(id, book);
        return book;
    }
}
```

BookNotFoundException.java JAVA

```
package com.example.library.book;

public class BookNotFoundException extends RuntimeException {

    private final long bookId;

    public BookNotFoundException(long bookId) {
        super("No book with id " + bookId);
        this.bookId = bookId;
    }

    public long bookId() {
        return bookId;
    }
}
```

BookController.java JAVA

```
package com.example.library.book;

import jakarta.validation.Valid;
import java.net.URI;
import java.time.Duration;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.CacheControl;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

@RestController
@RequestMapping(path = "/api/v1/books", produces = MediaType.APPLICATION_JSON_VALUE)
public class BookController {

    private final BookService books;

    public BookController(BookService books) {
        this.books = books;
    }

    @GetMapping
```

BookController.java (continued) JAVA

```java
    public Page<BookDto> list(@PageableDefault(size = 20, sort = "title") Pageable pageable) {
        return books.page(pageable);
    }

    @GetMapping("/{id}")
    public ResponseEntity<BookDto> get(@PathVariable long id) {
        BookDto book = books.find(id);
        return ResponseEntity.ok()
                .eTag("\"" + book.version() + "\"")
                .cacheControl(CacheControl.maxAge(Duration.ofSeconds(30)).cachePrivate())
                .body(book);
    }

    @PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<BookDto> create(@Valid @RequestBody BookDto.CreateRequest request) {
        BookDto created = books.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(created.id()).toUri();
        return ResponseEntity.created(location).body(created);
    }
}
```

ApiExceptionHandler.java JAVA

```java
package com.example.library.web;

import com.example.library.book.BookNotFoundException;
import java.net.URI;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(BookNotFoundException.class)
    ProblemDetail bookNotFound(BookNotFoundException ex) {
        ProblemDetail problem =
                ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Book not found");
        problem.setType(URI.create("https://library.example.com/problems/book-not-found"));
        problem.setProperty("bookId", ex.bookId());
        return problem;
    }

    @ExceptionHandler(IllegalArgumentException.class)
    ProblemDetail badArgument(IllegalArgumentException ex) {
        ProblemDetail problem =
                ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, ex.getMessage());
        problem.setTitle("Invalid request");
        return problem;
    }
}
```

`ResponseEntityExceptionHandler` already maps `MethodArgumentNotValidException`, `HttpMessageNotReadableException`, `HttpMediaTypeNotSupportedException`, `HttpMediaTypeNotAcceptableException` and the other standard exceptions to `ProblemDetail` responses, so only domain exceptions need handlers.

RequestIdFilter.java JAVA

```java
package com.example.library.web;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.core.Ordered;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;
```

RequestIdFilter.java (continued) JAVA

```java
import org.springframework.web.filter.OncePerRequestFilter;

@Component
@Order(Ordered.HIGHEST_PRECEDENCE + 10)
public class RequestIdFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws ServletException, IOException {
        String id = request.getHeader("X-Request-Id");
        if (id == null || id.isBlank()) {
            id = UUID.randomUUID().toString();
        }
        MDC.put("requestId", id);
        response.setHeader("X-Request-Id", id);
        try {
            chain.doFilter(request, response);
        } finally {
            MDC.remove("requestId");            // worker threads are reused: always clean up
        }
    }
}
```

WebConfig.java JAVA

```java
package com.example.library.web;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.HandlerInterceptor;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    private static final Logger log = LoggerFactory.getLogger(WebConfig.class);

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new HandlerInterceptor() {
            @Override
            public boolean preHandle(HttpServletRequest req, HttpServletResponse res,
                                     Object handler) {
                req.setAttribute("startNanos", System.nanoTime());
                return true;
            }

            @Override
            public void afterCompletion(HttpServletRequest req, HttpServletResponse res,
                                        Object handler, Exception ex) {
                long ms = (System.nanoTime() - (Long) req.getAttribute("startNanos")) / 1_000_000;
                log.info("{} {} -> {} in {} ms", req.getMethod(), req.getRequestURI(),
                        res.getStatus(), ms);
            }
        }).addPathPatterns("/api/**");
    }

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("https://app.library.example.com")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowedHeaders("Authorization", "Content-Type", "If-None-Match")
                .exposedHeaders("ETag", "Location", "X-Request-Id")
                .maxAge(3600);
    }
}
```

sreeram

What to notice: the filter cleans the `MDC` in `finally` because Tomcat reuses worker threads; the interceptor measures the handler time but cannot see time spent in filters; the CORS rule lists headers explicitly (the wildcard `*` combined with credentials is rejected by browsers) and exposes `ETag` so that browser JavaScript can read it.

1.4.3 Hands-on

Two plain-JDK programs reproduce the mechanisms of this chapter with real output. The first models the servlet filter chain and the `doDispatch` sequence: handler lookup, interceptor `preHandle`, `postHandle` and `afterCompletion` ordering, exception resolution, and short-circuiting by an interceptor.

MiniDispatcher.java JAVA

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

public class MiniDispatcher {

    record Request(String method, String path, Map<String, String> params) {}
    record Response(int status, String body) {}

    // ---- servlet filter chain ------------------------------------------------
    interface Filter { Response doFilter(Request req, Chain chain); }
    interface Chain { Response next(Request req); }

    static Chain chainOf(List<Filter> filters, Function<Request, Response> servlet) {
        return new Chain() {
            int index = 0;
            public Response next(Request req) {
                return index < filters.size()
                    ? filters.get(index++).doFilter(req, this) : servlet.apply(req);
            }
        };
    }

    // ---- handler interceptors and exception resolution -----------------------
    interface Interceptor {
        boolean preHandle(Request r);
        void postHandle(Request r, Response resp);
        void afterCompletion(Request r, Exception ex);
    }

    record Handler(String name, Function<Request, String> body) {}

    static class Dispatcher implements Function<Request, Response> {
        final Map<String, Handler> mappings = new LinkedHashMap<>();
        final List<Interceptor> interceptors = new ArrayList<>();

        public Response apply(Request req) {                        // doDispatch
            Handler h = mappings.get(req.method() + " " + req.path());
            if (h == null) return new Response(404, "no handler");
            int applied = 0;
            Exception failure = null;
            Response resp = null;
            try {
                for (Interceptor i : interceptors) {
                    if (!i.preHandle(req)) {
                        triggerAfter(req, applied, null);
                        return new Response(403, "blocked");
                    }
                    applied++;
                }
                resp = new Response(200, h.body().apply(req));
                for (int i = interceptors.size() - 1; i >= 0; i--) {
                    interceptors.get(i).postHandle(req, resp);
                }
            } catch (Exception ex) {                                // HandlerExceptionResolver
                failure = ex;
                int status = ex instanceof IllegalArgumentException ? 400 : 500;
                resp = new Response(status, ex.getMessage());
            }
            triggerAfter(req, applied, failure);
            return resp;
```

MiniDispatcher.java (continued) JAVA

```
        }

        void triggerAfter(Request req, int applied, Exception ex) {
            for (int i = applied - 1; i >= 0; i--) interceptors.get(i).afterCompletion(req, ex);
        }
    }

    static Interceptor named(String n, boolean allow) {
        return new Interceptor() {
            public boolean preHandle(Request r) {
                System.out.println("    " + n + ".preHandle -> " + allow);
                return allow;
            }
            public void postHandle(Request r, Response x) {
                System.out.println("    " + n + ".postHandle");
            }
            public void afterCompletion(Request r, Exception e) {
                String msg = e == null ? null : e.getMessage();
                System.out.println("    " + n + ".afterCompletion ex=" + msg);
            }
        };
    }

    static Filter named(String n) {
        return (req, chain) -> {
            System.out.println("  " + n + " before");
            Response r = chain.next(req);
            System.out.println("  " + n + " after  (status " + r.status() + ")");
            return r;
        };
    }

    public static void main(String[] args) {
        Dispatcher d = new Dispatcher();
        d.mappings.put("GET /books",
            new Handler("BookController#list", r -> "[book1, book2]"));
        d.mappings.put("GET /boom", new Handler("BookController#boom", r -> {
            throw new IllegalArgumentException("page must be >= 0"); }));
        d.interceptors.add(named("A", true));
        d.interceptors.add(named("B", true));
        List<Filter> filters = List.of(named("CharacterEncodingFilter"), named("SecurityFilter"));

        for (String path : List.of("/books", "/boom", "/missing")) {
            System.out.println("GET " + path);
            Response r = chainOf(filters, d).next(new Request("GET", path, Map.of()));
            System.out.println("  => " + r);
        }

        System.out.println("GET /books with interceptor B rejecting");
        d.interceptors.set(1, named("B", false));
        Request again = new Request("GET", "/books", Map.of());
        System.out.println("  => " + chainOf(filters, d).next(again));
    }
}
```

Real run on the build machine (JDK 21) TERMINAL

```
$ java MiniDispatcher.java
GET /books
  CharacterEncodingFilter before
  SecurityFilter before
    A.preHandle -> true
    B.preHandle -> true
    B.postHandle
    A.postHandle
    B.afterCompletion ex=null
    A.afterCompletion ex=null
  SecurityFilter after  (status 200)
  CharacterEncodingFilter after  (status 200)
  => Response[status=200, body=[book1, book2]]
GET /boom
  CharacterEncodingFilter before
  SecurityFilter before
    A.preHandle -> true
```

Real run on the build machine (JDK 21) (continued) TERMINAL

```
    B.preHandle -> true
    B.afterCompletion ex=page must be >= 0
    A.afterCompletion ex=page must be >= 0
  SecurityFilter after  (status 400)
  CharacterEncodingFilter after  (status 400)
  => Response[status=400, body=page must be >= 0]
GET /missing
  CharacterEncodingFilter before
  SecurityFilter before
  SecurityFilter after  (status 404)
  CharacterEncodingFilter after  (status 404)
  => Response[status=404, body=no handler]
GET /books with interceptor B rejecting
  CharacterEncodingFilter before
  SecurityFilter before
    A.preHandle -> true
    B.preHandle -> false
    A.afterCompletion ex=null
  SecurityFilter after  (status 403)
  CharacterEncodingFilter after  (status 403)
  => Response[status=403, body=blocked]
```

What to notice: for `/boom` the `postHandle` callbacks are skipped but both `afterCompletion` callbacks run and receive the exception; the filters see the final status 400 on the way out, which proves that exception resolution happens inside the dispatcher, before the filters unwind. For `/missing` no interceptor runs at all, because no handler was found, while both filters still run. When interceptor B rejects the request, only interceptor A (the one that already ran) receives `afterCompletion`.

The second program measures thread-per-request starvation. A pool of platform threads stands in for the Tomcat worker pool (scaled down from 200 to 10 and 50 threads) and each of 100 requests blocks for 200 ms, as a call to a slow downstream service would. The third line uses one virtual thread per request, which is what `spring.threads.virtual.enabled=true` configures.

PoolStarvation.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class PoolStarvation {

    static final int REQUESTS = 100;
    static final int BLOCKING_MS = 200;

    /** A handler that blocks on a downstream call; returns queueing delay plus service time. */
    static long handle(long submittedNanos) throws InterruptedException {
        Thread.sleep(BLOCKING_MS);
        return (System.nanoTime() - submittedNanos) / 1_000_000;
    }

    static void run(String label, ExecutorService pool) throws Exception {
        long t0 = System.nanoTime();
        List<Future<Long>> results = new ArrayList<>();
        for (int i = 0; i < REQUESTS; i++) {
            long submitted = System.nanoTime();
            results.add(pool.submit(() -> handle(submitted)));
        }
        long worst = 0;
        long sum = 0;
        for (Future<Long> f : results) {
            long ms = f.get();
            worst = Math.max(worst, ms);
            sum += ms;
        }
        long total = (System.nanoTime() - t0) / 1_000_000;
        pool.shutdown();
        System.out.printf("%-34s total=%5d ms  mean latency=%4d ms  worst=%4d ms%n",
            label, total, sum / REQUESTS, worst);
    }

    public static void main(String[] args) throws Exception {
```

PoolStarvation.java (continued) JAVA

```
        System.out.println(REQUESTS + " requests, each blocks " + BLOCKING_MS + " ms");
        run("platform pool, 10 threads", Executors.newFixedThreadPool(10));
        run("platform pool, 50 threads", Executors.newFixedThreadPool(50));
        run("virtual thread per request", Executors.newVirtualThreadPerTaskExecutor());
    }
}
```

Measured on the build machine (2 CPUs, JDK 21); timings vary slightly between runs TERMINAL

```
$ java PoolStarvation.java
100 requests, each blocks 200 ms
platform pool, 10 threads          total= 2015 ms  mean latency=1097 ms  worst=2002 ms
platform pool, 50 threads          total=  406 ms  mean latency= 298 ms  worst= 400 ms
virtual thread per request         total=  213 ms  mean latency= 203 ms  worst= 209 ms
```

The first row shows the mechanism of starvation: 100 requests of 200 ms through 10 workers need ten waves of 200 ms, so the last request waits about 2 seconds although each request needs only 200 ms of actual work; the mean latency of 1.1 seconds is queueing delay. The relation is Little's law: sustained concurrency equals arrival rate multiplied by latency. A service receiving 100 requests per second whose downstream call takes 5 seconds needs 500 concurrent workers, and a pool of 200 cannot supply them. The virtual-thread row removes the pool as the limit; the remaining limit would be the downstream capacity.

The API of the worked example can be exercised with `curl`. The responses below are typical (not run on the build machine); they show the failure classes of the next section in their wire form.

typical output (not run on the build machine) TERMINAL

```
$ curl -s 'localhost:8080/api/v1/books?page=0&size=2&sort=year,desc'
{"content":[{"id":2,"title":"Refactoring","author":"Fowler","year":1999,"version":1}, ...],
 "pageable":{"pageNumber":0,"pageSize":2, ...},"totalElements":5,"totalPages":3, ...}

$ curl -si localhost:8080/api/v1/books/2
HTTP/1.1 200
ETag: "1"
Cache-Control: max-age=30, private
Content-Type: application/json

$ curl -si localhost:8080/api/v1/books/2 -H 'If-None-Match: "1"'
HTTP/1.1 304
ETag: "1"

$ curl -si localhost:8080/api/v1/books/99
HTTP/1.1 404
Content-Type: application/problem+json

{"type":"https://library.example.com/problems/book-not-found","title":"Book not found",
 "status":404,"detail":"No book with id 99","instance":"/api/v1/books/99","bookId":99}
```

1.4.4 Real-world problems and solutions

Problem 1: 415, 406 and 400 responses from the message-conversion stage

Scenario. A new mobile client calls `POST /api/v1/books` and receives 415. A reporting tool calling `GET /api/v1/books` with `Accept: application/xml` receives 406. A third client receives 400 for requests that look correct.

Symptoms. The controller log shows nothing, because the controller method never runs. The response body is a `ProblemDetail` such as `"detail":"Content-Type 'application/x-www-form-urlencoded;charset=UTF-8' is not supported."`

Diagnosis. (1) Enable `logging.level.org.springframework.web=DEBUG`; `DefaultHandlerExceptionResolver` and `ExceptionHandlerExceptionResolver` log the resolved exception class, for example `Resolved [org.springframework.web.HttpMediaTypeNotSupportedException: Content-Type ... is not supported]`. The class name identifies the stage: `HttpMediaTypeNotSupportedException` (415, request `Content-Type`), `HttpMediaTypeNotAcceptableException` (406, request `Accept`), `HttpMessageNotReadableException` (400, body cannot be parsed), `MethodArgumentNotValidException` (400, validation). (2) Reproduce with `curl -v` and compare the `Content-Type` and `Accept` headers with the `consumes` and `produces` of the mapping. (3) For a 400, read the `detail`: Jackson messages name the field and type (`Cannot deserialize value of type int from String "abc"`).

Root cause. The first client sent form data; the endpoint declares `consumes = application/json`. The reporting tool requests a representation for which no converter is registered (Jackson XML is not on the classpath). The third client sent a number as a quoted string.

Fix. Correct the client (`Content-Type: application/json`, a numeric JSON value). If XML is a real requirement, add `jackson-dataformat-xml`, which registers `MappingJackson2XmlHttpMessageConverter`; otherwise return the 406. Keep the standard `ProblemDetail` responses so that clients receive a machine-readable explanation.

Prevention. Cover each endpoint with `MockMvc` tests that send a wrong content type and an unknown `Accept`, and publish the contract with OpenAPI.

Problem 2: all endpoints become slow because workers are blocked

Scenario. `GET /api/v1/prices` calls a pricing service without a timeout. The pricing service degrades and each call takes 30 seconds. Within a minute every endpoint of the application, including `/actuator/health/liveness`, times out, and Kubernetes restarts the pods, which makes the situation worse.

Symptoms. Latency of unrelated, previously fast endpoints rises; the busy-thread gauge sits at the maximum; CPU usage is low; new connections are accepted and then wait, or are refused when the accept queue is full.

Diagnosis. (1) Metrics: with `server.tomcat.mbeanregistry.enabled=true` Micrometer publishes `tomcat.threads.busy`, `tomcat.threads.current` and `tomcat.threads.config.max`; `busy` equal to `config.max` confirms saturation (`GET /actuator/metrics/tomcat.threads.busy`). (2) Thread dump: `jcmd <pid> Thread.print | grep -c "http-nio-8080-exec"` counts workers, and grouping the top frames with `grep -A6 "http-nio-8080-exec" | sort | uniq -c | sort -rn | head` shows that nearly all are in `SocketInputStream.socketRead0` or `Http11InputBuffer` below the pricing client call. (3) `GET /actuator/metrics/http.server.requests?tag=uri:/api/v1/prices` shows maximum duration near 30 s.

Root cause. Unbounded waiting on a slow dependency combined with a bounded pool: arrival rate multiplied by latency exceeded `threads.max`. The health endpoint shares the worker pool, so the probe failed and the orchestrator killed healthy instances.

Fix. In order of effect: (1) set connect and read timeouts on the outgoing client (next chapter); (2) isolate with a bulkhead and a circuit breaker, so that a slow dependency can occupy only a limited number of threads (see the chapter on resilience); (3) serve Actuator on a separate port (`management.server.port=9090`), so that probes do not compete with application traffic for workers; (4) consider `spring.threads.virtual.enabled=true` and explicit concurrency limits for each downstream.

Prevention. Alert when `tomcat.threads.busy / tomcat.threads.config.max` stays above 0.8; load-test with a slow stub; review every outgoing call for a timeout.

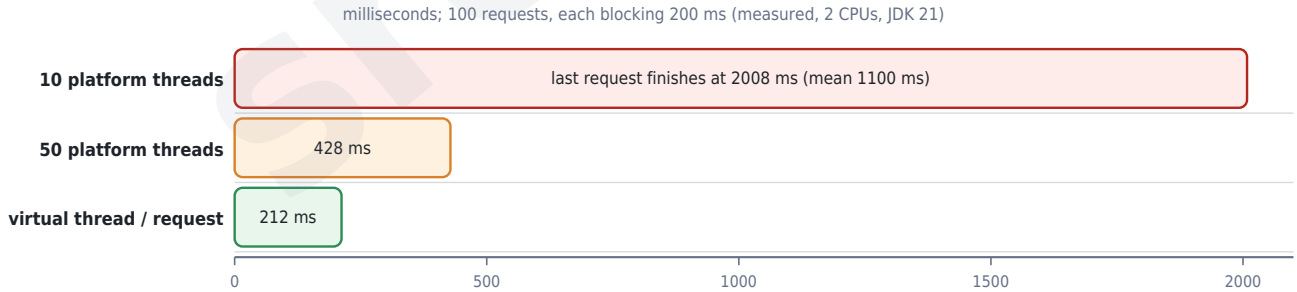


Figure 1.21. Measured values from the hands-on program: with 10 workers the last of 100 blocking requests completes after about 2 s; 50 workers about 0.43 s; one virtual thread per request about 0.21 s.

Problem 3: the browser reports a CORS error although the API works with curl

Scenario. A single-page application on `https://app.library.example.com` sends `GET /api/v1/books` with an `Authorization` header to `https://api.library.example.com`. The browser console reports: "Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource."

Symptoms. The request fails in the browser only. In the browser's network panel an `OPTIONS` request appears with status 401, 403 or 200 without CORS headers, and the actual `GET` is never sent.

Diagnosis. Reproduce the preflight with curl (typical command):

typical commands (not run on the build machine) TERMINAL

```
$ curl -si -X OPTIONS https://api.library.example.com/api/v1/books \
    -H 'Origin: https://app.library.example.com' \
    -H 'Access-Control-Request-Method: GET' \
    -H 'Access-Control-Request-Headers: authorization'
HTTP/1.1 403
Invalid CORS request
```

A body of `Invalid CORS request` means that Spring's CORS processing ran and rejected the origin, method or header (for example `Authorization` missing from `allowedHeaders`). A 401 or 403 **without** that body means the preflight never reached MVC: Spring Security rejected the unauthenticated `OPTIONS` request first. Preflight requests never carry credentials.

Root cause. Either the CORS configuration lists `allowedHeaders` that omit `Authorization`, or the security filter chain does not apply the CORS configuration, so the preflight is treated as an unauthenticated request.

Fix. Add the missing header to the policy and enable CORS in the security chain so that `CorsFilter` runs before authentication:

SecurityCorsConfig.java JAVA

```java
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http.cors(Customizer.withDefaults())      // uses the CorsConfigurationSource or MVC config
        .authorizeHttpRequests(a -> a.anyRequest().authenticated());
    return http.build();
}
```

With credentials (cookies), `allowedOrigins("*")` is rejected; list exact origins or use `allowedOriginPatterns`. The detailed treatment of CORS and CSRF in the security chain is in the Part on Spring Security.

Prevention. Add an integration test that sends `OPTIONS` with the `Origin`, `Access-Control-Request-Method` and `Access-Control-Request-Headers` headers and asserts the `Access-Control-Allow-*` response headers. Remember that CORS is enforced by browsers only; server-to-server calls are unaffected.

Problem 4: LazyInitializationException (or hidden N+1 queries) while serialising an entity

Scenario. `GET /api/v1/authors` returns `List<Author>` where `Author` is a JPA entity with `@OneToMany(fetch = FetchType.LAZY) List<Book> books`.

Symptoms. With `spring.jpa.open-in-view=false` the response is a 500 and the log shows `HttpMessageNotWritableException: Could not write JSON: failed to lazily initialize a collection of role 'Author.books': could not initialize proxy - no Session`. With open-in-view enabled (the Boot default, which logs a warning at start-up) there is no exception, but the SQL log shows one query for the authors followed by one query per author, and a database connection is held during JSON writing. With a bidirectional relationship the failure can instead be `JsonMappingException: Infinite recursion (StackOverflowError)`.

Diagnosis. The stack trace passes through `BeanSerializer`, `CollectionSerializer` and `HttpMessageConverter` classes, which shows that the failing code is serialisation, after the service method returned. Enable `logging.level.org.hibernate.SQL=DEBUG` and count the statements per request. Check `GET /actuator/configprops` (or the start-up warning) for `spring.jpa.open-in-view`.

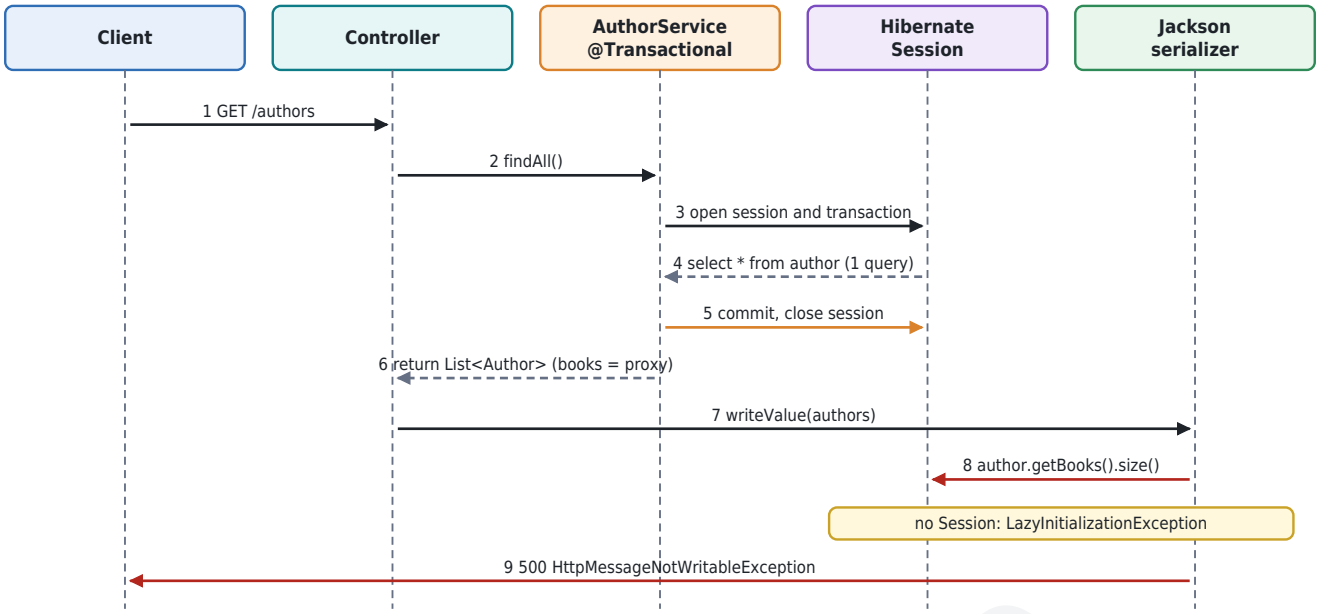


Figure 1.22. The transaction and session end when the service returns; Jackson later calls getBooks() on a proxy with no session, which fails, or with open-in-view keeps the session open and issues one query per author.

Root cause. Serialisation is a data-access operation when the object graph contains uninitialised proxies, and it occurs after the transactional boundary, in the message-conversion stage of the pipeline.

Fix. Never return entities from controllers. Map to a DTO inside the transactional service method, with the required associations loaded by a single query:

AuthorService.java JAVA

```java
public record AuthorDto(long id, String name, List<String> bookTitles) { }

@Service
public class AuthorService {

    private final AuthorRepository authors;

    public AuthorService(AuthorRepository authors) {
        this.authors = authors;
    }

    @Transactional(readOnly = true)
    public List<AuthorDto> findAll() {
        return authors.findAllWithBooks().stream()                    // one JOIN FETCH query
                .map(a -> new AuthorDto(a.getId(), a.getName(),
                        a.getBooks().stream().map(Book::getTitle).toList()))
                .toList();
    }
}
```

The repository method is `@Query("select distinct a from Author a left join fetch a.books")`. Set `spring.jpa.open-in-view=false` so that lazy access outside the service fails during development instead of silently issuing queries in production. The fetching strategies (JOIN FETCH, entity graphs, batch fetching) are treated in the Part on data access.

Prevention. Add an architecture test asserting that classes in the web package do not reference `@Entity` classes, and enable Hibernate statistics in tests to assert the number of statements.

1.4.5 Questions to expect

1. **Describe the lifecycle of an HTTP request in Spring MVC.** Tomcat acceptor and poller, a worker thread, servlet filters, `DispatcherServlet.doDispatch`: handler mapping, interceptor `preHandle`, argument resolution (including `@RequestBody` conversion and validation), controller, return-value handling with a message converter, `postHandle`, exception resolution if needed, `afterCompletion`.
2. **Filter versus interceptor?** A filter is a servlet component that wraps the entire request, can replace the request and response, and runs before Spring MVC; an interceptor runs inside the dispatcher with knowledge of the handler.

3. **What is `HttpMessageConverter` and how is one selected?** It reads and writes request and response bodies. Selection uses the request `Content-Type` for reading, and the `Accept` header with the `produces` condition for writing; failure gives 415 and 406 respectively.
4. **How are exceptions turned into responses?** `HandlerExceptionResolver` composite: `@ExceptionHandler` methods (including `@ControllerAdvice`), `ResponseStatusExceptionResolver`, `DefaultHandlerExceptionResolver`; unresolved exceptions go to `/error`.
5. **What is `ProblemDetail`?** The Spring representation of RFC 9457 (`application/problem+json`) with `type`, `title`, `status`, `detail`, `instance` and extensions; produced by extending `ResponseEntityExceptionHandler` or enabling `spring.mvc.problemdetails.enabled`.
6. **How many concurrent requests can a Boot application serve by default?** The Tomcat worker pool has up to 200 threads, so up to 200 requests run simultaneously; further connections wait in the executor queue and the accept queue (100). Virtual threads remove the 200 limit for blocked requests.
7. **What changes with `spring.threads.virtual.enabled=true`?** Request handling and the application task executor use virtual threads (Boot 3.2+, Java 21+); pinning on `synchronized` exists on JDK 21; downstream pools become the limit.
8. **How does CORS work and why can a preflight fail?** The browser sends `OPTIONS` with `Origin` and requested method and headers; the server must answer with `Access-Control-Allow-*`. Failures: header not allowed, or Spring Security rejecting the unauthenticated preflight.
9. **How does ETag support reduce load?** A matching `If-None-Match` yields 304 with no body, saving bandwidth; computation is saved only if the ETag is derived from a version without building the body.
10. **Why not expose JPA entities from controllers?** Lazy proxies fail or trigger queries during serialisation, internal fields leak, bidirectional relations recurse, and the API becomes coupled to the schema.

Check yourself

1. In `MiniDispatcher`, for the request `/boom`, which callbacks run on interceptor A and in what order relative to B?
2. A controller declares `@PostMapping(consumes = "application/json")` and a client posts `text/plain`. Which status results, and does the controller method execute?
3. Tomcat has 200 workers. 400 requests arrive in one second and each blocks for 1 second. Roughly how long does the last request wait?
4. Why can `OPTIONS` fail with 401 even though `@CrossOrigin` is configured?
5. What is wrong? `@GetMapping("/authors") List<Author> all() { return repo.findAll(); }` with `open-in-view=false` and a lazy collection.

Answers

1. `preHandle` for A then B ran; for `/boom` no `postHandle` runs; `afterCompletion` runs for B first, then A, each receiving the exception.
2. 415 (`HttpMediaTypeNotSupportedException`); the controller method does not execute because the failure occurs while matching and reading.
3. About one second of waiting for the second batch of 200, so the last request completes after about 2 seconds in total.
4. The preflight carries no credentials and is stopped by Spring Security before MVC's CORS processing unless `http.cors(...)` is enabled.
5. Serialisation touches an uninitialised proxy after the session closed: `LazyInitializationException` wrapped in `HttpMessageNotWritableException`; map to DTOs in the service.

sreeram

KEY POINTS

- One request occupies one Tomcat worker (up to 200) for its whole duration; unbounded waits on dependencies exhaust the pool (Little's law).
- `doDispatch` order: handler mapping, `preHandle`, argument resolvers, controller, return handler and converter, `postHandle`, exception resolvers, `afterCompletion`.
- Filters wrap everything and can replace the request; interceptors know the handler; `ResponseEntityExceptionHandler` plus `ProblemDetail` gives a uniform RFC 9457 error format.
- 415 means wrong `Content-Type`, 406 wrong `Accept`, 400 unreadable or invalid body; each is raised before the controller runs.
- Virtual threads (Boot 3.2, Java 21) lift the worker limit but not the database or downstream limits.
- Return DTOs, never entities; set `spring.jpa.open-in-view=false`.

1.5 HTTP Clients: RestClient, RestTemplate, WebClient and HTTP Interfaces

The previous chapter treated a service as the receiver of requests. Almost every service is also a caller: it asks a pricing service for a price, an identity provider for a key, a payment gateway for an authorisation. An outgoing HTTP call has three properties that the code does not show: it can block for an arbitrary time, it consumes a connection from a pool, and it fails in ways that differ from a local method call. Most cascading outages in microservice systems begin with an outgoing call that had no timeout. This chapter describes the client abstractions that Spring offers, the layers beneath them, and the settings that must be chosen deliberately.

1.5.1 Concept and mechanism

Four client styles

Client	Model	Notes
`RestClient`	Synchronous, fluent API (Spring Framework 6.1; auto-configured builder since Boot 3.2)	Recommended for new blocking code. Shares message converters, interceptors and request factories with RestTemplate.
`RestTemplate`	Synchronous, template methods (getForObject, exchange, ...)	In maintenance mode: it receives fixes and no new features. Boot does not create a RestTemplate bean; it provides RestTemplateBuilder.
`WebClient`	Non-blocking, reactive (Project Reactor); Reactor Netty by default	Required for reactive pipelines and streaming; usable from servlet applications with block(), which gives up the benefit.
`HTTP interface (@HttpExchange)`	A Java interface whose methods describe HTTP calls; a proxy implements it over RestClient, WebClient or RestTemplate	Declarative style; the proxy created by HttpServiceProxyFactory (Framework 6.0).

HTTP client options in Spring Boot 3.x.

The practical rule: a servlet application that uses blocking code and calls other services synchronously uses `RestClient` (directly or behind an HTTP interface). A WebFlux application, or a pipeline that fans out many calls and combines results asynchronously, uses `WebClient`. An existing `RestTemplate` does not need an urgent rewrite.

Layers beneath the API

The client API converts objects to bytes and back; it does not perform network I/O. The work is delegated to a `ClientHttpRequestFactory`, an adapter over a real HTTP library. The adapters supplied by Spring Framework are `JdkClientHttpRequestFactory` (`java.net.http.HttpClient`), `HttpComponentsClientHttpRequestFactory` (Apache HttpClient 5), `JettyClientHttpRequestFactory`, `ReactorNettyClientRequestFactory` and `SimpleClientHttpRequestFactory` (`HttpURLConnection`). Spring Boot detects the first available library in this order: Apache HttpClient, Jetty client, Reactor Netty, the JDK `HttpClient`, then the simple factory (`spring.http.client.factory` selects one explicitly in Boot 3.4 and later).

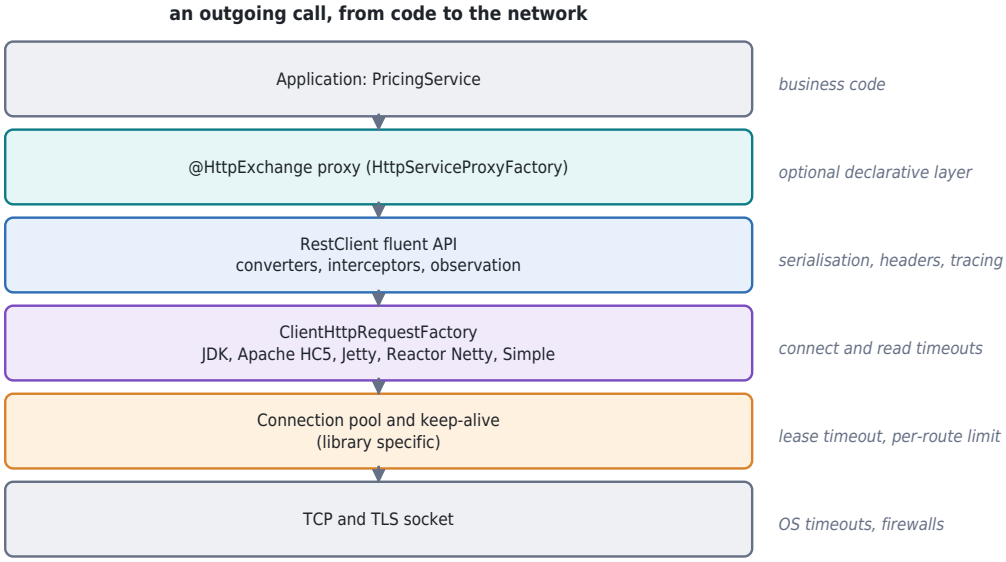


Figure 1.23. The fluent API is separated from the transport: converters and interceptors belong to the API layer, while timeouts and pooling belong to the library that the request factory adapts.

The `RestClient.Builder` bean that Boot provides is a **prototype**: each injection yields a new builder pre-configured with the `HttpMessageConverters` and the detected request factory, and with Micrometer observation, so that outgoing calls appear in the `http.client.requests` metric. `RestClient.create()` skips all of this auto-configuration, which is a common cause of calls that are missing from metrics and traces.

Timeouts and pools

A blocking call has up to four separate time limits, and each guards a different failure.

Limit	Guards against	Where it is configured
Connection request (lease) timeout	All connections of the pool are in use; the caller waits for one	Pool settings of the library (Apache: connection request timeout); the JDK client has no bounded pool
Connect timeout	A host that does not answer the TCP handshake (firewall drop, dead host)	Request factory settings; `spring.http.client.connect-timeout` (Boot 3.4+)
Read (response) timeout	A host that accepted the connection and then does not send data	Request factory settings; `spring.http.client.read-timeout` (Boot 3.4+)
Overall deadline	The sum of everything, including retries	Not provided by the blocking clients; implement with a resilience time limiter, or with `Mono.timeout` in WebClient

The time limits of an outgoing call.

The read timeout is **an idle timeout between bytes** in several libraries, not a bound on the total call duration: a server that sends one byte every second can keep a call alive beyond the read timeout. An overall deadline therefore needs a mechanism outside the HTTP library.

Neither the JDK client nor the Spring adapters impose a read timeout by default. The program below shows the consequence with a real `HttpClient` and a local server that delays every response by two seconds.

ClientTimeouts.java JAVA

```java
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpTimeoutException;
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
```

ClientTimeouts.java (continued) JAVA

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;

public class ClientTimeouts {

    static final int SERVER_DELAY_MS = 2000;

    static long since(long t0) { return (System.nanoTime() - t0) / 1_000_000; }

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.createContext("/slow", ex -> {
            try {
                Thread.sleep(SERVER_DELAY_MS);
            } catch (InterruptedException ignored) {
                // the server is being stopped
            }
            byte[] body = "done".getBytes();
            ex.sendResponseHeaders(200, body.length);
            ex.getResponseBody().write(body);
            ex.close();
        });
        server.start();
        URI uri = URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/slow");
        HttpClient client = HttpClient.newBuilder().connectTimeout(Duration.ofMillis(500)).build();

        // 1. Without a request timeout the caller waits for the full server delay.
        long t0 = System.nanoTime();
        HttpResponse<String> r = client.send(HttpRequest.newBuilder(uri).build(),
            HttpResponse.BodyHandlers.ofString());
        System.out.printf("no request timeout : status %d after %d ms%n",
            r.statusCode(), since(t0));

        // 2. With a request timeout the call fails fast and the thread is released.
        t0 = System.nanoTime();
        try {
            client.send(HttpRequest.newBuilder(uri).timeout(Duration.ofMillis(500)).build(),
                HttpResponse.BodyHandlers.ofString());
        } catch (HttpTimeoutException e) {
            System.out.printf("request timeout   : %s after %d ms%n",
                e.getClass().getSimpleName(), since(t0));
        }

        // 3. Model of a bounded connection pool: 2 connections, 6 callers, lease timeout 1 s.
        Semaphore pool = new Semaphore(2);
        ExecutorService callers = Executors.newFixedThreadPool(6);
        long start = System.nanoTime();
        List<Future<String>> results = new ArrayList<>();
        for (int i = 1; i <= 6; i++) {
            final int id = i;
            results.add(callers.submit(() -> {
                if (!pool.tryAcquire(1, TimeUnit.SECONDS)) {
                    return "caller " + id + ": lease timeout after " + since(start) + " ms";
                }
                try {
                    client.send(HttpRequest.newBuilder(uri).build(),
                        HttpResponse.BodyHandlers.ofString());
                    return "caller " + id + ": ok after " + since(start) + " ms";
                } finally {
                    pool.release();
                }
            }));
        }
        List<String> lines = new ArrayList<>();
        for (Future<String> f : results) lines.add(f.get());
        lines.stream().sorted().forEach(System.out::println);
        callers.shutdown();
        server.stop(0);
    }
}
```

Real run on the build machine (JDK 21); which callers win the pool varies between runs TERMINAL

```
$ java ClientTimeouts.java
no request timeout : status 200 after 2247 ms
request timeout    : HttpTimeoutException after 502 ms
caller 1: lease timeout after 1048 ms
caller 2: ok after 2058 ms
caller 3: ok after 2053 ms
caller 4: lease timeout after 1038 ms
caller 5: lease timeout after 1040 ms
caller 6: lease timeout after 1041 ms
```

The first line is the default behaviour: the call waits the full server delay. With a 500 ms request timeout the second line shows a fast failure (`HttpTimeoutException` after about 510 ms) that frees the thread. The third part is a **model**: a semaphore stands in for a pool of two connections and six callers share it with a one-second lease timeout. Two callers proceed; four wait one second and fail. A real pool (Apache `PoolingHttpClientConnectionManager`, default maximum 25 connections and 5 per route in HttpClient 5) behaves the same way, and the failure appears as a `ConnectionRequestTimeoutException`.

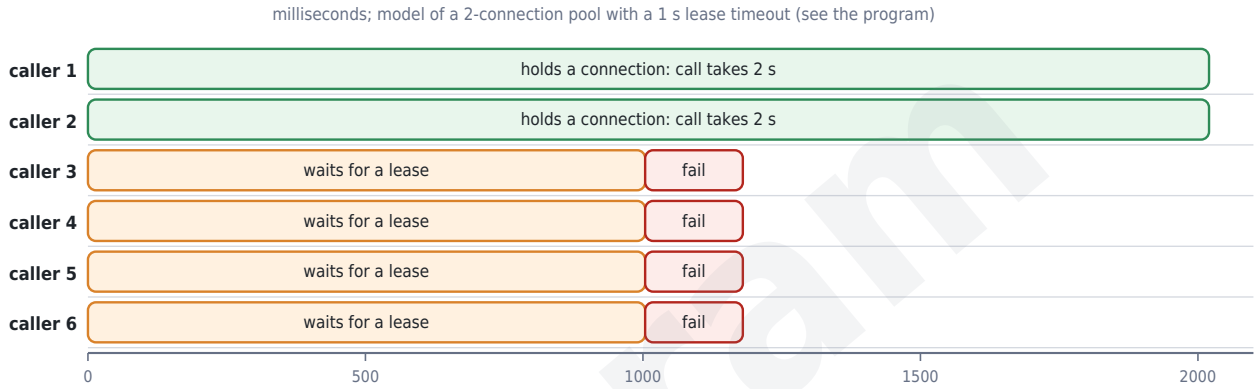


Figure 1.24. Timeline of the pool model: two callers hold the two connections for 2 s while four callers wait 1 s for a lease and then fail; without a lease timeout they would wait until a connection is free.

Error handling

`RestClient` throws `HttpClientErrorException` for 4xx and `HttpServerErrorException` for 5xx statuses (both subclasses of `RestClientResponseException`), and `ResourceAccessException` for I/O failures including timeouts. This default can be replaced with `onStatus(...)`. The distinction is operationally important: a 5xx or a timeout may justify a retry; a 4xx never does. Retries must be limited to idempotent operations or to requests that carry an idempotency key, and are covered in the chapter on resilience.

1.5.2 Worked example

The library service obtains prices from a pricing service. The configuration sets timeouts in properties, builds a `RestClient` from the auto-configured builder, and exposes the pricing API as an HTTP interface.

application.yml YAML

```yaml
spring:
  http:
    client:                                   # Spring Boot 3.4+; in Boot 4 the prefix is spring.http.clients
      factory: jdk
      connect-timeout: 1s
      read-timeout: 2s
pricing:
  base-url: https://pricing.internal.example.com
```

PricingClient.java JAVA

```java
package com.example.library.pricing;

import java.math.BigDecimal;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;

@HttpExchange(url = "/api/v1/prices", accept = "application/json")
```

PricingClient.java (continued) JAVA

```java
public interface PricingClient {

    @GetExchange("/{isbn}")
    Price priceOf(@PathVariable String isbn, @RequestParam(defaultValue = "EUR") String currency);

    record Price(String isbn, BigDecimal amount, String currency) { }
}
```

PricingClientConfig.java JAVA

```java
package com.example.library.pricing;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.support.RestClientAdapter;
import org.springframework.web.service.invoker.HttpServiceProxyFactory;

@Configuration(proxyBeanMethods = false)
public class PricingClientConfig {

    @Bean
    RestClient pricingRestClient(RestClient.Builder builder,
                                 @Value("${pricing.base-url}") String baseUrl) {
        return builder.baseUrl(baseUrl)                // timeouts: spring.http.client.*
                .defaultHeader("X-Caller", "library-service")
                .requestInterceptor((request, body, execution) -> {
                    request.getHeaders().set("X-Request-Id",
                            String.valueOf(org.slf4j.MDC.get("requestId")));
                    return execution.execute(request, body);
                })
                .build();
    }

    @Bean
    PricingClient pricingClient(RestClient pricingRestClient) {
        HttpServiceProxyFactory factory = HttpServiceProxyFactory
                .builderFor(RestClientAdapter.create(pricingRestClient)).build();
        return factory.createClient(PricingClient.class);
    }
}
```

PricingService.java JAVA

```java
package com.example.library.pricing;

import com.example.library.pricing.PricingClient.Price;
import java.math.BigDecimal;
import java.util.Optional;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.web.client.ResourceAccessException;

@Service
public class PricingService {

    private static final Logger log = LoggerFactory.getLogger(PricingService.class);

    private final PricingClient pricing;

    public PricingService(PricingClient pricing) {
        this.pricing = pricing;
    }

    public Optional<BigDecimal> priceOf(String isbn) {
        try {
            return Optional.of(pricing.priceOf(isbn, "EUR").amount());
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();                          // 404: no such price, not an outage
        } catch (HttpServerErrorException | ResourceAccessException e) {
```

Sreeram

PricingService.java (continued) JAVA

```
            log.warn("pricing unavailable for {}: {}", isbn, e.toString());
            throw new PricingUnavailableException(isbn, e);  // mapped to 503 by a handler
        }
    }
}
```

The same call written directly with the fluent API, with an explicit status handler, shows what the proxy does:

DirectCall.java JAVA

```
Price price = pricingRestClient.get()
        .uri("/api/v1/prices/{isbn}?currency={c}", isbn, "EUR")
        .retrieve()
        .onStatus(status -> status.value() == 404, (req, res) -> {
            throw new PriceNotFoundException(isbn);
        })
        .body(Price.class);
```

For per-client settings that differ from the global ones, build the factory explicitly (Boot 3.4 and later):

CustomTimeouts.java JAVA

```
ClientHttpRequestFactorySettings settings = ClientHttpRequestFactorySettings.defaults()
        .withConnectTimeout(Duration.ofMillis(500))
        .withReadTimeout(Duration.ofSeconds(1));
ClientHttpRequestFactory factory = ClientHttpRequestFactoryBuilder.detect().build(settings);
RestClient fast = builder.clone().requestFactory(factory).baseUrl(baseUrl).build();
```

For `WebClient` the equivalent protections are the connector settings (`spring.http.reactiveclient.connect-timeout` and `read-timeout` in Boot 3.4 and later) and a `timeout` on the reactive chain, which acts as an overall deadline: `webClient.get().uri(...).retrieve().bodyToMono(Price.class).timeout(Duration.ofSeconds(2))`.

What to notice: the interface contains no timeout or error handling, because both belong to the client configuration and to the caller; a 404 is translated into an empty `Optional` while 5xx and I/O errors are translated into a domain exception that is mapped to 503 at the web boundary; `builder.clone()` is required before reusing the prototype builder with different settings.

1.5.3 Hands-on

The timeout behaviour of the configured client can be checked without the real pricing service by starting a stub that delays its answer. WireMock (covered in the chapter on integration testing) provides `withFixedDelay`; the local server of the program above serves the same purpose. A test asserts that the failure appears within the configured limit:

PricingTimeoutTest.java JAVA

```
@Test
void slowPricingServiceFailsFast() {
    stubFor(get(urlPathEqualTo("/api/v1/prices/978-0134685991"))
            .willReturn(ok().withFixedDelay(5_000)));          // slower than read-timeout 2s

    long start = System.nanoTime();
    assertThatThrownBy(() -> pricingService.priceOf("978-0134685991"))
            .isInstanceOf(PricingUnavailableException.class);
    long ms = (System.nanoTime() - start) / 1_000_000;

    assertThat(ms).isBetween(1_900L, 3_000L);                  // ~2 s, not 5 s
}
```

To see outgoing latency in production, query the metric published for observed clients: `GET /actuator/metrics/http.client.requests?tag=uri:/api/v1/prices/{isbn}` returns count, total time and maximum for the pricing calls, and `tag=outcome:SERVER_ERROR` isolates failures (typical, not run on the build machine).

1.5.4 Real-world problems and solutions

Problem 1: a missing client timeout turns one slow dependency into a full outage

Scenario. The checkout endpoint calls the pricing service through a `RestTemplate` created with `new RestTemplate()`. The pricing service starts to respond after 40 seconds during a database failover.

Symptoms. Within a minute the checkout service stops responding to every request. Its CPU is idle. Dependent services report timeouts against it, so the failure spreads upstream.

Diagnosis. (1) A thread dump (`jcmd <pid> Thread.print`) shows nearly all `http-nio-*-exec` threads in `SocketInputStream.socketRead0` below `RestTemplate.execute`. (2) `http.client.requests` shows maximum duration at or near 40 s. (3) Inspect the request factory: `new RestTemplate()` uses `SimpleClientHttpRequestFactory` or the JDK client with no read timeout (the first line of the real run above).

Root cause. An unbounded wait in an outgoing call, combined with a bounded worker pool (see the pool starvation measurements in the previous chapter). Every request occupies a worker for as long as the dependency stalls.

Fix. Create all clients from the Boot builders (`RestClient.Builder`, `RestTemplateBuilder`, `WebClient.Builder`) and set connect and read timeouts through `spring.http.client.*` or explicit factory settings. Choose values from the dependency's service-level objective (for example read timeout slightly above the 99th percentile latency). Add a circuit breaker and bulkhead around the call (see the chapter on resilience) so that a persistent failure stops consuming threads.

Prevention. Forbid `new RestTemplate()`, `RestClient.create()` and `WebClient.create()` in code review or with an ArchUnit rule; test with a slow stub as in the example; alert on `http.client.requests` maximum.

Problem 2: connection pool exhaustion under load

Scenario. After the timeout fix the service runs with the Apache HttpClient 5 factory. Under load test the log shows `ConnectionRequestTimeoutException` for calls that the pricing service would have answered quickly.

Symptoms. Errors appear at a throughput well below the capacity of the pricing service, with low latency in the pricing service's own metrics. The error rate is stepwise: it grows when concurrency exceeds a threshold.

Diagnosis. The pool defaults of Apache HttpClient 5 are 25 connections in total and 5 per route (host). With 40 concurrent requests to one host, 35 wait for a lease. Confirm by comparing concurrency (`tomcat.threads.busy` for blocking callers) with the pool limit and by reading the pool statistics (`PoolingHttpClientConnectionManager.getTotalStats()`) or the lease-timeout exceptions.

Root cause. The per-route limit is lower than the concurrency directed at a single host, and the lease timeout is long or unset, so callers queue silently.

Fix. Size the per-route limit to the expected concurrent calls to that host (at most the downstream's capacity), set a short connection-request timeout (for example 500 ms) so that saturation fails fast, and define the pool in a custom `ClientHttpRequestFactoryBuilder` or `HttpComponentsClientHttpRequestFactory`. Raising the limit without bound only moves the queue to the dependency.

Prevention. Expose pool metrics, load-test at 2 times the expected peak, and treat the pool limit as an explicit bulkhead.

Problem 3: calls missing from metrics and traces

Scenario. The tracing system shows no outgoing spans for the pricing service, and `http.client.requests` is empty although traffic flows.

Symptoms. Traces end at the checkout service; the downstream spans appear as separate traces without a parent.

Diagnosis. Search for clients created with `RestClient.create()`, `new RestTemplate()` or `WebClient.create()`. These bypass the auto-configured builders and therefore the observation and trace-propagation customisers.

Root cause. The Micrometer observation and context propagation are applied through builder customisers; a client built without the builder has none.

Fix. Inject `RestClient.Builder`, `RestTemplateBuilder` or `WebClient.Builder` and build from them.

Prevention. Provide one internal factory for outgoing clients and use it throughout the code base.

1.5.5 Questions to expect

1. **RestTemplate, RestClient or WebClient?** `RestClient` for new synchronous code; `WebClient` for reactive or non-blocking composition; `RestTemplate` is in maintenance mode and acceptable in existing code.
2. **What is an HTTP interface client?** An interface annotated with `@HttpExchange` methods; `HttpServiceProxyFactory` creates a proxy that executes the calls through a `RestClient`, `WebClient` or `RestTemplate` adapter.
3. **Which timeouts must an outgoing call have?** Connect timeout, read timeout, a lease timeout on the pool, and an overall deadline where the client library does not provide one.
4. **Which timeout is applied by default?** None in the JDK or Spring adapters; the application must set them.
5. **How are 4xx and 5xx handled in `RestClient`?** By default as `HttpClientErrorException` and `HttpServerErrorException`; `onStatus` customises. 4xx are not retried; 5xx and I/O failures may be.
6. **Why use the injected builder instead of `RestClient.create()`?** The builder carries converters, the detected request factory, observation and propagation customisers, and the global timeouts.
7. **Why is a retry on POST dangerous?** The first request may have been executed despite a timeout; without an idempotency key the retry duplicates the effect.
8. **How does a blocking client relate to the Tomcat worker pool?** Each outgoing wait holds a worker thread; slow dependencies exhaust the pool unless timeouts and bulkheads bound the wait.

Check yourself

1. In the `ClientTimeouts` run, why does the first call take about 2 seconds although the connect timeout is 500 ms?
2. A `RestClient` is built with `RestClient.create()`. Which two production features are probably missing?
3. What is the difference between a connect timeout and a read timeout?
4. Four callers share a pool of two connections with a one-second lease timeout and each call takes 2 seconds. How many fail, and why?
5. Why must `builder.clone()` be used before changing the request factory on an injected builder?

Answers

1. The connect timeout bounds only the TCP handshake, which succeeded immediately. No read or request timeout was set, so the call waited for the server's two-second delay.
2. Global timeouts and converters configured through the builder, and the observation and trace propagation (metrics `http.client.requests`, trace headers).
3. Connect timeout: time to establish the TCP connection. Read timeout: maximum idle time waiting for response data after the request is sent.
4. Two fail: they wait one second and no connection is released before the lease timeout, since the holders need two seconds.
5. A builder is stateful: calls on it modify that instance. Cloning keeps the clients derived from one builder independent of each other.

KEY POINTS

- Use `RestClient` for blocking calls and `WebClient` for reactive ones; `@HttpExchange` interfaces give a declarative layer over either.
- Create clients from the Boot-provided builders so that converters, timeouts, observation and trace propagation apply.
- Every outgoing call needs a connect timeout, a read timeout, a pool lease timeout and, where possible, an overall deadline; the defaults are unbounded.
- Distinguish 4xx (caller error, no retry) from 5xx and I/O failures (candidate for retry with limits).
- A slow dependency without timeouts exhausts the worker pool of the caller; pool limits and bulkheads turn saturation into fast failures.

sreram

PART 2

Data Access in Depth

Persistence contexts, mappings, fetch plans, transactions, connection pools and query techniques, with the SQL traced and executed.

In this part

- **2.1** JPA and Hibernate Architecture
- **2.2** Mapping in Depth
- **2.3** Fetching and the N+1 Family
- **2.4** Transactions in Spring
- **2.5** Performance of the Data Layer
- **2.6** Query Techniques and Spring Data Internals

2.1 JPA and Hibernate Architecture

Relational databases store rows in tables that are related by foreign keys; Java programs manipulate graphs of objects that are related by references. The Java Persistence API (JPA, now Jakarta Persistence) standardises the translation between the two. Hibernate ORM is the implementation used by Spring Boot, Spring Data JPA is a repository abstraction placed above it, and Spring's transaction infrastructure binds the whole stack to a database connection. Most production data-access defects (extra queries, stale data, lost updates, pool exhaustion) are not mistakes in SQL; they are consequences of the behaviour of one of these layers that was not understood. This chapter describes the layers, the lifecycle of an entity, the mechanism of flushing and dirty checking, and the SQL that results.

All examples target Spring Boot 3.4/3.5 on JDK 21, which resolves to Hibernate ORM 6.6, Jakarta Persistence 3.1 and Spring Data JPA 3.4/3.5. Package names use `jakarta.persistence`. Hibernate-specific behaviour is identified as such, because it is not guaranteed by the JPA specification.

2.1.1 Concept and mechanism

The layers

A repository call crosses five layers before a SQL statement reaches PostgreSQL. Each layer has its own object, its own lifetime and its own failure modes.

sreeram

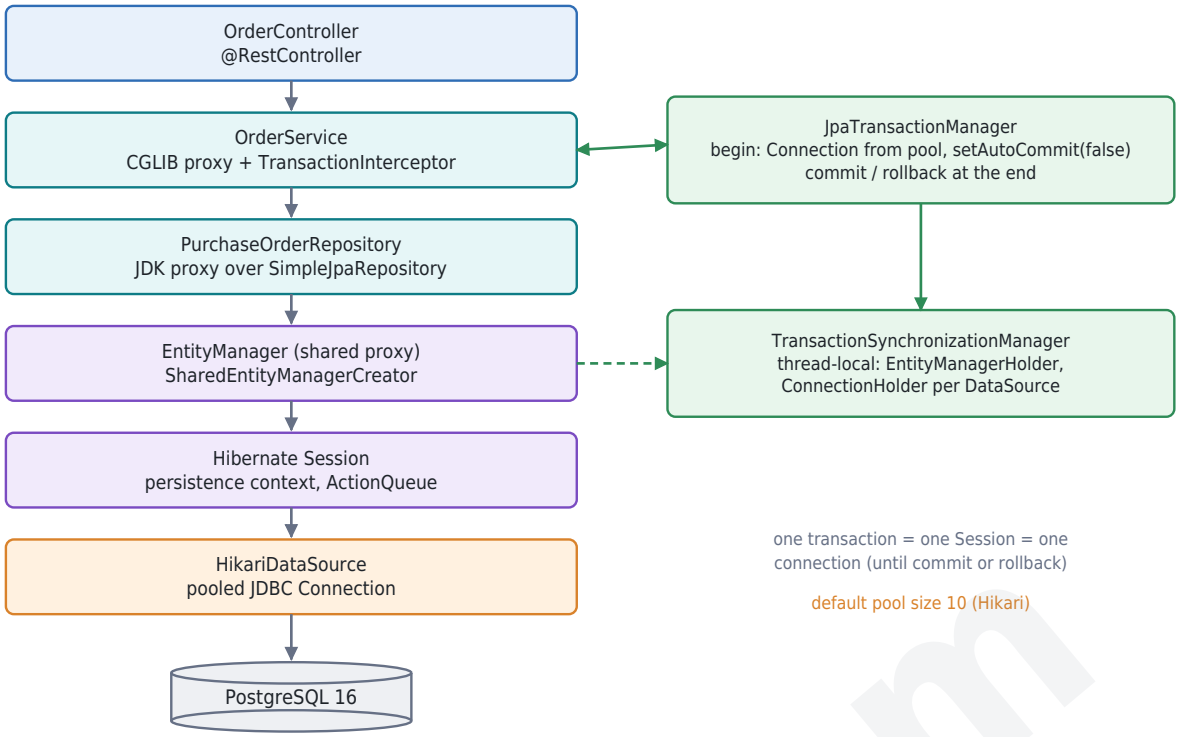


Figure 2.1. The data-access stack of a Spring Boot service: every repository method call passes through proxies that bind an EntityManager to the current transaction before Hibernate produces SQL for a pooled JDBC connection.

Type	Role	Lifetime
`EntityManagerFactory`	JPA factory; holds mappings, caches and the SQL generation metadata. Hibernate implementation: SessionFactory.	One per DataSource, application lifetime
`EntityManager`	JPA unit of work and persistence context. Hibernate implementation: Session.	One per transaction (or per request with open-in-view)
`Persistence context`	Identity map of managed entities plus change-tracking state	Same as the EntityManager
`JpaTransactionManager`	Spring PlatformTransactionManager that begins the JDBC transaction and binds an EntityManager	Application lifetime
`DataSource (HikariDataSource)`	Pool of physical JDBC connections	Application lifetime

Core types and their lifetimes.

Spring Boot assembles this stack in `HibernateJpaAutoConfiguration` (which extends the configuration in `JpaBaseConfiguration`). A `LocalContainerEntityManagerFactoryBean` creates the `EntityManagerFactory` from the `DataSource` and the entity classes found through `@EntityScan` (by default the package of the `@SpringBootApplication` class), and a `JpaTransactionManager` is registered as the `PlatformTransactionManager`. The `EntityManager` injected with `@PersistenceContext` or by constructor is not a real session: it is a shared proxy created by `SharedEntityManagerCreator`. On each method call the proxy asks `EntityManagerFactoryUtils` for the `EntityManager` that is bound to the current transaction (stored in `TransactionSynchronizationManager`, a set of thread-local maps). Inside a transaction the same `Session` is therefore returned for every call; outside a transaction a temporary `EntityManager` is created and closed for each call, so no persistence context spans two calls.

The persistence context

The persistence context is an identity map: for every database row that has been loaded in the current unit of work there is at most one entity instance, found by the key (entity class, primary key). Hibernate implements it in `StatefulPersistenceContext`. For each managed instance it keeps an `EntityEntry` containing the status (`MANAGED`, `READ_ONLY`, `DELETED`, `GONE`, `LOADING`, `SAVING`), the version, and the **loaded state**: a copy of the values of all persistent attributes at the time of loading. This structure explains three behaviours that are otherwise surprising.

- **First-level cache.** `find(Customer.class, 1L)` consults the identity map first. The second call within the same context returns the same Java instance without SQL. JPQL queries are not answered from the cache (the SQL is

always executed) but the rows are resolved to the instances that are already managed, so the instance created first wins and the state in the new result set is ignored for that instance.

- **Automatic updates.** An entity that is managed is written back at flush time when any attribute differs from the loaded state. No `save` call is required.
- **Memory growth.** Every loaded entity and its loaded-state copy stay reachable until the context is closed or cleared. A batch job that loads one million entities in one transaction holds two copies of each.

Entity states and transitions

JPA defines four states. The state is a property of the relation between an object and a persistence context, not of the object alone.

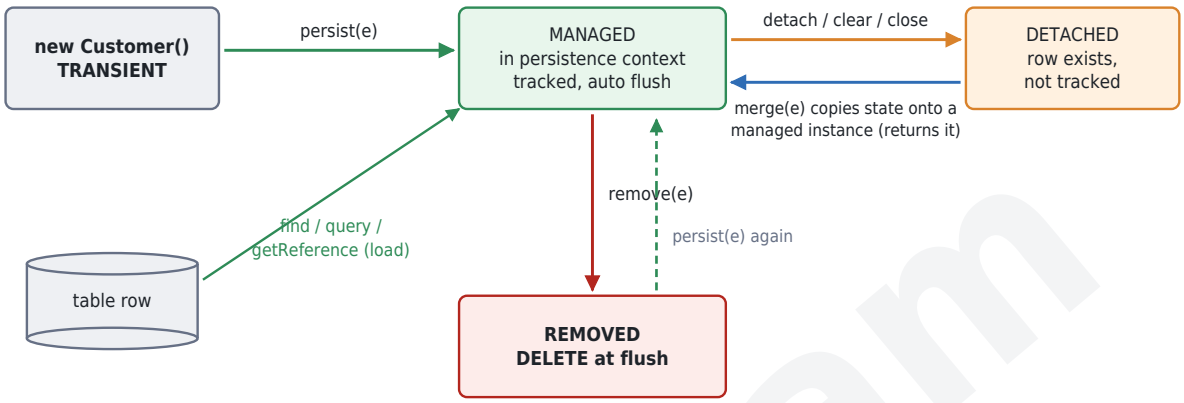


Figure 2.2. The four entity states and the operations that move an instance between them; only the managed state is tracked for changes.

Operation	Effect	SQL emitted
`persist(e)`	Transient to managed. The instance itself becomes managed. The identifier is assigned (immediately for IDENTITY, from the sequence for SEQUENCE).	INSERT at flush (immediately for IDENTITY)
`merge(e)`	Copies the state of a detached or transient instance onto a managed instance and returns that managed instance. The argument stays detached.	SELECT when the row is not in the context; UPDATE at flush
`remove(e)`	Managed to removed.	DELETE at flush
`detach(e)`, `clear()`, `close()`	Managed to detached. Later modifications are not tracked.	None
`refresh(e)`	Overwrites the instance with the current row.	SELECT
`find(C, id)`	Returns the managed instance, loading it when absent. Returns null when the row does not exist.	SELECT unless cached
`getReference(C, id)`	Returns a managed instance or a lazy proxy without SQL; the first access of a non-identifier attribute loads the row.	None until first access

EntityManager operations and the resulting state.

Spring Data's `save` is a thin wrapper: `SimpleJpaRepository.save` calls `persist` when `entityInformation.isNew(entity)` is true and `merge` otherwise. An entity is new when its identifier is null (or, for a primitive identifier, zero), or, when the entity has a version attribute of a wrapper type, when the version is null. An entity with an assigned identifier and no version attribute is therefore treated as existing, and `save` issues a `SELECT` followed by an `INSERT`. The remedy is a `@Version` attribute or an implementation of `Persistable<ID>`.

Flush and dirty checking

A flush synchronises the persistence context with the database: pending inserts, updates and deletes are executed as SQL on the current transaction. A flush is not a commit; the changes remain invisible to other transactions and can still be rolled back. With the default `FlushModeType.AUTO` a flush occurs (1) before the transaction commits, (2) before a JPQL or criteria query when pending changes touch tables that the query reads, and (3) on an explicit `flush()`. Whether a native SQL query triggers a flush depends on the Hibernate version and on how the session was bootstrapped, because Hibernate cannot know which tables a native statement reads; an explicit `flush()` before a native query that must observe pending changes removes that dependency.

During a flush Hibernate runs `DefaultFlushEntityEventListener` for every managed entity: the current attribute values are compared with the loaded state, and entities with a difference receive an update action in the `ActionQueue`. The queue executes actions in a fixed order, independent of the order of the Java calls: entity insertions, entity updates, collection removals, collection updates, entity deletions. This order is the reason that a delete followed by an insert of the same unique key in one transaction can violate the unique constraint (the insert runs first).

Two properties of dirty checking determine its cost and its output:

- **The comparison cost is proportional to the number of managed entities multiplied by their attribute count**, for each flush. A transaction that loads 50,000 entities pays this cost at every auto-flush. Read-only transactions avoid it (see the chapter on transactions) and bulk operations should use stateless techniques (see the chapter on performance).
- **By default all columns are written.** Hibernate prepares the update statement once per entity class, so a change to one attribute produces `update customer set name=? where id=?` only when `@DynamicUpdate` is present on the entity; otherwise every mapped column is included. The default is deliberate: it permits statement caching and JDBC batching, whereas dynamic statements differ per change set.

Proxies and lazy loading

A lazy association is represented by a proxy object, a runtime-generated subclass of the entity class (Hibernate 6 uses ByteBuddy) whose methods delegate to the real instance after loading it. Collections are represented by Hibernate's own `PersistentBag`, `PersistentSet` and `PersistentList` wrappers, which load their content on first access. Consequences:

- An entity class that is `final`, or has `final` methods that should trigger loading, cannot be proxied correctly. Entities need a no-argument constructor with at least protected visibility.
- Accessing an uninitialised proxy after the persistence context is closed throws `LazyInitializationException` (message: `could not initialize proxy [...] - no Session`). The exception is not a mapping error; it states that the data was not fetched while a session was available.
- Inside a proxy, `getClass()` is the generated subclass. `obj.getClass() == Customer.class` is false for a proxy; `instanceof` works. `Hibernate.unproxy(obj)` returns the underlying instance and `Hibernate.isInitialized(obj)` tests the state without triggering a load.

Identifier generation and its effect on batching

The strategy chosen for `@GeneratedValue` determines whether Hibernate can defer inserts until flush.

Strategy	Mechanism	Consequence
`IDENTITY`	Column `generated by default as identity`; the value is known only after the INSERT has run	`persist` executes the INSERT immediately; JDBC insert batching is disabled for that entity
`SEQUENCE`	`select nextval('customer_seq')`; with the pooled optimizer one call reserves `allocationSize` values (50 by default)	Inserts are deferred to flush and can be batched; one sequence round trip per 50 entities
`AUTO`	Hibernate 6 chooses SEQUENCE on PostgreSQL and names the sequence `<entity>_seq`	As SEQUENCE; the database sequence must have `INCREMENT BY 50` to match
`UUID`	Application generates the value (`@GeneratedValue` on a `UUID` attribute)	No round trip; larger index keys; random UUIDs fragment B-tree indexes
`TABLE`	Row in a generator table with locking	Rarely appropriate; serialises id generation

Identifier strategies on PostgreSQL.

The pooled optimizer relies on the sequence incrementing by the same amount as the `allocationSize` of the mapping. The SQL below was executed in PostgreSQL 16 on the build machine.

Sequence with INCREMENT BY 50: two calls reserve two blocks of ids — TERMINAL

```
$ psql -d shop -c "create sequence order_seq increment by 50 start 1"
$ psql -d shop -t -A -c "select nextval('order_seq') union all select nextval('order_seq')"
1
51
```

The first call returns 1 and the application may assign ids 1 to 50 without further database access; the next call returns 51. If the mapping declares `allocationSize = 50` but the database sequence increments by 1, Hibernate 6 validates this at start-up only when schema validation is active, and otherwise generates duplicate keys at run time. Matching the two values is a migration responsibility (see the chapter on performance for Flyway).

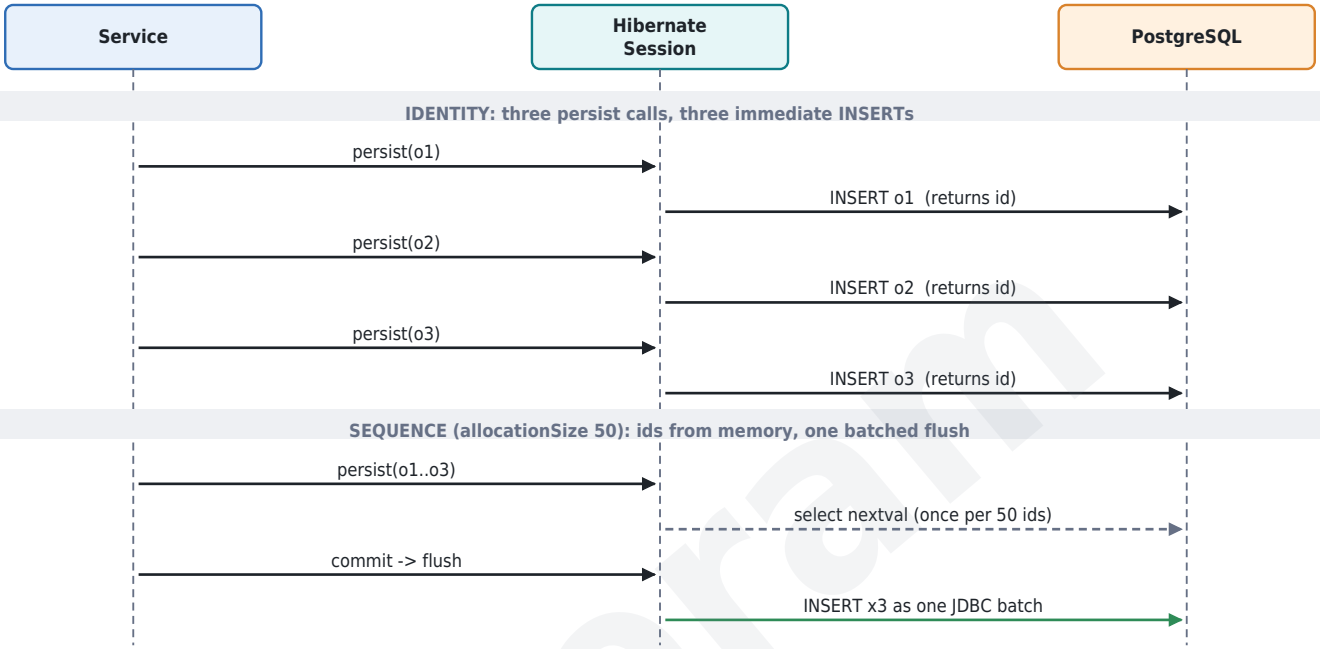


Figure 2.3. Insert behaviour for IDENTITY and SEQUENCE: with IDENTITY each persist forces an immediate round trip, whereas with SEQUENCE the inserts accumulate in the ActionQueue and are sent as a JDBC batch at flush.

equals and hashCode on entities

Entities are placed into `HashSet` and `HashMap` instances (for example as the elements of a `@OneToMany Set`), so `equals` and `hashCode` must stay stable over the lifetime of the object, including the moment `persist` assigns the identifier. Three implementations are in common use.

- **Default identity equality.** Correct as long as only one instance per row exists in the set. Detached copies loaded in different contexts are never equal.
- **Identifier-based equality** (the default output of many IDE generators): `hashCode` derived from a generated id changes when the id is assigned, so an instance added to a set before `persist` can no longer be found. This is the most frequent defect.
- **Business-key equality**, or **identifier equality with a constant hash code.** A natural key that is immutable (a SKU, an email address) can be used directly. Otherwise, equality can compare non-null identifiers while `hashCode` returns a class-based constant; the hash table then degrades to a list for very large sets, which is acceptable for the small collections that entities normally contain.

The effect is shown below with plain Java.

EqualsDemo.java — JAVA

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

public class EqualsDemo {

    /** Broken: hashCode depends on a generated id that is null until persist. */
    static class IdBased {
        Long id; final String sku;
        IdBased(String sku) { this.sku = sku; }
        @Override public boolean equals(Object o) {
```

EqualsDemo.java (continued) JAVA

```
            return o instanceof IdBased other && id != null && id.equals(other.id);
        }
        @Override public int hashCode() { return Objects.hashCode(id); }
    }

    /** Recommended when a natural key exists: immutable business key. */
    static class NaturalKey {
        Long id; final String sku;
        NaturalKey(String sku) { this.sku = sku; }
        @Override public boolean equals(Object o) {
            return o instanceof NaturalKey other && sku.equals(other.sku);
        }
        @Override public int hashCode() { return sku.hashCode(); }
    }

    /** Recommended otherwise: equal when ids are equal and non-null, constant hash per class. */
    static class StableHash {
        Long id;
        @Override public boolean equals(Object o) {
            return this == o || o instanceof StableHash other && id != null && id.equals(other.id);
        }
        @Override public int hashCode() { return StableHash.class.hashCode(); }
    }

    public static void main(String[] args) {
        Set<IdBased> a = new HashSet<>();
        IdBased x = new IdBased("A-1");
        a.add(x);
        x.id = 42L;                                  // "persist" assigns the id
        System.out.println("IdBased   contains after persist: " + a.contains(x));

        Set<NaturalKey> b = new HashSet<>();
        NaturalKey y = new NaturalKey("A-1");
        b.add(y);
        y.id = 42L;
        System.out.println("NaturalKey contains after persist: " + b.contains(y));

        Set<StableHash> c = new HashSet<>();
        StableHash z = new StableHash();
        c.add(z);
        z.id = 42L;
        System.out.println("StableHash contains after persist: " + c.contains(z));
        StableHash copy = new StableHash();
        copy.id = 42L;                               // detached copy of the same row
        System.out.println("StableHash detached copy equals: " + z.equals(copy));
    }
}
```

Real output (JDK 21) TERMINAL

```
$ java EqualsDemo.java
IdBased   contains after persist: false
NaturalKey contains after persist: true
StableHash contains after persist: true
StableHash detached copy equals: true
```

The `IdBased` variant fails because the element was stored in the bucket for `hashCode() == 0` and is searched in the bucket for `hashCode() == 42`. The set silently reports that its own element is absent, and operations such as `remove` on an association collection then do nothing.

2.1.2 Worked example

The domain used in this Part is a small order system. The PostgreSQL schema below is the one used for all SQL traces and measurements in the remaining chapters; the data set contains 50 customers, 20,000 orders, 60,000 order lines and 40,000 order tags.

V1__init.sql SQL

```sql
create table customer (
    id   bigint primary key,
    name text not null
);

create table purchase_order (
    id          bigint primary key,
    customer_id bigint not null references customer (id),
    status      text   not null,
    created_at  timestamptz not null default now(),
    version     bigint not null default 0
);

create table order_line (
    id       bigint primary key,
    order_id bigint not null references purchase_order (id),
    sku      text   not null,
    qty      int    not null
);

create index on order_line (order_id);
create index on purchase_order (created_at, id);
create sequence customer_seq       increment by 50;
create sequence purchase_order_seq increment by 50;
create sequence order_line_seq     increment by 50;
```

The entities map this schema. Field access is used (annotations on fields) because it keeps the identifier getter free of side effects and does not require setters.

Customer.java JAVA

```java
package com.example.shop.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "customer")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "customer_seq")
    @SequenceGenerator(name = "customer_seq", sequenceName = "customer_seq", allocationSize = 50)
    private Long id;

    @Column(nullable = false)
    private String name;

    protected Customer() { }                 // required by JPA, not for application code

    public Customer(String name) { this.name = name; }

    public Long getId() { return id; }
    public String getName() { return name; }
    public void rename(String newName) { this.name = newName; }

    @Override
    public boolean equals(Object o) {
        return this == o
                || o instanceof Customer other && id != null && id.equals(other.getId());
    }

    @Override
    public int hashCode() { return Customer.class.hashCode(); }
}
```

The entity exposes behaviour (`rename`) instead of a setter for every field. In `equals` the other instance is read through `getId()` and not through its field, because the argument may be a lazy proxy whose own fields are empty; a field read

Sreeram

on a proxy returns null. Hibernate populates the fields by reflection, so setters are unnecessary, and the domain methods keep invariants in one place.

PurchaseOrder.java JAVA

```java
package com.example.shop.domain;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.OneToMany;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;
import jakarta.persistence.Version;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

@Entity
@Table(name = "purchase_order")
public class PurchaseOrder {

    public enum Status { NEW, PAID, SHIPPED, CANCELLED }

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "purchase_order_seq")
    @SequenceGenerator(name = "purchase_order_seq", sequenceName = "purchase_order_seq",
            allocationSize = 50)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "customer_id")
    private Customer customer;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Status status = Status.NEW;

    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt = Instant.now();

    @Version
    private long version;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderLine> lines = new ArrayList<>();

    protected PurchaseOrder() { }

    public PurchaseOrder(Customer customer) { this.customer = customer; }

    public void addLine(String sku, int qty) {
        lines.add(new OrderLine(this, sku, qty));      // keeps both sides of the relation in sync
    }

    public void markPaid() {
        if (status != Status.NEW) {
            throw new IllegalStateException("Order " + id + " is " + status);
        }
        status = Status.PAID;
    }

    public Long getId() { return id; }
    public Customer getCustomer() { return customer; }
```

sreeram

PurchaseOrder.java (continued) JAVA

```
    public Status getStatus() { return status; }
    public Instant getCreatedAt() { return createdAt; }
    public List<OrderLine> getLines() { return Collections.unmodifiableList(lines); }
}
```

OrderLine.java JAVA

```
package com.example.shop.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "order_line")
public class OrderLine {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "order_line_seq")
    @SequenceGenerator(name = "order_line_seq", sequenceName = "order_line_seq",
            allocationSize = 50)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "order_id")
    private PurchaseOrder order;

    @Column(nullable = false)
    private String sku;

    @Column(nullable = false)
    private int qty;

    protected OrderLine() { }

    OrderLine(PurchaseOrder order, String sku, int qty) {
        this.order = order;
        this.sku = sku;
        this.qty = qty;
    }

    public Long getId() { return id; }
    public String getSku() { return sku; }
    public int getQty() { return qty; }
}
```

OrderService.java JAVA

```
package com.example.shop.service;

import com.example.shop.domain.Customer;
import com.example.shop.domain.PurchaseOrder;
import com.example.shop.repo.CustomerRepository;
import com.example.shop.repo.PurchaseOrderRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    private final CustomerRepository customers;
    private final PurchaseOrderRepository orders;

    public OrderService(CustomerRepository customers, PurchaseOrderRepository orders) {
        this.customers = customers;
        this.orders = orders;
    }
```

sreram

OrderService.java (continued) JAVA

```java
    @Transactional
    public Long place(long customerId, String sku, int qty) {
        Customer customer = customers.getReferenceById(customerId);   // proxy, no SQL
        PurchaseOrder order = new PurchaseOrder(customer);
        order.addLine(sku, qty);
        return orders.save(order).getId();                            // persist, cascades to lines
    }

    @Transactional
    public void pay(long orderId) {
        PurchaseOrder order = orders.findById(orderId).orElseThrow();
        order.markPaid();                                             // no save() call needed
    }
}
```

The two repositories are interfaces without a body (`interface CustomerRepository extends JpaRepository<Customer, Long> { }` and the same for `PurchaseOrderRepository`); their implementation is generated at start-up (see the chapter on query techniques).

What to notice: `place` obtains the customer with `getReferenceById`, which returns a proxy and avoids a `SELECT` for a row of which only the identifier is needed as the foreign key. `pay` never calls `save`: the managed `PurchaseOrder` is changed by `markPaid`, and the flush performed by the transaction interceptor before the commit detects the difference.

2.1.3 Hands-on

Making the SQL visible

SQL is made visible through the logging system, not through `spring.jpa.show-sql`, because the logger form includes the bind values and can be switched per environment without code changes.

application-dev.yml YAML

```yaml
logging:
  level:
    org.hibernate.SQL: DEBUG                # statements
    org.hibernate.orm.jdbc.bind: TRACE      # bind parameter values (Hibernate 6.x)
spring:
  jpa:
    open-in-view: false
    properties:
      hibernate:
        format_sql: true
        generate_statistics: true           # session metrics, logged at the end of each session
```

The logging category `org.hibernate.orm.jdbc.bind` is the Hibernate 6 name; in Hibernate 5 the category was `org.hibernate.type.descriptor.sql.BasicBinder`. The statistics property adds one log line per session that lists the number of JDBC statements, flushes and entity loads, which is the quickest indicator for an excessive statement count.

Tracing one transaction

Calling `place(7, "SKU-10", 2)` followed by `pay(...)` on an empty cache produces the following statements. The output is typical for Hibernate 6.6 on PostgreSQL (not run on the build machine); aliases such as `p1_0` are generated by Hibernate 6.

Typical Hibernate output for place(7, "SKU-10", 2) (not run on the build machine) TERMINAL

```
select nextval('purchase_order_seq')
select nextval('order_line_seq')
insert into purchase_order (created_at,customer_id,status,version,id) values (?,?,?,?,?)
insert into order_line (order_id,qty,sku,id) values (?,?,?,?)
```

Typical output for pay(1) (not run on the build machine) TERMINAL

```
select p1_0.id,p1_0.created_at,p1_0.customer_id,p1_0.status,p1_0.version
  from purchase_order p1_0 where p1_0.id=?
update purchase_order set customer_id=?,status=?,version=? where id=? and version=?
```

Preram

The sequence is queried once per entity type because the first `nextval` reserves 50 identifiers. No `SELECT` for the customer appears because `getReferenceById` supplied only the identifier for the foreign key. The `UPDATE` carries `and version=?` because of the `@Version` attribute and lists all updatable columns because `@DynamicUpdate` is absent.

The same mechanism in plain Java

The listing below implements the mechanism in about 100 lines: an identity map, loaded-state snapshots, a flush that compares them, and a lazy proxy created with `java.lang.reflect.Proxy`. It reproduces the observable behaviour of the persistence context but none of the optimisations of Hibernate.

MiniContext.java JAVA

```
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Proxy;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Objects;

public class MiniContext {

    interface Customer { long id(); String name(); }

    /** The "database": a map of rows; every access prints the SQL a provider would send. */
    static class Db {
        final Map<Long, String[]> rows = new HashMap<>();
        String[] select(long id) {
            System.out.println("  SQL> select id, name, tier from customer where id = " + id);
            return rows.get(id);
        }
        void update(long id, String sql) {
            System.out.println("  SQL> " + sql + " where id = " + id);
        }
    }

    static class CustomerEntity {
        long id; String name; String tier;
        CustomerEntity(long id, String name, String tier) {
            this.id = id; this.name = name; this.tier = tier;
        }
    }

    /** Minimal persistence context: identity map, snapshots, dirty checking on flush. */
    static class Context {
        final Db db;
        final Map<Long, CustomerEntity> managed = new LinkedHashMap<>();
        final Map<Long, String[]> snapshot = new HashMap<>();
        Context(Db db) { this.db = db; }

        CustomerEntity find(long id) {
            CustomerEntity e = managed.get(id);
            if (e != null) {
                System.out.println("  (first-level cache hit for id " + id + ")");
                return e;
            }
            String[] r = db.select(id);
            if (r == null) return null;
            e = new CustomerEntity(id, r[0], r[1]);
            managed.put(id, e);
            snapshot.put(id, r.clone());
            return e;
        }

        Customer getReference(long id) {      // lazy proxy: no SQL until a method is called
            InvocationHandler h = new InvocationHandler() {
                CustomerEntity target;
                public Object invoke(Object p, java.lang.reflect.Method m, Object[] a) {
                    if (m.getName().equals("id")) return id;      // identifier needs no SQL
                    if (target == null) target = find(id);
                    return m.getName().equals("name") ? target.name : null;
                }
            };
            return (Customer) Proxy.newProxyInstance(Customer.class.getClassLoader(),
                    new Class<?>[] {Customer.class}, h);
        }
```

Sreeram

MiniContext.java (continued) JAVA

```
        void flush() {
            System.out.println("  flush: comparing " + managed.size() + " managed instance(s)");
            for (CustomerEntity e : managed.values()) {
                String[] old = snapshot.get(e.id);
                boolean dirty = !Objects.equals(old[0], e.name) || !Objects.equals(old[1], e.tier);
                if (dirty) {   // like Hibernate's default, every column is written
                    db.update(e.id, "update customer set name = '" + e.name
                            + "', tier = '" + e.tier + "'");
                    snapshot.put(e.id, new String[] {e.name, e.tier});
                }
            }
        }
        void detach(CustomerEntity e) { managed.remove(e.id); snapshot.remove(e.id); }
    }

    public static void main(String[] args) {
        Db db = new Db();
        db.rows.put(1L, new String[] {"Asha", "SILVER"});
        db.rows.put(2L, new String[] {"Ravi", "GOLD"});
        Context ctx = new Context(db);

        System.out.println("1. find(1) twice:");
        CustomerEntity a = ctx.find(1), b = ctx.find(1);
        System.out.println("  same instance: " + (a == b));

        System.out.println("2. change a field, no explicit save, then flush:");
        a.tier = "GOLD";
        ctx.flush();
        System.out.println("  flush again (nothing changed):");
        ctx.flush();

        System.out.println("3. getReference(2) is a proxy:");
        Customer ref = ctx.getReference(2);
        System.out.println("  id() = " + ref.id() + " (no SQL)");
        System.out.println("  name() = " + ref.name());

        System.out.println("4. detached entity is no longer tracked:");
        ctx.detach(a);
        a.name = "Asha K";
        ctx.flush();
        System.out.println("  (no UPDATE was emitted for the detached instance)");
    }
}
```

Real output (JDK 21) TERMINAL

```
$ java MiniContext.java
1. find(1) twice:
  SQL> select id, name, tier from customer where id = 1
  (first-level cache hit for id 1)
  same instance: true
2. change a field, no explicit save, then flush:
  flush: comparing 1 managed instance(s)
  SQL> update customer set name = 'Asha', tier = 'GOLD' where id = 1
  flush again (nothing changed):
  flush: comparing 1 managed instance(s)
3. getReference(2) is a proxy:
  id() = 2 (no SQL)
  SQL> select id, name, tier from customer where id = 2
  name() = Ravi
4. detached entity is no longer tracked:
  flush: comparing 1 managed instance(s)
  (no UPDATE was emitted for the detached instance)
```

Step 2 is the central point: no write call exists in the program, yet the flush derives the `UPDATE` from the difference between the snapshot and the current values. The second flush finds no difference and emits nothing, because the snapshot was refreshed after the update. Step 4 shows the reverse: a change to a detached instance is lost silently unless the instance is merged.

sreeram

2.1.4 Real-world problems and solutions

Problem: updates that nobody asked for

Scenario. A reporting endpoint annotated `@Transactional` loads 2,000 orders to render a list. The database log shows an `UPDATE purchase_order` for many of the rows although the endpoint never calls `save`.

Symptoms. Latency grows with the result size. Row locks are taken on rows that were only read, and other transactions wait. The statistics line reports a nonzero entity update count.

Diagnosis. (1) Enable `org.hibernate.SQL=DEBUG` and `hibernate.generate_statistics` and find the `update` statements in the log. (2) Identify which attribute differs from the loaded state. A debugger breakpoint in `DefaultFlushEntityEventListener.onFlushEntity` or the log category `org.hibernate.event.internal.DefaultFlushEntityEventListener` at `TRACE` lists the dirty properties. (3) Search for code that writes to entity state in a read path: normalisation in a getter (`this.email = email.trim()`), a mutable attribute replaced by an equal copy (for example `Date`, arrays, or a `@Convert` converter that returns a new object without a value-based `equals`), and a `lastAccessed` timestamp set by an interceptor.

Root cause. The entities were managed and their state differed from the snapshot at flush time. Dirty checking compares values, not intent.

Fix. Remove state changes from read paths; mark the method `@Transactional(readOnly = true)`, which sets the Hibernate session to `FlushMode.MANUAL` so that no automatic flush occurs; return DTO projections for list endpoints so that no entity is managed (see the chapter on query techniques). For converters, implement `equals` on the converted value type or use `@Immutable` on the entity.

Prevention. A test that asserts the statement count for read endpoints (see the Hibernate `SessionFactory.getStatistics()` based assertion in the chapter on performance) turns a regression into a failing build.

Problem: an entity vanishes from its own HashSet

Scenario. An order holds `Set<OrderTag> tags`. After `order.addTag(tag)` followed by `save`, `order.removeTag(tag)` does nothing and the tag row is not deleted.

Symptoms. No exception; `set.contains(tag)` returns `false` for an element that was just added. The row remains after the transaction.

Diagnosis. (1) Print `tag.hashCode()` before and after the persist; the value differs. (2) Inspect `equals` and `hashCode` of `OrderTag`: an IDE-generated implementation based on the generated `id` attribute.

Root cause. The element was inserted into a hash bucket while the id was null (hash 0); `persist` assigned the id and the hash code changed (see `EqualsDemo`).

Fix. Use a business key that is assigned before the entity enters the collection, or equality on a non-null id with a constant hash code as shown in `Customer`. Never include lazy associations or mutable attributes in `equals` or `hashCode`; doing so can trigger loading inside hash operations.

Prevention. One repository-wide convention, enforced by a test that persists each entity inside a `HashSet` and asserts `contains`.

2.1.5 Questions to expect

INTERVIEW ANGLE

What is the persistence context, and how does it differ from a second-level cache? The persistence context is the first-level cache and identity map of one EntityManager: one instance per primary key, tracked for changes, destroyed when the EntityManager closes. It is mandatory and is scoped to a transaction. The second-level cache is optional, shared by all sessions of the SessionFactory and stores dehydrated entity state, not instances.

1. **What are the JPA entity states and which operations change them?** Transient, managed, detached and removed. `persist` makes a transient instance managed, `remove` makes a managed instance removed, `detach`, `clear` and closing the EntityManager make instances detached, and `merge` copies detached state onto a managed instance and returns it.
2. **Why does an update happen without calling `save`?** The managed entity differs from its loaded-state snapshot. At flush Hibernate compares them and queues an UPDATE. `save` is only required for transient or detached instances.

sheeran

3. **When does a flush occur?** Before commit, before a query that reads tables with pending changes (auto flush mode), and on explicit `flush()`. A flush sends SQL but does not commit.
4. **What is the difference between `find` and `getReference`?** `find` returns the entity or null and issues a SELECT unless the entity is cached; `getReference` returns a proxy without SQL and throws `EntityNotFoundException` on first access if the row does not exist.
5. **What is the difference between `persist` and `merge`?** `persist` makes the argument itself managed and requires a transient instance. `merge` returns a different managed instance carrying the state of the argument, which stays detached.
6. **Why is IDENTITY a poor choice for bulk inserts?** The value is known only after the INSERT is executed, so Hibernate must execute each INSERT during `persist` and cannot batch. SEQUENCE with a pooled optimizer permits deferred and batched inserts.
7. **Why can an entity not be a `final` class?** Lazy loading requires a runtime subclass. A final class cannot be subclassed, so references to it must be loaded eagerly or the proxy cannot be created.
8. **How should `equals` and `hashCode` be written for entities?** Stable across the persist transition: a business key, or equality on a non-null id with a constant hash code. Lazy associations must not participate.
9. **What does `Spring Data save` do?** It calls `persist` for a new entity (null id or null version) and `merge` otherwise. An entity with an assigned id and no version attribute therefore incurs an additional SELECT.

Check: what is the state?

```
Customer c = new Customer("Asha");          // transient
em.persist(c);                              // managed
em.flush();                                 // INSERT executed, still managed
em.detach(c);                               // detached
c.rename("Asha K");                         // not tracked, no UPDATE
Customer m = em.merge(c);                   // SELECT, then managed copy m with the new name
// c != m ; c remains detached ; UPDATE is issued at the next flush
```

KEY POINTS

The persistence context is an identity map with loaded-state snapshots; managed entities are written back by dirty checking at flush, without a `save` call.

- An EntityManager injected into a bean is a shared proxy; the real Session is bound to the current transaction by JpaTransactionManager.
- Entity states are relations to a persistence context; detached changes are lost unless merged.
- Auto flush occurs before commit and before queries on affected tables; the ActionQueue order is fixed, not the order of the calls.
- IDENTITY disables insert batching; SEQUENCE with allocationSize equal to the database increment enables it.
- Entity equals and hashCode must be stable across persist; a generated id must not feed the hash code.
- Make SQL visible through `org.hibernate.SQL` and `org.hibernate.orm.jdbc.bind` and count statements with Hibernate statistics.

Sreeram

2.2 Mapping in Depth

The mapping layer decides which SQL Hibernate can generate. Two mappings that describe the same table can differ by an order of magnitude in the number of statements: a unidirectional collection without a back reference, a `List` where a `Set` was required, or an inheritance strategy chosen without regard to polymorphic queries. This chapter treats the mapping annotations from the viewpoint of the SQL that each choice produces, and ends with optimistic locking, which is the mechanism that protects mapped data from concurrent updates.

2.2.1 Concept and mechanism

Relationships: ownership and direction

In a bidirectional relationship the database knows one foreign key, while the object model has two references. JPA resolves this with the concept of the **owning side**: the side whose changes are written to the foreign key column. The inverse side declares `mappedBy` and is ignored when the SQL is generated. For `PurchaseOrder` (one) and `OrderLine` (many) the owning side is `OrderLine.order`, because the foreign key `order_id` lives in `order_line`.

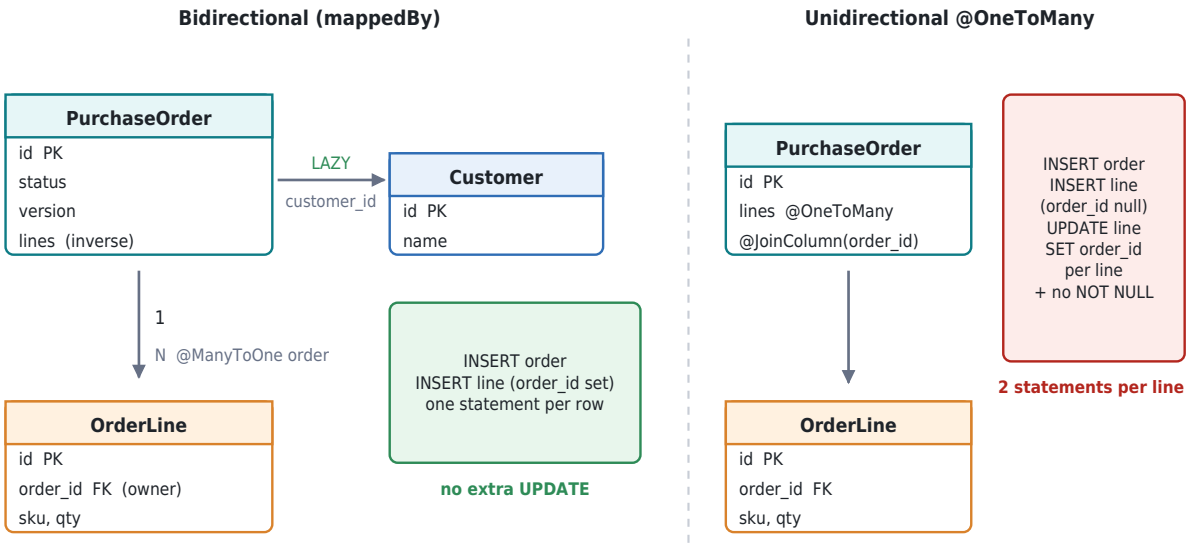


Figure 2.4. The order model: the many side (OrderLine.order) owns the foreign key and the one side declares mappedBy; the unidirectional variant on the right shows the extra statements that result when the owning side is missing.

Rules that follow from this:

- The application code must keep both sides consistent (`addLine` sets `line.order` and adds the line to the list). Hibernate reads only the owning side when it writes; the inverse collection is relevant only for the in-memory graph and for cascading.
- `@OneToMany(mappedBy = "order")` produces a single `INSERT` per line with the foreign key included. A unidirectional `@OneToMany` with `@JoinColumn(name = "order_id")` and no `@ManyToOne` produces `INSERT` with a null foreign key followed by `UPDATE order_line set order_id=? where id=?` for each line, and includes the key in the first insert only when the join column is declared `nullable = false`. A unidirectional `@OneToMany` without `@JoinColumn` uses an additional join table `purchase_order_lines`, which doubles the number of tables and joins.
- `cascade = CascadeType.ALL` propagates `persist`, `merge`, `remove`, `refresh` and `detach` from parent to children. `orphanRemoval = true` deletes a child that is removed from the parent collection. Both belong on aggregate-internal relations (an order and its lines); neither belongs on a reference to an independent entity (a line and its product).
- `@ManyToOne` and `@OneToOne` default to `FetchType.EAGER`; `@OneToMany` and `@ManyToMany` default to `LAZY`. A mapping that keeps the defaults for to-one relations imports an eager join into every query (see the chapter on fetching). All to-one associations are declared `LAZY` in this Part.
- A lazy `@OneToOne` on the side that does not hold the foreign key (`mappedBy`) cannot be proxied, because Hibernate cannot know from the owner row whether the target exists. Such an association is loaded eagerly unless bytecode enhancement is enabled. Sharing the primary key with `@MapsId` on the owning side avoids the problem.

Many-to-many and the join entity

`@ManyToMany` maps a join table with two foreign keys and no other columns. Three defects are common.

1. **List semantics.** With a `List` (a Hibernate bag) a single removal causes Hibernate to delete all join rows of the owner and re-insert the remaining ones, because a bag has no row identity. A `Set` removes exactly one row.
2. **Cascade REMOVE.** `CascadeType.REMOVE` or `ALL` on a many-to-many deletes the target entities, which are shared with other owners; the foreign key violation or the data loss follows. Only `PERSIST` and `MERGE` are meaningful there.
3. **No payload.** The moment the link needs a column (a quantity, a timestamp, a role), the mapping must become two `@ManyToOne` relations to a join entity. Starting with the join entity avoids the later migration.

The join entity pattern for the link between an order and a promotion is shown below; the composite identifier is explained in the section on keys.

OrderPromotion.java JAVA

```
package com.example.shop.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;
import jakarta.persistence.EmbeddedId;
import jakarta.persistence.Entity;
```

OrderPromotion.java (continued) JAVA

```java
import jakarta.persistence.FetchType;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.MapsId;
import java.io.Serializable;
import java.math.BigDecimal;
import java.util.Objects;

@Entity
public class OrderPromotion {

    @Embeddable
    public record Key(@Column(name = "order_id") Long orderId,
                      @Column(name = "promotion_id") Long promotionId) implements Serializable { }

    @EmbeddedId
    private Key id;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("orderId")
    private PurchaseOrder order;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("promotionId")
    private Promotion promotion;

    @Column(nullable = false)
    private BigDecimal discount;

    protected OrderPromotion() { }

    public OrderPromotion(PurchaseOrder order, Promotion promotion, BigDecimal discount) {
        this.id = new Key(order.getId(), promotion.getId());
        this.order = order;
        this.promotion = promotion;
        this.discount = discount;
    }

    @Override
    public boolean equals(Object o) {
        return o instanceof OrderPromotion other && Objects.equals(id, other.id);
    }

    @Override
    public int hashCode() { return Objects.hash(id); }
}
```

The record `Key` is a valid `@Embeddable` from Hibernate 6.2; records are immutable and provide value-based `equals` and `hashCode`, which are exactly the requirements of a composite key. `@MapsId` tells Hibernate that the foreign key columns of the associations are the same columns as the key parts, so no duplicate columns are mapped. A record cannot be an `@Entity`, because entities need a no-argument constructor and non-final, mutable state for proxies and dirty checking.

Embeddables and element collections

An `@Embeddable` groups attributes that have no identity of their own (an address, a money amount) into one value object whose columns live in the owner's table. `@AttributeOverride` renames columns when the same embeddable is used twice (billing and shipping address). `@ElementCollection` maps a collection of embeddables or basic values to a separate table; because the elements have no identifier, Hibernate treats the collection like a bag: any change deletes and re-inserts the whole collection unless the collection is a `Set` or has an `@OrderColumn`. Element collections are acceptable for small, rarely changing lists (tags of a document) and unsuitable when the rows must be queried or updated individually; in that case a real entity is required.

Inheritance strategies

JPA offers four ways to map a class hierarchy. The choice defines the table layout and the shape of the polymorphic query `select p from Payment p`. The three tables below were created in PostgreSQL 16 for a hierarchy `Payment` with subclasses `CardPayment` (attribute `last4`) and `UpiPayment` (attribute `upiId`), and the SQL shown is what a provider generates for the polymorphic query. The statements were executed against the same two rows in each layout.

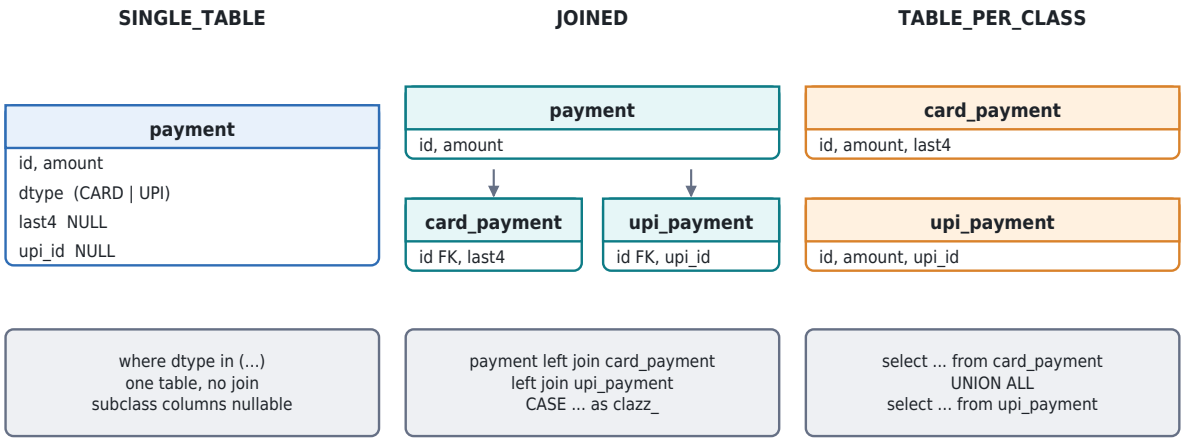


Figure 2.5. The three table layouts for a Payment hierarchy and the SQL shape each produces for a polymorphic query: a discriminator filter, joins to subclass tables, or a UNION ALL.

SINGLE_TABLE layout and polymorphic query SQL

```sql
create table pay_st (id bigint primary key, dtype varchar(31) not null,
    amount numeric(10,2) not null, last4 varchar(4), upi_id text);
select id, dtype, amount, last4, upi_id from pay_st;
```

SINGLE_TABLE result (real output, PostgreSQL 16) TERMINAL

```
 id | dtype | amount | last4 |  upi_id
----+-------+--------+-------+-----------
  1 | CARD  | 500.00 | 4242  |
  2 | UPI   | 250.00 |       | asha@bank
```

JOINED layout and polymorphic query SQL

```sql
create table pay_j      (id bigint primary key, amount numeric(10,2) not null);
create table pay_j_card (id bigint primary key references pay_j(id),
    last4 varchar(4) not null);
create table pay_j_upi  (id bigint primary key references pay_j(id),
    upi_id text not null);
select p.id, p.amount, c.last4, u.upi_id,
       case when c.id is not null then 1 when u.id is not null then 2 end as clazz_
from pay_j p left join pay_j_card c on c.id = p.id left join pay_j_upi u on u.id = p.id;
```

TABLE_PER_CLASS layout and polymorphic query SQL

```sql
create table pay_t_card (id bigint primary key, amount numeric(10,2) not null,
    last4 varchar(4) not null);
create table pay_t_upi  (id bigint primary key, amount numeric(10,2) not null,
    upi_id text not null);
select id, amount, last4, null as upi_id, 1 as clazz_ from pay_t_card
union all
select id, amount, null, upi_id, 2 from pay_t_upi;
```

The JOINED and TABLE_PER_CLASS queries returned the same rows as the single-table query, with the additional `clazz_` column that Hibernate uses to choose the Java subclass:

JOINED and TABLE_PER_CLASS result (real output, identical for both) TERMINAL

```
 id | amount | last4 |  upi_id   | clazz_
----+--------+-------+-----------+--------
  1 | 500.00 | 4242  |           |      1
  2 | 250.00 |       | asha@bank |      2
```

Aspect	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Polymorphic query	One table scan, discriminator filter	Left joins to every subclass table	UNION ALL of every concrete table
Subclass-specific columns	Nullable; no NOT NULL constraint possible	Normalised; constraints possible	Normalised; constraints possible
Insert cost	One INSERT	One INSERT per level	One INSERT

Aspect	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Identifier generation	Any	Any	Not IDENTITY (ids must be unique across tables)
Typical use	Small hierarchies, few subclass attributes	Many subclass attributes, strict constraints	Rarely; queries on the base type are expensive

Comparison of inheritance strategies.

`@MappedSuperclass` is a fourth option that is not an inheritance mapping in the JPA sense: the superclass is not an entity, cannot be queried, and only contributes its attributes to the tables of its subclasses. It is the right tool for shared columns such as audit fields. When polymorphic queries are not required, composition (a `Payment` that holds a `details` embeddable) is simpler than any hierarchy.

Enumerations, converters and JSON

`@Enumerated(EnumType.ORDINAL)` stores the position of the constant; inserting or reordering constants silently corrupts existing data. `EnumType.STRING` stores the name and is the correct default. On PostgreSQL a `text` column with a check constraint, or a native enum type mapped with `@JdbcTypeCode(SqlTypes.NAMED_ENUM)` (Hibernate 6.2 and later), enforces valid values in the database. A custom `AttributeConverter<X, Y>` annotated `@Converter(autoApply = true)` or applied with `@Convert` translates between a Java type and a column type (an `Email` value object to `text`). A converter must produce values whose `equals` is value-based, otherwise every flush sees a change (see the previous chapter). Structured documents are mapped with `@JdbcTypeCode(SqlTypes.JSON)` on a `jsonb` column; the content is then opaque to the relational model and cannot be joined.

Auditing

Spring Data JPA provides audit attributes through an entity listener. `@EnableJpaAuditing` activates the infrastructure, `@EntityListeners(AuditingEntityListener.class)` on a `@MappedSuperclass` attaches the listener, and the annotations `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy` and `@LastModifiedBy` mark the attributes. The auditor is resolved by an `AuditorAware<T>` bean, typically reading the `SecurityContext`. The listener runs in the `PrePersist` and `PreUpdate` callbacks, so the values are set at flush. A bulk `@Modifying` query bypasses entities and therefore the audit callbacks.

Soft deletion

Soft deletion keeps the row and marks it as deleted. Hibernate 6 provides three mechanisms: `@SQLDelete(sql = "update ... set deleted = true where id = ?")` combined with `@SQLRestriction("deleted = false")` (the replacement of the deprecated `@Where`, available since Hibernate 6.3), and `@SoftDelete` (since Hibernate 6.4), which generates both the update and the filter. Limits to be accepted: unique constraints must become partial indexes (`create unique index on customer (email) where not deleted`), native queries and foreign keys ignore the restriction, and cascades through soft-deleted parents must be reasoned about explicitly. Whether soft deletion is required at all is a data-retention question; an append-only audit table is often the simpler answer.

Composite keys

A composite key is mapped with `@EmbeddedId` (an embeddable holding the key columns, as in `OrderPromotion.Key`) or with `@IdClass` (the entity declares the key attributes itself and an identically named class mirrors them). The key class must be `Serializable` and implement value-based `equals` and `hashCode`; a record satisfies both. `@EmbeddedId` is preferred because `find(OrderPromotion.class, key)` takes a single typed argument. Composite keys prevent Spring Data's `isNew` detection from relying on a null id, so an entity with an assigned key either carries a `@Version` attribute or implements `Persistable`.

Optimistic locking with @Version

Two transactions that read the same row and later write it overwrite each other: the second update replaces the first without ever seeing it. This anomaly is a **lost update**. It happens at the default isolation level of PostgreSQL (READ COMMITTED) and at the level of the application, whatever the database. An attribute annotated `@Version` (a `long`, `int`, `short`, `Long`, `Instant` or `Timestamp`) turns every update into a compare-and-swap:

Update statement of a versioned entity SQL

```sql
update purchase_order set status=?, version=? where id=? and version=?
```

The version value read at load time is the last bind parameter. When the number of updated rows is zero, another transaction has changed the row; Hibernate throws `StaleObjectStateException` (wrapped in the JPA

`OptimisticLockException`), which Spring translates to `ObjectOptimisticLockingFailureException`. The following session was executed in PostgreSQL on the table `product(id, stock, version)` and reproduces the sequence of two clients that both read `stock = 10`.

Lost update without a version column (real output) TERMINAL

```
$ psql -d shop -t -A -c "select 'A read stock=' || stock from product where id=1"
A read stock=10
$ psql -d shop -t -A -c "select 'B read stock=' || stock from product where id=1"
B read stock=10
$ psql -d shop -c "update product set stock = 7 where id=1"      # A: 10 - 3
$ psql -d shop -c "update product set stock = 6 where id=1"      # B: 10 - 4
$ psql -d shop -t -A -c "select 'final stock=' || stock || ' (expected 3)' from product where id=1"
final stock=6 (expected 3)
```

The same sequence with a version predicate (real output) TERMINAL

```
$ psql -d shop -c "update product set stock = 7, version = version + 1 where id=2 and version = 0"
UPDATE 1
$ psql -d shop -c "update product set stock = 6, version = version + 1 where id=2 and version = 0"
UPDATE 0
$ psql -d shop -t -A -c "select 'stock=' || stock || ', v=' || version from product where id=2"
stock=7, v=1
```

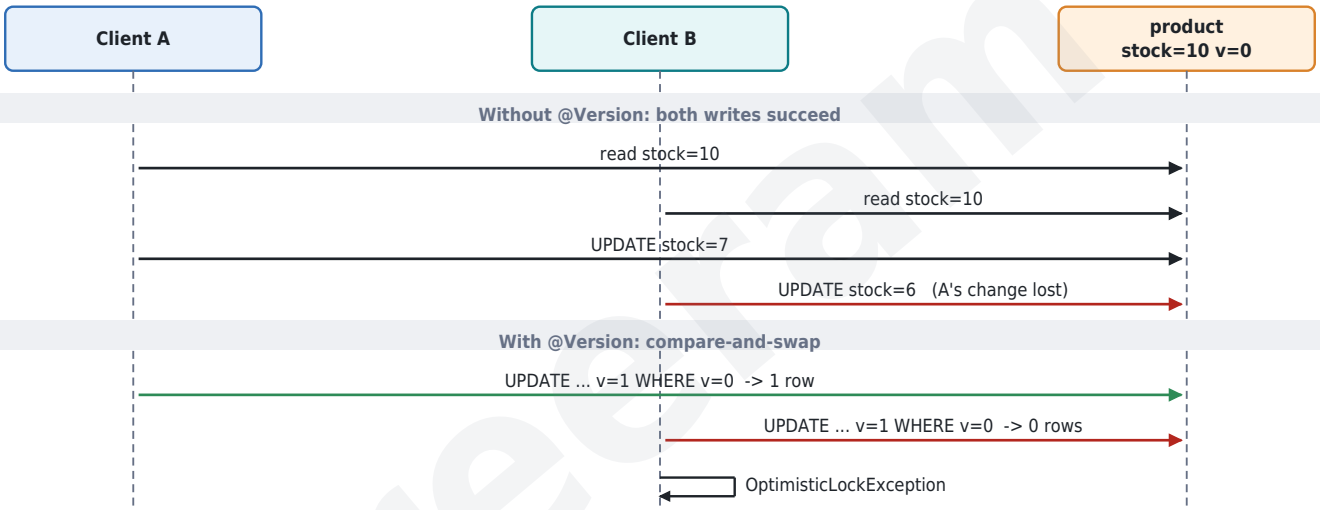


Figure 2.6. Lost update and its prevention: without a version predicate both writes succeed and A's change is lost; with the predicate the second UPDATE matches zero rows and the application is told.

The failure must be handled: a retry of the whole use case (reload, re-apply, save) for idempotent operations, or a message to the user in an interactive edit (`409 Conflict`). Two refinements exist. `LockModeType.OPTIMISTIC_FORCE_INCREMENT` raises the version of an entity that was only read, protecting an aggregate root when only a child changed. **Pessimistic locking** (`LockModeType.PESSIMISTIC_WRITE`, `@Lock(PESSIMISTIC_WRITE)` on a repository method) issues `select ... for update` and blocks other writers until the transaction ends; it is appropriate for short critical sections with high contention (stock reservation) and requires a lock timeout (`jakarta.persistence.lock.timeout`) and consistent ordering of locked rows to avoid deadlocks.

2.2.2 Worked example

The following classes complete the model: a shared audit superclass, an embeddable value object, and a single-table payment hierarchy.

Auditable.java JAVA

```java
package com.example.shop.domain;

import jakarta.persistence.Column;
import jakarta.persistence.EntityListeners;
import jakarta.persistence.MappedSuperclass;
import java.time.Instant;
import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
```

Auditable.java (continued) JAVA

```java
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class Auditable {

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @LastModifiedDate
    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;

    @CreatedBy
    @Column(name = "created_by", updatable = false)
    private String createdBy;

    @LastModifiedBy
    @Column(name = "updated_by")
    private String updatedBy;

    public Instant getCreatedAt() { return createdAt; }
    public Instant getUpdatedAt() { return updatedAt; }
}
```

Address.java JAVA

```java
package com.example.shop.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;

@Embeddable
public record Address(
        @Column(name = "street") String street,
        @Column(name = "city") String city,
        @Column(name = "postal_code", length = 12) String postalCode) { }
```

Payment.java JAVA

```java
package com.example.shop.domain;

import jakarta.persistence.DiscriminatorColumn;
import jakarta.persistence.DiscriminatorType;
import jakarta.persistence.DiscriminatorValue;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Inheritance;
import jakarta.persistence.InheritanceType;
import jakarta.persistence.SequenceGenerator;
import java.math.BigDecimal;

@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "dtype", discriminatorType = DiscriminatorType.STRING, length = 31)
public abstract class Payment extends Auditable {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "payment_seq")
    @SequenceGenerator(name = "payment_seq", sequenceName = "payment_seq", allocationSize = 50)
    private Long id;

    private BigDecimal amount;

    protected Payment() { }

    protected Payment(BigDecimal amount) { this.amount = amount; }

    public Long getId() { return id; }
    public BigDecimal getAmount() { return amount; }
}
```

Payment.java (continued) JAVA

```java

@Entity
@DiscriminatorValue("CARD")
class CardPayment extends Payment {
    private String last4;
    protected CardPayment() { }
    CardPayment(BigDecimal amount, String last4) { super(amount); this.last4 = last4; }
}

@Entity
@DiscriminatorValue("UPI")
class UpiPayment extends Payment {
    private String upiId;
    protected UpiPayment() { }
    UpiPayment(BigDecimal amount, String upiId) { super(amount); this.upiId = upiId; }
}
```

Java permits one public top-level type per file, so the two subclasses are declared package-private in the same file for brevity. In production code each entity has its own file. The auditing infrastructure is activated by two beans:

AuditConfig.java JAVA

```java
package com.example.shop.config;

import java.util.Optional;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.domain.AuditorAware;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;

@Configuration
@EnableJpaAuditing
public class AuditConfig {

    @Bean
    AuditorAware<String> auditorAware() {
        return () -> Optional.ofNullable(SecurityContextHolder.getContext().getAuthentication())
                .filter(Authentication::isAuthenticated)
                .map(Authentication::getName)
                .or(() -> Optional.of("system"));
    }
}
```

A retry wrapper is the second building block for optimistic locking. It must wrap the transaction, not run inside it, because a transaction that has received the exception is marked rollback-only.

StockService.java JAVA

```java
package com.example.shop.service;

import com.example.shop.domain.Product;
import com.example.shop.repo.ProductRepository;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.support.TransactionTemplate;

@Service
public class StockService {

    private static final int MAX_ATTEMPTS = 3;

    private final ProductRepository products;
    private final TransactionTemplate tx;

    public StockService(ProductRepository products, TransactionTemplate tx) {
        this.products = products;
        this.tx = tx;
    }

    public void reserve(long productId, int qty) {
        for (int attempt = 1; ; attempt++) {
            try {
```

sreeram

StockService.java (continued) JAVA

```
                tx.executeWithoutResult(status -> {          // one new transaction per attempt
                    Product p = products.findById(productId).orElseThrow();
                    p.decrease(qty);                         // throws when stock would be negative
                });
                return;
            } catch (ObjectOptimisticLockingFailureException e) {
                if (attempt == MAX_ATTEMPTS) throw e;        // surface as 409 or 503
            }
        }
    }
}
```

What to notice: `Product` carries a `@Version long version` attribute, the transaction is opened by the `TransactionTemplate` inside each loop iteration, and the exception is caught outside of it. The same method annotated `@Transactional` with the loop inside would reuse a poisoned transaction.

2.2.3 Hands-on

The mapping above produces, for `new CardPayment(500, "4242")` saved in a transaction, the following typical SQL (not run on the build machine): the audit columns are filled by the listener at `PrePersist`, the discriminator is a literal.

Typical Hibernate output (not run on the build machine) TERMINAL

```
select nextval('payment_seq')
insert into payment (amount,created_at,created_by,updated_at,updated_by,last4,dtype,id)
  values (?,?,?,?,?,?,'CARD',?)
```

A polymorphic `paymentRepository.findAll()` produces `select p1_0.id,p1_0.dtype,p1_0.amount,p1_0.last4,p1_0.upi_id,... from payment p1_0`, and a query for the subclass adds `where p1_0.dtype='CARD'`. To observe the lost-update protection, start two test threads that call `StockService.reserve` for the same product with a `CountDownLatch` after the read; one of them receives the exception at the first attempt and succeeds at the second.

2.2.4 Real-world problems and solutions

Problem: lost update on a shared record

Scenario. Two clerks edit the same order status or the same stock figure at the same time. After both save, the order shows the status of the clerk who saved last, and the stock count is higher than the physical stock.

Symptoms. No error in any log. Audit columns show two updates within a second. The business total no longer matches the sum of the movements.

Diagnosis. (1) Reproduce with two sessions as in the psql trace above. (2) Check the entity for a `@Version` attribute and the table for a `version` column. (3) Enable `org.hibernate.SQL=DEBUG`: an update without `and version=?` in the `where` clause confirms the absence of locking. (4) Compare the `updated_at` values of the two writes.

Root cause. Both transactions loaded the row, changed values in memory and wrote the full row. READ COMMITTED does not prevent this, because neither transaction read data that the other had already committed at the time of its own update statement.

Fix. Add `@Version private long version;` and the column `version bigint not null default 0`. Handle `ObjectOptimisticLockingFailureException` with a retry or a `409 Conflict` response that carries the current representation. For counters, replace read-modify-write with an atomic statement (`update product set stock = stock - ?1 where id = ?2 and stock >= ?1`) as a `@Modifying` query.

Prevention. Every aggregate root that can be edited by more than one actor carries a version; the version is exposed in the API as an ETag and checked with `If-Match` for client-driven edits.

Problem: a delete that removes more than expected, and slowly

Scenario. `orderRepository.deleteById(7)` is expected to remove one order and its 3 lines. The request takes seconds for a large order, and in another mapping it also deletes products.

Symptoms. The SQL log shows one `select` for the order, one `select` for the lines, one `delete` per line and finally `delete from purchase_order`. With a `@ManyToMany` mapped with `cascade = ALL` the shared target rows are deleted as well and a foreign key violation or data loss follows.

Diagnosis. (1) Log the SQL and count the `delete` statements. (2) Read the `cascade` attributes of the relations of the deleted entity. (3) For the many-to-many case, query the join table for other owners before the deletion.

Root cause. `CascadeType.REMOVE` is implemented by the persistence context: Hibernate loads the children, schedules a `DELETE` for each, and executes them before the parent delete. The cascade is correct for aggregate-internal children but applies equally to shared entities when it is declared on a many-to-many.

Fix. Keep `cascade` and `orphanRemoval` on aggregate-internal relations only; remove cascade from many-to-many and to-one references. For large child sets, use a bulk statement: `@Modifying @Query("delete from OrderLine l where l.order.id = :id")` followed by the parent delete, accepting that the persistence context is bypassed (clear it afterwards). Alternatively declare `@OnDelete(action = OnDeleteAction.CASCADE)` on the association together with `on delete cascade` in the foreign key, which delegates the cascade to the database in one statement; entities already loaded in the persistence context or the second-level cache are then not informed.

Prevention. Review every `cascade = ALL` in code review with the question: does the parent own the lifecycle of the child? Add a test that deletes a representative aggregate and asserts the statement count.

2.2.5 Questions to expect

1. **What is the owning side of a relationship?** The side that writes the foreign key. In a bidirectional association the side with `mappedBy` is the inverse side and is ignored for SQL generation; the application must update both sides.
2. **Why is a unidirectional `@OneToMany` inefficient?** Without a `@ManyToOne` on the child, Hibernate inserts the child first and then updates the foreign key per child (with `@JoinColumn`), or uses a join table. The bidirectional mapping with `mappedBy` writes the key during the insert.
3. **What is the difference between `cascade` and `orphanRemoval`?** `cascade` propagates an entity operation from parent to child. `orphanRemoval` deletes a child that is no longer referenced by the parent collection. A removal of the parent with cascade REMOVE also deletes the children, and a removal of a child from the collection requires `orphanRemoval`.
4. **Which inheritance strategy is the default and what are its trade-offs?** SINGLE_TABLE: one table, discriminator column, fast polymorphic queries, nullable subclass columns and no NOT NULL constraints. JOINED normalises at the cost of joins; TABLE_PER_CLASS issues UNION queries and cannot use IDENTITY.
5. **Why is `EnumType.ORDINAL` dangerous?** The stored value is the position of the constant, so reordering or inserting constants changes the meaning of existing rows. STRING stores the name.
6. **How does `@Version` work, and what does it protect against?** Every update adds `where version = ?` and increments the column. A zero update count raises `OptimisticLockException`, which prevents lost updates without holding database locks.
7. **When is pessimistic locking preferable?** For short, highly contended critical sections where retries would waste work, for example reserving the last units of stock. It requires a timeout and a consistent lock order.
8. **Why can a Java record not be an entity but can be an embeddable or a DTO?** Entities need a no-argument constructor, mutable state and non-final classes for proxying and dirty checking. Embeddables and projections only carry values, which records express well (Hibernate 6.2 and later).
9. **What is a `@MappedSuperclass`?** A non-entity base class whose mapped attributes are inherited by the entities that extend it. It cannot be queried or be the target of an association.
10. **What problem does `@ManyToMany` with a `List` cause?** A removal deletes all rows of the join table for that owner and re-inserts the remaining ones. A `Set`, or better a join entity, avoids this.

KEY POINTS

The owning side writes the foreign key; keep both sides consistent in domain methods and use `mappedBy` for the one side.

- Declare all to-one associations `LAZY`, `cascade` only for aggregate-internal children, and a `Set` or a join entity for many-to-many.
- Choose inheritance by the queries: SINGLE_TABLE for small hierarchies, JOINED for constraint-heavy ones; prefer composition when no polymorphic query exists.
- Store enums as STRING, give converters value-based equality, and use `@MappedSuperclass` for audit columns.
- `@Version` converts lost updates into detectable failures; handle the exception outside the transaction with a retry or a 409.
- Soft deletion has costs (unique indexes, native queries, foreign keys); apply it only when retention requires it.

2.3 Fetching and the N+1 Family

A mapped association is a promise that the related data can be reached by navigating the object graph. The cost of keeping that promise is decided at query time: a `PurchaseOrder` can be loaded together with its lines in one statement, in two statements, or in one statement per order. The wrong choice does not fail; it produces a correct result with hundreds of statements, and the defect becomes visible only with production-sized data or a network between the application and the database. This chapter explains the fetching strategies of Hibernate, the failure family that surrounds them (the N+1 select problem, `LazyInitializationException`, `MultipleBagFetchException`, in-memory pagination, open-in-view), and the way each is detected and corrected, with the SQL executed against PostgreSQL.

2.3.1 Concept and mechanism

Fetch timing and fetch strategy

Two independent decisions exist. The **fetch type** (`LAZY` or `EAGER`) states when the association is loaded relative to the owning entity. The **fetch strategy** states which SQL loads it: a join in the same statement, a secondary select, a batched select or a subselect. The annotation `@ManyToOne(fetch = EAGER)` fixes only the first decision, and Hibernate chooses the second depending on how the entity was requested.

- `entityManager.find` and `findById` load an eager to-one association with a join in the same statement.
- A JPQL or criteria query (including every Spring Data derived query and `findAll`) does not know about the join and loads the owners first; each eager association that is not yet in the persistence context is then loaded with an additional `select`, immediately and once per owner. Eager mappings therefore **cause** N+1 selects in queries.
- A lazy association issues its select on first access, once per owner.

The defaults are `EAGER` for `@ManyToOne` and `@OneToOne`, and `LAZY` for `@OneToMany` and `@ManyToMany`. The recommended configuration is `LAZY` everywhere, with the fetch plan declared per use case in the query.

The N+1 select problem

The N+1 select problem occurs when one query loads N parent rows and the code then touches an association of each parent, producing N further queries. A list endpoint that returns 20 orders with their lines is the standard case: one `select` for the page and one `select` for the lines of each order.

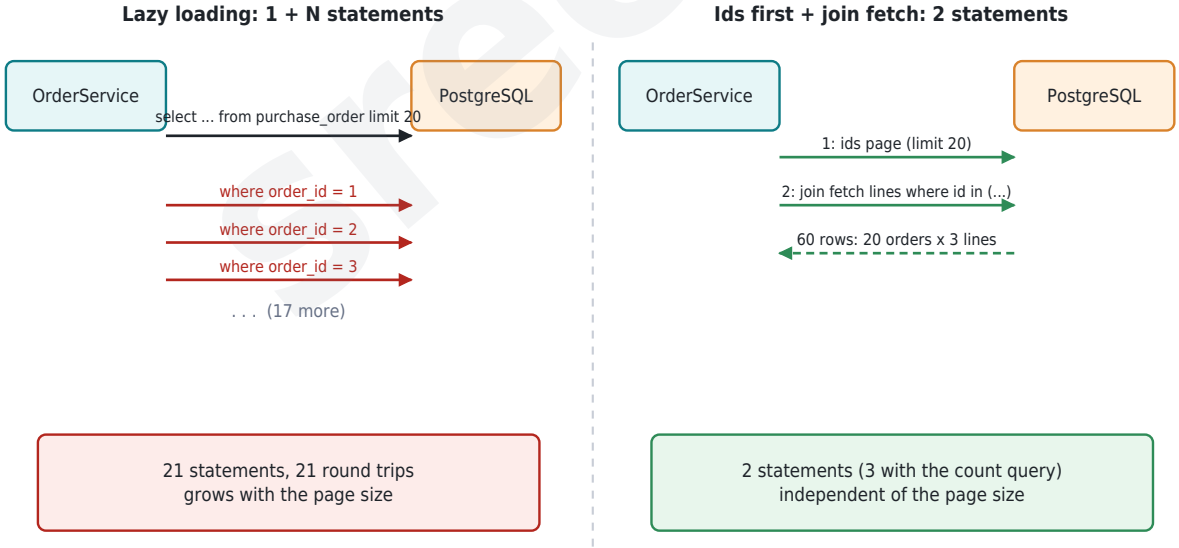


Figure 2.7. N+1 versus a fetch join for a page of 20 orders: the left sequence issues 21 statements; the right sequence issues one statement that carries the lines.

The statement count was measured on the build machine. The 20 order identifiers of the first page were loaded, and then the lines of each order were requested in turn, as lazy loading does; the PostgreSQL log (`log_statement = 'all'`) was then searched for the child statements.

Counting statements in the PostgreSQL log (real output) TERMINAL

```
$ grep -c "statement: select" /tmp/osw_s2/log
21
$ grep -c "order_line where order_id = [0-9]" /tmp/osw_s2/log
20
```

The same measurement for a page of 1,000 orders, executed through one psql session on the same machine over a Unix socket (no network latency), gives the proportion:

Strategy	Statements	Elapsed
One select per order (lazy loading)	1,001	203 ms
One join (join fetch)	1	44 ms

Loading the lines of 1,000 orders (measured on the build machine, typical).

The absolute numbers are small because the round trip within one machine costs microseconds. With a database 1 ms away (same availability zone) the first row of the table grows by roughly one second, and with 5 ms (another zone) by five seconds, while the join remains a single round trip. The count of statements is the metric to watch, not the elapsed time on a development machine.

Remedies and the SQL they produce

Five techniques change the number of statements. They differ in what they do to the result set size, pagination and flexibility.

Technique	SQL for 20 parents	Properties
`JOIN FETCH (JPQL)`	One statement, left join, 60 rows for 20 orders (3 lines each)	Parent columns repeat per child row; incompatible with SQL-level pagination; at most one bag per query
`@EntityGraph`	Same as join fetch, declared outside the query text	Reusable on derived queries; same restrictions
`@BatchSize(size = 20)` or `default_batch_fetch_size`	The parents: 1 statement. The lines: 1 statement with `order_id = any(?)` for up to 20 parents at a time	Works with pagination; the number of statements is ceil(N / batch size)
`@Fetch(FetchMode.SUBSELECT)`	The lines: `where order_id in (select id from purchase_order where <original query>)`	One extra statement for the whole page; the original predicate is executed twice; no pagination limit applied inside
`DTO projection`	One statement selecting only the needed columns, no managed entities	No lazy loading possible; no dirty checking; smallest payload

Fetching techniques for a collection association.

The three SQL shapes were executed against the 20-order page in PostgreSQL 16 (real output):

Row counts of the three fetch shapes (real output) TERMINAL

```
-- batch fetch form: children of 20 parents in one statement
 rows_returned
---------------
            60
-- join fetch form (20 orders, lines)
 rows_returned | distinct_orders
---------------+----------------
            60 |              20
```

The batch form is written as `where order_id = any (array[...])` with PostgreSQL's array parameter, or as `in (?, ?, ...)` with a padded parameter list, depending on the Hibernate version and the dialect; the effect is the same. Batch fetching is configured globally with `spring.jpa.properties.hibernate.default_batch_fetch_size=32` (a value between 10 and 100 is typical) or per association with `@BatchSize`. It turns N+1 into 1 + ceil(N / size) without changing any query, which makes it the safest default for an application with many read paths.

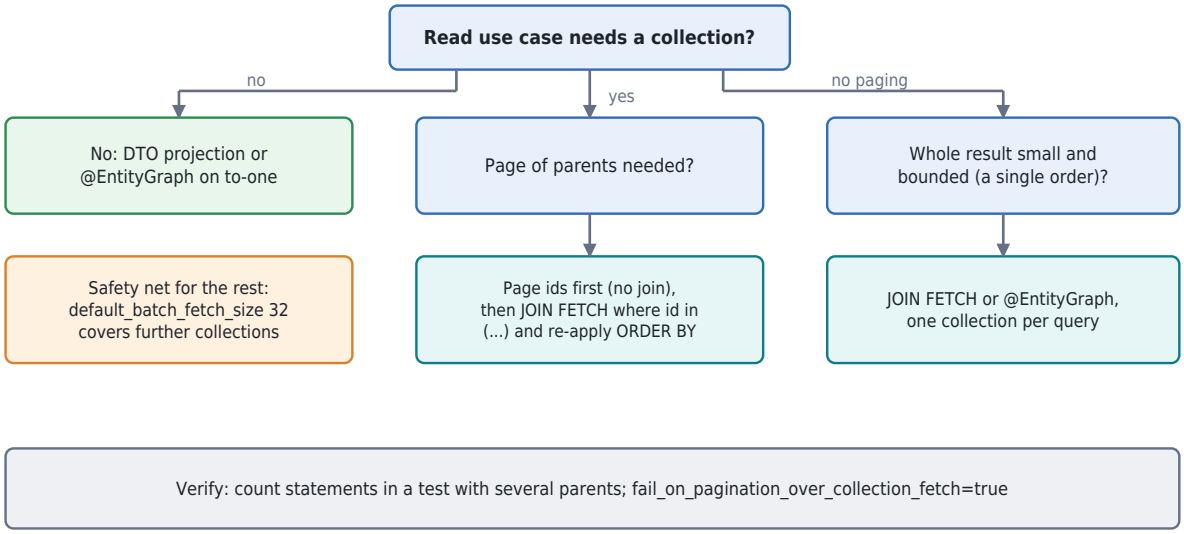


Figure 2.8. Decision path for choosing a fetch technique for a to-many association in a read use case.

Entity graphs

A JPA entity graph names the attributes to load for one operation. It can be declared on the entity with `@NamedEntityGraph(name = "PurchaseOrder.withLines", attributeNodes = @NamedAttributeNode("lines"))` and applied with the query hint `jakarta.persistence.fetchgraph` or, in Spring Data JPA, with `@EntityGraph("PurchaseOrder.withLines")` on a repository method. The ad hoc variant `@EntityGraph(attributePaths = {"lines", "customer"})` needs no declaration on the entity. A **fetch graph** (the Spring Data default, `EntityGraphType.FETCH`) treats attributes outside the graph as lazy; a **load graph** (`EntityGraphType.LOAD`) keeps the fetch types of the mapping for attributes outside the graph. The entity graph is translated into joins, so it has the same restrictions as `JOIN FETCH` for collections (bags, pagination).

MultipleBagFetchException and the cartesian product

A `List` without an `@OrderColumn` is a **bag**: it allows duplicates and has no row identity. When a single query fetches two bags, Hibernate cannot attribute the duplicated rows in the joined result to the correct collection and throws `MultipleBagFetchException` (message `cannot simultaneously fetch multiple bags`). Changing both collections to `Set` removes the exception but not the underlying problem, because the SQL result is still the cartesian product of the two collections for every parent. The product was measured with the 20-order page, where each order has 3 lines and 2 tags:

Rows returned by one join over two collections (real output) TERMINAL

```
 joined_rows
-------------
         120

 line_rows | tag_rows
-----------+----------
        60 |       40
```

One statement returns 120 rows to deliver 60 lines and 40 tags: the row count grows multiplicatively, 3 × 2 = 6 rows per order. With 50 lines and 20 tags per order the result holds 1,000 rows per order, and a page of 20 orders returns 20,000 rows. The correct approach is **one query per collection**: load the parents with the first collection, then load the second collection for the same parents in a second query (the persistence context completes the already loaded instances), or use batch fetching.

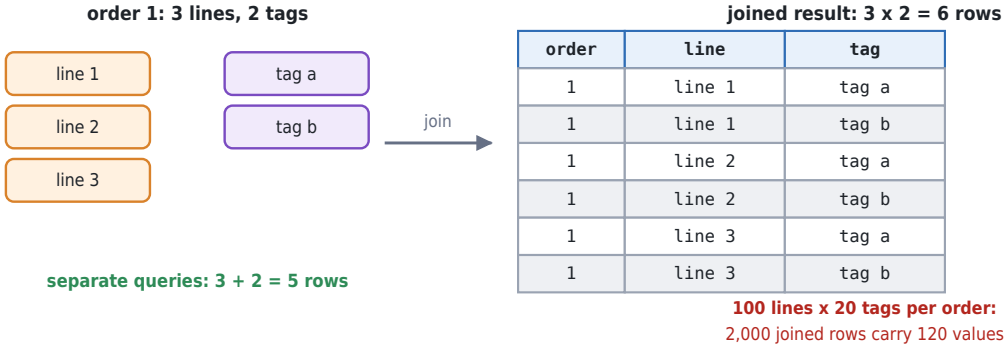


Figure 2.9. A join over two collections multiplies rows: the three lines of one order are paired with each of its two tags, so 5 child rows travel as 6 joined rows; two separate queries transfer only 5.

Pagination combined with a collection fetch

SQL limits count rows. A query that joins a collection and applies `LIMIT 10` returns ten joined rows, not ten parents. The effect was executed on the 20,000-order table:

LIMIT applied to a joined result (real output) TERMINAL

```
select o.id, l.id as line_id from purchase_order o join order_line l on l.order_id=o.id
  order by o.id limit 10;
 id | line_id
----+---------
  1 |       1
  1 |       2
  1 |       3
  2 |       4
  2 |       5
  2 |       6
  3 |       7
  3 |       8
  3 |       9
  4 |       10
(10 rows)
```

Ten rows contain only three complete orders and one incomplete order. Hibernate therefore refuses to push the limit into SQL when a collection is fetched: it loads the complete joined result, de-duplicates the parents and slices the list in memory, after logging the warning `HHH90003004: firstResult/maxResults specified with collection fetch; applying in memory`. For a table with 20,000 orders and 60,000 lines the statement transfers all 60,000 rows to return one page of 20. The property `hibernate.query.fail_on_pagination_over_collection_fetch=true` turns the warning into an exception, which is the recommended setting for a codebase in which the warning must never reach production.

The correct form paginates the parent identifiers first and fetches the collections for exactly those identifiers, or paginates the parents alone and uses batch fetching for the collections (see the worked example).

LazyInitializationException

A lazy association can be loaded only through a session that is still open. After the transaction of the service method ends (and without open-in-view) the persistence context is closed, the entities become detached, and touching an uninitialised association throws `LazyInitializationException: failed to lazily initialize a collection of role: com.example.shop.domain.PurchaseOrder.lines: could not initialize proxy - no Session`. The exception is the framework refusing to produce an incorrect answer; it does not indicate a configuration fault. Four legitimate corrections exist, in order of preference: load what the use case needs inside the service (join fetch, entity graph, batch fetching) and map to a DTO before the transaction ends; return a projection; call `Hibernate.initialize(order.getLines())` inside the transaction for a rare case; or keep the persistence context open longer (open-in-view, discussed below). The property `hibernate.enable_lazy_load_no_trans=true` silences the exception by opening a temporary session and connection for every lazy access; it is an anti-pattern that converts a loud error into an N+1 problem with N extra connections.

Open-in-view

Spring Boot registers `OpenEntityManagerInViewInterceptor` as a web request interceptor when `spring.jpa.open-in-view` is `true`, which is the default; at start-up Boot logs a warning stating that this setting is enabled. The interceptor creates an `EntityManager` before the controller is invoked and binds it to the thread until `afterCompletion`, after the response has been rendered. Transactions started by services reuse that `EntityManager`, and lazy associations can be loaded by the JSON serialiser after the service has returned.

The mechanism has a cost that is easily overlooked. `HibernateJpaVendorAdapter` configures Hibernate with the connection handling mode `DELAYED_ACQUISITION_AND_HOLD` (with the default `prepareConnection = true`): the JDBC connection is acquired at the first database access of the `EntityManager` and is released only when the `EntityManager` closes. With open-in-view that moment is the end of the request. A request that performs a 5 ms query and then spends 400 ms calling another service, or serialising a large response, holds its pooled connection for the whole time.

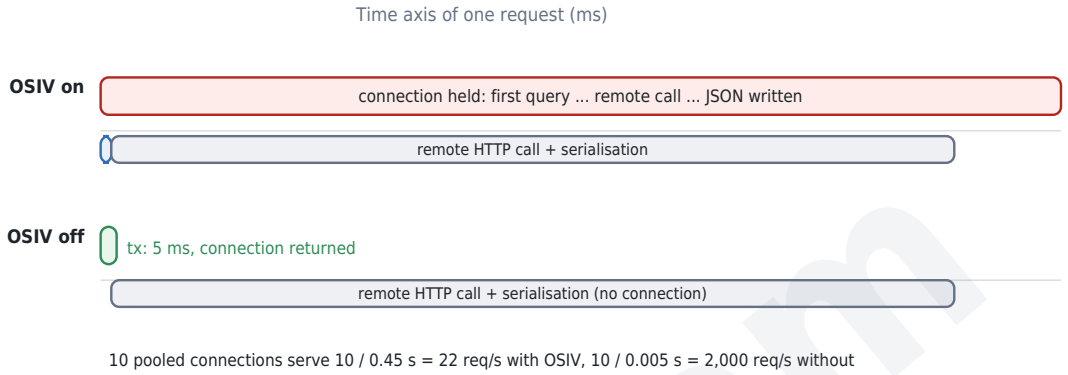


Figure 2.10. Connection hold time for one request: with open-in-view the connection is held from the first query until the response is written; with open-in-view disabled the connection is held only during the transaction.

Additional consequences: lazy loads in the view layer run outside any transaction (each in auto-commit mode, without the read-only and isolation settings of the service), and the number of queries is no longer visible in the service code. The remedy is `spring.jpa.open-in-view=false` together with DTO mapping inside the service transaction. The change is best made early in a project; in an existing application it exposes every `LazyInitializationException` that the setting had hidden.

2.3.2 Worked example

The use case is `GET /orders?status=NEW&page=0&size=20`: a page of orders with customer name and lines. The mapping from the previous chapters is used unchanged, with one addition: `PurchaseOrder` also holds `@OneToMany(mappedBy = "order") private List<OrderTag> tags`, which exists only to reproduce the multiple-bag failure. Configuration first:

application.yml YAML

```yaml
spring:
  jpa:
    open-in-view: false
    properties:
      hibernate:
        default_batch_fetch_size: 32
        query:
          fail_on_pagination_over_collection_fetch: true
        generate_statistics: true
logging:
  level:
    org.hibernate.SQL: DEBUG
```

`open-in-view: false` removes the safety net deliberately; `default_batch_fetch_size` makes every remaining lazy load cost at most one statement per 32 owners; `fail_on_pagination_over_collection_fetch` turns the in-memory pagination warning into an exception. The repository declares the two queries of the identifier-first technique and a third, entity-graph based, method for to-one data:

PurchaseOrderRepository.java JAVA

```java
package com.example.shop.repo;

import com.example.shop.domain.PurchaseOrder;
import com.example.shop.domain.PurchaseOrder.Status;
import java.util.Collection;
import java.util.List;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Slice;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface PurchaseOrderRepository extends JpaRepository<PurchaseOrder, Long> {

    /** Step 1: paginate identifiers only; no collection is joined, so LIMIT is applied in SQL. */
    @Query(value = "select o.id from PurchaseOrder o where o.status = :status "
            + "order by o.createdAt desc, o.id",
           countQuery = "select count(o) from PurchaseOrder o where o.status = :status")
    Page<Long> findIdsByStatus(@Param("status") Status status, Pageable pageable);

    /** Step 2: load the aggregates of exactly those identifiers with one join. */
    @Query("select o from PurchaseOrder o join fetch o.customer left join fetch o.lines "
            + "where o.id in :ids order by o.createdAt desc, o.id")
    List<PurchaseOrder> findWithLinesByIdIn(@Param("ids") Collection<Long> ids);

    /** To-one data only: join fetch is safe with pagination; Slice avoids the count query. */
    @EntityGraph(attributePaths = "customer")
    Slice<PurchaseOrder> findByStatus(Status status, Pageable pageable);
}
```

OrderView.java JAVA

```java
package com.example.shop.web;

import com.example.shop.domain.OrderLine;
import com.example.shop.domain.PurchaseOrder;
import java.util.List;

public record OrderView(Long id, String status, String customer, List<LineView> lines) {

    public record LineView(String sku, int qty) {
        static LineView of(OrderLine l) { return new LineView(l.getSku(), l.getQty()); }
    }

    public static OrderView of(PurchaseOrder o) {
        return new OrderView(o.getId(), o.getStatus().name(), o.getCustomer().getName(),
                o.getLines().stream().map(LineView::of).toList());
    }
}
```

OrderQueryService.java JAVA

```java
package com.example.shop.service;

import com.example.shop.domain.PurchaseOrder;
import com.example.shop.domain.PurchaseOrder.Status;
import com.example.shop.repo.PurchaseOrderRepository;
import com.example.shop.web.OrderView;
import java.util.List;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageImpl;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderQueryService {

    private final PurchaseOrderRepository orders;

    public OrderQueryService(PurchaseOrderRepository orders) { this.orders = orders; }
```

sreeram

OrderQueryService.java (continued) JAVA

```
    @Transactional(readOnly = true)
    public Page<OrderView> page(Status status, Pageable pageable) {
        Page<Long> ids = orders.findIdsByStatus(status, pageable);              // statements 1, 2
        List<PurchaseOrder> loaded = orders.findWithLinesByIdIn(ids.getContent()); // statement 3
        List<OrderView> views = loaded.stream().map(OrderView::of).toList();    // inside the tx
        return new PageImpl<>(views, pageable, ids.getTotalElements());
    }
}
```

OrderController.java JAVA

```
package com.example.shop.web;

import com.example.shop.domain.PurchaseOrder.Status;
import com.example.shop.service.OrderQueryService;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PagedModel;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderQueryService service;

    public OrderController(OrderQueryService service) { this.service = service; }

    @GetMapping
    public PagedModel<OrderView> list(@RequestParam(defaultValue = "NEW") Status status,
                                      Pageable pageable) {
        return new PagedModel<>(service.page(status, pageable));
    }
}
```

What to notice: (1) the entity never leaves the transaction; `OrderView.of` runs inside `page`, so no lazy access can occur in the controller or in Jackson; (2) the order of the result is re-established by repeating `order by` in the second query, because `in :ids` does not preserve the order of the page; (3) the controller returns `PagedModel`, available from Spring Data 3.3, because serialising a `PageImpl` directly is reported by Spring Data as unsupported (its JSON structure is not stable); (4) the pagination limit is applied in SQL in step 1, where no collection participates.

2.3.3 Hands-on

For a database of 20,000 orders, the request produces the following statements. The output is typical for Hibernate 6.6 (not run on the build machine); the PostgreSQL forms of the statements were executed manually in the preceding sections.

Typical output for GET /orders?status=NEW&size=20 (not run on the build machine) TERMINAL

```
select o1_0.id from purchase_order o1_0 where o1_0.status=?
  order by o1_0.created_at desc,o1_0.id offset ? rows fetch first ? rows only
select count(o1_0.id) from purchase_order o1_0 where o1_0.status=?
select o1_0.id,o1_0.created_at,c1_0.id,c1_0.name,l1_0.order_id,l1_0.id,l1_0.qty,l1_0.sku,
       o1_0.status,o1_0.version
  from purchase_order o1_0 join customer c1_0 on c1_0.id=o1_0.customer_id
  left join order_line l1_0 on o1_0.id=l1_0.order_id
 where o1_0.id in (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)
  order by o1_0.created_at desc,o1_0.id
```

Three statements serve the page regardless of its size; the naive version (`findAll(pageable)` followed by `order.getLines()` in the controller) issues 1 + 1 + 20. The statistics line at the end of the session can be used to verify the count: its text reports "preparing 3 JDBC statements" for the optimised request. To prove the claim in a test, enable `generate_statistics`, execute the request, and compare `sessionFactory.getStatistics().getPrepareStatementCount()` with the expected number (the chapter on performance shows the complete assertion).

The behaviour of the rejected variants can be reproduced by changing one line each: replacing the second query by `@Query("select o from PurchaseOrder o left join fetch o.lines")` with a `Pageable` parameter fails at execution

because `fail_on_pagination_over_collection_fetch` is enabled; adding `left join fetch o.tags` to the second query raises `MultipleBagFetchException: cannot simultaneously fetch multiple bags`.

2.3.4 Real-world problems and solutions

Problem: N+1 in a list endpoint

Scenario. `GET /orders` returns 100 orders with their lines. The endpoint takes 40 ms in development (10 orders) and 2.5 s in production, and the database CPU is low.

Symptoms. Latency grows linearly with page size. The database shows a high statement rate of identical statements that differ only in a bound id.

Diagnosis. (1) Enable `org.hibernate.SQL=DEBUG` for one request and count the statements; a repeating `select ... from order_line where order_id=?` is the signature. (2) With `hibernate.generate_statistics=true` read the session metrics line ("preparing N JDBC statements"). (3) In PostgreSQL, install `pg_stat_statements` (requires `shared_preload_libraries = 'pg_stat_statements'`) and run `select calls, mean_exec_time, query from pg_stat_statements order by calls desc limit 10;` to find statements with an abnormal call count. (4) Locate the access: the stack trace of a breakpoint in `AbstractPersistentCollection.initialize` or a Jackson serialisation frame (`BeanSerializer`) shows who triggers the lazy load.

Root cause. The entity list was returned, serialised, and every lazy collection was initialised one owner at a time, outside the query that loaded the owners.

Fix. Apply the fetch plan of the use case: identifier-first pagination with join fetch (worked example), `@EntityGraph` for to-one data, a DTO projection when entities are not needed, and `default_batch_fetch_size` as a global bound.

Prevention. An integration test per list endpoint that asserts the statement count on a data set with several parents (a test that passes with one parent proves nothing); `open-in-view=false` so that the accidental lazy load fails fast.

Problem: LazyInitializationException after the service returns

Scenario. After `spring.jpa.open-in-view=false` is introduced (or in a scheduler or message listener, which never had a view layer) a method fails with `LazyInitializationException: failed to lazily initialize a collection of role: ...PurchaseOrder.lines: could not initialize proxy - no Session`.

Symptoms. HTTP 500 or a failed message; the stack trace shows `AbstractPersistentCollection.throwLazyInitializationException`, called from a serialiser, a mapper or a stream operation.

Diagnosis. (1) Read the role named in the message: it identifies the association. (2) Find the first frame outside of any `@Transactional` boundary that touches it. (3) Check whether the entity was returned from a service method whose transaction already ended.

Root cause. The entity left the transaction in which it was loaded; its session closed; the collection was never fetched.

Fix. Move the mapping to DTO inside the transaction, or fetch the association in the query (join fetch, entity graph, batch fetching). Do not enable `hibernate.enable_lazy_load_no_trans`. Do not mark the controller `@Transactional` to keep the session open: that holds the connection during serialisation.

Prevention. Return DTOs from service methods that serve web or messaging boundaries; keep entities inside the transactional layer.

Problem: MultipleBagFetchException

Scenario. A developer adds `left join fetch o.tags` next to `left join fetch o.lines` and the application fails to start: `MultipleBagFetchException: cannot simultaneously fetch multiple bags: [...PurchaseOrder.lines, ...PurchaseOrder.tags]`.

Symptoms. The exception occurs at start-up for `@Query` methods (the query is validated when the repository is created) or at the first call for dynamically built queries.

Diagnosis. Both collections are `List` attributes without `@OrderColumn`, i.e. bags, and both appear in one fetch join or entity graph.

Root cause. A joined result over two bags cannot be mapped back to the two collections without ambiguity (see the cartesian product above).

Fix. Fetch one collection per query: load the parents with `lines` in the first query and `tags` in a second query for the same identifiers, or rely on batch fetching for the second collection. Changing the type to `Set` removes the exception but retains the cartesian product (120 rows for 100 child rows in the measured case), so it is not a performance fix.

Prevention. Allow at most one fetched collection per query as a code review rule; prefer `Set` for collections without order because it is semantically correct for unique children and avoids delete-and-reinsert behaviour.

Problem: pagination applied in memory

Scenario. A paged order list with `join fetch o.lines` returns the correct page but the endpoint becomes slower and consumes more heap as the table grows. The log contains `HHH90003004: firstResult/maxResults specified with collection fetch; applying in memory`.

Symptoms. Constant latency growth, `OutOfMemoryError` under load, a statement without `fetch first` or `limit` in the SQL log.

Diagnosis. Search the log for `HHH90003004`; inspect the generated SQL for the absence of a row limit; compare the row count returned by the statement (visible in the PostgreSQL slow-query log `auto_explain` output or in the statistics) with the page size.

Root cause. The collection join multiplies rows, so the database limit cannot be applied to parents; Hibernate loads everything and slices in memory.

Fix. Identifier-first pagination (worked example) or pagination of the parents alone plus batch fetching of the collections. Set `fail_on_pagination_over_collection_fetch=true` so that the defect fails in the first test.

Prevention. The same setting in every environment, and a repository test for each paginated query.

Problem: connection pool pressure from open-in-view

Scenario. Under moderate load a service that executes only 3 ms of SQL per request reports `HikariPool-1 - Connection is not available, request timed out after 30000ms`. The database is idle.

Symptoms. `hikaricp_connections_active` equals `hikaricp_connections_max`, `hikaricp_connections_pending` is non-zero, database CPU is low, request threads are in the JSON serialiser or in an outbound HTTP call.

Diagnosis. (1) Take a thread dump with `jcmd <pid> Thread.print` during the incident and look for request threads that are not in database code but are the owners of a connection (their stacks show `HttpMessageConverter`, `RestClient` or `Thread.sleep`). (2) Enable `spring.datasource.hikari.leak-detection-threshold=2000` (milliseconds), which logs the stack of the code that acquired a connection held longer than the threshold. (3) Check `spring.jpa.open-in-view` in the `/actuator/configprops` output or in the start-up log warning.

Root cause. With open-in-view the connection acquired by the first query is kept (`DELAYED_ACQUISITION_AND_HOLD`) until the `EntityManager` closes at the end of the request; time spent after the service call is time during which the connection is unavailable to others.

Fix. `spring.jpa.open-in-view=false` and DTO mapping inside the service. Remove remote calls from transactional methods or from request scopes that hold a connection (see the chapter on transactions).

Prevention. Alert on `hikaricp_connections_pending > 0` for longer than a few seconds, and keep the leak-detection threshold active in non-production environments.

2.3.5 Questions to expect

1. **What is the N+1 select problem and how is it detected?** One query loads N parents and a lazy association is then loaded with one query per parent. Detection: SQL logging or Hibernate statistics (statement count), `pg_stat_statements` call counts, and tests that assert the number of statements.
2. **Why do EAGER mappings not prevent N+1?** JPQL queries do not honour the fetch type with a join. The owners are loaded first, and each eager association is loaded with an extra select per owner. Only `find` by identifier uses a join for eager to-one associations.
3. **Compare `JOIN FETCH`, `@EntityGraph` and `@BatchSize`.** `JOIN FETCH` and entity graphs load the association in the same statement through a join (row multiplication for collections, no SQL pagination). `@BatchSize` loads lazily in groups of N owners with an `in`/`any` predicate, works with pagination and needs no query change.
4. **What causes `MultipleBagFetchException`, and what are the options?** Fetching two `List` collections without order column in one query. Options: fetch one collection per query, use batch fetching, or use `Set`; the last removes the exception but keeps the cartesian product.

5. **Why is pagination with a fetch join dangerous?** The database limit counts joined rows. Hibernate therefore loads all rows and paginates in memory (warning HHH90003004). Page the identifiers first, then fetch.
6. **What is open-in-view and why is it discouraged?** An interceptor that keeps the EntityManager open for the whole request so views can load lazily. It hides N+1, executes lazy loads outside transactions, and holds a JDBC connection until the request ends.
7. **How is `LazyInitializationException` correctly resolved?** By loading the required data inside the transaction (fetch plan) and returning DTOs. Not by `enable_lazy_load_no_trans` or by widening the transaction around the controller.
8. **What is the difference between a fetch graph and a load graph?** A fetch graph treats attributes outside the graph as lazy; a load graph keeps the mapping's fetch type for them. Spring Data's `@EntityGraph` defaults to the fetch graph (`EntityGraphType.FETCH`).
9. **When is a DTO projection better than an entity?** For read-only views: only the necessary columns are selected, no persistence context entries or snapshots are created, and lazy loading is impossible by construction.

KEY POINTS

Choose the fetch plan per use case in the query; keep every association `LAZY` in the mapping.

- N+1 is detected by counting statements (SQL log, Hibernate statistics, `pg_stat_statements`); a test with several parents is required to expose it.
- `default_batch_fetch_size` bounds the damage globally; join fetch or entity graphs serve known hot paths; projections serve read-only views.
- One collection per fetch join; page identifiers first when a collection is involved; enable `fail_on_pagination_over_collection_fetch`.
- Disable open-in-view, map to DTOs inside the transaction, and never reach for `enable_lazy_load_no_trans`.
- Open-in-view and long transactions are connection-pool problems as much as query problems.

2.4 Transactions in Spring

A transaction groups several database operations into one unit that either takes effect completely or not at all, and that is isolated from the effects of concurrent units. In a Spring application the unit is usually one service method, and the boundary is declared with `@Transactional`. The declaration looks trivial and hides a considerable mechanism: a proxy, an interceptor, a thread-bound resource registry, a connection from a pool, and a set of rules for exceptions, nesting and visibility. Most transaction defects (changes that are not rolled back, annotations that have no effect, connection starvation, unexpected `UnexpectedRollbackException`) follow directly from one of these rules. This chapter describes the mechanism, the propagation and isolation attributes, the rollback rules, programmatic and event-driven alternatives, deadlocks, and the interaction between transactions and the connection pool, with a plain-JDK simulation of the propagation stack.

2.4.1 Concept and mechanism

From annotation to JDBC transaction

`@EnableTransactionManagement` (applied automatically by Spring Boot's `TransactionAutoConfiguration`) registers three infrastructure beans: an `AnnotationTransactionAttributeSource`, which reads `@Transactional` from methods and classes; a `TransactionInterceptor`, which is the advice; and a `BeanFactoryTransactionAttributeSourceAdvisor`, which pairs them with a pointcut matching every method that carries transaction attributes. An auto-proxy creator wraps each matching bean in a proxy (a CGLIB subclass by default in Boot, `spring.aop.proxy-target-class=true`). Every call from another bean enters the proxy first; the proxy invokes `TransactionInterceptor.invoke`, which delegates to `TransactionAspectSupport.invokeWithinTransaction`.

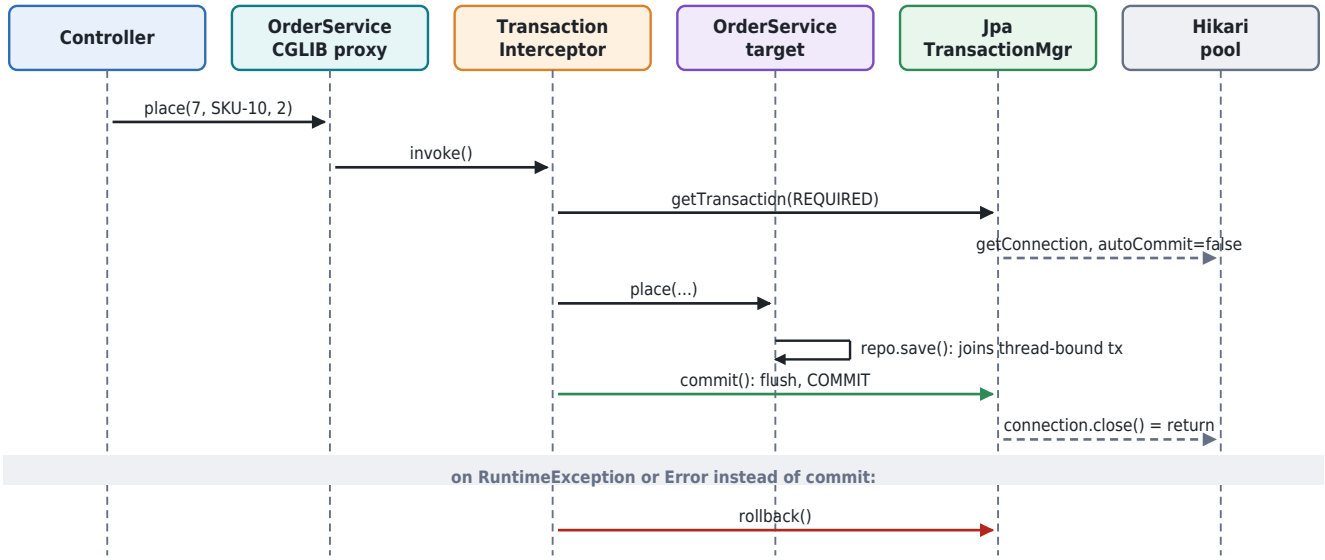


Figure 2.11. One call through the transactional proxy: the interceptor asks the transaction manager for a transaction, the target method runs inside it, and commit or rollback follows from the outcome; the repository calls reuse the Session and Connection bound by the manager.

`invokeWithinTransaction` performs the same steps for every call:

1. Determine the `TransactionAttribute` of the method (propagation, isolation, timeout, read-only flag, rollback rules) and the `PlatformTransactionManager` (the bean named by `@Transactional("name")`, or the only one available).
2. `createTransactionIfNecessary` calls `getTransaction(definition)`. For `JpaTransactionManager` the result of `doBegin` is: a Hibernate `Session` is created, a JDBC connection is taken from the pool, `setAutoCommit(false)` is applied (together with the isolation level and the read-only flag through `HibernateJpaDialect`), and the `EntityManagerHolder` and `ConnectionHolder` are bound to the thread in `TransactionSynchronizationManager`.
3. The target method is invoked. Any repository or `EntityManager` call finds the holder bound to the thread and joins the transaction.
4. On normal return `commitTransactionAfterReturning` calls `commit`: the persistence context is flushed (`beforeCommit` synchronisation), the JDBC `commit()` is executed, the connection goes back to the pool and the thread-bound resources are removed. On an exception `completeTransactionAfterThrowing` evaluates the rollback rules and calls `commit` or `rollback`.

The rules of the proxy follow from the architecture. Only calls that enter through the proxy are intercepted, so a method that calls another method of the same object (self-invocation) bypasses the interceptor. For class-based proxies methods may be `public`, `protected` or package-visible since Spring 6.0; `private` and `final` methods cannot be intercepted by a CGLIB proxy; for interface-based (JDK) proxies only interface methods are visible. The transaction is bound to the **thread**: a method that starts work on another thread (`@Async`, a parallel stream, `CompletableFuture.supplyAsync`) runs outside of it.

Transaction managers

Spring abstracts the resource-specific API behind `PlatformTransactionManager` (`getTransaction`, `commit`, `rollback`). `DataSourceTransactionManager` handles plain JDBC and `JdbcTemplate`; `JpaTransactionManager` handles JPA and also exposes the JDBC connection, so JPA and `JdbcTemplate` or `JdbcClient` calls on the same `DataSource` participate in one transaction; `JtaTransactionManager` coordinates several resources through a JTA provider. Boot configures `JpaTransactionManager` when JPA is on the class path, and `DataSourceTransactionManager` otherwise. With two `DataSource` instances, two managers exist, each transaction covers one `DataSource`, and `@Transactional("ordersTx")` selects the manager. A transaction across two databases is not provided without JTA; the alternatives are discussed in the part on microservices.

Propagation: physical and logical transactions

The propagation attribute defines what happens when a transactional method is called while a transaction already exists. The framework distinguishes a **physical** transaction (one connection, one JDBC `commit`) from **logical** transaction scopes (one per annotated method invocation). With `REQUIRED` several logical scopes share one physical transaction: an inner scope that decides to roll back cannot undo the others, so it marks the shared transaction as rollback-only, and the outermost scope then receives `UnexpectedRollbackException` when it attempts to commit.

Propagation	Behaviour	Typical use
`REQUIRED (default)`	Joins the existing transaction, or starts one	Almost all service methods
`REQUIRES_NEW`	Suspends the current transaction and starts an independent one with its own connection; commits or rolls back independently	Audit trail that must survive a rollback; work that must not share fate
`NESTED`	Savepoint inside the existing physical transaction (JDBC savepoints); rolls back to the savepoint on failure; behaves like REQUIRED without an outer transaction	Optional sub-step whose failure the caller handles
`SUPPORTS`	Joins a transaction if one exists, otherwise runs without	Read helpers
`NOT_SUPPORTED`	Suspends the current transaction and runs without one	Long non-transactional work called from a transactional caller
`MANDATORY`	Requires an existing transaction, otherwise `IllegalTransactionStateException`	Methods that must never be called alone
`NEVER`	Fails when a transaction exists	Assertions

Propagation behaviours.

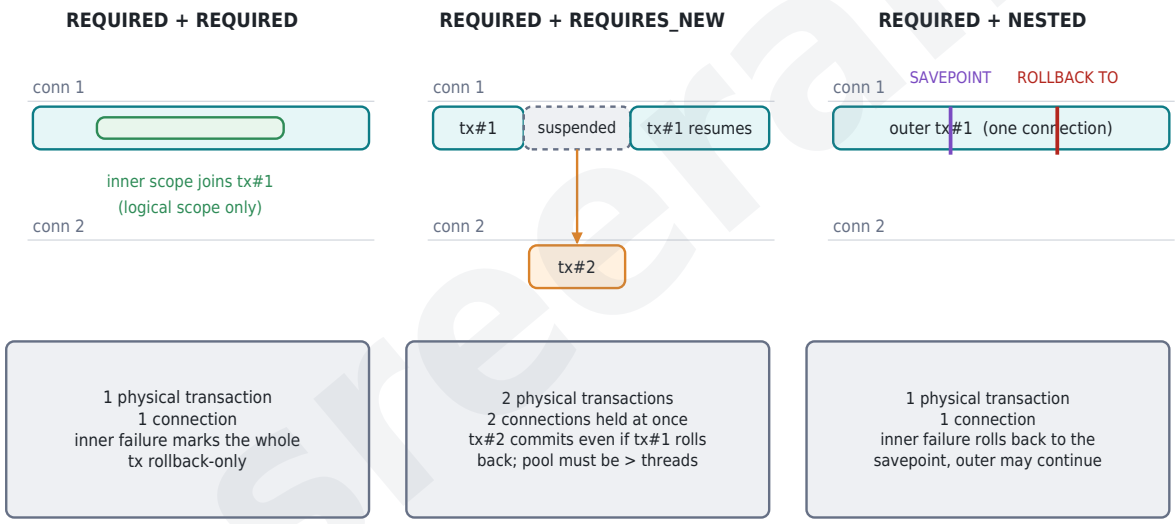


Figure 2.12. The transaction stack for REQUIRED, REQUIRES_NEW and NESTED: REQUIRES_NEW borrows a second pooled connection while the outer transaction keeps its own, NESTED stays on one connection and uses a savepoint.

`REQUIRES_NEW` has a cost that is easy to underestimate: the outer transaction keeps its connection while the inner transaction acquires another one. A thread therefore holds two connections at once. When all threads of a service do this simultaneously and the pool has the same number of connections as threads, every thread holds one connection and waits for a second one that never becomes available, which is a pool deadlock. The Spring documentation states that the pool must exceed the number of concurrent threads by at least one when `REQUIRES_NEW` is used. `NESTED` requires JDBC savepoint support and works with `JpaTransactionManager` through `HibernateJpaDialect`; it does not apply to JTA transactions.

The propagation stack in plain Java

The following program implements the decision logic of the interceptor with a thread-bound stack: the owner of a physical transaction pushes a `Tx`, a joiner reuses it, `NESTED` records a savepoint, and the default rollback rule distinguishes unchecked from checked exceptions. It does not use Spring; it reproduces the rules that `TransactionAspectSupport` applies.

TxSim.java JAVA

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.concurrent.Callable;

public class TxSim {
    enum Propagation { REQUIRED, REQUIRES_NEW, NESTED }

    static class UnexpectedRollbackException extends RuntimeException {
        UnexpectedRollbackException(String m) { super(m); }
    }

    static class AuditException extends Exception {
        AuditException(String m) { super(m); }
    }

    /** One physical transaction, i.e. one connection. */
    static class Tx {
        final int id;
        boolean rollbackOnly;
        Tx(int id) { this.id = id; }
    }

    static final Deque<Tx> STACK = new ArrayDeque<>();   // one thread only in this demo
    static int seq = 0;

    static void log(String s) { System.out.println("  ".repeat(STACK.size()) + s); }

    static <T> T execute(String name, Propagation p, Callable<T> body) throws Exception {
        Tx current = STACK.peek();
        boolean owner = current == null || p == Propagation.REQUIRES_NEW;
        boolean savepoint = !owner && p == Propagation.NESTED;
        Tx tx = owner ? new Tx(++seq) : current;
        if (owner) {
            String susp = current != null ? " (tx#" + current.id + " suspended)" : "";
            log(name + ": BEGIN tx#" + tx.id + susp);
            STACK.push(tx);
        } else {
            log(name + (savepoint ? ": SAVEPOINT in tx#" : ": joins tx#") + tx.id);
        }
        T result;
        try {
            result = body.call();
        } catch (RuntimeException | Error ex) {              // default rule: unchecked => rollback
            if (owner) {
                STACK.pop();
                log(name + ": ROLLBACK tx#" + tx.id + " (" + ex.getMessage() + ")");
            }
            else if (savepoint) log(name + ": ROLLBACK TO SAVEPOINT (" + ex.getMessage() + ")");
            else { tx.rollbackOnly = true; log(name + ": tx#" + tx.id + " marked rollback-only"); }
            throw ex;
        } catch (Exception ex) {                            // checked => commit anyway
            if (owner) {
                STACK.pop();
                log(name + ": COMMIT tx#" + tx.id + " despite " + ex.getMessage());
            }
            throw ex;
        }
        if (owner) {
            STACK.pop();
            if (tx.rollbackOnly) {
                log(name + ": ROLLBACK tx#" + tx.id + " (rollback-only)");
                throw new UnexpectedRollbackException("marked as rollback-only");
            }
            log(name + ": COMMIT tx#" + tx.id);
        }
        return result;
    }

    static Void fail(String m) { throw new IllegalStateException(m); }

    static void step(String title) { System.out.println("\n" + title); }

    public static void main(String[] args) throws Exception {
        step("A. REQUIRES_NEW commits independently of a failing outer transaction");
        try {
```

TxSim.java (continued) JAVA

```
            execute("placeOrder", Propagation.REQUIRED, () -> {
                execute("audit", Propagation.REQUIRES_NEW, () -> {
                    log("INSERT audit_log");
                    return null;
                });
                execute("reserve", Propagation.REQUIRED, () -> {
                    log("UPDATE stock");
                    return null;
                });
                return fail("payment declined");
            });
        } catch (RuntimeException e) { System.out.println("caller sees: " + e.getMessage()); }

        step("B. NESTED: failure rolls back to the savepoint only");
        execute("placeOrder", Propagation.REQUIRED, () -> {
            try {
                execute("loyalty", Propagation.NESTED, () -> fail("points service down"));
            }
            catch (RuntimeException e) { log("placeOrder caught: " + e.getMessage()); }
            return null;
        });

        step("C. Inner REQUIRED fails, outer catches: the transaction is poisoned");
        try {
            execute("placeOrder", Propagation.REQUIRED, () -> {
                try {
                    execute("notify", Propagation.REQUIRED, () -> fail("mail server down"));
                }
                catch (RuntimeException e) { log("placeOrder caught: " + e.getMessage()); }
                return null;
            });
        } catch (UnexpectedRollbackException e) {
            System.out.println("caller sees: " + e.getMessage());
        }

        step("D. Checked exception: the default rule commits");
        try {
            execute("importFile", Propagation.REQUIRED, () -> {
                throw new AuditException("bad row 17");
            });
        } catch (AuditException e) { System.out.println("caller sees: " + e.getMessage()); }
    }
}
```

Real output (JDK 21) TERMINAL

```
$ java TxSim.java
A. REQUIRES_NEW commits independently of a failing outer transaction
placeOrder: BEGIN tx#1
  audit: BEGIN tx#2 (tx#1 suspended)
    INSERT audit_log
  audit: COMMIT tx#2
  reserve: joins tx#1
  UPDATE stock
placeOrder: ROLLBACK tx#1 (payment declined)
caller sees: payment declined

B. NESTED: failure rolls back to the savepoint only
placeOrder: BEGIN tx#3
  loyalty: SAVEPOINT in tx#3
  loyalty: ROLLBACK TO SAVEPOINT (points service down)
  placeOrder caught: points service down
placeOrder: COMMIT tx#3

C. Inner REQUIRED fails, outer catches: the transaction is poisoned
placeOrder: BEGIN tx#4
  notify: joins tx#4
  notify: tx#4 marked rollback-only
  placeOrder caught: mail server down
placeOrder: ROLLBACK tx#4 (rollback-only)
caller sees: marked as rollback-only
```

Preview

Real output (JDK 21) (continued) TERMINAL

```
D. Checked exception: the default rule commits
importFile: BEGIN tx#5
importFile: COMMIT tx#5 despite bad row 17
caller sees: bad row 17
```

Four facts are visible in the output. In scenario A the audit row is committed in `tx#2` although `tx#1` rolls back. In B the failure is contained at the savepoint and the outer transaction commits. In C the outer method caught the exception and believed the situation handled, yet the transaction was already marked rollback-only by the inner joiner, so the commit became a rollback and the caller received `UnexpectedRollbackException` (in the simulation, the message differs from the framework's: `Transaction silently rolled back because it has been marked as rollback-only`). In D the checked exception does not roll back anything.

Rollback rules

The default rule of `TransactionAspectSupport` is: roll back on `RuntimeException` and `Error`; commit on checked exceptions (the exception is still propagated). The reasoning is the EJB tradition, in which a checked exception is a business outcome that the caller may handle after commit. The attributes `rollbackFor`, `rollbackForClassName`, `noRollbackFor` and `noRollbackForClassName` override the rule; the most specific matching rule wins. Since Spring 6.2 `@EnableTransactionManagement(rollbackOn = RollbackOn.ALL_EXCEPTIONS)` changes the default for the whole application. An exception thrown by a joined inner method sets the rollback-only flag at the moment it leaves the inner boundary, irrespective of what the outer method does with the exception afterwards.

A rollback applies only to the database. Changes to Java objects, to caches, to files, and messages already sent are not undone. Entities that were managed in the rolled-back persistence context are in an undefined state and must be discarded.

Isolation

Isolation defines which effects of concurrent transactions a transaction may observe. The SQL standard names three anomalies, and PostgreSQL adds the serialisation anomaly as the reason for its strictest level.

Level	Dirty read	Non-repeatable read, phantom	Notes
READ UNCOMMITTED	Not possible	Possible	PostgreSQL treats it as READ COMMITTED
READ COMMITTED (default)	Not possible	Possible	Each statement sees data committed before it started
REPEATABLE READ	Not possible	Not possible	One snapshot per transaction; a conflicting update fails with SQLSTATE 40001
SERIALIZABLE	Not possible	Not possible	Detects dangerous read-write dependencies; transactions fail with 40001 and must be retried

Isolation levels and the anomalies they permit (PostgreSQL behaviour).

`@Transactional(isolation = Isolation.REPEATABLE_READ)` applies only when the annotated method starts a new physical transaction (`REQUIRED` without an outer transaction, or `REQUIRES_NEW`); a joining method silently ignores it unless `validateExistingTransaction` is set on the manager. `Isolation.DEFAULT` leaves the database setting in force, which is READ COMMITTED for PostgreSQL (verified with `show default_transaction_isolation`). The following session executed in PostgreSQL shows what a failure at REPEATABLE READ looks like when another transaction updates the row that the first transaction later tries to update:

Two overlapping sessions (real output, abbreviated) TERMINAL

```
T1: begin isolation level repeatable read; select stock from product where id=1;
T1:  stock
T1: -------
T1:      7
T2: update product set stock=stock-1 where id=1          (autocommit, committed)
T1: update product set stock=stock-1 where id=1;
T1: ERROR:  could not serialize access due to concurrent update
```

Depending on the translation path (Hibernate's own conversion or Spring's SQLSTATE mapping) the failure appears as `CannotSerializeTransactionException`, another `ConcurrencyFailureException` subclass, or a `JpaSystemException`

wrapping the JDBC exception; the SQLSTATE `40001` is the reliable discriminator. The correct response is to repeat the whole transaction, which makes isolation levels above READ COMMITTED a design commitment to retry logic.

readOnly

`@Transactional(readOnly = true)` is a hint with real effects when the manager supports it. With `JpaTransactionManager` the `HibernateJpaDialect` sets the Hibernate session to `FlushMode.MANUAL`: the automatic flush and with it the dirty check of every managed entity are skipped. In addition `Connection.setReadOnly(true)` is applied; PostgreSQL then rejects writes, which was verified in psql:

Write inside a read-only transaction (real output) TERMINAL

```
BEGIN
ERROR:  cannot execute UPDATE in a read-only transaction
```

Two consequences follow. A read-only method that modifies an entity does not fail in Java but its change is never flushed, a silent loss that is worth a test. And `readOnly` does not make a method cheaper by itself in the database; its benefit lies in the skipped dirty check and in routing (a replica `DataSource` can be selected by the flag, see the chapter on performance). The attribute applies to new physical transactions only.

Programmatic transactions and synchronisation

`TransactionTemplate` (Boot registers a bean when exactly one `PlatformTransactionManager` exists) executes a callback inside a transaction without a proxy, so no self-invocation problem exists, and the boundary can be placed around a part of a method:

Fragment: TransactionTemplate JAVA

```java
Long id = tx.execute(status -> {            // TransactionCallback<Long>
    Order o = orders.save(new Order(...));
    if (o.getTotal().signum() < 0) {
        status.setRollbackOnly();           // programmatic rollback, no exception needed
    }
    return o.getId();
});
tx.executeWithoutResult(status -> audit.record("done"));   // no return value
```

Code that must react to the outcome registers a `TransactionSynchronization` through `TransactionSynchronizationManager.registerSynchronization`; the callbacks are `beforeCommit`, `beforeCompletion`, `afterCommit` and `afterCompletion(status)`. The annotation-based form is `@TransactionalEventListener(phase = AFTER_COMMIT)` on a listener method: the event published with `ApplicationEventPublisher` inside the transaction is delivered after the commit succeeded, so a mail or a message is never sent for data that was rolled back. The default phase is `AFTER_COMMIT`; a listener is not invoked at all when no transaction is active, unless `fallbackExecution = true`. The other phases are `BEFORE_COMMIT`, `AFTER_ROLLBACK` and `AFTER_COMPLETION`.

GOTCHA
In an `AFTER_COMMIT` callback the transaction has already been committed but its resources are still bound to the thread. Database access performed there joins the finished transaction and is never committed. Work that must be persisted from such a callback needs `@Transactional(propagation = Propagation.REQUIRES_NEW)` on the called method.

Deadlocks

A deadlock occurs when two transactions each hold a lock that the other needs. PostgreSQL detects the cycle after `deadlock_timeout` (1 second by default), aborts one transaction with SQLSTATE `40P01`, and lets the other proceed. The scenario below was reproduced with two sessions that update rows 1 and 2 of the same table in opposite order:

Deadlock between two sessions (real output) TERMINAL

```
S1: begin; update product set stock=stock+1 where id=1; select pg_sleep(1);
    update product set stock=stock+1 where id=2; commit;
S2: begin; update product set stock=stock+1 where id=2; select pg_sleep(1);
    update product set stock=stock+1 where id=1; commit;
S1: ERROR:  deadlock detected
S1: DETAIL:  Process 1123 waits for ShareLock on transaction 793; blocked by process 1126.
S1: Process 1126 waits for ShareLock on transaction 792; blocked by process 1123.
S1: HINT:  See server log for query details.
S1: CONTEXT:  while updating tuple (0,6) in relation "product"
```

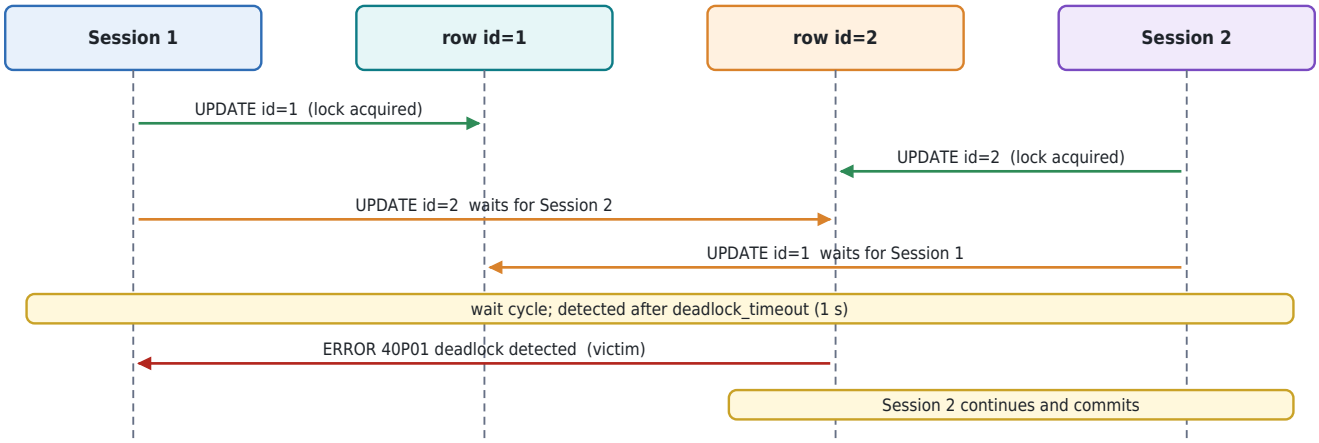


Figure 2.13. Opposite lock order produces a wait cycle; the database resolves it by aborting one transaction, which the application must be prepared to repeat.

Spring surfaces the failure as a subclass of `ConcurrencyFailureException` (`CannotAcquireLockException` or `PessimisticLockingFailureException`, depending on the path of translation). Prevention is structural: acquire locks in one global order (for example by ascending primary key), keep transactions short, and let the caller retry the entire transaction a bounded number of times with a jittered pause. Retrying inside the failed transaction is impossible, because the transaction is already aborted.

2.4.2 Worked example

A checkout use case shows the three recommended techniques together: short transactions around a remote call, an audit record that survives a rollback, and an after-commit notification. The orchestrating method is deliberately not annotated; it composes transactional units with `TransactionTemplate`, so the payment call happens while no connection is held.

CheckoutService.java JAVA

```java
package com.example.shop.service;

import com.example.shop.domain.PurchaseOrder;
import com.example.shop.repo.CustomerRepository;
import com.example.shop.repo.PurchaseOrderRepository;
import java.math.BigDecimal;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Service;
import org.springframework.transaction.support.TransactionTemplate;

@Service
public class CheckoutService {

    public record OrderPaid(long orderId) { }

    private final TransactionTemplate tx;
    private final CustomerRepository customers;
    private final PurchaseOrderRepository orders;
    private final PaymentGateway gateway;
    private final AuditService audit;
    private final ApplicationEventPublisher events;

    public CheckoutService(TransactionTemplate tx, CustomerRepository customers,
                           PurchaseOrderRepository orders, PaymentGateway gateway,
                           AuditService audit, ApplicationEventPublisher events) {
        this.tx = tx;
        this.customers = customers;
        this.orders = orders;
        this.gateway = gateway;
        this.audit = audit;
        this.events = events;
    }

    /** Not transactional: the remote call must not run while a connection is held. */
    public long checkout(long customerId, String sku, int qty, BigDecimal amount) {
        // unit 1: short transaction, connection returned to the pool at commit
        Long orderId = tx.execute(status -> {
```

CheckoutService.java (continued) JAVA

```
            PurchaseOrder order = new PurchaseOrder(customers.getReferenceById(customerId));
            order.addLine(sku, qty);
            return orders.save(order).getId();
        });

        // remote call: no transaction, no connection
        boolean approved = gateway.charge(orderId, amount);

        // unit 2: short transaction recording the outcome
        tx.executeWithoutResult(status -> {
            PurchaseOrder order = orders.findById(orderId).orElseThrow();
            if (approved) {
                order.markPaid();
                events.publishEvent(new OrderPaid(orderId));      // delivered after commit
            } else {
                order.cancel();
            }
            audit.record(orderId, approved ? "PAID" : "DECLINED"); // REQUIRES_NEW
        });
        return orderId;
    }
}
```

AuditService.java JAVA

```
package com.example.shop.service;

import com.example.shop.domain.AuditEntry;
import com.example.shop.repo.AuditEntryRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Service
public class AuditService {

    private final AuditEntryRepository entries;

    public AuditService(AuditEntryRepository entries) { this.entries = entries; }

    /** Independent transaction: the entry is committed even if the caller rolls back. */
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void record(long orderId, String event) {
        entries.save(new AuditEntry(orderId, event));
    }
}
```

Because `CheckoutService` calls `AuditService` through an injected reference, the call passes through the proxy and the `REQUIRES_NEW` attribute is honoured. The same method placed in `CheckoutService` and called as `this.record(...)` would run inside the existing transaction without any new connection. The notification is delivered by a listener bound to the commit:

ReceiptListener.java JAVA

```
package com.example.shop.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.transaction.event.TransactionPhase;
import org.springframework.transaction.event.TransactionalEventListener;

@Component
class ReceiptListener {

    private static final Logger log = LoggerFactory.getLogger(ReceiptListener.class);

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void onPaid(CheckoutService.OrderPaid event) {
        log.info("sending receipt for order {}", event.orderId());   // no database work here
    }
}
```

A bounded retry for concurrency failures wraps the whole transactional unit. It sits outside the `TransactionTemplate` call, because a failed transaction cannot be continued:

Retry.java JAVA

```java
package com.example.shop.support;

import java.util.concurrent.ThreadLocalRandom;
import java.util.function.Supplier;
import org.springframework.dao.ConcurrencyFailureException;

public final class Retry {

    private Retry() { }

    public static <T> T onConcurrencyFailure(int maxAttempts, Supplier<T> unitOfWork) {
        for (int attempt = 1; ; attempt++) {
            try {
                return unitOfWork.get();
            } catch (ConcurrencyFailureException e) {   // deadlock, serialisation, optimistic lock
                if (attempt >= maxAttempts) throw e;
                long backoffMs = (20L << attempt) + ThreadLocalRandom.current().nextLong(20);
                try {
                    Thread.sleep(backoffMs);
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                    throw e;
                }
            }
        }
    }
}
```

Usage: `Retry.onConcurrencyFailure(3, () -> tx.execute(s -> transfer(a, b, amount)))`. `ObjectOptimisticLockingFailureException` and the deadlock and serialisation exceptions are all subclasses of `ConcurrencyFailureException`, so one catch clause covers the three situations. The operation must be repeatable: it reloads its data in every attempt and has no side effects outside the transaction.

2.4.3 Hands-on

Transaction activity becomes visible with two logging categories:

application-dev.yml YAML

```yaml
logging:
  level:
    org.springframework.orm.jpa.JpaTransactionManager: DEBUG
    org.springframework.transaction.interceptor: TRACE
spring:
  transaction:
    default-timeout: 30s
```

The key `spring.transaction.default-timeout` sets a timeout for every transaction that does not declare its own; a transaction that exceeds it fails with `TransactionTimedOutException` at the next database access. Calling `checkout` produces log lines of the following kind (typical output, not run on the build machine):

Typical transaction log of checkout() (not run on the build machine) TERMINAL

```
Creating new transaction with name [null]: PROPAGATION_REQUIRED,ISOLATION_DEFAULT
Opened new EntityManager [SessionImpl(1022311872<open>)] for JPA transaction
Initiating transaction commit
Committing JPA transaction on EntityManager [SessionImpl(1022311872<open>)]
Closing JPA EntityManager [SessionImpl(1022311872<open>)] after transaction
Creating new transaction with name [null]: PROPAGATION_REQUIRED,ISOLATION_DEFAULT
Suspending current transaction, creating new transaction with name [..AuditService.record]
Committing JPA transaction on EntityManager [SessionImpl(1964052113<open>)]
Resuming suspended transaction after completion of inner transaction
Committing JPA transaction on EntityManager [SessionImpl(1022311872<open>)]
```

Two distinct `SessionImpl` identities are visible for the audit write and its caller, which is the evidence that `REQUIRES_NEW` used a separate session and connection. The name of a `TransactionTemplate` transaction is `null` because it has no method name.

sreeram

2.4.4 Real-world problems and solutions

Problem: a checked exception does not roll back the transaction

Scenario. `importOrders(file)` is annotated `@Transactional` and throws a checked `ImportException` when a row is invalid, after having saved the preceding rows. The caller reports an error, yet the saved rows remain in the database.

Symptoms. An exception reaches the caller, no rollback message is logged, and the rows exist after the call. At `TRACE` level the interceptor logs `Application exception overridden by commit` or `Completing transaction ... after exception`.

Diagnosis. (1) Enable `org.springframework.transaction.interceptor=TRACE` and read whether the interceptor decided to commit or to roll back. (2) Check whether the thrown type extends `RuntimeException` or `Error`. (3) Check `@Transactional(rollbackFor = ...)` on the method.

Root cause. The default rollback rule excludes checked exceptions (simulation scenario D).

Fix. `@Transactional(rollbackFor = ImportException.class)` or, for a whole code base, `rollbackFor = Exception.class` as a convention; in applications that adopt Spring 6.2 or later the global switch `@EnableTransactionManagement(rollbackOn = RollbackOn.ALL_EXCEPTIONS)` removes the exception from the rule. Alternatively make the exception unchecked.

Prevention. A test that triggers the failure and asserts that the row count did not change; a team convention that every domain exception either extends `RuntimeException` or is named in `rollbackFor`.

Problem: UnexpectedRollbackException after a handled exception

Scenario. `placeOrder` catches an exception from `notifyCustomer`, logs it and carries on. At the end the caller receives `UnexpectedRollbackException: Transaction silently rolled back because it has been marked as rollback-only`, and the order is not saved.

Symptoms. The exception is raised at the commit of the outer method, not at the failing call, which hides its origin. The log of the inner method shows an exception that appeared to be handled.

Diagnosis. (1) Find the first exception in the log that left a `@Transactional` boundary: with `JpaTransactionManager=DEBUG` the line `Participating transaction failed - marking existing transaction as rollback-only` identifies the inner method. (2) Check the propagation of that inner method (REQUIRED joins the outer transaction).

Root cause. The inner proxy marked the shared physical transaction as rollback-only when the runtime exception passed its boundary (simulation scenario C).

Fix. Choose one: remove `@Transactional` from the inner method when it needs no boundary of its own; declare it `REQUIRES_NEW` when its outcome must be independent; declare it `NESTED` to roll back to a savepoint only; or let the exception propagate and accept the rollback. Use `noRollbackFor` only when the exception is truly harmless to the data.

Prevention. The rule: catching an exception that crossed a transactional boundary never repairs the transaction. Review every `catch` around a call to another transactional bean.

Problem: connection pool exhaustion from long transactions

Scenario. A method annotated `@Transactional` saves an order, calls a payment provider (100 ms on average, 2 s during incidents) and saves the result. With a pool of 10 connections, throughput falls to a fraction of capacity and requests fail with `Connection is not available, request timed out after 30000ms`.

Symptoms. `hikaricp_connections_active` equals the maximum, `hikaricp_connections_pending` is positive, and `hikaricp_connections_usage_seconds` (time a connection is held) is far longer than the time spent in SQL. Database CPU is low; `pg_stat_activity` shows many connections in state `idle in transaction`.

Diagnosis. (1) `select pid, state, now() - xact_start as age, left(query, 60) from pg_stat_activity where state = 'idle in transaction' order by age desc;` lists transactions that hold a connection while the application does something else. (2) A thread dump (`jcmd <pid> Thread.print`) shows the owning threads in the HTTP client. (3) `spring.datasource.hikari.leak-detection-threshold` logs the acquiring stack of long-held connections.

Root cause. The remote call executes inside the transaction. By Little's law the number of connections in use equals the arrival rate multiplied by the holding time; extending the holding time by the remote latency divides the achievable rate by the same factor. The effect was measured with a plain-JDK pool of 10 connections, 50 concurrent clients and 20 requests each, in which each request needs 10 ms of database work and one 100 ms remote call:

PoolSim.java JAVA

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class PoolSim {
    static final int POOL = 10, CLIENTS = 50, REQUESTS_PER_CLIENT = 20, TIMEOUT_MS = 300;

    static void run(String label, int dbMs, int remoteMs, boolean remoteInsideTx) throws Exception {
        Semaphore pool = new Semaphore(POOL, true);
        List<Long> waits = Collections.synchronizedList(new ArrayList<>());
        AtomicInteger timeouts = new AtomicInteger(), done = new AtomicInteger();
        long t0 = System.nanoTime();
        List<Thread> ts = new ArrayList<>();
        for (int c = 0; c < CLIENTS; c++) {
            ts.add(Thread.ofPlatform().start(() -> {
                for (int i = 0; i < REQUESTS_PER_CLIENT; i++) {
                    long w0 = System.nanoTime();
                    try {
                        if (!pool.tryAcquire(TIMEOUT_MS, TimeUnit.MILLISECONDS)) {
                            timeouts.incrementAndGet();
                            continue;
                        }
                        waits.add((System.nanoTime() - w0) / 1_000_000);
                        try {
                            Thread.sleep(dbMs);                      // SQL work
                            if (remoteInsideTx) Thread.sleep(remoteMs);   // still holding
                        } finally { pool.release(); }
                        if (!remoteInsideTx) Thread.sleep(remoteMs);   // after release
                        done.incrementAndGet();
                    } catch (InterruptedException e) { return; }
                }
            }));
        }
        for (Thread t : ts) t.join();
        double secs = (System.nanoTime() - t0) / 1e9;
        Collections.sort(waits);
        long p95 = waits.isEmpty() ? -1 : waits.get((int) (waits.size() * 0.95) - 1);
        System.out.printf("%-34s done=%4d timeouts=%4d  req/s=%6.1f  wait p95=%4d ms%n",
                label, done.get(), timeouts.get(), done.get() / secs, p95);
    }

    public static void main(String[] args) throws Exception {
        System.out.println("pool=" + POOL + " clients=" + CLIENTS
                + " acquire timeout=" + TIMEOUT_MS + " ms");
        run("short tx (10 ms SQL), remote after", 10, 100, false);
        run("long tx (10 ms SQL + remote 100 ms)", 10, 100, true);
    }
}
```

Real output (JDK 21; measured on the build machine, typical) TERMINAL

```
$ java PoolSim.java
pool=10 clients=50 acquire timeout=300 ms
short tx (10 ms SQL), remote after done=1000 timeouts=   0  req/s= 442.1  wait p95=   0 ms
long tx (10 ms SQL + remote 100 ms) done= 600 timeouts= 400  req/s=  90.5  wait p95= 283 ms
```

When the remote call stays inside the transaction, each request holds a connection for about 110 ms, so 10 connections serve about 10 / 0.110 ≈ 90 requests per second (measured: 90.5), 40 percent of the requests wait longer than the acquire timeout (here 300 ms instead of 30 s, to keep the demonstration short) and fail. When the remote call runs after the connection was released, a request holds the connection for 10 ms, the pool serves about 1,000 requests per second in theory, and the 50 clients, which also spend 100 ms in the remote call, reach 442 requests per second without a single timeout.

Fix. Split the unit of work as in `CheckoutService`: transaction, remote call, transaction. Where atomicity across the remote call is required, use compensation (see the part on microservices) instead of holding a connection. Do not raise the pool size as the first response; it moves the saturation point but also increases load on the database.

Prevention. A review rule: no network call, file operation or `Thread.sleep` inside `@Transactional`. A `transaction timeout` of a few seconds turns a stuck provider into a fast failure.

Problem: intermittent deadlock under load

Scenario. A transfer between two accounts locks the source row and then the target row. Under load, 0.5 percent of transfers fail with `deadlock detected` and users see HTTP 500.

Symptoms. PostgreSQL log entries `ERROR: deadlock detected` with two process ids and the statements; the incidence rises with concurrency. The failures disappear when only one request is sent.

Diagnosis. (1) Read the `DETAIL` lines in the PostgreSQL log: they name the two statements and the relation. (2) Identify the two code paths that lock the same rows in opposite order (`transfer(a, b)` and `transfer(b, a)` run concurrently). (3) Optionally enable `log_lock_waits = on` to see waits that exceed `deadlock_timeout`.

Root cause. Opposite lock order (diagram above). The database aborts one transaction; the application reports it as an error.

Fix. Lock rows in a canonical order: `select ... where id in (?, ?) order by id for update` (or `@Lock(PESSIMISTIC_WRITE)` on a query ordered by id) before applying the changes, and wrap the unit in `Retry.onConcurrencyFailure(3, ...)` as a second line of defence.

Prevention. Document the lock order for each aggregate; load tests with concurrent opposite operations.

2.4.5 Questions to expect

1. **How does `@Transactional` work internally?** A proxy (CGLIB or JDK) around the bean routes calls through `TransactionInterceptor`, which asks the `PlatformTransactionManager` for a transaction, binds the resources (Session, Connection) to the thread, invokes the method, and then commits or rolls back according to the outcome and the rollback rules.
2. **Why does `@Transactional` have no effect on a self-invoked method?** The call does not pass through the proxy, so the interceptor never runs. The remedies are to move the method to another bean, to use `TransactionTemplate`, or to use AspectJ mode.
3. **Which exceptions cause a rollback by default?** `RuntimeException` and `Error`. Checked exceptions commit unless `rollbackFor` is declared.
4. **Explain REQUIRED, REQUIRES_NEW and NESTED.** REQUIRED joins or starts. REQUIRES_NEW suspends the current transaction and starts an independent one on a second connection. NESTED uses a savepoint in the same transaction and needs JDBC savepoint support.
5. **What is `UnexpectedRollbackException`?** The outer scope tried to commit a transaction that an inner joined scope had marked rollback-only. It tells the caller that the commit did not happen.
6. **What does `readOnly = true` do?** It marks the transaction read only, sets the Hibernate session to manual flush (no dirty checking) and sets the JDBC connection read only, so PostgreSQL rejects writes. It applies to new transactions only and can drive replica routing.
7. **Which isolation level does PostgreSQL use by default and which anomalies remain?** READ COMMITTED; non-repeatable reads, phantoms and lost updates are possible. REPEATABLE READ and SERIALIZABLE fail with SQLSTATE 40001 on conflict, which requires retries.
8. **What is the difference between `@TransactionalEventListener` and `@EventListener`?** The former is bound to a transaction phase (after commit by default) and is not invoked when no transaction runs; the latter is invoked immediately and synchronously.
9. **How are deadlocks avoided?** Acquire locks in a fixed order, keep transactions short, and retry on the concurrency exception with backoff.
10. **Why should a remote call not be made inside a transaction?** The connection is held for the call duration; pool capacity equals pool size divided by holding time, so latency of the remote system becomes a throughput limit and a source of pool exhaustion.

KEY POINTS

A transaction is a proxy, an interceptor and a thread-bound Session and Connection; only calls entering through the proxy are transactional.

- REQUIRED shares one physical transaction; an exception crossing an inner boundary marks it rollback-only even if the outer method catches it.
- Rollback by default only for unchecked exceptions and errors; declare `rollbackFor` for checked exceptions.
- REQUIRES_NEW uses a second connection; size the pool accordingly and use it sparingly. NESTED uses savepoints.
- `readOnly = true` skips dirty checking and makes the connection read only; isolation above READ COMMITTED requires retry logic.
- Keep transactions short: no remote calls, no sleeps; split the unit of work around slow operations and use after-commit events for side effects.
- Retry concurrency failures outside the transaction with bounded attempts, backoff and jitter; lock rows in a canonical order.

2.5 Performance of the Data Layer

The preceding chapters reduced the number of statements per use case. This chapter addresses the remaining cost centres of a data layer: the connection pool, the transfer of many rows, the repeated reading of unchanged data, pagination over large tables, the distribution of reads over several databases, and the evolution of the schema. Each topic is treated with the mechanism, a configuration or code listing, and, where the machine allows it, a measurement from PostgreSQL 16 on the build machine (2 CPUs, shared with other workloads; the numbers show orders of magnitude and are labelled typical).

2.5.1 Concept and mechanism

Connection pool: HikariCP

A JDBC connection is expensive to create (TCP and TLS handshake, authentication, backend process start in PostgreSQL), so Spring Boot uses a pool: HikariCP is selected automatically when it is on the class path, which is the case with `spring-boot-starter-data-jpa` and `spring-boot-starter-jdbc`. The pool is configured with the prefix `spring.datasource.hikari`.

Property	Default	Meaning and advice
`maximum-pool-size`	10	Maximum number of connections, idle plus in use. Size by measurement (see below), not by the number of request threads.
`minimum-idle`	same as maximum	Idle connections kept ready. Leaving the default yields a fixed-size pool, which HikariCP recommends for predictable behaviour.
`connection-timeout`	30000 ms	Maximum wait for a connection from the pool; after that a `SQLTransientConnectionException` is thrown (`Connection is not available, request timed out after 30000ms`). Values above a few seconds hide overload.
`max-lifetime`	1800000 ms	Maximum age of a connection; it is retired when idle after this age. Must be shorter than any limit imposed by the database, proxy or firewall.
`idle-timeout`	600000 ms	Applies only when `minimum-idle` is lower than `maximum-pool-size`.
`keepalive-time`	0 (off)	Interval of keep-alive probes on idle connections; useful behind devices that silently drop idle TCP connections.
`leak-detection-threshold`	0 (off)	Logs a stack trace when a connection is held longer than the value; a diagnostic tool for long transactions.

Main HikariCP settings and their defaults.

Sizing. A database can execute only as many statements in parallel as it has CPU cores and disk parallelism; additional concurrent statements wait inside the database and increase latency without increasing throughput. A rule of thumb from the PostgreSQL community, repeated in the HikariCP documentation on pool sizing, gives the order of magnitude: `connections = (cores x 2) + effective disk spindles`, which for typical servers gives pools of 10 to 30

connections. The effect was measured with `pgbench` (select-only workload, `-S`) against the PostgreSQL instance of the build machine, which has 2 cores:

pgbench -S, 2 cores, varying client count (measured, typical, shared machine) TERMINAL

```
$ pgbench -S -c N -j 2 -T 4 postgres
clients=1   tps=34170   avg latency ms=0.029
clients=2   tps=41774   avg latency ms=0.048
clients=4   tps=31222   avg latency ms=0.128
clients=8   tps=37228   avg latency ms=0.215
clients=16  tps=37444   avg latency ms=0.427
clients=32  tps=29190   avg latency ms=1.096
clients=64  tps=28581   avg latency ms=2.239
```

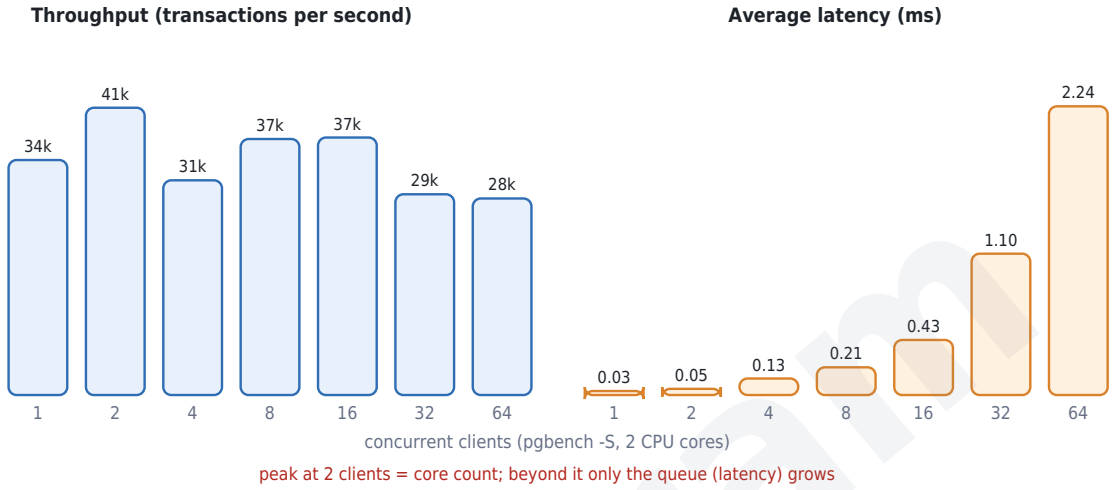


Figure 2.14. Throughput stops growing at about the core count while latency keeps rising with the number of concurrent clients; a pool larger than the database can serve moves the queue from the pool into the database.

Throughput peaked at 2 clients (equal to the number of cores) and fell afterwards, while the average latency at 64 clients was about 75 times that at one client. The measurement is noisy because the machine is shared; the shape, not the individual values, is the evidence. Two further constraints apply. The total of all instances must fit into the server limit: ten application instances with a pool of 10 consume 100 connections, which is the default value of PostgreSQL's `max_connections`. And Little's law relates the quantities: the number of connections in use equals the request rate multiplied by the time a connection is held (see the previous chapter), so shortening the holding time is more effective than enlarging the pool. When many instances need more connections than the server can afford, an external pooler (PgBouncer in transaction mode) is placed in front; in that case server-side prepared statements must be disabled in the driver (`prepareThreshold=0`).

HikariCP publishes the metrics `hikaricp.connections.active`, `.idle`, `.pending`, `.max`, `.acquire` and `.usage` to Micrometer; `pending > 0` for a sustained period is the alert condition for pool saturation.

Batch writes

Inserting 5,000 rows one statement at a time costs 5,000 network round trips and 5,000 statement executions. JDBC batching groups the parameter sets of one prepared statement into one request. Hibernate uses it when `hibernate.jdbc.batch_size` is positive, and only when the inserts are deferred, which excludes `IDENTITY` ids:

application.yml (batching) YAML

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/shop?reWriteBatchedInserts=true
  jpa:
    properties:
      hibernate:
        jdbc:
          batch_size: 50
          batch_versioned_data: true     # allow batching of entities with @Version
        order_inserts: true              # group inserts of the same table
        order_updates: true
        generate_statistics: true        # reports "executing N JDBC batches"
```

`order_inserts` and `order_updates` sort the actions by entity type so that interleaved inserts into two tables still form two batches. With the PostgreSQL driver, `reWriteBatchedInserts=true` rewrites a batch of single-row inserts into multi-row statements (`insert ... values (...),(...),(...)`), which reduces the work further. The relative effect was measured in psql by sending the same 5,000 rows as 5,000 single-row statements and as 100 statements of 50 rows each, inside one transaction on the local socket:

Single-row inserts versus 50-row inserts (measured, typical) TERMINAL

```
single ms: 347     (5,000 statements)
multi  ms: 89      (100 statements of 50 rows)
```

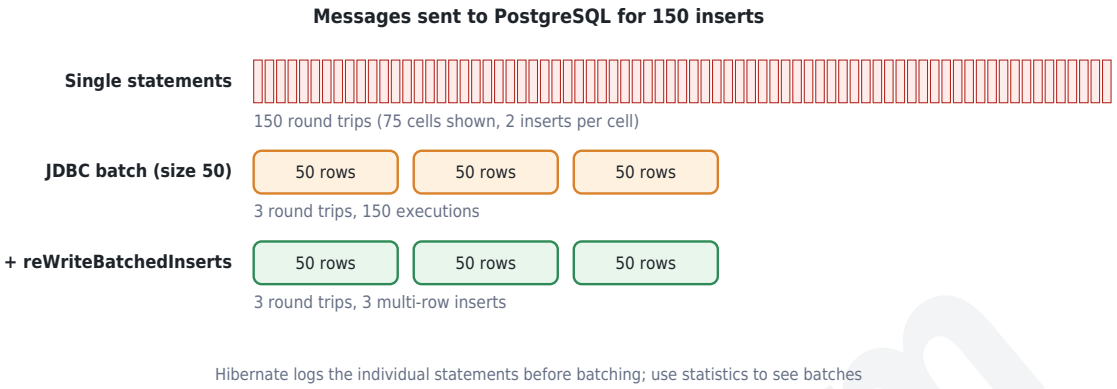


Figure 2.15. Round trips for 150 inserts: 150 single statements, three JDBC batches of 50, or three multi-row statements after reWriteBatchedInserts; the batches are visible only in statistics or in a proxying DataSource, not in the Hibernate SQL log.

The measurement used a local socket; on a network with 1 ms latency the single-row variant pays roughly another 5 seconds (5,000 round trips of 1 ms). A second hazard is memory: every persisted entity stays in the persistence context together with its snapshot. A loop that persists 100,000 entities in one transaction must call `flush()` and `clear()` every batch size, or split the work into transactions of a few hundred rows. For pure bulk loads that need no entity semantics, `JdbcClient`/`JdbcTemplate.batchUpdate`, Hibernate's `StatelessSession` (no persistence context, no dirty checking) or the PostgreSQL `COPY` command are faster than entities.

Bulk updates and deletes (`@Modifying @Query("update ...")`) are one SQL statement for any number of rows. They bypass the persistence context, entity listeners (auditing), the `@Version` increment (unless `update versioned` is written) and the second-level cache. Entities loaded before the statement are stale afterwards; `@Modifying(clearAutomatically = true)` clears the persistence context after execution.

Caching in the data layer

Four caches exist between the application and the table, with different scope and risk.

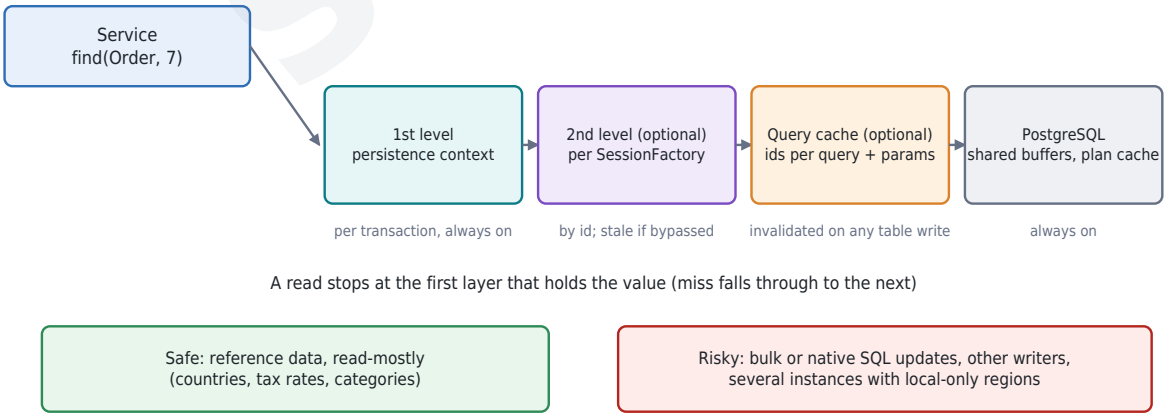


Figure 2.16. Cache layers of a read: the persistence context (per transaction), the optional second-level cache (per SessionFactory, shared by all sessions of one JVM), the optional query cache, and the PostgreSQL buffer cache; only the first is always active.

1. **Persistence context (first level).** Always on, per transaction, discussed in the first chapter of this Part.
2. **Second-level cache.** Optional, per `SessionFactory`, keyed by entity class and identifier, storing the dehydrated attribute values. It is activated with `hibernate.cache.use_second_level_cache=true` and a region factory (typically JCache with Ehcache or Caffeine through `hibernate-jcache`), and per entity with `@Cache(usage =`

`CacheConcurrencyStrategy.READ_WRITE)` together with `jakarta.persistence.Cacheable`. It answers `find` and lazy loads of single entities and collections; it does not answer arbitrary queries.

3. **Query cache.** Stores the identifiers returned by a query and the parameters, enabled with `hibernate.cache.use_query_cache` and per query. It is invalidated whenever any table of the query is written, so it helps only for read-mostly tables and rarely pays for itself.
4. **Statement and plan caches.** The PostgreSQL driver switches a statement to a server-side prepared statement after `prepareThreshold` executions (default 5) and caches up to `preparedStatementCacheQueries` (default 256) statements per connection. Hibernate caches compiled HQL plans (`hibernate.query.plan_cache_max_size`, default 2048). A query with a variable-length `in (...)` list produces one distinct SQL text per list length; `hibernate.query.in_clause_parameter_padding=true` pads the list to powers of two and reduces the number of distinct statements.

The second-level cache is correct only when all writes go through Hibernate in a cluster-aware setup. Direct SQL updates, bulk JPQL, other applications and (with a local-only region factory) other instances of the same service change rows without invalidating the cache, and users then read stale data. It suits reference data (countries, product categories, tax rates) that changes rarely. Business data with frequent changes, or service instances that scale horizontally with a local cache, are better served by the application-level cache (Spring Cache with an explicit eviction policy, covered in the part on delivery) or by the database itself.

Keyset pagination

Offset pagination (`limit 20 offset 19000`) is simple, but the database must produce and discard 19,000 rows before it returns the page, so the cost of page N grows with N. Keyset pagination (also called seek pagination) replaces the offset by a predicate on the sort key of the last row of the previous page, which an index resolves directly. The plans were captured on the 20,000-row `purchase_order` table with an index on `(created_at, id)`:

Offset and keyset form of the same page SQL

```sql
-- offset: page 951
select id, status from purchase_order order by created_at desc, id limit 20 offset 19000;
-- keyset: rows after the last row (created_at, id) of the previous page
select id, status from purchase_order
 where (created_at, id) < (now() - interval '19000 minutes', 19000)
 order by created_at desc, id limit 20;
```

EXPLAIN (ANALYZE) of both forms, abbreviated (real output, PostgreSQL 16) TERMINAL

```
Offset form:
 Limit (actual rows=20 loops=1)
   ->  Incremental Sort (actual rows=19020 loops=1)
         ->  Index Scan Backward using purchase_order_created_at_id_idx (actual rows=19021 loops=1)
 Execution Time: 4.855 ms
Keyset form:
 Limit (actual rows=20 loops=1)
   ->  Incremental Sort (actual rows=20 loops=1)
         ->  Index Scan Backward using purchase_order_created_at_id_idx (actual rows=21 loops=1)
               Index Cond: (ROW(created_at, id) < ROW((now() - '316:40:00'::interval), 19000))
 Execution Time: 0.056 ms
```

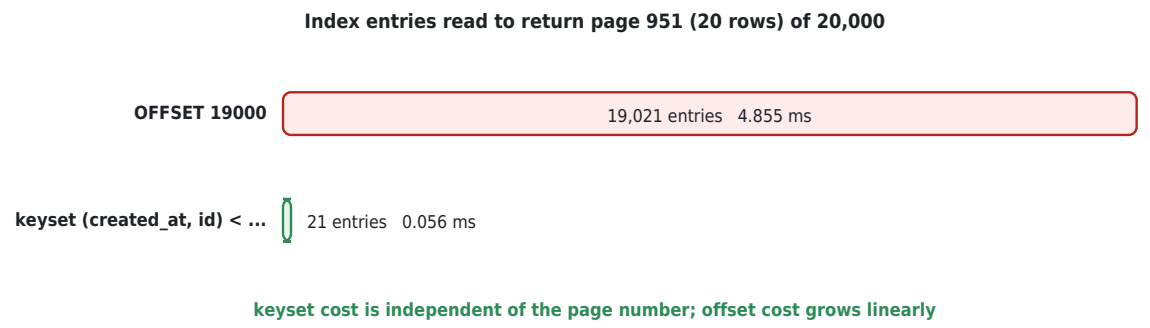


Figure 2.17. Rows read from the index to serve page 951: the offset form reads 19,021 index entries, the keyset form reads 21 and its cost does not depend on the page number.

The offset form read 19,021 index entries and the keyset form 21; the execution time was 4.9 ms against 0.06 ms on a table that fits in memory, and the gap widens with table size. The predicate `(created_at, id) < (...)` is a row-value comparison that PostgreSQL maps to the composite index; the id breaks ties between equal timestamps, which is why

the sort key must be unique. Limits of the technique: pages cannot be addressed by number (only next and previous), the sort order must be fixed and consistent with an index, and nullable sort columns lead to wrong results. Spring Data supports it from version 3.1 with `Window<T>` and `ScrollPosition.keyset()`, available for derived query methods but not for `@Query` strings; `Page` offers a total count, which costs one `count(*)` per page, and `Slice` only the information whether a next page exists.

Read replicas and routing

Reads can be served by a streaming replica of the primary, which increases read capacity. The application selects the target per transaction. Spring provides `AbstractRoutingDataSource`, whose `determineCurrentLookupKey()` is consulted when a connection is requested. The read-only flag of the current transaction is the natural key, but it is set after `JpaTransactionManager` has begun the transaction and obtained a connection. A `LazyConnectionDataSourceProxy` in front of the routing data source postpones the physical connection until the first statement, at which time the flag is available:

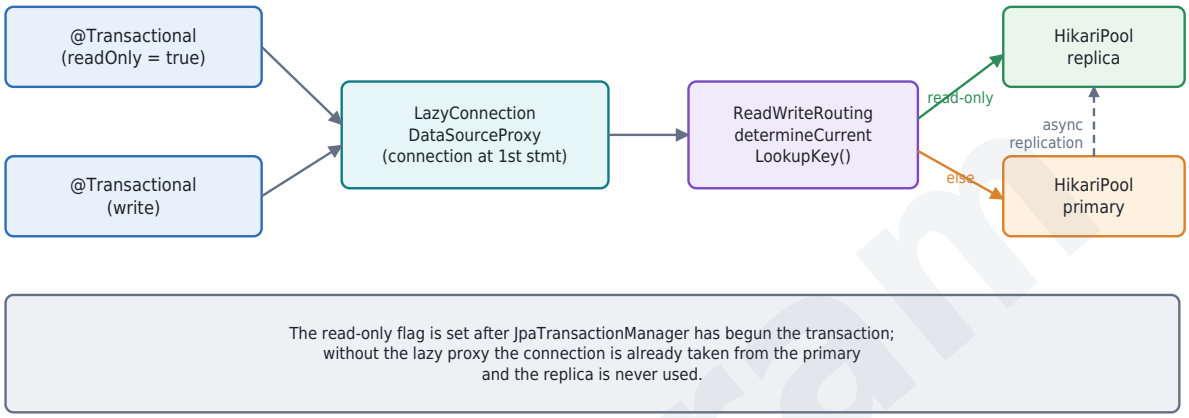


Figure 2.18. Routing by transaction flag: the lazy proxy defers connection acquisition until the first statement so that the read-only flag set during transaction begin can choose between primary and replica.

Replication is asynchronous, so a read that follows a write of the same user may not see it (read-your-writes violation); sessions that just wrote, and all methods that read data before deciding on a write, must use the primary. Replica lag must be monitored, and a replica failure needs a fallback to the primary.

Schema migrations

Hibernate can create the schema (`spring.jpa.hibernate.ddl-auto=create|update`), but a production system needs versioned, reviewable and repeatable changes. Flyway and Liquibase provide them, and Spring Boot runs them before the `EntityManagerFactory` is created, so Hibernate always sees the final schema. The Flyway convention is a file `V<version>__<description>.sql` in `classpath:db/migration`; the applied files are recorded with a checksum in the table `flyway_schema_history`, and a changed checksum aborts the start-up. Since Flyway 10 a database-specific module is required in addition to `flyway-core` (`org.flywaydb:flyway-database-postgresql`), as stated in the Spring Boot documentation. The Liquibase default master file is `db/changelog/db.changelog-master.yaml`.

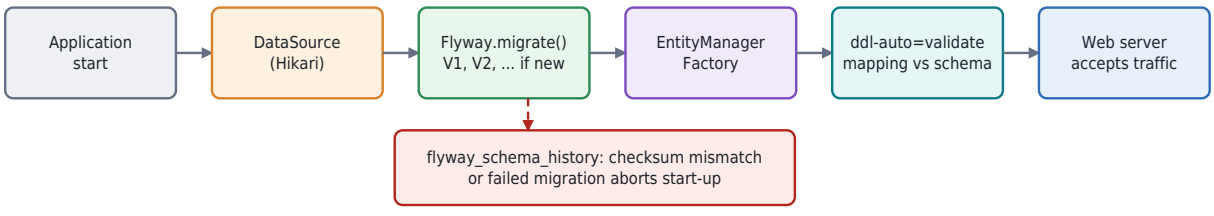


Figure 2.19. Start-up order with Flyway: migrations complete before the EntityManagerFactory is created, and Hibernate validates the mapping against the migrated schema.

A production-suitable configuration disables schema generation and validates the mapping:

application.yml (migrations) YAML

```yaml
spring:
  jpa:
    hibernate:
      ddl-auto: validate          # fail at start-up when entity and schema disagree
  flyway:
    locations: classpath:db/migration
    user: ${MIGRATION_USER}       # separate account with DDL rights
    password: ${MIGRATION_PASSWORD}   # the application user itself has DML rights only
```

Rules for safe migrations: a released migration is never edited (add a new one); every change must be compatible with the previous application version when rolling updates run old and new instances side by side; destructive changes are split in phases (**expand and contract**): add the new column and write to both, backfill, switch reads, and drop the old column in a later release. A column rename done in one step breaks the instances that are still running the old version. `ddl-auto=validate` also detects the sequence increment mismatch described in the first chapter when the mapping declares `allocationSize = 50`.

2.5.2 Worked example

Chunked batch import

The import service persists products in chunks of 500 rows, one transaction per chunk, with a flush and clear every 50 entities so that the persistence context stays small and the JDBC batches have the configured size. The `Product` entity uses a sequence generator with `allocationSize = 50` (not `IDENTITY`).

ProductImportService.java JAVA

```java
package com.example.shop.service;

import com.example.shop.domain.Product;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.support.TransactionTemplate;

@Service
public class ProductImportService {

    public record Row(String sku, String name) { }

    private static final int JDBC_BATCH = 50;       // equals hibernate.jdbc.batch_size
    private static final int CHUNK = 500;           // rows per transaction

    @PersistenceContext
    private EntityManager em;

    private final TransactionTemplate tx;

    public ProductImportService(TransactionTemplate tx) { this.tx = tx; }

    public int importAll(List<Row> rows) {
        int done = 0;
        for (int from = 0; from < rows.size(); from += CHUNK) {
            List<Row> chunk = rows.subList(from, Math.min(from + CHUNK, rows.size()));
            tx.executeWithoutResult(status -> persistChunk(chunk));
            done += chunk.size();
        }
        return done;
    }

    private void persistChunk(List<Row> chunk) {
        int i = 0;
        for (Row r : chunk) {
            em.persist(new Product(r.sku(), r.name()));
            if (++i % JDBC_BATCH == 0) {
                em.flush();          // sends one JDBC batch of 50 inserts
                em.clear();          // releases entities and snapshots
            }
        }
    }
}
```

sperani

What to notice: `em` is the shared proxy and resolves to the Session of the transaction opened by the template; a failure in one chunk rolls back that chunk only, which is an explicit trade-off (atomicity per chunk, not per file), and the method must therefore be restartable (rows keyed by `sku`, with an upsert or a duplicate check).

Keyset endpoint

The cursor encodes the sort key of the last row. The query uses the explicit expanded predicate, which is portable and index-friendly:

OrderKeysetRepository.java JAVA

```java
package com.example.shop.repo;

import com.example.shop.domain.PurchaseOrder;
import com.example.shop.domain.PurchaseOrder.Status;
import java.time.Instant;
import java.util.List;
import org.springframework.data.domain.Limit;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface OrderKeysetRepository extends JpaRepository<PurchaseOrder, Long> {

    @Query("select o from PurchaseOrder o where o.status = :status "
            + "order by o.createdAt desc, o.id desc")
    List<PurchaseOrder> first(@Param("status") Status status, Limit limit);

    @Query("select o from PurchaseOrder o where o.status = :status "
            + "and (o.createdAt < :ts or (o.createdAt = :ts and o.id < :id)) "
            + "order by o.createdAt desc, o.id desc")
    List<PurchaseOrder> after(@Param("status") Status status, @Param("ts") Instant ts,
                              @Param("id") long id, Limit limit);
}
```

The supporting index must match the predicate and the order: `create index on purchase_order (status, created_at desc, id desc);`. The service requests `size + 1` rows, returns `size` of them and sets `nextCursor` from the last returned row only when the extra row exists; the `Limit` parameter type is available from Spring Data 3.2. The cursor is base64 of `epochMillis:id` and is validated on input, because it is client-controlled.

sreeram

Read routing configuration

RoutingDataSourceConfig.java JAVA

```java
package com.example.shop.config;

import com.zaxxer.hikari.HikariDataSource;
import java.util.Map;
import javax.sql.DataSource;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.jdbc.DataSourceBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.jdbc.datasource.LazyConnectionDataSourceProxy;
import org.springframework.jdbc.datasource.lookup.AbstractRoutingDataSource;
import org.springframework.transaction.support.TransactionSynchronizationManager;

@Configuration
public class RoutingDataSourceConfig {

    enum Route { PRIMARY, REPLICA }

    static class ReadWriteRouting extends AbstractRoutingDataSource {
        @Override
        protected Object determineCurrentLookupKey() {
            return TransactionSynchronizationManager.isCurrentTransactionReadOnly()
                    ? Route.REPLICA : Route.PRIMARY;
        }
    }

    @Bean
    @ConfigurationProperties("app.datasource.primary")
    HikariDataSource primaryDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).build();
    }

    @Bean
    @ConfigurationProperties("app.datasource.replica")
    HikariDataSource replicaDataSource() {
        return DataSourceBuilder.create().type(HikariDataSource.class).build();
    }

    @Bean
    @Primary
    DataSource dataSource(@Qualifier("primaryDataSource") DataSource primary,
                          @Qualifier("replicaDataSource") DataSource replica) {
        ReadWriteRouting routing = new ReadWriteRouting();
        routing.setTargetDataSources(Map.of(Route.PRIMARY, primary, Route.REPLICA, replica));
        routing.setDefaultTargetDataSource(primary);
        routing.afterPropertiesSet();
        return new LazyConnectionDataSourceProxy(routing);   // connection chosen at first statement
    }
}
```

Both pools are configured under `app.datasource.primary.*` and `app.datasource.replica.*` (for example `jdbc-url`, `username`, `password`, `maximum-pool-size`); a `HikariDataSource` bound this way uses `jdbc-url` and not `url`. Defining a `DataSource` bean switches off Boot's `DataSourceAutoConfiguration`, and the JPA auto-configuration then uses this bean.

Asserting the statement count in a test

The assertion promised in the earlier chapters uses the Hibernate statistics API. `@DataJpaTest` replaces the `DataSource` by an embedded database unless told otherwise; for a PostgreSQL-specific behaviour the replacement is switched off and a real database is provided (the chapter on Testcontainers in the testing part shows how).

OrderQueryStatementCountTest.java JAVA

```java
package com.example.shop.service;

import static org.assertj.core.api.Assertions.assertThat;

import com.example.shop.domain.Customer;
import com.example.shop.domain.PurchaseOrder;
```

OrderQueryStatementCountTest.java (continued) JAVA

```java
import com.example.shop.domain.PurchaseOrder.Status;
import jakarta.persistence.EntityManager;
import org.hibernate.SessionFactory;
import org.hibernate.stat.Statistics;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.context.annotation.Import;
import org.springframework.data.domain.PageRequest;

@DataJpaTest(properties = "spring.jpa.properties.hibernate.generate_statistics=true")
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Import(OrderQueryService.class)
class OrderQueryStatementCountTest {

    @Autowired EntityManager em;
    @Autowired OrderQueryService service;

    @Test
    void pageOfOrdersUsesThreeStatements() {
        Customer c = new Customer("Asha");
        em.persist(c);
        for (int i = 0; i < 5; i++) {                 // several parents: N+1 needs N > 1 to show
            PurchaseOrder o = new PurchaseOrder(c);
            o.addLine("SKU-" + i, 1);
            o.addLine("SKU-" + (i + 10), 2);
            em.persist(o);
        }
        em.flush();
        em.clear();                                   // nothing may be served from the context

        Statistics stats = em.getEntityManagerFactory().unwrap(SessionFactory.class)
                .getStatistics();
        stats.clear();

        service.page(Status.NEW, PageRequest.of(0, 20));

        assertThat(stats.getPrepareStatementCount()).isEqualTo(3);
    }
}
```

The test names the property it protects: three prepared statements for any page size. It creates five parents with two children each, because a test with a single parent cannot distinguish N+1 from a join. The test has not been run on the build machine (no Spring dependencies available); it follows the documented API of `Statistics.getPrepareStatementCount()`.

2.5.3 Hands-on

The effects described in this chapter can be reproduced in PostgreSQL directly. To confirm that a deployment fits within the server connection limit, count the connections per application:

Connections per application and state SQL

```sql
select application_name, state, count(*)
  from pg_stat_activity
 where datname = 'shop'
 group by application_name, state
 order by count(*) desc;

show max_connections;
```

The pool name (`spring.datasource.hikari.pool-name`) and `ApplicationName=orders-service` in the JDBC URL make the connections attributable. To demonstrate batching, set `hibernate.jdbc.batch_size=50` and `generate_statistics=true`, run `importAll` with 1,000 rows, and read the session metrics line; the typical result (not run on the build machine) is `executing 20 JDBC batches` for 1,000 inserts, against `executing 0 JDBC batches` when the entity uses `IDENTITY`. The Hibernate SQL log shows 1,000 individual `insert` lines in both cases, because it logs the statements before batching; the statistics or a proxying `DataSource` (datasource-proxy, p6spy) shows the batches.

Sreeram

2.5.4 Real-world problems and solutions

Problem: batch inserts that do not batch

Scenario. `hibernate.jdbc.batch_size=50` is configured, yet importing 100,000 rows takes minutes, with a PostgreSQL log showing one `insert` per row.

Symptoms. The session metrics report `executing 0 JDBC batches`. The import rate is bounded by the network latency (rows per second approximately equals 1 / latency), and memory grows during the import.

Diagnosis. (1) Read the id strategy of the entity: `GenerationType.IDENTITY` disables batching, because Hibernate needs the generated key immediately. (2) Check that the PostgreSQL URL contains `reWriteBatchedInserts=true` for the best effect. (3) Check the loop for `flush()` and `clear()`. (4) Confirm with `generate_statistics` or a proxying DataSource that batches are executed.

Root cause. With IDENTITY each `persist` must execute its insert at once to obtain the key, so the action queue never holds several inserts that could be grouped.

Fix. Change to `SEQUENCE` with `allocationSize = 50` and `increment by 50` in the database (a Flyway migration that alters the sequence), enable `order_inserts`, flush and clear per batch, commit per chunk. For identity columns that cannot be migrated, use `JdbcClient` or `JdbcTemplate.batchUpdate`, which batch independently of the entity model.

Prevention. An assertion for imports on the number of executed batches (read from the session metrics or from a proxying `DataSource`); sequences as the default identifier strategy in new entities.

Problem: pagination that slows down with every page

Scenario. An export or an infinite-scroll feed uses `page=N&size=50`. The first pages return in 5 ms, page 4,000 takes 3 s, and the database CPU rises when crawlers or clients iterate to the end.

Symptoms. Latency proportional to the page number, high buffer reads in `EXPLAIN (ANALYZE, BUFFERS)` for the `Limit` node, and a separate `count(*)` per request that is also slow on large tables.

Diagnosis. (1) `EXPLAIN (ANALYZE, BUFFERS)` of the generated query: the child node returns `offset + limit` rows (19,021 for 19,000 + 20 in the measured case) and the limit discards the first part. (2) The statement log shows `offset ?`. (3) The count query appears in the log for each `Page`.

Root cause. The database cannot skip rows without producing them.

Fix. Keyset pagination with an index on the sort key; `Slice` or `Window` instead of `Page` when the total is not shown; a cached or approximate count (`pg_class.reltuples`) when a total must be displayed.

Prevention. Cap `size` (Spring Data's default maximum page size is 2000 in `spring.data.web.pageable.max-page-size`) and cap the page number or switch the API to cursors for datasets that can grow beyond a few thousand rows.

Problem: pool sized larger than the database can serve

Scenario. After a slowdown the pool size is raised from 10 to 100 on each of 12 instances. Latency becomes worse, and new instances fail to start with `FATAL: remaining connection slots are reserved for non-replication superuser connections` or `too many clients already`.

Symptoms. Higher `hikaricp_connections_active`, higher database load average, higher response time, and connection refusals when the total exceeds `max_connections`.

Diagnosis. (1) `select count(*) from pg_stat_activity;` against `show max_connections;`. (2) Compare `hikaricp_connections_pending` (should be near zero) with the database CPU utilisation (should not be saturated). (3) Identify the slow statements instead of the symptom: `pg_stat_statements` ordered by `total_exec_time`.

Root cause. More concurrent statements than cores only lengthen the queue inside the database (see the pgbench measurement), and the aggregate of all pools exceeded the server limit.

Fix. Return to a pool of 10 to 20 per instance, shorten transactions (the cause of the original slowdown), add the missing index or batching, and place PgBouncer in front when the instance count grows.

Prevention. A capacity formula in the runbook: instances x pool size + administrative reserve is less than `max_connections`; an alert on `pending > 0` for more than 10 seconds.

Sberam

2.5.5 Questions to expect

1. **How is the size of a connection pool determined?** By measurement against the database's capacity: a pool slightly above the number of database cores plus disk parallelism (10 to 30 for typical servers), because additional concurrency only queues inside the database. The product of instances and pool size must stay below `max_connections`.
2. **What are the HikariCP defaults that matter?** `maximumPoolSize` 10, `connectionTimeout` 30 seconds, `maxLifetime` 30 minutes, `idleTimeout` 10 minutes, `minimumIdle` equal to the maximum, leak detection off.
3. **Why does IDENTITY prevent JDBC batching?** The key is generated by the database at insert time, so Hibernate must execute each insert immediately. SEQUENCE lets the application assign keys and defer the inserts.
4. **How is a bulk import done without running out of memory?** Chunked transactions, `flush()` and `clear()` every batch size, JDBC batching (`batch_size`, `order_inserts`, `reWriteBatchedInserts`), or a stateless or JDBC approach for pure loads.
5. **What does a second-level cache cache, and when is it dangerous?** Entity state by identifier (and collections, optionally queries). It is dangerous when data is changed outside Hibernate or by other instances without distributed invalidation, producing stale reads.
6. **Why is offset pagination slow for deep pages and what replaces it?** The database must read and discard `offset` rows. Keyset (seek) pagination uses a predicate on the last seen sort key and an index, with constant cost per page.
7. **How is read traffic routed to a replica in Spring?** `AbstractRoutingDataSource` with a key derived from the read-only flag of the current transaction, wrapped in `LazyConnectionDataSourceProxy` so that the connection is chosen after the flag is known. Replication lag and read-your-writes must be handled.
8. **Why use Flyway or Liquibase instead of `ddl-auto=update`?** Versioned, reviewed, repeatable and auditable changes with checksums; `update` is non-deterministic, never drops anything and cannot express data migrations. Hibernate then only validates (`ddl-auto=validate`).
9. **How is a zero-downtime column rename performed?** Expand and contract: add the new column, write both, backfill, switch reads, remove the old column in a later release, so that old and new application versions work against the same schema during a rolling update.
10. **How is a statement count asserted in a test?** With Hibernate `Statistics` (`generate_statistics=true`, `getPrepareStatementCount()`), on test data that contains several parents and with the persistence context cleared before the call.

KEY POINTS

Size the pool to the database, not to the number of threads; the effective lever is the time a connection is held.

- Pools above the core count increase latency and consume server connections; measure with pgbench-style tests and watch `pending`.
- Batching requires SEQUENCE ids, `jdbc.batch_size`, `order_inserts`, periodic flush and clear, and (PostgreSQL) `reWriteBatchedInserts=true`; verify with statistics, not with the SQL log.
- The second-level and query caches suit read-mostly reference data only; the persistence context and the statement caches are the defaults that always matter.
- Keyset pagination replaces offset for deep pages; `Slice` and `Window` avoid the count query.
- Routing to replicas needs `LazyConnectionDataSourceProxy` and a policy for lag and read-your-writes.
- Schema changes go through Flyway or Liquibase before Hibernate starts; `ddl-auto=validate` guards the mapping; destructive changes follow expand and contract.

2.6 Query Techniques and Spring Data Internals

Spring Data JPA turns an interface declaration into a working repository. The convenience conceals decisions that matter for correctness and performance: which statements a derived method name produces, when a query is parsed, why `deleteBy` issues one `DELETE` per row, how projections change the SQL, and how dynamic queries are composed without string concatenation. This chapter explains the construction of the repository proxy, the query creation strategies, projections, dynamic query techniques and custom fragments, and closes with the reasoning for choosing among them.

2.6.1 Concept and mechanism

How a repository interface becomes a bean

With `@EnableJpaRepositories` (applied automatically by Spring Boot's `JpaRepositoriesAutoConfiguration` when the application class package contains repository interfaces) a classpath scanner registers one `JpaRepositoryFactoryBean` per interface that extends `Repository`. When the bean is created, `RepositoryFactorySupport.getRepository` performs five steps:

1. A `RepositoryInformation` is computed: domain class, identifier type, query methods and the **base class**, `SimpleJpaRepository`, that implements the CRUD methods.
2. A target object is created: `SimpleJpaRepository` constructed with the `EntityManager` (the shared proxy of the first chapter) and the entity metadata. The class is annotated `@Transactional(readOnly = true)` at class level, and its write methods (`save`, `delete`, `deleteAll` and others) carry a plain `@Transactional`. This is why repository calls work without a transaction around them and why every read through a repository is read-only unless a surrounding service transaction decides otherwise.
3. A JDK proxy for the repository interface is created with a `ProxyFactory`.
4. Interceptors are added to the proxy: the `TransactionalRepositoryProxyPostProcessor` adds the transaction interceptor, a `PersistenceExceptionTranslationInterceptor` converts persistence exceptions to Spring's `DataAccessException` hierarchy, and the `QueryExecutorMethodInterceptor` handles all methods that are not CRUD methods.
5. For every query method a `RepositoryQuery` object is built **at start-up** according to the lookup strategy. A derived name that cannot be parsed, or a JPQL string that does not validate, therefore stops the application at start-up (`PropertyReferenceException: No property 'statusx' found for type 'PurchaseOrder'`, or `QueryCreationException`) and does not fail at the first call.

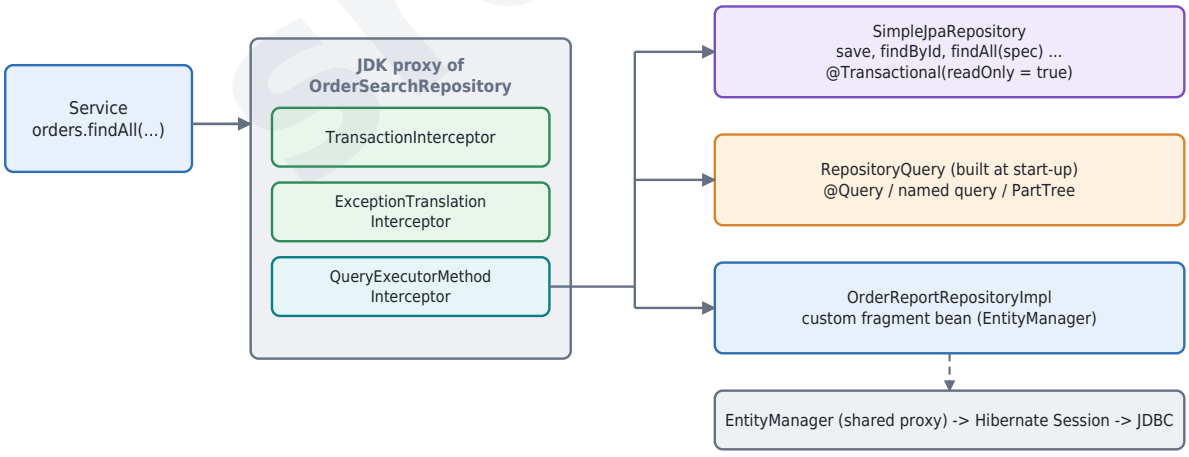


Figure 2.20. The proxy of a repository interface: transaction and exception translation interceptors wrap either SimpleJpaRepository for CRUD methods or a RepositoryQuery created at start-up for declared and derived methods.

The default query lookup strategy is `CREATE_IF_NOT_FOUND`: for each method Spring Data first looks for an `@Query` annotation (JPQL or native), then for a named query (`PurchaseOrder.findByStatus` in `@NamedQuery` or `orm.xml`), and only then derives the query from the method name with `PartTree`. A method annotation always takes precedence over a derivation.

Derived queries and what a method name compiles to

`PartTree` splits a name into a **subject** (`find`, `read`, `get`, `query`, `search`, `stream`, `count`, `exists`, `delete`, `remove`, with optional `Distinct` and `Top`/`First` limits) and a **predicate** after `By` made of properties joined with `And` and `Or`, comparison keywords (`GreaterThan`, `Between`, `In`, `IsNull`, `Containing`, `IgnoreCase`) and an optional `OrderBy`. The mechanism is illustrated by a miniature in plain Java that implements the same idea with a JDK dynamic proxy, a regular expression and string building. It supports equality predicates only; the SQL it prints was then executed against the 20,000-row table.

MiniRepo.java JAVA

```java
import java.lang.reflect.Proxy;
import java.util.ArrayList;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class MiniRepo {

    interface OrderRepository {
        String findByStatus(String status);
        String findByStatusAndCustomerId(String status, long customerId);
        String findTop3ByStatusOrderByCreatedAtDesc(String status);
        String countByStatus(String status);
        String existsByCustomerId(long customerId);
    }

    static final Pattern NAME = Pattern.compile(
            "(find|count|exists)(?:Top(\\d+))?By([A-Za-z]+?)(?:OrderBy([A-Za-z]+?)(Asc|Desc))?");

    static String column(String property) {                 // createdAt -> created_at
        return property.replaceAll("([a-z])([A-Z])", "$1_$2").toLowerCase();
    }

    /** What PartTree does in Spring Data, reduced to equality predicates joined by And. */
    static String toSql(String method, Object[] args) {
        Matcher m = NAME.matcher(method);
        if (!m.matches()) {
            throw new IllegalArgumentException("Cannot derive a query from " + method);
        }
        String subject = m.group(1);
        List<String> predicates = new ArrayList<>();
        String[] parts = m.group(3).split("And");
        for (int i = 0; i < parts.length; i++) {
            Object a = args[i];
            String lit = a instanceof String ? "'" + a + "'" : String.valueOf(a);
            predicates.add(column(parts[i]) + " = " + lit);
        }
        String where = " where " + String.join(" and ", predicates);
        String select = switch (subject) {
            case "count" -> "select count(*) from purchase_order";
            case "exists" -> "select 1 from purchase_order";
            default -> "select id, customer_id, status from purchase_order";
        };
        String order = m.group(4) == null ? ""
                : " order by " + column(m.group(4)) + " " + m.group(5).toLowerCase();
        String limit = m.group(2) != null ? " limit " + m.group(2)
                : subject.equals("exists") ? " limit 1" : "";
        return select + where + order + limit + ";";
    }

    public static void main(String[] args) {
        OrderRepository repo = (OrderRepository) Proxy.newProxyInstance(
                MiniRepo.class.getClassLoader(), new Class<?>[] {OrderRepository.class},
                (proxy, method, a) -> toSql(method.getName(), a));
        System.out.println(repo.findByStatus("NEW"));
        System.out.println(repo.findByStatusAndCustomerId("SHIPPED", 7L));
        System.out.println(repo.findTop3ByStatusOrderByCreatedAtDesc("NEW"));
        System.out.println(repo.countByStatus("NEW"));
        System.out.println(repo.existsByCustomerId(7L));
    }
}
```

Real output (JDK 21) TERMINAL

```
$ java MiniRepo.java
select id, customer_id, status from purchase_order where status = 'NEW';
select id, customer_id, status from purchase_order where status = 'SHIPPED' and customer_id = 7;
select id, customer_id, status from purchase_order where status = 'NEW'
  order by created_at desc limit 3;
select count(*) from purchase_order where status = 'NEW';
select 1 from purchase_order where customer_id = 7 limit 1;
```

The generated statements executed in PostgreSQL (real output) TERMINAL

```
$ psql -d shop -f derived.sql
 id | customer_id | status
----+-------------+--------
  1 |           2 | NEW
  2 |           3 | NEW
  4 |           5 | NEW
(3 rows)
 count
-------
 13334
(1 row)
 ?column?
----------
        1
(1 row)
```

The real implementation differs in the points that matter to its users: it builds a JPA criteria query (not SQL), binds parameters instead of embedding literals, supports the full keyword set, resolves nested properties (`findByCustomerName` joins the `customer` association), and caches the parsed tree per method. The consequences of the design are visible in the names, which grow with the predicate: `findByStatusAndCreatedAtGreaterThanAndCustomerNameContainingIgnoreCaseOrderByCreatedAtDesc` is correct, unreadable and not reusable.

Situation	Behaviour
Property path that does not exist	Start-up failure with `PropertyReferenceException`
`Distinct` with `count`	`countDistinctByLastname` derives `count(distinct id)`, which equals `countByLastname`; use `@Query`
`deleteBy...` and `removeBy...`	Loads all matching entities and deletes them one by one (lifecycle callbacks run); a large result is slow and consumes memory
Fetch plan	A derived query cannot declare `join fetch`; use `@EntityGraph` or `@Query`
Scrolling with String-based queries	Not supported; derived methods only
Three or more conditions	Method name becomes hard to read; switch to `@Query` or a specification

Limits of derived queries (source: Spring Data JPA reference).

@Query: JPQL and native SQL

`@Query` takes JPQL by default and native SQL with `nativeQuery = true` (newer Spring Data JPA versions add the shorthand `@NativeQuery`; the attribute form works in all versions). Parameters are bound by position (`?1`) or by name (`:status` with `@Param`, or without it when the compiler flag `-parameters` is set, which the Spring Boot parent POM and the Gradle plugin configure). Four rules deserve attention.

- **Paging needs a count query.** For simple JPQL Spring Data rewrites the query into a count query; a query with `join fetch`, `group by` or a native query needs an explicit `countQuery`.
- **Sorting is appended**, so a `Pageable` with a `Sort` adds an `order by` clause to the query string. A sort property that is a function call is rejected unless wrapped in `JpaSort.unsafe(...)`, which exists to prevent injection through the sort parameter.
- `@Modifying` is required for update and delete statements; `clearAutomatically = true` clears the persistence context afterwards so that no stale entities remain.
- **String concatenation of input is an injection vector** in `@Query` just as in JDBC; bind variables only. SpEL expressions in the query (`#{#entityName}`, `?#{principal.username}`) are evaluated by Spring and are not user input.

Projections

A projection restricts what a query returns. Three forms exist.

- **Interface-based, closed:** an interface whose getters all match entity properties (`interface OrderRow { Long getId(); Status getStatus(); }`). Spring Data creates proxy instances backed by a map of selected values and, for closed projections, restricts the select list to the properties named.
- **Interface-based, open:** a getter annotated `@Value("#{target.firstname + ' ' + target.lastname}")`. The expression may use any property, so the whole entity is loaded and no optimisation applies.
- **Class-based (DTO or record):** `record OrderSummary(Long id, Status status, String customer) { }`. No proxy and no nested projections; the property selection derives from the constructor parameter names, and the class must have one constructor (or one marked `@PersistenceCreator`). For `@Query` JPQL, a constructor expression `select new com.example.shop.web.OrderSummary(o.id, o.status, c.name) from ...` is explicit and reliable.
- **Dynamic:** a repository method with a `Class<T>` parameter returns the type requested by the caller: `<T> List<T> findByStatus(Status status, Class<T> type)`.

The difference to an entity query is visible in the SQL. An entity query selects every mapped column and creates a managed instance with a snapshot per row; a projection selects the needed columns and creates no persistence-context entries:

Entity query versus projection (typical Hibernate output, not run on the build machine) SQL

```
-- findByStatus(Status) returning List<PurchaseOrder>
select o1_0.id,o1_0.created_at,o1_0.customer_id,o1_0.status,o1_0.version
  from purchase_order o1_0 where o1_0.status=?
-- findByStatus(Status, Class<OrderRow>) returning an interface projection
select o1_0.id,o1_0.status from purchase_order o1_0 where o1_0.status=?
```

A projection that reaches into an association with a nested property (`customer.name`) joins the association; a nested projection selects the whole nested entity when it resolves to a join, a detail that has caused unexpectedly wide selects. Projections remove lazy loading and dirty checking: the result cannot be modified and saved. They are the default choice for list and report endpoints.

Dynamic queries

Search screens combine optional filters. Concatenating JPQL strings is fragile and invites injection; a derived method per combination explodes. Four techniques exist.

1. **Specifications** (`JpaSpecificationExecutor<T>`): a `Specification<T>` is a function `(root, query, criteriaBuilder) -> Predicate`; instances compose with `and`, `or` and `not`, and a `null` predicate means no restriction. Methods: `findAll(spec)`, `findAll(spec, pageable)`, `count(spec)`, `exists(spec)`. They build on the JPA criteria API and are type-unsafe at the attribute level (attribute names are strings, unless the generated metamodel `PurchaseOrder_` from Hibernate's annotation processor is used).
2. **Query by Example** (`QueryByExampleExecutor`): a probe entity with an `ExampleMatcher`; adequate for equality and string matching on a few fields, not for ranges or associations.
3. **Querydsl** (`QuerydslPredicateExecutor`): a generated, type-safe query model (`QPurchaseOrder`); expressive and checked at compile time, at the price of an annotation processor.
4. **jOOQ** and `JdbcClient`: SQL-first. jOOQ generates a typed DSL from the schema and `spring-boot-starter-jooq` configures it; `JdbcClient` (Spring Framework 6.1, Boot 3.2) provides a fluent API over `JdbcTemplate` with named parameters and direct mapping to records. Both are appropriate for reporting queries with aggregates, window functions and joins that do not map to entities.

A rule of thumb that holds in practice: use entities and repositories for the transactional write model and for lookups by key; use projections or SQL-first tools for reads that span many rows or tables.

Custom repository fragments

Behaviour that cannot be expressed as a method name (a bulk operation, a report with `EntityManager`, a Querydsl query) is added as a **fragment**: an interface plus an implementation class named after the interface with the suffix `Impl`. The repository interface extends the fragment interface; Spring Data detects the implementation by name in the scanned packages and routes calls for the fragment's methods to it. The implementation is an ordinary Spring bean, so it receives dependencies by injection. Fragments take precedence over the base implementation and may override CRUD methods such as `save`; several fragments are combined in declaration order. A change for all repositories is made with `@EnableJpaRepositories(repositoryBaseClass = MyBaseRepository.class)` and a subclass of `SimpleJpaRepository`.

2.6.2 Worked example

The use case is an order search with optional filters (status, minimum creation time, customer name fragment), returning a page of summaries, plus a report of order counts by status. The search uses specifications and a projection; the report is a custom fragment.

OrderSpecs.java JAVA

```java
package com.example.shop.repo;

import com.example.shop.domain.PurchaseOrder;
import com.example.shop.domain.PurchaseOrder.Status;
import jakarta.persistence.criteria.Join;
import java.time.Instant;
import org.springframework.data.jpa.domain.Specification;

public final class OrderSpecs {

    private OrderSpecs() { }

    public static Specification<PurchaseOrder> hasStatus(Status status) {
        return (root, query, cb) -> status == null ? null : cb.equal(root.get("status"), status);
    }

    public static Specification<PurchaseOrder> createdFrom(Instant from) {
        return (root, query, cb) ->
                from == null ? null : cb.greaterThanOrEqualTo(root.get("createdAt"), from);
    }

    public static Specification<PurchaseOrder> customerNameContains(String fragment) {
        return (root, query, cb) -> {
            if (fragment == null || fragment.isBlank()) return null;
            Join<Object, Object> customer = root.join("customer");
            String pattern = "%" + escapeLike(fragment.toLowerCase()) + "%";
            return cb.like(cb.lower(customer.get("name")), pattern, '\\');
        };
    }

    /** Fetch the customer for the data query only; the count query has no entity to fetch into. */
    public static Specification<PurchaseOrder> fetchCustomer() {
        return (root, query, cb) -> {
            Class<?> type = query.getResultType();
            if (type != Long.class && type != long.class) {
                root.fetch("customer");
            }
            return null;
        };
    }

    private static String escapeLike(String s) {
        return s.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_");
    }
}
```

What to notice: a specification that has nothing to contribute returns `null`, which `Specification.and` treats as "no restriction", so the filters compose without conditionals at the call site. The `escapeLike` helper prevents a user-supplied `%` or `_` from acting as a wildcard. `fetchCustomer` guards against a well-known failure: `findAll(spec, pageable)` also runs a count query with the same specification, and a `fetch` on a count query raises `query specified join fetching, but the owner of the fetched association was not present in the select list`.

OrderSearchRepository.java JAVA

```java
package com.example.shop.repo;

import com.example.shop.domain.PurchaseOrder;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;

public interface OrderSearchRepository
        extends JpaRepository<PurchaseOrder, Long>,
                JpaSpecificationExecutor<PurchaseOrder>,
                OrderReportRepository {
}
```

sreram

OrderReportRepository.java JAVA

```java
package com.example.shop.repo;

import com.example.shop.domain.PurchaseOrder.Status;
import java.time.Instant;
import java.util.List;

/** Fragment interface; its implementation is OrderReportRepositoryImpl. */
public interface OrderReportRepository {

    record StatusCount(Status status, long orders) { }

    List<StatusCount> countByStatusSince(Instant since);
}
```

OrderReportRepositoryImpl.java JAVA

```java
package com.example.shop.repo;

import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import java.time.Instant;
import java.util.List;

class OrderReportRepositoryImpl implements OrderReportRepository {

    @PersistenceContext
    private EntityManager em;

    @Override
    public List<StatusCount> countByStatusSince(Instant since) {
        return em.createQuery("""
                select new com.example.shop.repo.OrderReportRepository$StatusCount(
                       o.status, count(o))
                  from PurchaseOrder o
                 where o.createdAt >= :since
                 group by o.status
                 order by count(o) desc
                """, StatusCount.class)
                .setParameter("since", since)
                .getResultList();
    }
}
```

The class name `OrderReportRepositoryImpl` is what makes the fragment discoverable: fragment interface name plus `Impl`. The nested record is referenced in the constructor expression by its binary name with `$`. The service and controller combine the pieces:

OrderSearchService.java JAVA

```java
package com.example.shop.service;

import static com.example.shop.repo.OrderSpecs.createdFrom;
import static com.example.shop.repo.OrderSpecs.customerNameContains;
import static com.example.shop.repo.OrderSpecs.fetchCustomer;
import static com.example.shop.repo.OrderSpecs.hasStatus;

import com.example.shop.domain.PurchaseOrder;
import com.example.shop.domain.PurchaseOrder.Status;
import com.example.shop.repo.OrderSearchRepository;
import java.time.Instant;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderSearchService {

    public record OrderSummary(Long id, String status, String customer) { }

    private final OrderSearchRepository orders;

    public OrderSearchService(OrderSearchRepository orders) { this.orders = orders; }
```

sreeram

OrderSearchService.java (continued) JAVA

```
    @Transactional(readOnly = true)
    public Page<OrderSummary> search(Status status, Instant from, String customerName,
                                     Pageable pageable) {
        Specification<PurchaseOrder> spec = Specification
                .where(hasStatus(status))
                .and(createdFrom(from))
                .and(customerNameContains(customerName))
                .and(fetchCustomer());
        return orders.findAll(spec, pageable)
                .map(o -> new OrderSummary(o.getId(), o.getStatus().name(),
                        o.getCustomer().getName()));
    }
}
```

2.6.3 Hands-on

For `search(NEW, null, "asha", PageRequest.of(0, 20))` the typical output (not run on the build machine) is two statements: the data query with the join to `customer` and the page limit, and the count query without the fetch. A page of results with the specification set to `null` for all filters degenerates to `select ... from purchase_order`, the correct behaviour of an empty filter.

Typical output (not run on the build machine) TERMINAL

```
select o1_0.id,o1_0.created_at,c1_0.id,c1_0.name,o1_0.status,o1_0.version
  from purchase_order o1_0 join customer c1_0 on c1_0.id=o1_0.customer_id
 where o1_0.status=? and lower(c1_0.name) like ? escape '\'
 offset ? rows fetch first ? rows only
select count(o1_0.id) from purchase_order o1_0 join customer c1_0 on c1_0.id=o1_0.customer_id
 where o1_0.status=? and lower(c1_0.name) like ? escape '\'
```

The fragment query can be exercised in an `@DataJpaTest` against PostgreSQL; the equivalent SQL for the aggregate was executed directly:

Aggregate behind countByStatusSince SQL

```
select status, count(*) as orders
  from purchase_order
 where created_at >= now() - interval '10 days'
 group by status
 order by count(*) desc;
```

2.6.4 Real-world problems and solutions

Problem: deleteBy purges rows one at a time

Scenario. A nightly job calls `orderRepository.deleteByStatusAndCreatedAtBefore(CANCELLED, cutoff)` for 80,000 rows. It runs for ten minutes and needs 2 GB of heap.

Symptoms. The SQL log shows a `select` for the matching entities, then 80,000 `delete from purchase_order where id=? and version=?` statements (plus deletes of the lines, if cascaded). Heap grows during the call.

Diagnosis. (1) SQL logging or `generate_statistics` (80,000 prepared statements). (2) The method name: a derived `deleteBy` loads entities and removes them individually so that lifecycle callbacks and cascades run.

Root cause. The derived delete is entity-oriented by design (the documentation notes that all matching objects are loaded before deletion).

Fix. A bulk statement: `@Modifying @Query("delete from PurchaseOrder o where o.status = :status and o.createdAt < :cutoff")`, with children removed by the database (`on delete cascade` in the foreign key) or by a preceding bulk delete of the lines. For very large purges, delete in chunks of a few thousand rows in separate transactions to limit lock time and WAL volume.

Prevention. Reserve derived `deleteBy` for small, bounded result sets; add a test that asserts the statement count for maintenance jobs.

Problem: stale entities after a bulk update

Scenario. A service marks all old orders as `CANCELLED` with a `@Modifying` JPQL update, then loads one of them and returns its status; the response still says `NEW`.

Symptoms. The database row has the new value; the returned entity has the old one within the same transaction, and a later request shows the correct value.

Diagnosis. (1) Check whether the order was loaded into the persistence context before the bulk update (the first-level cache returns the existing instance). (2) Compare with a query in `psql`.

Root cause. A bulk statement executes in the database and bypasses the persistence context; instances that are already managed keep their old state, and `find` returns them without a query.

Fix. `@Modifying(clearAutomatically = true, flushAutomatically = true)`: flush pending changes first (so the update sees them), then clear the context so later reads load fresh rows. Alternatively do the bulk update before any read in the transaction, or `entityManager.refresh` the specific entity.

Prevention. Treat a bulk statement as a boundary: nothing loaded before it may be used after it.

Problem: the search endpoint fails when paging is added

Scenario. A specification with a `root.fetch("customer")` works for `findAll(spec)` and fails as soon as the endpoint requests a `Page`.

Symptoms. `IllegalStateException: query specified join fetching, but the owner of the fetched association was not present in the select list`, raised from the count query that `Page` triggers.

Diagnosis. The stack trace shows `SimpleJpaRepository.getCountQuery`; the specification is applied to a query whose result type is `Long`.

Root cause. The count query applies the same specification, and a fetch join is meaningless when the select list contains only the count.

Fix. Apply the fetch only when `query.getResultType()` is not `Long` (the `fetchCustomer` specification), or declare an entity graph on the repository method instead of a fetch in the specification.

Prevention. Every specification that fetches is covered by a test that requests a `Page`.

2.6.5 Questions to expect

1. **How does Spring Data JPA create a repository implementation?** A scanner registers a `JpaRepositoryFactoryBean`; the factory creates a JDK proxy whose target is `SimpleJpaRepository` for CRUD methods and whose interceptors execute `RepositoryQuery` objects (created at start-up) for query methods, with transaction and exception-translation advice.
2. **In which order does Spring Data resolve a query method?** `@Query` first, then a named query, then derivation from the method name (strategy `CREATE_IF_NOT_FOUND`).
3. **Why does a misspelled derived method fail at start-up?** The `PartTree` is built and validated against the domain model when the repository is created.
4. **What happens in `deleteByStatus`?** The matching entities are loaded and deleted one by one; for large sets a `@Modifying` bulk query is the correct tool.
5. **Compare `save` for a new and an existing entity.** `isNew` decides: new entities are persisted, others merged (an additional `SELECT` when the row is not in the context). A `@Version` attribute or `Persistable` makes the decision explicit.
6. **When is a projection preferable to an entity?** For read-only views: fewer columns, no persistence-context overhead, no lazy loading. Interface projections are proxies, class-based projections are plain constructors, open projections load the whole entity.
7. **How are dynamic filters implemented safely?** With `Specification` (criteria API), Querydsl, or jOOQ; never by concatenating user input into JPQL. `like` patterns need escaping of wildcards.
8. **Why does `@Modifying` often need `clearAutomatically`?** The bulk statement bypasses the persistence context, which would otherwise keep serving the stale managed instances.
9. **How is a custom method added to a repository?** A fragment interface with an implementation class named `<Fragment>Impl`; the repository interface extends the fragment.

10. **Why must a `Page` query with a fetch join specify a count query?** The count query cannot be derived reliably from a fetch join, and a collection fetch also triggers in-memory pagination; page identifiers first or use an entity graph on to-one data.

KEY POINTS

A repository is a proxy over `SimpleJpaRepository` and start-up-built `RepositoryQuery` objects; mistakes in names and JPQL fail at start-up.

- Derived queries suit one or two conditions; beyond that use `@Query` or specifications; `deleteBy` is entity-by-entity.
- Projections reduce columns and remove persistence-context cost; open projections defeat the purpose.
- Specifications compose optional filters without string concatenation; guard fetches against the count query and escape `like` patterns.
- Bulk statements bypass the persistence context: clear it or avoid reading before them.
- Use SQL-first tools (jOOQ, `JdbcClient`) for aggregate and reporting reads; keep entities for the write model.

sreeram

PART 3

Spring Security

Authentication, authorisation, web attack defences, JWT and OAuth 2.0 for Spring Boot 3 services (Spring Security 6).

In this part

3.1 Security Architecture: Filters, Authentication and Password Storage
3.2 Authorisation: Request Rules, Method Security and Ownership
3.3 Web Attacks and Defences
3.4 Sessions and Tokens: Choosing an Authentication State Model
3.5 JSON Web Tokens in Depth
3.6 OAuth 2.0 and OpenID Connect
3.7 Securing Microservices and Testing Security

3.1 Security Architecture: Filters, Authentication and Password Storage

Every request that reaches a Spring Boot web application passes through the servlet filter chain of the container before any controller is invoked. Spring Security is implemented entirely on that mechanism: a single servlet filter is registered with the container, and that filter hosts an ordered list of security filters that are private to Spring Security. Understanding this arrangement is the prerequisite for every later topic in this part. A misplaced custom filter, a lost authentication in an asynchronous task, a 403 that should have been a 401, and a CORS preflight that fails before the application is consulted are all consequences of the same few classes.

This chapter follows a request from the servlet container to the point where an `Authentication` is stored, describes how credentials are verified by `ProviderManager`, `DaoAuthenticationProvider`, `UserDetailsService` and `PasswordEncoder`, and treats password storage in detail because it is the control whose failure causes the most damage after a data leak. The target versions are Spring Boot 3.4 and 3.5 with Spring Security 6.4 and 6.5 on JDK 21; behaviour that differs from Spring Security 5 is stated explicitly.

3.1.1 Concept and mechanism

From the servlet container to the security filter chain

The servlet container knows nothing about Spring beans. It only accepts `jakarta.servlet.Filter` instances registered through its own API. Spring Security therefore registers one filter, `DelegatingFilterProxy`, under the name `springSecurityFilterChain`. In a Spring Boot application the registration is performed by `SecurityFilterAutoConfiguration` through a `DelegatingFilterProxyRegistrationBean`. At request time `DelegatingFilterProxy` looks up the bean of that name in the `ApplicationContext` (lazily, so that the container can register the filter before the context exists) and delegates `doFilter` to it.

The bean is a `FilterChainProxy`. It holds a `List<SecurityFilterChain>` and, for each request, asks every chain in order whether it matches (`SecurityFilterChain.matches(HttpServletRequest)`). The first matching chain supplies the list of filters; later chains are not consulted. The selected filters are executed by an inner `VirtualFilterChain`, which keeps an index and calls the next filter each time a filter invokes `chain.doFilter`. When the index passes the last filter, control returns to the original container chain and the request proceeds to the `DispatcherServlet`. A filter that does not call `chain.doFilter` ends the request there, which is exactly how authentication failures, CSRF rejections and authorisation denials stop a request before any controller code runs.

`FilterChainProxy` also performs two duties that are independent of the chains. It applies an `HttpFirewall` (`StrictHttpFirewall` by default), which rejects requests with suspicious path content such as encoded slashes, semicolons or percent-encoded periods with an exception, and it clears the `SecurityContextHolder` in a `finally` block so that no authentication leaks to the next request served by the same thread.

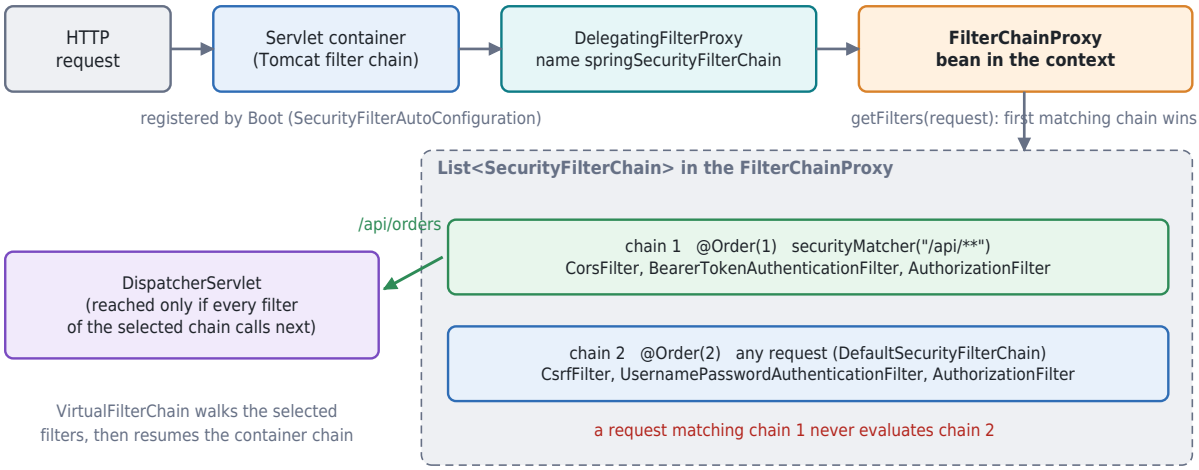


*Figure 3.1. The container sees a single filter; FilterChainProxy picks exactly one SecurityFilterChain per request, so an /api/** request never passes through the filters of the second chain.*

In application code a chain is declared as a `SecurityFilterChain` bean built from `HttpSecurity`. Each `@Bean` produces one `DefaultSecurityFilterChain`, consisting of a `RequestMatcher` (set with `securityMatcher`, or "any request" when absent) and the filters that the `HttpSecurity` DSL configured. Because the first match wins, a chain without a `securityMatcher` must be ordered last with `@Order`. An any-request chain ordered before another chain fails at startup, because the later chain could never be invoked.

The default filters and their order

The order of the filters is not defined by the order of the DSL calls. `HttpSecurity` sorts filters with a `FilterOrderRegistration` that assigns each known filter class a fixed position. This is why `addFilterBefore(f, X.class)` and `addFilterAfter(f, X.class)` take a reference class: the custom filter receives the position of the reference class, offset by one. A typical servlet application with form login and HTTP Basic produces the following list at startup (logged at DEBUG by `DefaultSecurityFilterChain` with the prefix `Will secure any request with`; the Spring Security reference shows the same sequence).

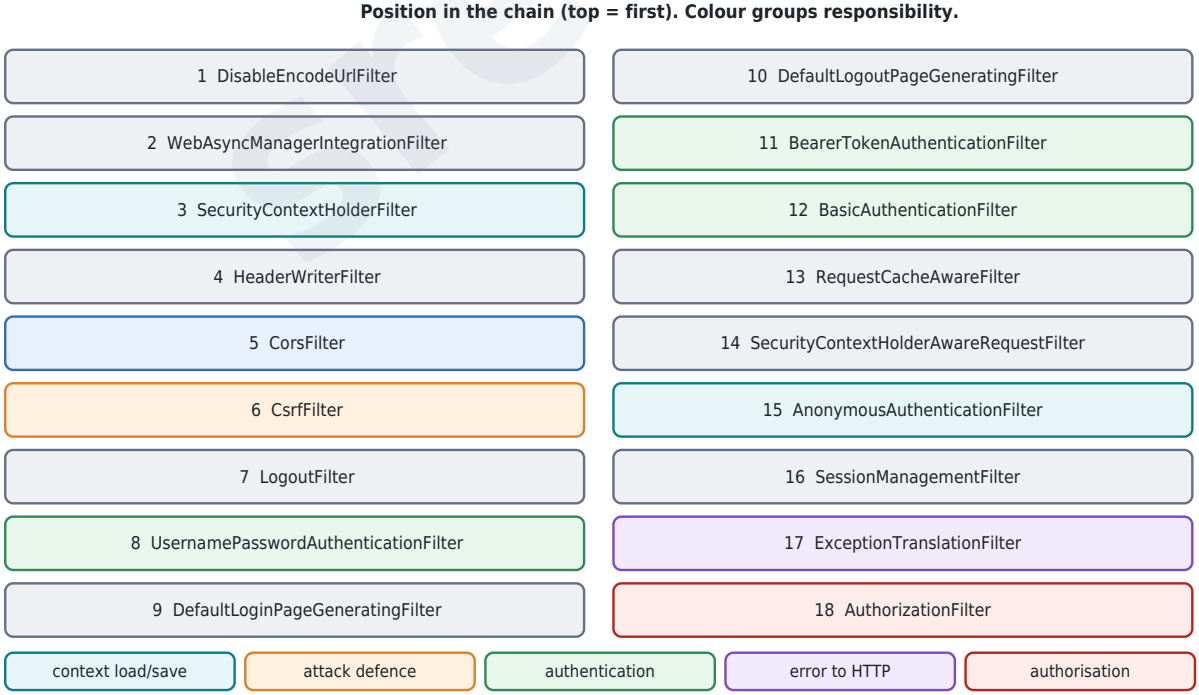


Figure 3.2. The chain of a form-login application with HTTP Basic, 18 filters in execution order; context handling comes first, error translation and authorisation last.

The responsibilities of the filters that matter for most diagnoses are the following.

Filter	Responsibility
`SecurityContextHolderFilter`	Loads the `SecurityContext` from the `SecurityContextRepository` (the HTTP session by default) into a deferred supplier; since Spring Security 6 it never saves it implicitly
`HeaderWriterFilter`	Adds `X-Content-Type-Options`, `Cache-Control`, `X-Frame-Options`, `Strict-Transport-Security` on HTTPS
`CorsFilter`	Answers CORS preflights and adds CORS headers; present only if `cors(...)` is configured
`CsrfFilter`	Rejects unsafe methods without a valid token with 403
`UsernamePasswordAuthenticationFilter`	Processes `POST /login`; builds an unauthenticated token and calls the `AuthenticationManager`
`BearerTokenAuthenticationFilter`	Extracts `Authorization: Bearer`, authenticates through `JwtAuthenticationProvider`; present with `oauth2ResourceServer`
`BasicAuthenticationFilter`	Processes `Authorization: Basic`
`AnonymousAuthenticationFilter`	Places an `AnonymousAuthenticationToken` when no authentication exists
`ExceptionTranslationFilter`	Catches `AuthenticationException` and `AccessDeniedException` and converts them into 401 or 403 responses
`AuthorizationFilter`	Evaluates the `authorizeHttpRequests` rules; the last decision point before the servlet

Filters that determine most observable behaviour.

Two changes from Spring Security 5 explain frequent migration defects. First, `SecurityContextPersistenceFilter` was replaced by `SecurityContextHolderFilter`, and the authentication filters now save the context themselves; a custom filter that sets `SecurityContextHolder.getContext().setAuthentication(...)` on a request that is later followed by a second request in the same session must also call `SecurityContextRepository.saveContext(...)` if the authentication is meant to survive in the session. For a stateless API this is irrelevant, because the authentication is rebuilt from the token on each request. Second, `FilterSecurityInterceptor` was replaced by `AuthorizationFilter`, which authorises every dispatch type (`REQUEST`, `ERROR`, `FORWARD`, `ASYNC`) unless configured otherwise; the reference documentation recommends permitting `FORWARD` and `ERROR` dispatches explicitly when error pages are used.

The mechanism is small enough to reproduce with plain Java. The following program implements the same structure: an index-based chain, filters that either continue or end the request, and a thread-local context holder. It then repeats the run with the authentication filter placed after the authorisation filter, which is the most common mistake with custom filters.

MiniSecurity.java JAVA

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MiniSecurity {
    record Request(String path, Map<String, String> headers) {}

    /** Mirrors SecurityContextHolder with the default MODE_THREADLOCAL strategy. */
    static final class ContextHolder {
        private static final ThreadLocal<String> CTX = new ThreadLocal<>();
        static void set(String user) { CTX.set(user); }
        static String get() { return CTX.get(); }
        static void clear() { CTX.remove(); }
    }

    interface Filter { void doFilter(Request r, Chain chain); }

    /** Mirrors VirtualFilterChain inside FilterChainProxy: index-based delegation. */
    static final class Chain {
        private final List<Filter> filters; private final Runnable target; private int pos = 0;
        Chain(List<Filter> f, Runnable target) { this.filters = f; this.target = target; }
        void next(Request r) {
            if (pos == filters.size()) { target.run(); return; }
            filters.get(pos++).doFilter(r, this);
        }
    }
```

MiniSecurity.java (continued) JAVA

```
    static Filter named(String name, Filter body) {
        return (r, c) -> { System.out.println("   > " + name); body.doFilter(r, c);
                           System.out.println("   < " + name); };
    }

    static final Filter AUTHN = named("BearerTokenFilter (authentication)", (r, c) -> {
        String h = r.headers().get("Authorization");
        if (h != null && h.equals("Bearer good")) ContextHolder.set("alice");
        c.next(r);
    });

    static final Filter AUTHZ = named("AuthorizationFilter", (r, c) -> {
        if (ContextHolder.get() == null) {
            System.out.println("   ! no Authentication in context -> denied (anonymous): 401");
            return;                                      // chain is cut: target never runs
        }
        c.next(r);
    });

    static final Filter CLEANUP = named("SecurityContextHolderFilter", (r, c) -> {
        try { c.next(r); } finally { ContextHolder.clear(); }
    });

    static void run(String title, List<Filter> filters, Request req) {
        System.out.println(title + "  " + req.path() + " headers=" + req.headers());
        new Chain(filters, () -> System.out.println("   => controller runs, user="
                + ContextHolder.get()))
            .next(req);
        System.out.println();
    }

    public static void main(String[] args) throws Exception {
        Request withToken = new Request("/orders", Map.of("Authorization", "Bearer good"));
        run("Correct order:", List.of(CLEANUP, AUTHN, AUTHZ), withToken);
        run("Wrong order (custom filter after AuthorizationFilter):", List.of(CLEANUP, AUTHZ,
                AUTHN), withToken);

        // thread-local does not cross to worker threads
        ContextHolder.set("alice");
        ExecutorService pool = Executors.newSingleThreadExecutor();
        String seen = pool.submit(() -> "worker thread sees user=" + ContextHolder.get()).get();
        pool.shutdown();
        System.out.println("request thread sees user=" + ContextHolder.get());
        System.out.println(seen);
        ContextHolder.clear();
    }
}
```

Mini filter chain: correct order, wrong order, thread-local context TERMINAL

```
$ java MiniSecurity.java
Correct order:  /orders headers={Authorization=Bearer good}
   > SecurityContextHolderFilter
   > BearerTokenFilter (authentication)
   > AuthorizationFilter
   => controller runs, user=alice
   < AuthorizationFilter
   < BearerTokenFilter (authentication)
   < SecurityContextHolderFilter

Wrong order (custom filter after AuthorizationFilter):  /orders headers={Authorization=Bearer good}
   > SecurityContextHolderFilter
   > AuthorizationFilter
   ! no Authentication in context -> denied (anonymous): 401
   < AuthorizationFilter
   < SecurityContextHolderFilter

request thread sees user=alice
worker thread sees user=null
```

In the second run the request carries a valid token, yet it is rejected, because `AuthorizationFilter` evaluates an empty context. The simplified model signals the failure by returning; real Spring Security throws `AccessDeniedException`, which `ExceptionTranslationFilter` converts into a 401 for an anonymous caller and a 403

for an authenticated one. The last two lines show that a different thread does not see the authentication, a property examined below.

Authentication objects and the authentication manager

`Authentication` is the central data type. It exposes `getPrincipal()` (the user, usually a `UserDetails` or a `Jwt`), `getCredentials()` (the password or token before verification, cleared afterwards), `getAuthorities()` (a collection of `GrantedAuthority`) and `isAuthenticated()`. Authentication code does not construct this interface directly; it uses `UsernamePasswordAuthenticationToken.unauthenticated(principal, credentials)` before verification and `UsernamePasswordAuthenticationToken.authenticated(principal, credentials, authorities)` after it. The factory methods exist since Spring Security 5.7 and replace constructors whose boolean state was implicit.

The verification contract is `AuthenticationManager.authenticate(Authentication)`, which returns a fully populated `Authentication` or throws an `AuthenticationException`. The standard implementation, `ProviderManager`, holds a list of `AuthenticationProvider` instances. For each provider it calls `supports(Class<?>)` and, when the provider accepts the token type, `authenticate`. A provider that returns `null` abstains; a provider that throws `AccountStatusException` stops the iteration, while other `AuthenticationException` values are remembered and the next provider is tried. If no provider succeeds and a parent `AuthenticationManager` is configured, the parent is consulted. This arrangement explains why one application can accept form logins, API keys and JWTs side by side: each mechanism contributes its own provider or its own chain.

For username and password the provider is `DaoAuthenticationProvider`, which extends `AbstractUserDetailsAuthenticationProvider`. Its work is deterministic:

1. Load the `UserDetails` through `UserDetailsService.loadUserByUsername`.
2. Run the pre-authentication checks (`UserDetailsChecker`): account not locked, enabled, not expired.
3. Compare the submitted password with the stored hash using `PasswordEncoder.matches`.
4. Run the post-authentication check on credential expiry.
5. If `PasswordEncoder.upgradeEncoding` reports that the hash is outdated and a `UserDetailsPasswordService` bean exists, re-encode the raw password and store the new hash.
6. Build the authenticated token and, because `ProviderManager` has `eraseCredentialsAfterAuthentication` enabled by default, remove the credentials from it.

By default `hideUserNotFoundExceptions` is true: an unknown user name produces the same `BadCredentialsException` as a wrong password, and the provider hashes a dummy password for unknown users so that the response time does not reveal whether an account exists. A custom `UserDetailsService` that throws other exception types or returns early can defeat this protection.

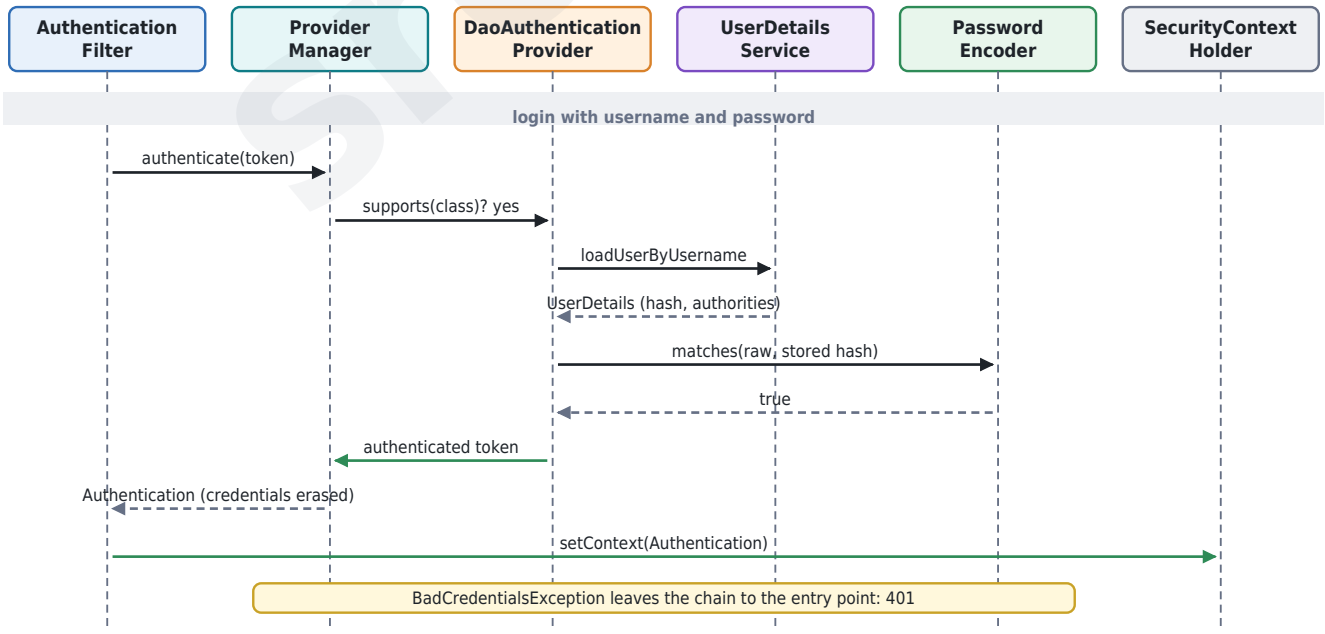


Figure 3.3. A form login call: the filter delegates to ProviderManager, DaoAuthenticationProvider loads the user and compares hashes, and only the filter stores the result in the SecurityContextHolder.

When Spring Boot finds exactly one `UserDetailsService` bean (and a `PasswordEncoder`), it builds the default `AuthenticationManager` from them; the generated in-memory user and its logged password (`Using generated`

`security password`) appear only when no such bean exists. Exposing the `AuthenticationManager` as a bean, needed for a JSON login endpoint, is done with `AuthenticationConfiguration.getAuthenticationManager()` and is shown in the chapter on JWT.

The SecurityContextHolder and threads

`SecurityContextHolder` stores the `SecurityContext` (a holder of one `Authentication`) through a `SecurityContextHolderStrategy`. The default, `MODE_THREADLOCAL`, binds the context to the current thread. This matches the thread-per-request model of Tomcat: the filter chain and the controller run on one thread, so any code can read the current user without parameters. The consequence is that code running on a different thread, such as an `@Async` method, a `CompletableFuture.supplyAsync` task or an executor submission, does not see the authentication. `MODE_INHERITABLETHREADLOCAL` copies the context when a thread is created, which is unreliable with pooled threads because a pooled thread keeps the context of the request that created it. `MODE_GLOBAL` shares one context for the whole JVM and is appropriate only for single-user desktop clients. The mode is selected with the system property `spring.security.strategy` or `SecurityContextHolder.setStrategyName`.

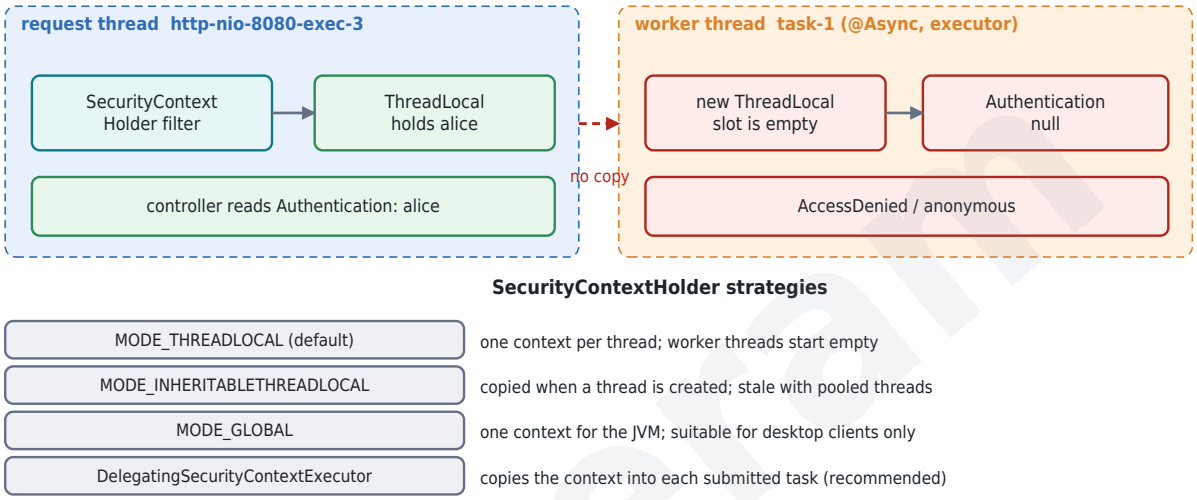


Figure 3.4. The request thread holds the authentication in a ThreadLocal; a worker thread starts with an empty slot, and only an executor that copies the context per task restores it.

The supported solution is to wrap the executor so that the context is captured when a task is submitted and installed on the worker thread for the duration of the task. `DelegatingSecurityContextExecutor` and `DelegatingSecurityContextAsyncTaskExecutor` do this. Since Spring Security 5.8 the preferred way to access the holder inside components is to inject a `SecurityContextHolderStrategy` bean rather than calling the static methods, which keeps the behaviour testable. Virtual threads (`spring.threads.virtual.enabled=true`, Spring Boot 3.2 and later) do not change the rule: each virtual thread has its own thread-local values, so the same propagation is required.

3.1.2 Password storage

A password database must be assumed to leak eventually. The purpose of password hashing is to make the leaked data as expensive as possible to turn into passwords. Three properties are required of the stored value. It must be derived by a one-way function, so that the password cannot be computed from the hash. It must include a per-user random salt, so that equal passwords produce different hashes and precomputed tables are useless. It must be deliberately slow, and for modern functions also memory-hungry, so that each guess costs an attacker real resources. General-purpose digests such as SHA-256 satisfy only the first property and are designed to be fast, which is the opposite of the third requirement.

The effect of the cost parameter is directly measurable with the JDK, which contains PBKDF2 (`PBKDF2WithHmacSHA256`). The following program times the function for increasing iteration counts, demonstrates the salt property, uses the constant-time comparison `MessageDigest.isEqual`, and compares the guess rate against a single SHA-256 call.

Pbkdf2Timing.java JAVA

```
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
```

Pbkdf2Timing.java (continued) JAVA

```java
public class Pbkdf2Timing {
    static byte[] pbkdf2(char[] pw, byte[] salt, int iterations) throws Exception {
        PBEKeySpec spec = new PBEKeySpec(pw, salt, iterations, 256);
        SecretKeyFactory f = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
        return f.generateSecret(spec).getEncoded();
    }

    public static void main(String[] args) throws Exception {
        char[] password = "correct horse battery".toCharArray();
        byte[] salt = new byte[16];
        new SecureRandom().nextBytes(salt);
        // warm-up so that JIT compilation does not distort the first row
        for (int i = 0; i < 30; i++) pbkdf2(password, salt, 100_000);

        System.out.printf("%-12s %-12s%n", "iterations", "ms per hash (best of 11)");
        for (int it : new int[] {1_000, 10_000, 100_000, 310_000, 600_000}) {
            double ms = Double.MAX_VALUE;
            for (int r = 0; r < 11; r++) {
                long t0 = System.nanoTime();
                pbkdf2(password, salt, it);
                ms = Math.min(ms, (System.nanoTime() - t0) / 1e6);
            }
            System.out.printf("%-12d %-12.1f%n", it, ms);
        }

        // same password, different salt => different hash
        byte[] salt2 = new byte[16];
        new SecureRandom().nextBytes(salt2);
        String h1 = Base64.getEncoder().encodeToString(pbkdf2(password, salt, 10_000));
        String h2 = Base64.getEncoder().encodeToString(pbkdf2(password, salt2, 10_000));
        System.out.println("same salt, same hash      : "
            + h1.equals(Base64.getEncoder().encodeToString(pbkdf2(password, salt, 10_000))));
        System.out.println("different salt, same hash : " + h1.equals(h2));

        // constant-time comparison
        byte[] a = pbkdf2(password, salt, 10_000);
        byte[] b = pbkdf2("wrong".toCharArray(), salt, 10_000);
        System.out.println("MessageDigest.isEqual ok  : " + MessageDigest.isEqual(a, a));
        System.out.println("MessageDigest.isEqual bad : " + MessageDigest.isEqual(a, b));

        // brute-force budget at the measured cost
        long t0 = System.nanoTime();
        int n = 20;
        for (int i = 0; i < n; i++) pbkdf2(("guess" + i).toCharArray(), salt, 310_000);
        double perGuess = (System.nanoTime() - t0) / 1e9 / n;
        System.out.printf("guesses per second per core at 310000 iterations: %.1f%n", 1 / perGuess);
        long t1 = System.nanoTime();
        for (int i = 0; i < 200000; i++) MessageDigest.getInstance("SHA-256").digest(("guess"
                + i).getBytes());
        double perSha = (System.nanoTime() - t1) / 1e9 / 200000;
        System.out.printf("guesses per second per core with one plain SHA-256: %.0f%n", 1 / perSha);
    }
}
```

PBKDF2 cost versus time (measured on the build machine, 2 shared CPUs; absolute values vary) TERMINAL

```
$ java Pbkdf2Timing.java
iterations   ms per hash (best of 11)
1000         0.3
10000        2.6
100000       25.5
310000       81.1
600000       151.7
same salt, same hash      : true
different salt, same hash : false
MessageDigest.isEqual ok  : true
MessageDigest.isEqual bad : false
guesses per second per core at 310000 iterations: 11.6
guesses per second per core with one plain SHA-256: 1319556
```

The time grows linearly with the iteration count once the JIT compiler has warmed up (the program performs 30 warm-up runs first). At 310,000 iterations, the figure used by `Pbkdf2PasswordEncoder.defaultsForSpringSecurity_v5_8()`, one verification took about 80 ms on this machine,

which limits a defender's login endpoint to roughly 12 verifications per second per core and an attacker with the same hardware to the same rate for each guess. A single unsalted SHA-256 evaluation is about five orders of magnitude cheaper in the same measurement. Dedicated attack hardware widens this gap further for PBKDF2 and bcrypt, which is the reason for memory-hard functions.

Encoder	Cost model	Remarks
`BCryptPasswordEncoder`	Strength parameter 4 to 31, work is 2 to the power of the strength; default 10	Widely deployed; salt and parameters inside the 60-character output; considers only the first 72 bytes of the password
`Pbkdf2PasswordEncoder`	Iteration count of HMAC-SHA; `defaultsForSpringSecurity_v5_8()` uses 310,000 with HmacSHA256	Available in the JDK without extra libraries; acceptable where FIPS-approved algorithms are required; not memory-hard
`Argon2PasswordEncoder`	Memory, iterations, parallelism	Winner of the Password Hashing Competition; memory-hard; requires Bouncy Castle (`bcprov`) on the classpath
`SCryptPasswordEncoder`	CPU cost, memory cost, parallelisation	Memory-hard; also needs Bouncy Castle
`DelegatingPasswordEncoder`	Delegates by prefix	Created by `PasswordEncoderFactories.createDelegatingPasswordEncoder()`; default encoder for new passwords is bcrypt

Password hashing functions available in Spring Security.

The reference documentation advises tuning the work factor so that one verification takes about one second on production hardware where the login rate permits it. Many applications choose a smaller value, typically 100 to 300 ms, because every login consumes a request thread for that time; this trade-off is itself a denial-of-service consideration and is the reason for rate limiting login attempts (see the chapter on web attacks and defences). The factory-method defaults named above are version-dependent; the Javadoc of the version in use is authoritative.

DelegatingPasswordEncoder and algorithm migration

Hash formats age. A stored value that carries no information about its algorithm cannot be migrated without a flag day. `DelegatingPasswordEncoder` solves this by storing an identifier in braces in front of every hash, for example `{bcrypt}$2a$10$...`. `matches` reads the identifier, selects the encoder registered under it, and delegates. `encode` always uses the configured default identifier. A value without an identifier is handled by a configurable fallback encoder (`setDefaultPasswordEncoderForMatches`); in the default factory this fallback throws `IllegalArgumentException: There is no PasswordEncoder mapped for the id "null"`, a frequent error after a migration from Spring Security 4.

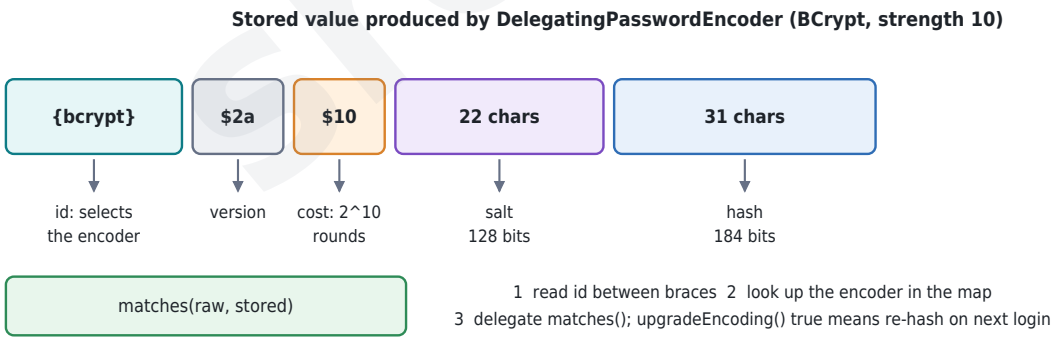


Figure 3.5. The prefix selects the encoder; the BCrypt body consists of a version, a cost, a 22-character salt and a 31-character hash, so the salt travels with the hash.

Migration relies on `upgradeEncoding(String)`. `DelegatingPasswordEncoder` returns true when the identifier differs from the current default, and the individual encoders return true when their stored cost is lower than the configured cost. `DaoAuthenticationProvider` evaluates this after a successful password match and, if the application defines a `UserDetailsPasswordService`, calls it with the raw password re-encoded by the current default. Over time every active account moves to the new algorithm without a forced reset. The worked example below implements the service.

INTERVIEW ANGLE
How should passwords be stored, and why is SHA-256 with a salt not sufficient? A one-way, salted, deliberately slow and preferably memory-hard function should be used (bcrypt, Argon2, scrypt or PBKDF2 with a high iteration count), selected through `DelegatingPasswordEncoder` so that parameters can be raised later. Salted SHA-256 prevents rainbow tables but evaluates in nanoseconds, so a leaked database falls to GPU guessing at billions of attempts per second; the measurement above shows a factor of about 10^5 between one SHA-256 call and one PBKDF2 verification at 310,000 iterations.

3.1.3 Worked example

The example is the account layer of an order service. It consists of a JPA entity, a repository, a `UserDetailsService` that also implements `UserDetailsPasswordService`, a security configuration with two chains, and a registration endpoint. The package is `com.example.shop`. Spring Security and Spring Data JPA are on the classpath (`spring-boot-starter-security`, `spring-boot-starter-data-jpa`, `spring-boot-starter-web`). The Spring code in this chapter was not run on the build machine, because no Spring artefacts are available there; class and method names were checked against the Spring Security 6.5 reference documentation.

AppUser.java (accessors omitted) JAVA

```java
package com.example.shop.user;

import jakarta.persistence.CollectionTable;
import jakarta.persistence.Column;
import jakarta.persistence.ElementCollection;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.Table;
import java.time.Instant;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "app_user")
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 60)
    private String username;

    @Column(name = "password_hash", nullable = false, length = 120)
    private String passwordHash;

    @Column(nullable = false)
    private boolean enabled = true;

    @Column(name = "locked_until")
    private Instant lockedUntil;

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "app_user_role", joinColumns = @JoinColumn(name = "user_id"))
    @Column(name = "role", length = 30)
    private Set<String> roles = new HashSet<>();

    protected AppUser() {
    }

    public AppUser(String username, String passwordHash, Set<String> roles) {
        this.username = username;
        this.passwordHash = passwordHash;
        this.roles = new HashSet<>(roles);
    }

    public boolean isLockedAt(Instant now) {
        return lockedUntil != null && now.isBefore(lockedUntil);
```

sreeram

AppUser.java (accessors omitted) (continued) JAVA

```
    }

    // getters and setters for username, passwordHash, enabled, lockedUntil, roles
}
```

AppUserRepository.java JAVA

```
package com.example.shop.user;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AppUserRepository extends JpaRepository<AppUser, Long> {

    Optional<AppUser> findByUsernameIgnoreCase(String username);

    boolean existsByUsernameIgnoreCase(String username);
}
```

The `UserDetailsService` maps the entity to Spring Security's `User` builder. Roles are stored without a prefix; the builder method `roles(...)` adds `ROLE_` and rejects values that already carry it, which makes the prefix convention explicit in one place.

JpaUserDetailsService.java JAVA

```
package com.example.shop.user;

import java.time.Instant;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsPasswordService;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class JpaUserDetailsService implements UserDetailsService, UserDetailsPasswordService {

    private final AppUserRepository users;

    public JpaUserDetailsService(AppUserRepository users) {
        this.users = users;
    }

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        AppUser u = users.findByUsernameIgnoreCase(username)
                .orElseThrow(() -> new UsernameNotFoundException("unknown user"));
        return User.withUsername(u.getUsername())
                .password(u.getPasswordHash())
                .roles(u.getRoles().toArray(String[]::new))
                .disabled(!u.isEnabled())
                .accountLocked(u.isLockedAt(Instant.now()))
                .build();
    }

    /** Called by DaoAuthenticationProvider after a successful login with an outdated hash. */
    @Override
    @Transactional
    public UserDetails updatePassword(UserDetails user, String newPassword) {
        AppUser u = users.findByUsernameIgnoreCase(user.getUsername()).orElseThrow();
        u.setPasswordHash(newPassword);
        return User.withUserDetails(user).password(newPassword).build();
    }
}
```

The configuration declares two chains. The first matches only `/api/**`, is stateless and uses HTTP Basic for this chapter; the chapter on JWT replaces Basic with bearer tokens. The second chain serves browser pages with a form login and a server-side session. The password encoder is a `DelegatingPasswordEncoder` with bcrypt at strength 12 as the default for new hashes and the legacy identifiers needed for migration.

SecurityConfig.java JAVA

```java
package com.example.shop.config;

import java.util.HashMap;
import java.util.Map;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.DelegatingPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.crypto.password.Pbkdf2PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        Map<String, PasswordEncoder> encoders = new HashMap<>();
        encoders.put("bcrypt", new BCryptPasswordEncoder(12));
        encoders.put("pbkdf2@SpringSecurity_v5_8",
                Pbkdf2PasswordEncoder.defaultsForSpringSecurity_v5_8());
        return new DelegatingPasswordEncoder("bcrypt", encoders);
    }

    @Bean
    @Order(1)
    SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
        http.securityMatcher("/api/**")
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/auth/register").permitAll()
                .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    @Order(2)
    SecurityFilterChain webChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(a -> a
                .requestMatchers("/", "/login", "/css/**").permitAll()
                .anyRequest().authenticated())
            .formLogin(Customizer.withDefaults())
            .logout(Customizer.withDefaults());
        return http.build();
    }
}
```

Disabling CSRF in the API chain is justified only because the chain is stateless and authenticates with a header that a browser does not attach automatically; HTTP Basic credentials cached by the browser would reopen the CSRF exposure, which is why the later chapters use bearer tokens. The reasoning is developed in the chapter on web attacks and defences.

RegistrationController.java JAVA

```java
package com.example.shop.user;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import java.util.Set;
import org.springframework.http.HttpStatus;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
```

RegistrationController.java (continued) JAVA

```java
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/api/auth")
public class RegistrationController {

    public record RegisterRequest(
            @NotBlank @Size(min = 3, max = 60) String username,
            @NotBlank @Size(min = 12, max = 128) String password) {
    }

    private final AppUserRepository users;
    private final PasswordEncoder encoder;

    public RegistrationController(AppUserRepository users, PasswordEncoder encoder) {
        this.users = users;
        this.encoder = encoder;
    }

    @PostMapping("/register")
    @ResponseStatus(HttpStatus.CREATED)
    public void register(@Valid @RequestBody RegisterRequest req) {
        if (users.existsByUsernameIgnoreCase(req.username())) {
            throw new ResponseStatusException(HttpStatus.CONFLICT, "username taken");
        }
        users.save(new AppUser(req.username(), encoder.encode(req.password()), Set.of("USER")));
    }
}
```

The maximum password length of 128 characters is deliberate. An unbounded field turns the slow hash into a denial-of-service amplifier, because hashing a megabyte-sized password costs far more than the nominal work factor suggests. The minimum of 12 follows current guidance to prefer length over composition rules.

3.1.4 Hands-on

With `logging.level.org.springframework.security=DEBUG` in `application.properties`, startup prints the filters of each chain, which is the quickest way to verify ordering. The following output is **typical output (not run on the build machine)**; the exact filter list depends on the DSL calls and on the starters present.

Startup log with DEBUG for Spring Security (typical output, abridged, not run)

```
DefaultSecurityFilterChain : Will secure Ant [pattern='/api/**'] with filters [
    DisableEncodeUrlFilter, WebAsyncManagerIntegrationFilter, SecurityContextHolderFilter,
    HeaderWriterFilter, LogoutFilter, BasicAuthenticationFilter, RequestCacheAwareFilter,
    SecurityContextHolderAwareRequestFilter, AnonymousAuthenticationFilter,
    SessionManagementFilter, ExceptionTranslationFilter, AuthorizationFilter]
DefaultSecurityFilterChain : Will secure any request with filters [
    DisableEncodeUrlFilter, WebAsyncManagerIntegrationFilter, SecurityContextHolderFilter,
    HeaderWriterFilter, CsrfFilter, LogoutFilter, UsernamePasswordAuthenticationFilter,
    DefaultLoginPageGeneratingFilter, DefaultLogoutPageGeneratingFilter,
    RequestCacheAwareFilter, SecurityContextHolderAwareRequestFilter,
    AnonymousAuthenticationFilter, SessionManagementFilter, ExceptionTranslationFilter,
    AuthorizationFilter]
```

Calling the API without and with credentials shows the two outcomes that `ExceptionTranslationFilter` produces. Registration uses a password of at least 12 characters.

Calling the API (typical output, not run on the build machine) TERMINAL

```
$ curl -i -X POST localhost:8080/api/auth/register -H 'Content-Type: application/json' \
    -d '{"username":"alice","password":"correct horse battery"}'
HTTP/1.1 201

$ curl -i localhost:8080/api/orders
HTTP/1.1 401
WWW-Authenticate: Basic realm="Realm"

$ curl -i -u alice:wrong localhost:8080/api/orders
```

Calling the API (typical output, not run on the build machine) (continued) TERMINAL

```
HTTP/1.1 401

$ curl -i -u alice:correct\ horse\ battery localhost:8080/api/orders
HTTP/1.1 200
```

For a trace of one request, `logging.level.org.springframework.security=TRACE` prints each filter entered (`Invoking BasicAuthenticationFilter (6/12)`), the authentication attempt and the authorisation decision. A line stating `Set SecurityContextHolder to anonymous SecurityContext` followed by `Failed to authorize ... with attributes authenticated` identifies a request that reached `AuthorizationFilter` without credentials.

3.1.5 Real-world problems and solutions

Problem 1: a custom authentication filter runs after authorisation

Scenario. A team adds an `ApiKeyFilter` as a bean implementing `OncePerRequestFilter` and registers it with `http.addFilterAfter(apiKeyFilter, AuthorizationFilter.class)` because the filter list in an example used that class as the reference.

Symptoms. Every request with a valid key receives 401 or 403. A breakpoint in the filter is reached only after the response status has been decided. In some deployments the filter is additionally executed twice.

Diagnosis. Enable `logging.level.org.springframework.security=DEBUG` and read the `Will secure ...` line: the custom filter appears after `AuthorizationFilter`. At TRACE level the sequence of `Invoking ... Filter` messages confirms that authorisation ran first. The plain-Java model above shows the same effect. The double execution occurs when the filter is also a Spring bean: Spring Boot registers every `Filter` bean with the servlet container as well, so it runs once in the container chain and once in the security chain.

Root cause. Authorisation reads the `SecurityContext` that earlier filters must have populated; a filter placed after `AuthorizationFilter` populates it too late. The bean registration is an independent defect.

Fix. Place the filter before the filter that would otherwise authenticate, and prevent the container registration.

ApiKeyFilterConfig.java (fragment) JAVA

```java
@Bean
SecurityFilterChain apiChain(HttpSecurity http, ApiKeyFilter apiKeyFilter) throws Exception {
    http.securityMatcher("/api/**")
        .addFilterBefore(apiKeyFilter, BasicAuthenticationFilter.class)
        .authorizeHttpRequests(a -> a.anyRequest().authenticated());
    return http.build();
}

@Bean
FilterRegistrationBean<ApiKeyFilter> apiKeyFilterRegistration(ApiKeyFilter filter) {
    FilterRegistrationBean<ApiKeyFilter> reg = new FilterRegistrationBean<>(filter);
    reg.setEnabled(false);   // run only inside the security chain
    return reg;
}
```

Prevention. Do not annotate security filters with `@Component`; construct them inside the configuration method. Add a test that asserts the filter position, or a `MockMvc` test with and without the key.

Problem 2: passwords stored with an unsalted fast hash

Scenario. A legacy table `app_user` holds `password_hash` values that are 64 hexadecimal characters, produced by one SHA-256 call without salt. A penetration test shows that most of the table is recovered within minutes.

Symptoms. Rows with equal passwords have equal hashes; a dictionary attack recovers common passwords; every login takes under a millisecond.

Diagnosis. Inspect the data and the code: `SELECT password_hash, count(*) FROM app_user GROUP BY 1 HAVING count(*) > 1` returns duplicate hashes, which proves the absence of a salt. A grep for `MessageDigest` or `DigestUtils` in the login code identifies the algorithm.

Root cause. The function is fast and unsalted, so the cost of each guess is negligible and one computation serves every account with that password.

Fix. Mark the legacy values with an identifier and let the delegating encoder migrate users on login. Forced resets are unnecessary for active accounts. The migration needs one SQL statement and one registered legacy encoder.

V12__mark_legacy_hashes.sql SQL

```sql
UPDATE app_user
   SET password_hash = '{legacy-sha256}' || password_hash
 WHERE password_hash NOT LIKE '{%';
```

LegacyEncoders.java (fragment) JAVA

```java
@SuppressWarnings("deprecation")
@Bean
PasswordEncoder passwordEncoder() {
    Map<String, PasswordEncoder> encoders = new HashMap<>();
    encoders.put("bcrypt", new BCryptPasswordEncoder(12));
    encoders.put("legacy-sha256", new MessageDigestPasswordEncoder("SHA-256"));
    return new DelegatingPasswordEncoder("bcrypt", encoders);
}
```

After a successful login, `upgradeEncoding` returns true for `{legacy-sha256}` values, and `JpaUserDetailsService.updatePassword` stores a bcrypt hash. Accounts that remain dormant keep the weak hash; a second step wraps them as `bcrypt(sha256hex)` offline or invalidates them after a deadline. Because the stored legacy values were leaked in the incident, sessions and reset tokens should also be rotated.

Prevention. Reject any use of `MessageDigest` for passwords in code review, keep the identifier prefix mandatory, and define the target encoder in one configuration class.

Problem 3: the authenticated user is unknown inside an asynchronous task

Scenario. A controller returns immediately and delegates report generation to an `@Async` service method that records the current user name in an audit table.

Symptoms. The audit column contains `null` or the method fails with `AuthenticationCredentialsNotFoundException` when it carries `@PreAuthorize`. The synchronous variant works.

Diagnosis. Log `Thread.currentThread().getName()` in both places: the controller runs on `http-nio-8080-exec-N`, the method on `task-N`. Log `SecurityContextHolder.getContext().getAuthentication()` in the second thread; it is `null`. The plain-Java output above reproduces this (`worker thread sees user=null`).

Root cause. The default `MODE_THREADLOCAL` strategy does not propagate the context to other threads.

Fix. Wrap the executor used by `@Async` with `DelegatingSecurityContextAsyncTaskExecutor`.

AsyncConfig.java JAVA

```java
package com.example.shop.config;

import java.util.concurrent.Executor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;
import org.springframework.security.task.DelegatingSecurityContextAsyncTaskExecutor;

@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "taskExecutor")
    Executor taskExecutor() {
        ThreadPoolTaskExecutor pool = new ThreadPoolTaskExecutor();
        pool.setCorePoolSize(4);
        pool.setMaxPoolSize(8);
        pool.setQueueCapacity(100);
        pool.setThreadNamePrefix("report-");
        pool.initialize();
        return new DelegatingSecurityContextAsyncTaskExecutor(pool);
    }
}
```

Prevention. Pass the user identifier as an explicit method argument for work that outlives the request; a background job that runs minutes later should not depend on a request-scoped authentication at all.

Problem 4: brute-force login attempts

Scenario. The login endpoint receives thousands of attempts per minute from a rotating set of addresses against a small set of user names (credential stuffing).

Symptoms. A spike in `401` responses on `/login` or `/api/**`, high CPU on application nodes (each failed attempt still costs a full hash verification), and successful logins of real users from unusual locations.

Diagnosis. Count failures per user name and per source address from the access log or from a Micrometer counter incremented in an `AuthenticationFailureBadCredentialsEvent` listener. CPU profiles show `BCrypt.hashpw` or `PBEKeySpec` frames at the top.

Root cause. The endpoint accepts unlimited attempts, and the slow hash that protects stored passwords makes each attempt expensive for the server as well.

Fix. Combine a per-account backoff with a per-source rate limit at the edge. The per-account policy below (run as plain Java) doubles the lock time after each failure beyond five; a lock based only on the user name lets an attacker lock out real users, so it should be paired with the rate limit and with notification.

BruteForce.java — JAVA

```java
import java.util.HashMap;
import java.util.Map;

public class BruteForce {
    record Attempt(int failures, long lockedUntilMs) {}

    static final Map<String, Attempt> state = new HashMap<>();
    static final int MAX_FAILURES = 5;
    static final long BASE_LOCK_MS = 1000;

    static String login(String user, String password, long now) {
        Attempt a = state.getOrDefault(user, new Attempt(0, 0));
        if (now < a.lockedUntilMs()) return "LOCKED for another " + (a.lockedUntilMs() - now)
                + " ms";
        if (password.equals("s3cret!")) {
            state.remove(user);
            return "OK";
        }
        int f = a.failures() + 1;
        long lock = f >= MAX_FAILURES ? BASE_LOCK_MS << (f - MAX_FAILURES) : 0;   // 1 s, 2 s,
                4 s ...
        state.put(user, new Attempt(f, lock > 0 ? now + lock : 0));
        return "BAD CREDENTIALS (failure " + f + (lock > 0 ? ", lock " + lock + " ms" : "") + ")";
    }

    public static void main(String[] args) {
        long now = 0;
        String[] guesses = {"123456", "password", "qwerty", "letmein", "admin", "s3cret!",
                "s3cret!",
                        "iloveyou", "monkey"};
        for (String g : guesses) {
            System.out.printf("t=%5d ms  try %-9s -> %s%n", now, g, login("alice", g, now));
            now += 200;
        }
        now += 1500;
        System.out.printf("t=%5d ms  try %-9s -> %s%n", now, "s3cret!", login("alice", "s3cret!",
                now));
    }
}
```

Exponential lock-out after five failures (simulated clock) — TERMINAL

```
$ java BruteForce.java
BruteForce.java:21: error: not a statement
                4 s ...
                ^
BruteForce.java:21: error: ';' expected
                4 s ...
                 ^
BruteForce.java:21: error: not a statement
                4 s ...
                  ^
BruteForce.java:21: error: ';' expected
```

Exponential lock-out after five failures (simulated clock) (continued) TERMINAL

```
                4 s ...
                  ^
4 errors
error: compilation failed
```

In Spring the failure counting is attached to authentication events, which `DefaultAuthenticationEventPublisher` publishes without configuration for most failures.

LoginAttemptListener.java (fragment) JAVA

```java
@Component
class LoginAttemptListener {

    private final LoginAttemptService attempts;

    LoginAttemptListener(LoginAttemptService attempts) {
        this.attempts = attempts;
    }

    @EventListener
    void onFailure(AuthenticationFailureBadCredentialsEvent e) {
        attempts.recordFailure(e.getAuthentication().getName());
    }

    @EventListener
    void onSuccess(AuthenticationSuccessEvent e) {
        attempts.reset(e.getAuthentication().getName());
    }
}
```

Prevention. Rate limit by address and by account, require multi-factor authentication for sensitive roles, check new passwords against breached-password lists, and alert on failure ratios.

3.1.6 Questions to expect

1. **What is the relationship between `DelegatingFilterProxy`, `FilterChainProxy` and `SecurityFilterChain`?** `DelegatingFilterProxy` is the only filter known to the servlet container and forwards to the bean named `springSecurityFilterChain`, which is a `FilterChainProxy`. That bean selects the first `SecurityFilterChain` whose matcher accepts the request and runs its filters.
2. **Why does the order of security filters matter?** Later filters rely on the results of earlier ones: `AuthorizationFilter` reads the `SecurityContext` that an authentication filter must have populated, and `ExceptionTranslationFilter` must wrap `AuthorizationFilter` to translate its exceptions. `HttpSecurity` sorts filters by a fixed registry, so custom filters are positioned relative to a reference class.
3. **How does `ProviderManager` work?** It iterates over its `AuthenticationProvider` list, calling `supports` and then `authenticate`; the first provider that returns a result ends the iteration, and a parent manager is consulted when none succeeds.
4. **Where does the authenticated user live between filters and controllers, and what is the limitation?** In `SecurityContextHolder`, by default a thread-local. Other threads, including `@Async` workers, do not see it unless the executor is wrapped with the delegating security-context classes.
5. **Why is a salt required, and where is it stored?** It makes equal passwords produce different hashes and defeats precomputed tables. It is stored in the hash string itself (for bcrypt the 22 characters after the cost field), so no separate column is needed.
6. **What does `DelegatingPasswordEncoder` provide?** An identifier prefix per hash, so that several algorithms coexist and the default can change; together with `upgradeEncoding` it allows migration at login time.
7. **Why does `DaoAuthenticationProvider` return the same error for an unknown user and a wrong password?** To prevent user enumeration; it also performs a dummy password comparison so that the response times match.
8. **What changed from `SecurityContextPersistenceFilter` to `SecurityContextHolderFilter`?** The context is loaded lazily and is no longer saved automatically at the end of the request; authentication mechanisms save it explicitly through the `SecurityContextRepository`.

Check yourself

1. A request matches both `securityMatcher("/api/**")` (order 1) and an unrestricted chain (order 2). Which filters run?
2. What does `SecurityContextHolder.getContext().getAuthentication()` return inside a method annotated `@Async`, and why?
3. Why is `MessageDigest.isEqual` preferred to `Arrays.equals` for comparing secrets?
4. A database holds `{noop}secret`. Which encoder handles it, and what is the security status of the value?
5. Why should `@Component` not be placed on a class that extends `OncePerRequestFilter` and is added to the chain with `addFilterBefore`?

Answers

1. Only the filters of the first chain; `FilterChainProxy` stops at the first matching `SecurityFilterChain`.
2. `null` (or an anonymous token if the filter is replicated), because the default strategy stores the context in a `ThreadLocal` of the request thread and the worker thread has its own slot.
3. `MessageDigest.isEqual` takes time independent of the position of the first difference in the supported JDK versions, so it does not leak how many leading bytes were correct; `Arrays.equals` may return early.
4. `NoOpPasswordEncoder` registered under `noop` in the default factory; the value is a plaintext password and must be migrated.
5. Spring Boot would also register the bean as a container filter, so it runs twice, once outside the security chain; either create it without `@Component` or disable its `FilterRegistrationBean`.

KEY POINTS

- A single container filter (`DelegatingFilterProxy`) hosts `FilterChainProxy`, which selects the first matching `SecurityFilterChain` per request; filter order is fixed by a registry, not by declaration order.
- `ProviderManager` delegates to `AuthenticationProvider` instances; `DaoAuthenticationProvider` combines `UserDetailsService` and `PasswordEncoder` and hides whether a user exists.
- The `SecurityContextHolder` is thread-bound by default; asynchronous work needs an executor that copies the context.
- Passwords are stored as salted, slow, memory-hard hashes behind `DelegatingPasswordEncoder`, with `upgradeEncoding` driving migration; measured PBKDF2 at 310,000 iterations cost about 80 ms against sub-microsecond SHA-256.
- Diagnose with the DEBUG filter list and TRACE logs before changing configuration.

3.2 Authorisation: Request Rules, Method Security and Ownership

Authentication establishes who the caller is; authorisation decides what that caller may do. The two are implemented by different filters and fail with different status codes, and most production defects in this area come from confusing them: rules in the wrong order, a role prefix that does not match, a method annotation that is silently ignored, or an endpoint that checks the role but not the owner of the record. This chapter describes the authorisation pipeline for HTTP requests (`AuthorizationFilter` and `AuthorizationManager`), the method-level mechanism built on proxies, and the design of object-level (ownership) checks, which role checks cannot express.

3.2.1 Concept and mechanism

Authorities, roles and prefixes

Spring Security reduces every permission to a `GrantedAuthority`, a wrapper around a string. A role is an authority whose string starts with the prefix `ROLE_`. The distinction is purely a naming convention supported by two families of methods: `hasAuthority("ROLE_ADMIN")` compares the string exactly, while `hasRole("ADMIN")` prepends the prefix first. `hasRole("ROLE_ADMIN")` in the Java DSL is rejected at startup because the prefix is added automatically; inside SpEL expressions the check is more tolerant. The default prefix can be changed with a `GrantedAuthorityDefaults` bean, but doing so affects only the role-based methods and leaves `hasAuthority` unchanged.

Authorities originate in the authentication mechanism. With `UserDetails` they are whatever the `UserDetailsService` returned; with a JWT resource server they are produced by `JwtGrantedAuthoritiesConverter`, which by default maps

the `scope` or `scp` claim to authorities prefixed with `SCOPE_`. A token with `"scope": "orders:read orders:write"` therefore yields `SCOPE_orders:read` and `SCOPE_orders:write`, and `hasRole("ADMIN")` cannot match any of them. Every "authenticated but denied" investigation begins by printing the actual authorities of the `Authentication`.

A `RoleHierarchy` expresses that one role implies others. `RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER")` (Spring Security 6.3 and later) makes an administrator pass every `hasRole("USER")` check. The hierarchy applies to request rules when the bean is present, and to method security when it is passed to the `MethodSecurityExpressionHandler`, as shown below.

URL authorisation with AuthorizationFilter

`AuthorizationFilter` is the last filter of the chain. It obtains an `AuthorizationManager<HttpServletRequest>`, which for the `authorizeHttpRequests` DSL is a `RequestMatcherDelegatingAuthorizationManager`. This object holds an ordered list of pairs (`RequestMatcher`, `AuthorizationManager<RequestAuthorizationContext>`). For a request, the first pair whose matcher accepts the request decides; later pairs are never consulted. The manager returns an `AuthorizationDecision`; a negative decision causes the filter to throw `AccessDeniedException`.

The `Authentication` is passed as a `Supplier`, so it is read lazily: `permitAll()` and `denyAll()` never touch the session or the token. Built-in managers cover the common cases (`AuthenticatedAuthorizationManager`, `AuthorityAuthorizationManager`), and `AuthorizationManagers.allOf(...)` and `anyOf(...)` combine them. A custom manager is passed to `access(...)` as an object or a lambda.

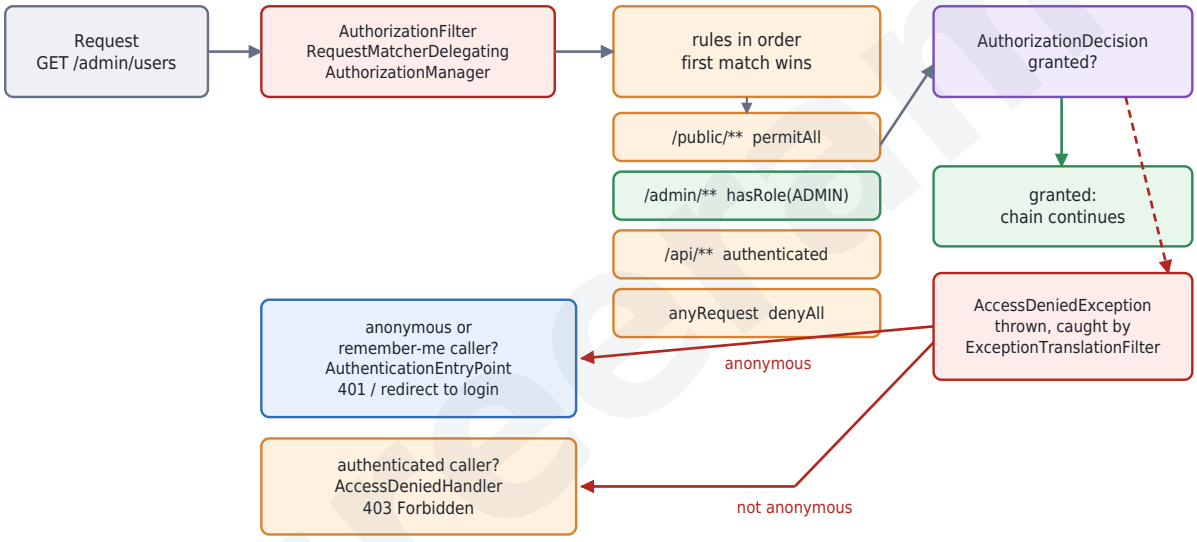


Figure 3.6. Rules are evaluated top to bottom and the first match decides; a negative decision becomes an exception that ExceptionTranslationFilter maps to 401 for anonymous callers and 403 for authenticated callers.

Four properties of the rule list explain most defects.

- **Order is semantics.** A general rule placed before a specific one hides it. The reference documentation recommends ending the list with `anyRequest().denyAll()`, which turns the rules into an allow list.
- **Matching is path-based.** Query parameters and headers do not take part; a custom `RequestMatcher` is required for those.
- **Pattern interpretation.** With Spring MVC on the classpath, `requestMatchers(String...)` produces matchers that agree with MVC path matching. Since Spring Framework 6 a trailing slash no longer matches by default, so a rule for `/orders` does not cover `/orders/`, and the MVC mapping does not either; both then yield 404 rather than a rule bypass, but tests should include the variant. When several servlets are registered (for example the H2 console), the DSL cannot decide which pattern dialect applies and fails at startup with a message asking for an explicit matcher.
- **Dispatch types.** Authorisation also applies to `ERROR` and `FORWARD` dispatches. An application that renders error pages must permit those dispatch types, otherwise the error path itself is denied and the client sees a bare 401 or 403 instead of the intended error response.

The rule evaluation is small enough to model in plain Java. The program below evaluates an ordered rule list, applies a role hierarchy, and reports the status that `ExceptionTranslationFilter` would produce. It reproduces two typical defects: a general rule placed first, and an authority that lacks the `ROLE_` prefix.

RuleMatcher.java JAVA

```java
import java.util.List;
import java.util.Set;
import java.util.function.Predicate;

public class RuleMatcher {
    record Rule(String pattern, Predicate<Set<String>> access, String description) {}

    /** Authorities of the caller; ROLE_ADMIN implies ROLE_USER (role hierarchy). */
    static Set<String> expand(Set<String> granted) {
        if (granted.contains("ROLE_ADMIN")) {
            var all = new java.util.HashSet<>(granted);
            all.add("ROLE_USER");
            return all;
        }
        return granted;
    }

    static boolean matches(String pattern, String path) {
        if (pattern.equals("/**")) return true;
        if (pattern.endsWith("/**")) {
            return path.startsWith(pattern.substring(0, pattern.length() - 3));
        }
        return pattern.equals(path);
    }

    /** hasRole("X") is hasAuthority("ROLE_X"). */
    static Predicate<Set<String>> hasRole(String role) {
        String authority = "ROLE_" + role;
        return a -> a.contains(authority);
    }

    static String decide(List<Rule> rules, String path, Set<String> granted, boolean anonymous) {
        Set<String> a = expand(granted);
        for (Rule r : rules) {
            if (matches(r.pattern(), path)) {
                boolean ok = r.access().test(a);
                String outcome = ok ? "200 allowed"
                    : anonymous ? "401 (entry point)" : "403 (denied handler)";
                return outcome + "   [first match: " + r.pattern() + " " + r.description() + "]";
            }
        }
        return "no rule";
    }

    public static void main(String[] args) {
        List<Rule> correct = List.of(
            new Rule("/public/**", a -> true, "permitAll"),
            new Rule("/admin/**", hasRole("ADMIN"), "hasRole(ADMIN)"),
            new Rule("/**", a -> a.stream().anyMatch(x -> x.startsWith("ROLE_")), "authenticated"));
        List<Rule> wrongOrder = List.of(
            new Rule("/**", a -> a.stream().anyMatch(x -> x.startsWith("ROLE_")), "authenticated"),
            new Rule("/admin/**", hasRole("ADMIN"), "hasRole(ADMIN)"));

        Set<String> anon = Set.of();
        Set<String> user = Set.of("ROLE_USER");
        Set<String> admin = Set.of("ROLE_ADMIN");
        Set<String> unprefixed = Set.of("ADMIN");   // authority mapped from a token claim

        System.out.println("-- correct order");
        System.out.println("anonymous  /admin/users : "
            + decide(correct, "/admin/users", anon, true));
        System.out.println("USER       /admin/users : "
            + decide(correct, "/admin/users", user, false));
        System.out.println("ADMIN      /admin/users : "
            + decide(correct, "/admin/users", admin, false));
        System.out.println("USER       /orders      : "
            + decide(correct, "/orders", user, false));
        System.out.println("-- /** placed before /admin/**");
        System.out.println("USER       /admin/users : "
            + decide(wrongOrder, "/admin/users", user, false));
        System.out.println("-- authority ADMIN (no prefix) against hasRole(ADMIN)");
        System.out.println("ADMIN      /admin/users : "
            + decide(correct, "/admin/users", unprefixed, false));
    }
}
```

Ordered rules, first match wins, 401 versus 403 TERMINAL

```
$ java RuleMatcher.java
-- correct order
anonymous  /admin/users : 401 (entry point)   [first match: /admin/** hasRole(ADMIN)]
USER       /admin/users : 403 (denied handler)   [first match: /admin/** hasRole(ADMIN)]
ADMIN      /admin/users : 200 allowed   [first match: /admin/** hasRole(ADMIN)]
USER       /orders      : 200 allowed   [first match: /** authenticated]
-- /** placed before /admin/**
USER       /admin/users : 200 allowed   [first match: /** authenticated]
-- authority ADMIN (no prefix) against hasRole(ADMIN)
ADMIN      /admin/users : 403 (denied handler)   [first match: /admin/** hasRole(ADMIN)]
```

401 versus 403

The two status codes answer different questions. **401 Unauthorized** means that the request lacks valid authentication; it should carry a `WWW-Authenticate` header naming the scheme. **403 Forbidden** means that the caller is known and the action is not permitted. `ExceptionTranslationFilter` decides between them: it catches `AccessDeniedException`; if the current authentication is anonymous (or remember-me), it starts authentication through the `AuthenticationEntryPoint` (401, or a redirect to the login page for browser chains); otherwise it delegates to the `AccessDeniedHandler` (403). An `AuthenticationException` thrown by an authentication filter, such as an invalid token, goes to the entry point directly.

The consequence for API design is that the entry point must match the client type. A browser chain redirects to `/login` with status 302, which is meaningless for a JSON client; an API chain should use `HttpStatusEntryPoint` or `BearerTokenAuthenticationEntryPoint`, which returns 401 with `WWW-Authenticate: Bearer`. Authentication that succeeded followed by 403 never produces a redirect.

Method security

Request rules protect URLs; they cannot protect a service method that is reachable from several controllers, from messaging listeners or from scheduled jobs. Method security attaches authorisation to the method itself. It is activated with `@EnableMethodSecurity` (Spring Security 5.6 and later; it supersedes `@EnableGlobalMethodSecurity`) and enables `@PreAuthorize`, `@PostAuthorize`, `@PreFilter` and `@PostFilter` by default; `@Secured` and JSR-250 annotations require `securedEnabled = true` and `jsr250Enabled = true`. Spring Boot does not enable it automatically.

The implementation is ordinary Spring AOP. `@EnableMethodSecurity` registers advisors whose interceptors are `AuthorizationManagerBeforeMethodInterceptor` (for `@PreAuthorize`) and `AuthorizationManagerAfterMethodInterceptor` (for `@PostAuthorize`). The expression is evaluated by `PreAuthorizeAuthorizationManager` against a `MethodSecurityExpressionRoot` that offers `hasRole`, `hasAuthority`, `authentication`, `principal`, the method arguments by name (`#id`) and, in `@PostAuthorize`, `returnObject`. A denied call raises `AuthorizationDeniedException`, a subclass of `AccessDeniedException` introduced in Spring Security 6.3.

Because the mechanism is a proxy, three limits apply, all inherited from the AOP chapter of the container part: calls from inside the same object (`this.method()`) bypass the proxy; `private` methods and `final` classes or methods cannot be intercepted by the CGLIB subclass; and a bean created with `new` is not proxied at all. The annotation is silently ignored in each case, which is why a test that asserts the denial is mandatory.

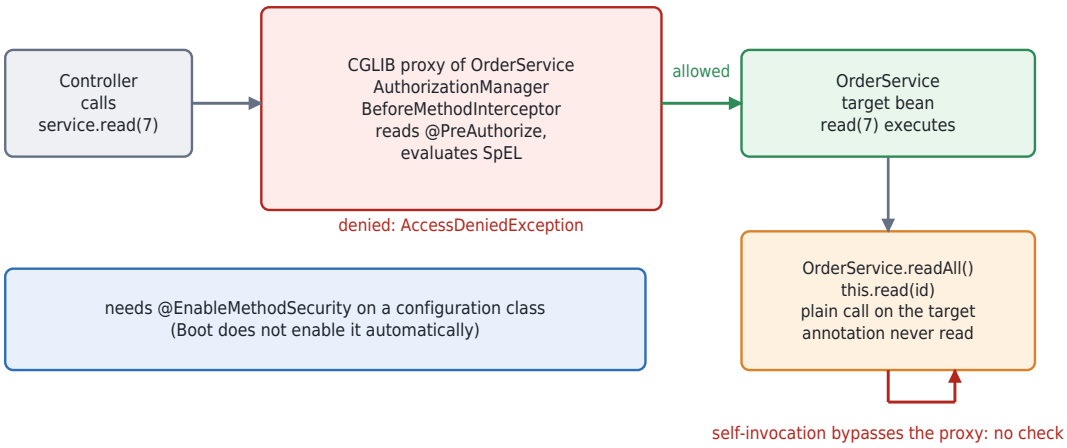


Figure 3.7. Method security sits in a proxy in front of the bean; a call made through this inside the target never crosses the proxy, so the annotation is not evaluated.

The interceptors are ordered through `AuthorizationInterceptorsOrder`: `@PreFilter` 100, `@PreAuthorize` 200, `@Secured` 300, JSR-250 400, `@PostAuthorize` 500, `@PostFilter` 600. `TransactionInterceptor` defaults to the lowest precedence, so pre-authorisation runs before the transaction opens, which is desirable, whereas `@PostAuthorize` runs after the transaction has committed. A `@PostAuthorize` on a method that writes data therefore does not roll back the write when the check fails; the reference documentation advises against that combination. Annotations on one method are combined conjunctively, and repeating the same annotation is not supported; boolean operators in one expression serve instead.

Ownership and the permission model

A role check answers "may this kind of caller perform this kind of action". Most applications also need "may this caller act on this record". Omitting the second question is the defect class known as insecure direct object reference (IDOR) and is listed by OWASP as broken object-level authorisation, the first item of the API Security Top 10. There are four implementation techniques, ordered by preference.

1. **Query-level scoping.** The repository query includes the owner: `findByIdAndOwner(id, owner)`. A record owned by someone else is simply not found, so the endpoint answers 404 and does not confirm that the identifier exists. This is the strongest option because the data is never loaded.
2. **A bean-based expression.** `@PreAuthorize("@orderAuthz.canCancel(#id, authentication)")` delegates to a Spring bean that may consult the database and the business state. It runs before the method and is testable in isolation.
3. `@PostAuthorize`**.** `@PostAuthorize("returnObject.owner == authentication.name")` checks the loaded object. The record is loaded before the decision, which costs a query and, with a side-effecting method, is unsafe.
4. **A** `PermissionEvaluator`**.** `hasPermission(#id, 'Order', 'read')` routes to a registered evaluator. It is suitable when permissions are stored in a table (access-control lists), and it is heavier than the other options.

Every technique reads the identity from the `Authentication`, never from a request parameter or body field. A body containing `"owner": "bob"` that is bound to the entity allows mass assignment of ownership (see the chapter on web attacks).

3.2.2 Worked example

The example secures an order API. Rules: product browsing is public; ordering requires the user role; deletion and the `/api/admin` area require the administrator role; the administrator inherits the user role; a user sees and cancels only their own orders. The listings continue the `com.example.shop` package of the previous chapter.

Order.java (accessors omitted) JAVA

```java
package com.example.shop.order;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.math.BigDecimal;

@Entity
@Table(name = "orders")
public class Order {

    public enum Status { NEW, PAID, SHIPPED, CANCELLED }

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, updatable = false, length = 60)
    private String owner;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal total;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Status status = Status.NEW;
```

Order.java (accessors omitted) (continued) JAVA

```
    protected Order() {
    }

    public Order(String owner, BigDecimal total) {
        this.owner = owner;
        this.total = total;
    }

    public void cancel() {
        if (status != Status.NEW) {
            throw new IllegalStateException("only NEW orders can be cancelled");
        }
        status = Status.CANCELLED;
    }

    // getters: getId, getOwner, getTotal, getStatus
}
```

OrderRepository.java JAVA

```
package com.example.shop.order;

import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, Long> {

    Optional<Order> findByIdAndOwner(Long id, String owner);

    List<Order> findByOwnerOrderByIdDesc(String owner);
}
```

The service combines three of the techniques so that they can be compared. `listAll` uses a role expression, `get` uses query-level scoping with an administrator exception, and `cancel` uses a bean expression.

OrderService.java JAVA

```
package com.example.shop.order;

import java.math.BigDecimal;
import java.util.List;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    public record OrderView(long id, String owner, BigDecimal total, String status) {
        static OrderView of(Order o) {
            return new OrderView(o.getId(), o.getOwner(), o.getTotal(), o.getStatus().name());
        }
    }

    private final OrderRepository orders;

    public OrderService(OrderRepository orders) {
        this.orders = orders;
    }

    @PreAuthorize("hasRole('ADMIN')")
    @Transactional(readOnly = true)
    public List<OrderView> listAll() {
        return orders.findAll().stream().map(OrderView::of).toList();
    }

    @PreAuthorize("hasRole('USER')")
    @Transactional(readOnly = true)
    public List<OrderView> listMine() {
        String me = currentUser().getName();
```

Sheeram

OrderService.java (continued) JAVA

```
        return orders.findByOwnerOrderByIdDesc(me).stream().map(OrderView::of).toList();
    }

    @PreAuthorize("hasRole('USER')")
    @Transactional(readOnly = true)
    public OrderView get(long id) {
        Authentication a = currentUser();
        boolean admin = a.getAuthorities().stream()
                .anyMatch(g -> g.getAuthority().equals("ROLE_ADMIN"));
        return (admin ? orders.findById(id) : orders.findByIdAndOwner(id, a.getName()))
                .map(OrderView::of)
                .orElseThrow(() -> new OrderNotFoundException(id));
    }

    @PreAuthorize("@orderAuthz.canCancel(#id, authentication)")
    @Transactional
    public void cancel(long id) {
        orders.findById(id).orElseThrow(() -> new OrderNotFoundException(id)).cancel();
    }

    private static Authentication currentUser() {
        return SecurityContextHolder.getContext().getAuthentication();
    }
}
```

OrderNotFoundException.java JAVA

```
package com.example.shop.order;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class OrderNotFoundException extends RuntimeException {

    public OrderNotFoundException(long id) {
        super("order " + id + " not found");
    }
}
```

The authorisation bean reads the state of the order and the owner. It is registered under the name `orderAuthz`, which the SpEL expression references with `@`. The expression receives the method argument `#id` by name; parameter names are available because Spring Boot's Maven and Gradle plugins compile with `-parameters`.

OrderAuthz.java JAVA

```
package com.example.shop.order;

import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

@Component("orderAuthz")
public class OrderAuthz {

    private final OrderRepository orders;

    public OrderAuthz(OrderRepository orders) {
        this.orders = orders;
    }

    @Transactional(readOnly = true)
    public boolean canCancel(long id, Authentication auth) {
        boolean admin = auth.getAuthorities().stream()
                .anyMatch(g -> g.getAuthority().equals("ROLE_ADMIN"));
        return orders.findById(id)
                .map(o -> admin || o.getOwner().equals(auth.getName()))
                .orElse(true);   // unknown id: let the service answer 404, not 403
    }
}
```

The controller contains no authorisation logic. A comment in the service documents each rule, and the URL rules in the configuration act as a coarse first barrier.

OrderController.java JAVA

```java
package com.example.shop.order;

import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/orders")
public class OrderController {

    private final OrderService service;

    public OrderController(OrderService service) {
        this.service = service;
    }

    @GetMapping
    public List<OrderService.OrderView> mine() {
        return service.listMine();
    }

    @GetMapping("/{id}")
    public OrderService.OrderView one(@PathVariable long id) {
        return service.get(id);
    }

    @PostMapping("/{id}/cancel")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void cancel(@PathVariable long id) {
        service.cancel(id);
    }
}
```

The configuration has two parts. The method security configuration enables the annotations and registers the role hierarchy through the expression handler. The `static` modifier is required for both beans: the handler must be available before Spring Security's method security configuration is initialised.

MethodSecurityConfig.java JAVA

```java
package com.example.shop.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.access.hierarchicalroles.RoleHierarchy;
import org.springframework.security.access.hierarchicalroles.RoleHierarchyImpl;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.access.expression.method.DefaultMethodSecurityExpressionHandler;
import org.springframework.security.access.expression.method.MethodSecurityExpressionHandler;

@Configuration
@EnableMethodSecurity
public class MethodSecurityConfig {

    @Bean
    static RoleHierarchy roleHierarchy() {
        return RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER");
    }

    @Bean
    static MethodSecurityExpressionHandler methodSecurityExpressionHandler(
            RoleHierarchy hierarchy) {
        DefaultMethodSecurityExpressionHandler handler =
                new DefaultMethodSecurityExpressionHandler();
        handler.setRoleHierarchy(hierarchy);
        return handler;
    }
}
```

The request rules replace the single `anyRequest().authenticated()` of the previous chapter. The matchers run top to bottom and the final `denyAll()` makes unknown URLs inaccessible. The access check for `/actuator/**` shows a custom `AuthorizationManager` expressed as a lambda that inspects the client address.

ApiChain.java (fragment of SecurityConfig) JAVA

```java
@Bean
@Order(1)
SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
    IpAddressMatcher internal = new IpAddressMatcher("10.0.0.0/8");
    http.securityMatcher("/api/**", "/actuator/**")
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/auth/register").permitAll()
            .requestMatchers("/actuator/health/**").permitAll()
            .requestMatchers("/actuator/**").access((auth, ctx) ->
                new AuthorizationDecision(internal.matches(ctx.getRequest())))
            .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .requestMatchers(HttpMethod.DELETE, "/api/orders/**").hasRole("ADMIN")
            .requestMatchers("/api/**").hasRole("USER")
            .anyRequest().denyAll())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

Two details are worth noting. `hasRole("USER")` on `/api/**` admits administrators because of the hierarchy, and the DELETE rule precedes the general rule on purpose. In Spring Security 6.4 and later the interface method for a custom manager is named `authorize` and returns an `AuthorizationResult`; the lambda form above targets the `check` contract documented for 6.5 and stays source-compatible in the 6.x line.

3.2.3 Hands-on

With users `alice` (role USER), `bob` (role USER) and `root` (role ADMIN), and one order owned by alice, the following calls demonstrate each layer. The output is **typical output (not run on the build machine)**.

Authorisation outcomes (typical output, not run on the build machine) TERMINAL

```
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8080/api/orders/1
401
$ curl -s -o /dev/null -w '%{http_code}\n' -u alice:pw localhost:8080/api/orders/1
200
$ curl -s -o /dev/null -w '%{http_code}\n' -u bob:pw localhost:8080/api/orders/1
404
$ curl -s -o /dev/null -w '%{http_code}\n' -u bob:pw -X POST localhost:8080/api/orders/1/cancel
403
$ curl -s -o /dev/null -w '%{http_code}\n' -u alice:pw -X DELETE localhost:8080/api/orders/1
403
$ curl -s -o /dev/null -w '%{http_code}\n' -u root:pw -X DELETE localhost:8080/api/orders/1
204
$ curl -s -o /dev/null -w '%{http_code}\n' -u alice:pw localhost:8080/api/unknown
403
```

Bob receives 404 for a read (query-level scoping hides the record) and 403 for a cancel (the expression denies). The inconsistency is a design decision to be made deliberately: hiding existence is preferable for reads of private data, and an explicit 403 is acceptable where the identifier is not secret. The final call returns 403 rather than 404 because `denyAll()` decides before routing. With TRACE logging a denied method call shows `AuthorizationDeniedException: Access Denied` raised in `AuthorizationManagerBeforeMethodInterceptor` and handled by `ExceptionTranslationFilter`.

3.2.4 Real-world problems and solutions

Problem 5: 401 and 403 returned in the wrong situations

Scenario. A single-page client calls the API with an expired bearer token and receives an HTML login page with status 200 after a redirect; another endpoint answers 403 to requests that carry no credentials at all.

Symptoms. `fetch` follows a 302 to `/login` and returns HTML that the JSON parser cannot read. Monitoring shows 403 for anonymous traffic on some paths and 401 on others.

Diagnosis. Log `ExceptionTranslationFilter` at DEBUG; it prints `Sending ... to authentication entry point since access is denied` for anonymous callers. Identify which entry point is configured for the matched chain: the log line `Will secure ... with filters` shows which chain applies, and the entry point of a form-login chain is `LoginUrlAuthenticationEntryPoint`. A 403 for anonymous callers arises when `httpBasic`, `formLogin` and `oauth2ResourceServer` are all absent from the chain, because no entry point is registered and `Http403ForbiddenEntryPoint` is the fallback.

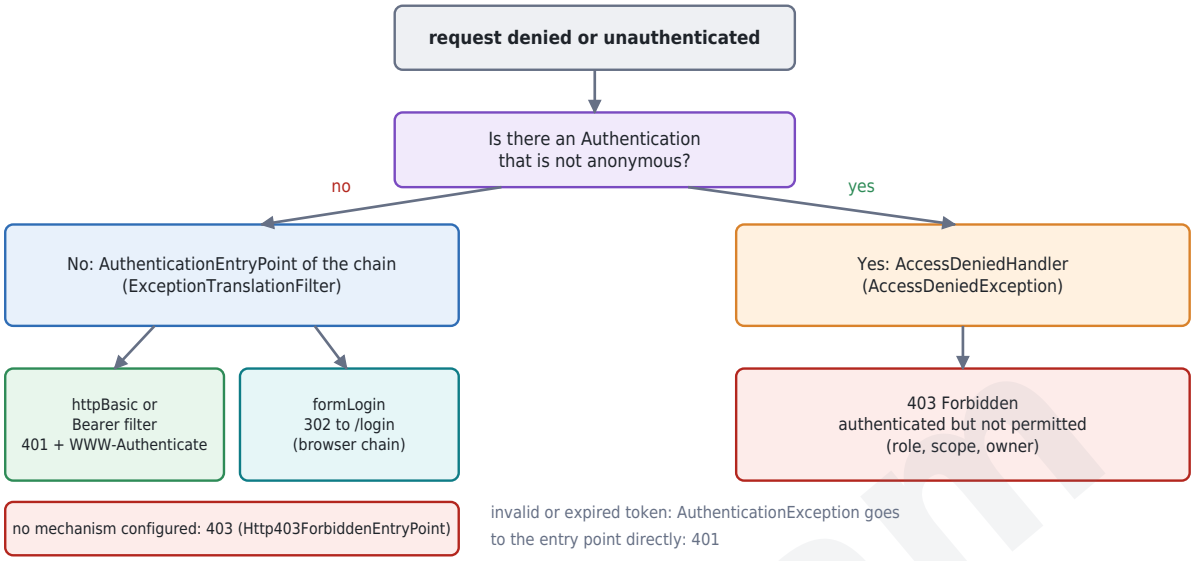


Figure 3.8. The decision path for 401 and 403: the outcome depends on whether authentication exists and which entry point the chain registers, not on the rule that failed.

Root cause. The API paths are served by a chain configured for browsers, or by a chain without any authentication mechanism.

Fix. Give API paths a dedicated chain whose entry point returns 401, and keep the redirect behaviour for page paths.

ApiEntryPoint.java (fragment) JAVA

```java
http.securityMatcher("/api/**")
    .exceptionHandling(e -> e
        .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED))
        .accessDeniedHandler((req, res, ex) -> res.sendError(HttpServletResponse.SC_FORBIDDEN)))
    .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));   // supplies Bearer entry point
```

Prevention. Test both codes explicitly for each chain: one request without credentials (expect 401) and one with insufficient authority (expect 403).

Problem 6: the role check never matches (prefix and mapping mismatch)

Scenario. After switching from a database login to JWT, every endpoint protected by `hasRole("ADMIN")` returns 403, although the token of the administrator contains `"roles": ["ADMIN"]`.

Symptoms. 403 for users who should pass; the same rule worked before the change.

Diagnosis. Print the authorities: a temporary controller method returns `authentication.getAuthorities()`. The output shows `[SCOPE_openid]` or `[ADMIN]`, but not `ROLE_ADMIN`. The plain-Java output above reproduces the effect: authority `ADMIN` against `hasRole(ADMIN)` is denied.

Root cause. `hasRole("ADMIN")` requires the authority `ROLE_ADMIN`. The default `JwtGrantedAuthoritiesConverter` reads only the `scope` or `scp` claims, and produces `SCOPE_` authorities; the `roles` claim is ignored.

Fix. Map the claim explicitly in a `JwtAuthenticationConverter`.

JwtRoleConverter.java (fragment) JAVA

```java
@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter roles = new JwtGrantedAuthoritiesConverter();
    roles.setAuthoritiesClaimName("roles");
    roles.setAuthorityPrefix("ROLE_");
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(roles);
    return converter;
}
```

Prevention. Document the authority naming scheme once (roles with prefix, permissions without), and assert in a test that the converter yields the expected authorities for a sample token.

Problem 7: method security is not applied

Scenario. A method annotated `@PreAuthorize("hasRole('ADMIN')")` is callable by any authenticated user. The annotation is present, the expression is correct, and no error is reported.

Symptoms. The method executes for unauthorised callers; no `AuthorizationDeniedException` appears in any log; unit tests that call the service through the Spring context confirm the bypass for some call paths and not for others.

Diagnosis. Work through the possible causes in this order. First, check whether `@EnableMethodSecurity` is present on a configuration class; Boot does not add it. Second, check whether the bean is proxied: `AopUtils.isAopProxy(bean)` or a debugger shows a class named `OrderService$$SpringCGLIB$$0`. Third, check the call path: a call from another method of the same class is a self-invocation. Fourth, check visibility and modifiers: private, static and final methods are not intercepted. Fifth, check whether the object was created with `new` or a `@Bean` method on a `static` factory executed before the infrastructure was ready. The diagram earlier in this chapter shows the self-invocation path.

Root cause. The authorisation is attached to the proxy, not to the method body.

Fix. Move the protected method into a separate bean or invoke it through the injected proxy; keep the annotation on public methods of non-final beans.

AdminReportService.java (fragment) JAVA

```java
@Service
public class AdminReportService {

    private final OrderService orders;   // injected proxy: calls pass the interceptor

    public AdminReportService(OrderService orders) {
        this.orders = orders;
    }

    public List<OrderService.OrderView> nightlyReport() {
        return orders.listAll();         // @PreAuthorize is evaluated here
    }
}
```

Prevention. A security test per protected method (`@WithMockUser(roles = "USER")` expecting `AccessDeniedException`) fails when the interception is missing. A catch-all request rule (`/api/admin/**` with `hasRole`) provides a second layer, since the reference documentation states that unannotated methods are not secured.

Problem 8: a user reads another user's data (broken object-level authorisation)

Scenario. `GET /api/orders/{id}` requires the user role. An authenticated customer iterates identifiers 1, 2, 3 and receives the orders of other customers.

Symptoms. Access logs show a single user requesting hundreds of sequential identifiers with status 200; there are no 401 or 403 responses to alert on.

Diagnosis. Review the endpoint: the controller calls `repository.findById(id)` without referring to the authentication. The condition is detectable by a test that authenticates as user B and requests a record of user A.

Root cause. The role check establishes that the caller is a customer; nothing establishes that the record belongs to that customer.

Fix. Scope the query by owner as in `OrderService.get`, or use a bean expression. Prefer non-guessable public identifiers (UUID) as defence in depth, never as the only control. Where records are shared among a team, resolve access through membership instead of a single owner column.

OwnershipTest.java (fragment) JAVA

```java
@Test
@WithMockUser(username = "bob", roles = "USER")
void bobCannotReadAlicesOrder() {
    long aliceOrderId = seed("alice");
    assertThatThrownBy(() -> service.get(aliceOrderId))
            .isInstanceOf(OrderNotFoundException.class);
}
```

Prevention. Include a cross-tenant negative test for every endpoint that takes an identifier; add an anomaly alert on sequential identifier access.

GOTCHA

A global `@RestControllerAdvice` with `@ExceptionHandler(Exception.class)` also receives `AccessDeniedException` and `AuthorizationDeniedException` thrown from method security, because those exceptions occur inside the controller call. The handler then converts a denial into a 500 or a 200 with an error body, and `ExceptionTranslationFilter` never sees it. Handlers must either exclude these types (declare a narrower handler that rethrows them) or map them deliberately to 403.

3.2.5 Questions to expect

1. **What is the difference between `hasRole("ADMIN")` and `hasAuthority("ADMIN")`?** `hasRole` prepends `ROLE_` and therefore matches the authority `ROLE_ADMIN`; `hasAuthority` compares the string exactly. Roles are authorities with a naming convention.
2. **In what order are `authorizeHttpRequests` rules evaluated?** In declaration order; the first matching rule decides and later rules are ignored, so specific rules precede general ones and the list should end with `anyRequest().denyAll()` or `authenticated()`.
3. **When does Spring answer 401 and when 403?** 401 when the request has no valid authentication and the entry point is invoked; 403 when an authenticated caller lacks permission. `ExceptionTranslationFilter` decides based on whether the current authentication is anonymous.
4. **How does `@PreAuthorize` work internally, and why can it be ignored?** An AOP advisor wraps the bean in a proxy whose interceptor evaluates the SpEL expression through an `AuthorizationManager` before delegating. Self-invocation, private or final methods, objects not managed by Spring and a missing `@EnableMethodSecurity` all bypass it.
5. **What is the difference between `@PreAuthorize` and `@PostAuthorize`?** The first runs before the method and can use arguments; the second runs after and can use `returnObject`, but the method has already executed and any side effects have occurred.
6. **How is object-level authorisation implemented?** By scoping the query by owner, by a bean-based expression, by `@PostAuthorize` on the result, or by a `PermissionEvaluator`; role checks alone do not prevent IDOR.
7. **How is a role hierarchy configured?** A `RoleHierarchy` bean (`ROLE_ADMIN > ROLE_USER`) for request rules, and the same hierarchy assigned to the `MethodSecurityExpressionHandler` for method security.
8. **Why is `anyRequest().denyAll()` recommended?** It converts the rule list into an allow list, so a newly added endpoint is inaccessible until a rule is written for it.

Check yourself

1. Rules: `requestMatchers("/**").authenticated()` followed by `requestMatchers("/admin/**").hasRole("ADMIN")`. Can a normal user call `/admin/x`?
2. A token has `"scope": "read write"`. Which authorities does the default converter create?
3. What is wrong with `@PostAuthorize("returnObject.owner == authentication.name")` on a method that deletes the order and returns it?
4. A service method `a()` annotated `@PreAuthorize("hasRole('ADMIN')")` is called by `b()` in the same class, and `b()` is called from a controller. Is the check performed?
5. Which status does an anonymous caller receive for a denied request on a chain with `httpBasic`, and which on a chain with `formLogin`?

Preview

Answers

1. Yes. The first rule matches every path and decides; the second is never reached (the plain-Java run above shows the same result).
2. `SCOPE_read` and `SCOPE_write`; no role authority exists, so `hasRole` cannot match.
3. The deletion has already happened when the check runs, and the transaction commits first because the post-authorisation interceptor is outside the transaction; a denial does not undo the write.
4. No. The call from `b()` is a self-invocation and bypasses the proxy.
5. 401 with `WWW-Authenticate: Basic` for the first; a 302 redirect to the login page for the second.

KEY POINTS

- Authorisation is a decision over `Authentication` plus a request or method; request rules are ordered and the first match wins, so a final `denyAll()` is the safe default.
- 401 means no valid authentication and 403 means insufficient permission; `ExceptionTranslationFilter` and the chain's entry point decide which one the client sees.
- `hasRole` adds the `ROLE_` prefix and `hasAuthority` does not; token claims must be mapped explicitly to the intended authority names.
- Method security is proxy-based: it is ignored for self-invocation, private or final methods and when `@EnableMethodSecurity` is missing; test the denial.
- Role checks do not protect records; scope queries by owner or use bean expressions to prevent broken object-level authorisation.

3.3 Web Attacks and Defences

Authentication and authorisation control who may call an endpoint. A separate class of attacks abuses the browser, the transport or the shape of the input rather than the identity check: a malicious page causes the browser to send authenticated requests (CSRF), a script reads data across origins, a session identifier is planted before login, an input string changes the meaning of a query, or a flood of requests exhausts the login endpoint. Spring Security provides defences for most of these by default, and several of those defaults are switched off by the most common configuration habit, `csrf.disable()`. This chapter states for each attack the precise condition under which it applies, the Spring mechanism that defends against it, and the configuration that keeps the defence working with a single-page client.

3.3.1 Concept and mechanism

Attack	Mechanism	Primary defence in Spring
CSRF	The browser attaches cookies or cached credentials to a request that a foreign page triggers	`CsrfFilter` with a token the foreign page cannot read; `SameSite` cookies
Cross-origin read	A foreign script reads a response using the victim's credentials	Same-origin policy; explicit CORS allow-list
Session fixation	The attacker sets a known session identifier, the victim authenticates, the identifier becomes privileged	`changeSessionId()` on login (default)
Clickjacking	The application is embedded in a hidden frame	`X-Frame-Options` or CSP `frame-ancestors`
XSS	Attacker-controlled markup runs in the page	Output encoding; Content-Security-Policy; `HttpOnly` cookies
Injection	Input is concatenated into a query or command	Parameter binding; allow-lists for identifiers
Mass assignment	A request body binds to fields that the client must not set	Dedicated request DTOs
Brute force and abuse	Unbounded attempts or request volume	Rate limiting and lock-out
Transport attacks	Credentials or tokens read or altered in transit	TLS and HSTS

Attack, mechanism and primary defence.

CSRF: when it applies and how the token works

Cross-site request forgery exists because browsers attach ambient credentials to requests regardless of the page that initiated them. A page on `evil.example` can submit a form to `bank.example/transfer`; the browser adds the session cookie of `bank.example`, and the server cannot distinguish the forged request from a genuine one. The condition for vulnerability is therefore precise: the application authenticates requests with credentials that the browser sends automatically (session cookies, HTTP Basic or client certificates) and performs state-changing operations on them. An API that authenticates only with an `Authorization: Bearer` header set by JavaScript is not exposed to this attack, because the foreign page cannot make the browser add that header, and there is no ambient credential to abuse. This is the sole legitimate reason to call `csrf.disable()`; the Spring Security reference states the same condition.

`CsrfFilter` implements the synchronizer-token pattern. On each request a `CsrfTokenRepository` provides the expected token (`HttpSessionCsrfTokenRepository`, the default, keeps it in the session; `CookieCsrfTokenRepository` keeps it in the `XSRF-TOKEN` cookie). For methods other than GET, HEAD, TRACE and OPTIONS the filter compares the expected value with the value submitted in the `X-CSRF-TOKEN` header (the cookie repository uses `X-XSRF-TOKEN`) or in the `_csrf` form parameter, and calls the `AccessDeniedHandler` with `InvalidCsrfTokenException` (403) on mismatch or absence. Since Spring Security 6 the token is loaded lazily: the session or cookie is touched only when a token is needed. Server-rendered forms obtain the token automatically from Thymeleaf's form support; a single-page application must read it from the cookie and send it in the header.

The default `XorCsrfTokenRequestAttributeHandler` masks the token with random data on every rendering to defeat BREACH-style compression attacks, so the value in a rendered page differs from request to request while remaining verifiable. A cookie-based single-page client reads the raw cookie value, which the masked handler cannot accept; the reference documentation therefore provides a `SpaCsrfTokenRequestHandler` that uses the masking handler for rendered values and the plain handler for the header (shown in the worked example). Cookies cleared at login and logout require the client to fetch a fresh token before the next unsafe request.

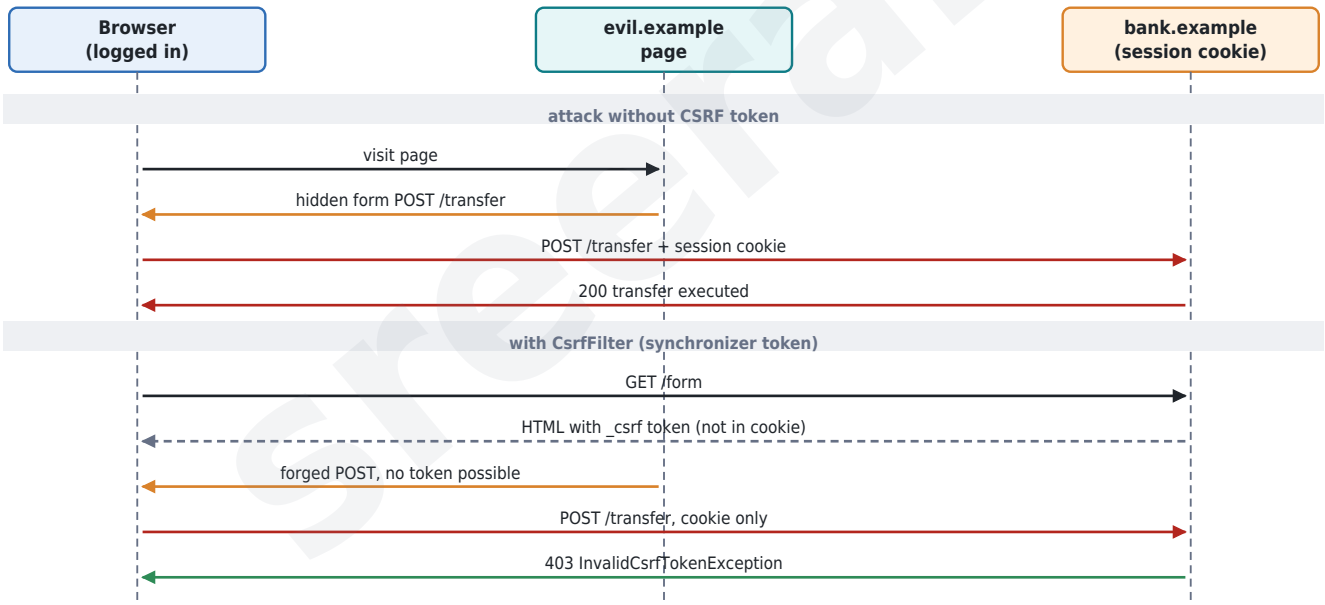


Figure 3.9. Without a token the forged POST succeeds because the browser attaches the session cookie; with CsrfFilter the page of the attacker cannot supply the token and receives 403.

`SameSite` cookies are a complementary defence: with `SameSite=Lax` (the default of current browsers) a cookie is withheld from cross-site POST requests. It reduces exposure but does not replace the token, because behaviour differs by browser version, `SameSite` relates to sites rather than origins (sibling subdomains remain trusted), and applications may need `SameSite=None` for legitimate cross-site embedding. In Spring Boot the attribute is set with `server.servlet.session.cookie.same-site`.

CORS: what it controls

The same-origin policy prevents a script from one origin reading responses from another. Cross-origin resource sharing relaxes it selectively through response headers. CORS is enforced by the browser, not by the server: a request from a non-browser client is unaffected, and a forged request from a foreign page is still *sent* (which is why CORS is not a CSRF defence); only the reading of the response is blocked. Requests that are not "simple" (any method other than GET, HEAD or POST with a simple content type, or any request with an `Authorization` header or JSON content type) are preceded by a **preflight**, an `OPTIONS` request carrying `Origin`, `Access-Control-Request-Method` and `Access-Control-Request-Headers`. The browser sends the preflight **without credentials**. The real request follows only if the preflight response carries matching `Access-Control-Allow-Origin`, `-Methods` and `-Headers`.

The security consequence is a classic ordering problem. If the security chain requires authentication for every request, the credential-less preflight is answered with 401, and the browser reports a CORS error even though the CORS configuration is correct. Spring Security solves it with `CorsFilter`, which is placed before the authentication and authorisation filters and answers preflights itself; it is added only when `http.cors(...)` is configured and a `CorsConfigurationSource` bean (or a `CorsFilter` bean) exists. A `@CrossOrigin` annotation or MVC-level `WebMvcConfigurer.addCorsMappings` configuration alone does not help, because the request never reaches Spring MVC.

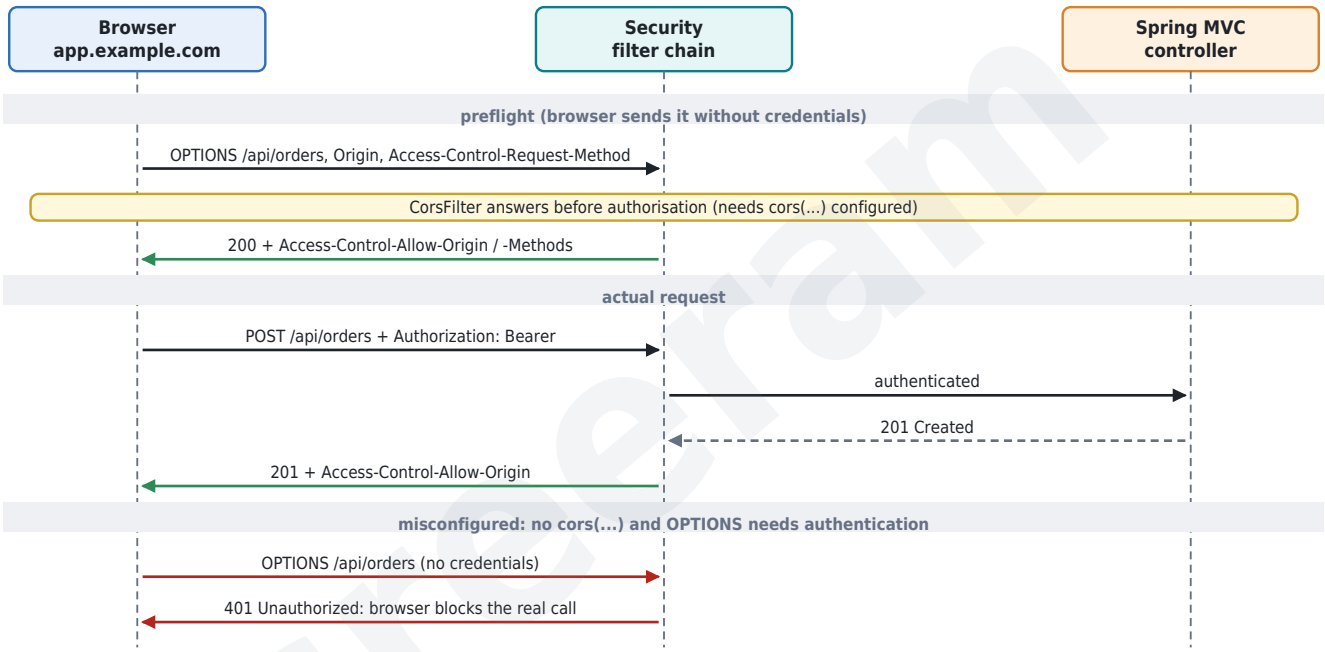


Figure 3.10. The preflight carries no credentials; it succeeds only if CorsFilter answers it before authorisation, as the first two exchanges show, whereas the third exchange shows the misconfigured chain.

Two rules constrain the configuration. `Access-Control-Allow-Credentials: true` cannot be combined with the wildcard origin `*`; the origin must be listed explicitly (or matched with `allowedOriginPatterns`). And reflecting the incoming `Origin` header without validation, which some copy-pasted filters do, is equivalent to disabling the protection.

Sessions, headers and transport

Session fixation. The attacker obtains a valid session identifier, causes the victim's browser to use it, and waits until the victim logs in. If the identifier is unchanged by authentication, the attacker's copy is now authenticated. Spring Security defends by default with `changeSessionId()` on the Servlet 3.1+ API, which keeps the attributes and issues a new identifier at login; `migrateSession` and `newSession` are alternatives. The related hygiene settings are `HttpOnly` (script cannot read the cookie), `Secure` (HTTPS only) and `SameSite`; in Boot they are `server.servlet.session.cookie.http-only`, `.secure` and `.same-site`. Concurrent-session limits are configured with `sessionConcurrency`.

Default response headers. `HeaderWriterFilter` adds `Cache-Control: no-cache, no-store, max-age=0, must-revalidate` with `Pragma` and `Expires`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `X-XSS-Protection: 0` (the legacy browser filter is disabled deliberately) and, only on HTTPS requests, `Strict-Transport-Security: max-age=31536000 ; includeSubDomains`. The header `Content-Security-Policy` is not sent by default and has to be configured.

XSS and CSP. Cross-site scripting is prevented first by output encoding at the template layer (Thymeleaf escapes with `th:text`; `th:utext` and string concatenation into HTML do not) and by never reflecting input into script contexts. A Content-Security-Policy limits damage when encoding fails: `default-src 'self'; object-src 'none'; frame-ancestors 'none'` forbids inline scripts, third-party script origins and framing. JSON APIs additionally return the correct `Content-Type` and `X-Content-Type-Options: nosniff`. `HttpOnly` prevents a successful script from reading the session cookie; it does not prevent the script from acting as the user.

Transport. TLS protects credentials and tokens in transit; HSTS instructs the browser to refuse plain HTTP for the host for the stated period. Behind a reverse proxy or ingress that terminates TLS, the application sees plain HTTP unless the forwarded headers are honoured; without `server.forward-headers-strategy=framework` (or `native`) HSTS is not emitted, redirects point to `http://`, and `Secure` cookies may be handled inconsistently.

Injection and mass assignment

An injection arises whenever data crosses into a language (SQL, JPQL, LDAP, shell) by string concatenation, so that the data can contain syntax. The defence is to keep code and data in separate channels: bound parameters for values, and an allow-list for identifiers (column names, sort fields) that cannot be bound. The following session against PostgreSQL 16, run on the build machine, executes the SQL that results from concatenation and then the same hostile inputs through a prepared statement, which is the form that JDBC, Hibernate and Spring Data use for bound parameters.

inj.sql SQL

```sql
CREATE TABLE app_user (id serial PRIMARY KEY, username text UNIQUE, password_hash text, role text);
INSERT INTO app_user (username, password_hash, role) VALUES
  ('alice', '{bcrypt}$2a$10$xxxxxxxxxxxxxxxxxxxxxx', 'USER'),
  ('bob',   '{bcrypt}$2a$10$yyyyyyyyyyyyyyyyyyyyyy', 'USER'),
  ('root',  '{bcrypt}$2a$10$zzzzzzzzzzzzzzzzzzzzzz', 'ADMIN');

\echo -- 1. concatenated query, input alice
SELECT username, role FROM app_user WHERE username = 'alice';
\echo -- 2. concatenated query, input is a tautology condition
SELECT username, role FROM app_user WHERE username = '' OR '1'='1';
\echo -- 3. concatenated query, input closes the quote and appends a UNION
SELECT username, role FROM app_user WHERE username = 'x'
  UNION SELECT username, password_hash FROM app_user --'
;
\echo -- 4. prepared statement with the same malicious inputs as data
PREPARE find_user(text) AS SELECT username, role FROM app_user WHERE username = $1;
EXECUTE find_user('alice');
EXECUTE find_user(''' OR ''1''=''1');
EXECUTE find_user('x'' UNION SELECT username, password_hash FROM app_user --');
```

Concatenation versus bound parameters (PostgreSQL 16, run on the build machine) TERMINAL

```
$ psql -d postgres -X -q -f inj.sql
-- 1. concatenated query, input alice
 username | role
----------+------
 alice    | USER
(1 row)

-- 2. concatenated query, input is a tautology condition
 username | role
----------+-------
 alice    | USER
 bob      | USER
 root     | ADMIN
(3 rows)

-- 3. concatenated query, input closes the quote and appends a UNION
 username |                 role
----------+--------------------------------------
 alice    | {bcrypt}$2a$10$xxxxxxxxxxxxxxxxxxxxxx
 bob      | {bcrypt}$2a$10$yyyyyyyyyyyyyyyyyyyyyy
 root     | {bcrypt}$2a$10$zzzzzzzzzzzzzzzzzzzzzz
(3 rows)

-- 4. prepared statement with the same malicious inputs as data
 username | role
----------+------
 alice    | USER
(1 row)
```

Sriram

Concatenation versus bound parameters (PostgreSQL 16, run on the build machine) (continued) TERMINAL

```
 username | role
----------+------
(0 rows)

 username | role
----------+------
(0 rows)
```

Query 2 returns every row; query 3 appends the password hashes to the result set through a `UNION`; the prepared statement treats both inputs as literal user names and returns nothing. In Spring Data, derived queries, `@Query` with `:name` or `?1` parameters, and `JdbcTemplate` with `?` placeholders are safe; string concatenation into `@Query(nativeQuery = true)`, `createQuery`, `JdbcTemplate.query` or `Sort` expressions built from request parameters is not.

Mass assignment is the structural counterpart: the framework binds all properties present in the request body to the target object, so a client that adds `"role":"ADMIN"` to a profile update elevates its own privileges if the body is bound to the entity. The defence is to bind requests only to purpose-built DTOs that contain exactly the fields the client may set, and to copy them explicitly.

Rate limiting with a token bucket

A token bucket allows a sustained rate with a bounded burst: the bucket holds at most `capacity` tokens, tokens are added at `refill` per second, and each request removes one. An empty bucket means the request is rejected with 429 and a `Retry-After` header. The state per key is two numbers (token count and last refill time), which makes the algorithm cheap enough to run in a filter. The following program runs the algorithm with an injectable clock so that its behaviour is deterministic: a client sending 20 requests per second against a limit of 5 per second with a burst of 10.

TokenBucketDemo.java JAVA

```java
import java.util.LinkedHashMap;
import java.util.Map;

public class TokenBucketDemo {
    /** Token bucket with an injectable clock (milliseconds) so that the run is deterministic. */
    static final class TokenBucket {
        private final long capacity;
        private final double refillPerMs;
        private double tokens;
        private long lastMs;

        TokenBucket(long capacity, double refillPerSecond, long nowMs) {
            this.capacity = capacity;
            this.refillPerMs = refillPerSecond / 1000.0;
            this.tokens = capacity;
            this.lastMs = nowMs;
        }

        synchronized boolean tryAcquire(long nowMs) {
            tokens = Math.min(capacity, tokens + (nowMs - lastMs) * refillPerMs);
            lastMs = nowMs;
            if (tokens >= 1.0) {
                tokens -= 1.0;
                return true;
            }
            return false;
        }

        synchronized long retryAfterMs() {
            return (long) Math.ceil((1.0 - tokens) / refillPerMs);
        }
    }

    public static void main(String[] args) {
        // 5 requests/second sustained, burst of 10, one bucket per client key
        Map<String, TokenBucket> buckets = new LinkedHashMap<>();
        int allowed = 0, rejected = 0;
        System.out.println("client A sends 40 requests, one every 50 ms (20 req/s) for 2 s");
        for (int i = 0; i < 40; i++) {
            long now = i * 50L;
            TokenBucket b = buckets.computeIfAbsent("A", k -> new TokenBucket(10, 5, now));
            boolean ok = b.tryAcquire(now);
            if (ok) allowed++; else rejected++;
            if (i < 2 || (i >= 9 && i <= 14) || i % 10 == 0) {
```

TokenBucketDemo.java (continued) JAVA

```
                System.out.printf("t=%4d ms  request %2d  %s%s%n", now, i + 1, ok ? "200" : "429",
                    ok ? "" : "  Retry-After ~" + b.retryAfterMs() + " ms");
            }
        }
        System.out.println("allowed=" + allowed + " rejected=" + rejected);
        System.out.println("expected allowed = burst 10 + 5 req/s * 1.95 s = about 19");

        TokenBucket other = buckets.computeIfAbsent("B", k -> new TokenBucket(10, 5, 2000));
        System.out.println("client B first request at t=2000 ms: " + (other.tryAcquire(2000)
                ? "200" : "429")
            + "  (buckets are independent per key)");
    }
}
```

Token bucket: 10 burst, 5 per second sustained TERMINAL

```
$ java TokenBucketDemo.java
client A sends 40 requests, one every 50 ms (20 req/s) for 2 s
t=   0 ms  request  1  200
t=  50 ms  request  2  200
t= 450 ms  request 10  200
t= 500 ms  request 11  200
t= 550 ms  request 12  200
t= 600 ms  request 13  200
t= 650 ms  request 14  429  Retry-After ~150 ms
t= 700 ms  request 15  429  Retry-After ~100 ms
t=1000 ms  request 21  200
t=1500 ms  request 31  429  Retry-After ~100 ms
allowed=19 rejected=21
expected allowed = burst 10 + 5 req/s * 1.95 s = about 19
client B first request at t=2000 ms: 200  (buckets are independent per key)
```

The first 13 requests pass (10 from the full bucket plus 3 refilled during the first 650 ms), after which rejections begin with a computed `Retry-After`; over two seconds 19 of 40 requests pass, consistent with the formula in the output. A second client has its own bucket. The key determines the fairness of the limit: the authenticated user name protects against one account being abused, the source address protects unauthenticated endpoints such as login, and a limit by address alone penalises users behind a shared proxy.

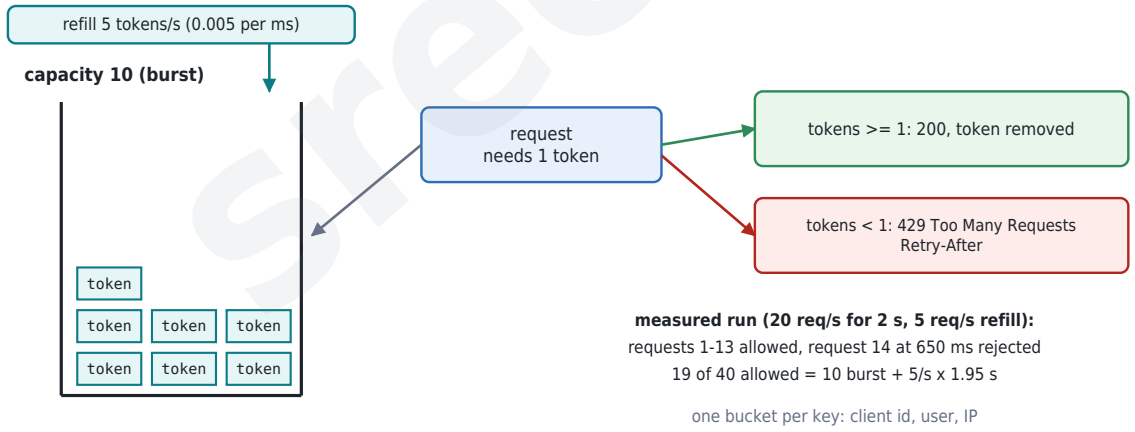


Figure 3.11. The bucket refills continuously and each request consumes one token; the measured run shows the burst phase followed by the sustained rate.

In-process limiters apply per instance: with four replicas a limit of five requests per second becomes up to twenty for the service as a whole. Limits that must be global belong in the gateway or in a shared store such as Redis (see the chapters on gateways and on Redis caching). The in-process filter below is appropriate as a safety net for the login endpoint and for single-instance services.

3.3.2 Worked example

The web chain below serves a single-page client that authenticates with a server session. It enables CSRF with the cookie repository and the SPA handler from the reference documentation, configures CORS for one origin, hardens headers, limits login attempts per address, and keeps session fixation protection at its default. The API chain from the previous chapters stays separate and stateless.

SpaCsrfTokenRequestHandler.java JAVA

```java
package com.example.shop.config;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.util.function.Supplier;
import org.springframework.security.web.csrf.CsrfToken;
import org.springframework.security.web.csrf.CsrfTokenRequestAttributeHandler;
import org.springframework.security.web.csrf.CsrfTokenRequestHandler;
import org.springframework.security.web.csrf.XorCsrfTokenRequestAttributeHandler;
import org.springframework.util.StringUtils;

/** Plain token from the X-XSRF-TOKEN header, masked token from rendered forms. */
final class SpaCsrfTokenRequestHandler implements CsrfTokenRequestHandler {

    private final CsrfTokenRequestHandler plain = new CsrfTokenRequestAttributeHandler();
    private final CsrfTokenRequestHandler xor = new XorCsrfTokenRequestAttributeHandler();

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                       Supplier<CsrfToken> csrfToken) {
        this.xor.handle(request, response, csrfToken);
        csrfToken.get();   // forces the deferred token to load, which writes the cookie
    }

    @Override
    public String resolveCsrfTokenValue(HttpServletRequest request, CsrfToken csrfToken) {
        String header = request.getHeader(csrfToken.getHeaderName());
        return (StringUtils.hasText(header) ? this.plain : this.xor)
                .resolveCsrfTokenValue(request, csrfToken);
    }
}
```

The rate limiter has two parts. The state lives in a Spring bean so that idle buckets can be evicted on a schedule (an unbounded map keyed by client address is itself a memory-exhaustion vector); the filter is a plain object created inside the security configuration, so that Boot does not register it a second time with the container.

RateLimiter.java JAVA

```java
package com.example.shop.security;

import java.time.Duration;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class RateLimiter {

    static final class Bucket {
        private final long capacity;
        private final double refillPerNano;
        private double tokens;
        private long lastNanos;
        private volatile long lastUsedNanos;

        Bucket(long capacity, double perSecond, long now) {
            this.capacity = capacity;
            this.refillPerNano = perSecond / 1_000_000_000.0;
            this.tokens = capacity;
            this.lastNanos = now;
            this.lastUsedNanos = now;
        }

        synchronized boolean tryAcquire(long now) {
            tokens = Math.min(capacity, tokens + (now - lastNanos) * refillPerNano);
```

RateLimiter.java (continued) JAVA

```
            lastNanos = now;
            lastUsedNanos = now;
            if (tokens >= 1.0) {
                tokens -= 1.0;
                return true;
            }
            return false;
        }

        synchronized long retryAfterSeconds() {
            return Math.max(1, (long) Math.ceil((1.0 - tokens) / (refillPerNano * 1e9)));
        }
    }

    private final ConcurrentMap<String, Bucket> buckets = new ConcurrentHashMap<>();
    private static final long CAPACITY = 10;
    private static final double PER_SECOND = 5;

    /** Returns 0 when the request is allowed, otherwise the number of seconds to wait. */
    public long check(String key) {
        long now = System.nanoTime();
        Bucket b = buckets.computeIfAbsent(key, k -> new Bucket(CAPACITY, PER_SECOND, now));
        return b.tryAcquire(now) ? 0 : b.retryAfterSeconds();
    }

    @Scheduled(fixedDelay = 60_000)
    void evictIdle() {
        long cutoff = System.nanoTime() - Duration.ofMinutes(10).toNanos();
        buckets.values().removeIf(b -> b.lastUsedNanos - cutoff < 0);
    }
}
```

RateLimitFilter.java JAVA

```
package com.example.shop.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import org.springframework.http.MediaType;
import org.springframework.web.filter.OncePerRequestFilter;

/** Not a Spring bean: constructed in the security configuration. */
public class RateLimitFilter extends OncePerRequestFilter {

    private final RateLimiter limiter;

    public RateLimitFilter(RateLimiter limiter) {
        this.limiter = limiter;
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        return !(request.getRequestURI().equals("/login")
                || request.getRequestURI().startsWith("/api/auth/"));
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws ServletException, IOException {
        long wait = limiter.check(request.getRemoteAddr());   // see forward-headers-strategy
        if (wait == 0) {
            chain.doFilter(request, response);
            return;
        }
        response.setStatus(429);
        response.setHeader("Retry-After", Long.toString(wait));
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.getWriter().write("{\"error\":\"too_many_requests\"}");
    }
}
```

sreeram

`request.getRemoteAddr()` is the address of the direct peer. Behind a trusted proxy it is the proxy address, and every client shares one bucket; with `server.forward-headers-strategy=framework` the container resolves the client address from `X-Forwarded-For` for requests from trusted proxies. Trusting that header from the open internet lets an attacker choose a new key with every request, so the trust boundary has to be configured, not assumed.

WebChain.java (fragment of SecurityConfig) JAVA

```java
@Bean
@Order(2)
SecurityFilterChain webChain(HttpSecurity http, RateLimiter limiter) throws Exception {
    http
        .cors(Customizer.withDefaults())            // uses the CorsConfigurationSource bean
        .csrf(csrf -> csrf
            .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
            .csrfTokenRequestHandler(new SpaCsrfTokenRequestHandler()))
        .addFilterBefore(new RateLimitFilter(limiter), UsernamePasswordAuthenticationFilter.class)
        .sessionManagement(s -> s
            .sessionFixation(f -> f.changeSessionId())        // the default, stated explicitly
            .sessionConcurrency(c -> c.maximumSessions(3)))
        .headers(h -> h
            .contentSecurityPolicy(csp -> csp.policyDirectives(
                "default-src 'self'; object-src 'none'; frame-ancestors 'none'"))
            .referrerPolicy(r -> r.policy(ReferrerPolicy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN))
            .httpStrictTransportSecurity(hsts -> hsts
                .includeSubDomains(true).maxAgeInSeconds(31_536_000)))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/", "/login", "/assets/**").permitAll()
            .anyRequest().authenticated())
        .formLogin(Customizer.withDefaults())
        .logout(Customizer.withDefaults());
    return http.build();
}
```

CorsConfig.java JAVA

```java
package com.example.shop.config;

import java.time.Duration;
import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
public class CorsConfig {

    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration c = new CorsConfiguration();
        // explicit origin; never "*" together with credentials
        c.setAllowedOrigins(List.of("https://app.example.com"));
        c.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
        c.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-XSRF-TOKEN"));
        c.setAllowCredentials(true);
        c.setMaxAge(Duration.ofHours(1));                          // browsers cache the preflight
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", c);
        return source;
    }
}
```

application.yml (fragment) YAML

```yaml
server:
  forward-headers-strategy: framework        # honour X-Forwarded-* from the ingress
  servlet:
    session:
      timeout: 30m
      cookie:
        http-only: true
        secure: true
        same-site: lax
        name: SHOPSESSION
```

The mass-assignment defence is a matter of types. The vulnerable form binds the body to the entity; the safe form binds it to a record and copies two fields.

ProfileController.java JAVA

```java
package com.example.shop.user;

import jakarta.validation.Valid;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import java.security.Principal;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/profile")
public class ProfileController {

    /** Exactly the fields the client may change; role, enabled and id are absent. */
    public record ProfileUpdate(@NotBlank String displayName, @Email String email) {
    }

    private final ProfileService profiles;

    public ProfileController(ProfileService profiles) {
        this.profiles = profiles;
    }

    @PutMapping
    public void update(Principal principal, @Valid @RequestBody ProfileUpdate body) {
        profiles.update(principal.getName(), body.displayName(), body.email());
    }

    // VULNERABLE variant, for contrast:
    // @PutMapping public AppUser update(@RequestBody AppUser body) { return repo.save(body); }
}
```

3.3.3 Hands-on

The header defaults and the CORS handshake can be checked with `curl`. The output is **typical output (not run on the build machine)**.

Response headers and a preflight (typical output, not run on the build machine) TERMINAL

```
$ curl -sI https://app.example.com/api/orders -H 'Authorization: Bearer x'
HTTP/1.1 401
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Strict-Transport-Security: max-age=31536000 ; includeSubDomains
WWW-Authenticate: Bearer

$ curl -si -X OPTIONS https://api.example.com/api/orders \
    -H 'Origin: https://app.example.com' \
    -H 'Access-Control-Request-Method: POST' \
    -H 'Access-Control-Request-Headers: authorization,content-type'
HTTP/1.1 200
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET,POST,PUT,DELETE
Access-Control-Allow-Headers: authorization,content-type
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 3600
```

A CSRF failure is visible both in the response and at DEBUG level, where `CsrfFilter` logs `Invalid CSRF token found for http://localhost:8080/api/orders` or, when no session exists, `Could not verify the provided CSRF token because your session was not found`.

3.3.4 Real-world problems and solutions

Problem 9: the browser reports a CORS error although CORS is configured

Scenario. The frontend at `https://app.example.com` calls `POST https://api.example.com/api/orders` with an `Authorization` header. The browser console reports that the preflight response did not pass the access control check. `@CrossOrigin` is present on the controller and the request works from `curl`.

Symptoms. The network panel shows an `OPTIONS` request with status 401 and no `Access-Control-Allow-Origin` header; the actual POST is never sent.

Diagnosis. Reproduce the preflight with `curl -i -X OPTIONS` including the `Origin` and `Access-Control-Request-Method` headers and no credentials. A 401 or 403 means that the security chain rejected it. Confirm with the startup filter list that `CorsFilter` is missing, and in the TRACE log that `AuthorizationFilter` denied an `OPTIONS` request. The third exchange of the preflight diagram shows this sequence.

Root cause. The `@CrossOrigin` annotation is processed by Spring MVC, which the request never reaches. The security chain requires authentication for every request, and the preflight is sent without credentials by specification.

Fix. Configure CORS in the security chain with a `CorsConfigurationSource` bean and `http.cors(Customizer.withDefaults())`, as in the worked example. Do not permit all `OPTIONS` requests globally as a substitute.

Prevention. Keep one source of CORS configuration, the security chain; list origins per environment in configuration properties; add a `MockMvc` test that performs an `OPTIONS` request with an `Origin` header and expects the allow headers.

Problem 10: POST returns 403 after enabling Spring Security

Scenario. A form post or `fetch` call that worked before security was added now fails with 403 on every POST, PUT and DELETE; GET requests are fine.

Symptoms. Response status 403 with an empty or generic body; the DEBUG log shows `Invalid CSRF token found for ...` or `Could not verify the provided CSRF token because your session was not found`.

Diagnosis. Inspect the request: is a token sent in the `X-CSRF-TOKEN` or `X-XSRF-TOKEN` header or in a `_csrf` field? Is the session cookie present (a missing cookie yields the second message)? Is the cookie `XSRF-TOKEN` set in a previous response, and does the client copy it into the header? For a pure token-based API the filter should not be active at all.

Root cause. One of three situations: a server-rendered form lacks the hidden token field; a single-page client does not send the token header; or a stateless API chain authenticated by bearer token still has CSRF enabled.

Fix. For forms use the template integration that emits `_csrf`. For a single-page client use the cookie repository with `SpaCsrfTokenRequestHandler`, read the `XSRF-TOKEN` cookie and send it as `X-XSRF-TOKEN`, and fetch the token again after login and logout because the cookie is cleared. For a stateless bearer-token API, disable CSRF **only on that chain**, which is justified because no ambient credentials exist. A broad `ignoringRequestMatchers("/api/**")` on a chain that also accepts session cookies is the dangerous variant.

Prevention. Document per chain which credential type it accepts and derive the CSRF setting from it; add a test that a session-authenticated POST without a token receives 403.

Problem 11: a client elevates its own role through a profile update

Scenario. `PUT /api/profile` accepts the `AppUser` entity as body and saves it. A tester sends `{"displayName":"x","role":"ADMIN"}` (or the property that backs the role set) and subsequently has administrator rights.

Symptoms. An audit shows role changes without an administrative action; the request log contains unexpected properties in a profile update.

Diagnosis. Search the code base for `@RequestBody` parameters whose type is an entity, and for repository `save` calls on a deserialised object. Write a test that posts extra fields and asserts that they are ignored.

Root cause. Jackson binds every matching property; the entity contains fields that only the server may change.

Fix. Bind to a dedicated record (`ProfileUpdate` in the worked example) and copy the permitted fields. As additional protection, set `spring.jackson.deserialization.fail-on-unknown-properties=true` so that unexpected properties are rejected with 400 rather than ignored, and make fields that must never change `updatable = false` in the mapping.

Prevention. Rule for reviews: no entity types in controller signatures; separate command and view types. Authorisation-relevant fields change only through dedicated administrative operations that are themselves method-secured.

Leseprobe

Problem 12: injection through a sort parameter

Scenario. A reporting endpoint accepts `?sort=createdAt` and builds a native query: `"SELECT * FROM orders ORDER BY " + sort`. Parameter binding cannot be used for column names, so the developer concatenated.

Symptoms. Requests such as `sort=(CASE WHEN (SELECT substr(password_hash,1,1) FROM app_user LIMIT 1)='$' THEN id ELSE total END)` change the order of results, which leaks data one bit at a time; error messages from PostgreSQL appear in responses.

Diagnosis. Search for string concatenation in `@Query(nativeQuery = true)`, `EntityManager.createNativeQuery`, `JdbcTemplate` and `Sort.by` calls whose arguments come from requests. Enable statement logging (`spring.jpa.show-sql=true` or the `org.hibernate.SQL` logger) and look for statements containing request content.

Root cause. Identifiers cannot be bound as parameters, so the code treated them as trusted text.

Fix. Map the request value to a closed set.

SortAllowList.java — JAVA

```java
package com.example.shop.order;

import java.util.Map;
import org.springframework.data.domain.Sort;
import org.springframework.http.HttpStatus;
import org.springframework.web.server.ResponseStatusException;

final class SortAllowList {

    private static final Map<String, String> ALLOWED = Map.of(
            "created", "createdAt",
            "total", "total",
            "status", "status");

    private SortAllowList() {
    }

    static Sort resolve(String requested, Sort.Direction direction) {
        String property = ALLOWED.get(requested);
        if (property == null) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "unsupported sort field");
        }
        return Sort.by(direction, property);   // Spring Data validates against the entity model
    }
}
```

Prevention. Prefer derived queries, Specifications and `Pageable` with property validation to hand-written SQL; ban string concatenation into queries in static analysis; run the application database user with the minimum privileges so that an injection cannot read the user table.

3.3.5 Questions to expect

1. **When is CSRF protection necessary, and when may it be disabled?** It is necessary when authentication relies on credentials that the browser attaches automatically, such as session cookies or Basic authentication. It may be disabled for stateless APIs that authenticate only with a header that scripts set explicitly, such as a bearer token, because a forged cross-site request cannot carry it.
2. **How does Spring Security prevent CSRF?** `CsrfFilter` requires, for unsafe methods, a token that is not available to a foreign page, submitted as a header or form parameter and compared with the value stored in the session or cookie; failures raise `InvalidCsrfTokenException` and produce 403.
3. **Why does a CORS preflight fail on a secured API, and how is it fixed?** The preflight carries no credentials, so an authenticated chain rejects it. `http.cors(...)` with a `CorsConfigurationSource` adds `CorsFilter` ahead of authentication so that it answers the preflight.
4. **Is CORS a security feature of the server?** No. It is enforced by browsers and only controls whether a page may read a cross-origin response; it does not stop requests from being sent and does not protect against non-browser clients.
5. **What is session fixation and how is it prevented?** An attacker fixes the session identifier before login; the defence is issuing a new identifier at authentication, which Spring Security does by default with `changeSessionId()`.
6. **How are SQL injection attacks prevented in a Spring application?** By bound parameters in derived queries, `@Query` with named or positional parameters and `JdbcTemplate` placeholders, by allow-lists for identifiers, and by least-privilege database accounts.

sieeram

7. **What is mass assignment?** Binding request data to fields the client must not control. The defence is request DTOs with only the permitted fields, optionally combined with failing on unknown properties.
8. **Explain the token bucket algorithm and its advantage over a fixed window.** Tokens refill at a constant rate up to a capacity and each request consumes one, so bursts up to the capacity are allowed while the long-run rate is bounded; fixed windows permit double the limit around a window boundary.
9. **Which headers does Spring Security add by default?** `Cache-Control` (no caching), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `X-XSS-Protection: 0` and, on HTTPS, `Strict-Transport-Security`; a Content-Security-Policy has to be configured explicitly.

Check yourself

1. A stateless API authenticates with `Authorization: Bearer`. Why is CSRF not an issue, and what changes if the same token is stored in a cookie?
2. What is wrong with `setAllowedOrigins(List.of("*"))` combined with `setAllowCredentials(true)`?
3. In the token bucket run above, why is request 14 the first rejection although requests arrive at a constant 20 per second?
4. Which of the following is safe: (a) `"... WHERE name = '" + name + "'"`; (b) `@Query("select o from Order o where o.owner = :owner")`; (c) `"ORDER BY " + sort`?
5. A page served by `bank.example` sets `SameSite=Lax` cookies. Can a form on `evil.example` POST with the cookie attached?

Answers

1. The foreign page cannot make the browser add the header, so there is no ambient credential. With the token in a cookie the browser attaches it automatically, and CSRF protection is required again.
2. Browsers reject the wildcard with credentials, and the combination would otherwise allow any site to read authenticated responses; the origins must be listed explicitly or matched with patterns.
3. The bucket starts with 10 tokens and refills 5 per second; 13 requests have been served after 600 ms (10 plus 3 refilled), and at 650 ms the bucket holds less than one token.
4. Only (b). (a) is injectable and (c) cannot be parameterised, so it needs an allow-list.
5. No for a cross-site POST in browsers that enforce `Lax`: the cookie is withheld, although a top-level GET navigation would carry it, which is why GET must not change state.

KEY POINTS

- CSRF applies when credentials are attached automatically; disable it only on stateless bearer-token chains, and use the cookie repository with the SPA handler for session-based single-page clients.
- CORS is a browser rule enforced by response headers; the preflight carries no credentials, so `CorsFilter` must run before authorisation, and wildcard origins cannot be combined with credentials.
- Defaults cover session fixation, framing and content sniffing; a Content-Security-Policy, HSTS behind a proxy and cookie flags need explicit configuration.
- Keep data and code separate: bound parameters for values, allow-lists for identifiers, DTOs for request bodies.
- Rate limit by key with a token bucket; per-instance limits multiply by the replica count, so global limits belong at the gateway or in a shared store.

3.4 Sessions and Tokens: Choosing an Authentication State Model

An authenticated caller must be recognised on every request after the first. Two designs exist. In the **stateful** design the server stores the authentication and the client holds only an opaque reference; in the **stateless** design the client holds a signed statement of the authentication and the server stores nothing. Each choice determines scaling behaviour, revocation, exposure to CSRF and XSS, and the operational dependencies of the service. This chapter compares both with the mechanisms Spring provides, as the basis for the token chapters that follow.

3.4.1 Concept and mechanism

Server sessions

With form login the `UsernamePasswordAuthenticationFilter` stores the `SecurityContext` in the `HttpSession` through `HttpSessionSecurityContextRepository`, and the container returns the session identifier in the `JSESSIONID` cookie. Each later request carries the cookie; `SecurityContextHolderFilter` loads the context from the session into the holder. The session is a server-side map; the identifier is a random 128-bit value that is worthless without the map entry. Logout invalidates the entry and the authentication ends immediately, which is the central advantage of the design. The costs are memory per session, a lookup for every request and a requirement that all instances can reach the session.

With several instances behind a load balancer there are two approaches. **Sticky sessions** pin a client to one instance by cookie or address; they are simple but lose the sessions of an instance that restarts or is scaled in, and distribute load unevenly. **Externalised sessions** store the session in a shared store; Spring Session replaces the container's `HttpSession` implementation with one backed by Redis (or JDBC), transparently to the application code. In Spring Boot the Redis store is configured by adding Spring Session's Redis module (`spring-session-data-redis` with a Redis client; Spring Boot's own documentation names a starter for it in recent versions) and setting `spring.session.timeout`; the auto-configuration replaces the need for `@EnableRedisHttpSession`. Session data is serialised, so session attributes must be serialisable, and the session store becomes a dependency of every request: an outage of Redis becomes an outage of the application unless sessions are allowed to degrade.

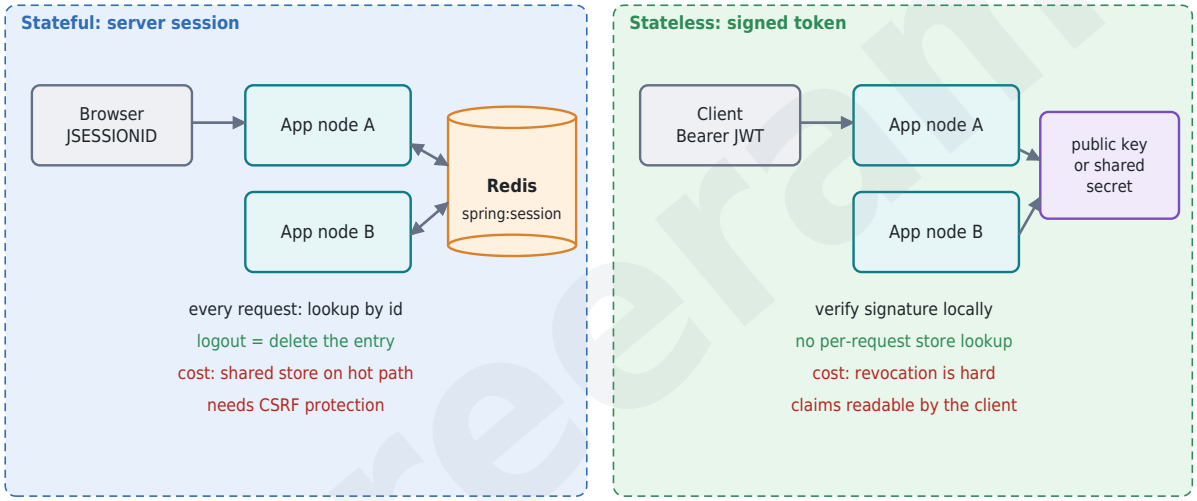


Figure 3.12. The stateful design looks up a session in a shared store on every request; the stateless design verifies a signature locally, which removes the store but also removes cheap revocation.

Signed tokens

In the stateless design the server signs a set of claims (subject, expiry, authorities) and the client presents the result with each request, normally in the `Authorization` header. Any instance that holds the verification key can validate the token without a store lookup, so horizontal scaling and cross-service calls are straightforward: a downstream service can verify the same token. The price is that a token is valid until it expires, wherever the user logs out; revocation requires extra state (a deny list or short lifetimes with refresh tokens), and claims are visible to the client because a signature provides integrity, not confidentiality.

Property	Server session (Redis-backed)	Signed token (JWT)
State on the server	One entry per session	None for access tokens; refresh tokens need a store
Revocation	Immediate: delete the entry	Not before expiry without extra state
Per-request cost	Store lookup	Signature verification (CPU, no network)
CSRF exposure	Yes (cookie is ambient)	No, if sent in a header; yes, if stored in a cookie
XSS exposure	Cookie `HttpOnly` limits theft	Token in script-readable storage is stolen by XSS
Cross-service use	Needs the shared store or a gateway	Verifiable by every service with the key

Property	Server session (Redis-backed)	Signed token (JWT)
Typical clients	Server-rendered sites, same-site SPAs	Mobile apps, third-party clients, service-to-service

Sessions and signed tokens compared.

A decision rule

The choice is not "tokens are modern". A same-site web application with one team and a Redis instance already in place is simpler and safer with server sessions and the cookie-based CSRF protection of the previous chapter: logout works, there is no token storage problem in the browser, and permissions can change immediately. Tokens are the better fit when clients are not browsers, when several independently deployed services must verify the caller without calling a central store, or when the identity provider is a separate system (see the chapter on OAuth 2.0). A frequent hybrid is the backend-for-frontend pattern: the browser keeps only a session cookie with a small backend that holds the tokens and calls the APIs, so that no token is ever exposed to browser scripts.

INTERVIEW ANGLE

Why not store a JWT in `localStorage`, and what is the alternative? Any script that runs in the page, including a script injected by an XSS vulnerability or a compromised dependency, can read `localStorage` and send the token away; the stolen token works until it expires, from anywhere. A token in an `HttpOnly` cookie cannot be read by script but is sent automatically, which reintroduces CSRF and requires the protections of the previous chapter. The robust alternative is a backend-for-frontend that keeps tokens server-side and gives the browser only a session cookie.

Check yourself

1. Which of the two designs supports immediate logout without extra components, and why?
2. What must be true of objects stored in an externalised session?
3. A mobile app and three microservices share one login. Which design fits and why?
4. Why does storing a token in a cookie bring back the need for CSRF protection?

Answers

1. The stateful design: deleting the server-side entry invalidates the identifier at once; a signed token stays valid until its expiry.
2. They must be serialisable, because the store holds a serialised copy and every instance deserialises it.
3. Signed tokens: the app is not a browser, and each service can verify the token with a public key without contacting a central session store.
4. Browsers attach cookies automatically to cross-site requests, so a forged request carries the credential; the cookie is again an ambient credential.

KEY POINTS

- Sessions keep state on the server and give immediate revocation at the cost of a shared store; tokens keep state in the client and give local verification at the cost of revocation and storage safety.
- Spring Session moves the session to Redis or JDBC without changing application code; session attributes must be serialisable.
- Cookie-based credentials need CSRF protection; header-based tokens do not, but are exposed to script theft when kept in script-readable storage.
- A backend-for-frontend keeps tokens away from the browser and combines stateless services with cookie sessions at the edge.

sreeram

3.5 JSON Web Tokens in Depth

A JSON Web Token (JWT, RFC 7519) is a compact, URL-safe representation of a set of claims, usually signed so that a receiver can verify who issued it and that it has not been altered. In a Spring Boot service the JWT replaces the server-side session: the login endpoint issues a token, the client sends it in the `Authorization` header, and every instance verifies it without a shared store. The simplicity of the format hides a large number of ways to use it incorrectly. Most published JWT vulnerabilities are not weaknesses of the cryptography but of verification code that trusts the token to describe how it should be verified. This chapter therefore begins with the byte layout, builds a complete issuer and verifier in plain Java that is attacked for real, and then shows the Spring Security 6 implementation, including refresh-token rotation and logout.

3.5.1 Concept and mechanism

The three parts

A signed JWT (technically a JWS in compact serialisation) consists of three Base64url strings joined by periods. The first is the **header**, a JSON object naming the signature algorithm (`alg`) and optionally the key identifier (`kid`) and type (`typ`). The second is the **payload**, a JSON object of claims. The third is the **signature**, computed over the ASCII bytes of the first two parts joined by a period. Base64url is the URL-safe Base64 variant that replaces `+` and `/` by `-` and `_` and omits the `=` padding, so that a token can appear in a header, a cookie or a URL without escaping.

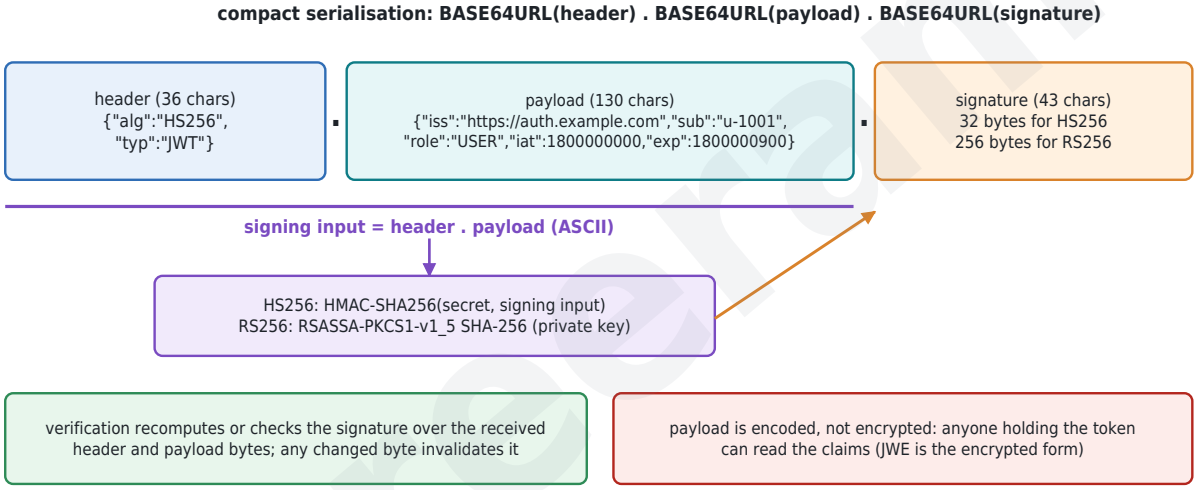


Figure 3.13. The signature covers header and payload exactly as transmitted; the payload is merely encoded, so every holder of the token can read it.

The registered claims defined by RFC 7519 that matter for validation are `iss` (issuer), `sub` (subject, the user identifier), `aud` (audience, the intended recipient), `exp` (expiry, seconds since the epoch), `nbf` (not before), `iat` (issued at) and `jti` (a unique token identifier). Application claims such as `roles` are added next to them. Two properties follow from the format. A JWT is not encrypted: placing an e-mail address, a national identifier or a secret in the payload discloses it to anyone who sees the token, including browser extensions and log aggregators; encryption requires the separate JWE format. And a JWT is self-contained: the receiver has no way to learn that the user has since been disabled or has logged out unless it consults additional state.

Algorithms and keys

The `alg` header selects one of the algorithms of RFC 7518. Three are in common use.

Algorithm	Key material	Signature size	Remarks
HS256	One shared secret of at least 256 bits	32 bytes	Every verifier can also forge tokens; suitable when issuer and verifier are the same service
RS256	RSA key pair, 2048 bits or more	256 bytes (2048-bit key)	Verifiers need only the public key; the default of the Spring Security resource server

Algorithm	Key material	Signature size	Remarks
ES256	ECDSA key pair on curve P-256	64 bytes	Smaller tokens and faster signing than RSA; JWS uses the raw R and S values, not the DER encoding that `java.security.Signature` returns

Signature algorithms for JWTs.

Asymmetric algorithms are preferred as soon as the issuer and the verifier are different services, because a compromised verifier then cannot mint tokens. Public keys are distributed as a **JWK Set** (JWKS) at a well-known URL; each key has a `kid`, and the token header names the key used. Key rotation follows from that: the issuer publishes the new key, begins signing with it, and retires the old key after the longest token lifetime has passed. The Spring resource server caches the JWKS and refetches it when it meets an unknown `kid`.

Verification rules

A verifier must decide the algorithm itself and must check every claim that determines whether the token is meant for it. The minimum list is:

1. Parse the token strictly: three parts, valid Base64url, JSON objects.
2. Select the key and the algorithm from **server configuration**, then check that the header `alg` equals it. The header is attacker-controlled input.
3. Verify the signature with a constant-time comparison.
4. Check `exp` (reject expired tokens) and `nbf`, with a small tolerance for clock differences between machines (clock skew). `JwtTimestampValidator` in Spring Security uses 60 seconds by default.
5. Check `iss` against the expected issuer and `aud` against the identifier of this service, so that a token issued for another service of the same issuer is rejected.
6. Check application-level conditions: required scope or role, token type, deny list.

The next program implements issuing and verification for HS256 and RS256 with nothing but the JDK (`javax.crypto.Mac`, `java.security.Signature`, `java.util.Base64`), including two deliberately vulnerable verifier variants that trust the header. It then runs the attacks against both a naive and a strict verifier.

MiniJwt.java JAVA

```
import java.nio.charset.StandardCharsets;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.MessageDigest;
import java.security.PublicKey;
import java.security.Signature;
import java.time.Instant;
import java.util.Base64;
import java.util.HashMap;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

public class MiniJwt {
    static final Base64.Encoder B64 = Base64.getUrlEncoder().withoutPadding();
    static final Base64.Decoder UNB64 = Base64.getUrlDecoder();

    static String b64(byte[] b) { return B64.encodeToString(b); }
    static String b64(String s) { return b64(s.getBytes(StandardCharsets.UTF_8)); }
    static String text(String part) { return new String(UNB64.decode(part),
            StandardCharsets.UTF_8); }

    static byte[] hmac(byte[] key, String data) throws Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(key, "HmacSHA256"));
        return mac.doFinal(data.getBytes(StandardCharsets.US_ASCII));
    }

    static String signHs256(String payloadJson, byte[] key) throws Exception {
        String signingInput = b64("{\"alg\":\"HS256\",\"typ\":\"JWT\"}") + "." + b64(payloadJson);
        return signingInput + "." + b64(hmac(key, signingInput));
    }
```

MiniJwt.java (continued) JAVA

```
    static String signRs256(String payloadJson, KeyPair kp) throws Exception {
        String signingInput = b64("{\"alg\":\"RS256\",\"typ\":\"JWT\"}") + "." + b64(payloadJson);
        Signature s = Signature.getInstance("SHA256withRSA");
        s.initSign(kp.getPrivate());
        s.update(signingInput.getBytes(StandardCharsets.US_ASCII));
        return signingInput + "." + b64(s.sign());
    }

    /** Flat-JSON claim reader, sufficient for this demonstration only. */
    static String claim(String json, String name) {
        Matcher m = Pattern.compile("\"" + name + "\"\\s*:\\s*(\"([^\"]*)\"|[0-9]+)").matcher(json);
        if (!m.find()) return null;
        return m.group(2) != null ? m.group(2) : m.group(1);
    }

    /** VULNERABLE: trusts the alg header supplied by the token. */
    static String verifyNaive(String token, byte[] hmacKey, PublicKey rsaPub) throws Exception {
        String[] p = token.split("\\.", -1);
        String alg = claim(text(p[0]), "alg");
        String signingInput = p[0] + "." + p[1];
        switch (alg) {
            case "none": return text(p[1]);
            case "HS256":
                byte[] key = hmacKey != null ? hmacKey : rsaPub.getEncoded();
                if (MessageDigest.isEqual(hmac(key, signingInput),
                        UNB64.decode(p[2]))) return text(p[1]);
                break;
            case "RS256":
                Signature s = Signature.getInstance("SHA256withRSA");
                s.initVerify(rsaPub);
                s.update(signingInput.getBytes(StandardCharsets.US_ASCII));
                if (s.verify(UNB64.decode(p[2]))) return text(p[1]);
                break;
            default:
        }
        throw new SecurityException("bad signature");
    }

    /** STRICT: fixed algorithm, time claims checked with a clock skew allowance. */
    static String verifyStrictHs256(String token, byte[] key, Instant now, long skewSeconds)
            throws Exception {
        String[] p = token.split("\\.", -1);
        if (p.length != 3) throw new SecurityException("malformed");
        if (!"HS256".equals(claim(text(p[0]),
                "alg"))) throw new SecurityException("alg not allowed");
        if (!MessageDigest.isEqual(hmac(key, p[0] + "." + p[1]), UNB64.decode(p[2])))
            throw new SecurityException("signature mismatch");
        String json = text(p[1]);
        long exp = Long.parseLong(claim(json, "exp"));
        if (now.getEpochSecond() > exp + skewSeconds) throw new SecurityException("expired");
        String nbf = claim(json, "nbf");
        if (nbf != null && now.getEpochSecond() + skewSeconds < Long.parseLong(nbf))
            throw new SecurityException("not yet valid");
        if (!"https://auth.example.com".equals(claim(json,
                "iss"))) throw new SecurityException("issuer");
        return json;
    }

    static void attempt(String label, java.util.concurrent.Callable<String> c) {
        try { String r = c.call();
            System.out.println(label + " -> ACCEPTED sub=" + claim(r, "sub") + " role=" + claim(r,
                    "role")); }
        catch (Exception e) { System.out.println(label + " -> rejected: " + e.getMessage()); }
    }

    public static void main(String[] args) throws Exception {
        byte[] key = "0123456789abcdef0123456789abcdef".getBytes(StandardCharsets.UTF_8);
        Instant now = Instant.ofEpochSecond(1_800_000_000L);
        String payload = "{\"iss\":\"https://auth.example.com\",\"sub\":\"u-1001\","
            + "\"role\":\"USER\",\"iat\":" + now.getEpochSecond() + ",\"exp\":"
                    + (now.getEpochSecond() + 900) + "}";

        String token = signHs256(payload, key);
        String[] p = token.split("\\.");
        System.out.println("token parts    : " + p[0].length() + " + " + p[1].length() + " + "
```

MiniJwt.java (continued) JAVA

```
                + p[2].length() + " characters");
        System.out.println("token         : " + token.substring(0, 36) + "...");
        System.out.println("header        : " + text(p[0]));
        System.out.println("payload       : " + text(p[1]).replace(",\"iat",
                ",\n               \"iat"));
        System.out.println("signature len : " + UNB64.decode(p[2]).length + " bytes");
        System.out.println();

        System.out.println("== verification ==");
        attempt("valid token at t+0        ", () -> verifyStrictHs256(token, key, now, 30));
        attempt("valid token at t+920 (skew 30)", () -> verifyStrictHs256(token, key,
                now.plusSeconds(920), 30));
        attempt("valid token at t+931 (skew 30)", () -> verifyStrictHs256(token, key,
                now.plusSeconds(931), 30));

        System.out.println();
        System.out.println("== tampering: role USER -> ADMIN, signature unchanged ==");
        String forgedPayload = b64(text(p[1]).replace("USER", "ADMIN"));
        String tampered = p[0] + "." + forgedPayload + "." + p[2];
        attempt("strict verifier", () -> verifyStrictHs256(tampered, key, now, 30));

        System.out.println();
        System.out.println("== attack 1: alg=none ==");
        String none = b64("{\"alg\":\"none\",\"typ\":\"JWT\"}") + "." + forgedPayload + ".";
        System.out.println("forged token  : " + none.substring(0, 36) + "..."
                + none.substring(none.length() - 12));
        attempt("naive verifier ", () -> verifyNaive(none, key, null));
        attempt("strict verifier", () -> verifyStrictHs256(none, key, now, 30));

        System.out.println();
        System.out.println("== attack 2: RS256 -> HS256 algorithm confusion ==");
        KeyPairGenerator g = KeyPairGenerator.getInstance("RSA");
        g.initialize(2048);
        KeyPair kp = g.generateKeyPair();
        String legit = signRs256(payload, kp);
        attempt("legitimate RS256 token, naive verifier", () -> verifyNaive(legit, null,
                kp.getPublic()));
        // attacker knows only the public key (published at the JWKS endpoint)
        String signingInput = b64("{\"alg\":\"HS256\",\"typ\":\"JWT\"}") + "." + forgedPayload;
        String confused = signingInput + "." + b64(hmac(kp.getPublic().getEncoded(), signingInput));
        attempt("HS256 token signed with the public key, naive ", () -> verifyNaive(confused, null,
                kp.getPublic()));
        System.out.println("(a verifier fixed to RS256 would reject the header alg=HS256)");

        System.out.println();
        System.out.println("== key size ==");
        System.out.println("HS256 secret used above: " + key.length * 8
                + " bits (RFC 7518 requires at least 256)");
        System.out.println("RS256 signature length : " + UNB64.decode(legit.split("\\.")[2]).length
                + " bytes");
    }
}
```

Issuing, verifying and attacking a JWT (run on the build machine; signature bytes differ per run only for RS256 tokens) TERMINAL

```
$ java MiniJwt.java
token parts    : 36 + 130 + 43 characters
token         : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
header        : {"alg":"HS256","typ":"JWT"}
payload       : {"iss":"https://auth.example.com","sub":"u-1001","role":"USER",
                "iat":1800000000,"exp":1800000900}
signature len : 32 bytes

== verification ==
valid token at t+0         -> ACCEPTED sub=u-1001 role=USER
valid token at t+920 (skew 30) -> ACCEPTED sub=u-1001 role=USER
valid token at t+931 (skew 30) -> rejected: expired

== tampering: role USER -> ADMIN, signature unchanged ==
strict verifier -> rejected: signature mismatch

== attack 1: alg=none ==
forged token  : eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0....MDAwMDkwMH0.
naive verifier  -> ACCEPTED sub=u-1001 role=ADMIN
```

Issuing, verifying and attacking a JWT (run on the build machine; signature bytes differ per run only for RS256 tokens) (continued) TERMINAL

```
strict verifier -> rejected: alg not allowed

== attack 2: RS256 -> HS256 algorithm confusion ==
legitimate RS256 token, naive verifier -> ACCEPTED sub=u-1001 role=USER
HS256 token signed with the public key, naive  -> ACCEPTED sub=u-1001 role=ADMIN
(a verifier fixed to RS256 would reject the header alg=HS256)

== key size ==
HS256 secret used above: 256 bits (RFC 7518 requires at least 256)
RS256 signature length : 256 bits
```

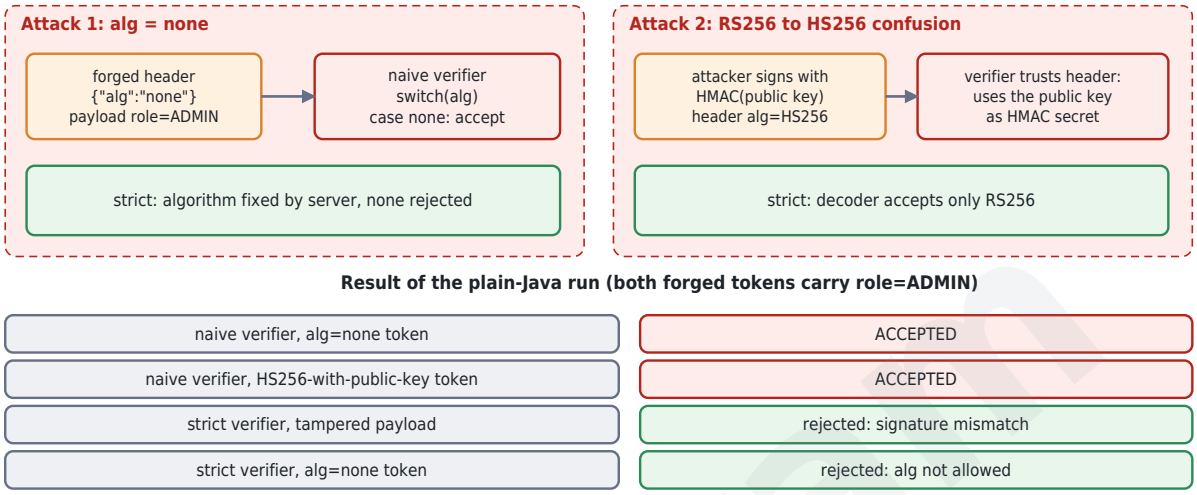


Figure 3.14. Both forged tokens are accepted by a verifier that reads the algorithm from the token and are rejected by a verifier that fixes the algorithm.

Reading the output: the first block shows the layout (36 + 130 + 43 characters), the 15-minute lifetime, and the clock-skew behaviour: at 920 s the token is 20 s past its `exp` and still accepted with a 30 s allowance, at 931 s it is rejected. The tampering test changes `role` from `USER` to `ADMIN` without re-signing and fails on the signature. The two attacks succeed against the naive verifier for different reasons.

- `alg` = `none`**.** The JWS specification defines an unsecured variant. A verifier that honours `none` accepts any payload with an empty signature part.
- **Algorithm confusion.** A service that expects RS256 verifies with a public key. If the verifier selects the algorithm from the header and passes the same key object to an HMAC routine, an attacker signs a token with HMAC using the *public key bytes* as the secret, which are public by design. The naive verifier computes the same HMAC and accepts it.

Other recurring attack vectors are weak HMAC secrets (next section), `kid` values that the server uses unsafely in a file path or SQL query, header parameters such as `jku` or `x5u` that make the verifier fetch a key from an attacker-controlled URL, and tokens leaked in URLs and logs. Mature libraries such as Nimbus JOSE+JWT, which Spring Security uses, bind the algorithm to the key and reject `none`, but custom verification code does not benefit from that.

Access tokens, refresh tokens and revocation

A signed token cannot be recalled, so its lifetime is the revocation delay. The standard design uses two tokens. The **access token** is short-lived (5 to 15 minutes), is sent to the API on every request and is not stored by the server. The **refresh token** is long-lived (days), is sent only to the token endpoint, and is an opaque random value stored (hashed) on the server so that it can be revoked.

Refresh tokens are the highest-value target, because one of them yields new access tokens for weeks. **Rotation** makes each refresh token single-use: every refresh returns a new refresh token and marks the old one as used. All tokens issued from one login share a **family** identifier. If a used token is presented again, either the legitimate client or an attacker holds a copy, and the server cannot tell which; it revokes the entire family, which forces a new login and ends the attacker's access. The following simulation shows the sequence.

RefreshRotation.java JAVA

```java
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class RefreshRotation {
    record Token(String family, boolean used) {}

    static final Map<String, Token> store = new HashMap<>();
    static final Set<String> revokedFamilies = new HashSet<>();
    static int counter = 0;

    static String issue(String family) {
        String id = "rt" + (++counter);
        store.put(id, new Token(family, false));
        return id;
    }

    static String refresh(String id) {
        Token t = store.get(id);
        if (t == null) return "401 unknown token";
        if (revokedFamilies.contains(t.family())) {
            return "401 family " + t.family() + " already revoked";
        }
        if (t.used()) {
            revokedFamilies.add(t.family());
            return "401 reuse detected: family " + t.family() + " revoked";
        }
        store.put(id, new Token(t.family(), true));
        return "200 issued " + issue(t.family()) + " (" + id + " is now used)";
    }

    public static void main(String[] args) {
        String rt1 = issue("login-1");
        System.out.println("login                    : issued " + rt1);
        System.out.println("client refreshes rt1     : " + refresh("rt1"));
        System.out.println("attacker replays rt1     : " + refresh("rt1"));
        System.out.println("client refreshes rt2     : " + refresh("rt2"));
        System.out.println("client logs in again     : issued " + issue("login-2"));
        System.out.println("new session refreshes rt3: " + refresh("rt3"));
    }
}
```

Refresh token rotation with reuse detection TERMINAL

```
$ java RefreshRotation.java
login                    : issued rt1
client refreshes rt1     : 200 issued rt2 (rt1 is now used)
attacker replays rt1     : 401 reuse detected: family login-1 revoked
client refreshes rt2     : 401 family login-1 already revoked
client logs in again     : issued rt3
new session refreshes rt3: 200 issued rt4 (rt3 is now used)
```

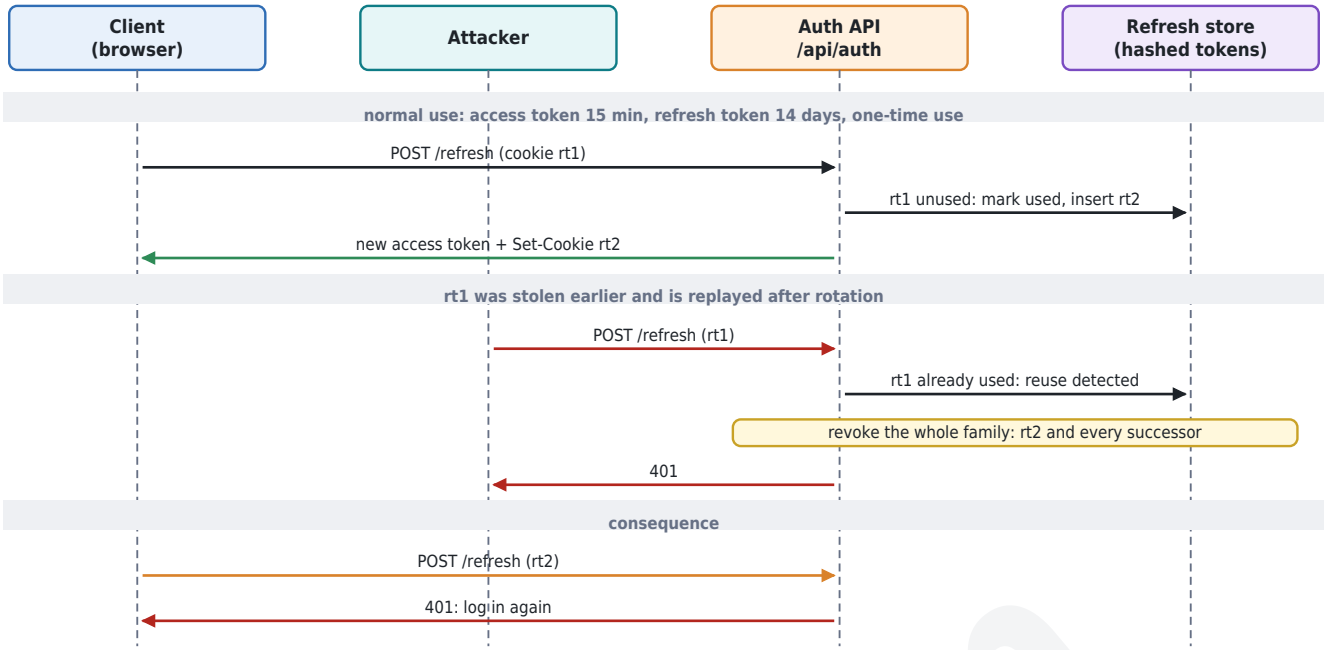


Figure 3.15. The replay of an already-used refresh token is detected and revokes the family, so the stolen copy and its legitimate successor both stop working.

Rotation has a known practical cost: two concurrent refresh calls with the same token (two browser tabs) look like reuse. The refresh endpoint should serialise on the token row, and clients should coordinate refreshes through a single-flight mechanism; some systems add a grace period of a few seconds during which a used token is tolerated.

Access-token revocation (logout, disabled account) needs state or short lifetimes. Three strategies exist. **Short lifetimes alone** accept a revocation delay equal to the lifetime. A **deny list** stores the `jti` of revoked tokens until their `exp`, at the cost of one lookup per request and a store shared by all instances (Redis with a TTL equal to the remaining lifetime is the usual choice). A **token version** claim compares a per-user counter in the token with the current counter in the database or cache, which revokes all tokens of a user at once with one lookup.

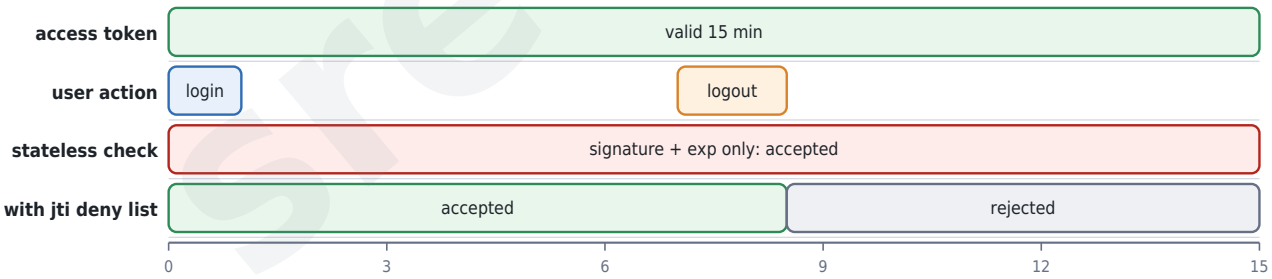


Figure 3.16. A purely stateless check accepts the token until its exp even after logout; a jti deny list closes the window at the cost of a lookup per request.

Where the client keeps tokens

Two places exist in a browser, and both have a cost. Script-readable storage (`localStorage`, `sessionStorage`, a JavaScript variable) is exposed to every script on the page, so an XSS flaw or a compromised third-party script exfiltrates the token. An `HttpOnly` cookie is invisible to scripts but is attached automatically, so it needs CSRF protection and `SameSite`. A widely used compromise keeps the short-lived access token in memory (a JavaScript variable, lost on reload) and the refresh token in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie scoped to the refresh path (`Path=/api/auth`). The page obtains a new access token on load through the refresh endpoint. The stronger design is the backend-for-frontend described in the previous chapter. Mobile clients store refresh tokens in the platform keystore (Keychain, Keystore), not in preferences.

3.5.2 Worked example

The implementation issues RS256 access tokens, stores hashed refresh tokens with rotation and reuse detection, and protects the API with the Spring Security resource server. It builds on the `AppUser` model and the delegating password encoder of the first chapter. Dependencies: `spring-boot-starter-oauth2-resource-server` (which brings `spring-security-oauth2-jose` and Nimbus), `spring-boot-starter-data-jpa`, `spring-boot-starter-validation`. The Spring code was not run on the build machine; the signatures were checked against the Spring Security 6.5 reference documentation.

First, the key pair. Keys are generated once, stored outside the repository (a secret store, a mounted volume), and referenced by location.

Generating an RSA key pair in PKCS#8 and X.509 PEM form (openssl 3.0.13, run on the build machine) TERMINAL

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out private.pem
$ openssl rsa -pubout -in private.pem -out public.pem
writing RSA key
$ head -1 private.pem public.pem
==> private.pem <==
-----BEGIN PRIVATE KEY-----

==> public.pem <==
-----BEGIN PUBLIC KEY-----
```

application.yml (fragment) YAML

```yaml
app:
  jwt:
    issuer: https://api.shop.example.com
    private-key: file:${JWT_PRIVATE_KEY_PATH}
    public-key: file:${JWT_PUBLIC_KEY_PATH}
    access-ttl: 15m
    refresh-ttl: 14d
```

JwtProperties.java JAVA

```java
package com.example.shop.auth;

import java.time.Duration;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.core.io.Resource;

@ConfigurationProperties(prefix = "app.jwt")
public record JwtProperties(String issuer, Resource privateKey, Resource publicKey,
                            Duration accessTtl, Duration refreshTtl) {
}
```

The configuration class loads the keys with `RsaKeyConverters` (from `spring-security-core`), builds the encoder for issuing and the decoder for verifying, and combines the default validators (timestamp and issuer) with the deny-list validator. Using `withPublicKey(...).signatureAlgorithm(RS256)` fixes the algorithm on the verifying side; the header `alg` of a presented token cannot change it.

JwtConfig.java JAVA

```java
package com.example.shop.auth;

import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;
import java.io.IOException;
import java.io.InputStream;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.converter.RsaKeyConverters;
import org.springframework.security.oauth2.core.DelegatingOAuth2TokenValidator;
import org.springframework.security.oauth2.jose.jws.SignatureAlgorithm;
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.JwtValidators;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;
```

JwtConfig.java (continued) JAVA

```java

@Configuration
@EnableConfigurationProperties(JwtProperties.class)
public class JwtConfig {

    private static RSAPublicKey publicKey(JwtProperties p) throws IOException {
        try (InputStream in = p.publicKey().getInputStream()) {
            return RsaKeyConverters.x509().convert(in);
        }
    }

    private static RSAPrivateKey privateKey(JwtProperties p) throws IOException {
        try (InputStream in = p.privateKey().getInputStream()) {
            return RsaKeyConverters.pkcs8().convert(in);
        }
    }

    @Bean
    JwtEncoder jwtEncoder(JwtProperties props) throws IOException {
        RSAKey jwk = new RSAKey.Builder(publicKey(props))
                .privateKey(privateKey(props))
                .keyID("shop-2026-10")
                .build();
        return new NimbusJwtEncoder(new ImmutableJWKSet<>(new JWKSet(jwk)));
    }

    @Bean
    JwtDecoder jwtDecoder(JwtProperties props, TokenDenyList denyList) throws IOException {
        NimbusJwtDecoder decoder = NimbusJwtDecoder.withPublicKey(publicKey(props))
                .signatureAlgorithm(SignatureAlgorithm.RS256)
                .build();
        decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
                JwtValidators.createDefaultWithIssuer(props.issuer()), denyList));
        return decoder;
    }
}
```

The deny list is a validator. The in-memory map is adequate for one instance; with several instances the same interface is implemented over Redis with `SET jti 1 EX <remaining seconds>`.

TokenDenyList.java JAVA

```java
package com.example.shop.auth;

import java.time.Instant;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.security.oauth2.core.OAuth2Error;
import org.springframework.security.oauth2.core.OAuth2TokenValidator;
import org.springframework.security.oauth2.core.OAuth2TokenValidatorResult;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.stereotype.Component;

@Component
public class TokenDenyList implements OAuth2TokenValidator<Jwt> {

    private final ConcurrentMap<String, Instant> revoked = new ConcurrentHashMap<>();

    public void revoke(String jti, Instant expiresAt) {
        if (jti != null && expiresAt != null) {
            revoked.put(jti, expiresAt);
        }
    }

    @Override
    public OAuth2TokenValidatorResult validate(Jwt jwt) {
        if (jwt.getId() != null && revoked.containsKey(jwt.getId())) {
            return OAuth2TokenValidatorResult.failure(
                    new OAuth2Error("invalid_token", "token has been revoked", null));
        }
        return OAuth2TokenValidatorResult.success();
    }
```

TokenDenyList.java (continued) JAVA

```
    @Scheduled(fixedDelay = 60_000)
    void purgeExpired() {
        Instant now = Instant.now();
        revoked.values().removeIf(exp -> exp.isBefore(now));
    }
}
```

The access-token factory writes the standard claims, a unique `jti`, and the roles without the `ROLE_` prefix; the converter in the security configuration restores the prefix.

TokenService.java JAVA

```
package com.example.shop.auth;

import java.time.Clock;
import java.time.Instant;
import java.util.Collection;
import java.util.List;
import java.util.UUID;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.oauth2.jose.jws.SignatureAlgorithm;
import org.springframework.security.oauth2.jwt.JwsHeader;
import org.springframework.security.oauth2.jwt.JwtClaimsSet;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.JwtEncoderParameters;
import org.springframework.stereotype.Service;

@Service
public class TokenService {

    private final JwtEncoder encoder;
    private final JwtProperties props;
    private final Clock clock = Clock.systemUTC();

    public TokenService(JwtEncoder encoder, JwtProperties props) {
        this.encoder = encoder;
        this.props = props;
    }

    public String accessToken(String username, Collection<? extends GrantedAuthority> authorities) {
        Instant now = clock.instant();
        List<String> roles = authorities.stream()
                .map(GrantedAuthority::getAuthority)
                .filter(a -> a.startsWith("ROLE_"))
                .map(a -> a.substring("ROLE_".length()))
                .toList();
        JwtClaimsSet claims = JwtClaimsSet.builder()
                .issuer(props.issuer())
                .subject(username)
                .audience(List.of("shop-api"))
                .issuedAt(now)
                .expiresAt(now.plus(props.accessTtl()))
                .id(UUID.randomUUID().toString())
                .claim("roles", roles)
                .build();
        JwsHeader header = JwsHeader.with(SignatureAlgorithm.RS256).build();
        return encoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
    }
}
```

The refresh token is a random 256-bit value. Only its SHA-256 digest is stored, so a database leak does not yield usable tokens (a fast hash is acceptable here because the input is 256 bits of entropy, unlike a password).

RefreshToken.java (accessors omitted) JAVA

```
package com.example.shop.auth;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.time.Instant;
```

RefreshToken.java (accessors omitted) (continued) JAVA

```java
@Entity
@Table(name = "refresh_token")
public class RefreshToken {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "token_hash", nullable = false, unique = true, length = 64)
    private String tokenHash;

    @Column(nullable = false, length = 60)
    private String username;

    @Column(name = "family_id", nullable = false, length = 36)
    private String familyId;

    @Column(name = "expires_at", nullable = false)
    private Instant expiresAt;

    @Column(nullable = false)
    private boolean used;

    @Column(nullable = false)
    private boolean revoked;

    protected RefreshToken() {
    }

    public RefreshToken(String tokenHash, String username, String familyId, Instant expiresAt) {
        this.tokenHash = tokenHash;
        this.username = username;
        this.familyId = familyId;
        this.expiresAt = expiresAt;
    }

    public void markUsed() {
        this.used = true;
    }

    // getters: getTokenHash, getUsername, getFamilyId, getExpiresAt, isUsed, isRevoked
}
```

RefreshTokenRepository.java JAVA

```java
package com.example.shop.auth;

import jakarta.persistence.LockModeType;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select t from RefreshToken t where t.tokenHash = :hash")
    Optional<RefreshToken> findForUpdate(@Param("hash") String hash);

    @Modifying
    @Query("update RefreshToken t set t.revoked = true where t.familyId = :family")
    int revokeFamily(@Param("family") String familyId);
}
```

The rotation method carries the key subtlety: when reuse is detected it must revoke the family **and commit**, although it then throws an exception. By default a runtime exception rolls the transaction back, which would undo the revocation; `noRollbackFor` prevents it.

RefreshTokenService.java JAVA

```java
package com.example.shop.auth;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.time.Instant;
import java.util.Base64;
import java.util.HexFormat;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class RefreshTokenService {

    public static class InvalidRefreshException extends RuntimeException {
    }

    public static class RefreshReuseException extends RuntimeException {
    }

    public record Rotation(String username, String newRefreshToken) {
    }

    private final RefreshTokenRepository repo;
    private final JwtProperties props;
    private final SecureRandom random = new SecureRandom();

    public RefreshTokenService(RefreshTokenRepository repo, JwtProperties props) {
        this.repo = repo;
        this.props = props;
    }

    @Transactional
    public String issue(String username, String familyId) {
        byte[] raw = new byte[32];
        random.nextBytes(raw);
        String token = Base64.getUrlEncoder().withoutPadding().encodeToString(raw);
        repo.save(new RefreshToken(sha256(token), username, familyId,
                Instant.now().plus(props.refreshTtl())));
        return token;
    }

    @Transactional(noRollbackFor = RefreshReuseException.class)
    public Rotation rotate(String presented) {
        RefreshToken t = repo.findForUpdate(sha256(presented))
                .orElseThrow(InvalidRefreshException::new);
        if (t.isRevoked() || t.getExpiresAt().isBefore(Instant.now())) {
            throw new InvalidRefreshException();
        }
        if (t.isUsed()) {
            repo.revokeFamily(t.getFamilyId());       // committed despite the exception
            throw new RefreshReuseException();
        }
        t.markUsed();
        return new Rotation(t.getUsername(), issue(t.getUsername(), t.getFamilyId()));
    }

    @Transactional
    public void revokeFamilyOf(String presented) {
        repo.findForUpdate(sha256(presented)).ifPresent(t -> repo.revokeFamily(t.getFamilyId()));
    }

    private static String sha256(String value) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            return HexFormat.of().formatHex(md.digest(value.getBytes(StandardCharsets.UTF_8)));
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

Sleeram

The controller exposes login, refresh and logout. The refresh token travels only in a cookie restricted to the authentication path; the access token is returned in the body. The `AuthenticationManager` bean is obtained from `AuthenticationConfiguration`, which uses the `JpaUserDetailsService` and the delegating encoder of the first chapter. On refresh the user is loaded again, so that a disabled account or changed roles take effect at the next refresh rather than at the end of the refresh token lifetime.

AuthController.java JAVA

```
package com.example.shop.auth;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import java.time.Duration;
import java.util.UUID;
import org.springframework.http.HttpHeaders;
import org.springframework.http.ResponseCookie;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.CookieValue;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    static final String COOKIE = "refresh_token";

    public record LoginRequest(@NotBlank String username, @NotBlank String password) {
    }

    public record TokenResponse(String accessToken, String tokenType, long expiresIn) {
    }

    private final AuthenticationManager authentication;
    private final UserDetailsService users;
    private final TokenService tokens;
    private final RefreshTokenService refreshTokens;
    private final TokenDenyList denyList;
    private final JwtProperties props;

    public AuthController(AuthenticationManager authentication, UserDetailsService users,
                          TokenService tokens, RefreshTokenService refreshTokens,
                          TokenDenyList denyList, JwtProperties props) {
        this.authentication = authentication;
        this.users = users;
        this.tokens = tokens;
        this.refreshTokens = refreshTokens;
        this.denyList = denyList;
        this.props = props;
    }

    @PostMapping("/login")
    public ResponseEntity<TokenResponse> login(@Valid @RequestBody LoginRequest req) {
        Authentication auth = authentication.authenticate(
                UsernamePasswordAuthenticationToken.unauthenticated(
                        req.username(), req.password()));
        String refresh = refreshTokens.issue(auth.getName(), UUID.randomUUID().toString());
        return respond(tokens.accessToken(auth.getName(), auth.getAuthorities()), refresh);
    }

    @PostMapping("/refresh")
    public ResponseEntity<TokenResponse> refresh(
            @CookieValue(name = COOKIE, required = false) String presented) {
        if (presented == null) {
            throw new RefreshTokenService.InvalidRefreshException();
        }
        RefreshTokenService.Rotation r = refreshTokens.rotate(presented);
```

AuthController.java (continued) JAVA

```
        UserDetails user = users.loadUserByUsername(r.username());   // current roles and status
        if (!user.isEnabled() || !user.isAccountNonLocked()) {
            throw new RefreshTokenService.InvalidRefreshException();
        }
        return respond(tokens.accessToken(user.getUsername(), user.getAuthorities()),
                r.newRefreshToken());
    }

    @PostMapping("/logout")
    public ResponseEntity<Void> logout(
            @AuthenticationPrincipal Jwt jwt,
            @CookieValue(name = COOKIE, required = false) String presented) {
        if (presented != null) {
            refreshTokens.revokeFamilyOf(presented);
        }
        denyList.revoke(jwt.getId(), jwt.getExpiresAt());
        return ResponseEntity.noContent()
                .header(HttpHeaders.SET_COOKIE, cookie("", Duration.ZERO).toString())
                .build();
    }

    private ResponseEntity<TokenResponse> respond(String access, String refresh) {
        return ResponseEntity.ok()
                .header(HttpHeaders.SET_COOKIE, cookie(refresh, props.refreshTtl()).toString())
                .body(new TokenResponse(access, "Bearer", props.accessTtl().toSeconds()));
    }

    private static ResponseCookie cookie(String value, Duration maxAge) {
        return ResponseCookie.from(COOKIE, value)
                .httpOnly(true)
                .secure(true)
                .sameSite("Strict")
                .path("/api/auth")
                .maxAge(maxAge)
                .build();
    }
}
```

Exceptions thrown in the controller layer are translated by an advice. Because `authenticate` throws subclasses of `AuthenticationException` (`BadCredentialsException`, `LockedException`, `DisabledException`), a single handler yields a uniform 401 that does not reveal which condition applied.

AuthExceptionHandler.java JAVA

```
package com.example.shop.auth;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.security.core.AuthenticationException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice(assignableTypes = AuthController.class)
public class AuthExceptionHandler {

    @ExceptionHandler({AuthenticationException.class,
            RefreshTokenService.InvalidRefreshException.class,
            RefreshTokenService.RefreshReuseException.class})
    ProblemDetail unauthorized() {
        return ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, "authentication failed");
    }
}
```

The security configuration for the API chain activates the resource server, maps the `roles` claim to authorities and permits the three public endpoints. The decoder bean from `JwtConfig` is picked up automatically; no property is needed.

JwtSecurityConfig.java JAVA

```
package com.example.shop.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
```

JwtSecurityConfig.java (continued) JAVA

```java
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.authentication.configuration
        .AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.oauth2.server.resource.authentication
        .JwtAuthenticationConverter;
import org.springframework.security.oauth2.server.resource.authentication
        .JwtGrantedAuthoritiesConverter;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class JwtSecurityConfig {

    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration cfg) throws Exception {
        return cfg.getAuthenticationManager();
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtGrantedAuthoritiesConverter roles = new JwtGrantedAuthoritiesConverter();
        roles.setAuthoritiesClaimName("roles");
        roles.setAuthorityPrefix("ROLE_");
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(roles);
        return converter;
    }

    @Bean
    @Order(1)
    SecurityFilterChain apiChain(HttpSecurity http, JwtAuthenticationConverter converter)
            throws Exception {
        http.securityMatcher("/api/**")
            // bearer header only; the refresh cookie is SameSite=Strict
            .csrf(csrf -> csrf.disable())
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/auth/login", "/api/auth/refresh",
                        "/api/auth/register").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(o -> o
                .jwt(j -> j.jwtAuthenticationConverter(converter)));
        return http.build();
    }
}
```

The refresh endpoint is cookie-authenticated, so a forged cross-site POST to it is conceivable; the cookie attributes `SameSite=Strict` and `Path=/api/auth` prevent the browser from attaching it to cross-site requests. The comment on `csrf.disable()` records the reason for the decision, which a reviewer must be able to check.

For understanding the mechanism, and for the rare case in which the resource server DSL cannot be used, the same behaviour can be written as a filter. The sketch below is educational; production code should use `oauth2ResourceServer`, which also handles error responses, `WWW-Authenticate` headers and DPoP or opaque-token variants.

BearerFilter.java (educational sketch; prefer oauth2ResourceServer) JAVA

```java
public class BearerFilter extends OncePerRequestFilter {

    private final JwtDecoder decoder;
    private final JwtAuthenticationConverter converter;
    private final AuthenticationEntryPoint entryPoint = new BearerTokenAuthenticationEntryPoint();

    public BearerFilter(JwtDecoder decoder, JwtAuthenticationConverter converter) {
        this.decoder = decoder;
        this.converter = converter;
    }
```

Sriram

BearerFilter.java (educational sketch; prefer oauth2ResourceServer) (continued) JAVA

```java
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws ServletException, IOException {
        String header = req.getHeader(HttpHeaders.AUTHORIZATION);
        if (header != null && header.startsWith("Bearer ")) {
            try {
                Jwt jwt = decoder.decode(header.substring(7));   // signature, exp, iss, deny list
                SecurityContext ctx = SecurityContextHolder.createEmptyContext();
                ctx.setAuthentication(converter.convert(jwt));
                SecurityContextHolder.setContext(ctx);
            } catch (JwtException e) {
                SecurityContextHolder.clearContext();
                entryPoint.commence(req, res, new InvalidBearerTokenException(e.getMessage(), e));
                return;
            }
        }
        chain.doFilter(req, res);
    }
}
```

3.5.3 Hands-on

The sequence below exercises login, an API call, refresh with rotation, replay of the old refresh token, and logout. The output is **typical output (not run on the build machine)**; token values are shortened.

Login, call, refresh, replay, logout (typical output, not run on the build machine) TERMINAL

```
$ curl -si -X POST localhost:8080/api/auth/login -H 'Content-Type: application/json' \
    -d '{"username":"alice","password":"correct horse battery"}' -c jar.txt
HTTP/1.1 200
Set-Cookie: refresh_token=Qm3x...; Path=/api/auth; Max-Age=1209600; Secure;
    HttpOnly; SameSite=Strict
{"accessToken":"eyJhbGciOiJSUzI1NiJ9...","tokenType":"Bearer","expiresIn":900}

$ curl -s localhost:8080/api/orders -H "Authorization: Bearer $AT"
[]

$ curl -s -X POST localhost:8080/api/auth/refresh -b jar.txt -c jar.txt
{"accessToken":"eyJhbGciOiJSUzI1NiJ9...","tokenType":"Bearer","expiresIn":900}

$ curl -si -X POST localhost:8080/api/auth/refresh -b old_cookie.txt
HTTP/1.1 401
{"status":401,"detail":"authentication failed", ...}

$ curl -si localhost:8080/api/orders -H "Authorization: Bearer $EXPIRED"
HTTP/1.1 401
WWW-Authenticate: Bearer error="invalid_token",
    error_description="Jwt expired at 2026-10-19T09:15:00Z"
```

The decoded header of the issued token contains `"alg":"RS256"` and the `kid`; the payload contains `iss`, `sub`, `aud`, `iat`, `exp`, `jti` and `roles`. A quick decode on the command line is `echo "$AT" | cut -d. -f2 | base64 -d` (the padding may need to be completed, and the result is readable by anyone, which is the point of the earlier remark that a JWT is not encrypted).

3.5.4 Real-world problems and solutions

Problem 13: a token is accepted after logout or after its expiry

Scenario. After the user presses "log out", a copy of the access token still works for several minutes. In a second incident a token created by a custom issuer remains valid for a month although its lifetime should be 15 minutes.

Symptoms. API calls with a logged-out token return 200; the audit log shows requests with tokens older than the configured lifetime.

Diagnosis. For the first incident the timeline is the one in the logout diagram: the logout handler deleted the refresh token but the access token was verified only by signature and `exp`. For the second, decode the payload and compare `exp - iat` with the expected value: a difference of 900000 instead of 900 reveals that the issuer wrote milliseconds (`System.currentTimeMillis()`) into a claim that is defined in seconds, so the token expires in the year 50000. Also check the server and issuer clocks (`chronyc tracking`, `date -u`) and the configured skew.

Root cause. Stateless verification cannot know about logout; and a unit mistake in `exp`.

Fix. Revoke on logout through the deny list (`TokenDenyList` above) and keep access tokens short; for the unit mistake use `Instant` and the `JwtClaimsSet` builder, which writes seconds, rather than hand-built maps, and add a test that asserts `exp - iat == 900`.

Prevention. Document the revocation delay as a property of the design; for high-risk operations (changing a password, transferring money) require re-authentication or a recent `auth_time` claim instead of relying on the access token alone.

Problem 14: signing secret hard-coded, weak or leaked

Scenario. The HS256 secret is a string literal in `application.properties` (`jwt.secret=secret123`), committed to the repository, and identical in every environment.

Symptoms. A security review reproduces a forged administrator token; or, after a repository leak, tokens with arbitrary claims appear in the logs.

Diagnosis. Search the repository history for the secret (`git log -S secret123`), and test the strength offline: one token is enough for a dictionary attack. The following run is the attacker's view on the build machine; the throughput varies with hardware, the structure does not.

WeakSecret.java — JAVA

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.Base64;
import java.util.List;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

public class WeakSecret {
    static final Base64.Encoder B64 = Base64.getUrlEncoder().withoutPadding();

    static byte[] hmac(byte[] key, String data) throws Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(key, "HmacSHA256"));
        return mac.doFinal(data.getBytes(StandardCharsets.US_ASCII));
    }

    public static void main(String[] args) throws Exception {
        String header = B64.encodeToString("{\"alg\":\"HS256\",\"typ\":\"JWT\"}".getBytes());
        String payload = B64.encodeToString("{\"sub\":\"u-1001\",\"role\":\"USER\"}".getBytes());
        String signingInput = header + "." + payload;
        // signature produced by the server with its (weak) key
        byte[] observedSignature = hmac("secret123".getBytes(), signingInput);

        // The attacker holds one token and tries candidate secrets offline.
        List<String> dictionary = List.of("password", "123456", "changeme", "secret", "jwtsecret",
            "mysecretkey", "supersecret", "secret123", "admin", "letmein");
        long t0 = System.nanoTime();
        int tries = 0;
        String found = null;
        for (String candidate : dictionary) {
            tries++;
            byte[] guess = hmac(candidate.getBytes(), signingInput);
            if (MessageDigest.isEqual(guess, observedSignature)) {
                found = candidate;
                break;
            }
        }
        System.out.println("dictionary attack: secret = " + found + " after " + tries + " guesses");

        // Throughput of an offline attack on a single CPU thread
        int n = 300_000;
        t0 = System.nanoTime();
        for (int i = 0; i < n; i++) hmac(("k" + i).getBytes(), signingInput);
        double perSec = n / ((System.nanoTime() - t0) / 1e9);
        System.out.printf("offline HMAC guesses per second on one thread: %.0f%n", perSec);

        double lower8 = Math.pow(26, 8);
        System.out.printf("8 lowercase letters : %.2e candidates, %.1f hours on one thread%n",
            lower8, lower8 / perSec / 3600);
```

WeakSecret.java (continued) JAVA

```
        double bits256 = Math.pow(2, 256);
        System.out.printf("256-bit random key  : %.2e candidates, not searchable%n", bits256);
    }
}
```

Offline guessing of an HS256 secret from a single token (typical output; throughput varies) TERMINAL

```
$ java WeakSecret.java
dictionary attack: secret = secret123 after 8 guesses
offline HMAC guesses per second on one thread: 433642
8 lowercase letters : 2.09e+11 candidates, 133.8 hours on one thread
256-bit random key  : 1.16e+77 candidates, not searchable
```

Root cause. HMAC verification is fast, so a short or guessable secret falls in seconds, and a secret in version control is public to everyone with read access, now and in the future history. Because the same secret signs and verifies, any service that holds it can also forge tokens.

Fix. Rotate immediately: generate a new key (`openssl rand -base64 32` for HS256, or an RSA or EC key pair), inject it from the environment or a secret store (Kubernetes Secret, Vault, cloud secret manager), never the repository, and invalidate old tokens by changing the `iss`, the `kid` set or the key. Prefer asymmetric keys when more than one service verifies. A start-up check removes the weak-key class of defect.

KeyStrengthCheck.java (fragment) JAVA

```
@Bean
SecretKey hmacKey(@Value("${app.jwt.hmac-secret}") String base64Secret) {
    byte[] key = Base64.getDecoder().decode(base64Secret);
    if (key.length < 32) {
        throw new IllegalStateException("HS256 secret must be at least 256 bits");
    }
    return new SecretKeySpec(key, "HmacSHA256");
}
```

Prevention. Secret scanning in CI (a pre-commit hook and a pipeline step), distinct keys per environment, a documented rotation procedure that is exercised, and `kid`-based rotation so that old and new keys overlap.

Problem 15: a forged token is accepted (alg = none, algorithm confusion)

Scenario. A service uses a small hand-written verifier derived from a blog post, or an old library version that honours the header algorithm. A tester submits a token with `{"alg":"none"}` and role `ADMIN`.

Symptoms. Requests with an unsigned or re-signed token succeed; the signature part of the accepted token is empty or was computed with the public key.

Diagnosis. Review the verifier: does it read `alg` from the token and branch on it? Does it pass one key variable to several algorithm implementations? Reproduce both attacks as in the plain-Java run above. In Spring, inspect the decoder bean: it must be built with a fixed algorithm (`withPublicKey(...)` with RS256, or `jws-algorithms` in properties), and no custom `JwtDecoder` should parse the header manually.

Root cause. The token, which is untrusted input, selected the verification method.

Fix. Use `NimbusJwtDecoder` (or the library of the platform) with a fixed algorithm and key, never a hand-written verifier; if a custom one is unavoidable, implement the strict variant shown in the run, which compares the header `alg` with the configured value before any cryptographic operation.

Prevention. Upgrade libraries regularly, add a regression test with an `alg=none` token and an HS256 token signed with the public key, and restrict the accepted algorithm list in configuration (`spring.security.oauth2.resourceserver.jwt.jws-algorithms`).

Problem 16: a stolen refresh token

Scenario. A malware-infected browser extension copied the refresh cookie of a customer. The attacker uses it from another country to mint access tokens.

Symptoms. Two clients refresh in alternation; the legitimate user is logged out unexpectedly; logs show refresh calls from different addresses and user agents for one family.

Diagnosis. Query the `refresh_token` table by `family_id`: used tokens followed by new ones issued from two different source addresses (store the address and user agent with each token) indicate a fork. The rotation simulation above shows the server-side signature of the event: a presented token that is already marked used.

Root cause. A long-lived bearer credential without single-use semantics remains valid for whoever holds it.

Fix. Rotation with reuse detection (`RefreshTokenService`), revocation of the family, a notification to the user, and binding the token to context where possible (sender-constrained tokens such as DPoP or mTLS in the OAuth ecosystem). Absolute lifetime limits (for example 30 days since login regardless of rotation) cap the damage of an undetected theft.

Prevention. Store the token only in an `HttpOnly` cookie or the platform keystore, keep refresh tokens off script-readable storage, alert on reuse events, and provide the user with a list of active sessions with a "sign out everywhere" action that revokes all families of the user.

WATCH OUT
Never log tokens. Access and refresh tokens in request logs, exception messages or analytics events are credentials in plain text for everyone with log access. Mask the `Authorization` header and the refresh cookie in access logs, and in front proxies.

3.5.5 Questions to expect

1. **What are the parts of a JWT and what does the signature cover?** Header, payload and signature, each Base64url encoded; the signature covers the ASCII bytes of `header.payload` exactly as transmitted. It provides integrity and authenticity, not confidentiality.
2. **HS256 versus RS256: when is each appropriate?** HS256 uses one shared secret for signing and verifying, so it suits a single service that is both issuer and verifier. RS256 separates the private signing key from the public verification key, which is required when several services or third parties verify tokens; it is the Spring resource server default.
3. **Which checks must a verifier perform?** Signature with a server-fixed algorithm and key, `exp` and `nbf` with limited skew, `iss`, `aud`, and application conditions such as scope, token type and revocation.
4. **What is the `alg=none` attack and how is it prevented?** The attacker declares the unsecured algorithm and supplies an empty signature; a verifier that honours the header accepts the token. Prevention is a fixed algorithm in the verifier and the use of a maintained library.
5. **How is a JWT revoked?** Not directly. Short lifetimes bound the exposure; a `jti` deny list or a per-user token version closes the window at the price of a lookup; refresh tokens are stored server-side and revoked there.
6. **Describe refresh token rotation.** Each refresh invalidates the presented token and issues a new one in the same family; presenting an already-used token signals theft and revokes the family.
7. **Where should a browser application store tokens?** Not in `localStorage`. The access token in memory and the refresh token in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie scoped to the refresh path, or a backend-for-frontend that keeps tokens server-side.
8. **How does Spring Security validate a bearer token?** `BearerTokenAuthenticationFilter` extracts the token and calls `JwtAuthenticationProvider`, which uses a `JwtDecoder` (signature and validators) and a `JwtAuthenticationConverter` to build a `JwtAuthenticationToken`; failures go to `BearerTokenAuthenticationEntryPoint` and yield 401.
9. **What is clock skew and what does Spring default to?** The tolerated difference between the clocks of issuer and verifier when checking `exp` and `nbf`; `JwtTimestampValidator` allows 60 seconds by default.

Check yourself

1. What does `Base64.getUrlEncoder().withoutPadding().encodeToString("{\"alg\":\"none\"}".getBytes())` return?
2. A token's payload decodes to `{"sub":"u-1","role":"USER"}`. An attacker edits it to `"role":"ADMIN"` and re-encodes it. Why does verification fail?
3. A verifier is configured for RS256 and receives a token with header `{"alg":"HS256"}`. What must it do?
4. Why is it acceptable to store a SHA-256 digest of a refresh token but not of a password?
5. What is wrong with `claims.put("exp", System.currentTimeMillis() + 900_000)`?

Answers

1. `eyJhbGciOiJub25lIn0` (verified by running it).
2. The signature was computed over the original header and payload bytes; the changed payload produces a different signing input, so the signature no longer matches, and the attacker cannot recompute it without the key.

3. Reject the token before any cryptographic operation, because the algorithm is not the configured one.
4. A refresh token is 256 random bits, so guessing it is infeasible even against a fast hash; passwords have low entropy and need a slow, salted function.
5. `exp` is defined in seconds since the epoch; milliseconds put the expiry about a thousand times too far in the future, so the token effectively never expires.

KEY POINTS

- A JWT is signed, not encrypted: integrity and authenticity yes, confidentiality no; the verifier, not the token, must decide the algorithm and key.
- Validate signature, `exp`, `nbf` with limited skew, `iss`, `aud` and application rules; attacks such as `alg=none` and RS256-to-HS256 confusion succeed only against verifiers that obey the header, as the run above demonstrates.
- Short-lived access tokens plus single-use, hashed, server-stored refresh tokens with family revocation give revocation without a lookup on every request; add a `jti` deny list where immediate logout matters.
- In Spring Boot, `oauth2ResourceServer().jwt()` with a `NimbusJwtDecoder` that has a fixed algorithm and a `JwtAuthenticationConverter` for the `roles` claim is the standard implementation; hand-written filters are rarely justified.
- Keys belong in a secret store, never in the repository; weak HMAC secrets fall to offline guessing in seconds.

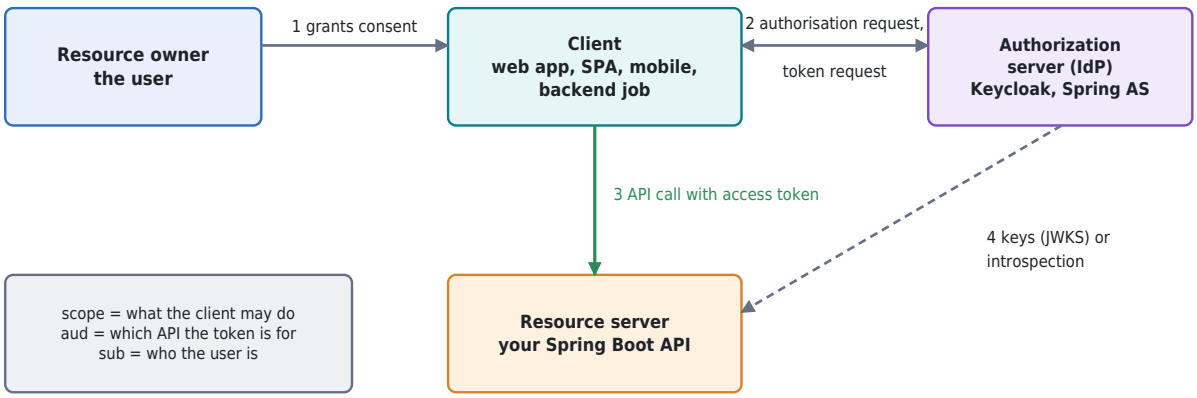
3.6 OAuth 2.0 and OpenID Connect

JWT login endpoints written by each team solve authentication again and again, and they cannot offer what users and partners expect: single sign-on across applications, login with an existing corporate or social account, and delegated access in which a third-party application acts for a user without ever learning the password. OAuth 2.0 (RFC 6749) is the standard framework for delegated authorisation, and OpenID Connect (OIDC) adds an identity layer on top of it. In a Spring Boot landscape the typical arrangement is a dedicated authorisation server (Keycloak, Spring Authorization Server, a cloud identity provider), web or mobile clients that obtain tokens from it, and Spring Boot services that act as resource servers and only validate tokens. This chapter explains the roles, the grant types, the authorisation code flow with PKCE in detail, the OIDC tokens, and the Spring Security configuration of the three Spring roles: client, resource server and authorisation server.

3.6.1 Concept and mechanism

Roles and tokens

OAuth 2.0 defines four roles. The **resource owner** is the user who owns the data. The **client** is the application that wants access, such as a web application, a single-page application, a mobile app or a backend job. The **authorisation server** authenticates the user, obtains consent and issues tokens. The **resource server** is the API that accepts access tokens. The central idea is that the client never handles the user's password: the user authenticates at the authorisation server, and the client receives a token with limited scope and lifetime.



The client never sees the password; the resource server never sees the user's credentials.

Figure 3.17. The user grants consent at the authorisation server, the client exchanges the grant for a token, and the resource server only validates the token it receives.

The **access token** is the credential presented to the resource server. It may be a JWT, which the resource server verifies locally with keys from the JWKS endpoint of the issuer, or an opaque string, which the resource server sends to the introspection endpoint (RFC 7662). Its important claims are `scope` (what the client may do), `aud` (which API it is for), `sub` (the user, or the client in machine flows) and `exp`. The **refresh token** lets the client obtain new access tokens and is handled as in the previous chapter. The **ID token** exists only in OpenID Connect and is a JWT addressed to the client; it states who authenticated and how.

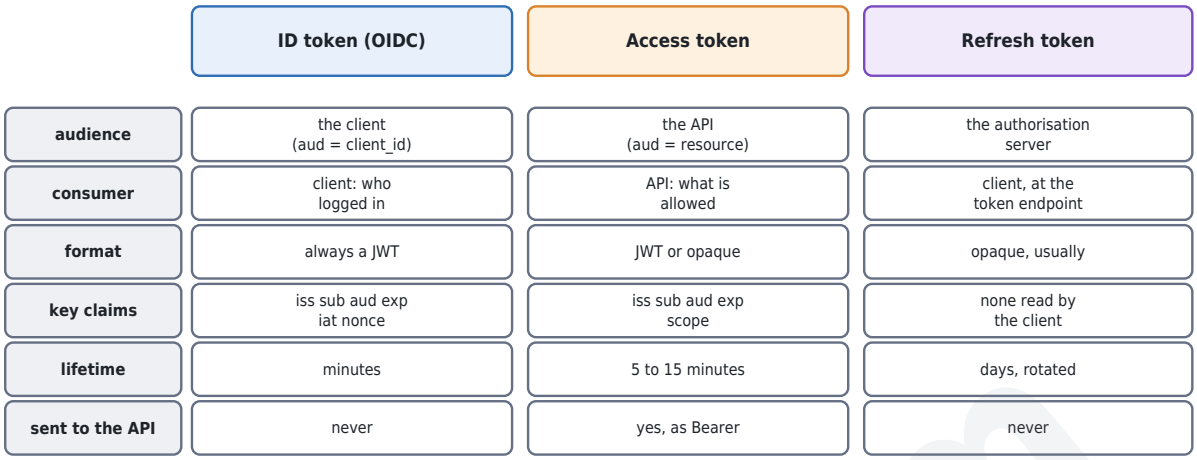


Figure 3.18. The three tokens have different audiences: the ID token is for the client, the access token is for the API, and the refresh token is for the authorisation server; only the access token is ever sent to the API.

A frequent error is to use the ID token as an API credential, or to read user attributes from the access token of a third party. OIDC separates the concerns: the client reads the ID token to learn who logged in (validating `iss`, `aud` equal to its own client id, `exp` and the `nonce` it sent) and optionally calls the UserInfo endpoint for more claims; it sends the access token to the API. OIDC providers publish a discovery document at `/.well-known/openid-configuration` that lists the endpoints and the JWKS location, which is what Spring Security uses when `issuer-uri` is configured.

Grant types

A grant type is the way in which a client obtains a token. The choice follows from who calls the API.

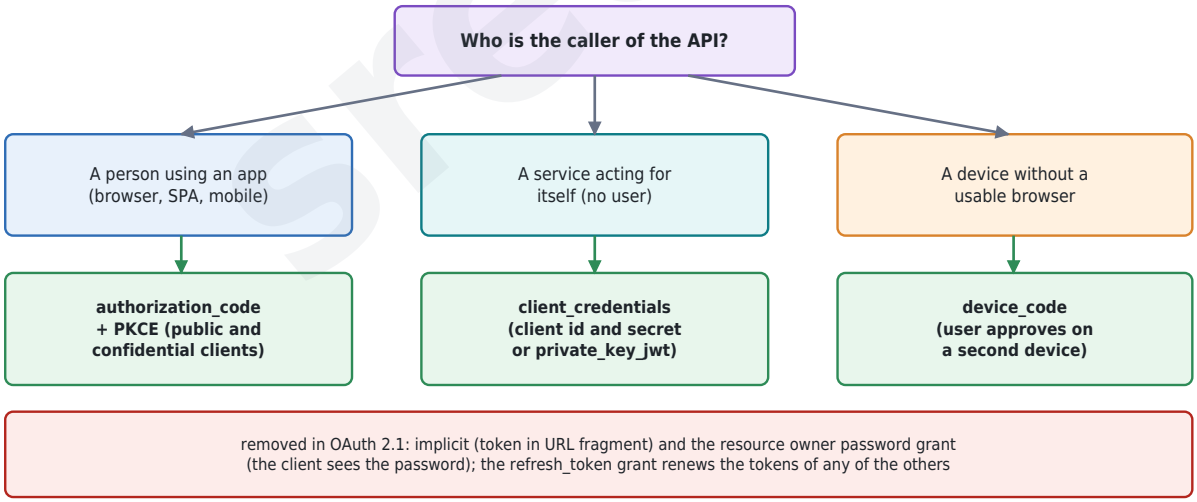


Figure 3.19. The caller determines the grant: a person uses the authorisation code with PKCE, a service acting on its own uses client credentials, and a device without a browser uses the device code.

- **Authorisation code with PKCE** is the grant for every client that acts for a user, including confidential server-side applications since OAuth 2.1. The client redirects the browser to the authorisation server, receives a short-lived code through the redirect, and exchanges it for tokens over a back channel.
- **Client credentials** has no user: the client authenticates with its own credentials (secret, or a signed assertion, or mTLS) and receives a token whose `sub` is the client. It is the grant for service-to-service calls and scheduled jobs.
- **Refresh token** renews tokens without involving the user.
- **Device code** serves televisions and command-line tools: the device shows a code, and the user approves on a second device.

The **implicit grant** (token returned in the URL fragment) and the **resource owner password credentials grant** (the client collects the password) are removed in the OAuth 2.1 draft and in current security best practice. A client that asks for the user's password defeats the purpose of OAuth, and the implicit grant exposes tokens in browser history and referrer headers.

Authorisation code with PKCE

The flow has a front channel (browser redirects, visible to the user and to anything on the device) and a back channel (direct HTTP between client and server). The front channel carries only a short-lived, single-use **code**; the tokens travel on the back channel. The code alone is not enough to obtain tokens, because the token request must also prove that it comes from the party that started the flow.

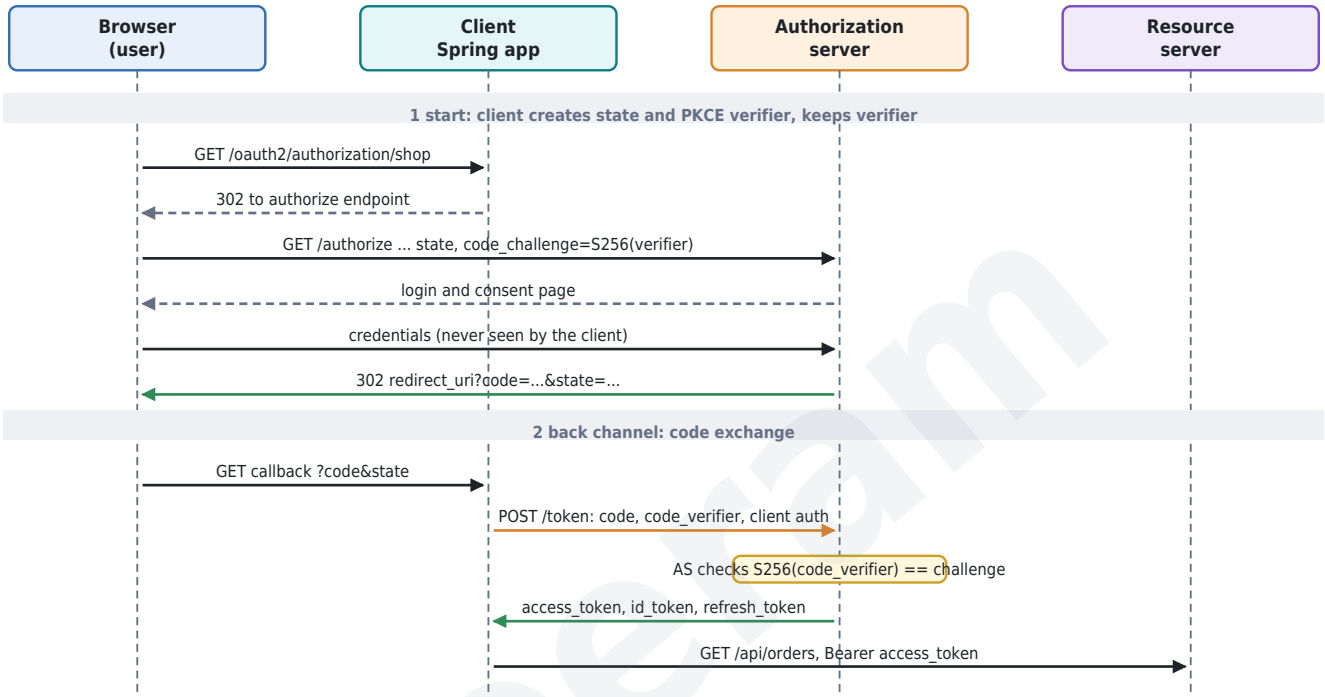


Figure 3.20. The browser carries the code and the state through redirects, while the tokens are obtained by the client directly from the token endpoint with the code and the PKCE verifier.

Four parameters protect the flow. `state` is a random value the client creates, stores and compares when the redirect returns; it binds the response to the session that started the request and defends against CSRF on the callback. `redirect_uri` must match a value registered for the client exactly; this prevents the code from being delivered to an attacker. `nonce` (OIDC) ties the ID token to the request and defends against token replay. **PKCE** (Proof Key for Code Exchange, RFC 7636) defends against interception of the code: the client generates a random `code_verifier`, sends `code_challenge = BASE64URL(SHA-256(code_verifier))` with the authorisation request (method `S256`), and sends the verifier itself with the token request. The server recomputes the challenge and rejects a mismatch. An attacker who steals only the code, for example through a malicious app registered for the same custom URL scheme, cannot redeem it. The computation can be verified with the test vector of RFC 7636:

Pkce.java JAVA

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;

public class Pkce {
    static String s256(String verifier) throws Exception {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] d = md.digest(verifier.getBytes(StandardCharsets.US_ASCII));
        return Base64.getUrlEncoder().withoutPadding().encodeToString(d);
    }

    public static void main(String[] args) throws Exception {
        // RFC 7636 appendix B test vector
        String verifier = "dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk";
        System.out.println("RFC 7636 verifier  : " + verifier);
        System.out.println("computed challenge : " + s256(verifier));
```

Pkce.java (continued) JAVA

```
        System.out.println("RFC 7636 challenge : E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM");
        System.out.println("equal              : "
            + s256(verifier).equals("E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM"));

        // fresh verifier from 32 random bytes
        byte[] raw = new byte[32];
        new SecureRandom().nextBytes(raw);
        String v = Base64.getUrlEncoder().withoutPadding().encodeToString(raw);
        System.out.println("generated verifier length: " + v.length()
                + " characters (allowed 43..128)");

        // Authorization server check at the token endpoint
        String challengeStored = s256(v);
        System.out.println("token request with the right verifier : "
            + (s256(v).equals(challengeStored) ? "code accepted" : "invalid_grant"));
        System.out.println("token request by an interceptor (no verifier): "
            + (s256("guess").equals(challengeStored) ? "code accepted" : "invalid_grant"));
    }
}
```

PKCE challenge computation (RFC 7636 test vector, run on the build machine) TERMINAL

```
$ java Pkce.java
RFC 7636 verifier  : dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
computed challenge : E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
RFC 7636 challenge : E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
equal              : true
generated verifier length: 43 characters (allowed 43..128)
token request with the right verifier : code accepted
token request by an interceptor (no verifier): invalid_grant

== refresh token rotation ==
login issued         : rt-...
legitimate refresh   : 200 new refresh to...
attacker replays t1  : 401 REUSE DETECTED -> family fam-1 revoked
legitimate uses t2   : 401 family revoked
```

An authorisation request therefore looks like the following (line breaks added), and a token request with the code and verifier follows on the back channel.

Authorisation request and token request (HTTP, abridged)

```
GET /realms/shop/protocol/openid-connect/auth
    ?response_type=code
    &client_id=shop-web
    &redirect_uri=https%3A%2F%2Fapp.example.com%2Flogin%2Foauth2%2Fcode%2Fshop
    &scope=openid%20profile%20orders.read
    &state=Zm9vYmFy
    &nonce=n-0S6_WzA2Mj
    &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
    &code_challenge_method=S256

POST /realms/shop/protocol/openid-connect/token
Content-Type: application/x-www-form-urlencoded
Authorization: Basic c2hvcC13ZWI6Y2xpZW50LXNlY3JldA==   (confidential client only)

grant_type=authorization_code&code=SplxlOBeZQQYbYS6WxSbIA
    &redirect_uri=https%3A%2F%2Fapp.example.com%2Flogin%2Foauth2%2Fcode%2Fshop
    &code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

The Spring roles

Spring offers a module for each role. **OAuth2 Client** (`spring-boot-starter-oauth2-client`) implements the client side: `oauth2Login()` provides login through an OIDC provider (the `OAuth2AuthorizationRequestRedirectFilter` starts the flow, `OAuth2LoginAuthenticationFilter` handles the callback at `{baseUrl}/login/oauth2/code/{registrationId}`), stores the tokens in an `OAuth2AuthorizedClient`, and can call APIs with them. **OAuth2 Resource Server** (`spring-boot-starter-oauth2-resource-server`) validates bearer tokens, as in the previous chapter, with either a `JwtDecoder` or an opaque-token introspector. **Spring Authorization Server** (the `spring-security-oauth2-authorization-server` module, project version 1.x for Spring Security 6) implements the server role for teams that need their own issuer; for many organisations an existing product such as Keycloak, Okta, Auth0 or Microsoft Entra ID is the more economical choice and Spring Boot services only consume it.

A service can combine roles: an API that validates incoming tokens (resource server) and calls another API with a client credentials token (client). A web application with server-side rendering that logs users in with OIDC (client) and then relays the user's token to its APIs does the same. The gateway case, where the token is relayed automatically, is covered in the chapter on securing microservices.

3.6.2 Worked example

The example secures the order API as a resource server of a Keycloak realm, adds browser login with `oauth2Login`, and shows a service-to-service call with client credentials. The Spring code was not run on the build machine.

Resource server. The properties name the issuer; at startup (or on first use) Spring discovers the JWKS URI from the issuer's metadata and caches the keys. The scope claim becomes `SCOPE_` authorities, so scope rules use `hasAuthority`.

application.yml (resource server) YAML

```yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://sso.example.com/realms/shop
          audiences: shop-api
```

ResourceServerConfig.java (fragment of a configuration class) JAVA

```java
@Bean
@Order(1)
SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers(HttpMethod.GET, "/api/orders/**").hasAuthority("SCOPE_orders.read")
            .requestMatchers(HttpMethod.POST, "/api/orders/**").hasAuthority("SCOPE_orders.write")
            .anyRequest().denyAll())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
    return http.build();
}
```

With `issuer-uri`, Boot configures a decoder that validates the signature, the timestamps and the `iss` claim; the `audiences` property adds the `aud` check. Keycloak places realm roles in `realm_access.roles` rather than `scope`, so role-based rules need a custom `JwtAuthenticationConverter` that reads that nested claim; the claim structure differs per provider, which is why the converter is application code. Each decision should be taken from the token that the provider actually issues (decode a sample first).

Browser login. A server-rendered or backend-for-frontend application logs users in through OIDC. Boot builds the `ClientRegistration` from the properties; the redirect URI default is `{baseUrl}/login/oauth2/code/{registrationId}`, and this value must be registered at the provider.

application.yml (OIDC login) YAML

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          shop:
            client-id: shop-web
            client-secret: ${SHOP_WEB_SECRET}
            scope: openid,profile,email,orders.read
            authorization-grant-type: authorization_code
        provider:
          shop:
            issuer-uri: https://sso.example.com/realms/shop
```

WebLoginConfig.java (fragment) JAVA

```java
@Bean
@Order(2)
SecurityFilterChain webChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(a -> a
            .requestMatchers("/", "/assets/**").permitAll()
            .anyRequest().authenticated())
        .oauth2Login(Customizer.withDefaults())
        .logout(l -> l.logoutSuccessUrl("/"));
    return http.build();
}
```

The authenticated principal is an `OidcUser`; a controller reads claims from it with `@AuthenticationPrincipal OidcUser user`. Spring Security applies PKCE automatically for public clients (a registration without a secret and with `client-authentication-method: none`); recent versions of Spring Security also apply it to confidential clients, in which case the authorisation request contains `code_challenge` and `code_challenge_method=S256`. The exact version in which this default was introduced should be checked against the reference documentation of the version in use.

Service-to-service with client credentials. The order service reads stock from an inventory service. It has its own client registration with the `client_credentials` grant, and a `WebClient` that obtains and caches the token and refreshes it before expiry. (The `WebClient` type comes from `spring-boot-starter-webflux`, which may be added next to the servlet stack for this purpose.)

application.yml (client credentials) YAML

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          inventory:
            provider: shop
            client-id: order-service
            client-secret: ${ORDER_SERVICE_SECRET}
            authorization-grant-type: client_credentials
            scope: inventory.read
```

InventoryClientConfig.java JAVA

```java
package com.example.shop.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.client
        .AuthorizedClientServiceOAuth2AuthorizedClientManager;
import org.springframework.security.oauth2.client.OAuth2AuthorizedClientProviderBuilder;
import org.springframework.security.oauth2.client.OAuth2AuthorizedClientService;
import org.springframework.security.oauth2.client.registration.ClientRegistrationRepository;
import org.springframework.security.oauth2.client.web.reactive.function.client
        .ServletOAuth2AuthorizedClientExchangeFilterFunction;
import org.springframework.web.reactive.function.client.WebClient;

@Configuration
public class InventoryClientConfig {

    @Bean
    AuthorizedClientServiceOAuth2AuthorizedClientManager authorizedClientManager(
            ClientRegistrationRepository registrations, OAuth2AuthorizedClientService clients) {
        AuthorizedClientServiceOAuth2AuthorizedClientManager manager =
                new AuthorizedClientServiceOAuth2AuthorizedClientManager(registrations, clients);
        manager.setAuthorizedClientProvider(
                OAuth2AuthorizedClientProviderBuilder.builder().clientCredentials().build());
        return manager;
    }

    @Bean
    WebClient inventoryClient(AuthorizedClientServiceOAuth2AuthorizedClientManager manager) {
        ServletOAuth2AuthorizedClientExchangeFilterFunction oauth =
                new ServletOAuth2AuthorizedClientExchangeFilterFunction(manager);
        oauth.setDefaultClientRegistrationId("inventory");
        return WebClient.builder()
                .baseUrl("http://inventory-service")
```

InventoryClientConfig.java (continued) JAVA

```
                .apply(oauth.oauth2Configuration())
                .build();
    }
}
```

The service-based manager (`AuthorizedClientServiceOAuth2AuthorizedClientManager`) works outside an HTTP request, for example in a scheduled job, because it does not need a request-bound repository. The access token is cached in the `OAuth2AuthorizedClientService` (in memory by default) and renewed when it expires, so the token endpoint is not called for every inventory request.

Spring Authorization Server. Teams that issue their own tokens add `spring-security-oauth2-authorization-server`, define a `RegisteredClientRepository` with the clients, a `JWKSource` with the signing key, and apply `OAuth2AuthorizationServerConfiguration.applyDefaultSecurity(http)` to a dedicated security chain. The configuration is considerably longer than that of a resource server and requires decisions about key rotation, consent storage and token customisation; it is justified when the organisation needs to own the identity provider, not merely for one application.

3.6.3 Hands-on

A client credentials token can be requested and decoded from the command line. The output is **typical output (not run on the build machine)**; token values are shortened.

Client credentials request against Keycloak (typical output, not run on the build machine) TERMINAL

```
$ curl -s -X POST https://sso.example.com/realms/shop/protocol/openid-connect/token \
    -d grant_type=client_credentials -d client_id=order-service \
    -d client_secret="$ORDER_SERVICE_SECRET" -d scope=inventory.read
{"access_token":"eyJhbGciOiJSUzI1NiIs...","expires_in":300,"token_type":"Bearer",
 "scope":"inventory.read"}

$ curl -s https://sso.example.com/realms/shop/.well-known/openid-configuration \
    | head -c 200
{"issuer":"https://sso.example.com/realms/shop","authorization_endpoint":
"https://sso.example.com/realms/shop/protocol/openid-connect/auth",...
```

The decoded payload of the access token for the machine client carries `iss`, `aud`, `sub` (the service-account identifier), `exp`, `scope` and `azp` (the authorised party, the client id). The user flows additionally yield an ID token with `nonce` and `auth_time`.

3.6.4 Real-world problems and solutions

Problem 17: the redirect fails with invalid redirect URI

Scenario. Login works on a developer machine and fails after deployment behind a load balancer: the provider shows "Invalid parameter: redirect_uri" or redirects to a different host.

Symptoms. The authorisation server displays an error page before the login form; or the browser reaches the provider with `redirect_uri=http://app.internal:8080/login/oauth2/code/shop`.

Diagnosis. Read the `redirect_uri` query parameter of the authorisation request in the browser's address bar and compare it character by character with the values registered for the client at the provider. Behind a TLS-terminating proxy the application builds the URI from the request it sees (plain HTTP and the internal host name), unless forwarded headers are honoured.

Root cause. The default template `{baseUrl}/login/oauth2/code/{registrationId}` is expanded from the incoming request; the scheme, host or port differs from the registered value, and the provider requires an exact match.

Fix. Make the application derive the external URL (`server.forward-headers-strategy=framework`, and a proxy that sets `X-Forwarded-Proto` and `X-Forwarded-Host`) or set the redirect URI explicitly in the registration (`redirect-uri: https://app.example.com/login/oauth2/code/{registrationId}`), and register exactly that value, without wildcards, at the provider.

Prevention. One registered redirect URI per environment, tested in the staging environment; never register wildcard redirect URIs, which let an attacker receive the code.

Problem 18: valid tokens are rejected with an issuer mismatch

Scenario. A resource server in Kubernetes is configured with `issuer-uri: http://keycloak:8080/realms/shop` (the internal service name). Clients obtain tokens through the public URL `https://sso.example.com/realms/shop`.

Symptoms. Every call returns 401 with `WWW-Authenticate: Bearer error="invalid_token"` and the log message `The iss claim is not valid`; alternatively the service fails to start because the issuer metadata cannot be retrieved.

Diagnosis. Decode the token payload and read `iss`; compare it with the configured `issuer-uri` and with the `issuer` field of `/.well-known/openid-configuration` fetched from the URL the service uses. The three must be identical strings. For start-up failures check network reachability and TLS trust from the pod (`kubectl exec` into the pod and run `curl -v` against the issuer URL).

Root cause. The `iss` claim contains the URL under which the authorisation server was reached when the token was issued; the validator compares it with the configured issuer literally. Internal and external names produce different strings.

Fix. Configure the authorisation server with a fixed frontend URL (for Keycloak, the hostname setting) so that tokens always carry the public issuer, then configure the resource server with that public issuer for validation and, if the public URL is not reachable from the cluster, supply the key location explicitly with `jwk-set-uri` while the issuer check keeps using the public value.

application.yml YAML

```yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://sso.example.com/realms/shop
          jwk-set-uri: http://keycloak:8080/realms/shop/protocol/openid-connect/certs
```

Prevention. Treat the issuer as a contract string; test token validation in an environment with the same topology as production.

Problem 19: authenticated but forbidden (scope and audience confusion)

Scenario. A single-page client logs in successfully and obtains an access token, yet every API call returns 403. In another case the API accepts a token that was issued for a different API of the same realm.

Symptoms. 403 with a valid token (first case); access to data by a client that should not have it (second case).

Diagnosis. Decode the access token: compare `scope` (or `scp`) with the authority the endpoint requires (`SCOPE_orders.read` expects the scope `orders.read`), and check `aud`. A token that lacks the scope was requested without it: the client did not include `orders.read` in its authorisation request, or the provider's client configuration does not allow it. A token accepted by the wrong API shows an `aud` that does not name this API, and a resource server without an audience validator.

Root cause. Scopes are granted per client and per request; they do not originate in the user's roles. The default `JwtGrantedAuthoritiesConverter` maps only the scope claim. Audience validation is not performed unless configured.

Fix. Request the scope in the client registration and permit it for the client at the provider; configure `spring.security.oauth2.resourceserver.jwt.audiences` or an `OAuth2TokenValidator<Jwt>` that requires the expected `aud`.

AudienceValidator.java (fragment) JAVA

```java
OAuth2TokenValidator<Jwt> audience = jwt -> jwt.getAudience().contains("shop-api")
        ? OAuth2TokenValidatorResult.success()
        : OAuth2TokenValidatorResult.failure(
                new OAuth2Error("invalid_token", "wrong audience", null));
decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
        JwtValidators.createDefaultWithIssuer(issuer), audience));
```

Prevention. Decide per API which scope names exist (`orders.read`, `orders.write`) and which claim carries the audience; include a negative test that a token for another audience is rejected.

> **NOTE**
> Single sign-on with social providers uses the same mechanism: `oauth2Login()` with a registration for Google or GitHub. The identity provider authenticates the person; the application still decides authorisation locally, typically by mapping the verified e-mail or the `sub` claim to a local account and assigning roles itself.

3.6.5 Questions to expect

1. **What problem does OAuth 2.0 solve?** Delegated access: a client obtains limited, revocable access to a resource on behalf of a user without receiving the user's password.
2. **Distinguish OAuth 2.0 from OpenID Connect.** OAuth 2.0 authorises access to APIs and says nothing about who the user is. OIDC adds an ID token, the UserInfo endpoint and standard claims, and thereby standardises authentication.
3. **Explain the authorisation code flow with PKCE.** The client sends the user to the authorisation server with `state` and a `code_challenge`; after login the server redirects back with a code; the client exchanges the code and the `code_verifier` for tokens on the back channel; the server verifies that the verifier hashes to the challenge.
4. **Why is the implicit grant deprecated?** The access token is returned in the URL fragment, where it is exposed to browser history, extensions and referrers, and there is no client authentication or code binding; the code flow with PKCE replaces it, even for single-page applications.
5. **When is client credentials used?** For machine-to-machine calls in which no user is involved, with the client's own identity as subject.
6. **What is the difference between an ID token and an access token?** The ID token is addressed to the client and proves authentication; the access token is addressed to the resource server and authorises calls. The ID token must not be sent to an API as a credential.
7. **How does a Spring Boot resource server validate a JWT?** It fetches the signing keys from the JWKS URI discovered through the issuer, verifies the signature, checks `exp`, `nbf` and `iss` and optionally `aud`, and maps `scope` to `SCOPE_` authorities.
8. **JWT access tokens versus opaque tokens?** JWTs are verified locally and scale well but cannot be revoked before expiry; opaque tokens require an introspection call per request (cacheable) and allow immediate revocation.
9. **What is the purpose of `state` and of `nonce`?** `state` binds the callback to the browser session that started the flow (CSRF on the redirect); `nonce` binds the ID token to the request and prevents replay.

Check yourself

1. Which of `state`, `nonce` and `code_verifier` is sent to the token endpoint?
2. A resource server receives a token whose `aud` is `billing-api`. It serves `orders-api`. What should happen and which setting enforces it?
3. Why is a wildcard `redirect_uri` dangerous?
4. For a nightly job that calls the inventory API, which grant fits, and which Spring type obtains the token outside an HTTP request?
5. What does the computed challenge equal for the verifier `dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk`?

Answers

1. `code_verifier` (together with the code and, for confidential clients, client authentication). `state` and `nonce` travel with the authorisation request and are checked by the client.
2. The token must be rejected with 401; an audience validator or `spring.security.oauth2.resourceserver.jwt.audiences` enforces it.
3. The authorisation server could deliver the code to an attacker-controlled URL matching the wildcard, and the attacker could then redeem it; exact matching prevents that.
4. `client_credentials`; `AuthorizedClientServiceOAuth2AuthorizedClientManager` works without a request.
5. `E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM`, as the RFC 7636 test vector and the run above show.

KEY POINTS

- OAuth 2.0 delegates authorisation without sharing passwords; OIDC adds authentication through the ID token; the access token is for the API, the ID token for the client.
- Use authorisation code with PKCE for users and client credentials for services; the implicit and password grants are obsolete.
- `state`, `nonce`, exact `redirect_uri` matching and PKCE each close a specific attack; a computed PKCE challenge can be checked against the RFC 7636 test vector.
- Spring Security provides `oauth2Login`, `oauth2Client` and `oauth2ResourceServer`; a Boot service is usually a resource server of an existing identity provider.
- Issuer string, audience and scope mapping are the three usual causes of valid-looking tokens being rejected or wrongly accepted.

3.7 Securing Microservices and Testing Security

In a monolith the perimeter is the application: one login, one session, one process. In a set of services the same request crosses a gateway and several services, each reachable from inside the network. Security that relies on the network boundary ("internal calls are trusted") fails as soon as one service is compromised or a misrouted request arrives, so each service must establish for itself who is calling and what the caller may do. This chapter covers the identity models for service-to-service calls (user token relay, service identity, mutual TLS, API keys), where validation belongs, and how to test all of it with `spring-security-test`.

3.7.1 Concept and mechanism

Two identities in every internal call

An internal call has two possible subjects. The **end user** on whose behalf the request is made, and the **calling service** that makes it. Authorisation decisions in the callee usually need one or both: "this user may read this order" and "only the order service may reserve stock". Conflating them produces either over-broad access (every service acts as an administrator) or lost audit trails (every action appears to come from a service account). Three mechanisms carry the identities.

- **Token relay.** The gateway or the first service forwards the user's access token to downstream services, which validate it as resource servers. The user identity and scopes arrive unchanged. The token must carry an audience that every receiving service accepts, and a service that relays the token gains the ability to act with the full authority of the user; therefore the audience and scope design must be narrow.
- **Service identity with client credentials.** The calling service obtains its own token (see the previous chapter) and the callee authorises on the service identity and scopes such as `inventory.reserve`. Background jobs and event consumers have no user token and must use this model.
- **Token exchange (RFC 8693).** The service presents the user's token to the authorisation server and receives a new token with a different audience and reduced scope that still identifies the user (and records the acting service in an `act` claim). It combines both identities without handing the original token around; support depends on the identity provider.

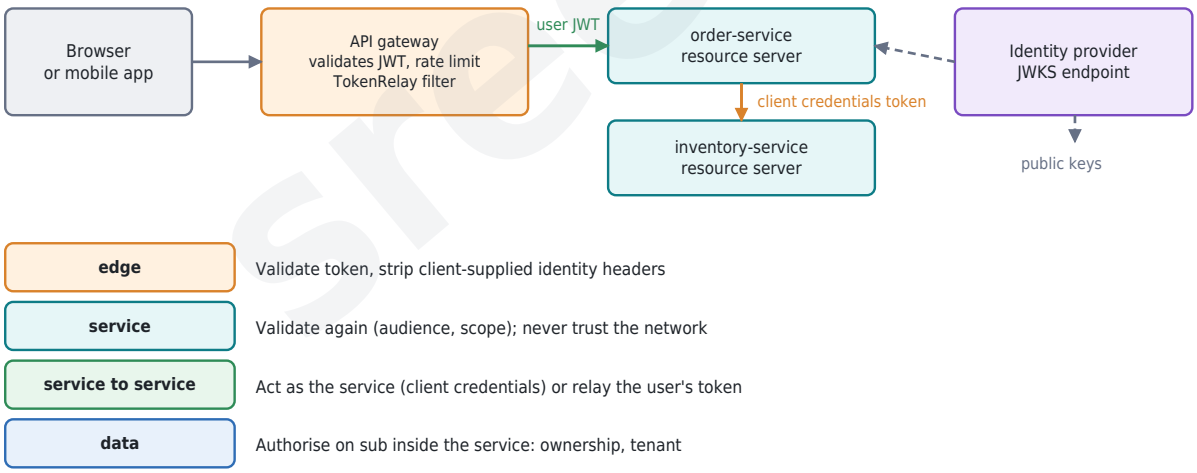


Figure 3.21. Every layer validates the token itself; the gateway strips identity headers supplied by clients, and services authorise on the subject inside the token rather than on network position.

Where validation belongs

Validation at the gateway alone is a common design with a standing weakness: any caller that reaches a service directly (a compromised pod, a mistaken route, a developer port-forward) bypasses it. The robust pattern is defence in depth. The gateway validates the token, applies rate limits and rejects obviously invalid requests early, which protects the services from load. Each service additionally runs as a resource server, validates signature, issuer, audience and expiry, and performs ownership checks against the `sub` claim. The cost is one signature verification per hop, which is small compared with the network call.

Identity must never be carried in plain headers that a client could set. A header such as `X-User-Id` written by the gateway is trustworthy only if the gateway removes any such header arriving from outside and the services are reachable exclusively through the gateway; the first condition is easy to forget and the second is difficult to guarantee. Signed tokens do not have this weakness.

Mutual TLS

Transport Layer Security normally authenticates only the server. In **mutual TLS** (mTLS) the client also presents a certificate during the handshake, so the server knows which service is calling before any application data is exchanged, and the channel is encrypted in both directions. Certificates are issued by an internal certificate authority and are kept short-lived (hours to days) so that revocation is rarely needed. In Kubernetes the usual solution is a service mesh (Istio, Linkerd) or a certificate manager that issues and rotates the certificates, leaving the application unchanged. A Spring Boot service can also terminate mTLS itself with `server.ssl.client-auth=need` and a trust store, and read the client identity from the certificate with `http.x509(...)`. mTLS authenticates the service, not the user, so it complements token relay rather than replacing it.

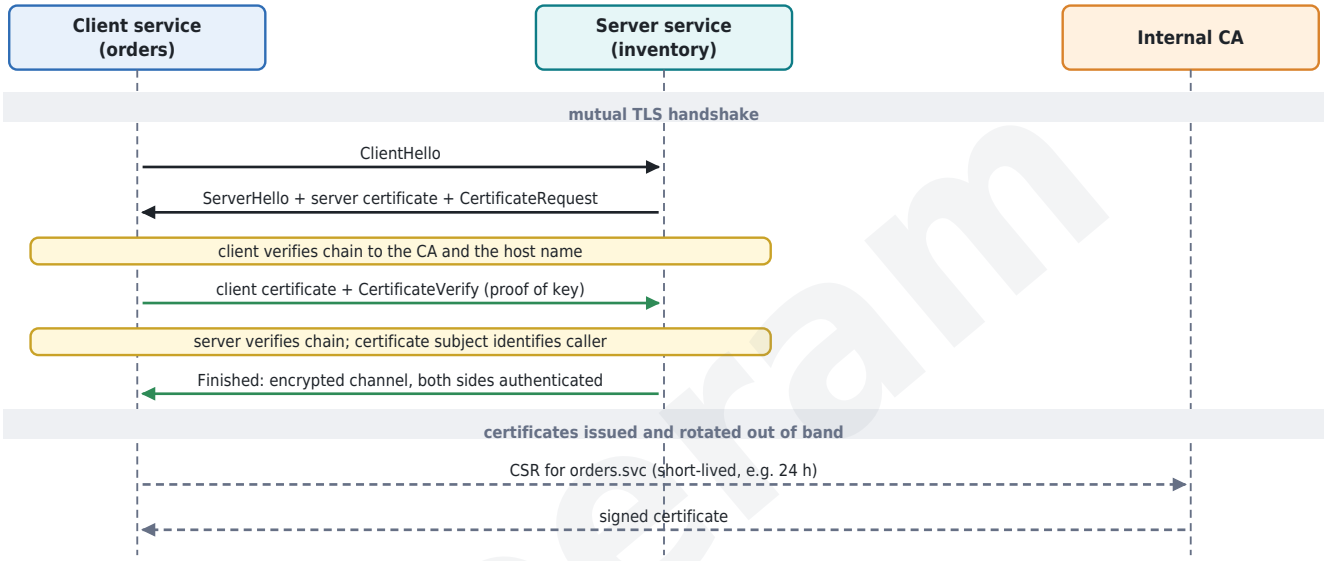


Figure 3.22. In mutual TLS both sides present certificates signed by the internal CA, so the server learns the caller identity at the connection level; the certificates are issued and rotated outside the application.

API keys

An API key is a long random secret that identifies a client application, used where OAuth is not offered, for example for partner integrations and webhooks. Treat it as a password: generate 32 or more random bytes, show it to the owner once, store only its digest (SHA-256 suffices for high-entropy values), compare in constant time, scope it to specific operations, support several active keys per client for rotation, and transmit it only in a header over TLS, never in a URL, because URLs are logged. A short non-secret prefix (for example `shk_live_ab12`) lets the owner and the support staff identify a key without revealing it.

3.7.2 Worked example

The order service accepts bearer tokens from users and API keys from partner systems on a separate path. The first part is the gateway route that validates the token and relays it; the second is a key filter for `/partner/**`.

application.yml (gateway; token relay) YAML

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - TokenRelay=
            - RemoveRequestHeader=Cookie
            - RemoveRequestHeader=X-User-Id
  security:
```

application.yml (gateway; token relay) (continued) YAML

```yaml
    oauth2:
      client:
        registration:
          shop:
            client-id: gateway
            client-secret: ${GATEWAY_SECRET}
            scope: openid,orders.read,orders.write
        provider:
          shop:
            issuer-uri: https://sso.example.com/realms/shop
```

The `TokenRelay` filter takes the access token of the logged-in user from the gateway's `OAuth2AuthorizedClient` and adds it as a bearer header to the downstream request; it belongs to Spring Cloud Gateway (a reactive application, with the OAuth2 client starter). A gateway that merely validates tokens presented by clients forwards the incoming `Authorization` header unchanged and needs only the resource server configuration. The module naming of Spring Cloud Gateway changed in recent Spring Cloud releases, so the starter name must be taken from the release train in use.

The partner key filter looks the digest up in a table. The filter is constructed inside the security configuration, as established earlier, and populates an authentication with an authority per scope.

ApiKeyFilter.java JAVA

```java
package com.example.shop.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.HexFormat;
import java.util.List;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;

/** Not a Spring bean: constructed in the security configuration. */
public class ApiKeyFilter extends OncePerRequestFilter {

    public record Client(String name, List<String> scopes, boolean active) {
    }

    public interface KeyStore {
        java.util.Optional<Client> findByDigest(String sha256Hex);
    }

    private final KeyStore keys;

    public ApiKeyFilter(KeyStore keys) {
        this.keys = keys;
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        return !request.getRequestURI().startsWith("/partner/");
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws ServletException, IOException {
        String presented = req.getHeader("X-API-Key");
        if (presented == null) {
            res.sendError(HttpServletResponse.SC_UNAUTHORIZED);
            return;
        }
        // lookup by digest: no secret comparison happens in application code
        var client = keys.findByDigest(sha256(presented)).filter(Client::active);
        if (client.isEmpty()) {
```

ApiKeyFilter.java (continued) JAVA

```
            res.sendError(HttpServletResponse.SC_UNAUTHORIZED);
            return;
        }
        var authorities = client.get().scopes().stream()
                .map(s -> new SimpleGrantedAuthority("SCOPE_" + s)).toList();
        SecurityContext ctx = SecurityContextHolder.createEmptyContext();
        ctx.setAuthentication(UsernamePasswordAuthenticationToken.authenticated(
                client.get().name(), null, authorities));
        SecurityContextHolder.setContext(ctx);
        chain.doFilter(req, res);
    }

    private static String sha256(String v) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            return HexFormat.of().formatHex(md.digest(v.getBytes(StandardCharsets.UTF_8)));
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

Looking the key up by its digest in an indexed column avoids comparing secrets in application code at all; the database equality on a 256-bit digest leaks no useful timing information about the key itself. The filter is registered with `addFilterBefore(new ApiKeyFilter(store), AuthorizationFilter.class)` on a chain whose `securityMatcher` is `/partner/**`, and the rules for that chain use `hasAuthority("SCOPE_orders.write")`.

3.7.3 Testing security

Security rules are code and need tests with the same status as business logic: a rule that is never exercised is a rule that fails silently after a refactoring. The module `spring-security-test` supplies the tools.

- `@WithMockUser` installs a `UsernamePasswordAuthenticationToken` with the given name, roles (prefixed with `ROLE_`) or authorities (used as given). Defaults: user `user`, role `USER`. It needs no user store.
- `@WithUserDetails` loads the principal through the `UserDetailsService`; the user must exist.
- `jwt()` (`SecurityMockMvcRequestPostProcessors`) builds a `JwtAuthenticationToken` for a request without signing anything; `jwt().authorities(...)` sets authorities, `jwt().jwt(j -> j.claim(...))` sets claims. The default token has `sub` of `user` and scope `read`.
- `csrf()` adds a valid CSRF token to a request; without it every state-changing request in a CSRF-protected chain returns 403.
- `springSecurity()` applies the filter chain to a `MockMvc` built manually; `@WebMvcTest` and `@AutoConfigureMockMvc` do it automatically.

A `@WebMvcTest` loads only the web layer and does not include application configuration classes unless they are imported: the security configuration must be imported with `@Import`, otherwise the test runs against Boot's default security and passes or fails for the wrong reason. In Spring Boot 3.4 and later `@MockitoBean` replaces `@MockBean` for the service dependencies.

OrderControllerSecurityTest.java JAVA

```
package com.example.shop.order;

import static org.mockito.BDDMockito.given;
import static org.springframework.security.test.web.servlet.request
        .SecurityMockMvcRequestPostProcessors.csrf;
import static org.springframework.security.test.web.servlet.request
        .SecurityMockMvcRequestPostProcessors.jwt;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import com.example.shop.config.SecurityConfig;
import java.math.BigDecimal;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
```

OrderControllerSecurityTest.java (continued) JAVA

```java
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(OrderController.class)
@Import(SecurityConfig.class)
class OrderControllerSecurityTest {

    @Autowired
    MockMvc mvc;

    @MockitoBean
    OrderService service;

    @Test
    void anonymousIsUnauthorized() throws Exception {
        mvc.perform(get("/api/orders/1")).andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(username = "alice", roles = "USER")
    void userMayReadOwnOrder() throws Exception {
        given(service.get(1)).willReturn(
                new OrderService.OrderView(1, "alice", BigDecimal.TEN, "NEW"));
        mvc.perform(get("/api/orders/1")).andExpect(status().isOk());
    }

    @Test
    @WithMockUser(username = "alice", roles = "USER")
    void userMayNotDelete() throws Exception {
        mvc.perform(org.springframework.test.web.servlet.request.MockMvcRequestBuilders
                .delete("/api/orders/1").with(csrf())).andExpect(status().isForbidden());
    }

    @Test
    void tokenWithScopeIsAccepted() throws Exception {
        mvc.perform(get("/api/orders/1")
                .with(jwt().authorities(new SimpleGrantedAuthority("SCOPE_orders.read"))))
                .andExpect(status().isOk());
    }

    @Test
    void tokenWithoutScopeIsForbidden() throws Exception {
        mvc.perform(post("/api/orders/1/cancel").with(jwt())).andExpect(status().isForbidden());
    }
}
```

Two details of this class matter. The anonymous test asserts 401, not merely "not 200", which documents the entry point behaviour; and the last test proves that an authenticated token with the wrong authority is denied, which is the situation the role-mapping problems of the previous chapters produce. Method security needs its own tests at the service level, with `@SpringBootTest` or a slice that includes the method security configuration, because a `@MockitoBean` replaces the proxy that carries the annotation:

OrderServiceSecurityTest.java (fragment) JAVA

```java
@SpringBootTest
class OrderServiceSecurityTest {

    @Autowired
    OrderService service;

    @Test
    @WithMockUser(roles = "USER")
    void userCannotListAllOrders() {
        assertThatThrownBy(() -> service.listAll()).isInstanceOf(AccessDeniedException.class);
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void adminCanListAllOrders() {
        assertThatCode(() -> service.listAll()).doesNotThrowAnyException();
    }
}
```

3.7.4 Real-world problems and solutions

Problem 20: a downstream service trusts an identity header

Scenario. The gateway validates the JWT and adds `X-User-Id: 42` and `X-Roles: ADMIN` to the downstream request. The services read these headers. A new route is added that does not pass through the gateway filter, or a client simply sends the headers itself.

Symptoms. Requests acting as other users or as administrators, with no token validation in the logs of the target service; the audit trail attributes actions to arbitrary identities.

Diagnosis. Call the service directly (from inside the cluster, or through the new route) with `curl -H 'X-User-Id: 1' -H 'X-Roles: ADMIN'` and no token; a 200 confirms the defect. Review whether the gateway removes incoming copies of these headers (`RemoveRequestHeader`) and whether network policies restrict who may reach the service.

Root cause. Authentication information in unauthenticated headers is data supplied by the caller. The service has no means to distinguish the gateway's header from a forged one.

Fix. Make every service a resource server that validates the token and reads identity from its claims; where the gateway must pass derived data, sign it (a short-lived internal JWT) or restrict reachability with network policies and mTLS so that only the gateway can call the service, and strip the headers at the edge as well.

Prevention. A test per service that a request with forged identity headers and no token receives 401; Kubernetes `NetworkPolicy` that denies all ingress except from the gateway.

Problem 21: service-to-service calls fail intermittently with 401 after token expiry

Scenario. The order service obtains a client credentials token at startup and stores it in a field. Every few minutes inventory calls return 401.

Symptoms. A burst of 401 responses every five minutes (the token lifetime), recovering after restarts; logs in the callee show `Jwt expired at ...`.

Diagnosis. Decode the token held by the caller and compare `exp` with the time of the first failure; check the token lifetime at the provider (Keycloak default for access tokens is five minutes).

Root cause. The token is a short-lived credential and was treated as a constant.

Fix. Obtain tokens through `OAuth2AuthorizedClientManager`, which caches and renews them before expiry (`InventoryClientConfig` in the previous chapter), and map a single 401 on a call to one retry after a forced refresh, not to repeated failures.

Prevention. Never cache tokens by hand; load-test across a token lifetime boundary; alert on the ratio of 401 to total responses between services.

Problem 22: tests pass with @WithMockUser, production returns 403

Scenario. Controller tests annotated `@WithMockUser(roles = "ADMIN")` pass. In production the same endpoint rejects administrators with 403, or throws `ClassCastException` on `(UserDetails) authentication.getPrincipal()`.

Symptoms. A green build and a failing release; the production stack trace names `Jwt cannot be cast to UserDetails`.

Diagnosis. Compare the authentication type the test installs (`UsernamePasswordAuthenticationToken` with a `User` principal) with the type production produces (`JwtAuthenticationToken` with a `Jwt` principal) and the authorities of each (`ROLE_ADMIN` from the annotation, versus `SCOPE_...` from the real converter).

Root cause. The test fabricated an authentication that the production chain never creates, so neither the authority mapping nor the principal type was tested.

Fix. Use the post-processor that matches the production authentication (`jwt()`), run the real `JwtAuthenticationConverter` where roles come from claims (`jwt().jwt(j -> j.claim("roles", List.of("ADMIN")))` combined with a test that applies the converter), and read the principal through `@AuthenticationPrincipal Jwt` or `Authentication.getName()` rather than casting.

Prevention. One end-to-end security test per chain with a real signed token from a test key, using the same decoder bean as production; Testcontainers with the real identity provider for the critical flows (see the testing part).

sreeram

3.7.5 Questions to expect

1. **How should service-to-service authentication be designed?** With a service identity (client credentials or mTLS) for calls made by the service itself, and relay or exchange of the user token when the user's identity matters; each service validates tokens and does not trust the network.
2. **Why validate JWTs in every service and not only at the gateway?** Anything that reaches a service without passing the gateway would otherwise be unauthenticated; per-service validation also provides the audience check and the ownership decision on `sub`.
3. **What is mutual TLS and what does it add?** Both client and server present certificates in the handshake, so the server authenticates the calling service and the channel is encrypted; it identifies services, not users, and certificates must be issued and rotated automatically.
4. **What is wrong with an `X-User-Id` header set by the gateway?** It is unauthenticated data; without guaranteed stripping at the edge and exclusive reachability through the gateway it can be forged. Use signed tokens.
5. **How are API keys stored safely?** Random with at least 256 bits of entropy, shown once, stored as a digest, looked up by digest, scoped, rotatable and carried in a header, never a URL.
6. **How is `@PreAuthorize` tested?** Through a Spring-managed bean in a context that includes method security, with `@WithMockUser` or `jwt()`, asserting `AccessDeniedException`; a mocked bean bypasses the proxy and tests nothing.
7. **What does the `jwt()` post-processor do?** It places a `JwtAuthenticationToken` in the security context of a `MockMvc` request without a real token or signature; the default has scope `read`, and authorities and claims can be overridden.
8. **Why does a state-changing `MockMvc` test fail with 403 although the user has the role?** The chain has CSRF protection enabled and the request lacks a token; `with(csrf())` adds one.

Check yourself

1. A service reads `X-Roles` set by the gateway. Name two conditions that make this safe, and say why neither is easy to guarantee.
2. What does `@WithMockUser` create when no attributes are given?
3. A test replaces `OrderService` with `@MockitoBean` and asserts that a `@PreAuthorize("hasRole('ADMIN')")` method denies a user. What happens?
4. Which identity does a client credentials token carry in `sub`?
5. Why is a key stored as SHA-256 acceptable but a password is not?

Answers

1. The gateway must strip client-supplied copies, and no other path may reach the service; both depend on configuration that drifts as routes and deployments change.
2. A user named `user` with password `password` and the authority `ROLE_USER`.
3. The mock is not the proxied bean, so the annotation is never evaluated and the test does not test the rule (it fails to throw, or passes meaninglessly).
4. The identity of the client (the service account), not a person.
5. The key has 256 bits of random entropy and cannot be guessed against a fast hash; a password has low entropy and needs a slow, salted function.

KEY POINTS

- Each internal call has a user identity and a service identity; carry them with token relay or exchange, client credentials, or mutual TLS, never with plain headers.
- Validate at the gateway for load protection and in every service for security; authorise on `sub` and scopes inside the service.
- API keys are random, hashed at rest, scoped, rotatable and sent in headers only.
- Test the production authentication type: `jwt()` for resource servers, `@WithMockUser` for form login, `csrf()` for state-changing requests, and assert 401 and 403 separately.
- Method security tests need the real proxied bean; a mocked bean tests nothing.

Sheeram

PART 4

Microservices and Resilience

From the decision to split a system, through communication, gateways and failure handling, to distributed data, tracing and API design.

In this part

- **4.1** Microservice Architecture
- **4.2** Synchronous Communication
- **4.3** Service Discovery, Configuration and Gateways
- **4.4** Resilience: Timeouts, Retries, Circuit Breakers and Bulkheads
- **4.5** Distributed Data and Transactions
- **4.6** Distributed Tracing and Correlated Logging
- **4.7** API Design for Services

4.1 Microservice Architecture

A single deployable application is the simplest unit to build, test, deploy and reason about. It stops being adequate when the number of developers working on it prevents independent release, when parts of the system need very different scaling or availability characteristics, or when one failure domain takes down unrelated functionality. A microservice architecture addresses these organisational and scaling constraints by decomposing a system into independently deployable services, each owning a narrow business capability and its own data.

The decomposition is not free. Every in-process method call that becomes a network call acquires latency, a failure mode, a serialisation cost and a versioning problem. Every transaction that crossed a table boundary becomes a distributed consistency problem. The remainder of this Part is largely a catalogue of the mechanisms that make those costs manageable: timeouts, retries, circuit breakers, sagas, outbox tables, tracing and contract design. This first chapter establishes the vocabulary and the criteria that decide whether the cost is justified at all.

4.1.1 Concept and mechanism

Three architectural styles

Three styles occupy the practical design space. A **monolith** is one deployable unit with one process and, usually, one database. A **modular monolith** is also one deployable unit, but its internal structure enforces module boundaries: each module owns its tables, exposes a small public API, and is forbidden from reaching into the internals of another. **Microservices** place each module in its own deployable unit with its own database, and the boundaries become network boundaries.

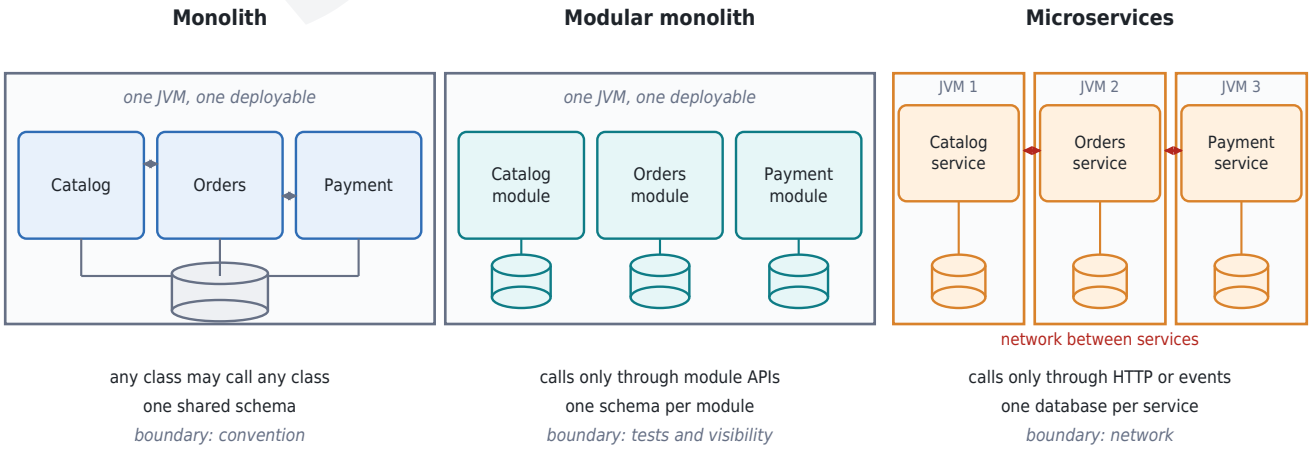


Figure 4.1. The three styles keep the same business modules; what changes is where the boundary is enforced (compiler and tests, or network) and how many deployable units and databases exist.

Property	Monolith	Modular monolith	Microservices
Deployable units	1	1	one per service
Boundary enforced by	convention	package visibility, architecture tests	network and separate repository or build
Data consistency	local ACID transactions	local ACID transactions	eventual consistency between services
Call cost between modules	method call (nanoseconds)	method call (nanoseconds)	network call (about a millisecond or more)
Independent release	no	no	yes
Operational burden	low	low	high: discovery, gateway, tracing, per-service pipelines
Typical fit	small team, early product	most teams of up to a few dozen developers	many teams, divergent scaling needs

Comparison of the three styles on the properties that matter operationally.

The modular monolith is frequently the correct intermediate step. Its boundaries are cheap to move because a refactoring is a code change rather than a data migration between databases. Spring Modulith (a separate project built on Spring Boot 3) provides verification of module dependencies and application events between modules; the same rules can be enforced with ArchUnit tests. A service extracted later from a well-formed module inherits an already minimal interface.

Bounded contexts

The unit of decomposition in a sound design is the **bounded context** from domain-driven design: a region of the domain within which each term has exactly one meaning and one model. The word "product" illustrates the point. In a catalogue context a product has a description, images and categories. In an inventory context it is a stock keeping unit with a warehouse quantity. In a billing context it is a tax category and a unit price at a point in time. Forcing one shared `Product` class across all three produces a model that every team must change for every other team's reason. Separate contexts hold separate models and exchange only the data that the other side needs, typically by identifier.

> **DEFINITION**
> Bounded context. A boundary within which a domain model and its vocabulary are consistent. Two contexts may use the same word for different concepts; translation between them occurs at an explicit interface (an anti-corruption layer, a published event schema or an API contract).

A **context map** records the relationships: which context is upstream (publishes a model), which is downstream (consumes it), and whether the consumer adapts to the producer (conformist), insulates itself with a translation layer (anti-corruption layer), or participates in a shared kernel. In service terms an anti-corruption layer is the mapper class that converts the remote DTO of another service into the local domain object, so that a change in the remote schema affects one class.

Database per service

Each service owns its persistent state and no other service reads or writes it directly. Access occurs only through the owning service's API or its published events. This rule is the one that distinguishes microservices from a monolith that merely happens to run in several processes. The consequences are structural:

- A join across two services is impossible in SQL. The caller composes the result in code (API composition), maintains a local read model fed by events (CQRS), or the data is duplicated deliberately.
- A transaction spanning two services cannot be an ACID transaction. Consistency is achieved through sagas and idempotent messages (see the chapter on distributed data).
- Each service may select a storage technology suited to its workload (relational for orders, a document store for the catalogue, a key-value store for sessions), at the cost of operating several technologies.
- A schema change is local to the owner, which removes the largest source of cross-team coordination.

Granularity, ownership and Conway's law

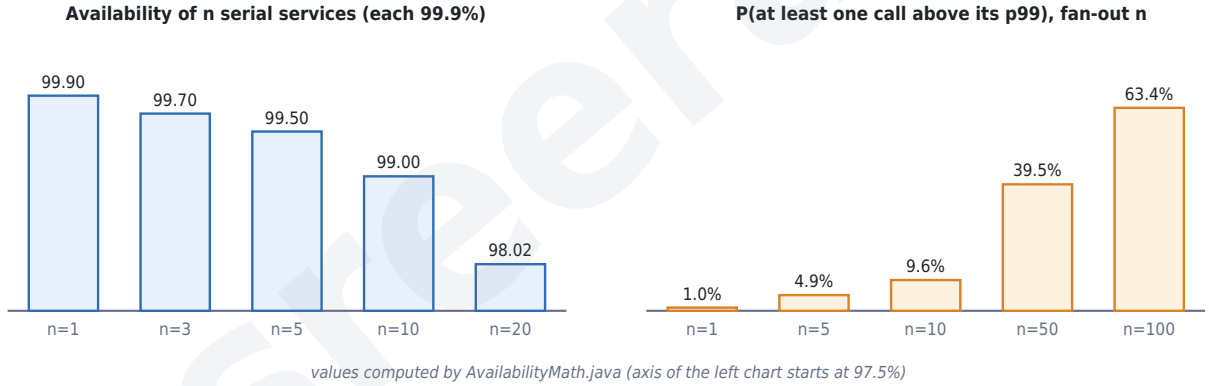
Conway's law states that the structure of a system mirrors the communication structure of the organisation that builds it. The practical corollary is the **inverse Conway manoeuvre**: the team structure is arranged first to match the desired architecture. A service owned by three teams has three sets of priorities and is released at the speed of the slowest negotiation; a service owned by one team that is also responsible for it in production (build it, run it) has a single decision maker.

Granularity has no universal measure. The following heuristics are useful in order of reliability:

1. **Data ownership.** A service should own a coherent set of entities and the invariants between them. If two candidate services must always be updated atomically, they are one service.
2. **Rate of change.** Code that changes together belongs together. If every feature requires coordinated changes in three services, the boundaries are wrong.
3. **Team ownership.** One team can reasonably own and operate a few services. A service that requires more than one team indicates a boundary that is too coarse.
4. **Scaling and failure profile.** A component with a distinct load profile (image resizing, search) or a distinct availability requirement (payment capture) is a candidate for isolation.
5. **Size is not a criterion.** Lines of code and the number of endpoints do not determine correct granularity. A service that can be rewritten by a small team in a few weeks is a useful sanity check, not a rule.

The costs, quantified

Two calculations explain most of the operational cost of fine-grained decomposition. First, availability multiplies. If a request requires five services in series and each is available 99.9 percent of the time, the request succeeds with probability 0.999 raised to the fifth power, about 99.5 percent. Second, tail latency amplifies with fan-out: if each call exceeds its own 99th percentile latency one time in a hundred, a request that makes 100 such calls sees at least one slow call 63 percent of the time. The program in the Hands-on section computes both.



values computed by AvailabilityMath.java (axis of the left chart starts at 97.5%)

Figure 4.2. Composite availability falls with the number of serial dependencies, and the probability of meeting at least one slow call grows quickly with fan-out; both curves come from the program in the Hands-on section.

The eight fallacies of distributed computing (the network is reliable, latency is zero, bandwidth is infinite, the network is secure, topology does not change, there is one administrator, transport cost is zero, the network is homogeneous) are each violated by the first remote call that replaces a method call. A service developer must handle each of them explicitly in code and configuration.

When not to use microservices

The following conditions favour a monolith or a modular monolith, and a decomposition attempted without them tends to produce a distributed monolith (services that must be deployed together and fail together):

- The domain is not yet understood. Boundaries drawn early are usually wrong, and moving a boundary between services costs far more than moving it between packages.
- The organisation has few teams. Independent deployment is valuable only when more than one team wants to deploy independently.
- There is no automated delivery pipeline, no centralised logging, no metrics and no distributed tracing. Without these, a failure spanning four services cannot be diagnosed.
- The workload needs strong consistency across the entities that would be split, for example a ledger.
- The team lacks the capacity to operate additional infrastructure (a registry or Kubernetes, a gateway, a broker, per-service databases).

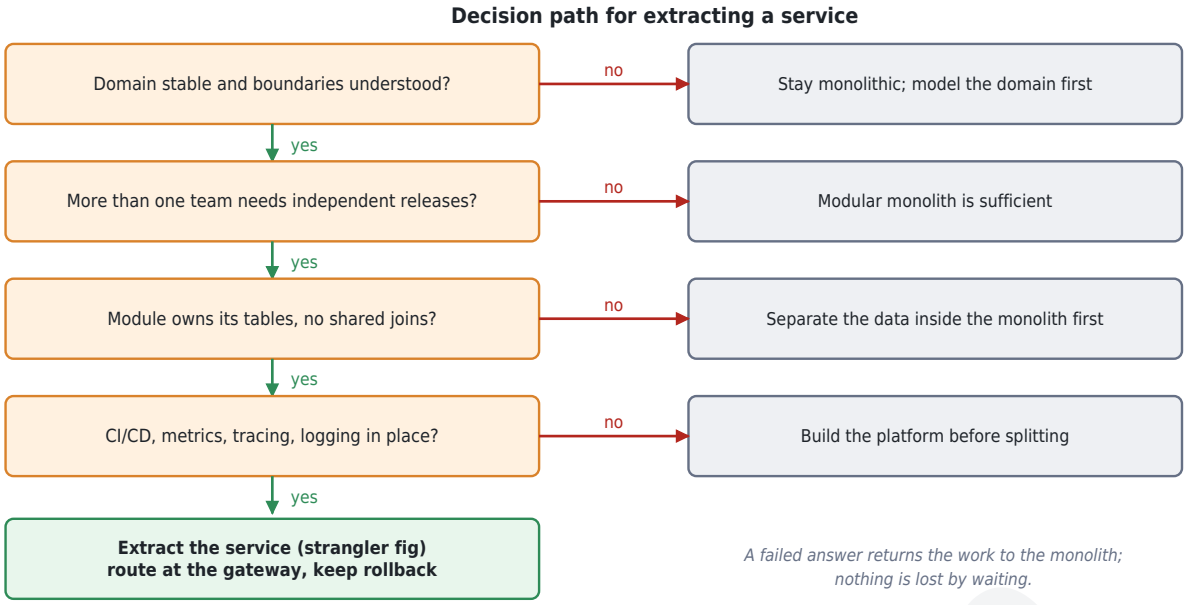


Figure 4.3. A decision path for extracting a service: every question that fails returns the module to the modular monolith rather than forcing a split.

Decomposition techniques

Two approaches dominate. **Decomposition by business capability** identifies what the business does (catalogue management, order fulfilment, payment) and creates one service per capability. **Decomposition by subdomain** uses the domain-driven design classification into core, supporting and generic subdomains; core subdomains receive the most investment and the most careful boundaries, generic ones (identity, notifications) are candidates for off-the-shelf products.

For an existing monolith the standard migration technique is the **strangler fig** pattern: a routing layer (the API gateway) is placed in front of the monolith, one capability at a time is reimplemented as a service, and the gateway route for that capability is switched from the monolith to the new service. The monolith shrinks until it can be retired or remains as a small core. Because the route can be switched back, each step is reversible.

4.1.2 Worked example

The worked example used through this Part is an e-commerce platform decomposed along business capabilities. Seven services and one gateway result:

Service	Responsibility	Owned data	Store
catalog-service	products, categories, prices	product, category	PostgreSQL
inventory-service	stock levels and reservations	stock_item, reservation	PostgreSQL
order-service	order lifecycle and checkout orchestration	orders, order_line, outbox	PostgreSQL
payment-service	payment capture and refunds	payment, idempotency_key	PostgreSQL
shipping-service	shipments and carrier integration	shipment	PostgreSQL
customer-service	customer profile and addresses	customer, address	PostgreSQL
notification-service	e-mail and SMS dispatch	template, delivery_log	PostgreSQL
api-gateway	routing, authentication, rate limiting	none	none

Services of the example platform, their responsibilities and their data stores.

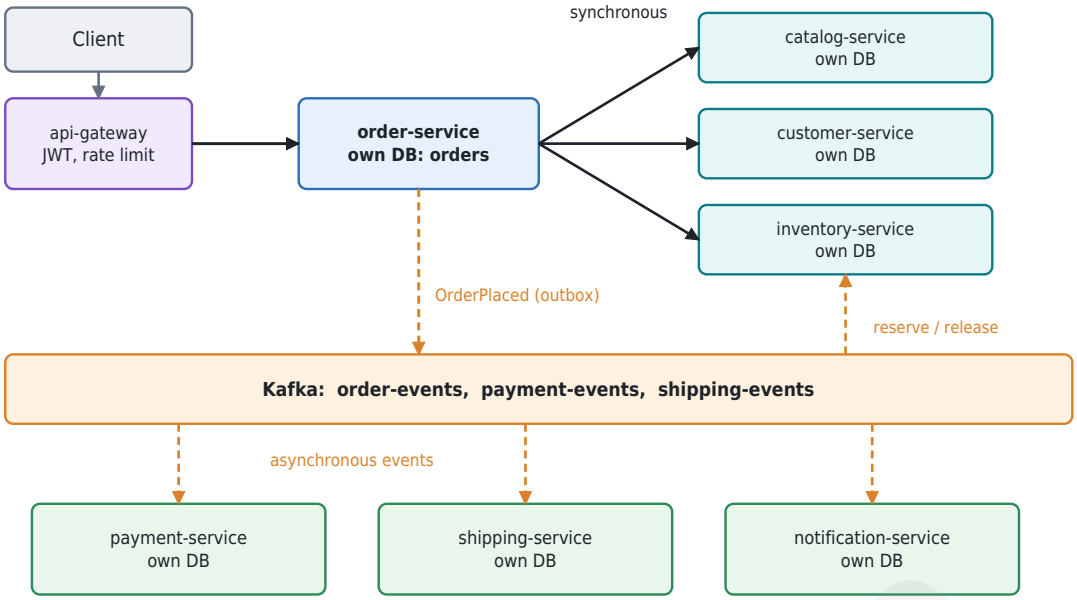


Figure 4.4. The example platform: synchronous calls (solid) are limited to data needed to answer a request, while state changes that other services react to travel as events (dashed) through the broker.

The design applies four rules that recur in later chapters. Services call each other synchronously only when the caller cannot answer its own request without the answer (the order service needs a price and a stock check before accepting an order). State changes that other services merely react to (order placed, payment captured, shipment dispatched) are published as events. The order service owns the checkout process and its compensations. The gateway is the only component reachable from outside the cluster.

Repository and build layout

A multi-module Maven layout with a parent that imports the Spring Boot and Spring Cloud bills of materials keeps dependency versions consistent without coupling the services at run time. Each service remains an independently built and released artifact; the parent only fixes versions.

Repository layout

```
shop-platform/
├── pom.xml                    parent: Boot parent, Spring Cloud BOM, shared plugin versions
├── api-gateway/               Spring Cloud Gateway application
├── catalog-service/
├── inventory-service/
├── order-service/
│   ├── pom.xml
│   └── src/main/java/com/shop/order/
│       ├── OrderServiceApplication.java
│       ├── api/               controllers, request and response DTOs
│       ├── domain/            entities, domain events, repository interfaces
│       ├── application/       use cases (services), transaction boundaries
│       └── infrastructure/    HTTP clients to other services, outbox, configuration
├── payment-service/
├── shipping-service/
├── customer-service/
└── notification-service/
```

The package layout inside a service follows the same discipline as the module layout in a modular monolith: `api` depends on `application`, `application` on `domain`, and only `infrastructure` knows about remote services. The class that calls the inventory service lives in `infrastructure`, and it returns a local type, which keeps the remote schema out of the domain.

shop-platform/pom.xml (excerpt) XML

```xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.6</version>
  <relativePath/>
</parent>
```

shop-platform/pom.xml (excerpt) (continued) XML

```xml
<properties>
  <java.version>21</java.version>
  <spring-cloud.version>2025.0.0</spring-cloud.version>
</properties>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Spring Cloud release trains are matched to Spring Boot lines: the 2024.0.x train targets Boot 3.4, the 2025.0.x train targets Boot 3.5, and 2025.1.x targets Boot 4.0. Using a train with a Boot version outside its range produces startup failures of the kind `NoSuchMethodError` or a failed compatibility check, so the pair is always taken from the Spring Cloud project page. At the time of writing the Boot 3.4 and 3.5 lines are past open-source support and Boot 4.0 with Spring Cloud 2025.1.x is current; the mechanisms in this Part are the same, and differences that affect code are noted where they occur.

One service: order-service

The order service demonstrates database-per-service in code. It owns an `Order` aggregate, has a private datasource, and refers to customers and products only by identifier.

Order.java JAVA

```java
package com.shop.order.domain;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

@Entity
@Table(name = "orders")
public class Order {

    public enum Status { PENDING, CONFIRMED, CANCELLED, SHIPPED }

    @Id
    private UUID id = UUID.randomUUID();

    // reference to another bounded context by identifier only, never by @ManyToOne
    @Column(nullable = false)
    private UUID customerId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Status status = Status.PENDING;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal total = BigDecimal.ZERO;

    @Column(nullable = false)
    private Instant createdAt = Instant.now();

    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "order_id")
    private List<OrderLine> lines = new ArrayList<>();

    @Version
    private long version;

    protected Order() { }
```

sreram

Order.java (continued) JAVA

```
    public Order(UUID customerId) {
        this.customerId = customerId;
    }

    public void addLine(String sku, int quantity, BigDecimal unitPrice) {
        lines.add(new OrderLine(sku, quantity, unitPrice));
        total = total.add(unitPrice.multiply(BigDecimal.valueOf(quantity)));
    }

    public void confirm() {
        if (status != Status.PENDING) {
            throw new IllegalStateException("Order " + id + " is " + status);
        }
        status = Status.CONFIRMED;
    }

    public void cancel() {
        if (status == Status.SHIPPED) {
            throw new IllegalStateException("Shipped order cannot be cancelled");
        }
        status = Status.CANCELLED;
    }

    public UUID getId() { return id; }
    public UUID getCustomerId() { return customerId; }
    public Status getStatus() { return status; }
    public BigDecimal getTotal() { return total; }
    public List<OrderLine> getLines() { return List.copyOf(lines); }
}
```

OrderLine.java JAVA

```
package com.shop.order.domain;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.util.UUID;

@Entity
@Table(name = "order_line")
public class OrderLine {

    @Id
    private UUID id = UUID.randomUUID();

    @Column(nullable = false, length = 40)
    private String sku;

    private int quantity;

    @Column(precision = 12, scale = 2)
    private BigDecimal unitPrice;

    protected OrderLine() { }

    OrderLine(String sku, int quantity, BigDecimal unitPrice) {
        this.sku = sku;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    public String getSku() { return sku; }
    public int getQuantity() { return quantity; }
    public BigDecimal getUnitPrice() { return unitPrice; }
}
```

The price is copied into the order line at purchase time. This is a deliberate duplication: the order must record the price that was valid when it was placed, and a later change in the catalogue must not rewrite history. It also removes the need to join against the catalogue to display an old order.

The application service coordinates the use case. Its dependencies on other services are expressed as interfaces owned by the order service (`PriceLookup`, `StockReservation`), implemented in `infrastructure` with the HTTP clients introduced in the next chapter.

sreeram

OrderService.java JAVA

```java
package com.shop.order.application;

import com.shop.order.domain.Order;
import com.shop.order.domain.OrderRepository;
import java.math.BigDecimal;
import java.util.List;
import java.util.UUID;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderService {

    public record Item(String sku, int quantity) { }

    private final OrderRepository orders;
    // ports implemented by HTTP clients to catalog-service and inventory-service
    private final PriceLookup prices;
    private final StockReservation stock;

    public OrderService(OrderRepository orders, PriceLookup prices, StockReservation stock) {
        this.orders = orders;
        this.prices = prices;
        this.stock = stock;
    }

    @Transactional
    public Order place(UUID customerId, List<Item> items) {
        Order order = new Order(customerId);
        for (Item item : items) {
            BigDecimal price = prices.priceOf(item.sku());
            order.addLine(item.sku(), item.quantity(), price);
        }
        orders.save(order);
        // remote call inside a local transaction: see the chapters on resilience and sagas
        stock.reserve(order.getId(), items);
        return order;
    }
}
```

The comment on the last line marks a defect that the later chapters resolve: a remote call is made while a database transaction and its connection are held open, and the local commit and the remote reservation are not atomic. The corrected design replaces the inline call with a saga step and an outbox event.

order-service/src/main/resources/application.yml YAML

```yaml
spring:
  application:
    name: order-service
  datasource:
    url: jdbc:postgresql://orders-db:5432/orders
    username: ${ORDERS_DB_USER}
    password: ${ORDERS_DB_PASSWORD}
    hikari:
      maximum-pool-size: 10
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
  flyway:
    enabled: true

server:
  port: 8080
  shutdown: graceful

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
```

sreram

order-service/src/main/resources/application.yml (continued) YAML

```yaml
  endpoint:
    health:
      probes:
        enabled: true
```

The datasource points at a database that only this service can reach. In a Kubernetes deployment that property is enforced by network policy and by credentials that exist only in this service's Secret; a convention without enforcement erodes within a few releases.

4.1.3 Hands-on

The arithmetic from the concept section is reproduced by a short program. The program is plain JDK and was run on the build machine.

AvailabilityMath.java JAVA

```java
public class AvailabilityMath {

    public static void main(String[] args) {
        System.out.println("Serial chain: every service must be up for a request to succeed");
        System.out.printf("%-10s %-14s %-14s %s%n", "services", "each 99.9%", "each 99.99%",
                "downtime/month at 99.9% each");
        for (int n : new int[] {1, 3, 5, 10, 20}) {
            double a = Math.pow(0.999, n), b = Math.pow(0.9999, n);
            System.out.printf("%-10d %-14.4f %-14.4f %.0f minutes%n", n, a * 100, b * 100,
                    (1 - a) * 30 * 24 * 60);
        }
        System.out.println();
        System.out.println(
                "Tail latency: fraction of user requests that see at least one slow (p99) call");
        System.out.printf("%-12s %s%n", "fan-out", "P(at least one call slower than its p99)");
        for (int n : new int[] {1, 5, 10, 50, 100}) {
            System.out.printf("%-12d %.1f%%%n", n, (1 - Math.pow(0.99, n)) * 100);
        }
    }
}
```

Run the availability and tail-latency calculation TERMINAL

```
$ java AvailabilityMath.java
Serial chain: every service must be up for a request to succeed
services   each 99.9%     each 99.99%    downtime/month at 99.9% each
1          99.9000        99.9900        43 minutes
3          99.7003        99.9700        129 minutes
5          99.5010        99.9500        216 minutes
10         99.0045        99.9000        430 minutes
20         98.0189        99.8002        856 minutes

Tail latency: fraction of user requests that see at least one slow (p99) call
fan-out      P(at least one call slower than its p99)
1            1.0%
5            4.9%
10           9.6%
50           39.5%
100          63.4%
```

Reading the output: twenty serial dependencies of 99.9 percent each give about 98 percent, which corresponds to more than 14 hours of unavailability per month. The mitigation is not better individual services alone; it is fewer synchronous hops on the critical path, fallbacks that allow a request to succeed without a dependency, and asynchronous handling of everything that need not complete before the response.

The module boundaries of a modular monolith can be verified by a test. The sketch below assumes the ArchUnit library and the package layout shown above (typical usage; not run on the build machine).

ArchitectureTest.java JAVA

```java
package com.shop.order;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.importer.ImportOption;
import com.tngtech.archunit.junit.AnalyzeClasses;
```

ArchitectureTest.java (continued) JAVA

```java
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.lang.ArchRule;

@AnalyzeClasses(packages = "com.shop.order", importOptions = ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {

    @ArchTest
    static final ArchRule domainDoesNotDependOnInfrastructure =
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAPackage("..infrastructure..");

    @ArchTest
    static final ArchRule controllersDoNotUseRepositoriesDirectly =
        noClasses().that().resideInAPackage("..api..")
            .should().dependOnClassesThat().resideInAPackage("..domain..")
            .because("controllers must go through application services");
}
```

4.1.4 Real-world problems and solutions

Problem 1: the distributed monolith

Scenario. An organisation split a monolith into twelve services over a year. Releases are scheduled monthly because the services "must go out together".

Symptoms. A change to one field of the customer representation requires pull requests in five repositories merged in a fixed order. A staging environment with one service at an older version fails integration tests. An outage in the inventory service makes the catalogue pages return errors although they display no stock information.

Diagnosis. Three measurements locate the coupling. First, the deployment history: services whose release timestamps always fall within the same hour are coupled at deploy time. Second, the dependency graph taken from tracing data (a service map) or from the gateway access logs: a synchronous call from catalogue to inventory on a read path shows the failure propagation path. Third, schema ownership: `grep` for the other services' table names in each repository and inspect the database grants.

Root cause. Boundaries were drawn by technical layer or by entity rather than by business capability, services share a database or a shared DTO library, and calls are synchronous where the result is not needed to answer the request.

Fix. Replace the shared DTO library by per-consumer DTOs (consumer-driven contracts, see the chapter on API design). Replace synchronous calls on read paths by local read models fed by events. Merge services that always change together. Introduce an expand-and-contract policy for every interface change, so that old and new versions are accepted simultaneously.

Prevention. Track the lead time and the number of services touched per feature as architectural metrics. Require an independent deploy of each service in the pipeline (deploy the new version of a single service against the current versions of the others).

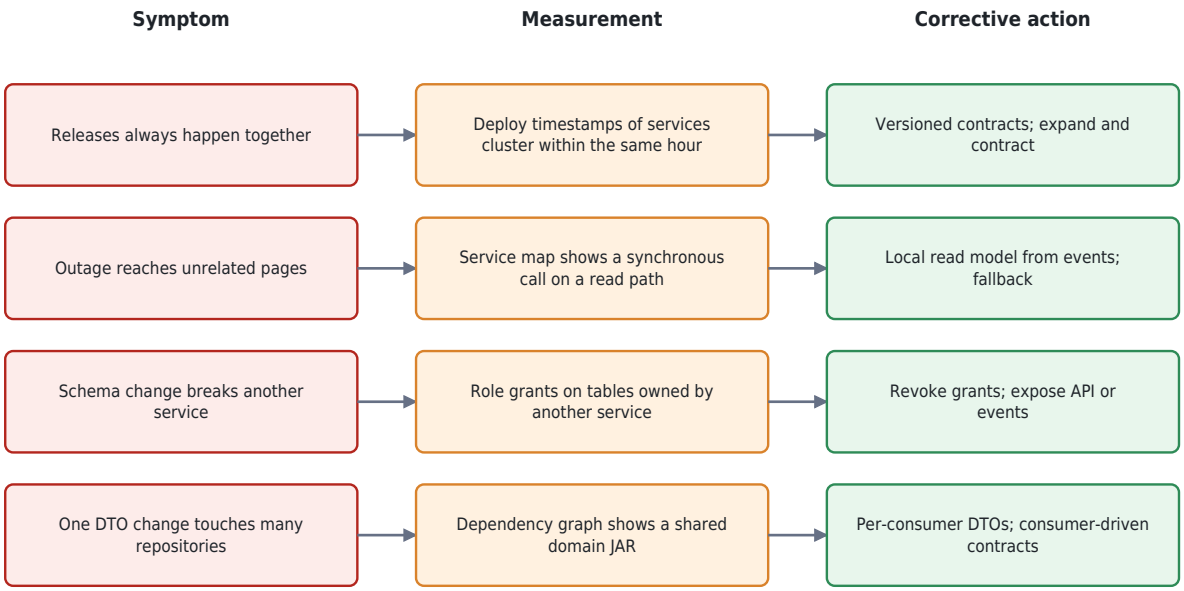


Figure 4.5. The diagnosis path for a suspected distributed monolith: each symptom maps to a measurable coupling and to the corrective action named at the end of the branch.

Problem 2: schema change breaks another service

Scenario. The payment service reads the `orders` table of the order service directly "for reporting". A migration renames `total` to `total_amount`.

Symptoms. Payment captures begin to fail with `ERROR: column "total" does not exist` immediately after the order service is deployed; the order service itself is healthy and its tests pass.

Diagnosis. The PostgreSQL log names the failing statement and the role. Mapping the role to a service shows that the payment service holds a grant on the order service's schema. `SELECT grantee, privilege_type FROM information_schema.role_table_grants WHERE table_name = 'orders'` lists every consumer.

Root cause. The database is an integration point. Its schema became a public interface without versioning, review or tests.

Fix. Revoke the foreign grants. Expose the needed data through an API endpoint or an event, and have the payment service store the fields it needs (the order total at capture time) in its own table. For the transition, publish a database view with a stable name and version it like an API while consumers migrate.

Prevention. Provision one database role per service with privileges on its own schema only, and run the check query above in the pipeline.

Problem 3: a monolith decomposed too early

Scenario. A three-person team building a first product starts with eight services.

Symptoms. Most of the sprint is spent on build pipelines, version pinning and local environment start-up (eight processes, three databases and a broker); a feature that touches ordering and inventory needs two coordinated deployments; local integration testing needs the whole stack.

Diagnosis. Count the share of work items that touch more than one service, and compare the time spent on infrastructure tasks with feature tasks over a few sprints.

Root cause. The architecture was selected for an organisation size and scale that do not exist yet.

Fix. Merge the services into a modular monolith along the same boundaries (the package structure already exists), keep the module APIs as the former service interfaces, and extract a module only when a measured need appears (independent scaling, a separate team, a different availability target).

Prevention. Apply the decision path in the diagram: extraction requires a stated, measurable reason.

Sreeram

4.1.5 Questions to expect

1. **What is the difference between a monolith, a modular monolith and microservices?** A monolith is one deployable unit with no enforced internal boundaries. A modular monolith is one deployable unit whose modules own their data and expose narrow APIs, enforced by tests or build rules. Microservices are separately deployable units with their own databases; the boundaries are network boundaries and consistency between them is eventual.
2. **What is a bounded context and why does it matter for service boundaries?** It is a boundary within which a domain model is consistent and each term has one meaning. Service boundaries aligned with bounded contexts minimise coupling because the same real-world entity is modelled separately in each context and shared only by identifier.
3. **Why database per service?** It makes the service the single owner of its schema, so schema changes do not require cross-team coordination, and it prevents hidden coupling through shared tables. The cost is the loss of cross-service joins and ACID transactions.
4. **What is a distributed monolith?** A system split into services that must still be deployed together and that fail together, typically because of a shared database, shared libraries with domain types, or chains of synchronous calls. It has the costs of both styles and the benefits of neither.
5. **How is the size of a service decided?** By data ownership and invariants, rate of change, team ownership and scaling profile, not by lines of code. Two entities that must be updated atomically belong in one service.
6. **When should microservices be avoided?** When the domain is unclear, the team is small, delivery automation and observability are missing, or strong cross-entity consistency is required.
7. **State Conway's law and a practical consequence.** System structure mirrors the communication structure of the organisation. The practical consequence is to organise teams around the desired service boundaries (inverse Conway manoeuvre) and give each service a single owning team.
8. **How is a monolith migrated to microservices safely?** With the strangler fig pattern: route traffic through a gateway, reimplement one capability at a time as a service, switch its route, and keep the old path available for rollback until the new service is verified.
9. **Why does availability decrease with more services?** Serial dependencies multiply availability: five services of 99.9 percent give about 99.5 percent. Mitigations are fewer synchronous hops, fallbacks, caching and asynchronous processing.

Check yourself

1. Four serial dependencies each have an availability of 99.5 percent. What is the availability of the request path?
2. The order entity declares `@ManyToOne Customer customer` where `Customer` is a class from the customer service's JAR. Which rule is broken and what replaces the association?
3. Two services always need to be released together. Name two possible corrective actions.
4. A request fans out to 20 downstream calls, each exceeding its p99 one time in a hundred. Estimate the probability of at least one slow call.

Answers

1. 0.995 raised to the fourth power is about 0.9801, so 98.0 percent.
2. Database per service and bounded-context isolation: the association couples the order model to another context. A `UUID customerId` field replaces it, with data fetched through the API or a local read model.
3. Merge the services (they belong to one boundary), or remove the coupling (versioned contract, expand-and-contract migration, event instead of synchronous call).
4. One minus 0.99 raised to the 20th power: about 18 percent.

KEY POINTS

Microservices trade local simplicity for independent deployment; the trade is justified only by team scale, divergent scaling needs or isolation requirements.

- Boundaries follow bounded contexts and business capabilities, not technical layers or entity tables.
- Each service owns its data; cross-service joins and ACID transactions are replaced by composition, events and sagas.
- Availability multiplies and tail latency amplifies across synchronous hops, so the critical path is kept short.
- A modular monolith with enforced module boundaries is the default starting point; extraction needs a measurable reason.
- A distributed monolith (shared database, lockstep releases) is the characteristic failure of premature or poorly bounded decomposition.

sreeram

4.2 Synchronous Communication

A synchronous call blocks the caller, and the thread that issued it, until the callee answers or a timeout expires. It is the simplest interaction model and the one that most frequently causes production incidents, because every property of a local method call that a developer takes for granted is absent: the call can be slow, can fail after the work was performed, can be delivered twice, and can leave the caller waiting on a callee that has stopped responding. This chapter describes how an HTTP call is decomposed into separately bounded phases, how a timeout budget is derived across a chain of services, how retries are made safe with idempotency keys, and how the number of calls per request is controlled.

4.2.1 Concept and mechanism

Protocol choices

REST over HTTP with JSON is the default for service-to-service calls that are also consumed by browsers and third parties. **gRPC** uses HTTP/2 with Protocol Buffers; the contract is a `.proto` file from which client stubs and server skeletons are generated, messages are binary and compact, calls are multiplexed over a long-lived connection, streaming is first class, and a call carries a deadline that is propagated automatically to downstream calls. Messaging (the Part on Kafka) is the alternative when the caller does not need an immediate answer.

Criterion	REST and JSON	gRPC
Contract	OpenAPI document, often written after the code	`.proto` file, written first; code generated
Payload	text JSON, human readable	binary Protocol Buffers, smaller and faster to parse
Transport	HTTP/1.1 or HTTP/2	HTTP/2 only
Streaming	server-sent events or WebSocket as add-ons	unary, server, client and bidirectional streaming
Deadline propagation	manual (headers, conventions)	built in
Browser and third-party use	direct	needs gRPC-Web or a gateway translation
Debuggability	curl and browser	needs grpcurl or tooling
Fit	public and cross-team APIs, moderate throughput	internal high-volume calls with strict contracts

Selection criteria for synchronous protocols between services.

The remainder of the chapter uses HTTP and JSON, which the Spring Boot stack supports directly. The timeout, retry and idempotency principles apply unchanged to gRPC.

Client options in Spring Boot 3

Four client abstractions exist in a Boot 3 application, and the choice affects how timeouts and pools are configured.

- `RestClient` (Spring Framework 6.1, Boot 3.2 and later) is the synchronous fluent client and the recommended replacement for `RestTemplate` in new code. Boot provides a prototype `RestClient.Builder` bean pre-configured with message converters and an appropriate `ClientHttpRequestFactory`.
- `RestTemplate` is in maintenance mode. It remains supported and widely deployed; its behaviour with respect to timeouts and pools is identical because both delegate to a `ClientHttpRequestFactory`.
- `WebClient` is the reactive client. Using it from blocking code (calling `block()`) loses its benefits and is justified only when the rest of the stack is reactive.
- **HTTP interface clients** (`@HttpExchange` interfaces created with `HttpServiceProxyFactory`) describe a remote API as a Java interface; the proxy delegates to a `RestClient`, `WebClient` or `RestTemplate` through an adapter. Spring Cloud OpenFeign is documented as feature-complete and its documentation recommends migrating to HTTP interface clients. Spring Framework 7 (Boot 4) adds declarative registration of such clients; the Boot 3 idiom shown below creates the proxy explicitly.

Boot detects the HTTP library for `RestClient` and `RestTemplate` in this order: Apache HttpClient, Jetty HttpClient, Reactor Netty HttpClient, the JDK `java.net.http.HttpClient`, and finally `java.net.HttpURLConnection`. Since Boot 3.4 global defaults can be set with `spring.http.client.connect-timeout`, `spring.http.client.read-timeout`, `spring.http.client.redirects` and `spring.http.client.factory`; the reactive equivalents use the prefix `spring.http.reactiveclient`. A pool size or a per-phase timeout that these properties do not cover requires constructing the factory explicitly, as in the worked example.

The phases of one HTTP call

A call is a sequence of phases, and each phase has its own failure mode and its own timeout setting. Confusing them is the origin of most incidents attributed to "the network".

1. **Connection acquisition** from the client pool. If all connections to the route are in use, the caller waits in a queue. The bound is the pool acquisition timeout (Apache: `connectionRequestTimeout`).
2. **Connect**: TCP handshake, and the TLS handshake if used. The bound is the connect timeout. A failure here (refused, unreachable) means the request was never sent, so a retry is always safe.
3. **Send and wait for the first byte**: the callee processes the request. The bound is the read (socket) or response timeout. A failure here is ambiguous: the callee may have completed the work, partially completed it, or not started it.
4. **Receive the body.** A callee that sends headers and stalls is bounded by the same socket read timeout, applied per read, not to the whole response.

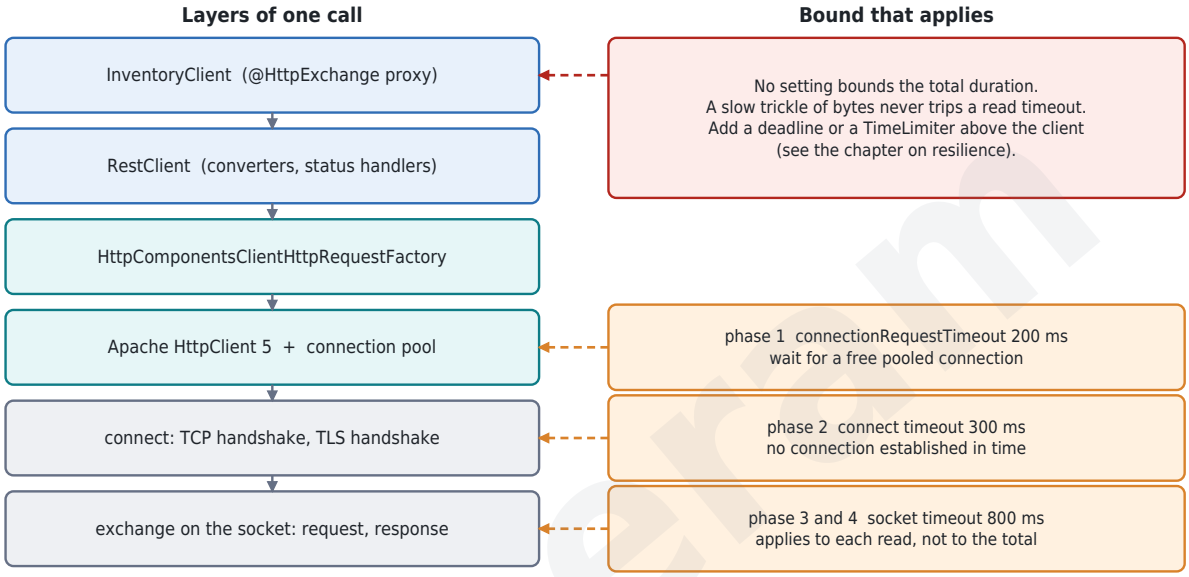


Figure 4.6. The layers between a call on an HTTP interface and the wire, and the timeout that bounds each phase; a total deadline for the whole exchange must be added at the application level because none of the four settings bounds the sum.

No standard client timeout bounds the total duration of the exchange: a slow callee that sends one byte every second never triggers a read timeout of two seconds. An overall deadline is obtained with a time limiter (Resilience4j `TimeLimiter`, discussed in the resilience chapter), an HTTP/2 stream deadline, or a server-side budget.

Timeout budget across a chain

A request that crosses several services shares one budget. Each upstream timeout must exceed the total time its callee may spend, including retries, or the upstream gives up while the downstream continues to work on a result nobody will read. The relation for a caller that makes the downstream call with up to `n` attempts, a per-attempt timeout `t` and a delay `b` between attempts is:

Budget rule

```
caller timeout  >  own work  +  n * t  +  (n - 1) * b  +  margin

example: gateway 3000 ms
  order-service   own work 100 ms, 2 attempts * 800 ms, 1 * 100 ms backoff  =  1800 ms
  margin          1200 ms          (3000 - 1800)
```

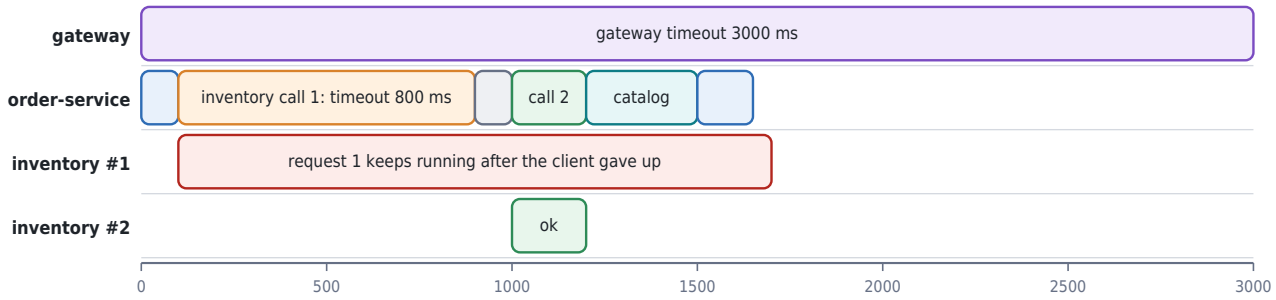


Figure 4.7. A request that meets one slow inventory call: the client of the order service gives up at 800 ms and retries, while the first inventory call continues to consume a worker; the total stays inside the 3000 ms gateway budget.

The diagram also shows the cost that retries add on the callee: the abandoned first call is still executing. The callee should therefore be built to tolerate duplicated and abandoned work, which leads to idempotency.

Idempotency

An operation is **idempotent** if executing it several times has the same effect as executing it once. In HTTP, `GET`, `PUT`, `DELETE` and `HEAD` are defined as idempotent, and `POST` is not. A client may retry an idempotent request after an ambiguous failure without further protection. A `POST` that creates a payment or reserves stock must be made idempotent explicitly, and the standard technique is a client-generated **idempotency key**:

1. The client generates a unique key (a UUID) once per logical operation and sends it in a header (the convention `Idempotency-Key`), reusing the same value on every retry.
2. The server records the key together with a hash of the request and, when finished, the response, in a table with a unique constraint on the key.
3. A repeated request with a known key returns the stored response without repeating the work. A repeated key with a different request hash is a client error (HTTP 422).

The unique constraint is what makes the technique correct under concurrency: two simultaneous requests with the same key cannot both insert the row.

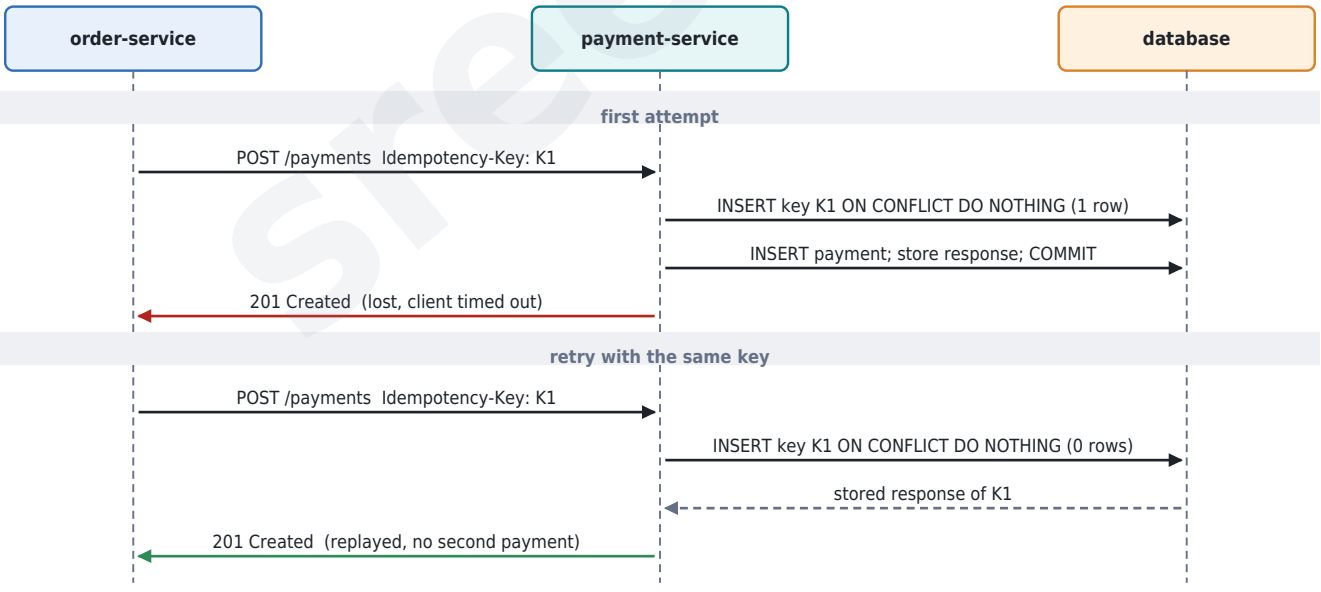


Figure 4.8. The first request commits but its response is lost; the retry with the same key finds the stored response and returns it, so the payment is captured exactly once.

Calls per request: the N+1 pattern across services

The N+1 selects problem has a distributed form. A list endpoint of the order service returns 50 orders and, for each, fetches the customer name from the customer service: one request for the list plus 50 requests for the customers. Every call adds serialisation, a network round trip and a slot in the callee's thread pool, and the latencies add up when the calls are sequential. Three remedies exist, in decreasing order of preference:

1. **Batch endpoint.** The callee offers `GET /customers?ids=...` (or a `POST` with a body when the list is long) and the caller makes one request for the distinct identifiers.
2. **Local read model.** The caller stores the few customer attributes it displays, updated from customer events. The call disappears from the read path entirely, at the price of eventual consistency.
3. **Bounded parallelism.** Independent calls are issued concurrently with virtual threads or a bounded executor. This reduces latency but not load, and unbounded parallelism turns the caller into a load generator against its dependency.

4.2.2 Worked example

The order service needs prices from the catalog service and stock reservations from the inventory service. Both dependencies are described as HTTP interfaces; the inventory client carries an idempotency key.

CatalogClient.java JAVA

```java
package com.shop.order.infrastructure;

import java.math.BigDecimal;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;

@HttpExchange(url = "/api/v1/products", accept = "application/json")
public interface CatalogClient {

    record PriceResponse(String sku, BigDecimal amount, String currency) { }

    @GetExchange("/{sku}/price")
    PriceResponse price(@PathVariable("sku") String sku);
}
```

InventoryClient.java JAVA

```java
package com.shop.order.infrastructure;

import java.util.List;
import java.util.UUID;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.service.annotation.DeleteExchange;
import org.springframework.web.service.annotation.HttpExchange;
import org.springframework.web.service.annotation.PostExchange;

@HttpExchange("/api/v1/reservations")
public interface InventoryClient {

    record Line(String sku, int quantity) { }

    record ReservationRequest(UUID orderId, List<Line> lines) { }

    record ReservationResponse(UUID reservationId, String state) { }

    @PostExchange
    ReservationResponse reserve(@RequestHeader("Idempotency-Key") String idempotencyKey,
                                @RequestBody ReservationRequest request);

    @DeleteExchange("/{orderId}")
    void release(@PathVariable("orderId") UUID orderId);
}
```

The factory that creates the proxies configures each phase explicitly. Apache HttpClient 5 is used because it exposes the pool size and the acquisition timeout; adding `org.apache.httpcomponents.client5:httpclient5` to the classpath is sufficient for Boot to prefer it, but the explicit configuration documents the limits.

InventoryClientConfig.java JAVA

```java
package com.shop.order.infrastructure;

import org.apache.hc.client5.http.config.ConnectionConfig;
import org.apache.hc.client5.http.config.RequestConfig;
import org.apache.hc.client5.http.impl.classic.CloseableHttpClient;
import org.apache.hc.client5.http.impl.classic.HttpClients;
```

InventoryClientConfig.java (continued) JAVA

```java
import org.apache.hc.client5.http.impl.io.PoolingHttpClientConnectionManager;
import org.apache.hc.client5.http.impl.io.PoolingHttpClientConnectionManagerBuilder;
import org.apache.hc.core5.util.TimeValue;
import org.apache.hc.core5.util.Timeout;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.HttpComponentsClientHttpRequestFactory;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.support.RestClientAdapter;
import org.springframework.web.service.invoker.HttpServiceProxyFactory;

@Configuration(proxyBeanMethods = false)
class InventoryClientConfig {

    @Bean
    RestClient inventoryRestClient(RestClient.Builder builder,
                                   @Value("${clients.inventory.base-url}") String baseUrl) {
        PoolingHttpClientConnectionManager pool = PoolingHttpClientConnectionManagerBuilder.create()
            .setMaxConnTotal(50)
            .setMaxConnPerRoute(25)
            .setDefaultConnectionConfig(ConnectionConfig.custom()
                .setConnectTimeout(Timeout.ofMillis(300))      // phase 2
                .setSocketTimeout(Timeout.ofMillis(800))       // phase 3 and 4, per read
                .build())
            .build();
        CloseableHttpClient http = HttpClients.custom()
            .setConnectionManager(pool)
            .setDefaultRequestConfig(RequestConfig.custom()
                .setConnectionRequestTimeout(Timeout.ofMillis(200))   // phase 1
                .setResponseTimeout(Timeout.ofMillis(800))
                .build())
            .evictExpiredConnections()
            .evictIdleConnections(TimeValue.ofSeconds(30))
            .build();
        return builder.baseUrl(baseUrl)
            .requestFactory(new HttpComponentsClientHttpRequestFactory(http))
            .defaultStatusHandler(status -> status.is5xxServerError(), (request, response) -> {
                int code = response.getStatusCode().value();
                throw new DependencyUnavailableException("inventory", code);
            })
            .build();
    }

    @Bean
    InventoryClient inventoryClient(RestClient inventoryRestClient) {
        return HttpServiceProxyFactory
            .builderFor(RestClientAdapter.create(inventoryRestClient))
            .build()
            .createClient(InventoryClient.class);
    }
}
```

DependencyUnavailableException.java JAVA

```java
package com.shop.order.infrastructure;

public class DependencyUnavailableException extends RuntimeException {

    private final String dependency;
    private final int status;

    public DependencyUnavailableException(String dependency, int status) {
        super(dependency + " responded with HTTP " + status);
        this.dependency = dependency;
        this.status = status;
    }

    public String getDependency() { return dependency; }
    public int getStatus() { return status; }
}
```

sreeram

The mapping of every 5xx answer to one exception type matters later: the retry and circuit-breaker configuration lists exactly the exception types that count as failures, and a 4xx answer (a client error) must not be counted, because retrying it cannot help and it does not indicate a sick dependency.

The callee: an idempotent payment endpoint

The payment service implements the server side of the idempotency key. In the simplest correct design the claim of the key and the business work run in one database transaction. A concurrent duplicate then blocks inside the database on the unique index until the first transaction finishes, and afterwards finds the committed row.

IdempotencyRecord.java JAVA

```java
package com.shop.payment.idempotency;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.time.Instant;
import java.util.Map;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;

@Entity
@Table(name = "idempotency_key")
public class IdempotencyRecord {

    @Id
    @Column(name = "key", length = 100)
    private String key;

    @Column(name = "request_hash", nullable = false, length = 64)
    private String requestHash;

    @Column(name = "response_code")
    private Integer responseCode;

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(name = "response_body")
    private Map<String, Object> responseBody;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt = Instant.now();

    protected IdempotencyRecord() { }

    public String getRequestHash() { return requestHash; }
    public Integer getResponseCode() { return responseCode; }
    public Map<String, Object> getResponseBody() { return responseBody; }

    public void complete(int code, Map<String, Object> body) {
        this.responseCode = code;
        this.responseBody = body;
    }
}
```

IdempotencyRepository.java JAVA

```java
package com.shop.payment.idempotency;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface IdempotencyRepository extends JpaRepository<IdempotencyRecord, String> {

    /** Returns 1 if this call claimed the key, 0 if the key already existed. */
    @Modifying
    @Query(value = """
        INSERT INTO idempotency_key (key, request_hash, created_at)
        VALUES (:key, :hash, now())
```

IdempotencyRepository.java (continued) JAVA

```
        ON CONFLICT (key) DO NOTHING
        """, nativeQuery = true)
    int claim(@Param("key") String key, @Param("hash") String hash);
}
```

PaymentService.java JAVA

```
package com.shop.payment;

import com.shop.payment.idempotency.IdempotencyRecord;
import com.shop.payment.idempotency.IdempotencyRepository;
import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.HexFormat;
import java.util.Map;
import java.util.UUID;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;

@Service
public class PaymentService {

    public record CaptureRequest(UUID orderId, BigDecimal amount, String currency) { }

    private final IdempotencyRepository keys;
    private final PaymentRepository payments;

    public PaymentService(IdempotencyRepository keys, PaymentRepository payments) {
        this.keys = keys;
        this.payments = payments;
    }

    @Transactional
    public ResponseEntity<Map<String, Object>> capture(String key, CaptureRequest request) {
        String hash = sha256(request.orderId() + "|" + request.amount() + "|" + request.currency());
        if (keys.claim(key, hash) == 0) {
            IdempotencyRecord stored = keys.findById(key).orElseThrow();
            if (!stored.getRequestHash().equals(hash)) {
                throw new ResponseStatusException(HttpStatus.UNPROCESSABLE_ENTITY,
                    "Idempotency-Key reused with a different request");
            }
            return ResponseEntity.status(stored.getResponseCode()).body(stored.getResponseBody());
        }
        Payment payment = payments.save(
            new Payment(request.orderId(), request.amount(), "CAPTURED"));
        Map<String, Object> body =
            Map.of("paymentId", payment.getId(), "state", payment.getState());
        keys.findById(key).orElseThrow().complete(HttpStatus.CREATED.value(), body);
        return ResponseEntity.status(HttpStatus.CREATED).body(body);
    }

    private static String sha256(String value) {
        try {
            byte[] digest = MessageDigest.getInstance("SHA-256")
                .digest(value.getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(digest);
        } catch (java.security.NoSuchAlgorithmException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

The `claim` statement is a native query because JPA has no portable equivalent of `ON CONFLICT DO NOTHING`; the alternative of saving an entity and catching `DataIntegrityViolationException` aborts the transaction in PostgreSQL, which makes the subsequent read fail with "current transaction is aborted". The row ends as part of the same commit as the payment, so a rollback of the business work also releases the key and the client may retry.

NOTE
The second design, claiming the key in its own short transaction with an `IN_PROGRESS` marker and completing it later, allows a concurrent duplicate to receive an immediate `409 Conflict` instead of waiting. It costs more code (a recovery job for keys stuck in `IN_PROGRESS` after a crash) and is justified when the business work is long. The SQL session below demonstrates this second design.

4.2.3 Hands-on

The distributed N+1 effect is measurable with plain JDK classes. The program starts a small "customer service" that spends 5 ms per request, then fetches 50 customers three ways: sequentially, in parallel with virtual threads, and with one batch request.

DistributedNPlusOne.java JAVA

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class DistributedNPlusOne {

    static final AtomicInteger requests = new AtomicInteger();

    public static void main(String[] args) throws Exception {
        // avoid Nagle/delayed-ACK stalls
        System.setProperty("sun.net.httpserver.nodelay", "true");
        // "customer-service": 5 ms of work per request, regardless of batch size
        HttpServer srv = HttpServer.create(new InetSocketAddress("127.0.0.1", 21420), 0);
        srv.setExecutor(Executors.newFixedThreadPool(16));
        srv.createContext("/customers", ex -> {
            requests.incrementAndGet();
            try { Thread.sleep(5); } catch (InterruptedException e) { }
            String q = ex.getRequestURI().getQuery();
            String body = q == null ? "{\"id\":" + ex.getRequestURI().getPath().replaceAll("\\D",
                    "") + "}"
                    : "[" + Arrays.stream(q.substring(4).split(",")).map(i -> "{\"id\":" + i + "}")
                            .collect(Collectors.joining(",")) + "]";
            byte[] b = body.getBytes();
            ex.sendResponseHeaders(200, b.length);
            ex.getResponseBody().write(b); ex.close();
        });
        srv.start();

        HttpClient http = HttpClient.newBuilder().version(HttpClient.Version.HTTP_1_1).build();
        // 50 orders, 50 customers
        List<Integer> customerIds = IntStream.rangeClosed(1, 50).boxed().toList();
        http.send(HttpRequest.newBuilder(URI.create("http://127.0.0.1:21420/customers/1")).build(),
                HttpResponse.BodyHandlers.ofString());                                    // warm-up
        requests.set(0);

        long t = System.nanoTime();
        for (int id : customerIds)
            http.send(HttpRequest.newBuilder(URI.create("http://127.0.0.1:21420/customers/"
                    + id)).build(),
                    HttpResponse.BodyHandlers.ofString());
        report("1. one call per order, sequential", t);

        requests.set(0); t = System.nanoTime();
        try (var vt = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<String>> fs = new ArrayList<>();
            for (int id : customerIds) fs.add(vt.submit(() -> http.send(
                    HttpRequest.newBuilder(URI.create("http://127.0.0.1:21420/customers/"
                            + id)).build(),
                    HttpResponse.BodyHandlers.ofString()).body()));
            for (var f : fs) f.get();
        }
        report("2. one call per order, parallel (virtual threads)", t);
```

DistributedNPlusOne.java (continued) JAVA

```
        requests.set(0); t = System.nanoTime();
        String ids = customerIds.stream().map(String::valueOf).collect(Collectors.joining(","));
        http.send(HttpRequest.newBuilder(URI.create("http://127.0.0.1:21420/customers?ids="
                + ids)).build(),
                HttpResponse.BodyHandlers.ofString());
        report("3. one batch call /customers?ids=1,2,...,50", t);
        srv.stop(0);
        System.exit(0);
    }

    static void report(String what, long t0) {
        System.out.printf("%-52s requests=%2d  elapsed=%4d ms%n", what, requests.get(),
                (System.nanoTime() - t0) / 1_000_000);
    }
}
```

Run the N+1 comparison (timings are typical, measured on the build machine) TERMINAL

```
$ java DistributedNPlusOne.java
1. one call per order, sequential                    requests=50  elapsed= 472 ms
2. one call per order, parallel (virtual threads)    requests=50  elapsed=  79 ms
3. one batch call /customers?ids=1,2,...,50          requests= 1  elapsed=  13 ms
```

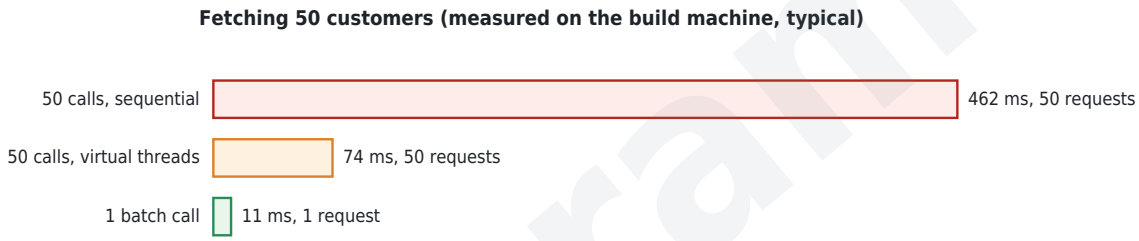


Figure 4.9. The same 50 customers fetched with 50 sequential calls, 50 parallel calls and one batch call; the batch removes both the latency and the load of 49 requests.

The first version of this program, without the `sun.net.httpserver.nodelay` property, measured about 52 ms per request instead of about 9 ms. The server wrote the response headers and the body in two small segments, and the interaction of the Nagle algorithm with delayed acknowledgements on the client held the second segment for roughly 40 ms. The effect is a reminder that a measurement of a network call includes transport behaviour, and that a latency number of this kind has to be explained before it is accepted.

The idempotency table of the second design is exercised below in PostgreSQL 16, which was run for real. The `INSERT 0 1` tag means one row was inserted (the key was claimed) and `INSERT 0 0` means the conflict clause suppressed the insert.

Idempotency key states in PostgreSQL TERMINAL

```
$ psql -d shop -f idem_http.sql
CREATE TABLE
-- first request with key K1: claim the key
INSERT 0 1
-- retry arrives while the first is still running: claim fails (0 rows), caller must answer 409
INSERT 0 0
-- first request finishes: store the response
UPDATE 1
-- later retry: claim fails again, the stored response is replayed
INSERT 0 0
  status   | response_code |              response_body
-----------+---------------+----------------------------------------
 COMPLETED |           201 | {"state": "CAPTURED", "paymentId": 981}
(1 row)
```

4.2.4 Real-world problems and solutions

Problem 1: the order list endpoint takes seconds (distributed N+1)

Scenario. `GET /api/v1/orders?size=50` of the order service enriches every order with the customer name by calling the customer service once per order.

Symptoms. The endpoint responds in 2 to 3 seconds in production and 80 ms in the developer environment. The customer service shows 50 times more requests than the order service on its dashboard, and its thread pool is busy although its CPU is idle.

Diagnosis. (1) The metric `http.client.requests` of the order service, filtered by the `uri` tag of the customer call, shows a request count 50 times the count of `http.server.requests` for the list endpoint. (2) A trace of one request shows 50 sequential child spans of identical name and duration below the controller span (see the chapter on tracing). (3) With debug logging of the client (`logging.level.org.springframework.web.client=DEBUG`) the log contains 50 lines `HTTP GET .../customers/{id}`.

Root cause. A loop over the result of the first query contains a remote call; locally the callee answers in about 1 ms, so the cost is invisible in development.

Fix. Add `GET /api/v1/customers?ids=` to the customer service, collect the distinct customer identifiers of the page, call once, and join in memory:

OrderQueryService.java (fragment) — JAVA

```java
public Page<OrderSummary> list(Pageable pageable) {
    Page<Order> page = orders.findAll(pageable);
    Set<UUID> ids = page.stream().map(Order::getCustomerId).collect(Collectors.toSet());
    Map<UUID, String> names = customers.byIds(ids).stream()
        .collect(Collectors.toMap(CustomerSummary::id, CustomerSummary::name));
    return page.map(o -> new OrderSummary(o.getId(), o.getStatus(),
        names.getOrDefault(o.getCustomerId(), "unknown")));
}
```

Prevention. Assert the number of outbound calls in an integration test with a stub server (for WireMock, `verify(1, getRequestedFor(urlPathEqualTo("/customers")))`), and set a maximum page size in the API.

Problem 2: customer charged twice after a timeout

Scenario. The order service calls `POST /payments`; the payment service captures the payment but is slow, the client times out and a retry interceptor repeats the request.

Symptoms. Customers report two charges for one order. The payment table contains two rows with the same `order_id` a few seconds apart.

Diagnosis. `SELECT order_id, count(*) FROM payment GROUP BY order_id HAVING count(*) > 1;` lists affected orders. The access log of the payment service contains two `POST /api/v1/payments` lines for the same order, the first with a duration above the client read timeout, which shows that the server completed work after the client had abandoned it.

Root cause. The retry policy treated `POST` like an idempotent method and the endpoint had no deduplication.

Fix. Introduce the `Idempotency-Key` header as in the worked example, and configure the retry to repeat only requests that carry a key or that use idempotent methods.

Prevention. A contract test per mutating endpoint that sends the same request twice with the same key and asserts a single side effect.

Problem 3: all endpoints hang when one dependency slows down (missing client timeout)

Scenario. The order service uses a `RestTemplate` created with `new RestTemplate()` for the inventory service. The inventory service begins to respond after 60 seconds or not at all.

Symptoms. After a few minutes every endpoint of the order service returns slowly or not at all, including `/actuator/health`, although the CPU is idle and the heap is small. Kubernetes liveness probes fail and the pods restart repeatedly.

Diagnosis. A thread dump shows the request threads waiting in the HTTP client. Taken with `jcmd <pid> Thread.print`, the dump contains many entries of this shape (typical output; frames differ by client library):

Thread dump excerpt (typical output, not run on the build machine)

```
"http-nio-8080-exec-37" #92 daemon prio=5
   java.lang.Thread.State: RUNNABLE
        at sun.nio.ch.NioSocketImpl.park(NioSocketImpl.java:186)
        at sun.nio.ch.NioSocketImpl.implRead(NioSocketImpl.java:312)
        at java.net.Socket$SocketInputStream.read(Socket.java:1099)
        at org.springframework.web.client.RestTemplate.execute(RestTemplate.java)
        at com.shop.order.infrastructure.InventoryGateway.reserve(InventoryGateway.java)
```

Counting such threads (`jcmd <pid> Thread.print | grep -c "InventoryGateway.reserve"`) and comparing with the metric `tomcat.threads.busy` against `tomcat.threads.config.max` (the latter available when `server.tomcat.mbeanregistry.enabled=true`) confirms exhaustion of the 200 default worker threads.

Root cause. No read timeout was configured, so a stalled callee holds a worker thread indefinitely, and a few minutes of request arrivals consume the whole pool. The liveness probe is served by the same pool and fails as a consequence.

Fix. Set connect and read timeouts on every client (`spring.http.client.connect-timeout` and `spring.http.client.read-timeout` as defaults, or the explicit factory shown above), add a bulkhead so that one dependency cannot take all threads (see the resilience chapter), and serve liveness from a separate management port.

Prevention. A startup check or an architecture test that fails when a `RestClient` or `RestTemplate` is created without a configured factory, and a dashboard panel with busy threads against maximum threads.

4.2.5 Questions to expect

1. **RestTemplate, RestClient or WebClient?** `RestClient` for new blocking code (Boot 3.2 and later); `RestTemplate` exists in large code bases and is in maintenance mode; `WebClient` when the application is reactive or needs streaming. All delegate to a pluggable HTTP library.
2. **Which timeouts does an HTTP client have?** Connection acquisition from the pool, connect, read (socket or response), and optionally a total deadline implemented outside the HTTP library. Each bounds a different phase and all should be set.
3. **Why is a retry unsafe after a read timeout?** The request may have been processed; the failure is ambiguous. Retrying a non-idempotent operation can duplicate its effect.
4. **How is a `POST` made safe to retry?** With an idempotency key generated by the client for each logical operation and stored by the server under a unique constraint together with the response.
5. **How are timeouts chosen across a chain?** Each upstream timeout must exceed the downstream's total time including retries; per-call timeouts are derived from the downstream p99 plus a margin, and the sum along the critical path stays below the user-facing timeout.
6. **What is the N+1 problem across services and how is it avoided?** One call per element of a list. Avoided with batch endpoints, local read models and bounded parallelism; detected by comparing client request counts with server request counts and by trace waterfalls.
7. **What does an HTTP interface client provide?** A Java interface with `@HttpExchange` annotations whose proxy performs the calls through a `RestClient`, `WebClient` or `RestTemplate`; it separates the description of a remote API from the transport configuration.
8. **When is gRPC preferable to REST?** For internal, high-volume calls with strict contracts and streaming needs, where generated stubs, binary payloads, multiplexing and built-in deadlines pay off; REST remains better for public and browser-facing APIs.
9. **Why map 5xx responses to a dedicated exception?** So that resilience configuration can state exactly which outcomes count as dependency failures; 4xx responses indicate a bad request and must not trip breakers or be retried.

Check yourself

1. A client has `connectTimeout=300 ms` and `readTimeout=800 ms`. A server sends one byte every 500 ms for a total of 40 seconds. Does the call time out?
2. Which HTTP methods are idempotent by definition, and why is that insufficient to make a `POST /payments` retry safe?
3. A gateway has a 3 s timeout. The order service calls inventory with 3 attempts of 1.5 s each. What is wrong?
4. What does `INSERT ... ON CONFLICT (key) DO NOTHING` report when the key already exists, and how does the application use it?

Answers

1. No. The read timeout applies to each read separately and every read completes within 500 ms, so the call runs for 40 seconds unless an overall deadline exists.
2. GET, PUT, DELETE and HEAD. `POST` is not, so a retry could capture a second payment; the idempotency key supplies the missing guarantee.
3. The downstream may need up to 4.5 s plus delays, which exceeds the 3 s upstream budget: the gateway gives up while the order service continues, wasting work and producing duplicates.
4. It reports zero affected rows (`INSERT 0 0`); the application treats that as "already claimed" and replays the stored response or rejects a mismatching request.

KEY POINTS

Synchronous calls need explicit bounds, explicit repetition rules and an explicit budget.

- A call has four phases (acquire, connect, read, body); each needs its own timeout and none bounds the total, so a deadline must be added.
- Timeouts shrink along the chain: every upstream timeout exceeds the downstream total including retries.
- Retry only idempotent operations; make `POST` safe with an idempotency key stored under a unique constraint.
- Remote calls inside loops are the distributed N+1; use batch endpoints or local read models and verify the call count in tests.
- `RestClient` with an `@HttpExchange` interface is the Boot 3 idiom; map 5xx to a dedicated exception so resilience rules can name it.

4.3 Service Discovery, Configuration and Gateways

Service instances are short-lived: a deployment replaces every instance, an autoscaler adds and removes instances, and a node failure relocates them. The address of an instance therefore cannot be written into the configuration of its callers. Three mechanisms remove that dependence. **Service discovery** maps a stable logical name to the current set of healthy instances. **Load balancing** selects one instance per request or per connection. **Externalised configuration** delivers environment-specific settings to every instance without rebuilding the artifact. A fourth component, the **API gateway**, provides the single entry point at which routing, authentication, rate limiting and edge resilience are applied once instead of in every service.

4.3.1 Concept and mechanism

Discovery models

Two models exist. In **client-side discovery** each client obtains the list of instances from a registry (Eureka, Consul) and chooses among them itself; the registry is an additional component that every service registers with and that every caller queries. In **server-side discovery** the client addresses a stable name or virtual address and the platform picks the instance: a cloud load balancer, or on Kubernetes the `Service` object with its cluster DNS name (for example `inventory-service.shop.svc.cluster.local`, which resolves to a virtual IP that `kube-proxy` maps to the ready pods).

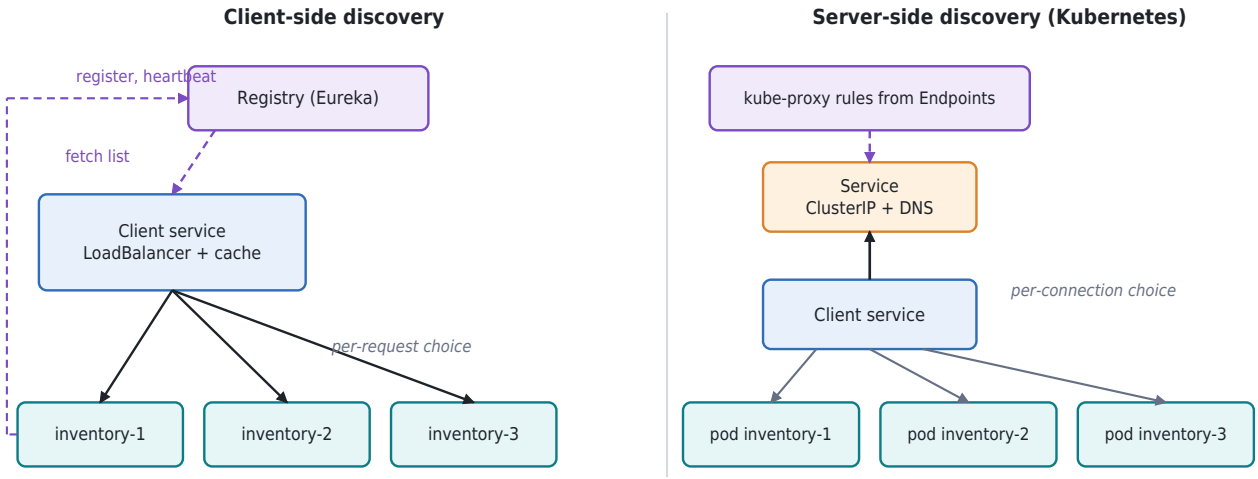


Figure 4.10. Client-side discovery moves the instance list and the balancing decision into every client process; server-side discovery keeps the client simple and places the decision in the platform.

Property	Client-side (Spring Cloud)	Kubernetes Service and DNS
Registry	separate component to run and secure	built into the cluster, always present
Registration	each instance registers and sends heartbeats	automatic from pod readiness
Balancing granularity	per request (client library)	per connection (`kube-proxy`)
Staleness window	minutes with default timers	seconds (endpoint update and readiness)
Language support	Spring clients only unless sidecars are used	any language, any HTTP client
Typical use	VMs, mixed environments, non-Kubernetes platforms	Kubernetes deployments (the common case)

Client-side registry (Eureka or Consul) compared with Kubernetes service discovery.

On Kubernetes the registry adds no value and complicates operations: readiness already decides which pods receive traffic. The client-side model remains relevant on virtual machines and for features that need per-request control (zone preference, weighted routing, retry on a different instance). Spring Cloud Kubernetes provides a `DiscoveryClient` implementation backed by the Kubernetes API for the cases where a Spring client needs the instance list, but the default recommendation is to address the Service name directly.

How a registry works: Eureka as the reference

An Eureka client performs three periodic activities. It **registers** with the server at start-up. It sends a **heartbeat** (lease renewal) every 30 seconds by default. It **fetches the registry** delta and keeps a local cache that is refreshed on a timer; the lookups of the application use only this cache. The server removes an instance whose lease has not been renewed within the expiration duration (90 seconds by default, evicted by a periodic task). If too many leases lapse at once the server assumes a network partition and enters **self-preservation mode**, in which it stops evicting, preferring stale entries to a registry that empties during a partial outage.

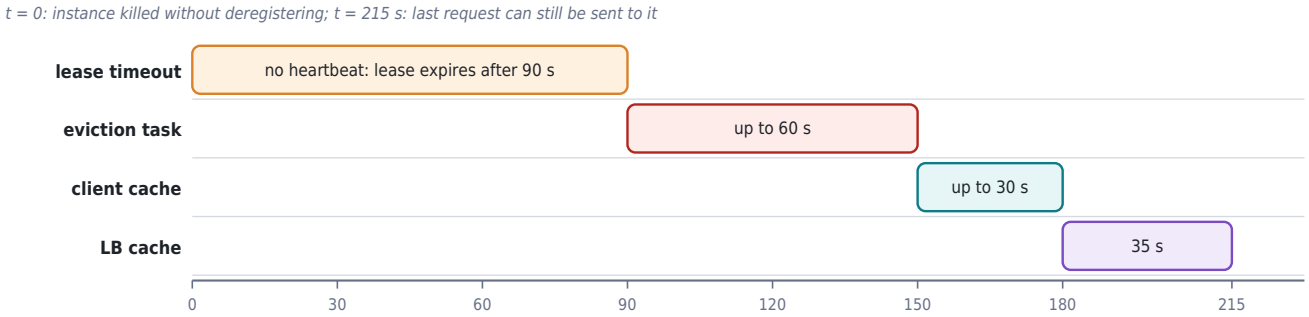


Figure 4.11. The worst-case time between the death of an instance and the last request routed to it is the sum of independent timers (lease expiry, eviction, registry cache, load-balancer cache); with default values it is measured in minutes.

With the defaults mentioned (values to be confirmed for the version in use, since they are configurable and are not stated in the Spring Cloud reference), an instance killed without deregistering can still be selected for several minutes.

The standard mitigations are graceful shutdown (the instance deregisters, then drains), retries that fall through to another instance on connection failure, and shortening the timers consciously, because the server's computations assume the heartbeat period.

Client-side load balancing with Spring Cloud LoadBalancer

Spring Cloud LoadBalancer is the client-side balancer of Spring Cloud. A client that uses a logical name (`http://inventory-service/...` or `lb://inventory-service` in a gateway route) is intercepted: `BlockingLoadBalancerClient` serves `RestClient` and `RestTemplate` beans marked `@LoadBalanced`, and a `ReactorLoadBalancerExchangeFilterFunction` serves `WebClient`. The interceptor asks a per-service `ReactorLoadBalancer` for an instance; the default implementation is `RoundRobinLoadBalancer`, with `RandomLoadBalancer` as the alternative. The candidate list comes from a `ServiceInstanceListSupplier`, which is composed from a delegate chain (discovery client, caching, health checks, zone preference, weighting, same-instance preference). Each logical service gets its own child application context, created lazily on first use unless `spring.cloud.loadbalancer.eager-load.clients` lists it.

Instance lists are cached; the documented defaults are a time to live of 35 seconds (`spring.cloud.loadbalancer.cache.ttl`) and a capacity of 256 entries. Load-balanced retry is controlled by `spring.cloud.loadbalancer.retry.enabled`. When a discovery client with its own cache (Eureka) is used, the second cache adds to the staleness window shown above.

The plain round-robin strategy ignores load: a slow instance receives the same share of requests as a fast one. Alternatives are weighted selection, a supplier that filters on health, and, at the platform level, least-connections balancing in the ingress or service mesh.

Kubernetes balances connections, not requests

`kube-proxy` makes a routing decision when a TCP connection is established. An HTTP client that keeps a connection alive and reuses it for thousands of requests sends all of them to the pod chosen at connect time. When a deployment scales from 2 to 6 pods, the existing connections stay on the original two pods and the new pods receive almost nothing; gRPC, which multiplexes all calls over one HTTP/2 connection, shows the effect most strongly. Remedies: a headless Service (`clusterIP: None`) with client-side balancing across the returned addresses, a maximum connection age on the client or server so that connections are recycled, an L7 proxy or service mesh that balances per request, or an ingress controller configured with a least-requests algorithm.

The API gateway

Spring Cloud Gateway is built on Spring WebFlux and Reactor Netty. Its building blocks are:

- **Route**: an identifier, a destination URI, a list of predicates and a list of filters.
- **Predicate**: a condition on the request (`Path`, `Method`, `Header`, `Host`, `Query`, `Cookie`, `After`, `Weight`). A route is selected when all its predicates match.
- **Filter**: logic applied before the request is forwarded (pre) and after the response returns (post): `StripPrefix`, `RewritePath`, `AddRequestHeader`, `Retry`, `CircuitBreaker`, `RequestRateLimiter`.
- **Global filter**: applied to every route (for example `ReactiveLoadBalancerClientFilter`, which resolves `lb://` URIs, and `NettyRoutingFilter`, which performs the forwarding).

Processing follows the WebFlux handler pipeline: `RoutePredicateHandlerMapping` evaluates the routes and stores the match in the exchange; `FilteringWebHandler` builds a `GatewayFilterChain` from the global filters and the route filters, sorted by `Ordered`; each filter may act before calling `chain.filter(exchange)` (pre phase) and in a continuation after it (post phase). The filter with the highest precedence (lowest order value) runs first on the way in and last on the way out.

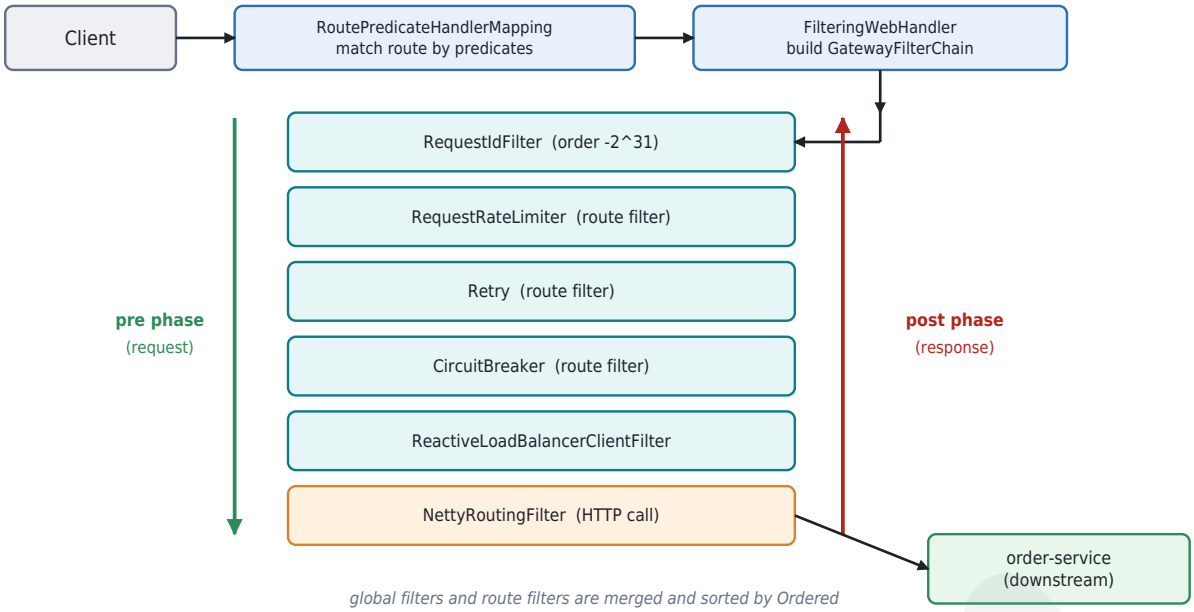


Figure 4.12. The path of one request through the gateway: route matching, the ordered filter chain with its pre and post phases, and the Netty client that performs the downstream call.

Version note: with the Spring Cloud 2025.0 release train (Gateway 4.3 and later, used with Boot 3.5) the artifact is named `spring-cloud-starter-gateway-server-webflux` and the properties live under `spring.cloud.gateway.server.webflux`; the earlier artifact `spring-cloud-starter-gateway` and the prefix `spring.cloud.gateway` apply to the 2024.0 train (Boot 3.4) and are deprecated afterwards. The examples below use the new names. A servlet-based variant (`spring-cloud-starter-gateway-server-webmvc`) exists for teams that prefer to stay on the blocking stack.

Rate limiting at the edge

`RequestRateLimiter` implements the **token bucket** algorithm: a bucket holds up to `burstCapacity` tokens, is refilled at `replenishRate` tokens per second, and each request removes `requestedTokens` (default 1); a request that finds the bucket empty receives HTTP 429. The state is kept in Redis (hence the dependency on `spring-boot-starter-data-redis-reactive`), which makes the limit global across all gateway instances. A `KeyResolver` bean decides what is limited: the authenticated user, an API key, or the client address. The mini gateway in the Hands-on section implements the same algorithm in memory.

Externalised configuration

Three mechanisms dominate in Spring applications.

1. **Environment variables and Kubernetes `ConfigMap` and `Secret` objects.** Values are injected as environment variables (read once at pod start) or mounted as files. Boot can import a mounted directory with `spring.config.import=optional:configtree:/etc/config/`, where each file name becomes a property key and its content the value; this is the usual way to consume mounted Secrets.
2. **Spring Cloud Config Server**, a service that serves property files from a Git repository (or another backend) per application name and profile. Clients declare `spring.config.import=optional:configserver:http://config-server:8888`. A change in Git becomes visible after the client refreshes (`POST /actuator/refresh` re-binds `@ConfigurationProperties` and rebuilds `@RefreshScope` beans).
3. **Cloud-provider or Vault stores**, imported through further `spring.config.import` locations.

Precedence, from highest to lowest, follows the order documented for Boot: command-line arguments, system properties, environment variables, imported and profile-specific files, then defaults packaged in the JAR. A value that is set in two places is a latent source of drift.

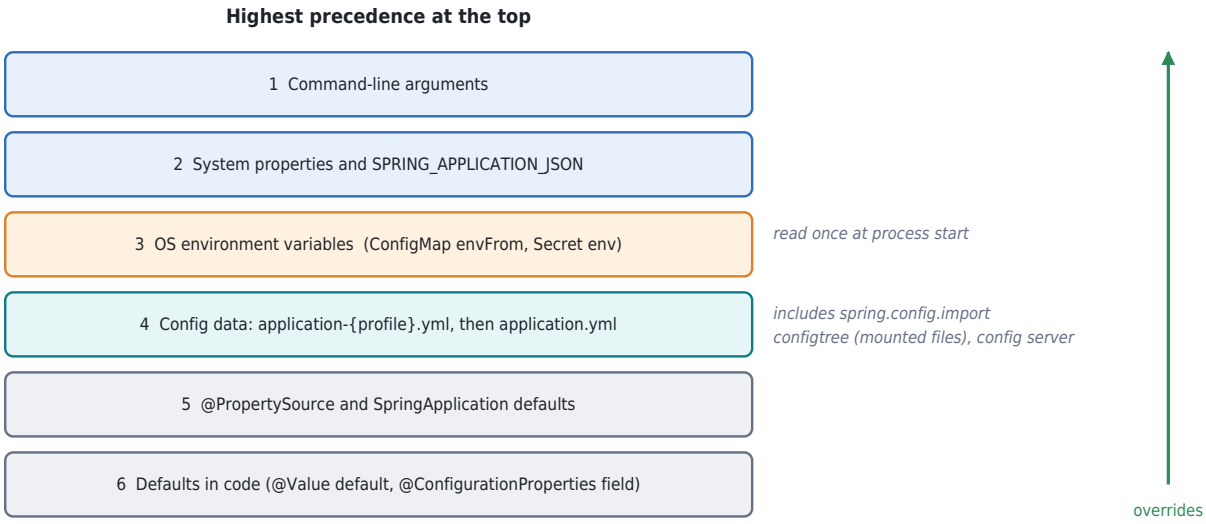


Figure 4.13. Configuration sources and the order in which they override each other for one service instance; the ConfigMap volume and the environment variable differ in when a change becomes visible.

4.3.2 Worked example

The gateway of the example platform terminates edge concerns and routes to the services by Kubernetes Service name.

api-gateway/pom.xml (dependencies) XML

```xml
<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway-server-webflux</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-reactor-resilience4j</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

api-gateway/src/main/resources/application.yml YAML

```yaml
spring:
  application:
    name: api-gateway
  data:
    redis:
      host: ${REDIS_HOST:localhost}
  cloud:
    gateway:
      server:
        webflux:
          httpclient:
            connect-timeout: 500          # milliseconds
            response-timeout: 3s          # java.time.Duration
          routes:
            - id: orders
              uri: http://order-service:8080
              predicates:
                - Path=/api/orders/**
              filters:
                - RewritePath=/api/orders(?<segment>/?.*), /api/v1/orders${segment}
                - name: RequestRateLimiter
                  args:
```

api-gateway/src/main/resources/application.yml (continued) YAML

```yaml
                redis-rate-limiter.replenishRate: 20
                redis-rate-limiter.burstCapacity: 40
                redis-rate-limiter.requestedTokens: 1
                key-resolver: "#{@clientKeyResolver}"
            - name: Retry
              args:
                retries: 2
                methods: GET
                statuses: BAD_GATEWAY, SERVICE_UNAVAILABLE
                backoff:
                  firstBackoff: 50ms
                  maxBackoff: 500ms
                  factor: 2
                  basedOnPreviousValue: false
            - name: CircuitBreaker
              args:
                name: ordersBreaker
                fallbackUri: forward:/fallback/orders
        - id: reports
          uri: http://report-service:8080
          predicates:
            - Path=/api/reports/**
          metadata:
            response-timeout: 20000     # per-route override in milliseconds
            connect-timeout: 500

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,gateway
```

The `Retry` filter is limited to `GET` and to two gateway-class statuses; a `POST` is never retried by the gateway because the gateway cannot know whether the operation is idempotent. The `gateway` actuator endpoint (`/actuator/gateway/routes`) lists the effective routes and is the first diagnostic tool when a route does not match. On the 2024.0 release train the same file is written with `spring.cloud.gateway.routes` and `spring.cloud.gateway.httpclient`, without the `server.webflux` levels.

GatewayConfig.java JAVA

```java
package com.shop.gateway;

import java.util.Optional;
import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import reactor.core.publisher.Mono;

@Configuration(proxyBeanMethods = false)
class GatewayConfig {

    /** Rate-limit key: the client id header when present, otherwise the remote address. */
    @Bean
    KeyResolver clientKeyResolver() {
        return exchange -> {
            String clientId = exchange.getRequest().getHeaders().getFirst("X-Client-Id");
            if (clientId != null && !clientId.isBlank()) {
                return Mono.just(clientId);
            }
            return Mono.just(Optional.ofNullable(exchange.getRequest().getRemoteAddress())
                .map(address -> address.getAddress().getHostAddress())
                .orElse("unknown"));
        };
    }
}
```

Behind a cloud load balancer the remote address is that of the balancer; an `X-Forwarded-For` aware resolver (`XForwardedRemoteAddressResolver`) or an authenticated identity is then the correct key, and the header `X-Client-Id` is only trustworthy if the gateway itself sets it after authentication.

RequestIdFilter.java JAVA

```java
package com.shop.gateway;

import java.util.UUID;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.core.Ordered;
import org.springframework.http.server.reactive.ServerHttpRequest;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

@Component
public class RequestIdFilter implements GlobalFilter, Ordered {

    public static final String HEADER = "X-Request-Id";

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String id = exchange.getRequest().getHeaders().getFirst(HEADER);
        if (id == null || id.isBlank()) {
            id = UUID.randomUUID().toString();
        }
        ServerHttpRequest request = exchange.getRequest().mutate().header(HEADER, id).build();
        exchange.getResponse().getHeaders().set(HEADER, id);
        return chain.filter(exchange.mutate().request(request).build());
    }

    @Override
    public int getOrder() {
        return Ordered.HIGHEST_PRECEDENCE;     // first in the pre phase, last in the post phase
    }
}
```

For service-to-service calls that need client-side balancing (a non-Kubernetes deployment), the `RestClient.Builder` is marked `@LoadBalanced` and the base URL contains the logical name:

LoadBalancedClientConfig.java JAVA

```java
package com.shop.order.infrastructure;

import org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

@Configuration(proxyBeanMethods = false)
class LoadBalancedClientConfig {

    @Bean
    @LoadBalanced
    RestClient.Builder loadBalancedRestClientBuilder() {
        return RestClient.builder();
    }

    @Bean
    RestClient inventoryRestClient(RestClient.Builder loadBalancedRestClientBuilder) {
        return loadBalancedRestClientBuilder.baseUrl("http://inventory-service").build();
    }
}
```

The Kubernetes side of the same system consists of a `Service`, which provides the stable name, and a `ConfigMap`, which provides the non-secret configuration. The checksum annotation forces a rolling restart when the ConfigMap changes, which closes the drift problem described below.

k8s/order-service.yaml (excerpt) YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service
  ports:
```

k8s/order-service.yaml (excerpt) (continued) YAML

```yaml
    - port: 8080
      targetPort: 8080
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: order-service-config
data:
  clients.inventory.base-url: "http://inventory-service:8080"
  resilience.timeout: "800ms"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
      annotations:
        checksum/config: "<sha256 of the ConfigMap, computed by Helm or the pipeline>"
    spec:
      containers:
        - name: order-service
          image: registry.example.com/shop/order-service:1.4.2
          env:
            - name: SPRING_CONFIG_IMPORT
              value: "optional:configtree:/etc/config/"
          volumeMounts:
            - name: config
              mountPath: /etc/config
      volumes:
        - name: config
          configMap:
            name: order-service-config
```

4.3.3 Hands-on

The behaviour of the gateway pipeline, round-robin selection, failover on a failed instance, path rewriting and token-bucket limiting is reproduced with plain JDK classes (`com.sun.net.httpserver` and `java.net.http`). The program starts four backend instances and a gateway, then sends requests.

MiniGateway.java JAVA

```java
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

public class MiniGateway {

    record Instance(String id, int port) {}

    /** Route = Path predicate + StripPrefix filter + client-side round-robin over instances. */
    static class Route {
        final String id, prefix;
        final List<Instance> instances;
        final AtomicInteger next = new AtomicInteger();
        final Map<String, Long> downUntil = new ConcurrentHashMap<>();
```

sreeram

MiniGateway.java (continued) JAVA

```
        Route(String id, String prefix, List<Instance> instances) {
            this.id = id;
            this.prefix = prefix;
            this.instances = instances;
        }

        boolean matches(String path) { return path.startsWith(prefix); }
        Optional<Instance> choose(Set<String> tried) {
            long now = System.currentTimeMillis();
            for (int i = 0; i < instances.size(); i++) {
                Instance c = instances.get(Math.floorMod(next.getAndIncrement(), instances.size()));
                if (tried.contains(c.id()) || downUntil.getOrDefault(c.id(), 0L) > now) continue;
                return Optional.of(c);
            }
            return Optional.empty();
        }
    }

    /** Token bucket: capacity 5, refilled at 5 tokens per second, keyed by client. */
    static class Bucket {
        double tokens = 5; long last = System.nanoTime();
        synchronized boolean take() {
            long n = System.nanoTime();
            tokens = Math.min(5, tokens + (n - last) / 1e9 * 5); last = n;
            if (tokens >= 1) { tokens -= 1; return true; }
            return false;
        }
    }

    static HttpServer backend(String id, int port) throws IOException {
        HttpServer s = HttpServer.create(new InetSocketAddress("127.0.0.1", port), 0);
        s.createContext("/", ex -> {
            String body = "{\"instance\":\"" + id + "\",\"path\":\"" + ex.getRequestURI()
                    + "\",\"requestId\":\""
                    + ex.getRequestHeaders().getFirst("X-Request-Id") + "\"}";
            reply(ex, 200, body);
        });
        s.start();
        return s;
    }

    static void reply(HttpExchange ex, int code, String body) throws IOException {
        byte[] b = body.getBytes(StandardCharsets.UTF_8);
        ex.getResponseHeaders().add("Content-Type", "application/json");
        ex.sendResponseHeaders(code, b.length);
        try (var os = ex.getResponseBody()) { os.write(b); }
    }

    public static void main(String[] args) throws Exception {
        HttpServer o1 = backend("order-1", 21401), o2 = backend("order-2", 21402),
                o3 = backend("order-3", 21403);
        HttpServer u1 = backend("user-1", 21404);
        List<Route> routes = List.of(
                new Route("orders", "/api/orders", List.of(new Instance("order-1", 21401),
                        new Instance("order-2", 21402), new Instance("order-3", 21403))),
                new Route("users", "/api/users", List.of(new Instance("user-1", 21404))));
        Map<String, Bucket> buckets = new ConcurrentHashMap<>();
        HttpClient http = HttpClient.newBuilder().connectTimeout(Duration.ofMillis(300)).build();
        AtomicInteger ids = new AtomicInteger(1000);

        HttpServer gw = HttpServer.create(new InetSocketAddress("127.0.0.1", 21410), 0);
        gw.createContext("/", ex -> {
            String client = ex.getRequestHeaders().getFirst("X-Client");
            if (client != null && !buckets.computeIfAbsent(client, k -> new Bucket()).take()) {
                reply(ex, 429, "{\"error\":\"rate limit exceeded\"}"); return;
            }
            String path = ex.getRequestURI().getPath();
            Route r = routes.stream().filter(x -> x.matches(path)).findFirst().orElse(null);
            if (r == null) { reply(ex, 404, "{\"error\":\"no route\"}"); return; }
            String stripped = path.substring("/api".length());       // StripPrefix=1
            Set<String> tried = new HashSet<>();
            for (int attempt = 0; attempt < 3; attempt++) {
                Optional<Instance> inst = r.choose(tried);
                if (inst.isEmpty()) break;
```

MiniGateway.java (continued) JAVA

```
                tried.add(inst.get().id());
                try {
                    HttpRequest req = HttpRequest.newBuilder(URI.create("http://127.0.0.1:"
                            + inst.get().port() + stripped))
                            .timeout(Duration.ofSeconds(1)).header("X-Request-Id", "req-"
                                    + ids.incrementAndGet()).GET().build();
                    HttpResponse<String> resp = http.send(req,
                            HttpResponse.BodyHandlers.ofString());
                    reply(ex, resp.statusCode(), resp.body());
                    return;
                } catch (IOException | InterruptedException e) {
                    r.downUntil.put(inst.get().id(), System.currentTimeMillis() + 2000);
                    System.out.println("  gateway: " + inst.get().id() + " failed ("
                            + e.getClass().getSimpleName()
                            + "), marked down for 2 s, trying next");
                }
            }
            reply(ex, 503, "{\"error\":\"no healthy instance for route " + r.id + "\"}");
        });
        gw.start();

        HttpClient c = HttpClient.newHttpClient();
        String[] last = new String[1];
        java.util.function.BiConsumer<String, String> get = (path, client) -> {
            try {
                var b = HttpRequest.newBuilder(URI.create("http://127.0.0.1:21410" + path)).GET();
                if (client != null) b.header("X-Client", client);
                var r = c.send(b.build(), HttpResponse.BodyHandlers.ofString());
                System.out.println("GET " + path + " -> " + r.statusCode() + " " + r.body());
            } catch (Exception e) { System.out.println("client error " + e); }
        };
        System.out.println("-- round robin over 3 order instances");
        for (int i = 0; i < 6; i++) get.accept("/api/orders/" + (100 + i), null);
        System.out.println("-- other route and unknown route");
        get.accept("/api/users/7", null);
        get.accept("/api/payments/1", null);
        System.out.println("-- order-2 is stopped; the gateway fails over");
        o2.stop(0);
        for (int i = 0; i < 4; i++) get.accept("/api/orders/" + (200 + i), null);
        System.out.println(
                "-- rate limit: bucket of 5 tokens, 8 rapid requests from client 'mobile-app'");
        for (int i = 0; i < 8; i++) get.accept("/api/users/" + i, "mobile-app");
        o1.stop(0); o3.stop(0); u1.stop(0); gw.stop(0);
    }
}
```

Run the mini gateway (request ids and the number of admitted requests vary with timing) TERMINAL

```
$ java MiniGateway.java
-- round robin over 3 order instances
GET /api/orders/100 -> 200 {"instance":"order-1","path":"/orders/100","requestId":"req-1001"}
GET /api/orders/101 -> 200 {"instance":"order-2","path":"/orders/101","requestId":"req-1002"}
GET /api/orders/102 -> 200 {"instance":"order-3","path":"/orders/102","requestId":"req-1003"}
GET /api/orders/103 -> 200 {"instance":"order-1","path":"/orders/103","requestId":"req-1004"}
GET /api/orders/104 -> 200 {"instance":"order-2","path":"/orders/104","requestId":"req-1005"}
GET /api/orders/105 -> 200 {"instance":"order-3","path":"/orders/105","requestId":"req-1006"}
-- other route and unknown route
GET /api/users/7 -> 200 {"instance":"user-1","path":"/users/7","requestId":"req-1007"}
GET /api/payments/1 -> 404 {"error":"no route"}
-- order-2 is stopped; the gateway fails over
GET /api/orders/200 -> 200 {"instance":"order-1","path":"/orders/200","requestId":"req-1008"}
  gateway: order-2 failed (ConnectException), marked down for 2 s, trying next
GET /api/orders/201 -> 200 {"instance":"order-3","path":"/orders/201","requestId":"req-1010"}
GET /api/orders/202 -> 200 {"instance":"order-1","path":"/orders/202","requestId":"req-1011"}
GET /api/orders/203 -> 200 {"instance":"order-3","path":"/orders/203","requestId":"req-1012"}
-- rate limit: bucket of 5 tokens, 8 rapid requests from client 'mobile-app'
GET /api/users/0 -> 200 {"instance":"user-1","path":"/users/0","requestId":"req-1013"}
GET /api/users/1 -> 200 {"instance":"user-1","path":"/users/1","requestId":"req-1014"}
GET /api/users/2 -> 200 {"instance":"user-1","path":"/users/2","requestId":"req-1015"}
GET /api/users/3 -> 200 {"instance":"user-1","path":"/users/3","requestId":"req-1016"}
GET /api/users/4 -> 200 {"instance":"user-1","path":"/users/4","requestId":"req-1017"}
GET /api/users/5 -> 200 {"instance":"user-1","path":"/users/5","requestId":"req-1018"}
GET /api/users/6 -> 429 {"error":"rate limit exceeded"}
GET /api/users/7 -> 429 {"error":"rate limit exceeded"}
```

Observations. The first six requests rotate over `order-1`, `order-2`, `order-3`. The prefix `/api` is stripped before forwarding, which is the effect of `StripPrefix=1`. After `order-2` is stopped, the first request routed to it fails with a connection error, the gateway marks the instance down for two seconds (a passive health check) and retries the same request on the next instance, so the client sees no error; the request ids skip a number because the failed attempt consumed one. Six rather than five of the eight rapid requests passed the limiter because the bucket refills at five tokens per second while the requests are being issued.

4.3.4 Real-world problems and solutions

Problem 1: gateway returns 504 although the service completed the work

Scenario. The gateway has `response-timeout: 3s`. The order service takes 3.4 seconds for a checkout during a database slowdown.

Symptoms. Clients receive `504 Gateway Timeout` and retry. The order table later contains two orders for one cart. The order service logs show the request completing successfully after the client was gone.

Diagnosis. Compare timestamps: the gateway access log shows the 504 at 3.0 s, the order service shows the completion at 3.4 s. The metric `spring.cloud.gateway.requests` (tags `routeId`, `status`) shows the 504 count rising on the route. `GET /actuator/gateway/routes` shows the effective route configuration, including any per-route `response-timeout` metadata.

Root cause. The timeouts do not form a budget: the service has no deadline shorter than the gateway's, so it continues to work for a client that has already been answered, and the client's retry creates a duplicate.

Fix. Order the timeouts: client greater than gateway greater than service call chain. Give the service its own bounded work (query timeout via `spring.jpa.properties.jakarta.persistence.query.timeout`, client timeouts as in the previous chapter), set long-running routes explicitly through the `response-timeout` metadata, and make checkout idempotent.

Prevention. Keep the budget table (client, gateway, service, database timeouts) in the repository next to the configuration and review it with every timeout change.

Problem 2: 503 and connection errors for minutes after each deployment

Scenario. A rolling update replaces the pods of the inventory service. During each deployment the order service logs `Connection refused` and `503` for a few minutes.

Symptoms. The error rate spikes at every release and recovers without intervention. Pods that were terminated are still receiving requests.

Diagnosis. (1) `kubectl get endpoints inventory-service -w` while the rollout runs shows when the old addresses disappear. (2) Compare the timestamp of the pod termination with the last request logged by that pod. (3) For a registry-based setup, `GET /eureka/apps/INVENTORY-SERVICE` shows entries for instances that no longer exist, and the metric or log of the load balancer cache shows the 35 second window.

Root cause. Termination and endpoint removal are asynchronous. On Kubernetes, the pod receives SIGTERM at about the same moment that the endpoint is removed from the Service; components that have not yet observed the change keep sending requests. In a registry model the staleness window is the sum of the timers in the earlier diagram.

Fix. Enable graceful shutdown (`server.shutdown=graceful` with `spring.lifecycle.timeout-per-shutdown-phase=30s`), add a `preStop` hook with a short sleep (for example five seconds) so that the endpoint removal propagates before the server stops accepting connections, make readiness fail first, and let clients retry idempotent calls on a different instance.

Prevention. A deployment test that runs a constant request load during a rolling update and fails the pipeline on any non-2xx result.

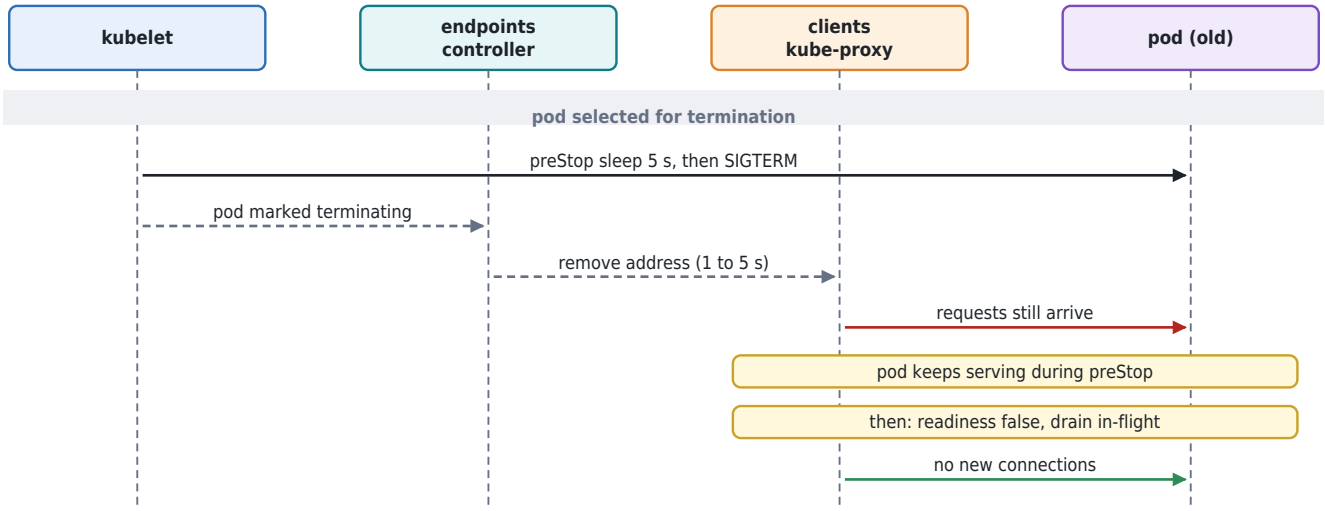


Figure 4.14. The sequence during pod termination: the kubelet signal and the endpoint removal are independent, so the pod must keep serving for the propagation interval before it begins to refuse connections.

Problem 3: one instance behaves differently (configuration drift)

Scenario. Rate limiting is configured as 20 requests per second in the ConfigMap. One of five gateway pods rejects clients at 10 requests per second.

Symptoms. Intermittent 429 responses for the same client; identical requests behave differently depending on the pod that serves them.

Diagnosis. Compare the effective configuration of each pod: `kubectl exec <pod> -- env | grep -i rate`, `GET /actuator/env/<property>` (the response lists every property source that defines the value and which one won) and `GET /actuator/configprops` for bound beans (both endpoints require exposure and protection; see the Actuator chapter). A pod that started before the ConfigMap edit has the old value because environment variables are read once at start-up.

Root cause. The ConfigMap was edited in place. Pods that consume it as environment variables keep the old values until restarted, and pods that mount it as a volume receive the new file after the kubelet sync delay, but Boot reads the files only at start-up unless a refresh is triggered.

Fix. Treat configuration as part of the release: add the checksum annotation shown above so that a ConfigMap change causes a rolling restart, or use Spring Cloud Config or Kubernetes configuration reload with an explicit refresh.

Prevention. Expose the effective value of critical settings as a metric or an info contribution, and alert when instances of one application report different values.

Problem 4: new pods receive no traffic after scaling out

Scenario. The inventory service is scaled from 2 to 6 pods during a sale; latency on the original two pods does not improve.

Symptoms. CPU of two pods at saturation, four pods idle; the request rate per pod is uneven.

Diagnosis. Per-pod request rate (`http.server.requests` grouped by pod label); the number of established connections per pod (`ss -tn state established | grep :8080 | wc -l` inside each pod); client pool settings show a small pool of long-lived keep-alive connections.

Root cause. The clients hold persistent connections created before the scale-out, and `kube-proxy` balanced those connections, not the requests.

Fix. Limit connection lifetime on the clients (the Apache `ConnectionConfig` time-to-live, for example 30 seconds) or on the server (`server.tomcat.keep-alive-timeout` and `server.tomcat.max-keep-alive-requests`), or use a request-level balancer.

Prevention. Scale-out is validated in load tests by checking the per-pod request distribution, not only the aggregate latency.

4.3.5 Questions to expect

1. **Why is service discovery needed?** Instances are ephemeral and their addresses change; callers need a stable logical name that resolves to the currently healthy instances.
2. **Client-side versus server-side discovery?** Client-side: the client queries a registry and balances itself (Eureka plus Spring Cloud LoadBalancer), giving per-request control at the cost of library coupling. Server-side: the platform balances (Kubernetes Service), keeping clients simple but balancing per connection.
3. **Do Spring applications on Kubernetes need Eureka?** Normally not; the Service DNS name and readiness probes provide discovery and health filtering. Eureka remains useful off Kubernetes.
4. **What is the default load-balancing algorithm of Spring Cloud LoadBalancer?** Round robin (`RoundRobinLoadBalancer`); a random implementation is available, and the instance list comes from a `ServiceInstanceListSupplier` chain.
5. **What does an API gateway do and what should it not do?** It routes, authenticates, rate limits, applies edge resilience and adds correlation data. It should not contain business logic, because it is shared by all teams and becomes a deployment bottleneck.
6. **How does the gateway filter chain work?** The matched route and the global filters form an ordered chain; each filter has a pre phase before calling the chain and a post phase afterwards; the highest precedence runs first on the way in and last on the way out.
7. **How does `RequestRateLimiter` work and where is the state?** Token bucket with `replenishRate` and `burstCapacity`, state in Redis so that the limit holds across gateway instances; a `KeyResolver` selects the limited identity; excess requests receive 429.
8. **How is configuration kept consistent across instances?** One source of truth (ConfigMap or Config Server), rolled out through the release process, with a restart or an explicit refresh and with the effective values observable at run time.
9. **Why can scaling out not help immediately on Kubernetes?** The Service balances connections; long-lived keep-alive or HTTP/2 connections remain pinned to the old pods.

Check yourself

1. A gateway has `response-timeout: 2s`. A route to a report service needs 15 s. Which setting solves this without raising the global value?
2. In the `RequestIdFilter` the order is `Ordered.HIGHEST_PRECEDENCE`. In which phase does it run first, and in which does it run last?
3. A pod mounts a ConfigMap as a volume and the ConfigMap is edited. Does the Spring Boot application see the new value?
4. Why does `kubectl scale` from 2 to 6 replicas not reduce the load of the two original pods when clients use keep-alive?

Answers

1. The per-route metadata `response-timeout` (milliseconds) on the report route.
2. First in the pre phase (before the request is forwarded) and last in the post phase.
3. Not automatically. The file in the volume is updated after the kubelet sync delay, but Boot binds properties at start-up; a restart (for example via a checksum annotation) or an explicit refresh is needed.
4. Balancing happens per connection when it is created; existing connections remain on the original pods.

KEY POINTS

Discovery, balancing, gateway and configuration decide how a request finds a healthy, correctly configured instance.

- On Kubernetes the Service name is the discovery mechanism and `kube-proxy` balances connections; client-side registries remain for non-Kubernetes platforms.
- Spring Cloud Gateway evaluates predicates, then runs an ordered chain of global and route filters with pre and post phases; rate limiting is a Redis-backed token bucket.
- Registry staleness is the sum of lease, eviction and cache timers; graceful shutdown, preStop delay and retries cover the gap.
- Timeouts form a budget (client, gateway, service); a gateway timeout without a bounded service produces duplicate work.
- Configuration drift is prevented by treating configuration as a release artifact: checksum-triggered rollouts and observable effective values.

sreram

4.4 Resilience: Timeouts, Retries, Circuit Breakers and Bulkheads

A dependency that fails quickly is an inconvenience; a dependency that responds slowly is an outage. A failed call releases its thread within milliseconds, whereas a slow call holds a thread, a pooled connection and often a database transaction for as long as the call lasts, and the caller's capacity is consumed by requests that cannot complete. Resilience engineering for services is therefore concerned with bounding the damage that one slow or failing component can cause to the components that depend on it. The mechanisms in this chapter are timeouts, retries with backoff and jitter, circuit breakers, bulkheads, rate limiters, fallbacks and load shedding. Each addresses a specific failure mode, and several of them cause incidents of their own when combined carelessly; the chapter pays as much attention to those interactions as to the mechanisms.

4.4.1 Concept and mechanism

Why slowness cascades

Little's law relates the average number of requests in a system (`L`) to the arrival rate (`λ`) and the average time each spends in it (`W`): `L = λ × W`. A service that receives 100 requests per second and answers in 20 ms holds two requests in flight on average. If one of its dependencies begins to answer in 5 seconds, the same arrival rate requires `100 × 5 = 500` concurrent requests. A Tomcat pool of 200 threads is exhausted within about two seconds; from then on new requests queue, wait for the accept queue, and time out in the caller, which in turn begins to accumulate waiting threads. The failure moves up the call chain at the speed of the thread pools filling.

Dependency latency	Requests in flight	Fits in 200 threads	Consequence
20 ms	2	yes	normal operation
200 ms	20	yes	slower but healthy
2 s	200	borderline	saturation at any burst
5 s	500	no	pool exhausted in about 2 s, queue grows

Concurrency demanded by Little's law for an arrival rate of 100 requests per second.

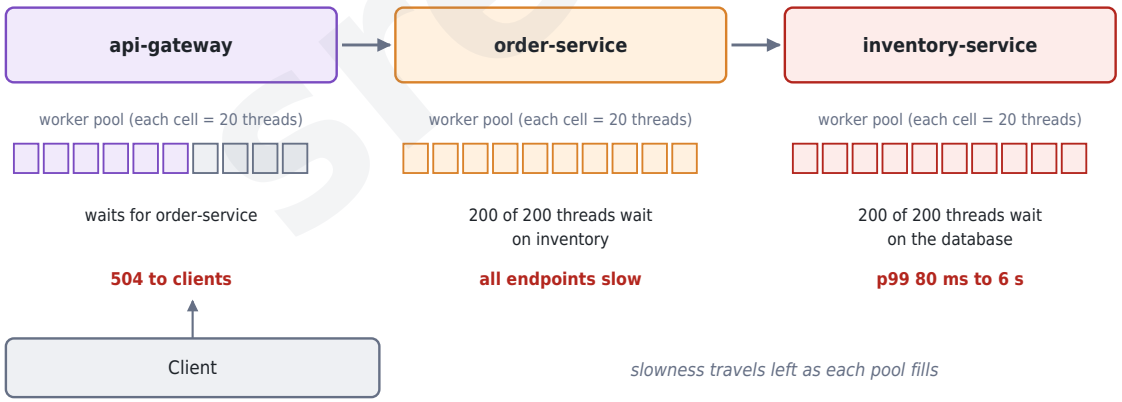


Figure 4.15. A slow inventory service fills the worker pool of the order service, whose own callers then wait on it; the pool occupancy shown is the evidence that appears in metrics.

The defences operate at different points of this process. A **timeout** limits how long one call may hold a thread. A **bulkhead** limits how many threads one dependency may hold. A **circuit breaker** stops sending calls to a dependency that is known to be failing, so that no thread is spent on it at all. **Load shedding** refuses work early when the service itself is saturated. A **fallback** supplies a degraded answer instead of an error.

Timeouts

Timeouts are the foundation, since every other mechanism relies on calls that end. The phases and the budget rule were described in the chapter on synchronous communication. Resilience4j adds the `TimeLimiter`, which bounds the total duration of an asynchronous call (`CompletableFuture` or other `CompletionStage`) and can cancel it; its configuration keys are `timeoutDuration` and `cancelRunningFuture`. A `TimeLimiter` cannot interrupt a blocking method that is called synchronously from the caller's thread, so for blocking `RestClient` code the client library timeouts remain the effective bound.

Retries, backoff and jitter

A retry converts a transient failure (a dropped connection, a pod being replaced, a short overload) into a success at the cost of extra load and extra latency. Three conditions must hold for a retry to be correct: the failure is plausibly transient, the operation is idempotent or protected by a key, and the extra load does not endanger the dependency.

Backoff spaces the attempts: the delay before attempt `n` grows exponentially, `delay(n) = base × multiplier^(n-1)`, optionally capped. **Jitter** randomises the delay. Without jitter, clients that failed at the same instant (because the same dependency failed) retry at the same instants, so the dependency receives synchronised waves just as it tries to recover. Two common schemes are full jitter (a uniform random delay between zero and the exponential value) and equal jitter (half of the exponential value plus a random half). Resilience4j exposes this as `enableExponentialBackoff`, `exponentialBackoffMultiplier`, `enableRandomizedWait` and `randomizedWaitFactor`.

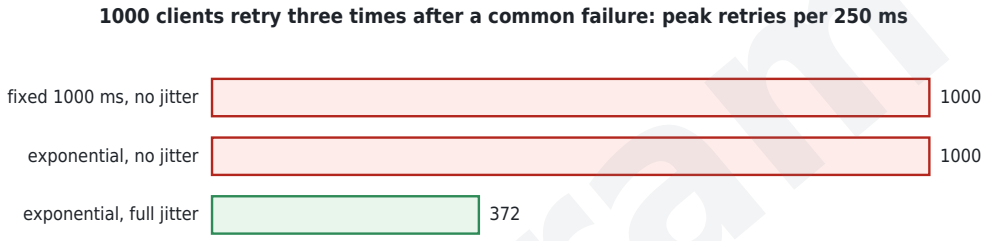


Figure 4.16. One thousand clients that fail together and retry: without jitter every retry arrives in the same 250 ms bucket (peak 1000), with full jitter the peak is 372; values come from the program in the Hands-on section.

The more serious hazard is **retry amplification**. If each of three layers in a call chain (gateway, order service, inventory client) retries three times, one user request produces `3 × 3 × 3 = 27` calls against the failing dependency. A dependency that was overloaded at 100 requests per second now receives 2700 per second: the retries convert a partial failure into a total one, which is a **retry storm**. The rules that prevent it are retry at one layer only (usually the one closest to the failing call), a small attempt count, a retry budget (for example, allow retries only up to 10 percent of the request volume), and no retry when the circuit is open.

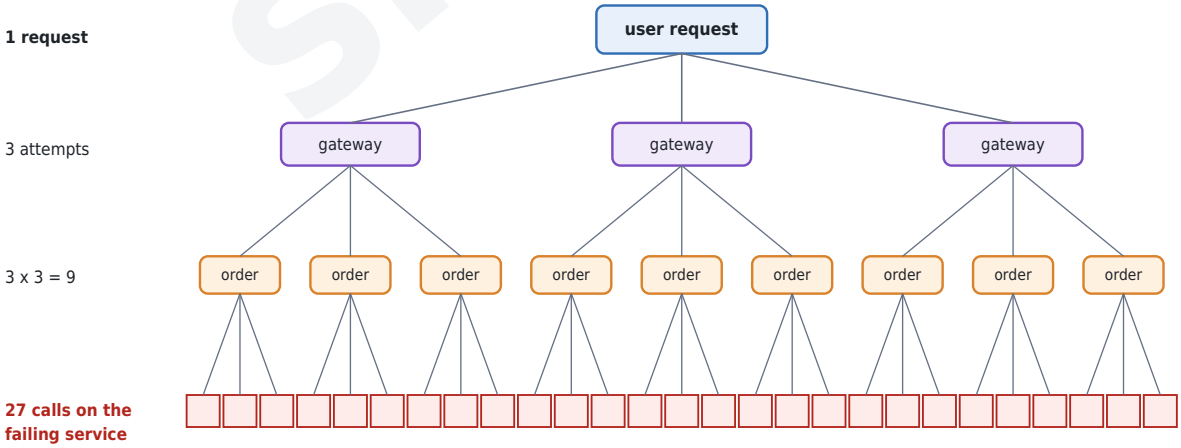


Figure 4.17. The call tree for three layers with three attempts each: the number of leaf calls multiplies along the chain, so one request becomes 27 calls on the failing dependency.

Circuit breaker

A circuit breaker wraps a call and observes its outcomes. While the dependency is healthy it is transparent. When the observed failure rate exceeds a threshold it opens and rejects calls immediately without touching the dependency, which protects the caller's threads and gives the dependency room to recover. After a wait period it lets a limited number of probe calls through; their result decides whether to close or to open again.

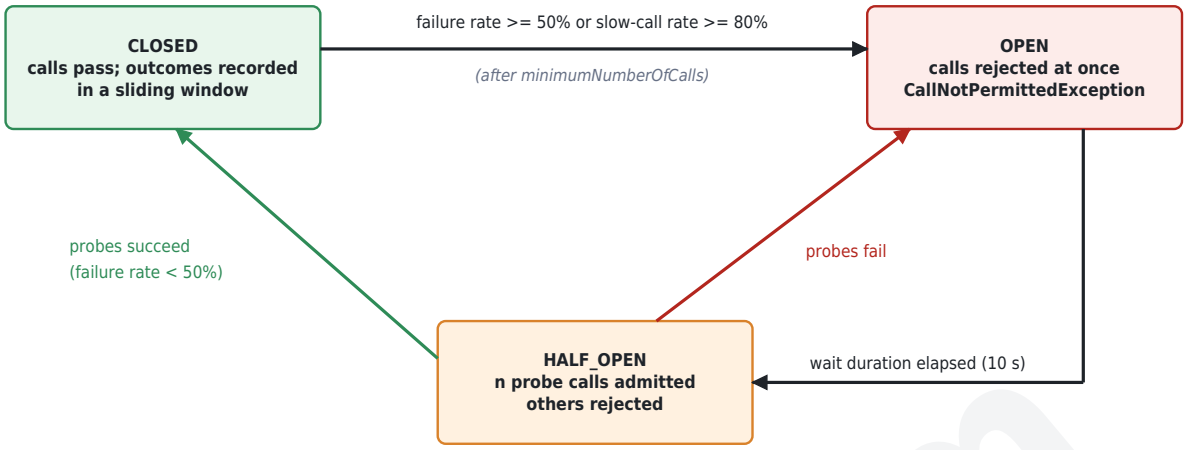


Figure 4.18. The three states and the conditions of each transition; the open state rejects calls with `CallNotPermittedException`, and the half-open state admits only a limited number of probes.

The parameters of the Resilience4j breaker, with the documented defaults:

Property	Default	Meaning
`slidingWindowType`	COUNT_BASED	the window is the last N calls, or the last N seconds (TIME_BASED)
`slidingWindowSize`	100	window size in calls or seconds
`minimumNumberOfCalls`	100	calls needed before the failure rate is evaluated at all
`failureRateThreshold`	50	percentage of failures that opens the breaker
`slowCallDurationThreshold`	60 s	a call longer than this counts as slow
`slowCallRateThreshold`	100	percentage of slow calls that opens the breaker
`waitDurationInOpenState`	60 s	time in OPEN before half-open
`permittedNumberOfCallsInHalfOpenState`	10	probes admitted in HALF_OPEN
`automaticTransitionFromOpenToHalfOpenEnabled`	false	move to HALF_OPEN by timer, not on the next call
`recordExceptions`, `ignoreExceptions`	all exceptions record	which exceptions count as failures, and which are neutral

Resilience4j circuit breaker configuration properties.

Two defaults deserve attention. `minimumNumberOfCalls` equal to 100 means that a service with ten calls per minute needs more than ten minutes of traffic before the breaker can open at all. `waitDurationInOpenState` of 60 seconds keeps a recovered dependency isolated for a minute after it has returned. Both are rarely appropriate unchanged. The slow-call settings are valuable because most real outages present first as latency, not as errors.

What counts as a failure is a design decision: timeouts, connection errors and 5xx responses indicate a sick dependency and are recorded; 4xx responses and business rejections (insufficient stock) are the dependency working correctly and must be ignored, or a burst of invalid requests would open the breaker.

Bulkhead

A bulkhead partitions capacity, as the compartments of a ship limit the spread of flooding. The **semaphore bulkhead** limits the number of concurrent calls to a dependency (`maxConcurrentCalls`, default 25) and rejects excess calls after `maxWaitDuration` (default 0, which means immediately) with `BulkheadFullException`. The **thread-pool bulkhead** executes the calls on a dedicated pool with a queue, isolating them completely from the caller's threads; it requires asynchronous invocation and costs a thread hand-off. The semaphore variant is sufficient for most blocking Spring applications.

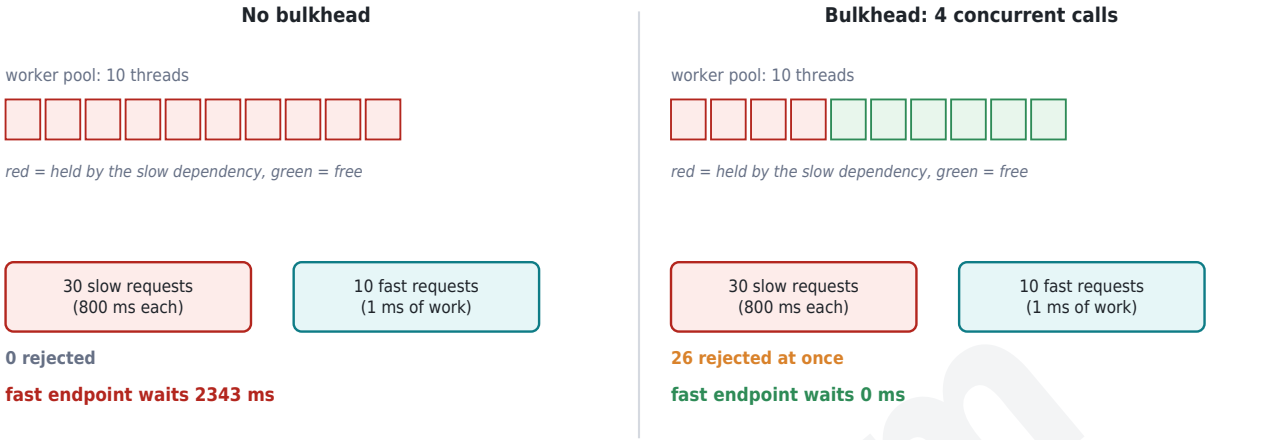


Figure 4.19. A shared worker pool serves two endpoints; without a bulkhead the slow dependency takes every thread and the fast endpoint starves, with a bulkhead of four the excess slow calls are rejected and the fast endpoint is unaffected.

Rate limiter, load shedding and fallbacks

A **client-side rate limiter** (`limitForPeriod`, `limitRefreshPeriod`, `timeoutDuration`) protects a dependency with a known quota (a payment provider with 50 requests per second). A **server-side limiter** at the gateway protects the system from abusive or runaway clients. **Load shedding** is the server protecting itself: when a queue exceeds a bound the service answers `503 Service Unavailable` with a `Retry-After` header at once instead of accepting work it cannot finish. In Spring Boot the relevant Tomcat settings are `server.tomcat.threads.max`, `server.tomcat.accept-count` (the queue of pending connections) and `server.tomcat.max-connections`; an application-level limiter such as a semaphore around expensive endpoints gives finer control.

A **fallback** is an alternative result when the primary call fails or is rejected. Useful forms are a cached or last-known value (a stale price is better than no price), a default (an empty recommendation list), a simplified computation, and a queued request processed later. A fallback that hides a failure which the caller must act on is harmful: for a stock reservation the correct reaction to a failed call is to reject the order, not to assume success. The design question for each dependency is whether degraded operation is acceptable, and if so, in what form; this is **graceful degradation**.

Composition order

Resilience4j annotations and decorators are proxy-based, and the order in which they wrap the call matters. The default aspect order is Retry outermost, then CircuitBreaker, RateLimiter, TimeLimiter, Bulkhead innermost, which is written `Retry(CircuitBreaker(RateLimiter(TimeLimiter(Bulkhead(call)))))`. The order is adjustable through `resilience4j.retry.retryAspectOrder`, `circuitBreakerAspectOrder` and the equivalent keys (a higher value means higher priority).

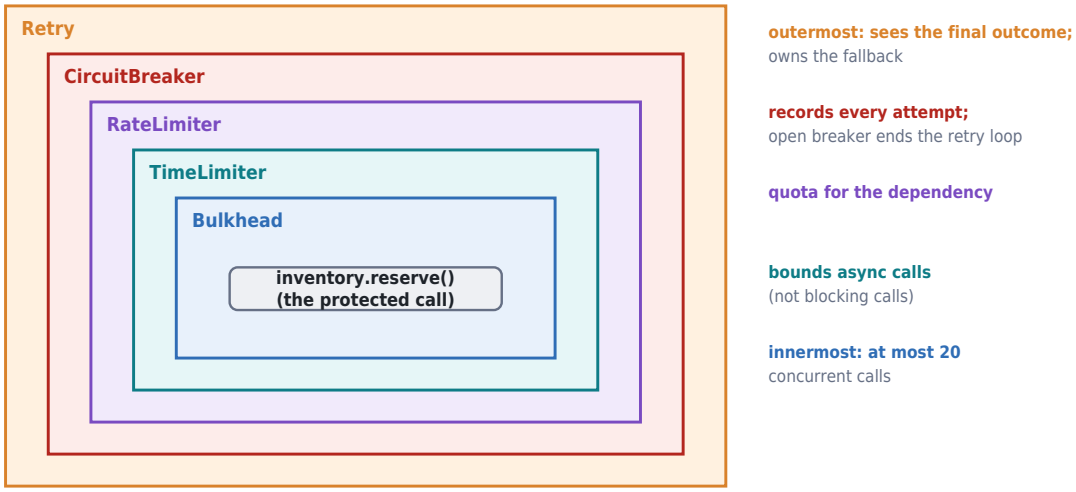


Figure 4.20. The default nesting of the decorators: every retry attempt passes through the breaker, so each attempt is recorded, and an open breaker causes the retry to receive `CallNotPermittedException`, which the retry configuration should ignore.

4.4.2 Worked example

The order service calls the inventory service for stock reservations. The gateway class adds the decorators; the configuration lives in YAML so that thresholds are changed without code changes.

order-service/pom.xml (dependencies added) XML

```xml
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

The Resilience4j version is managed by the Spring Cloud BOM when the Spring Cloud circuit breaker starter is used; when only `resilience4j-spring-boot3` is added, its version is declared explicitly.

InventoryGateway.java JAVA

```java
package com.shop.order.infrastructure;

import com.shop.order.application.OrderService;
import com.shop.order.application.StockReservation;
import io.github.resilience4j.bulkhead.annotation.Bulkhead;
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import io.github.resilience4j.retry.annotation.Retry;
import java.util.List;
import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Component
public class InventoryGateway implements StockReservation {

    private static final Logger log = LoggerFactory.getLogger(InventoryGateway.class);

    private final InventoryClient client;

    public InventoryGateway(InventoryClient client) {
        this.client = client;
    }

    @Bulkhead(name = "inventory")
    @CircuitBreaker(name = "inventory")
```

sreeram

InventoryGateway.java (continued) JAVA

```java
    @Retry(name = "inventory", fallbackMethod = "reservationUnavailable")
    @Override
    public void reserve(UUID orderId, List<OrderService.Item> items) {
        List<InventoryClient.Line> lines = items.stream()
            .map(i -> new InventoryClient.Line(i.sku(), i.quantity()))
            .toList();
        // the order id is the idempotency key: every retry of this reservation carries the same key
        client.reserve(orderId.toString(), new InventoryClient.ReservationRequest(orderId, lines));
    }

    /** Fallback: the failure is translated into a domain exception, not hidden. */
    void reservationUnavailable(UUID orderId, List<OrderService.Item> items, Throwable cause) {
        log.warn("Stock reservation for order {} unavailable: {}", orderId, cause.toString());
        throw new StockUnavailableException(orderId, cause);
    }
}
```

The fallback method has the same parameters as the protected method plus a trailing exception parameter and the same return type; Resilience4j selects the most specific matching fallback. The fallback is attached to `@Retry`, the outermost decorator, deliberately: a fallback on the inner `@CircuitBreaker` would convert every failure into `StockUnavailableException` before the retry could see it, and since that type is not listed in `retryExceptions` no retry would ever occur. On the outermost decorator it runs only after the retries are exhausted, and it also receives `CallNotPermittedException` and `BulkheadFullException`. The fallback rethrows a domain exception because a reservation cannot be assumed: the order service answers the caller with `503` and a `Retry-After` header (see the chapter on API design), and the order is not created.

order-service/src/main/resources/application.yml (resilience part) YAML

```yaml
resilience4j:
  circuitbreaker:
    configs:
      default:
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 20
        minimumNumberOfCalls: 10
        failureRateThreshold: 50
        slowCallDurationThreshold: 1s
        slowCallRateThreshold: 80
        waitDurationInOpenState: 10s
        permittedNumberOfCallsInHalfOpenState: 5
        automaticTransitionFromOpenToHalfOpenEnabled: true
        registerHealthIndicator: true
        recordExceptions:
          - com.shop.order.infrastructure.DependencyUnavailableException
          - org.springframework.web.client.ResourceAccessException
        ignoreExceptions:
          - org.springframework.web.client.HttpClientErrorException
    instances:
      inventory:
        baseConfig: default
  retry:
    instances:
      inventory:
        maxAttempts: 3
        waitDuration: 100ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        enableRandomizedWait: true
        randomizedWaitFactor: 0.5
        retryExceptions:
          - com.shop.order.infrastructure.DependencyUnavailableException
          - org.springframework.web.client.ResourceAccessException
        ignoreExceptions:
          - io.github.resilience4j.circuitbreaker.CallNotPermittedException
  bulkhead:
    instances:
      inventory:
        maxConcurrentCalls: 20
        maxWaitDuration: 0

management:
  health:
    circuitbreakers:
```

sreeram

order-service/src/main/resources/application.yml (resilience part) (continued) YAML

```yaml
      enabled: true
  endpoints:
    web:
      exposure:
        include: health,prometheus,circuitbreakers,circuitbreakerevents,retries,bulkheads
```

Points that the configuration encodes. Retry and breaker list the same two exception types as failures, which excludes 4xx responses. The retry ignores `CallNotPermittedException`, so an open breaker ends the call immediately instead of consuming the retry attempts. The slow-call threshold (1 second) is above the HTTP client's 800 ms read timeout, so timeouts count as failures and the slow-call rule catches a dependency that answers just inside the timeout. The bulkhead limit (20) is below the Tomcat pool size (200), so the inventory service can never take more than a tenth of the worker threads.

State transitions are logged through the event publisher; the listener documents the breaker's decisions in the log, which is the first evidence examined during an incident:

ResilienceEventLogging.java JAVA

```java
package com.shop.order.infrastructure;

import io.github.resilience4j.circuitbreaker.CircuitBreakerRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration(proxyBeanMethods = false)
class ResilienceEventLogging {

    private static final Logger log = LoggerFactory.getLogger(ResilienceEventLogging.class);

    @Bean
    ApplicationRunner circuitBreakerTransitionLogger(CircuitBreakerRegistry registry) {
        return args -> {
            registry.getAllCircuitBreakers().forEach(this::register);
            registry.getEventPublisher().onEntryAdded(event -> register(event.getAddedEntry()));
        };
    }

    private void register(io.github.resilience4j.circuitbreaker.CircuitBreaker breaker) {
        breaker.getEventPublisher().onStateTransition(event -> log.warn(
            "circuit breaker {} transitioned {}", event.getCircuitBreakerName(),
            event.getStateTransition()));
    }
}
```

4.4.3 Hands-on

Because Spring jars are not available on the build machine, the mechanisms are demonstrated with plain JDK implementations that follow the Resilience4j semantics described above. The first program computes backoff schedules, retry amplification and the thundering-herd effect.

RetryDemo.java JAVA

```java
import java.util.Random;
import java.util.TreeMap;
import java.util.Map;

public class RetryDemo {

    static int downstreamCalls;

    /** One layer calls the next and retries on failure; the innermost dependency always fails. */
    static boolean call(int layer, int[] attempts) {
        if (layer == attempts.length) { downstreamCalls++; return false; }
        for (int i = 0; i < attempts[layer]; i++) {
            if (call(layer + 1, attempts)) return true;
        }
        return false;
    }
```

RetryDemo.java (continued) JAVA

```
    static void backoff() {
        System.out.println("== backoff schedule (base 100 ms, multiplier 2, 5 attempts, seed 42)");
        Random r = new Random(42);
        long delay = 100;
        for (int attempt = 1; attempt <= 4; attempt++) {
            long full = (long) (r.nextDouble() * delay);
            long equal = delay / 2 + (long) (r.nextDouble() * delay / 2);
            System.out.printf(
                    "after failure %d: exponential=%4d  full-jitter=%4d  equal-jitter=%4d (ms)%n",
                    attempt, delay, full, equal);
            delay *= 2;
        }
        System.out.println();
    }

    static void storm() {
        System.out.println("== retry amplification when the innermost dependency is down");
        int[][] cfgs = {{3, 3, 3}, {3, 3, 1}, {1, 1, 3}};
        String[] names = {"gateway=3 order=3 inventory-client=3",
                "gateway=3 order=3 inventory-client=1",
                "gateway=1 order=1 inventory-client=3"};
        for (int i = 0; i < cfgs.length; i++) {
            downstreamCalls = 0;
            call(0, cfgs[i]);
            System.out.printf("%-42s -> %2d calls reach the failing dependency per user request%n",
                    names[i], downstreamCalls);
        }
        System.out.println();
    }

    static void herd() {
        System.out.println("== 1000 clients fail at t=0 and retry; arrivals per 250 ms bucket");
        String[] modes = {"fixed 1000 ms, no jitter", "exponential 1000*2^n, no jitter",
                "exponential, full jitter"};
        for (int m = 0; m < modes.length; m++) {
            Random r = new Random(7);
            TreeMap<Long, Integer> buckets = new TreeMap<>();
            for (int c = 0; c < 1000; c++) {
                long t = 0;
                for (int attempt = 0; attempt < 3; attempt++) {
                    long cap = m == 0 ? 1000 : 1000L << attempt;
                    long wait = m == 2 ? (long) (r.nextDouble() * cap) : cap;
                    t += wait;
                    buckets.merge(t / 250 * 250, 1, Integer::sum);
                }
            }
            int peak = buckets.values().stream().mapToInt(Integer::intValue).max().orElse(0);
            System.out.printf("%-34s buckets=%3d peak=%4d retries per 250 ms%n", modes[m],
                    buckets.size(), peak);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        backoff();
        storm();
        herd();
    }
}
```

Retry backoff, amplification and herd (seeded, deterministic) TERMINAL

```
$ java RetryDemo.java
== backoff schedule (base 100 ms, multiplier 2, 5 attempts, seed 42)
after failure 1: exponential= 100  full-jitter=  72  equal-jitter=  84 (ms)
after failure 2: exponential= 200  full-jitter=  61  equal-jitter= 127 (ms)
after failure 3: exponential= 400  full-jitter= 266  equal-jitter= 380 (ms)
after failure 4: exponential= 800  full-jitter= 295  equal-jitter= 510 (ms)

== retry amplification when the innermost dependency is down
gateway=3 order=3 inventory-client=3       -> 27 calls reach the failing dependency per user request
gateway=3 order=3 inventory-client=1       ->  9 calls reach the failing dependency per user request
gateway=1 order=1 inventory-client=3       ->  3 calls reach the failing dependency per user request
```

Retry backoff, amplification and herd (seeded, deterministic) (continued) TERMINAL

```
== 1000 clients fail at t=0 and retry; arrivals per 250 ms bucket
fixed 1000 ms, no jitter          buckets=  3 peak=1000 retries per 250 ms
exponential 1000*2^n, no jitter   buckets=  3 peak=1000 retries per 250 ms
exponential, full jitter          buckets= 27 peak= 372 retries per 250 ms
```

The second program is a count-based circuit breaker with the same parameters as the YAML above in miniature (window 10, minimum 5 calls, 50 percent, 1000 ms open, 3 probes). A simulated dependency becomes unavailable between t=500 ms and t=2000 ms (a call then hangs for 400 ms and the client times out after 100 ms). The caller issues a call every 50 ms, once without and once with the breaker.

CircuitBreakerDemo.java JAVA

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.concurrent.TimeoutException;

public class CircuitBreakerDemo {

    enum State { CLOSED, OPEN, HALF_OPEN }

    static class CallNotPermitted extends RuntimeException {
        CallNotPermitted() { super("circuit open", null, false, false); }
    }

    interface Call<T> { T run() throws Exception; }

    /** Count-based sliding window breaker, modelled on Resilience4j semantics. */
    static class Breaker {
        final int windowSize, minCalls, halfOpenPermits;
        final double failureRate;
        final long waitOpenMs;
        State state = State.CLOSED;
        final Deque<Boolean> window = new ArrayDeque<>(); // true = failure
        long openedAt;
        int halfOpenInFlight, halfOpenDone, halfOpenFailed;
        final long t0 = System.nanoTime();

        Breaker(int windowSize, int minCalls, double failureRate, long waitOpenMs,
                int halfOpenPermits) {
            this.windowSize = windowSize; this.minCalls = minCalls; this.failureRate = failureRate;
            this.waitOpenMs = waitOpenMs; this.halfOpenPermits = halfOpenPermits;
        }

        long now() { return (System.nanoTime() - t0) / 1_000_000; }

        void transition(State to, String why) {
            System.out.printf("t=%4d ms  %-9s -> %-9s  (%s)%n", now(), state, to, why);
            state = to;
            if (to == State.OPEN) openedAt = now();
            if (to == State.HALF_OPEN) {
                halfOpenInFlight = 0; halfOpenDone = 0; halfOpenFailed = 0;
            }
            if (to == State.CLOSED) window.clear();
        }

        <T> T execute(Call<T> c) throws Exception {
            if (state == State.OPEN) {
                if (now() - openedAt >= waitOpenMs) transition(State.HALF_OPEN, "wait elapsed");
                else throw new CallNotPermitted();
            }
            if (state == State.HALF_OPEN) {
                if (halfOpenInFlight >= halfOpenPermits) throw new CallNotPermitted();
                halfOpenInFlight++;
            }
            try {
                T r = c.run();
                record(false);
                return r;
            } catch (Exception e) {
                record(true);
                throw e;
            }
        }
```

Freeram

CircuitBreakerDemo.java (continued) JAVA

```
        void record(boolean failure) {
            if (state == State.HALF_OPEN) {
                halfOpenDone++;
                if (failure) halfOpenFailed++;
                if (halfOpenDone == halfOpenPermits) {
                    double rate = 100.0 * halfOpenFailed / halfOpenDone;
                    if (rate >= failureRate) transition(State.OPEN, "probe failure rate " + rate
                            + "%");
                    else transition(State.CLOSED, "probe failure rate " + rate + "%");
                }
                return;
            }
            window.addLast(failure);
            if (window.size() > windowSize) window.removeFirst();
            if (window.size() >= minCalls) {
                long f = window.stream().filter(b -> b).count();
                double rate = 100.0 * f / window.size();
                if (rate >= failureRate) transition(State.OPEN, "failure rate " + rate + "% of "
                        + window.size());
            }
        }
    }

    static final long TIMEOUT_MS = 100;
    static long outageStart = 500, outageEnd = 2000;
    static int reached = 0;

    /** Simulated remote call: healthy 20 ms; during the outage it hangs for 400 ms,
            the client times out at 100 ms. */
    static String remote(long tNowMs) throws Exception {
        reached++;
        boolean down = tNowMs >= outageStart && tNowMs < outageEnd;
        long latency = down ? 400 : 20;
        Thread.sleep(Math.min(latency, TIMEOUT_MS));
        if (latency > TIMEOUT_MS) throw new TimeoutException("no response in " + TIMEOUT_MS
                + " ms");
        return "ok";
    }

    static void run(String title, boolean useBreaker) throws Exception {
        System.out.println("== " + title);
        Breaker cb = new Breaker(10, 5, 50.0, 1000, 3);
        reached = 0;
        int ok = 0, failed = 0, rejected = 0;
        long blocked = 0;
        long start = System.nanoTime();
        for (int i = 0; i < 70; i++) {
            long t = (System.nanoTime() - start) / 1_000_000;
            long c0 = System.nanoTime();
            try {
                if (useBreaker) cb.execute(() -> remote(t)); else remote(t);
                ok++;
            } catch (CallNotPermitted e) { rejected++;
            } catch (Exception e) { failed++; }
            blocked += (System.nanoTime() - c0) / 1_000_000;
            long wait = (i + 1) * 50L - (System.nanoTime() - start) / 1_000_000;
            if (wait > 0) Thread.sleep(wait);
        }
        System.out.printf(
                "calls=70 ok=%d timeout=%d rejected=%d  reached=%d  blocked=%d ms%n%n",
                ok, failed, rejected, reached, blocked);
    }

    public static void main(String[] a) throws Exception {
        run("without circuit breaker", false);
        run("with circuit breaker (window 10, min 5, 50%, wait 1000 ms, 3 probes)", true);
    }
}
```

Sreeram

Run the circuit breaker simulation (timings measured on the build machine, typical) TERMINAL

```
$ java CircuitBreakerDemo.java
== without circuit breaker
calls=70 ok=55 timeout=15 rejected=0  reached=70  blocked=2600 ms

== with circuit breaker (window 10, min 5, 50%, wait 1000 ms, 3 probes)
t=1014 ms  CLOSED    -> OPEN       (failure rate 50.0% of 10)
t=2050 ms  OPEN      -> HALF_OPEN  (wait elapsed)
t=2172 ms  HALF_OPEN -> CLOSED     (probe failure rate 0.0%)
calls=70 ok=39 timeout=5 rejected=26  reached=44  blocked=1298 ms
```

orange segment: HALF_OPEN from 2050 ms to 2172 ms (3 probes succeed), then CLOSED again; times in ms from the run on the build machine

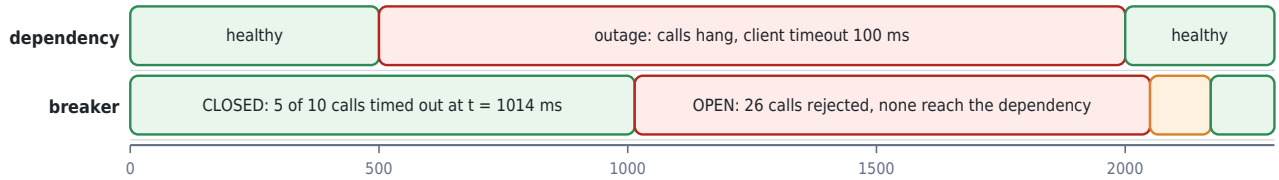


Figure 4.21. The breaker opens at about 1000 ms, after five timeouts have filled half of the window, rejects 26 calls without touching the dependency, probes at about 2050 ms and closes once three probes succeed.

Reading the result. Without the breaker the caller was blocked for 2.6 seconds in total and every call during the outage waited for its full timeout. With the breaker the dependency received 44 instead of 70 calls, the caller blocked for 1.3 seconds, and 26 calls were answered in microseconds with a rejection that the application can turn into a fallback. The five timeouts before the opening are the price of detection; the window and the minimum call count control that price. The breaker did not return to CLOSED at the end of the outage (t=2000) but at 2172 ms, because the open period of one second began at 1014 ms and lasted until 2050 ms.

The third program shows the effect of a bulkhead on an endpoint that does not use the failing dependency.

BulkheadDemo.java JAVA

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class BulkheadDemo {

    static final int WORKERS = 10;        // stands in for the Tomcat worker pool
    static final long SLOW_MS = 800;      // latency of the degraded dependency
    static final Semaphore BULKHEAD = new Semaphore(4);

    static void scenario(String title, boolean bulkhead) throws Exception {
        ExecutorService workers = Executors.newFixedThreadPool(WORKERS);
        AtomicInteger rejected = new AtomicInteger();
        List<Future<Long>> fast = new ArrayList<>();
        // 30 requests to /orders (needs the slow dependency) arrive first
        for (int i = 0; i < 30; i++) {
            workers.submit(() -> {
                if (bulkhead && !BULKHEAD.tryAcquire()) { rejected.incrementAndGet(); return; }
                try { Thread.sleep(SLOW_MS); } catch (
                        InterruptedException e) { Thread.currentThread().interrupt(); }
                finally { if (bulkhead) BULKHEAD.release(); }
            });
        }
        Thread.sleep(50);
        // then 10 requests to /health-like endpoint that need no dependency (1 ms of work)
        for (int i = 0; i < 10; i++) {
            long enq = System.nanoTime();
            fast.add(workers.submit(() -> (System.nanoTime() - enq) / 1_000_000));
        }
        long max = 0, sum = 0;
        for (Future<Long> f : fast) { long v = f.get(); max = Math.max(max, v); sum += v; }
        System.out.printf(
                "%-24s fast endpoint: avg wait=%4d ms  max wait=%4d ms  slow calls rejected=%d%n",
                title, sum / fast.size(), max, rejected.get());
        workers.shutdownNow();
    }

    public static void main(String[] a) throws Exception {
        System.out.println("worker pool = " + WORKERS + " threads, dependency latency = "
```

BulkheadDemo.java (continued) JAVA

```
                + SLOW_MS + " ms");
        scenario("no bulkhead", false);
        scenario("bulkhead (4 concurrent)", true);
    }
}
```

Run the bulkhead simulation TERMINAL

```
$ java BulkheadDemo.java
worker pool = 10 threads, dependency latency = 800 ms
no bulkhead              fast endpoint: avg wait=2346 ms  max wait=2347 ms  slow calls rejected=0
bulkhead (4 concurrent)  fast endpoint: avg wait=   0 ms  max wait=   0 ms  slow calls rejected=26
```

Ten worker threads serve 30 slow requests (800 ms dependency latency) followed by ten fast requests. Without isolation the fast requests wait 2.3 seconds in the queue behind the slow ones. With a bulkhead of four concurrent calls, 26 slow requests are refused instantly, the workers are free, and the fast requests are served with no wait.

4.4.4 Real-world problems and solutions

Problem 1: a slow dependency takes down unrelated endpoints (cascading failure)

Scenario. The inventory service has a slow database query. Its p99 rises from 80 ms to 6 s. The order service and the gateway have no breaker or bulkhead.

Symptoms. Within minutes the order service times out on every endpoint, including those that never call inventory; the gateway returns 504; liveness probes of the order service fail and pods restart in a loop, which makes recovery slower.

Diagnosis. (1) `http.server.requests` (the p99 or the `max` per `uri`) shows the latency rise first on the inventory service, then upstream. (2) The metrics `tomcat.threads.busy` against `tomcat.threads.config.max`, and `hikaricp.connections.active` against the pool maximum, show saturation. (3) `jcmd <pid> Thread.print` lists most `http-nio-*-exec-*` threads in `InventoryGateway.reserve`. (4) `http.client.requests` with `outcome=SERVER_ERROR` or `CLIENT_ERROR` for the inventory `uri` and a rising `max`. (5) Gateway metric `spring.cloud.gateway.requests` with `status=504`.

Root cause. The shared worker pool was the only isolation between endpoints, and every call to the slow dependency was allowed to wait for the full client timeout.

Fix. Bulkhead and breaker on the inventory call as in the worked example, a slow-call threshold, tighter timeouts derived from the dependency's normal p99, and a fallback or fail-fast response. Move liveness to a separate management port or a light-weight check that does not use the worker pool.

Prevention. A load test that injects latency into each dependency (a proxy such as Toxiproxy, or a stub with a configurable delay) and asserts that endpoints without that dependency keep their latency.

Problem 2: a retry storm prolongs an outage

Scenario. The inventory service reaches 90 percent CPU. The order service retries three times, the gateway retries twice, and the mobile client retries three times.

Symptoms. Inventory request rate rises by an order of magnitude after the first error spike although the number of users is constant; the service does not recover when the original cause (a slow query) disappears; logs show many near-identical requests per order identifier.

Diagnosis. Compare the inbound request rate of the inventory service with the rate of distinct user requests at the gateway; the ratio is the amplification factor (27 for three layers of three). Count requests per `X-Request-Id` in the inventory access log to see the multiplication directly. The retry metrics (`resilience4j_retry_calls_total` with the `kind` tag values `successful_with_retry` and `failed_with_retry`) quantify the retry share at each layer.

Root cause. Each layer retries independently, none has a budget, and the retries are not jittered, so they arrive in synchronised waves.

Fix. Retry at one layer; limit attempts to two or three with exponential backoff and jitter; make the circuit breaker open on the failure rate so that retries stop; return `429` or `503` with `Retry-After` from the overloaded service so that well-behaved clients wait.

Prevention. Document which layer owns retries in the architecture decision record, and review any new `@Retry` or gateway `Retry` filter against it.

Problem 3: the dependency fails again immediately after recovery (thundering herd)

Scenario. After a five-minute outage of the catalogue database, the catalogue service restarts. Thousands of waiting clients and cache-less workers send requests at the same instant.

Symptoms. The service becomes healthy, receives a vertical spike of requests, becomes unhealthy again within seconds, and the pattern repeats (flapping). The breaker alternates between half-open and open.

Diagnosis. A request-rate graph with one-second resolution shows the spike at the moment of recovery; the breaker transition log shows `HALF_OPEN to OPEN` repeated; connection counts on the database reach the pool maximum within one second.

Root cause. Synchronised behaviour: fixed retry intervals, caches that expire at the same time, and breakers in all instances that move to half-open at the same wall-clock moment.

Fix. Jitter on every retry delay (see the simulation: peak 1000 retries per 250 ms bucket without jitter, 372 with full jitter), randomised cache expiry, a small number of probe calls in half-open (`permittedNumberOfCallsInHalfOpenState`), request coalescing for identical cache misses, and a warm-up ramp (slow start) behind the load balancer.

Prevention. Include recovery in failure-injection tests: stop the dependency, restore it and observe the request rate, not only the outage.

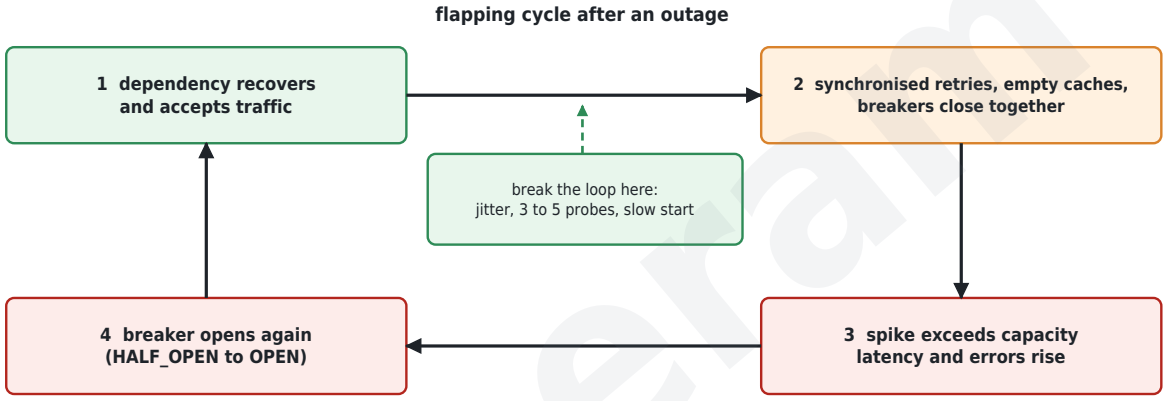


Figure 4.22. The flapping cycle after an outage: recovery, synchronised retries, overload, breaker opens, and the same cycle again; jitter and small probe counts break the loop at the second step.

Problem 4: the circuit breaker never opens, or never closes

Scenario A: never opens. The inventory service is down and the order service logs a timeout for every call, yet `/actuator/circuitbreakers` shows `CLOSED` and a failure rate of `-1.0%`.

Diagnosis. `GET /actuator/circuitbreakers` shows `bufferedCalls` below `minimumNumberOfCalls` (the failure rate is reported as `-1.0%` until enough calls have been recorded). Further causes are checked in this order: (1) `minimumNumberOfCalls` left at the default 100 with a low call rate; (2) the thrown exception type is not in `recordExceptions` or is in `ignoreExceptions`, so failures are classified as neutral; (3) a `try`/`catch` or a fallback inside the decorated method returns a default value, so the breaker records a success; (4) the method is invoked from the same class (self-invocation), so no proxy intercepts it (see the chapter on AOP and proxies); (5) the annotation is on a private method or a class that is not a Spring bean.

Fix. Set `minimumNumberOfCalls` to a value reachable within the window at the real traffic (10 or fewer for low-volume dependencies), list the exceptions explicitly, throw from the protected method and place the fallback on the decorator annotation (`fallbackMethod`), not inside the method body.

Scenario B: never closes. The breaker stays OPEN although the dependency recovered long ago.

Diagnosis. `waitDurationInOpenState` is long and `automaticTransitionFromOpenToHalfOpenEnabled` is false, so the breaker moves to half-open only when a call arrives, and no call arrives because upstream also protects itself; or the probes fail because they hit a still-cold instance (`permittedNumberOfCallsInHalfOpenState` of one, with a failure rate threshold of 50 percent, reopens on a single failure).

Fix. Enable the automatic transition, shorten the wait (10 to 30 seconds), and use several probes (3 to 5).

Prevention. Alert on `resilience4j_circuitbreaker_state` (the gauge has one series per state) and on the transition log line, and test the breaker in an integration test with a stub that fails and then recovers.

4.4.5 Questions to expect

1. **Why is a slow dependency worse than a failing one?** A failure releases the thread at once, a slow call holds a thread, a connection and often a transaction for its whole duration; by Little's law, concurrency equals rate times latency, so a latency increase exhausts pools.
2. **Explain the circuit breaker states.** Closed: calls pass and outcomes are recorded in a sliding window. Open: calls are rejected immediately after the failure or slow-call rate exceeded the threshold. Half-open: after the wait period a limited number of probe calls decide between closing and reopening.
3. **What is the difference between a circuit breaker and a retry?** A retry repeats a call to survive a transient failure and adds load; a breaker stops calls to a failing dependency and removes load. They are combined so that retry attempts feed the breaker and an open breaker stops further attempts.
4. **Why add jitter to exponential backoff?** Without it, clients that failed together retry together, producing synchronised waves; randomisation spreads the retries over time.
5. **What is a retry storm and how is it prevented?** Multiplied retries across layers (3 by 3 by 3 gives 27 calls) overload a recovering dependency. Prevent by retrying at one layer, limiting attempts, using backoff and jitter, retry budgets, and a breaker.
6. **Semaphore bulkhead versus thread-pool bulkhead?** The semaphore limits concurrent calls on the caller's thread and is cheap; the thread-pool variant runs calls on a dedicated pool with a queue, which isolates the caller's threads fully but needs asynchronous calls and a hand-off.
7. **How should a fallback be designed?** According to the business meaning of the call: cached or default data for reads that tolerate staleness; an explicit failure for operations whose result cannot be assumed; never a fallback that converts an error into a silent success.
8. **What is the default Resilience4j aspect order?** Retry, CircuitBreaker, RateLimiter, TimeLimiter, Bulkhead from outermost to innermost; adjustable with the `*AspectOrder` properties.
9. **Which failures should not open a breaker?** Client errors (4xx) and business rejections; they show that the dependency works. Timeouts, connection errors, 5xx responses and slow calls are the relevant signals.
10. **What is load shedding?** A server rejecting excess work early (HTTP 503 with `Retry-After`) when queues exceed a bound, so that the accepted requests still complete within their deadlines.

Check yourself

1. A service handles 50 requests per second. A dependency latency rises from 40 ms to 4 s. How many requests are in flight on average, and what happens to a pool of 200 threads?
2. A breaker has `minimumNumberOfCalls: 100` and the service makes 5 calls per minute to the dependency, which is down. What does the breaker report?
3. A `@CircuitBreaker` method catches `Exception` and returns `null`. What does the breaker record?
4. Three layers retry three times each with no jitter. The dependency handles 1000 calls per second at full capacity and the user load is 100 requests per second. What load does it receive during an outage?

Answers

1. `50 × 4 = 200` in flight, so the pool of 200 is at its limit and any burst exhausts it; before the incident it was `50 × 0.04 = 2`.
2. It stays CLOSED and reports a failure rate of `-1.0%` until 100 calls have been recorded in the window, which takes about 20 minutes.
3. Successful calls: the exception does not leave the method, so the failures are invisible to the breaker.
4. 100 × 27 = 2700 calls per second, 2.7 times its capacity, and synchronised.

KEY POINTS

Resilience limits how far a failure travels: every dependency call needs a bound, a rule for repetition and a way to stop.

- Slowness cascades through thread pools (concurrency equals rate times latency); timeouts, bulkheads and breakers cut the chain at different points.
- Retry only transient failures on idempotent operations, at one layer, with exponential backoff and jitter; layered retries multiply (3 x 3 x 3 = 27).
- A breaker needs sensible `minimumNumberOfCalls`, a slow-call threshold, an explicit list of recorded exceptions and a fallback outside the protected method.
- Fallbacks must reflect business meaning: stale data for reads, explicit failure for operations that cannot be assumed.
- Recovery is a failure mode of its own: jitter, few probes and warm-up prevent the thundering herd.

Stefan

4.5 Distributed Data and Transactions

With a database per service, one business operation touches several databases, and no transaction manager spans them. Placing an order must reserve stock in the inventory database, capture a payment in the payment database and create a shipment in the shipping database; any of these steps may fail after the others have succeeded. This chapter explains why two-phase commit is not used for this, how a saga replaces the atomic transaction with a sequence of local transactions and compensations, how the outbox pattern makes "change the database and publish an event" reliable, and how consumers tolerate the duplicates that reliable delivery implies.

4.5.1 Concept and mechanism

Two-phase commit and its limits

The classical answer to a distributed transaction is the **two-phase commit** protocol (2PC), exposed in Java as XA and JTA. A coordinator asks every participant to prepare; each participant makes its changes durable, holds its locks and votes; if all vote yes the coordinator tells all to commit, otherwise to roll back. Spring supports it through JTA transaction managers (Narayana, Atomikos), but it is rarely used between microservices, for structural reasons:

- **Blocking.** After voting yes, a participant cannot release its locks until the coordinator decides. If the coordinator fails between the phases, participants hold locks for an unbounded time.
- **Availability multiplies.** The transaction succeeds only if every participant and the coordinator are available, the serial-availability arithmetic of the first chapter applied to writes.
- **Support.** REST over HTTP and message brokers such as Kafka do not participate in XA. The services would have to share a transaction protocol and a resource-manager technology, which removes the independence that motivated the split.
- **Latency and lock duration** grow with the slowest participant and the number of round trips.

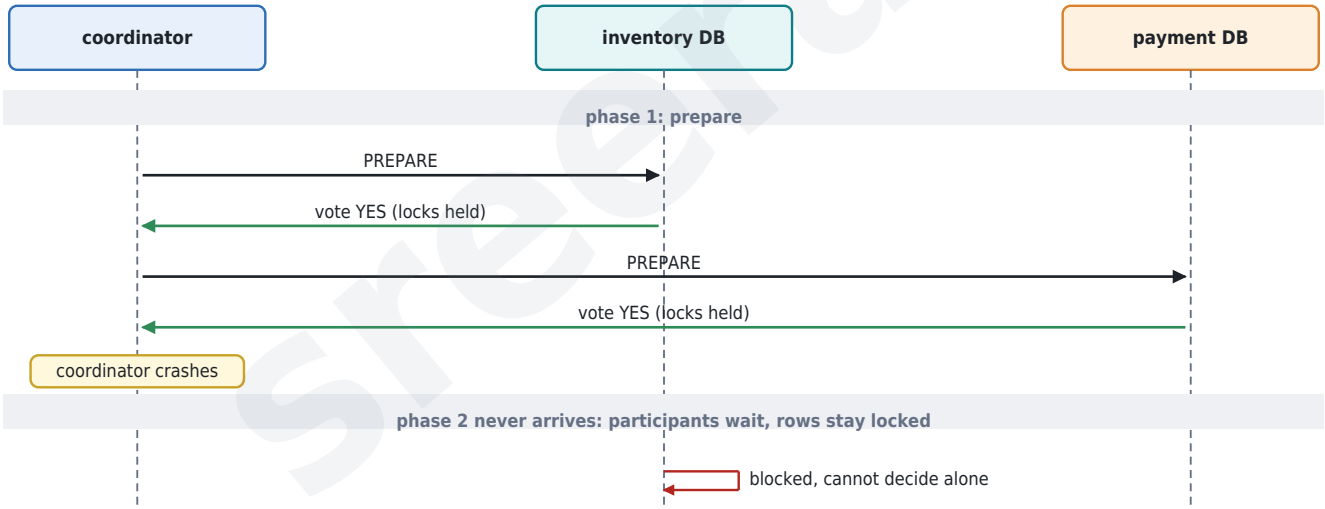


Figure 4.23. If the coordinator fails after both participants have voted yes, both hold their row locks until it returns; no participant may decide alone.

Eventual consistency and the saga pattern

Without a global transaction, consistency between services is **eventual**: the data may be temporarily inconsistent, and the system converges to a consistent state. A **saga** structures this convergence. It is a sequence of local transactions, one per service; each commits independently and triggers the next, and for each step that has an externally visible effect a **compensating transaction** exists that semantically undoes it. If step `k` fails, the compensations for steps `k-1` down to `1` run in reverse order.

A compensation is a business operation, not a database rollback: a captured payment is refunded (and the refund appears in the ledger), reserved stock is released, an order is marked cancelled. Three properties are required. Compensations must be **idempotent**, because they are retried. They must be **retryable until they succeed**, because a saga that cannot undo a step is stuck. And some steps cannot be undone at all (an e-mail has been sent), so such steps are placed last, after the **pivot transaction**, the step after which the saga is committed to completing; steps before it are compensable, steps after it are retried forward until they succeed.

Two coordination styles exist.

- **Orchestration.** A central orchestrator (in the example, the order service) holds the saga state, sends a command to each participant, receives the reply and decides the next step or the compensations. The flow is explicit in one place and easy to monitor; the orchestrator is a component to build and a risk of accumulating logic.
- **Choreography.** There is no coordinator. Each service reacts to events and publishes its own: the order service publishes `OrderPlaced`, inventory reacts and publishes `StockReserved`, payment reacts and publishes `PaymentCaptured`, and so on. Failures are events as well (`PaymentFailed`) to which earlier participants react by compensating. Coupling is low and no single point exists, but the overall flow exists only implicitly in the subscriptions and becomes hard to follow beyond three or four services; cyclic dependencies appear easily.

Aspect	Orchestration	Choreography
Flow visibility	one place (state machine)	spread across subscriptions
Coupling	participants know the orchestrator's commands	participants know each other's events
Adding a step	change the orchestrator	add a subscriber, possibly change several services
Failure handling	explicit compensation logic in one place	each service reacts to failure events
Risk	orchestrator becomes a god service	implicit cycles, hard-to-trace flows
Suitable for	long flows with branching and many compensations	short flows with few participants

Orchestration compared with choreography.

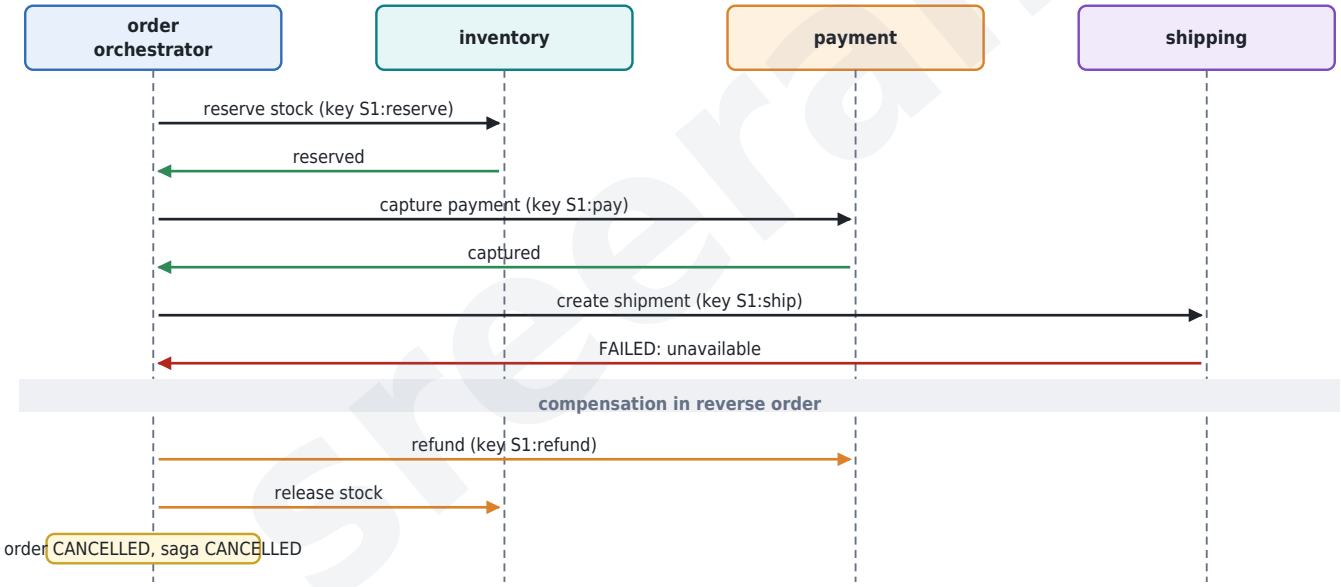


Figure 4.24. The orchestrated saga: each step is a local transaction in one service; when shipping fails, the orchestrator issues the compensations in reverse order and cancels the order.

sreeram

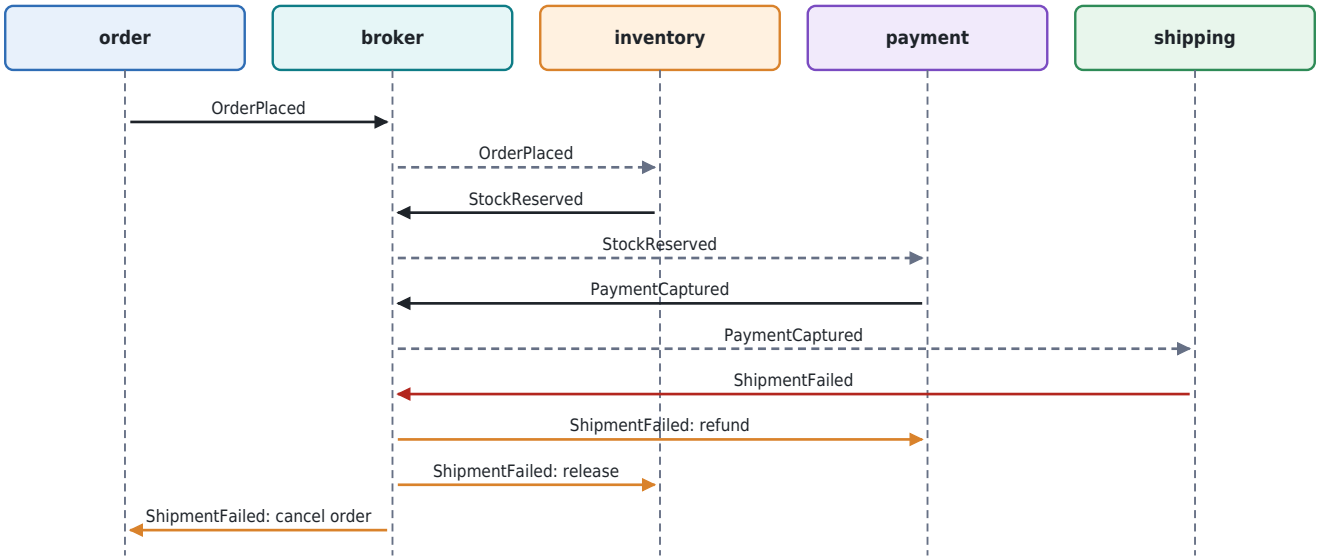


Figure 4.25. The same checkout as choreography: services communicate only through events on the broker, and a failure is published as an event to which earlier participants react by compensating.

Isolation anomalies in sagas

A saga lacks the isolation of an ACID transaction: the intermediate states of steps are visible to other transactions. Two concurrent sagas may read each other's partial results, and a compensation may overwrite a change made in between. The countermeasures are semantic: a **semantic lock** (an order in state `PENDING` is not shipped or modified), **commutative updates** (decrement stock rather than set it), **pessimistic ordering** (take the hardest-to-reverse step last), **versioned reads** and a reread-before-write check. The design question is always which states are visible, and which operations are legal in each of them.

The dual-write problem and the outbox pattern

A service that updates its database and then publishes an event to a broker performs two writes to two systems, and no transaction covers both. A crash between them loses the event (database committed, event never sent) or, if the order is reversed, publishes an event for a change that was rolled back. Retrying the publish does not help when the process itself has died.

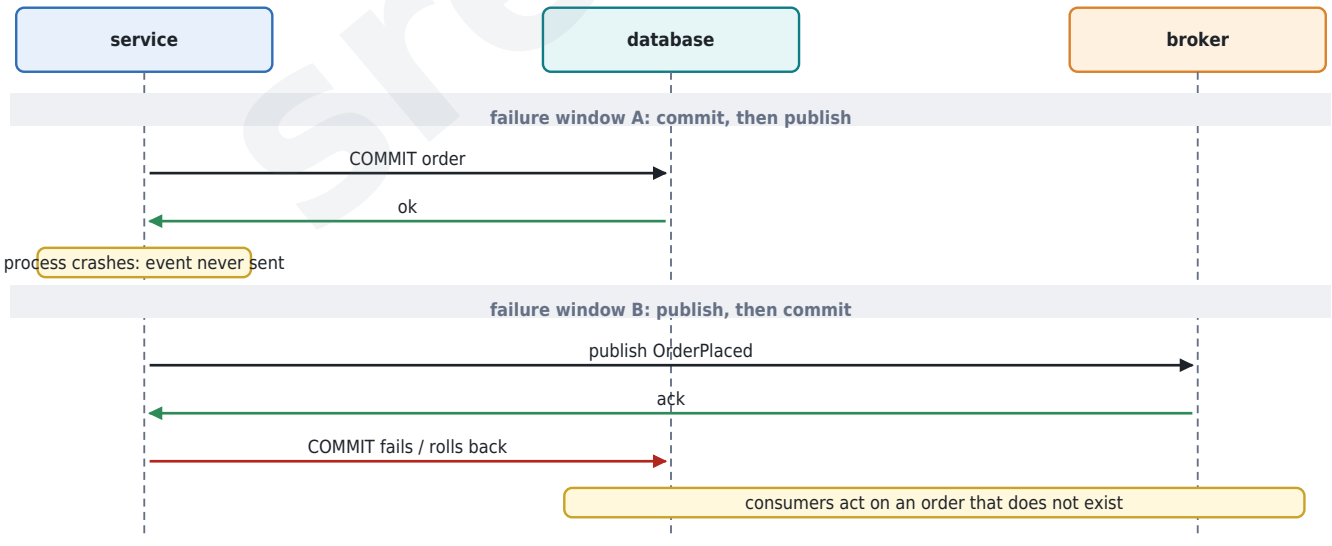


Figure 4.26. The two failure windows of a dual write: a crash after the commit loses the event, and a publish before the commit announces a change that may never exist.

The **transactional outbox** removes the second write from the request path. The service inserts the event into an `outbox` table in the same local transaction as the business change; either both rows commit or neither does. A separate **relay** reads unpublished outbox rows and publishes them to the broker, then marks them published. The relay is either a poller (a scheduled job issuing `SELECT ... FOR UPDATE SKIP LOCKED`, which lets several relay instances work on

disjoint rows) or a change-data-capture tool (Debezium reading the database log), which avoids polling latency and load.

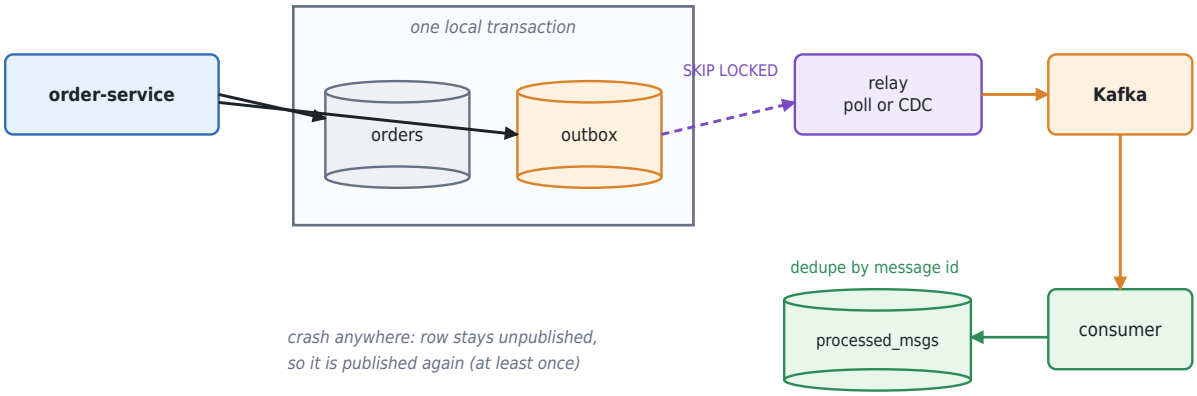


Figure 4.27. The business row and the outbox row commit together; the relay publishes afterwards and marks the row, so a crash at any point leads to a redelivery, never to a loss.

The relay can crash after publishing and before marking the row, so the event is published twice: the outbox guarantees **at-least-once** delivery. The consumer side must therefore be an **idempotent consumer**: it records the identifier of each processed message in a table with a unique constraint, in the same transaction as the processing, and ignores a message whose identifier is already present. Operations that are naturally idempotent (setting a state, an upsert keyed on a business identifier) need no extra table. Ordering is preserved per aggregate by using the aggregate identifier as the message key, so that all events of one order go to one partition (see the chapter on Kafka in the last Part).

CQRS and event sourcing

CQRS (command query responsibility segregation) separates the model that accepts changes from the model that serves queries. The write side enforces invariants; the read side keeps denormalised views, tailored to screens, built by consuming the events of the write side. It answers the cross-service join problem: the "order history with product names" view is a table in the order service fed by catalog events, and no remote call is made at query time. The price is eventual consistency of the view and the need to rebuild it when its shape changes.

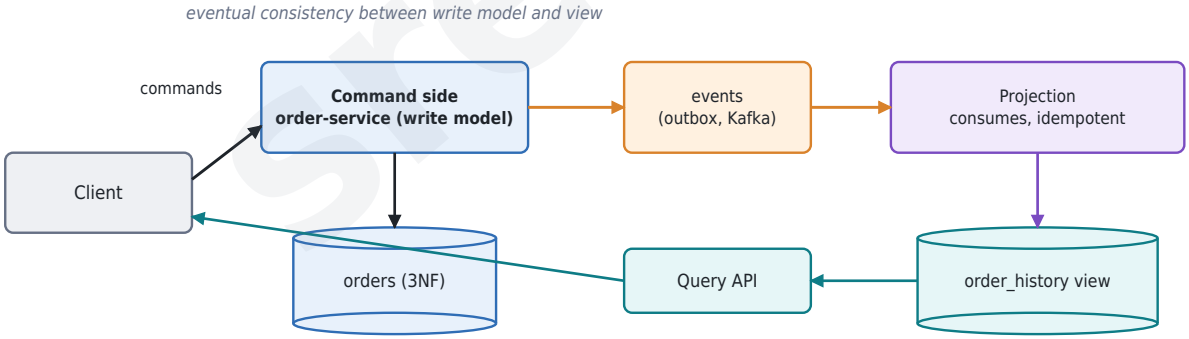


Figure 4.28. The write model emits events through the outbox; one or more projections consume them and maintain read models, which are queried without any cross-service call.

Event sourcing stores the state of an aggregate as the sequence of events that produced it, rather than as the current row. The current state is derived by replaying events (with snapshots for long histories). It gives a complete audit trail and the ability to build new projections from history, and it requires event schema versioning, replay tooling and a different way of thinking about queries. It is justified for domains where history is the product (ledgers, bookings); for most services the outbox and ordinary tables are sufficient, and event sourcing is a deliberate commitment rather than a default.

4.5.2 Worked example

The order service orchestrates the checkout saga. Its state is persisted so that the saga survives a restart: every step is recorded before the next begins, and a recovery job resumes unfinished sagas. All remote calls carry an idempotency key derived from the saga identifier and the step name, so that a repeated step does not repeat its effect (the server side was shown in the chapter on synchronous communication).

SagaState.java JAVA

```java
package com.shop.order.saga;

public enum SagaState {
    STARTED, STOCK_RESERVED, PAYMENT_CAPTURED, SHIPMENT_CREATED, COMPLETED,
    COMPENSATING, CANCELLED;

    public boolean atLeast(SagaState other) {
        return ordinal() >= other.ordinal();
    }
}
```

CheckoutSaga.java JAVA

```java
package com.shop.order.saga;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Version;
import java.time.Instant;
import java.util.UUID;

@Entity
@Table(name = "checkout_saga")
public class CheckoutSaga {

    @Id
    private UUID id = UUID.randomUUID();

    @Column(nullable = false)
    private UUID orderId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private SagaState state = SagaState.STARTED;

    /** Highest forward step that completed; compensation starts here. */
    @Enumerated(EnumType.STRING)
    @Column(length = 20)
    private SagaState reached = SagaState.STARTED;

    private String failureReason;

    @Column(nullable = false)
    private Instant updatedAt = Instant.now();

    @Version
    private long version;

    protected CheckoutSaga() { }

    public CheckoutSaga(UUID orderId) {
        this.orderId = orderId;
    }

    public UUID getId() { return id; }
    public UUID getOrderId() { return orderId; }
    public SagaState getState() { return state; }
    public SagaState getReached() { return reached; }

    public void advance(SagaState next) {
        this.state = next;
        this.reached = next;
        this.updatedAt = Instant.now();
```

Sreeram

CheckoutSaga.java (continued) JAVA

```
    }

    public void startCompensation(String reason) {
        this.state = SagaState.COMPENSATING;
        this.failureReason = reason;
        this.updatedAt = Instant.now();
    }

    public void cancelled() {
        this.state = SagaState.CANCELLED;
        this.updatedAt = Instant.now();
    }
}
```

CheckoutOrchestrator.java JAVA

```
package com.shop.order.saga;

import com.shop.order.domain.Order;
import com.shop.order.domain.OrderRepository;
import com.shop.order.infrastructure.InventoryClient;
import com.shop.order.infrastructure.PaymentClient;
import com.shop.order.infrastructure.ShippingClient;
import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.OptimisticLockingFailureException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.support.TransactionTemplate;

@Service
public class CheckoutOrchestrator {

    private static final Logger log = LoggerFactory.getLogger(CheckoutOrchestrator.class);

    private final CheckoutSagaRepository sagas;
    private final OrderRepository orders;
    private final InventoryClient inventory;
    private final PaymentClient payments;
    private final ShippingClient shipping;
    private final TransactionTemplate tx;

    public CheckoutOrchestrator(CheckoutSagaRepository sagas, OrderRepository orders,
                                InventoryClient inventory, PaymentClient payments,
                                ShippingClient shipping, TransactionTemplate tx) {
        this.sagas = sagas;
        this.orders = orders;
        this.inventory = inventory;
        this.payments = payments;
        this.shipping = shipping;
        this.tx = tx;
    }

    /** Runs or resumes a saga. Each step is persisted before the next one starts. */
    public void run(UUID sagaId) {
        CheckoutSaga saga = sagas.findById(sagaId).orElseThrow();
        Order order = orders.findWithLinesById(saga.getOrderId()).orElseThrow();
        try {
            if (saga.getState() == SagaState.STARTED) {
                inventory.reserve(key(saga, "reserve"), InventoryClient.requestFor(order));
                saga = persist(saga, SagaState.STOCK_RESERVED);
            }
            if (saga.getState() == SagaState.STOCK_RESERVED) {
                payments.capture(key(saga, "pay"), PaymentClient.requestFor(order));
                saga = persist(saga, SagaState.PAYMENT_CAPTURED);
            }
            if (saga.getState() == SagaState.PAYMENT_CAPTURED) {
                shipping.create(key(saga, "ship"), ShippingClient.requestFor(order));
                saga = persist(saga, SagaState.SHIPMENT_CREATED);
            }
            if (saga.getState() == SagaState.SHIPMENT_CREATED) {
                final CheckoutSaga done = saga;
                tx.executeWithoutResult(s -> {
                    orders.findById(done.getOrderId()).orElseThrow().confirm();
```

Sreeram

CheckoutOrchestrator.java (continued) JAVA

```
                    CheckoutSaga fresh = sagas.findById(done.getId()).orElseThrow();
                    fresh.advance(SagaState.COMPLETED);
                });
            }
        } catch (OptimisticLockingFailureException e) {
            log.info("saga {} is being advanced by another instance", sagaId);
        } catch (RuntimeException e) {
            log.warn("saga {} failed in state {}: {}", sagaId, saga.getState(), e.toString());
            compensate(saga, e.getMessage());
        }
    }

    private void compensate(CheckoutSaga current, String reason) {
        final UUID sagaId = current.getId();
        final UUID orderId = current.getOrderId();
        CheckoutSaga saga = current;
        if (saga.getState() != SagaState.COMPENSATING) {
            saga = tx.execute(s -> {
                CheckoutSaga fresh = sagas.findById(sagaId).orElseThrow();
                fresh.startCompensation(reason);
                return fresh;
            });
        }
        // reverse order; every compensation is idempotent and is retried by the recovery job
        if (saga.getReached().atLeast(SagaState.PAYMENT_CAPTURED)) {
            payments.refund(key(saga, "refund"), orderId);
        }
        if (saga.getReached().atLeast(SagaState.STOCK_RESERVED)) {
            inventory.release(orderId);
        }
        tx.executeWithoutResult(s -> {
            orders.findById(orderId).orElseThrow().cancel();
            sagas.findById(sagaId).orElseThrow().cancelled();
        });
    }

    private CheckoutSaga persist(CheckoutSaga saga, SagaState next) {
        final UUID id = saga.getId();
        return tx.execute(s -> {
            CheckoutSaga fresh = sagas.findById(id).orElseThrow();
            fresh.advance(next);
            return fresh;
        });
    }

    private static String key(CheckoutSaga saga, String step) {
        return saga.getId() + ":" + step;
    }
}
```

The clients that the orchestrator uses follow the pattern of the inventory client. Each takes an idempotency key, and each offers a static helper that builds its request from the order; the helper is a plain static method of the interface and is not intercepted by the proxy. The repository method `findWithLinesById` is declared with an entity graph so that the order lines are loaded without an open transaction.

PaymentClient.java JAVA

```
package com.shop.order.infrastructure;

import com.shop.order.domain.Order;
import java.math.BigDecimal;
import java.util.UUID;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.service.annotation.HttpExchange;
import org.springframework.web.service.annotation.PostExchange;

@HttpExchange("/api/v1/payments")
public interface PaymentClient {

    record CaptureRequest(UUID orderId, BigDecimal amount, String currency) { }

    @PostExchange
```

sreeram

PaymentClient.java (continued) JAVA

```
    void capture(@RequestHeader("Idempotency-Key") String key, @RequestBody CaptureRequest body);

    @PostExchange("/{orderId}/refund")
    void refund(@RequestHeader("Idempotency-Key") String key,
                @PathVariable("orderId") UUID orderId);

    static CaptureRequest requestFor(Order order) {
        return new CaptureRequest(order.getId(), order.getTotal(), "EUR");
    }
}
```

OrderRepository.java JAVA

```
package com.shop.order.domain;

import java.util.Optional;
import java.util.UUID;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, UUID> {

    @EntityGraph(attributePaths = "lines")
    Optional<Order> findWithLinesById(UUID id);
}
```

The `reached` field is what makes compensation exact: if payment capture failed, the saga reached `STOCK_RESERVED`, so only the stock is released; if shipping failed, it reached `PAYMENT_CAPTURED`, so the payment is refunded first and the stock is released second. The idempotency keys make a resumed saga safe: if the process crashes after `payments.capture` succeeded but before the state was saved, the recovery job runs the capture again with the same key, and the payment service replays the stored response.

A recovery job completes the picture. It selects sagas that have been in a non-terminal state for longer than their expected duration and re-runs them:

SagaRecoveryJob.java JAVA

```
package com.shop.order.saga;

import java.time.Duration;
import java.time.Instant;
import java.util.List;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class SagaRecoveryJob {

    private final CheckoutSagaRepository sagas;
    private final CheckoutOrchestrator orchestrator;

    public SagaRecoveryJob(CheckoutSagaRepository sagas, CheckoutOrchestrator orchestrator) {
        this.sagas = sagas;
        this.orchestrator = orchestrator;
    }

    @Scheduled(fixedDelayString = "PT30S")
    public void resumeStalled() {
        Instant cutoff = Instant.now().minus(Duration.ofMinutes(2));
        List<CheckoutSaga> stalled = sagas.findStalled(cutoff);
        for (CheckoutSaga saga : stalled) {
            orchestrator.run(saga.getId());
        }
    }
}
```

CheckoutSagaRepository.java JAVA

```
package com.shop.order.saga;

import java.time.Instant;
import java.util.List;
import java.util.UUID;
```

CheckoutSagaRepository.java (continued) JAVA

```java
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface CheckoutSagaRepository extends JpaRepository<CheckoutSaga, UUID> {

    @Query("""
        select s from CheckoutSaga s
        where s.state not in (com.shop.order.saga.SagaState.COMPLETED,
                              com.shop.order.saga.SagaState.CANCELLED)
          and s.updatedAt < :cutoff
        """)
    List<CheckoutSaga> findStalled(@Param("cutoff") Instant cutoff);
}
```

When several order-service instances run, two instances may resume the same saga at once. The `@Version` field on `CheckoutSaga` makes the second state update fail with an optimistic-lock exception, and the idempotent remote calls make the duplicated step harmless. A lock on the saga row (`PESSIMISTIC_WRITE` with `SKIP LOCKED`) avoids the duplicate work.

The outbox in Spring

The outbox writer runs inside the business transaction. It uses `JdbcClient` (Spring Framework 6.1, auto-configured in Boot 3.2 and later), which participates in the current transaction because it uses the same `DataSource` as JPA.

OutboxWriter.java JAVA

```java
package com.shop.order.outbox;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Component
public class OutboxWriter {

    private final JdbcClient jdbc;
    private final ObjectMapper json;

    public OutboxWriter(JdbcClient jdbc, ObjectMapper json) {
        this.jdbc = jdbc;
        this.json = json;
    }

    /** MANDATORY: fails fast if called without a surrounding business transaction. */
    @Transactional(propagation = Propagation.MANDATORY)
    public void write(String aggregateType, String aggregateId, String eventType, Object event) {
        try {
            jdbc.sql("""
                INSERT INTO outbox (aggregate_type, aggregate_id, event_type, payload)
                VALUES (:type, :id, :event, CAST(:payload AS jsonb))
                """)
                .param("type", aggregateType)
                .param("id", aggregateId)
                .param("event", eventType)
                .param("payload", json.writeValueAsString(event))
                .update();
        } catch (JsonProcessingException e) {
            throw new IllegalArgumentException("cannot serialise " + eventType, e);
        }
    }
}
```

The use in the order service is one added line inside the transaction that creates the order:

OrderService.java (fragment, replaces the inline stock call) JAVA

```java
@Transactional
public Order place(UUID customerId, List<Item> items) {
    Order order = new Order(customerId);
    for (Item item : items) {
        order.addLine(item.sku(), item.quantity(), prices.priceOf(item.sku()));
    }
    orders.save(order);
    sagas.save(new CheckoutSaga(order.getId()));
    outbox.write("Order", order.getId().toString(), "OrderPlaced",
        new OrderPlaced(order.getId(), customerId, order.getTotal()));
    return order;
}
```

The relay is a scheduled component calling a transactional batch method on a separate bean (a call from a method of the same class would bypass the transactional proxy; see the chapter on AOP and proxies).

OutboxBatchPublisher.java JAVA

```java
package com.shop.order.outbox;

import java.nio.charset.StandardCharsets;
import java.util.List;
import java.util.concurrent.TimeUnit;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OutboxBatchPublisher {

    private static final Logger log = LoggerFactory.getLogger(OutboxBatchPublisher.class);

    record Row(long id, String aggregateType, String aggregateId, String eventType,
               String payload) { }

    private final JdbcClient jdbc;
    private final KafkaTemplate<String, String> kafka;

    public OutboxBatchPublisher(JdbcClient jdbc, KafkaTemplate<String, String> kafka) {
        this.jdbc = jdbc;
        this.kafka = kafka;
    }

    @Transactional
    public int publishPending() {
        List<Row> rows = jdbc.sql("""
            SELECT id, aggregate_type, aggregate_id, event_type, payload::text AS payload
            FROM outbox
            WHERE published_at IS NULL
            ORDER BY id
            LIMIT 100
            FOR UPDATE SKIP LOCKED
            """)
            .query((rs, n) -> new Row(rs.getLong("id"), rs.getString("aggregate_type"),
                rs.getString("aggregate_id"), rs.getString("event_type"), rs.getString("payload")))
            .list();
        int published = 0;
        for (Row row : rows) {
            ProducerRecord<String, String> record = new ProducerRecord<>(
                "order-events", row.aggregateId(), row.payload());
            record.headers().add("message-id",
                Long.toString(row.id()).getBytes(StandardCharsets.UTF_8));
            record.headers().add("event-type", row.eventType().getBytes(StandardCharsets.UTF_8));
            try {
                kafka.send(record).get(5, TimeUnit.SECONDS);
            } catch (Exception e) {
                log.warn("outbox row {} not published: {}", row.id(), e.toString());
                break;                          // keep per-aggregate order: stop, retry on the next run
            }
            jdbc.sql("UPDATE outbox SET published_at = now() WHERE id = :id")
```

OutboxBatchPublisher.java (continued) JAVA

```
                .param("id", row.id()).update();
            published++;
        }
        return published;
    }
}
```

OutboxRelay.java JAVA

```
package com.shop.order.outbox;

import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
public class OutboxRelay {

    private final OutboxBatchPublisher publisher;

    public OutboxRelay(OutboxBatchPublisher publisher) {
        this.publisher = publisher;
    }

    @Scheduled(fixedDelayString = "PT0.5S")
    public void run() {
        while (publisher.publishPending() > 0) {
            // drain the backlog before sleeping again
        }
    }
}
```

The consumer in another service deduplicates on the message identifier and the consumer name. The identifier check and the processing share one transaction, so a crash between them leaves no trace of either:

PaymentEventsListener.java JAVA

```
package com.shop.shipping;

import java.nio.charset.StandardCharsets;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

@Component
public class PaymentEventsListener {

    private final JdbcClient jdbc;
    private final ShipmentService shipments;

    public PaymentEventsListener(JdbcClient jdbc, ShipmentService shipments) {
        this.jdbc = jdbc;
        this.shipments = shipments;
    }

    @KafkaListener(topics = "payment-events", groupId = "shipping-service")
    @Transactional
    public void onEvent(ConsumerRecord<String, String> record) {
        String messageId = new String(record.headers().lastHeader("message-id").value(),
            StandardCharsets.UTF_8);
        int claimed = jdbc.sql("""
            INSERT INTO processed_messages (message_id, consumer)
            VALUES (:id, 'shipping-service')
            ON CONFLICT DO NOTHING
            """).param("id", messageId).update();
        if (claimed == 0) {
            return;                     // duplicate delivery: already processed
        }
        shipments.handlePaymentEvent(record.key(), record.value());
    }
}
```

4.5.3 Hands-on

The saga logic is reproduced as a plain JDK program with in-memory services: the same orchestration loop, compensation in reverse order, a failing shipping step, and an idempotent payment endpoint. It was run on the build machine.

SagaDemo.java JAVA

```
import java.util.*;

public class SagaDemo {

    // ---- in-memory "services" ---------------------------------------------------------
    static final Map<String, String> orders = new LinkedHashMap<>();
    static int stock = 5;
    static int chargedCents = 0;
    // idempotency store of the payment service
    static final Set<String> processedKeys = new HashSet<>();
    static boolean shippingDown = false;
    static int paymentCalls = 0;

    static void createOrder(String id) { orders.put(id, "PENDING"); }
    static void cancelOrder(String id) { orders.put(id, "CANCELLED"); }
    static void approveOrder(String id) { orders.put(id, "CONFIRMED"); }
    static void reserve(int qty) { if (stock < qty) throw new IllegalStateException(
            "out of stock"); stock -= qty; }
    static void release(int qty) { stock += qty; }
    static void charge(String key, int cents) {
        paymentCalls++;
        if (!processedKeys.add(key)) { System.out.println("    payment: duplicate key " + key
                + ", ignored"); return; }
        chargedCents += cents;
    }
    static void refund(String key, int cents) {
        if (processedKeys.add("refund-" + key)) chargedCents -= cents;
    }
    static void ship(String id) { if (shippingDown) throw new IllegalStateException(
            "shipping unavailable"); }

    interface Action { void run(); }
    record Step(String name, Action action, Action compensation) {}

    static void runSaga(String orderId, List<Step> steps) {
        Deque<Step> done = new ArrayDeque<>();
        try {
            for (Step s : steps) {
                System.out.println("  do   " + s.name());
                s.action().run();
                done.push(s);
            }
            approveOrder(orderId);
            System.out.println("  saga completed");
        } catch (RuntimeException e) {
            System.out.println("  FAILED: " + e.getMessage() + " -> compensating " + done.size()
                    + " step(s)");
            while (!done.isEmpty()) {
                Step s = done.pop();
                System.out.println("  undo " + s.name());
                s.compensation().run();
            }
            cancelOrder(orderId);
        }
    }

    static List<Step> checkout(String id, int qty, int cents) {
        return List.of(
                new Step("create order " + id, () -> createOrder(id), () -> {}),
                new Step("reserve stock x" + qty, () -> reserve(qty), () -> release(qty)),
                new Step("charge " + cents + " cents", () -> charge("pay-" + id, cents), ()
                        -> refund("pay-" + id, cents)),
                new Step("create shipment", () -> ship(id), () -> {}));
    }

    static void state(String label) {
        System.out.printf("  state [%s]: orders=%s stock=%d charged=%d%n", label, orders, stock,
                chargedCents);
```

SagaDemo.java (continued) JAVA

```
    }

    public static void main(String[] args) {
        System.out.println("== A: no saga, shipping fails after payment (partial failure)");
        shippingDown = true;
        try { createOrder("o-1"); reserve(1); charge("pay-o-1", 4999); ship("o-1"); }
        catch (RuntimeException e) { System.out.println("  exception: " + e.getMessage()); }
        state("after");

        System.out.println("== B: orchestrated saga, same failure");
        orders.clear(); stock = 5; chargedCents = 0; processedKeys.clear();
        runSaga("o-2", checkout("o-2", 1, 4999));
        state("after");

        System.out.println("== C: saga, everything healthy");
        shippingDown = false;
        runSaga("o-3", checkout("o-3", 2, 1500));
        state("after");

        System.out.println("== D: payment request retried with the same idempotency key");
        int before = chargedCents, callsBefore = paymentCalls;
        charge("pay-o-4", 700); charge("pay-o-4", 700); charge("pay-o-4", 700);
        System.out.println("  requests received=" + (paymentCalls - callsBefore)
                + ", amount charged=" + (chargedCents - before) + " cents");
    }
}
```

Run the saga demonstration TERMINAL

```
$ java SagaDemo.java
== A: no saga, shipping fails after payment (partial failure)
  exception: shipping unavailable
  state [after]: orders={o-1=PENDING} stock=4 charged=4999
== B: orchestrated saga, same failure
  do   create order o-2
  do   reserve stock x1
  do   charge 4999 cents
  do   create shipment
  FAILED: shipping unavailable -> compensating 3 step(s)
  undo charge 4999 cents
  undo reserve stock x1
  undo create order o-2
  state [after]: orders={o-2=CANCELLED} stock=5 charged=0
== C: saga, everything healthy
  do   create order o-3
  do   reserve stock x2
  do   charge 1500 cents
  do   create shipment
  saga completed
  state [after]: orders={o-2=CANCELLED, o-3=CONFIRMED} stock=3 charged=1500
== D: payment request retried with the same idempotency key
    payment: duplicate key pay-o-4, ignored
    payment: duplicate key pay-o-4, ignored
  requests received=3, amount charged=700 cents
```

Scenario A shows the problem without a saga: shipping failed after payment, leaving an order stuck in `PENDING`, stock reduced by one and 4999 cents charged, which is the inconsistent state that a customer eventually reports. Scenario B runs the same failure under the saga: the compensations execute in reverse order (refund, then stock release, then order cancellation) and the final state is clean. Scenario D sends the same payment request three times with one idempotency key and charges once.

The outbox pattern is demonstrated in PostgreSQL 16 (real output). The first script creates the order and the outbox row in one transaction, shows that a rolled-back transaction leaves neither, and adds three orders. The sequence values are consumed by the rolled-back transaction, which is why the identifiers skip 2: sequences are not transactional.

Outbox written atomically with the business row TERMINAL

```
$ psql -d shop -f outbox_run1.sql
-- 1. order and event written in one transaction, committed
BEGIN
INSERT 0 1
INSERT 0 1
```

Outbox written atomically with the business row (continued) TERMINAL

```
COMMIT
-- 2. same pair, but the transaction rolls back: neither row survives
BEGIN
INSERT 0 1
INSERT 0 1
ROLLBACK
 orders | outbox_rows
--------+-------------
      1 |           1
(1 row)

-- 3. three more orders so that two relay workers have something to share
INSERT 0 3
INSERT 0 3
 id | aggregate_id |  event_type  | unpublished
----+--------------+--------------+-------------
  1 | 1            | OrderCreated | t
  3 | 3            | OrderCreated | t
  4 | 4            | OrderCreated | t
  5 | 5            | OrderCreated | t
(4 rows)
```

Two relay workers then run concurrently with `FOR UPDATE SKIP LOCKED`; worker A is started first and holds its rows for 1.5 seconds. Worker B does not wait for A and does not take A's rows; it claims the next two.

Two relay workers share the backlog without blocking each other TERMINAL

```
$ psql -d shop -v worker=A -f relay.sql  # and, 0.5 s later, worker=B
BEGIN
relay A : claiming up to 2 rows
 id | aggregate_id
----+--------------
  1 | 1
  3 | 3
(2 rows)
relay A : publishing to the broker, then marking rows
UPDATE 2
COMMIT
BEGIN
relay B : claiming up to 2 rows
 id | aggregate_id
----+--------------
  4 | 4
  5 | 5
(2 rows)
relay B : publishing to the broker, then marking rows
UPDATE 2
COMMIT
```

Finally, the consumer-side deduplication: the same message identifier is claimed twice by one consumer (the second claim inserts nothing) and once by a different consumer, which is a separate claim because the key is the pair of message and consumer.

Idempotent consumer table TERMINAL

```
$ psql -d shop -f idem.sql
INSERT 0 1
INSERT 0 0
INSERT 0 1
 id | published
----+-----------
  1 | t
  3 | t
  4 | t
  5 | t
(4 rows)
```

4.5.4 Real-world problems and solutions

Problem 1: payment taken but no shipment (partial failure without a saga)

Scenario. The checkout method calls inventory, payment and shipping in sequence within one `@Transactional` method. The shipping service is unavailable for twenty minutes.

Symptoms. Customers are charged and receive no order confirmation. The order table shows orders in `PENDING`; the payment table shows captured payments for the same order identifiers; support receives complaints before the monitoring raises an alert.

Diagnosis. A reconciliation query across the exports of the services finds the inconsistency: payments with state `CAPTURED` whose order is not `CONFIRMED` for more than a few minutes. In the order service log the stack trace of the failing shipping call is followed by a rollback of the local transaction, which does not undo the remote effects (this is visible as the absence of any `refund` request in the payment service log).

Root cause. The local `@Transactional` annotation covers only the local database. The remote calls are not transactional, and the code has no compensation.

Fix. Replace the inline sequence with the orchestrated saga above: persisted state, idempotent steps, compensations in reverse order, and a recovery job. Order the steps so that the least reversible one is last.

Prevention. A failure-injection test for each step (a stub that fails on the nth call) asserting that the final state is either complete or fully compensated; a reconciliation job that raises an alert when sagas remain non-terminal beyond their service-level time.

Problem 2: events are lost when the broker is down (dual write)

Scenario. After saving the order the service calls `kafkaTemplate.send(...)`. The broker is unreachable for a minute during a rolling upgrade; the call throws after the database commit (or the process is killed between the two).

Symptoms. Orders exist in the order database but the shipping and notification services never received them; no error is visible to the customer. A comparison of counts (orders in the database against messages produced to the topic) differs.

Diagnosis. Count orders created in the interval and compare with the topic offsets of the interval (`kafka-run-class kafka.tools.GetOffsetShell` or the consumer group lag tools). Search the order service log for `TimeoutException` from the producer in the same interval, and check whether the send was executed inside or after the transaction.

Root cause. Two independent writes without atomicity; the failure window between them loses the event permanently.

Fix. The transactional outbox shown above. The relay retries until the broker accepts the message; the service answers the request as soon as the local transaction commits and is not coupled to broker availability. For events already lost, regenerate them from the source rows (a replay job that inserts outbox rows for orders without a corresponding published event).

Prevention. Forbid direct broker calls in transactional code by an architecture test; monitor the age of the oldest unpublished outbox row (`SELECT now() - min(created_at) FROM outbox WHERE published_at IS NULL`) and alert when it exceeds a minute.

Problem 3: duplicate processing of a message

Scenario. The relay publishes a message and the process stops before it marks the row. After restart the row is published again. In a separate incident a consumer rebalance redelivers messages that were processed but not yet committed.

Symptoms. A customer receives two confirmation e-mails, stock is decremented twice for one order, or a ledger shows two entries.

Diagnosis. Find messages that appear more than once: the same `message-id` header on two offsets of the topic, or two rows in the target table with the same business key. In the consumer log, the same message identifier is handled twice with different timestamps.

Root cause. At-least-once delivery combined with a consumer whose processing is not idempotent.

Fix. Deduplicate on the message identifier within the processing transaction (`PaymentEventsListener`), or make the operation idempotent by construction (a unique constraint on the business key, an upsert, a state machine that ignores a transition already applied). For side effects outside the database (e-mail), record the intent in the table in the same transaction and send from that table, with a provider idempotency key if the provider supports one.

Prevention. A test that delivers each message type twice and asserts a single effect; retention of the `processed_messages` rows for at least the maximum redelivery window of the broker.

Problem 4: stuck sagas and failing compensations

Scenario. Payment refund fails during compensation because the payment provider has an outage of two hours.

Symptoms. Orders remain in a non-terminal state; customers see "processing" for hours; the stock stays reserved.

Diagnosis. `SELECT state, count(*), min(updated_at) FROM checkout_saga WHERE state NOT IN ('COMPLETED','CANCELLED') GROUP BY state;` lists the stuck sagas by state and age; the log lines of the recovery job show repeated failure of the same compensation.

Root cause. Compensations are as fallible as the forward steps; a saga with a failed compensation cannot reach a terminal state automatically.

Fix. The recovery job retries compensation with backoff; after a threshold the saga is flagged for manual handling and an alert is raised (a dead-letter table or a work queue for operators). Compensations are idempotent, so retrying after a partial success is safe.

Prevention. A metric `sagas.stuck` (count of non-terminal sagas older than the service-level time) with an alert, and a runbook that states the manual action for each state.

4.5.5 Questions to expect

1. **Why not use distributed transactions (2PC) between microservices?** The coordinator blocks participants while they hold locks, availability is the product of all participants, brokers and REST do not support XA, and it couples services at the data layer.
2. **What is a saga?** A sequence of local transactions across services with compensating transactions that semantically undo completed steps when a later step fails; it provides eventual consistency without global locks.
3. **Orchestration or choreography?** Orchestration keeps the flow and compensations in one component (clear, easy to monitor, a coordinator to maintain); choreography uses events only (loose coupling, but the flow is implicit). Orchestration suits longer flows with many compensations.
4. **What properties must a compensating transaction have?** Idempotent, retryable until success, and commutative with concurrent operations as far as possible; steps that cannot be compensated are placed after the pivot.
5. **What is the dual-write problem and how does the outbox pattern solve it?** Updating a database and publishing to a broker are two writes that cannot be atomic; the outbox writes the event into a table in the same local transaction, and a relay publishes it afterwards with at-least-once delivery.
6. **How does an idempotent consumer work?** It stores each processed message identifier in a table with a unique constraint inside the processing transaction and skips messages already present; alternatively, the operation itself is idempotent.
7. **Polling relay or change-data-capture?** Polling is simple and needs no extra infrastructure but adds latency and database load; CDC (Debezium) reads the transaction log with low latency and load at the cost of operating Kafka Connect.
8. **What is eventual consistency and how is it explained to a product owner?** After a change, different services may disagree for a short time and then converge; the user interface and the business rules must be designed for the intermediate states.
9. **What is CQRS and when is it worthwhile?** Separate write and read models, the latter built from events; worthwhile when read shapes differ strongly from the write model or when queries would otherwise require cross-service joins.
10. **When is event sourcing appropriate?** When the history of changes is itself valuable (ledgers, auditing, temporal queries); it requires schema versioning, replay tooling and different query design, and is not a default.

Check yourself

1. In the orchestrator, shipping fails and the saga had `reached` the state `PAYMENT_CAPTURED`. Which compensations run and in which order?
2. A relay publishes a row and crashes before `UPDATE outbox SET published_at`. What happens on restart and what must the consumer do?
3. Why is `@Transactional(propagation = MANDATORY)` used on `OutboxWriter.write`?

4. Two relay instances run `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 100` at the same time. What does the second one see?

Answers

1. Refund the payment first, then release the stock, then cancel the order and mark the saga `CANCELLED`.
2. The row is still unpublished, so it is published again; the consumer receives a duplicate and must ignore it by message identifier or by an idempotent operation.
3. It throws an exception when no transaction is active, so an outbox row can never be written outside the business transaction, which would defeat atomicity.
4. The rows locked by the first instance are skipped; it receives other unlocked rows, so the two work on disjoint subsets.

KEY POINTS

Without distributed transactions, consistency is built from local transactions, idempotence and reliable events.

- Two-phase commit blocks, multiplies unavailability and is not supported by REST or Kafka, so it is not the default between services.
- A saga is a chain of local transactions with idempotent compensations; orchestration centralises the flow, choreography distributes it.
- The outbox pattern writes the event in the business transaction and publishes it afterwards, giving at-least-once delivery without a dual write.
- Consumers must be idempotent: deduplicate on message id in the processing transaction, or make the operation naturally idempotent.
- Persist saga state, retry compensations, and alert on sagas that stay non-terminal.

4.6 Distributed Tracing and Correlated Logging

In a monolith a stack trace and one log file describe a failed request. In a system of services one user request produces log lines in six processes, and nothing links them unless the system arranges it. **Distributed tracing** assigns every request a trace identifier at the edge, propagates it through every call and message, and records each unit of work as a timed **span**. A trace is the tree of spans of one request; it answers where the time went and which service failed. **Correlated logging** writes the same identifiers into every log line, so that a single search on a trace identifier returns the complete story across services.

4.6.1 Concept and mechanism

Traces, spans and propagation

A **trace** has a 128-bit identifier shared by all its spans. A **span** has a 64-bit identifier, a parent span identifier, a name, start time, duration, status and attributes (for example `http.request.method`, `db.system`). The root span is created where the request enters the system (the gateway or the first service). When a service makes an outbound call it creates a child span and sends the trace identifier and the child's span identifier in a request header, so that the callee continues the same trace with the caller's span as parent. This transmission of context across process boundaries is **propagation**.

The standard header is the W3C Trace Context `traceparent`, with four fields separated by hyphens: a version (`00`), the 32-hex-digit trace id, the 16-hex-digit parent span id, and trace flags (`01` means sampled). A second header, `tracestate`, carries vendor-specific data, and the separate **baggage** mechanism propagates user-defined key-value pairs (a tenant identifier) to all services. An older format, B3 (`X-B3-TraceId` and related headers) from Zipkin, is still found; Spring Boot supports both and selects W3C by default with the OpenTelemetry bridge.

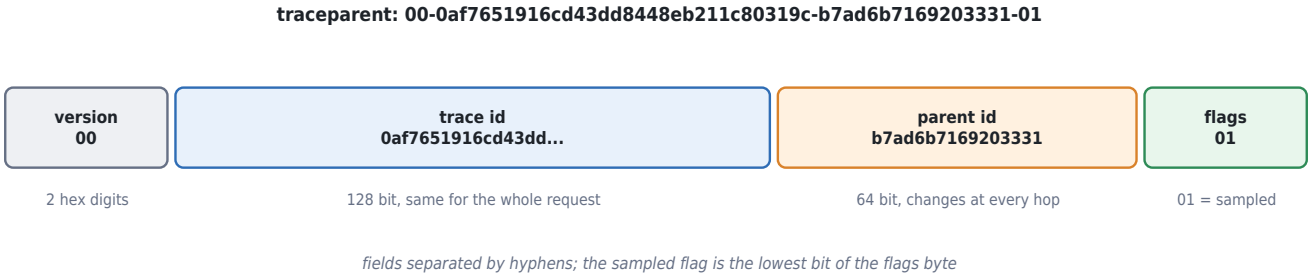


Figure 4.29. The fields of the traceparent header: the trace id is constant for the whole request, the parent id changes at every hop, and the flags carry the sampling decision.

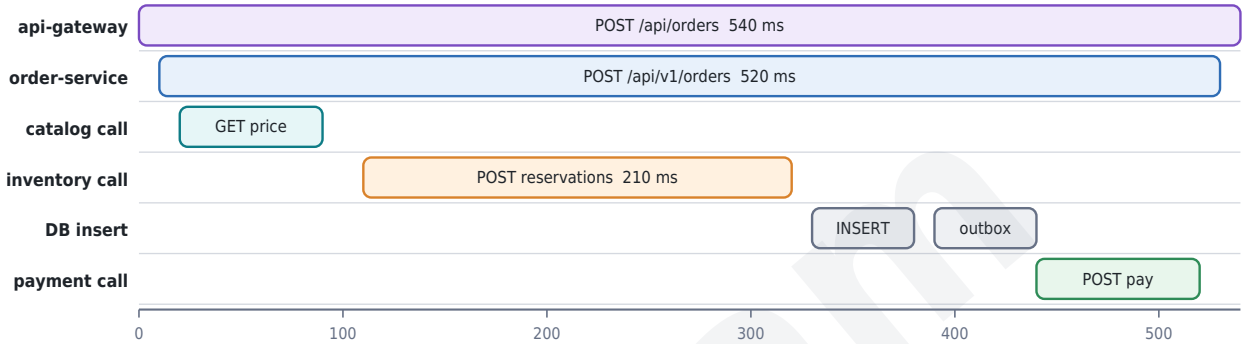


Figure 4.30. A trace of one checkout as a tracing backend draws it: the gateway span contains the order span, which contains sequential child spans; the 50 repeated customer spans of the earlier N+1 example would appear here as a long staircase.

Micrometer Tracing and Spring Boot

Spring Boot 3 instruments applications through the **Micrometer Observation API**: one instrumentation point (an HTTP server request, a `RestClient` call, a JDBC statement through a data-source proxy, a Kafka send or receive) produces both metrics and, if a tracer is present, spans. **Micrometer Tracing** is the facade over the tracer; a **bridge** connects it to a concrete implementation, either OpenTelemetry (`micrometer-tracing-bridge-otel`) or Brave (`micrometer-tracing-bridge-brave`). An exporter sends finished spans to a backend (Zipkin, Jaeger, Tempo, any OTLP collector). The setup in Boot 3.x is therefore three dependencies in addition to the Actuator: the bridge and an exporter (`opentelemetry-exporter-otlp` or `opentelemetry-exporter-zipkin`).

Instrumentation applies only to clients created from the auto-configured builders. A `RestClient` obtained through the injected `RestClient.Builder` (or `RestTemplateBuilder`, `WebClient.Builder`) carries the observation interceptor and adds the `traceparent` header; a client created with `RestClient.create()` or `new RestTemplate()` does not, and the trace ends at that call. The same holds for the Kafka template and listeners, where observation is enabled with `spring.kafka.template.observation-enabled=true` and `spring.kafka.listener.observation-enabled=true`.

By default only a fraction of requests is sampled: `management.tracing.sampling.probability` defaults to 0.1 (10 percent). The sampling decision is made at the root and travels in the flags, so downstream services honour it. A trace that is not sampled still has an identifier (and the identifier still reaches the logs), but no spans are exported.

Correlation in logs

Micrometer Tracing places the current identifiers in the logging **MDC** (mapped diagnostic context, a thread-local map of the logging framework) under the keys `traceId` and `spanId`. Boot's log pattern includes them through `logging.pattern.correlation`; the default format differs between Boot lines, so setting it explicitly is advisable:

application.properties (logging) PROPERTIES

```
logging.pattern.correlation=[${spring.application.name:},%X{traceId:-},%X{spanId:-}]
logging.include-application-name=false
```

With structured JSON logging (Boot 3.4 and later), the identifiers appear as fields automatically. In both forms a log aggregator can then be queried by `traceId`, which joins the lines of all services.

Context across threads

The MDC and the tracer's current span are thread-local. When work moves to another thread (an `@Async` method, an executor, a `CompletableFuture`, a reactive operator) the new thread has no context unless it is copied. A thread pool adds a second hazard: a worker that never clears its MDC keeps the identifiers of the previous request and writes them into the next one. The demonstration in the Hands-on section reproduces both effects.

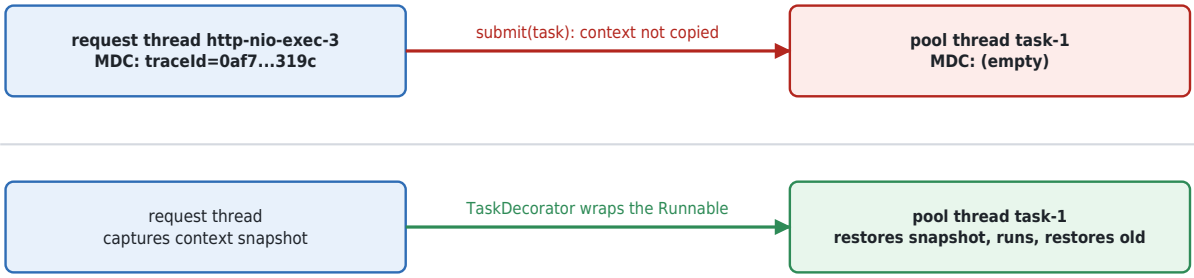


Figure 4.31. The request thread holds the trace context in thread-local storage; a task submitted to a pool starts with an empty context unless a decorator copies it at submission and restores the previous state afterwards.

Spring offers the `TaskDecorator` hook for this: a decorator wraps every `Runnable` submitted to an executor, captures the context at submission time on the calling thread, and installs it on the worker thread around the execution. Boot applies a single `TaskDecorator` bean to the auto-configured `applicationTaskExecutor`. Context propagation of the tracing context needs the `io.micrometer:context-propagation` library, which Boot's observation support uses; for reactive pipelines `Hooks.enableAutomaticContextPropagation()` performs the equivalent step.

4.6.2 Worked example

The dependencies and properties for the order service export traces over OTLP and keep the correlation in the logs.

order-service/pom.xml (dependencies added) XML

```xml
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>context-propagation</artifactId>
</dependency>
```

order-service/src/main/resources/application.yml (tracing part) YAML

```yaml
management:
  tracing:
    sampling:
      probability: 1.0          # 1.0 in test environments; 0.05 to 0.1 under production load
  otlp:
    tracing:
      endpoint: http://otel-collector:4318/v1/traces
spring:
  kafka:
    template:
      observation-enabled: true
    listener:
      observation-enabled: true
logging:
  pattern:
    correlation: "[${spring.application.name:},%X{traceId:-},%X{spanId:-}] "
  include-application-name: false
```

The property names of the OTLP and Zipkin endpoints (`management.otlp.tracing.endpoint` and `management.zipkin.tracing.endpoint`) belong to the Boot 3.x line; Boot 4 reorganised the tracing modules and the properties, so the reference of the version in use decides.

An `@Async` method needs the decorator so that the log lines written there keep the trace identifier. The decorator below is the explicit form of what `context-propagation` provides, and it matches the structure of the plain JDK demonstration:

AsyncConfig.java JAVA

```java
package com.shop.order.config;

import io.micrometer.context.ContextSnapshot;
import io.micrometer.context.ContextSnapshotFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.task.TaskDecorator;
import org.springframework.scheduling.annotation.EnableAsync;

@Configuration(proxyBeanMethods = false)
@EnableAsync
class AsyncConfig {

    private final ContextSnapshotFactory snapshots = ContextSnapshotFactory.builder().build();

    /** Captured on the submitting thread, restored on the worker around the task. */
    @Bean
    TaskDecorator contextPropagatingDecorator() {
        return runnable -> {
            ContextSnapshot snapshot = snapshots.captureAll();
            return snapshot.wrap(runnable);
        };
    }
}
```

A custom span around a business operation uses the Observation API; the observation produces a span and a timer with the same name and low-cardinality tags:

ReservationObserver.java JAVA

```java
package com.shop.order.infrastructure;

import io.micrometer.observation.Observation;
import io.micrometer.observation.ObservationRegistry;
import java.util.function.Supplier;
import org.springframework.stereotype.Component;

@Component
public class ReservationObserver {

    private final ObservationRegistry registry;

    public ReservationObserver(ObservationRegistry registry) {
        this.registry = registry;
    }

    public <T> T observe(int lineCount, Supplier<T> call) {
        return Observation.createNotStarted("inventory.reserve", registry)
            .lowCardinalityKeyValue("lines.bucket", lineCount > 10 ? "large" : "small")
            .observe(call);
    }
}
```

Only bounded values belong in tag values: the tag `lines.bucket` has two values, whereas an order identifier as a tag would create one time series per order and exhaust the metrics backend (see the chapter on observability in the next Part).

4.6.3 Hands-on

The propagation format and the thread hand-off problem are reproduced with plain JDK classes: a parser for `traceparent` that rejects malformed and all-zero identifiers, a thread-local MDC, a pool without propagation, the same pool with a decorator, and a pool whose worker never clears the MDC.

sreeram

TraceDemo.java JAVA

```java
import java.security.SecureRandom;
import java.util.*;
import java.util.concurrent.*;
import java.util.regex.*;

public class TraceDemo {

    static final SecureRandom RND = new SecureRandom();
    static final ThreadLocal<Map<String, String>> MDC = ThreadLocal.withInitial(HashMap::new);

    static String hex(int bytes) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < bytes; i++) sb.append(String.format("%02x", RND.nextInt(256)));
        return sb.toString();
    }

    record TraceParent(String traceId, String parentId, boolean sampled) {
        static final Pattern P = Pattern.compile(
                "^00-([0-9a-f]{32})-([0-9a-f]{16})-([0-9a-f]{2})$");
        static Optional<TraceParent> parse(String h) {
            if (h == null) return Optional.empty();
            Matcher m = P.matcher(h);
            if (!m.matches() || m.group(1).equals("0".repeat(32))
                    || m.group(2).equals("0".repeat(16)))
                return Optional.empty();
            return Optional.of(new TraceParent(m.group(1), m.group(2),
                    (Integer.parseInt(m.group(3), 16) & 1) == 1));
        }
        String header() { return "00-" + traceId + "-" + parentId + "-" + (sampled ? "01" : "00"); }
        TraceParent child() { return new TraceParent(traceId, hex(8), sampled); }
    }

    static void log(String msg) {
        System.out.printf("[%-22s] traceId=%s %s%n", Thread.currentThread().getName(),
                MDC.get().getOrDefault("traceId", "-"), msg);
    }

    /** Equivalent of a Spring TaskDecorator: copy the caller MDC onto the worker thread. */
    static Runnable propagating(Runnable r) {
        Map<String, String> captured = new HashMap<>(MDC.get());
        return () -> {
            Map<String, String> old = MDC.get();
            MDC.set(captured);
            try { r.run(); } finally { MDC.set(old); }
        };
    }

    public static void main(String[] args) throws Exception {
        System.out.println("== W3C traceparent parsing");
        String incoming = "00-0af7651916cd43dd8448eb211c80319c-b7ad6b7169203331-01";
        TraceParent tp = TraceParent.parse(incoming).orElseThrow();
        System.out.println("incoming : " + incoming);
        System.out.println("parsed   : traceId=" + tp.traceId());
        System.out.println("           parentId=" + tp.parentId() + " sampled=" + tp.sampled());
        System.out.println("outgoing : " + tp.child().header() + "  (same trace id, new span id)");
        System.out.println("malformed: " + TraceParent.parse("00-xyz-123-01"));
        System.out.println("all-zero : " + TraceParent.parse("00-" + "0".repeat(32)
                + "-b7ad6b7169203331-01"));

        System.out.println("== MDC across threads");
        MDC.get().put("traceId", tp.traceId());
        ExecutorService pool = Executors.newFixedThreadPool(1, r -> new Thread(r, "async-pool-1"));
        log("request thread, before dispatch");
        pool.submit(() -> log("async task WITHOUT propagation")).get();
        pool.submit(propagating(() -> log("async task WITH propagation"))).get();
        pool.submit(() -> log("next task on same worker (no leak expected)")).get();
        pool.shutdown();

        System.out.println("== stale context leak on a pooled thread");
        ExecutorService leaky = Executors.newFixedThreadPool(1, r -> new Thread(r, "leaky-pool-1"));
        leaky.submit(() -> { MDC.get().put("traceId",
                "trace-of-request-A"); log("request A runs, never clears MDC"); }).get();
```

TraceDemo.java (continued) JAVA

```
        leaky.submit(() -> log("request B on the same thread")).get();
        leaky.shutdown();
    }
}
```

Run the trace propagation demonstration (the generated span id varies per run) TERMINAL

```
$ java TraceDemo.java
== W3C traceparent parsing
incoming : 00-0af7651916cd43dd8448eb211c80319c-b7ad6b7169203331-01
parsed   : traceId=0af7651916cd43dd8448eb211c80319c
           parentId=b7ad6b7169203331 sampled=true
outgoing : 00-0af7651916cd43dd8448eb211c80319c-77bd951498d87e38-01  (same trace id, new span id)
malformed: Optional.empty
all-zero : Optional.empty
== MDC across threads
[main                ] traceId=0af7651916cd43dd8448eb211c80319c request thread, before dispatch
[async-pool-1        ] traceId=- async task WITHOUT propagation
[async-pool-1        ] traceId=0af7651916cd43dd8448eb211c80319c async task WITH propagation
[async-pool-1        ] traceId=- next task on same worker (no leak expected)
== stale context leak on a pooled thread
[leaky-pool-1        ] traceId=trace-of-request-A request A runs, never clears MDC
[leaky-pool-1        ] traceId=trace-of-request-A request B on the same thread
```

The first three lines of the second block are the essential result: the task submitted without propagation logs `traceId=-`, the decorated task logs the original identifier, and the next undecorated task on the same worker shows `-` again because the decorator restored the previous state. The last block shows the opposite defect: a worker that writes to the MDC and never clears it leaks request A's identifier into request B, producing log lines that point investigators at the wrong request. Servlet filters in Spring (`ServerHttpObservationFilter`) clear the context at the end of each request for exactly this reason.

4.6.4 Real-world problems and solutions

Problem 1: trace identifier missing in the logs of asynchronous work

Scenario. After a payment the order service sends a confirmation e-mail from an `@Async` method. Searching the logs for the trace id of a failed order returns the request lines but not the e-mail lines.

Symptoms. Log lines of the async method show an empty trace id (`[order-service,,]`), and the trace in the backend ends at the controller span, without the e-mail span.

Diagnosis. Log `Thread.currentThread().getName()` together with the MDC in the controller and in the async method: the first shows `http-nio-8080-exec-N` with identifiers, the second `task-N` without. Check for a `TaskDecorator` bean and for the `context-propagation` dependency (`mvn dependency:tree | grep context-propagation`). If a custom executor bean is declared, check that it is built from the Boot-provided builder, since a hand-built `ThreadPoolTaskExecutor` does not receive the decorator.

Root cause. The tracing context is thread-local and the executor does not copy it.

Fix. Register the `TaskDecorator` as above; for hand-built executors call `setTaskDecorator`. For `CompletableFuture.supplyAsync`, pass the decorated executor explicitly.

Prevention. A test that triggers the async path inside an observation and asserts the identifier in the log output; a review rule that every executor is created by the shared configuration.

Problem 2: the trace breaks at one service

Scenario. The trace of a checkout shows the gateway and the order service, but the payment service appears as a separate trace with a different trace id.

Symptoms. Two traces for one request; the payment trace has no parent. Latency attribution across the two services is impossible.

Diagnosis. Capture the headers on the wire or in the callee (`curl -v`, a debug log of the incoming request, or the `http.server.requests` observation with the `traceparent` header logged by a filter): if `traceparent` is absent on the incoming request, the caller did not send it. Check how the caller's client was created (`RestClient.create()` or `new RestTemplate()` instead of the injected builder) and whether a different propagation format (B3 on one side, W3C on the other) is configured. A callee that uses Brave with B3 and a caller that sends W3C produce exactly this break.

Root cause. The propagation chain is broken by a client that bypasses the auto-configured instrumentation, or by mixed formats.

Fix. Create all clients from the injected builders; align the propagation type in all services (`management.tracing.propagation.type`); for messaging enable observation on templates and listeners.

Prevention. A cross-service smoke test that makes one request through the gateway and asserts that the trace backend returns spans of every service under one trace id.

Problem 3: sampling hides the failing request

Scenario. Support reports a failed order but the trace backend has no trace for it.

Symptoms. The log line with the trace id exists, but the trace is not found.

Diagnosis. The sampling flag in the logged `traceparent` is `00`, or `management.tracing.sampling.probability` is 0.1, so nine of ten requests are not exported.

Root cause. Head-based sampling decides at the start of the request, before it is known to be interesting.

Fix. Use tail-based sampling in the collector (keep all traces that contain an error or exceed a latency threshold, sample the rest), and keep trace ids in all logs, including unsampled requests, so that the log search still works.

Prevention. Document the sampling policy and include the trace id in error responses (the `ProblemDetail` of the next chapter), so that a report from a customer carries the identifier.

4.6.5 Questions to expect

1. **What is the difference between a trace and a span?** A trace is the whole journey of one request across services; a span is one timed unit of work within it, with a parent span forming a tree.
2. **What does the `traceparent` header contain?** Version, trace id (32 hex digits), parent span id (16 hex digits) and flags, where the sampled flag drives the sampling decision downstream.
3. **How does a trace id reach the log lines?** The tracer puts `traceId` and `spanId` in the MDC; the log pattern prints them.
4. **Why is the trace id lost in an `@Async` method?** The MDC and the current span are thread-local, and the pool thread has none; a `TaskDecorator` with context propagation copies it.
5. **Which clients propagate the trace automatically?** Those built from the auto-configured `RestClient.Builder`, `RestTemplateBuilder` and `WebClient.Builder`; clients constructed directly do not.
6. **What is Micrometer Tracing?** A facade over tracers with bridges for OpenTelemetry and Brave; instrumentation is written once against the Observation API.
7. **Head-based versus tail-based sampling?** Head-based decides at the root before the outcome is known and is cheap; tail-based decides after the trace completes and can keep all errors and slow traces, at the price of buffering in the collector.
8. **Why must tag values have low cardinality?** Each distinct combination creates a time series in the metrics backend; unbounded values (identifiers) cause memory and cost explosions.

Check yourself

1. In `00-0af7651916cd43dd8448eb211c80319c-b7ad6b7169203331-01`, which field changes at each hop, and what does `01` mean?
2. A pooled worker writes `MDC.put("traceId", id)` and never removes it. What appears in the logs of the next task on that worker?
3. Why does a service that builds its client with `RestClient.create()` appear as the root of a new trace?
4. What does `management.tracing.sampling.probability=0.1` mean for a request with a logged trace id?
5. Which header propagates a tenant id to every service, and what is it called?

Answers

1. The parent span id; `01` means the trace is sampled.
2. The previous request's trace id, so the lines are attributed to the wrong request.
3. That client has no observation interceptor, sends no `traceparent`, and the callee starts a new trace.
4. About one request in ten is exported to the backend; the trace id exists for all requests.
5. Baggage (configured with `management.tracing.baggage.remote-fields`).

KEY POINTS

Tracing turns scattered log lines and timings into one tree per request, provided the context travels everywhere the request goes.

- A trace id is created at the edge and propagated by the W3C `traceparent` header; each hop adds a child span.
- Micrometer Tracing with an OpenTelemetry bridge and an exporter is the Boot 3 setup; clients must come from the injected builders.
- The MDC holds `traceId` and `spanId` for log correlation; threads need a `TaskDecorator` to inherit and to clean up context.
- Sampling is a policy: head-based is cheap, tail-based keeps the interesting traces; the id stays in the logs either way.
- Metric tags must be low-cardinality; identifiers belong in span attributes and logs, not in tags.

4.7 API Design for Services

An API between services is a long-lived contract. Its consumers are deployed independently and cannot be changed in the same release, so every change to a request, a response or an error format is judged by one criterion: does an existing consumer keep working. This chapter covers the decisions that determine whether an API can evolve: pagination that remains correct and fast as data grows, an error contract with a defined shape, rules for compatible change, and machine-readable documentation.

4.7.1 Concept and mechanism

Pagination: offset and keyset

Offset pagination (`page` and `size`, translated to `LIMIT size OFFSET page*size`) is simple and supports jumping to an arbitrary page, but it has two defects. The database must produce and discard all rows before the offset, so the cost grows with the page number; and if rows are inserted or deleted between two requests, items are skipped or repeated. **Keyset pagination** (also called seek or cursor pagination) passes the sort key of the last item seen, and the query continues after it: `WHERE (created_at, id) < (:ts, :id) ORDER BY created_at DESC, id DESC LIMIT n`. With an index on the sort columns the cost is independent of position, and the result is stable under concurrent writes. Its restriction is that only "next" (and, with the reversed comparison, "previous") navigation is possible, and the sort key must be unique, hence the identifier as tie-breaker.

The cursor returned to the client is an opaque string (a Base64-encoded key); opacity lets the server change its encoding without breaking clients. Spring Data supports keyset scrolling in recent versions (`ScrollPosition`, `Window`), and the same query is easily written explicitly, as in the worked example.

Two further rules protect the service. The maximum page size is capped on the server (a request for `size=100000` is clamped), and a response whose element count is unbounded is never returned. Serialising Spring Data's `Page` implementation directly produces a JSON shape that is not a stable contract (Spring Data 3.3 logs a warning for it); a dedicated response record, or `PagedModel` with `@EnableSpringDataWebSupport(pageSerializationMode = PageSerializationMode.VIA_DTO)`, fixes the shape.

Filtering and sorting

Filters and sort fields are an allow-list mapped explicitly to columns. Passing a client-supplied `sort=` value straight into a query exposes internal column names and permits sorting on unindexed columns that scan the table. Each supported filter combination should correspond to an index that the team has verified.

Error contract: ProblemDetail

RFC 9457 (which obsoletes RFC 7807) defines `application/problem+json` with the members `type` (a URI identifying the problem type), `title`, `status`, `detail`, `instance`, and arbitrary extension members. Spring Framework 6 represents this as `ProblemDetail`; with `spring.mvc.problemdetails.enabled=true` the framework renders its own exceptions (validation, missing parameters, unsupported media types) in this format, and an `@RestControllerAdvice` extending `ResponseEntityExceptionHandler` adds application exceptions. A consistent contract means that every service, and every error class, uses the same shape, and that clients can branch on `type` rather than parse messages. The error body should carry the trace id so that a customer report leads directly to the trace.

Compatible change

The rule for evolving a JSON API is that changes are **additive**. Adding an optional response field, a new optional request parameter or a new endpoint is compatible. Removing or renaming a field, changing its type or meaning, making an optional parameter required, and tightening validation are breaking. Consumers must follow the **tolerant reader** rule: ignore fields they do not know (Jackson in Spring Boot is configured with `FAIL_ON_UNKNOWN_PROPERTIES` disabled by default) and not depend on field order.

A rename that cannot be avoided is performed in the **expand and contract** sequence: add the new field while keeping the old one (expand); announce the deprecation with the `Deprecation` and `Sunset` response headers; migrate and verify that no consumer uses the old field (request metrics per consumer, or consumer-driven contract tests); remove the old field (contract). The same sequence applies to database columns behind the API.

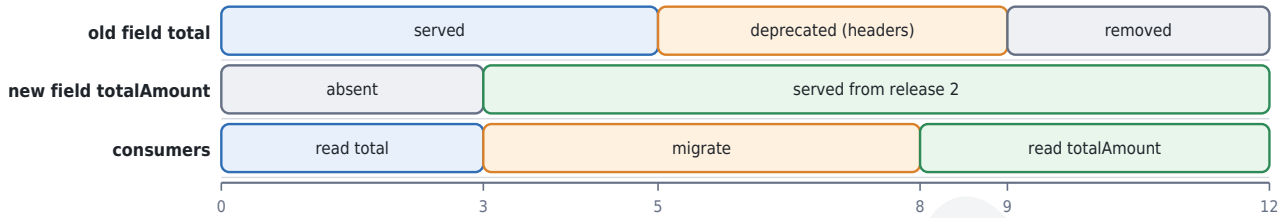


Figure 4.32. Expand and contract over three releases: both fields are served during the overlap, and the old one is removed only after usage has dropped to zero.

When a breaking change is unavoidable, a new version is introduced next to the old. The common mechanisms are a version in the path (`/api/v2/orders`, simple and cache-friendly, and used by the example platform), a request header, or a media type (`application/vnd.shop.v2+json`). Spring Framework 7 (Boot 4) adds first-class support for API versioning in the MVC and WebFlux stacks; in Boot 3 the path form with separate controllers is the practical choice. Two versions in operation double the test surface, so the number of live versions is kept small and each has a published end date.

Contracts and documentation

An OpenAPI description is the machine-readable form of the contract. The springdoc library (`springdoc-openapi-starter-webmvc-ui` for Spring MVC) generates it from the controllers and serves it at `/v3/api-docs`, with a Swagger UI at `/swagger-ui.html`. A generated description documents what the code does; it does not by itself protect consumers. **Consumer-driven contract testing** (Pact, or Spring Cloud Contract) records what each consumer actually depends on and verifies it against the provider in the provider's build, so that a breaking change fails the provider's pipeline before release. Optimistic concurrency over HTTP uses `ETag` and `If-Match`: a `PUT` carrying a stale tag is answered with `412 Precondition Failed`, which protects clients from lost updates without server-side locks.

4.7.2 Worked example

The order list endpoint uses keyset pagination, returns a stable response record, validates its parameters and reports errors as problem details.

OrderSummary.java JAVA

```java
package com.shop.order.api;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

public record OrderSummary(UUID id, String status, BigDecimal total, Instant createdAt) { }
```

CursorPage.java JAVA

```java
package com.shop.order.api;

import java.util.List;

/** Stable response shape: items plus an opaque cursor, null on the last page. */
public record CursorPage<T>(List<T> items, String nextCursor, int size) { }
```

OrderQueryRepository.java JAVA

```java
package com.shop.order.api;

import java.time.Instant;
import java.util.List;
import java.util.UUID;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.stereotype.Repository;

@Repository
public class OrderQueryRepository {

    private final JdbcClient jdbc;

    public OrderQueryRepository(JdbcClient jdbc) {
        this.jdbc = jdbc;
    }

    /** Fetches size + 1 rows so that the caller can tell whether another page exists. */
    public List<OrderSummary> firstPage(int sizePlusOne) {
        return jdbc.sql("""
            SELECT id, status, total, created_at FROM orders
            ORDER BY created_at DESC, id DESC LIMIT :n
            """).param("n", sizePlusOne)
            .query((rs, i) -> new OrderSummary(rs.getObject("id", UUID.class),
                rs.getString("status"), rs.getBigDecimal("total"),
                rs.getTimestamp("created_at").toInstant()))
            .list();
    }

    public List<OrderSummary> after(Instant createdAt, UUID id, int sizePlusOne) {
        return jdbc.sql("""
            SELECT id, status, total, created_at FROM orders
            WHERE (created_at, id) < (:ts, :id)
            ORDER BY created_at DESC, id DESC LIMIT :n
            """).param("ts", java.sql.Timestamp.from(createdAt))
            .param("id", id).param("n", sizePlusOne)
            .query((rs, i) -> new OrderSummary(rs.getObject("id", UUID.class),
                rs.getString("status"), rs.getBigDecimal("total"),
                rs.getTimestamp("created_at").toInstant()))
            .list();
    }
}
```

OrderController.java JAVA

```java
package com.shop.order.api;

import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.Base64;
import java.util.List;
import java.util.UUID;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/api/v2/orders")
public class OrderController {

    private static final int MAX_SIZE = 100;

    private final OrderQueryRepository queries;

    public OrderController(OrderQueryRepository queries) {
        this.queries = queries;
    }

    @GetMapping
    public CursorPage<OrderSummary> list(@RequestParam(defaultValue = "25") int size,
                                         @RequestParam(required = false) String cursor) {
```

Sreeram

OrderController.java (continued) JAVA

```
        int limit = Math.max(1, Math.min(size, MAX_SIZE));
        List<OrderSummary> rows = (cursor == null)
            ? queries.firstPage(limit + 1)
            : decodeAndQuery(cursor, limit + 1);
        boolean more = rows.size() > limit;
        List<OrderSummary> items = more ? rows.subList(0, limit) : rows;
        String next = more ? encode(items.get(items.size() - 1)) : null;
        return new CursorPage<>(items, next, items.size());
    }

    private List<OrderSummary> decodeAndQuery(String cursor, int n) {
        try {
            String raw = new String(Base64.getUrlDecoder().decode(cursor), StandardCharsets.UTF_8);
            String[] parts = raw.split("\\|");
            return queries.after(Instant.parse(parts[0]), UUID.fromString(parts[1]), n);
        } catch (RuntimeException e) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "invalid cursor");
        }
    }

    private static String encode(OrderSummary last) {
        String raw = last.createdAt() + "|" + last.id();
        return Base64.getUrlEncoder().withoutPadding()
            .encodeToString(raw.getBytes(StandardCharsets.UTF_8));
    }
}
```

The error contract is a single advice class. It extends the framework's handler so that framework exceptions are rendered as problem details too, and adds the application's own exceptions with a stable `type` and the trace id:

ApiExceptionHandler.java JAVA

```
package com.shop.order.api;

import com.shop.order.infrastructure.StockUnavailableException;
import io.micrometer.tracing.Tracer;
import java.net.URI;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    private final Tracer tracer;

    public ApiExceptionHandler(Tracer tracer) {
        this.tracer = tracer;
    }

    @ExceptionHandler(StockUnavailableException.class)
    ResponseEntity<ProblemDetail> stockUnavailable(StockUnavailableException e) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.SERVICE_UNAVAILABLE,
            "Stock reservation is temporarily unavailable");
        problem.setType(URI.create("https://api.shop.example/problems/stock-unavailable"));
        problem.setTitle("Stock unavailable");
        problem.setProperty("traceId", tracer.currentSpan() == null
            ? null : tracer.currentSpan().context().traceId());
        HttpHeaders headers = new HttpHeaders();
        headers.set(HttpHeaders.RETRY_AFTER, "5");
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).headers(headers).body(problem);
    }
}
```

sreeram

application.yml (API settings) YAML

```yaml
spring:
  mvc:
    problemdetails:
      enabled: true
springdoc:
  api-docs:
    path: /v3/api-docs
  swagger-ui:
    path: /swagger-ui.html
```

A typical error body produced by this handler (not run on the build machine) is shown below; the `traceId` member is the link between a customer report and the trace.

503 response (typical output, not run on the build machine) JSON

```json
{
  "type": "https://api.shop.example/problems/stock-unavailable",
  "title": "Stock unavailable",
  "status": 503,
  "detail": "Stock reservation is temporarily unavailable",
  "instance": "/api/v2/orders",
  "traceId": "0af7651916cd43dd8448eb211c80319c"
}
```

4.7.3 Hands-on

The cost difference between offset and keyset pagination was measured in PostgreSQL 16 on a table of 500 000 orders with an index on `(created_at DESC, id DESC)`. The page requested is page 10 000 of size 25. The statements in the file are the SQL that the repository above executes (with literal values in place of the bind parameters).

Offset against keyset pagination (real output, timings measured on the build machine) TERMINAL

```
$ psql -d shop -f page.sql
-- offset pagination, page 10000 of size 25 (OFFSET 249975)
 Limit (actual rows=25 loops=1)
   Buffers: shared hit=1839 read=960
   ->  Index Scan using orders_big_created_idx on orders_big (actual rows=250000 loops=1)
         Buffers: shared hit=1839 read=960
 Planning:
   Buffers: shared hit=39 read=1
 Planning Time: 0.130 ms
 Execution Time: 22.798 ms
-- keyset pagination at the same position (cursor = last row of the previous page)
 Limit (actual rows=25 loops=1)
   Buffers: shared hit=3 read=1
   ->  Index Scan using orders_big_created_idx on orders_big (actual rows=25 loops=1)
         Index Cond: (ROW(created_at, id) < ROW('2026-10-16 17:29:11.82463+00'::timestamp with
        time zone, 249975))
         Buffers: shared hit=3 read=1
 Planning Time: 0.076 ms
 Execution Time: 0.034 ms
```



Figure 4.33. The offset query reads 250 000 index entries to return 25 rows (22.8 ms, 2799 buffers), whereas the keyset query reads 25 (0.03 ms, 4 buffers); the gap widens linearly with the page number.

The `actual rows=250000` line of the offset plan is the discarded work; the keyset plan shows the cursor condition as an index condition, so the scan starts at the cursor position. On a table that is read concurrently the difference is amplified by the buffers held and the I/O of each request.

4.7.4 Real-world problems and solutions

Problem 1: a field rename breaks a consumer in production

Scenario. The order service renames `total` to `totalAmount` in `OrderSummary` as part of a cleanup. The mobile back-end service that consumes the list deserialises into its own DTO.

Symptoms. After the release the consumer shows zero totals (the unknown field is ignored and the old field is missing, so the value defaults to null or zero), no exception is logged, and the consumer's tests pass.

Diagnosis. Compare the response body before and after with a saved sample (`curl` against both versions), or diff the OpenAPI documents of the two builds (`/v3/api-docs`). In the consumer, enable failure on missing required properties in its contract test and inspect its DTO. The access logs by `User-Agent` or client identifier identify who still reads the field.

Root cause. A breaking change was released as if it were additive, and the tolerant-reader behaviour of the consumer turned the break into silent data loss.

Fix. Restore the old field and add the new one next to it (expand); mark the old field deprecated in the OpenAPI document and send `Deprecation` and `Sunset` headers; remove it after the metrics show no use.

Prevention. An OpenAPI diff step (for example `openapi-diff`) in the provider pipeline that fails on removed or renamed fields, and consumer-driven contract tests.

Problem 2: the list endpoint slows down as data grows

Scenario. `GET /api/v1/orders?page=N&size=25` is fast for years and then degrades to seconds for a crawler that reads all pages.

Symptoms. Latency grows linearly with the page number; the database CPU rises while the application is idle; the slow query log shows `OFFSET` values in the hundreds of thousands.

Diagnosis. `EXPLAIN (ANALYZE, BUFFERS)` of the logged statement shows an index scan returning all rows up to the offset (as in the measurement above); `pg_stat_statements` ranks the statement by total time.

Root cause. Offset pagination reads and discards all earlier rows.

Fix. Introduce keyset pagination in `/api/v2/orders` as above, keep v1 with a hard cap on `page`, and publish the end date of v1.

Prevention. A maximum page size and a maximum offset in the API; load tests against a table of production size.

Problem 3: clients cannot parse errors

Scenario. Four services return errors in four formats: a plain string, Spring's default `timestamp, status, error, path` map, a custom envelope, and HTML from the gateway.

Symptoms. The shared client library contains a chain of parsing attempts, and a new error class is shown to users as "unknown error".

Diagnosis. Request an invalid payload on each service with `curl -i` and compare `Content-Type` and body.

Root cause. No common error contract.

Fix. Enable `spring.mvc.problemdetails.enabled=true`, derive each service's advice from `ResponseEntityExceptionHandler`, define the problem `type` URIs in a shared catalogue, and have the gateway translate its own errors to the same format.

Prevention. A contract test that checks the content type `application/problem+json` for each error status of each endpoint.

4.7.5 Questions to expect

1. **Offset or keyset pagination?** Offset allows arbitrary page jumps but costs grow with the offset and results shift under concurrent writes; keyset is constant-time and stable but supports only sequential navigation and needs a unique sort key with an index.
2. **What is an opaque cursor and why?** An encoded position token whose structure is not part of the contract, so the server can change it without breaking clients.
3. **What is `ProblemDetail`?** Spring's representation of RFC 9457 `application/problem+json` with `type`, `title`, `status`, `detail`, `instance` and extension members; enabled for framework exceptions with `spring.mvc.problemdetails.enabled`.

4. **Which changes are backward compatible?** Adding optional response fields, optional request parameters and new endpoints; removing or renaming fields, changing types or meanings, and making parameters required are breaking.
5. **What is the tolerant reader principle?** Consumers ignore unknown fields and do not depend on order, so providers can add fields safely.
6. **How is a field renamed safely?** Expand and contract: serve both, deprecate with headers, observe usage, then remove.
7. **How are API versions exposed?** Path (`/v2/`), header or media type; path versioning is the simplest in Boot 3; keep few versions with published end dates.
8. **What does consumer-driven contract testing add over OpenAPI?** OpenAPI describes what the provider offers; contracts record what consumers actually use and fail the provider build when a used feature changes.
9. **How are lost updates prevented over HTTP?** `ETag` on reads, `If-Match` on writes, and `412 Precondition Failed` for a stale tag, backed by a version column.

Check yourself

1. A request asks for `size=100000`. What must the controller do?
2. Why is the identifier part of the keyset comparison `(created_at, id) < (:ts, :id)`?
3. What is wrong with serialising a `PageImpl` directly as the response?
4. A consumer ignores unknown fields. Which change is still breaking for it: adding `currency`, or renaming `total`?

Answers

1. Clamp it to the server maximum (100 in the example) or reject it; an unbounded response is never produced.
2. Timestamps are not unique; without the tie-breaker, rows with equal timestamps are skipped or repeated at page boundaries.
3. The JSON shape is an implementation detail of Spring Data and not a stable contract; use a dedicated record or `PagedModel` via DTO mode.
4. Renaming `total`: the consumer no longer finds the field it reads.

KEY POINTS

A service API is a contract that evolves under live consumers, so its shape, errors and growth are designed deliberately.

- Keyset pagination costs the same at any position and is stable under writes; offset pagination degrades linearly (22.8 ms against 0.03 ms in the measurement).
- Responses use dedicated records, capped sizes and allow-listed filters and sorts.
- Errors follow RFC 9457 `ProblemDetail` everywhere, with stable `type` URIs and the trace id.
- Change is additive; renames follow expand and contract with deprecation headers and usage metrics.
- OpenAPI documents the contract; consumer-driven contract tests and an OpenAPI diff in the pipeline enforce it.

sreeram

PART 5

Testing, Observability and Performance

Verifying behaviour before release, observing it in production, and tuning it when the numbers are wrong.

In this part

- **5.1** Testing Strategy and Unit Tests
- **5.2** Spring Slice Tests and Context Caching
- **5.3** Integration Testing with Testcontainers
- **5.4** Actuator and Micrometer
- **5.5** Logging in Spring Boot Applications
- **5.6** Performance Tuning

5.1 Testing Strategy and Unit Tests

A test suite has two jobs: it detects regressions quickly, and it documents intended behaviour in executable form. A suite that is slow, flaky or coupled to implementation details fails both jobs, because developers stop running it or stop trusting it. Test design therefore starts with a strategy: which behaviour is verified at which level, at what cost, and with which tools. This chapter fixes the strategy and then treats the lowest level in depth: unit tests written with JUnit 5, AssertJ and Mockito, including the handling of time, randomness and test data. The Spring-specific levels are covered in the chapters that follow.

5.1.1 Concept and mechanism

The test pyramid and its cost model

Tests differ along three axes: scope (how much real code runs), speed (milliseconds against seconds or minutes) and diagnostic precision (a failing unit test names a method; a failing end-to-end test names a symptom). The test pyramid allocates many cheap, precise tests at the base and few expensive, broad tests at the top. For a Spring Boot service, the levels map onto tools as follows.

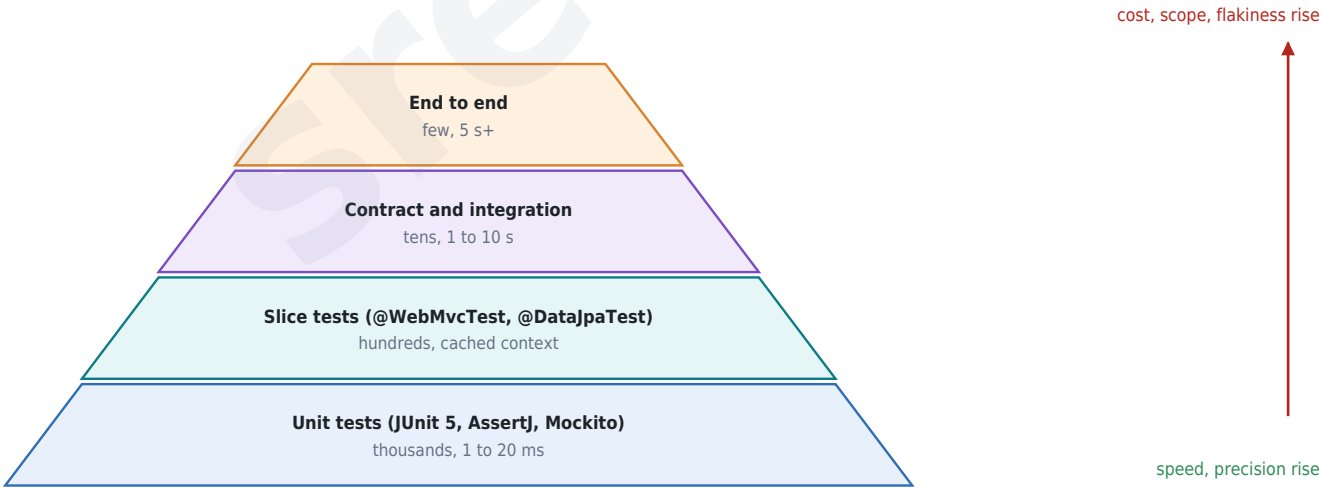


Figure 5.1. The test pyramid for a Spring Boot service: cost and scope grow toward the apex, so the number of tests should shrink in the same direction.

Level	Scope	Typical tools	Time per test
Unit	One class, collaborators replaced	JUnit 5, AssertJ, Mockito	1 to 20 ms
Slice	One layer with a partial context	`@WebMvcTest`, `@DataJpaTest`, `@JsonTest`	0.1 to 3 s (context cached)
Integration	Full context, real database or broker	`@SpringBootTest`, Testcontainers	1 to 10 s
Contract	Agreement between two services	Spring Cloud Contract, Pact	0.1 to 2 s

Level	Scope	Typical tools	Time per test
End to end	Deployed system through the gateway	HTTP client, browser automation	5 s and above

Test levels in a Spring Boot service.

The pyramid is a heuristic rather than a law. Services that consist mostly of data access and little logic gain more from integration tests against a real database than from mocked unit tests; the shape is then closer to a "honeycomb" with a thick middle layer. The governing rule is to test each behaviour at the lowest level that can exhibit the failure. A pricing rule is a unit-level concern. A mapping that depends on PostgreSQL column types is not, because no mock reproduces that behaviour.

JUnit 5 architecture

JUnit 5 consists of three components. The **JUnit Platform** is the launcher and the `TestEngine` SPI used by build tools and IDEs. **JUnit Jupiter** is the programming model (`@Test`, extensions) together with its engine. **JUnit Vintage** runs JUnit 3 and 4 tests on the same platform. Spring Boot's `spring-boot-starter-test` brings Jupiter, AssertJ, Mockito, Hamcrest, JSONassert, JsonPath and the Spring test support.

The central lifecycle rule is that Jupiter creates a **new instance of the test class for every test method** (`@TestInstance(Lifecycle.PER_METHOD)`, the default). Instance fields therefore cannot leak state between tests, and `@BeforeEach` runs against a fresh object. `@BeforeAll` and `@AfterAll` run once per class and must be `static` unless the class is annotated `@TestInstance(Lifecycle.PER_CLASS)`. The following program reproduces this mechanism in about forty lines of plain JDK code: it finds annotated methods by reflection, creates a new instance per test and runs the callbacks around the test body.

MiniJUnit.java JAVA

```java
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

public class MiniJUnit {
    @Retention(RetentionPolicy.RUNTIME) @interface Test {}
    @Retention(RetentionPolicy.RUNTIME) @interface BeforeEach {}
    @Retention(RetentionPolicy.RUNTIME) @interface AfterEach {}

    static class Counter {
        int value;
        void inc() { value++; }
    }

    public static class CounterTests {
        private Counter counter;

        @BeforeEach void setUp()    { counter = new Counter(); System.out.println("  setUp"); }
        @AfterEach  void tearDown() { System.out.println("  tearDown"); }

        @Test void startsAtZero() { check(counter.value == 0, "expected 0"); }
        @Test void incrementsOnce() { counter.inc(); check(counter.value == 1, "expected 1"); }
        @Test void deliberatelyFailing() { counter.inc(); check(counter.value == 2, "expected 2"); }

        static void check(boolean ok, String msg) { if (!ok) throw new AssertionError(msg); }
    }

    public static void main(String[] args) throws Exception {
        Class<?> type = CounterTests.class;
        int passed = 0, failed = 0;
        for (Method m : type.getDeclaredMethods()) {
            if (!m.isAnnotationPresent(Test.class)) continue;
            Object instance =
                    type.getDeclaredConstructor().newInstance();   // new instance per test
            System.out.println(m.getName());
            run(instance, BeforeEach.class);
            try {
                m.invoke(instance);
                passed++;
                System.out.println("  PASS");
            } catch (InvocationTargetException e) {
                failed++;
                System.out.println("  FAIL: " + e.getCause().getMessage());
            } finally {
                run(instance, AfterEach.class);
```

MiniJUnit.java (continued) JAVA

```
            }
        }
        System.out.println(passed + " passed, " + failed + " failed");
    }

    static void run(Object o, Class<? extends Annotation> a) throws Exception {
        for (Method m : o.getClass().getDeclaredMethods())
            if (m.isAnnotationPresent(a)) { m.setAccessible(true); m.invoke(o); }
    }
}
```

Output of the harness (real run; method order from reflection is not specified and may differ) TERMINAL

```
$ java MiniJUnit.java
startsAtZero
  setUp
  PASS
  tearDown
incrementsOnce
  setUp
  PASS
  tearDown
deliberatelyFailing
  setUp
  FAIL: expected 2
  tearDown
2 passed, 1 failed
```

The harness omits what the real engine adds: extension points (`BeforeEachCallback`, `ParameterResolver`, `TestExecutionExceptionHandler`), parameter injection, nested classes, conditional execution, parallelism and reporting. Spring's `SpringExtension` is an ordinary Jupiter extension; it builds or fetches the `ApplicationContext` in `postProcessTestInstance` and injects fields and parameters from it. Order of test methods in Jupiter is deterministic but intentionally non-obvious; tests must not depend on it, and `@TestMethodOrder` exists for the rare exception.

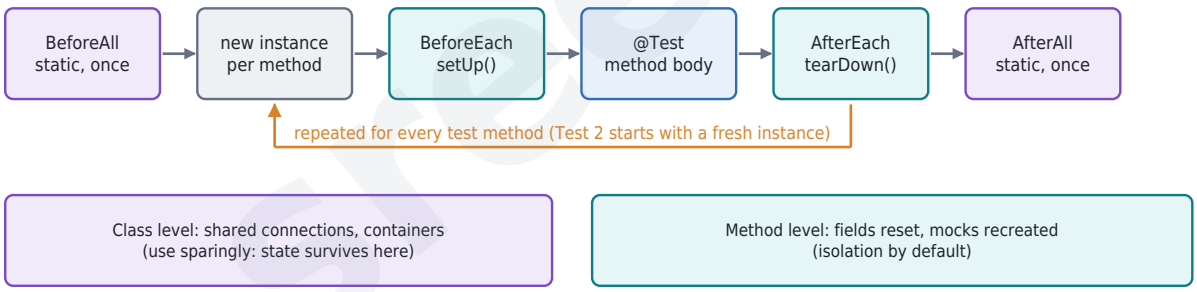


Figure 5.2. The Jupiter callback sequence for two test methods: static class-level callbacks run once, while a fresh instance and the per-method callbacks surround every test.

Writing readable tests with Jupiter and AssertJ

Jupiter supplies several constructs that reduce duplication without hiding intent.

- `@ParameterizedTest` runs one method against many inputs. Sources: `@ValueSource`, `@CsvSource`, `@MethodSource`, `@EnumSource`.
- `@Nested` classes group the cases of one scenario (for example "when the cart is empty") and inherit the enclosing `@BeforeEach`.
- `@DisplayName` and `@Tag` control reporting and selection (for example `-Dgroups=slow` with Surefire).
- `@TempDir` supplies an isolated directory; `@Timeout` fails a test that runs too long.
- `assertThrows` and `assertAll` in Jupiter, or the richer AssertJ equivalents.

AssertJ is preferred for assertions because the fluent API is discoverable in the IDE and failure messages show actual and expected values. The pattern `assertThat(actual).isEqualTo(expected)` is the entry point; `extracting`, `containsExactly`, `usingRecursiveComparison` and `assertThatThrownBy` cover most needs. Two details cause frequent confusion. First, `BigDecimal.equals` compares scale: `new BigDecimal("180.0000")` is not equal to `new`

`BigDecimal("180.00")`; the AssertJ method `isEqualByComparingTo` compares numeric value. Second, `SoftAssertions` collect every failure of a block instead of stopping at the first, which shortens the edit-run cycle when several fields of a result are verified.

Test doubles with Mockito

A **test double** replaces a collaborator. The vocabulary (after Meszaros) distinguishes a dummy (passed but unused), a stub (returns canned answers), a spy (wraps a real object and records calls), a mock (verifies interactions) and a fake (a working lightweight implementation such as an in-memory repository). Mockito creates stubs, spies and mocks.

Mockito 5 uses the inline mock maker by default, which instruments classes through a Java agent and therefore mocks final classes, final methods and (with `mockStatic`) static methods. The generated mock records every invocation, matches stubbing rules in the order they were declared (the last matching rule wins), and returns type-appropriate defaults (`null`, `0`, `false`, empty collection, empty `Optional`) for unstubbed calls. The plain-JDK program below implements the same idea with `java.lang.reflect.Proxy`: an `InvocationHandler` records calls and arguments and returns a configured answer. It also shows the standard technique for time: a `java.time.Clock` is injected, and tests supply `Clock.fixed`.

MiniMock.java JAVA

```
import java.lang.reflect.*;
import java.math.BigDecimal;
import java.time.*;
import java.util.*;

public class MiniMock {
    interface PaymentGateway { boolean charge(String account, BigDecimal amount); }

    /** Unit under test: applies a 10% promotion until the promotion deadline. */
    static class CheckoutService {
        private final PaymentGateway gateway;
        private final Clock clock;
        private final Instant promoEnds;
        CheckoutService(PaymentGateway gateway, Clock clock, Instant promoEnds) {
            this.gateway = gateway; this.clock = clock; this.promoEnds = promoEnds;
        }
        boolean checkout(String account, BigDecimal price) {
            BigDecimal due = clock.instant().isBefore(promoEnds)
                    ? price.multiply(new BigDecimal("0.90")) : price;
            return gateway.charge(account, due);
        }
    }

    /** A tiny Mockito-like recorder built on java.lang.reflect.Proxy. */
    static class Mock implements InvocationHandler {
        final List<String> calls = new ArrayList<>();
        final List<Object[]> argsSeen = new ArrayList<>();
        Object answer = false;
        @Override public Object invoke(Object proxy, Method m, Object[] args) {
            calls.add(m.getName());
            argsSeen.add(args);
            return answer;
        }
    }

    public static void main(String[] args) {
        Instant ends = Instant.parse("2026-12-31T23:59:59Z");

        Mock handler = new Mock();
        handler.answer = true;
        ClassLoader loader = PaymentGateway.class.getClassLoader();
        PaymentGateway gateway = (PaymentGateway) Proxy.newProxyInstance(
                loader, new Class<?>[]{PaymentGateway.class}, handler);

        Clock before = Clock.fixed(Instant.parse("2026-12-01T10:00:00Z"), ZoneOffset.UTC);
        Clock after  = Clock.fixed(Instant.parse("2027-01-01T00:00:00Z"), ZoneOffset.UTC);

        new CheckoutService(gateway, before, ends).checkout("acc-1", new BigDecimal("200.00"));
        new CheckoutService(gateway, after,  ends).checkout("acc-1", new BigDecimal("200.00"));

        System.out.println("calls     = " + handler.calls);
        System.out.println("captured  = " + handler.argsSeen.get(0)[1]
```

MiniMock.java (continued) JAVA

```
                + " , " + handler.argsSeen.get(1)[1]);
        System.out.println("verify times(2) -> " + (handler.calls.size() == 2));
    }
}
```

Output (real run) TERMINAL

```
$ java MiniMock.java
calls     = [charge, charge]
captured  = 180.0000 , 200.00
verify times(2) -> true
```

The captured amounts show the `BigDecimal` scale effect described above: multiplying `200.00` by `0.90` yields scale 4. A test asserting `isEqualTo(new BigDecimal("180.00"))` fails, while `isEqualByComparingTo("180.00")` passes. The program also demonstrates why `Clock` injection makes the promotion boundary testable without sleeping or touching the system clock.

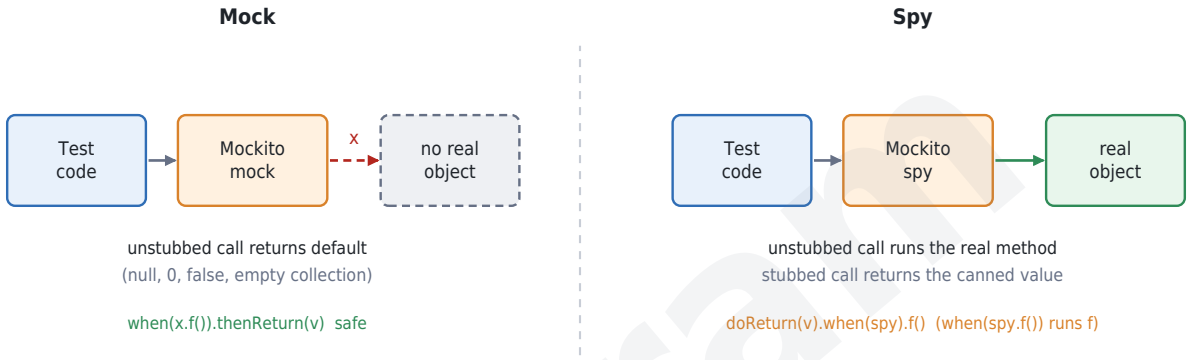


Figure 5.3. Behaviour of a Mockito mock and a spy: the mock never calls real code, whereas the spy delegates unstubbed calls to the wrapped object; both record invocations for later verification.

Mockito rules that prevent fragile tests

- Use `@ExtendWith(MockitoExtension.class)`. It installs **strict stubs**: an unused stubbing raises `UnnecessaryStubbingException`, and a stubbed call made with different arguments raises `PotentialStubbingProblem`. Both indicate dead or wrong test setup.
- Mock only collaborators that cross a boundary (repository, HTTP client, clock, message sender). Do not mock value objects, records, collections or the class under test.
- Verify outcomes before interactions. `verify(gateway).charge(...)` is appropriate for a side effect that is the observable result (a payment was requested). Verifying every internal call couples the test to the implementation and breaks on refactoring.
- Use `ArgumentCaptor` when the argument is constructed inside the unit and its content matters.
- Prefer `doReturn(x).when(spy).method()` for spies, because `when(spy.method())` executes the real method during stubbing.
- Never share a mock between tests through a static field.

Time, randomness and test data

Non-deterministic inputs are the second most common cause of flaky tests, after shared state. The remedy is the same for all of them: pass the source of non-determinism in as a dependency. Time is injected as `java.time.Clock` (a `@Bean` in production, `Clock.fixed` or `Clock.offset` in tests). Randomness is injected as a `java.util.random.RandomGenerator` or a seeded `java.util.Random`. Identifiers are injected as a `Supplier<UUID>`. Scheduling is tested by invoking the scheduled method directly rather than waiting for the trigger.

Test data builders keep fixtures readable. A builder supplies valid defaults and lets each test override only the fields relevant to the scenario, so the intent of the test is visible and a new mandatory field is added in one place.

OrderBuilder.java JAVA

```java
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

public class OrderBuilder {
    public record Line(String sku, int quantity, BigDecimal unitPrice) { }
    public record Order(String customerId, List<Line> lines, String coupon) { }

    private String customerId = "cust-1";
    private final List<Line> lines = new ArrayList<>(
            List.of(new Line("SKU-1", 1, new BigDecimal("10.00"))));
    private String coupon = null;

    public static OrderBuilder anOrder() { return new OrderBuilder(); }

    public OrderBuilder forCustomer(String id) { this.customerId = id; return this; }
    public OrderBuilder withCoupon(String code) { this.coupon = code; return this; }
    public OrderBuilder withLine(String sku, int qty, String price) {
        lines.add(new Line(sku, qty, new BigDecimal(price)));
        return this;
    }
    public Order build() { return new Order(customerId, List.copyOf(lines), coupon); }
}
```

Mutation testing complements coverage. Line coverage reports which code ran, not whether any assertion would notice a defect. PIT (`pitest-maven`) mutates the bytecode (flips a comparison, removes a call, changes a constant) and re-runs the tests; a mutant that no test kills indicates a missing assertion. The cost is a long run, so PIT is normally applied to the domain package or to changed files in continuous integration.

5.1.2 Worked example

The service under test calculates an order total with a coupon and charges a gateway. The listing shows the production class and its JUnit 5, AssertJ and Mockito test. The code is idiomatic for Spring Boot 3.x on JDK 21; it was not executed on the build machine, because the Spring and Mockito artefacts are not available offline (the mechanisms above were executed with plain-JDK equivalents).

PricingService.java JAVA

```java
package shop.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Clock;
import java.time.Instant;
import org.springframework.stereotype.Service;

@Service
public class PricingService {

    private final CouponRepository coupons;
    private final Clock clock;

    public PricingService(CouponRepository coupons, Clock clock) {
        this.coupons = coupons;
        this.clock = clock;
    }

    public BigDecimal total(BigDecimal subtotal, String couponCode) {
        if (couponCode == null) {
            return subtotal;
        }
        Coupon coupon = coupons.findByCode(couponCode)
                .orElseThrow(() -> new UnknownCouponException(couponCode));
        Instant now = clock.instant();
        if (now.isAfter(coupon.expiresAt())) {
            throw new ExpiredCouponException(couponCode);
        }
        BigDecimal discount = subtotal.multiply(coupon.percent())
                .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
        return subtotal.subtract(discount);
    }
}
```

sreeram

PricingServiceTest.java JAVA

```java
package shop.pricing;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.BDDMockito.given;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.Optional;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class PricingServiceTest {

    private static final Instant NOW = Instant.parse("2026-06-01T12:00:00Z");

    @Mock CouponRepository coupons;
    PricingService service;

    @org.junit.jupiter.api.BeforeEach
    void setUp() {
        service = new PricingService(coupons, Clock.fixed(NOW, ZoneOffset.UTC));
    }

    @Test
    @DisplayName("without a coupon the subtotal is returned unchanged")
    void noCoupon() {
        assertThat(service.total(new BigDecimal("50.00"), null)).isEqualByComparingTo("50.00");
    }

    @Nested
    class WithCoupon {

        @ParameterizedTest(name = "{0}% of {1} leaves {2}")
        @CsvSource({"10, 200.00, 180.00", "25, 80.00, 60.00", "33, 10.00, 6.70"})
        void appliesPercentage(String percent, String subtotal, String expected) {
            given(coupons.findByCode("SAVE"))
                    .willReturn(Optional.of(new Coupon("SAVE", new BigDecimal(percent),
                            NOW.plusSeconds(60))));

            assertThat(service.total(new BigDecimal(subtotal), "SAVE"))
                    .isEqualByComparingTo(expected);
        }

        @Test
        void expiredCouponIsRejected() {
            given(coupons.findByCode("OLD"))
                    .willReturn(Optional.of(
                            new Coupon("OLD", BigDecimal.TEN, NOW.minusSeconds(1))));

            assertThatThrownBy(() -> service.total(new BigDecimal("10.00"), "OLD"))
                    .isInstanceOf(ExpiredCouponException.class)
                    .hasMessageContaining("OLD");
        }

        @Test
        void unknownCouponIsRejected() {
            given(coupons.findByCode("NOPE")).willReturn(Optional.empty());

            assertThatThrownBy(() -> service.total(new BigDecimal("10.00"), "NOPE"))
                    .isInstanceOf(UnknownCouponException.class);
        }
    }
}
```

Seeram

What to notice: the clock is fixed at one instant, so the expiry boundary (one second before and after) is tested exactly; `isEqualByComparingTo` avoids the scale pitfall; the parameterized test covers three rounding cases with one method; strict stubs fail the build if a stubbing becomes unused after a refactoring.

5.1.3 Hands-on

Unit tests are executed by the Maven Surefire plugin (`mvn test`) or the Gradle `test` task. Integration tests conventionally use the suffix `IT` and run in the Failsafe plugin (`mvn verify`), which keeps the fast loop fast.

Typical output (not run on the build machine) TERMINAL

```
$ ./mvnw -q test -Dtest=PricingServiceTest
[INFO] Running shop.pricing.PricingServiceTest
[INFO] Running shop.pricing.PricingServiceTest$WithCoupon
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.412 s
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
```

Parallel execution is enabled in `src/test/resources/junit-platform.properties`:

junit-platform.properties PROPERTIES

```
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = same_thread
junit.jupiter.execution.parallel.mode.classes.default = concurrent
```

With these settings, test classes run concurrently while the methods of one class run sequentially. Parallelism is safe only when tests share no mutable state; the next section shows what happens otherwise.

5.1.4 Real-world problems and solutions

Problem 1: A test suite fails intermittently after parallel execution is enabled

Scenario. A team enables class-level parallelism to shorten a six-minute build. The suite passes locally and fails on roughly one pipeline run in a hundred, with different tests failing each time.

Symptoms. Assertion failures such as `expected size 1 but was 2` that do not reproduce when a single test is re-run. The failures disappear with `junit.jupiter.execution.parallel.enabled=false`.

Diagnosis. (1) Re-run the failing class alone ten times; if it is stable, the cause lies in interaction with other tests. (2) Search the test and main sources for `static` mutable fields, singletons and caches: `grep -rn "static.*\(List\|Map\|Set\)<" src`. (3) Run with `-Djunit.jupiter.execution.parallel.mode.classes.default=same_thread` to confirm. (4) Order the suspects by reading `@BeforeAll`/`@AfterAll` blocks that clear shared structures.

The following program reproduces the mechanism: two tests clear and fill the same static list. Executed one after another they always pass; executed on two threads the interleaving corrupts the list.

FlakyDemo.java JAVA

```
import java.util.*;
import java.util.concurrent.*;

public class FlakyDemo {
    static final List<String> SHARED = new ArrayList<>();          // static state shared by "tests"

    static boolean testAddsOne() {
        SHARED.clear();
        SHARED.add("a");
        pause();
        return SHARED.size() == 1;
    }
    static boolean testAddsTwo() {
        SHARED.clear();
        SHARED.add("a"); SHARED.add("b");
        pause();
        return SHARED.size() == 2;
    }
    static void pause() { Thread.onSpinWait(); }

    public static void main(String[] args) throws Exception {
        int runs = 200, sequentialFailures = 0, parallelFailures = 0;
        ExecutorService pool = Executors.newFixedThreadPool(2);
```

FlakyDemo.java (continued) JAVA

```java
        for (int i = 0; i < runs; i++) {
            if (!testAddsOne() || !testAddsTwo()) sequentialFailures++;
            Future<Boolean> f1 = pool.submit(FlakyDemo::testAddsOne);
            Future<Boolean> f2 = pool.submit(FlakyDemo::testAddsTwo);
            if (!(f1.get() && f2.get())) parallelFailures++;
        }
        pool.shutdown();
        System.out.println("sequential: " + sequentialFailures + " failing suites out of " + runs);
        System.out.println("parallel  : " + parallelFailures + " failing suites out of " + runs);
    }
}
```

Three consecutive runs (real; the parallel figure varies between runs, which is the defect) TERMINAL

```
$ for i in 1 2 3; do java FlakyDemo.java | grep parallel; done
parallel  : 0 failing suites out of 200
parallel  : 1 failing suites out of 200
parallel  : 0 failing suites out of 200
```

Root cause. Shared mutable state with no synchronisation. The failure probability per run is small, which is why it survives code review and local runs, and why it is expensive to find by retrying.

Fix. Remove the shared state: create collaborators per test (fields of the test instance are per-test under the default lifecycle), replace static caches with injected beans, and use `@ResourceLock("shared-resource")` from Jupiter for the rare unavoidable shared resource such as a port or a system property. Do not add retries to hide the failure.

Prevention. Enable parallel execution early, while the suite is small. Treat any test that needs a specific execution order as defective. Keep a CI report of per-test failure rates (Surefire XML reports) so that a test failing in 1 of 200 runs is visible.

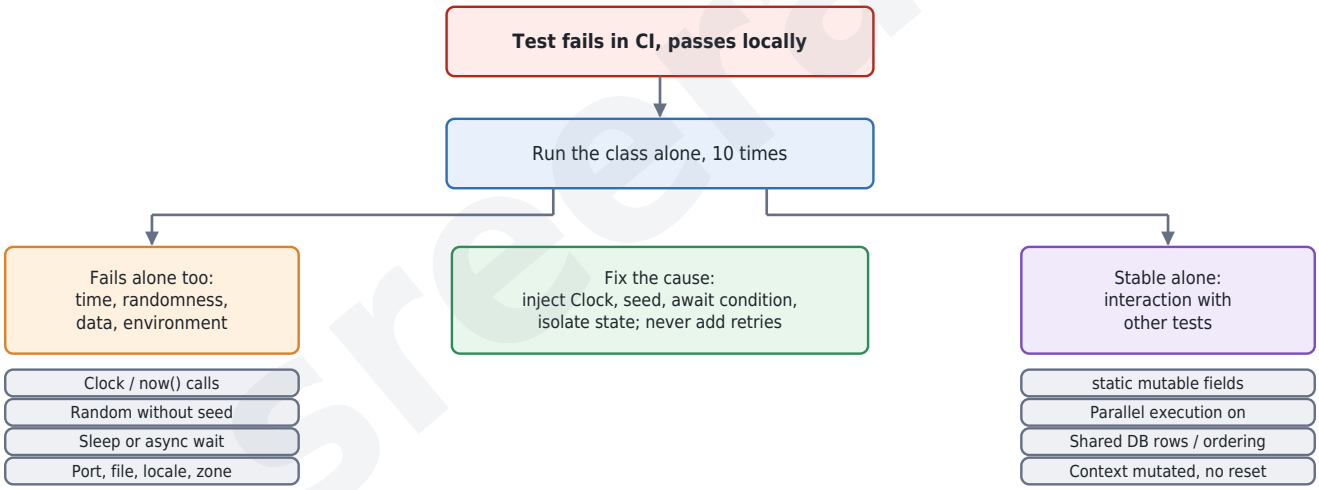


Figure 5.4. Diagnosis path for a flaky test: reproduce in isolation first, then classify the cause as shared state, time, randomness, asynchrony or environment.

Problem 2: A time-dependent test fails every day after midnight UTC

Scenario. A test creates an order and asserts that `order.createdAt().toLocalDate()` equals `LocalDate.now()`. It fails in the nightly build, which starts at 00:00:03 UTC on a server in another time zone.

Symptoms. The failure occurs only within a window of a few seconds or hours each day, and only on some machines.

Diagnosis. Search for `LocalDate.now()`, `Instant.now()`, `System.currentTimeMillis()` and `new Date()` in both main and test code. Check the JVM default time zone (`-Duser.timezone`, the `TZ` variable) on the failing agent. Compare the timestamps in the assertion message.

Root cause. The production code and the test each read the system clock independently, and a day boundary falls between the two reads; in addition, the two reads may use different zones.

Fix. Inject one `Clock` into the production code, read the time once, and construct the test with `Clock.fixed`.

ClockConfig.java JAVA

```java
package shop.config;

import java.time.Clock;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class ClockConfig {
    @Bean
    Clock clock() {
        return Clock.systemUTC();
    }
}
```

Prevention. Forbid direct calls to the static `now()` methods outside the configuration class by an ArchUnit rule or a Checkstyle regular expression. Tests that need a Spring context replace the `Clock` bean with a `@TestConfiguration` that returns a fixed clock.

Problem 3: A mock-heavy test passes while the production code is broken

Scenario. After a refactoring that renamed a repository method, all unit tests of the service remain green but the endpoint returns HTTP 500 in the staging environment.

Symptoms. High line coverage, no failing test, and a failing deployed application. The stack trace points at a query that Spring Data cannot derive (`PropertyReferenceException`).

Diagnosis. The test stubs `repository.findByEmailAddress(...)`, a method on a mocked interface. A mock accepts any method signature declared on the interface, so it cannot detect that Spring Data derives the query from the method name and the entity has no `emailAddress` property.

Root cause. The test verified the interaction with a double that cannot reproduce the behaviour that failed. The behaviour belongs to the persistence layer and must be tested there.

Fix. Keep the service unit test, and add a slice or integration test (`@DataJpaTest` with a real database; see the chapters that follow) that executes each repository method. A single context start-up detects invalid derived queries, because Spring Data validates them when the repository bean is created.

Prevention. Apply the rule from the pyramid section: assert each behaviour at the lowest level that can fail for that reason. Mock the repository in service tests, but test the repository against a real database.

5.1.5 Questions to expect

What is the difference between a mock, a stub and a spy?

A stub returns canned answers and is not asserted upon. A mock is a double whose interactions are verified. A spy wraps a real object, so unstubbed calls execute real code while calls are still recorded. Mockito uses the term "mock" for objects that act as stubs or mocks depending on whether the test calls `verify`.

Why does JUnit 5 create a new instance of the test class for each test method?

Isolation: instance fields cannot carry state from one test to another, and the order of execution does not matter. `@TestInstance(Lifecycle.PER_CLASS)` changes this behaviour and makes `@BeforeAll` non-static, but also makes shared state the responsibility of the test author.

What does MockitoExtension add compared with MockitoAnnotations.openMocks?

It integrates with the Jupiter lifecycle, closes the mocks after each test and applies strict stubs by default, so unused or mismatched stubbings fail the test.

How is code that depends on the current time tested?

By injecting a `java.time.Clock` and substituting `Clock.fixed` or `Clock.offset` in tests. Sleeping, comparing against the real clock with tolerances, and mocking static methods are inferior alternatives.

Which tests belong at the base of the pyramid, and why not test everything end to end?

Business rules, mappers, validators and algorithms belong at the base because they run in milliseconds and fail with a precise location. End-to-end tests are slow, depend on every component and fail for environmental reasons; they verify a few critical journeys only.

Does 100 percent line coverage prove the tests are adequate?

No. Coverage measures executed lines, not asserted behaviour. A test without assertions can reach full coverage. Mutation testing (PIT) estimates assertion quality by checking whether the tests detect injected defects.

How are void methods and exceptions verified?

Void methods through their side effects (`verify`, captured arguments, state of a fake) and exceptions with `assertThrows` or `assertThatThrownBy`, which also permit assertions on type, message and cause.

When should a spy be used?

Rarely: for legacy classes that cannot be split, or to verify a call on a real object while keeping its behaviour. Frequent need for spies on the class under test indicates a design problem (the class does too much).

KEY POINTS

Key points

- Test each behaviour at the lowest level that can fail for that reason; use the pyramid as a budget for cost and speed, not as a fixed ratio.
- Jupiter creates one test instance per method; shared mutable state, static caches and ordering assumptions are the main sources of flaky tests.
- Inject `Clock`, random generators and identifier suppliers; then time and randomness become ordinary inputs.
- Use `MockitoExtension` with strict stubs; mock boundaries, not value objects, and avoid verifying internals.
- Compare `BigDecimal` values with `isEqualByComparingTo`; `equals` includes scale.
- Mocks cannot detect persistence or serialisation defects; those need slice or integration tests.

5.2 Spring Slice Tests and Context Caching

Unit tests prove that a class behaves correctly in isolation; they cannot prove that the annotations around it work. Request mapping, JSON binding, validation, exception translation, derived queries and transaction boundaries are all declared in annotations and executed by framework code, so a test must run some of the Spring infrastructure to verify them. Starting the full application for every such test is wasteful. Spring Boot therefore provides **test slices**: annotations that start a deliberately small application context containing one layer (web, persistence, JSON, HTTP client) and nothing else. This chapter explains how slices are built, what each one contains, how the test context cache decides whether to reuse a context, and why suites become slow when it does not.

5.2.1 Concept and mechanism

Anatomy of a Spring Boot test

A test class annotated with a Boot test annotation is processed by the Spring TestContext Framework. The sequence is as follows.

1. `SpringExtension` (the Jupiter extension, registered by the meta-annotation `@ExtendWith(SpringExtension.class)` on every Boot test annotation) asks the `TestContextManager` for a `TestContext`.
2. The `TestContextManager` builds a `MergedContextConfiguration` from the annotations on the class: configuration classes, active profiles, property sources, `ContextCustomizer` instances (contributed by `@MockitoBean`, `@DynamicPropertySource`, the Testcontainers support and others) and the `ContextLoader`. For Boot tests the loader is `SpringBootContextLoader`.
3. The `DefaultCacheAwareContextLoaderDelegate` looks up this configuration in the **context cache** (`DefaultContextCache`). A hit returns the existing `ApplicationContext`; a miss loads a new one through `SpringApplication` and stores it.

4. `DependencyInjectionTestExecutionListener` injects `@Autowired` fields and parameters of the test instance from the context. Other listeners handle transactions (`TransactionalTestExecutionListener`), `@Sql` scripts and dirty-context handling.

The cache key is the `MergedContextConfiguration`. Two test classes share a context only when every component of the key is equal. The practical consequence is that a different set of `@MockitoBean` declarations, a different `@TestPropertySource`, a different profile list or a different `@Import` produces a new context with a full start-up. The cache holds 32 contexts by default and evicts the least recently used (`spring.test.context.cache.maxSize`).

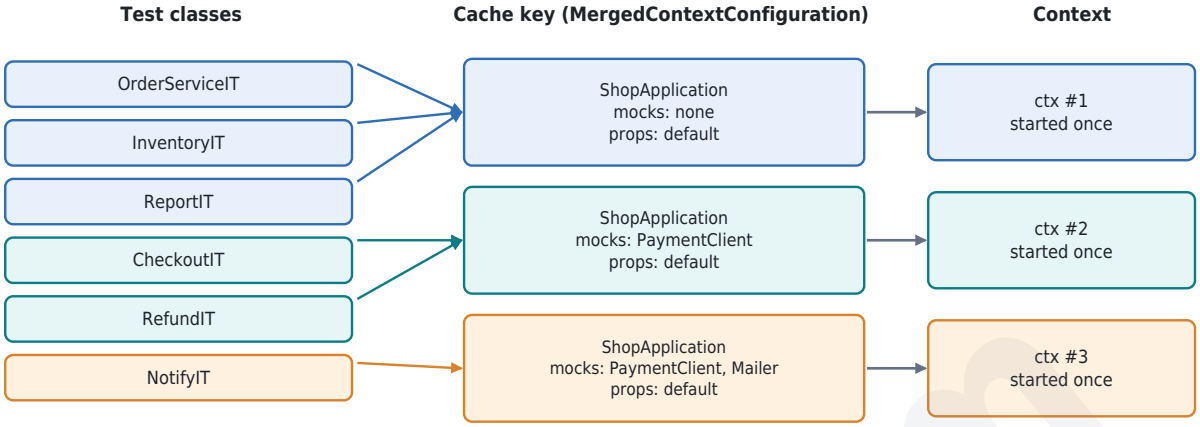


Figure 5.5. The context cache as a map from merged configuration to a started ApplicationContext: three test classes with identical configuration share one entry, while a different mock set forces a second start-up.

The next program models the cache faithfully enough to show the cost structure. The key is a record of configuration class, mock set and properties; a miss costs 150 ms (standing for a context start), a hit costs nothing.

ContextCacheDemo.java JAVA

```
import java.util.*;

public class ContextCacheDemo {
    record ContextKey(String config, Set<String> mockBeans, Map<String, String> properties) { }

    static final Map<ContextKey, Object> CACHE = new HashMap<>();
    static int startups = 0;

    static Object contextFor(ContextKey key) throws InterruptedException {
        Object ctx = CACHE.get(key);
        if (ctx == null) {
            Thread.sleep(150);                // stands for a 150 ms context start-up
            startups++;
            ctx = new Object();
            CACHE.put(key, ctx);
        }
        return ctx;
    }

    static ContextKey key(Set<String> mocks, Map<String, String> props) {
        return new ContextKey("ShopApplication", mocks, props);
    }

    public static void main(String[] args) throws Exception {
        List<ContextKey> suite = List.of(
            key(Set.of(), Map.of()),                                  // OrderServiceIT
            key(Set.of("PaymentClient"), Map.of()),                   // CheckoutIT
            key(Set.of(), Map.of()),                                  // InventoryIT
            key(Set.of("PaymentClient"), Map.of("shop.fast", "true")),// PromoIT
            key(Set.of("PaymentClient", "Mailer"), Map.of()),         // NotifyIT
            key(Set.of(), Map.of()),                                  // ReportIT
            key(Set.of("PaymentClient"), Map.of())                    // RefundIT
        );
        long t0 = System.nanoTime();
        for (ContextKey k : suite) contextFor(k);
        long ms = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("test classes     : " + suite.size());
        System.out.println("context start-ups: " + startups
                + " (distinct keys: " + new HashSet<>(suite).size() + ")");
        System.out.println("elapsed          : ~" + (ms / 50 * 50) + " ms");
    }
```

ContextCacheDemo.java (continued) JAVA

```
        // same suite with one shared configuration (mock declared once, in a shared base class)
        CACHE.clear(); startups = 0;
        ContextKey shared = key(Set.of("PaymentClient"), Map.of());
        t0 = System.nanoTime();
        for (int i = 0; i < suite.size(); i++) contextFor(shared);
        ms = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("shared config    : " + startups + " start-up, ~"
                + (ms / 50 * 50) + " ms");
    }
}
```

Output (real run; the elapsed figures are rounded) TERMINAL

```
$ java ContextCacheDemo.java
test classes     : 7
context start-ups: 4 (distinct keys: 4)
elapsed          : ~600 ms
shared config    : 1 start-up, ~150 ms
```

Seven test classes cost four start-ups because four distinct keys exist. In a real application a start-up takes two to fifteen seconds rather than 150 ms, so the arithmetic dominates the build time. The remedy is structural: keep the number of distinct configurations small.

What each slice loads

A slice annotation combines three things: a `@TypeExcludeFilter` that restricts component scanning to the relevant stereotypes, an `@AutoConfigure...` set that enables only the relevant auto-configuration classes (listed in `META-INF/spring/org.springframework.boot.test.autoconfigure...imports` files), and default properties. Anything outside the slice is absent from the context and must be supplied by `@MockitoBean`, `@Import` or `@TestConfiguration`.

Annotation	Beans and auto-configuration loaded	Not loaded
`@WebMvcTest`	`@Controller`, `@ControllerAdvice`, `@JsonComponent`, `Converter`, `Filter`, `HandlerInterceptor`, `WebMvcConfigurer`; MVC, Jackson, validation, MockMvc, Spring Security	`@Service`, `@Repository`, `@Component`, DataSource
`@DataJpaTest`	`@Entity` scan, Spring Data repositories, DataSource, Hibernate, `TestEntityManager`, Flyway or Liquibase	Controllers, services, web layer
`@JsonTest`	Jackson `ObjectMapper`, modules, `JacksonTester`	Everything else
`@RestClientTest`	`RestClient.Builder` or `RestTemplateBuilder`, `MockRestServiceServer`, Jackson	Controllers, repositories
`@WebFluxTest`	Reactive controllers, `WebTestClient`	Servlet MVC
`@SpringBootTest`	The complete application context	Nothing excluded

Slice annotations (Boot 3.4 and 3.5).

In Boot 3.x these annotations live in `org.springframework.boot.test.autoconfigure...` packages (for example `org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest`). Spring Boot 4 reorganised the test modules and moved several of them (for example the web slice moved to a `webmvc.test.autoconfigure` package); code in this chapter targets Boot 3.4 and 3.5.

A slice never contains the services that a controller calls. A controller test that omits `@MockitoBean` for its service fails at context start with `UnsatisfiedDependencyException` naming the missing bean. This is a feature: the failure message lists exactly the collaborators the layer needs.

@MockitoBean, @MockitoSpyBean and @MockBean

Spring Framework 6.2 introduced bean override support: `@MockitoBean` and `@MockitoSpyBean` in the package `org.springframework.test.context.bean.override.mockito`. Boot 3.4 deprecated the older `@MockBean` and `@SpyBean` (package `org.springframework.boot.test.mock.mockito`), whose behaviour the new annotations replace; removal is announced for Boot 4. Differences that matter: `@MockitoBean` can be declared on the test class as well as on fields, replaces or creates the bean by type or name, and registers its definition through a `ContextCustomizer`, so the declared set is part of the cache key. Mocks are reset after each test by default (`MockReset.AFTER`).

MockMvc, MockMvcTester and the real server

`MockMvc` drives the `DispatcherServlet` directly through a mock servlet request and response. No socket is opened and no Tomcat runs; filters, interceptors, argument resolution, message conversion and exception handlers all execute, because they belong to the Spring MVC pipeline. What is **not** exercised: the servlet container (error pages, `server.*` properties, HTTP/2, compression), real serialisation over the wire and multi-threaded behaviour. `@SpringBootTest(webEnvironment = RANDOM_PORT)` with `TestRestTemplate` or `RestClient` covers those.

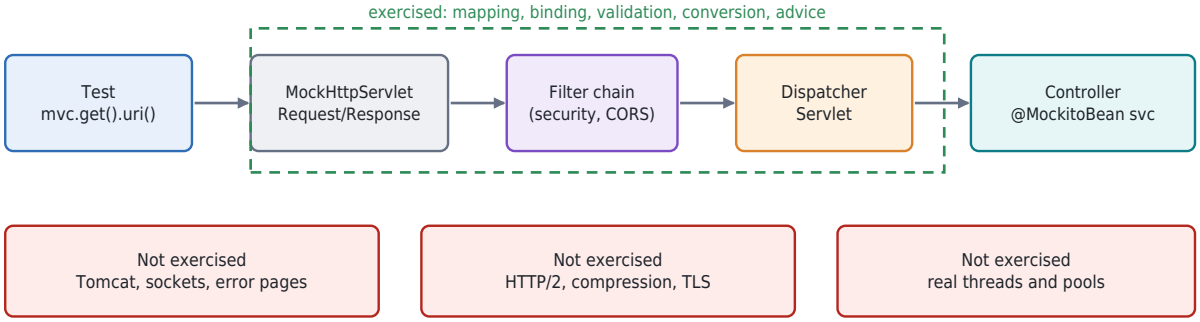


Figure 5.6. What MockMvc exercises: the request skips the network and the servlet container but passes through the filter chain, DispatcherServlet and handler, so mapping, binding and error translation are tested.

`MockMvcTester` (Spring Framework 6.2, therefore Boot 3.4 and later) is an AssertJ-based facade over `MockMvc`. It replaces the `andExpect(...)` chain with `assertThat(...)`, gives better failure messages and treats exceptions raised by the handler as assertion targets (`hasFailed()`, `failure()`). It is auto-configured by `@WebMvcTest` when AssertJ is on the classpath.

5.2.2 Worked example

The scenario is an order endpoint: `POST /orders` validates the body, delegates to `OrderService` and returns `201` with the created resource, while `GET /orders/{id}` returns `404` as an RFC 9457 `ProblemDetail` for an unknown identifier. The production code is shown first.

OrderController.java JAVA

```java
package shop.orders;

import jakarta.validation.Valid;
import java.net.URI;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
class OrderController {

    private final OrderService orders;

    OrderController(OrderService orders) {
        this.orders = orders;
    }

    @PostMapping
    ResponseEntity<OrderView> create(@Valid @RequestBody CreateOrderRequest request) {
        OrderView created = orders.create(request);
        return ResponseEntity.created(URI.create("/orders/" + created.id())).body(created);
    }

    @GetMapping("/{id}")
    OrderView get(@PathVariable long id) {
        return orders.find(id).orElseThrow(() -> new OrderNotFoundException(id));
    }
}
```

CreateOrderRequest.java JAVA

```java
package shop.orders;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

record CreateOrderRequest(@NotBlank String sku, @Min(1) int quantity) { }
```

OrderView.java JAVA

```java
package shop.orders;

import java.time.Instant;

record OrderView(long id, String sku, int quantity, Instant createdAt) { }
```

ApiExceptionHandler.java JAVA

```java
package shop.orders;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
class ApiExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException ex) {
        ProblemDetail problem =
                ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Order not found");
        return problem;
    }
}
```

The slice test loads only the controller, the advice and the MVC infrastructure. `OrderService` is replaced by a Mockito mock.

OrderControllerTest.java JAVA

```java
package shop.orders;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.BDDMockito.given;

import java.time.Instant;
import java.util.Optional;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.assertj.MockMvcTester;

@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired MockMvcTester mvc;
    @MockitoBean OrderService orders;

    @Test
    void createReturns201WithLocation() {
        given(orders.create(any())).willReturn(new OrderView(7, "SKU-1", 2, Instant.EPOCH));

        assertThat(mvc.post().uri("/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content("{\"sku\":\"SKU-1\",\"quantity\":2}"))
            .hasStatus(201)
            .hasHeader("Location", "/orders/7")
            .bodyJson().extractingPath("$.sku").isEqualTo("SKU-1");
    }
```

OrderControllerTest.java (continued) JAVA

```
    @Test
    void invalidBodyReturns400() {
        assertThat(mvc.post().uri("/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content("{\"sku\":\"\",\"quantity\":0}"))
            .hasStatus(400);
    }

    @Test
    void unknownOrderReturnsProblemDetail() {
        given(orders.find(99L)).willReturn(Optional.empty());

        assertThat(mvc.get().uri("/orders/{id}", 99))
            .hasStatus(404)
            .bodyJson().extractingPath("$.title").isEqualTo("Order not found");
    }
}
```

What to notice: no `@SpringBootTest`, hence no database and no unrelated beans; the `400` test needs no stubbing because validation fails before the service is called; the `404` test verifies the advice and the `ProblemDetail` serialisation together. Content type `application/problem+json` is also asserted in a complete suite (`hasContentType`).

Persistence slice with a real database

`@DataJpaTest` configures an in-memory database by default (`@AutoConfigureTestDatabase(replace = ANY)`), wraps each test in a transaction that is rolled back, and exposes `TestEntityManager`. The default replacement is a hazard when production uses PostgreSQL, as discussed in the problems below. With Testcontainers the replacement is switched off so that the container database is used.

OrderRepositoryTest.java JAVA

```
package shop.orders;

import static org.assertj.core.api.Assertions.assertThat;

import java.util.List;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Testcontainers
class OrderRepositoryTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired TestEntityManager em;
    @Autowired OrderRepository repository;

    @Test
    void findsOrdersBySkuIgnoringCase() {
        em.persist(new OrderEntity("SKU-1", 2));
        em.persist(new OrderEntity("sku-2", 1));
        em.flush();
        em.clear();

        List<OrderEntity> found = repository.findBySkuIgnoreCase("SKU-2");

        assertThat(found).extracting(OrderEntity::getSku).containsExactly("sku-2");
    }
}
```

The calls `em.flush()` and `em.clear()` are essential: without them the persistence context returns the managed instance from the first-level cache and the test never reads the database, so a wrong query or a mapping error remains undetected.

Two further slices are small and fast. `@JsonTest` verifies serialisation contracts (field names, date formats, `@JsonIgnore`) through `JacksonTester<T>` without any web layer. `@RestClientTest(PaymentClient.class)` binds a `MockRestServiceServer` to the client's `RestClient.Builder` so that URL, headers and response mapping of an outgoing call are verified with canned responses.

5.2.3 Hands-on

Running a single slice test and inspecting the context start-up is the quickest way to learn what a slice contains. The condition evaluation report lists the auto-configurations that matched; for a web slice, no `DataSourceAutoConfiguration` entry appears under "Positive matches".

Typical output (not run on the build machine) TERMINAL

```
$ ./mvnw -q test -Dtest=OrderControllerTest -Ddebug
...
Positive matches:
-----------------
   JacksonAutoConfiguration matched:
      - @ConditionalOnClass found required class 'com.fasterxml.jackson.databind.ObjectMapper'
   WebMvcAutoConfiguration matched:
      - @ConditionalOnWebApplication (required) found 'session' scope
...
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.9 s
```

The first test class that uses a given configuration pays the start-up cost (about two seconds here); later classes with an identical key report milliseconds. To see cache behaviour, enable the logger `org.springframework.test.context.cache` at `DEBUG`: each lookup logs the cache statistics (`size`, `hits`, `misses`), and the ratio of misses to test classes is the number to reduce.

src/test/resources/application-test.properties PROPERTIES

```
logging.level.org.springframework.test.context.cache=DEBUG
spring.test.context.cache.maxSize=32
```

5.2.4 Real-world problems and solutions

Problem 1: The test suite takes twelve minutes because every class starts its own context

Scenario. A service has 60 integration test classes. Each class declares its own `@MockitoBean` set or `@TestPropertySource`, and the pipeline spends most of its time starting the application.

Symptoms. The log shows the Spring banner dozens of times; the Surefire report lists per-class times of five to nine seconds for the first test and milliseconds for the rest; JVM memory grows during the run because up to 32 contexts stay alive.

Diagnosis. (1) Count banners: `grep -c "Started .* in" target/surefire-reports/*-output.txt`. (2) Enable the cache logger shown above and compare the `misses` figure with the number of classes. (3) For two classes that should share a context, print the differences: unequal `@MockitoBean` sets, `@ActiveProfiles` order, `@TestPropertySource` values, `@Import` lists and `@DirtiesContext` usage are the usual causes.

Root cause. The cache key includes every configuration element, so cosmetic differences (a property set in one class only, a mock declared in one class only) create separate contexts. `@DirtiesContext` discards the context after the class or method, forcing a rebuild.

Fix. Move shared declarations to an abstract base class or a meta-annotation so that all classes present an identical key; declare mocks needed by few tests on a `@TestConfiguration`-free slice instead of the full context; replace `@DirtiesContext` with explicit cleanup (truncate tables, reset a fake) wherever possible. Where a few tests need distinct configuration, group them in a separate Failsafe execution so the common contexts are not evicted.

AbstractIntegrationTest.java JAVA

```java
package shop.support;

import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import shop.payment.PaymentClient;

@SpringBootTest
@ActiveProfiles("test")
public abstract class AbstractIntegrationTest {

    @MockitoBean
    protected PaymentClient paymentClient;     // declared once: one cache key for all subclasses
}
```

Prevention. Track the number of context start-ups per build as a metric; fail the build when it exceeds an agreed budget (for example five). Prefer slices to `@SpringBootTest` for new tests.

Problem 2: A web slice test fails at start-up with a missing bean

Scenario. A new `@WebMvcTest` for a controller fails immediately after a developer adds a second dependency (`AuditService`) to the controller constructor.

Symptoms. `UnsatisfiedDependencyException: Error creating bean with name 'orderController' ... No qualifying bean of type 'shop.audit.AuditService'`.

Diagnosis. Read the innermost cause in the trace; it names the type. Confirm that the type is a `@Service` or `@Component`, which a web slice never scans.

Root cause. Slices include only the stereotypes of their layer. The new collaborator is outside the slice.

Fix. Declare `@MockitoBean AuditService audit;` in the test. If the collaborator is configuration-heavy and not mockable (for example a `Clock` or `PasswordEncoder` bean), supply it through `@Import(TestBeans.class)` or a nested `@TestConfiguration` class. Do not switch to `@SpringBootTest` to avoid the error: that hides the dependency and slows the test.

Prevention. Keep controllers thin so that their dependency list stays short; a controller with five collaborators is a design signal.

Problem 3: A mock is injected into the wrong place, or the test sees the real bean

Scenario. A test declares `@MockitoBean PaymentClient client` but the service under test still calls the real HTTP client, or two beans of the same type exist and the wrong one is replaced.

Symptoms. Real network calls from tests, or a mock that is never invoked (`Wanted but not invoked`).

Diagnosis. (1) Check whether the production code constructs the collaborator with `new` rather than receiving it from the container; a mock cannot replace an object that is not a bean. (2) List bean definitions of the type: `ctx.getBeanNamesForType(PaymentClient.class)`. (3) With two candidates, `@MockitoBean` without a name fails on ambiguity or replaces the primary candidate, depending on the version.

Root cause. Bean override works on the application context only. Objects created with `new`, static factories and singletons outside Spring are invisible to it.

Fix. Obtain collaborators through constructor injection; when two beans exist, name the one to replace: `@MockitoBean(name = "primaryPaymentClient")`.

Prevention. Forbid `new` for infrastructure collaborators in services; the design that makes mocking possible is the same design that makes the class testable without Spring.

Problem 4: Tests alter each other through a shared cached context

Scenario. A test class changes a singleton bean (for example, sets a feature flag in an in-memory holder) and the next class, sharing the cached context, fails.

Symptoms. Order-dependent failures that disappear when the class runs alone.

Diagnosis. Run the two classes in both orders. Search for tests that mutate bean state (`ReflectionTestUtils.setField`, `holder.set(...)`, caches filled by the test).

Root cause. A cached context shares singleton beans across all classes that use it; state in those beans persists.

Fix. Reset state in `@AfterEach`, or prefer `@DirtiesContext` only for the one class that cannot clean up. Cache and holder beans should expose a reset method for tests, or the test should use a fresh instance supplied through `@TestConfiguration`.

Prevention. Treat singleton beans in tests as shared infrastructure: tests may read them freely and must restore any change they make.

5.2.5 Questions to expect

What is the difference between @SpringBootTest and @WebMvcTest?

`@SpringBootTest` starts the whole application context (optionally with a real server). `@WebMvcTest` starts only the MVC layer: controllers, advice, converters, filters and MVC auto-configuration, with collaborators supplied as mocks. The slice is faster and checks only web behaviour.

How does the test context cache work and what invalidates it?

The `TestContext` framework caches started contexts in a map keyed by the merged context configuration (classes, profiles, properties, customizers such as mock declarations). A different key produces a new context. `@DirtiesContext` removes the context from the cache. The default capacity is 32 contexts with LRU eviction.

Why can @MockitoBean slow down a test suite?

Each distinct set of declared overrides is part of the cache key. Tests that declare different mocks cannot share a context, so each combination costs one context start-up.

What does MockMvc test, and what does it not test?

It tests everything inside the DispatcherServlet pipeline: filters, mapping, binding, validation, conversion and exception handling. It does not start a servlet container, so error pages, container configuration, compression and real HTTP behaviour are untested.

Why are flush() and clear() needed in a repository test?

Writes are buffered in the persistence context until a flush, and reads are served from its first-level cache. Without flush and clear, the test verifies the cache rather than the SQL and the mapping.

How is a controller secured in a slice test?

`@WebMvcTest` includes Spring Security; tests use `@WithMockUser` or the `jwt()` request post-processor from `spring-security-test`, and `csrf()` for state-changing requests. Custom `SecurityFilterChain` beans must be imported with `@Import`, because configuration classes are outside the scan filter.

What replaced @MockBean?

`@MockitoBean` and `@MockitoSpyBean` from Spring Framework 6.2 (usable from Boot 3.4). The older annotations are deprecated.

KEY POINTS

Key points

- A slice test starts one layer: web, persistence, JSON or HTTP client. Everything else is mocked or imported explicitly.
- The context cache key is the full merged configuration; a different mock set, property or profile means a new context start-up.
- Reduce distinct configurations with a shared base class or meta-annotation, and avoid `@DirtiesContext` unless state cannot be reset.
- `@MockitoBean` replaces `@MockBean` from Boot 3.4; mocks only replace beans, not objects created with `new`.
- MockMvc exercises the whole MVC pipeline without a server; use a random-port test when container behaviour matters.
- In repository tests, flush and clear the persistence context before asserting, and use the production database engine.

sferam

5.3 Integration Testing with Testcontainers

An integration test verifies that the application works together with the systems it depends on: the relational database with its real SQL dialect and constraints, the message broker with its real delivery semantics, the cache with its real eviction behaviour. Embedded substitutes (H2 for PostgreSQL, an in-memory fake for Kafka) are fast but differ in exactly the places where production defects occur. Testcontainers closes this gap by starting the real software in a throw-away Docker container from inside the test run. Spring Boot 3.1 added first-class support (`@ServiceConnection`), so that a container replaces connection properties without manual wiring. This chapter covers the mechanism, the configuration patterns that keep a suite fast, stubbing of HTTP dependencies, contract testing and security tests.

5.3.1 Concept and mechanism

How Testcontainers works

Testcontainers is a Java library that talks to the Docker Engine API through the socket (`/var/run/docker.sock`, or `DOCKER_HOST`). For each `GenericContainer` or specialised subclass (`PostgreSQLContainer`, `KafkaContainer`) the library performs these steps.

1. Pulls the image if absent (the first run on a build agent can take a minute).
2. Creates and starts the container with a random host port mapped to each exposed container port, which avoids port conflicts between parallel builds.
3. Evaluates a **wait strategy** until the service is ready: a log line (`Wait.forLogMessage`), a listening port, an HTTP endpoint or a health check. A container whose process has started is not necessarily ready; the wait strategy is the readiness contract.
4. Exposes the mapped host, port and credentials (`getJdbcUrl()`, `getBootstrapServers()`).
5. Registers the container with **Ryuk**, a sidecar container that deletes containers, networks and volumes created by the JVM when the JVM exits, even after a crash.

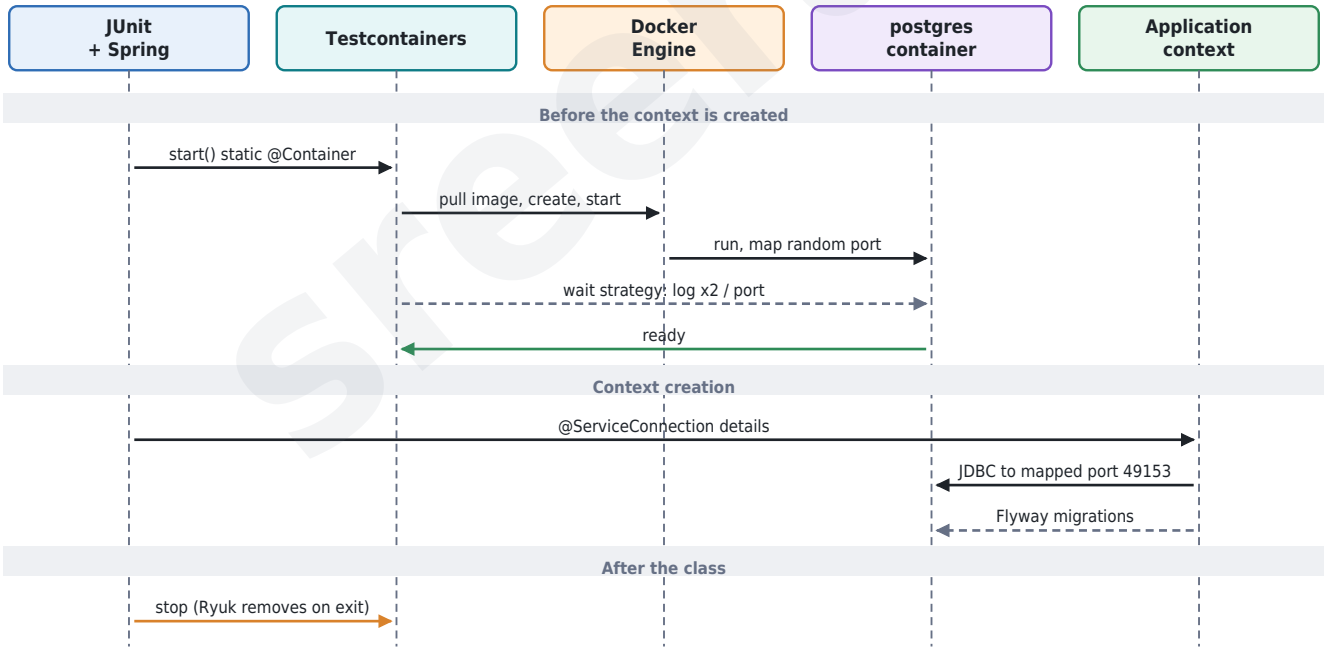


Figure 5.7. Start-up sequence of a Testcontainers-backed Spring test: the container is started and awaited before the application context is created, and its mapped port reaches the context through connection details.

Wiring the container into Spring Boot

Three mechanisms exist, in decreasing order of preference.

`@ServiceConnection` (Boot 3.1 and later; package `org.springframework.boot.testcontainers.service.connection`). Annotating a container field or `@Bean` method makes Boot create a `ConnectionDetails` bean (for example `JdbcConnectionDetails`, `KafkaConnectionDetails`) that the auto-configuration uses instead of the property values. No property names appear in the test, so the test remains valid when property names change. Supported types include `PostgreSQLContainer`, `MySQLContainer`, `KafkaContainer`,

`MongoDBContainer`, `RedisContainer`; a plain `GenericContainer` needs `@ServiceConnection(name = "redis")` so Boot can determine the service. The dependency `spring-boot-testcontainers` is required (test scope).

`@DynamicPropertySource` registers properties computed after the container starts. It is more verbose and works for any container and any property, including application-specific ones.

`@ImportTestcontainers` imports container declarations from an interface, which allows one declaration to be shared by many test classes and by the development-time launcher.

A container declared as a `static` field with the Jupiter `@Container` annotation is started before the first test of the class and stopped after the last. Container beans (`@Bean @ServiceConnection PostgreSQLContainer<?>`) are managed by the Spring context instead: they start with the context and stop with it, which matches the context cache lifetime and avoids a stopped container behind a cached context.

The singleton container pattern

Starting a PostgreSQL container takes two to five seconds. A container per test class multiplies that cost by the number of classes. The singleton pattern starts one container per JVM, in a shared holder, and shares it among all classes; tests isolate themselves through unique data or by truncating tables.

Containers.java JAVA

```java
package shop.support;

import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.context.annotation.Bean;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.utility.DockerImageName;

@TestConfiguration(proxyBeanMethods = false)
public class Containers {

    @Bean
    @ServiceConnection
    PostgreSQLContainer<?> postgres() {
        return new PostgreSQLContainer<>(DockerImageName.parse("postgres:16-alpine"));
    }
}
```

Because the container is a bean of a configuration that is imported by the shared base class, every subclass presents the same context key and therefore the same container. The two optimisations compose: one cached context and one container.

5.3.2 Worked example

The example verifies an order repository, a Flyway migration and the HTTP endpoint against real PostgreSQL, a Kafka producer against a real broker, and an outgoing payment call against a stub.

Shared base class

AbstractIntegrationTest.java JAVA

```java
package shop.support;

import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.context.SpringBootTest.WebEnvironment;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.ActiveProfiles;

@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
@Import(Containers.class)
public abstract class AbstractIntegrationTest {
}
```

sreeram

Database, migration and endpoint

OrderApiIT.java JAVA

```java
package shop.orders;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.http.ResponseEntity;
import org.springframework.jdbc.core.JdbcTemplate;
import shop.support.AbstractIntegrationTest;

class OrderApiIT extends AbstractIntegrationTest {

    @Autowired TestRestTemplate http;
    @Autowired JdbcTemplate jdbc;

    @BeforeEach
    void clean() {
        jdbc.execute("TRUNCATE TABLE orders RESTART IDENTITY CASCADE");
    }

    @Test
    void createdOrderIsPersisted() {
        ResponseEntity<OrderView> response = http.postForEntity(
                "/orders", new CreateOrderRequest("SKU-1", 3), OrderView.class);

        assertThat(response.getStatusCode().value()).isEqualTo(201);
        Integer rows = jdbc.queryForObject(
                "SELECT count(*) FROM orders WHERE sku = 'SKU-1' AND quantity = 3", Integer.class);
        assertThat(rows).isEqualTo(1);
    }

    @Test
    void flywayAppliedAllMigrations() {
        Integer failed = jdbc.queryForObject(
                "SELECT count(*) FROM flyway_schema_history WHERE success = false", Integer.class);
        assertThat(failed).isZero();
    }
}
```

Because the server runs in its own thread, a `@Transactional` test method does **not** roll back what the server wrote (the HTTP handler uses a different connection and transaction). Cleanup is therefore explicit: `TRUNCATE ... RESTART IDENTITY CASCADE` in `@BeforeEach`.

sreeram

Kafka with a real broker

ShipmentPublisherIT.java JAVA

```java
package shop.shipping;

import static org.assertj.core.api.Assertions.assertThat;

import java.time.Duration;
import java.util.List;
import java.util.Map;
import org.apache.kafka.clients.consumer.Consumer;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.kafka.core.DefaultKafkaConsumerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.test.utils.KafkaTestUtils;
import org.testcontainers.containers.KafkaContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;

@SpringBootTest
@Testcontainers
class ShipmentPublisherIT {

    @Container
    @ServiceConnection
    static KafkaContainer kafka = new KafkaContainer(DockerImageName.parse("apache/kafka:3.8.0"));

    @Autowired ShipmentPublisher publisher;

    @Test
    void publishesShipmentEventWithOrderIdAsKey() {
        publisher.publish(new ShipmentEvent("order-42", "DISPATCHED"));

        Map<String, Object> props = KafkaTestUtils.consumerProps(
                kafka.getBootstrapServers(), "it-group", "true");
        props.put("auto.offset.reset", "earliest");
        try (Consumer<String, String> consumer = new DefaultKafkaConsumerFactory<>(
                props, new StringDeserializer(), new StringDeserializer()).createConsumer()) {
            consumer.subscribe(List.of("shipments"));
            ConsumerRecord<String, String> record =
                    KafkaTestUtils.getSingleRecord(consumer, "shipments", Duration.ofSeconds(10));
            assertThat(record.key()).isEqualTo("order-42");
            assertThat(record.value()).contains("DISPATCHED");
        }
    }
}
```

The assertion on the **key** matters: partition assignment follows the key, so ordering guarantees depend on it. A mocked `KafkaTemplate` would never reveal a wrong key. The Kafka container class and image name depend on the Testcontainers version (`org.testcontainers.containers.KafkaContainer` accepts `apache/kafka` images from Testcontainers 1.20; older versions use `confluentinc/cp-kafka`); the version in the build file governs.

Stubbing an HTTP dependency

Calls to third-party HTTP services are not made in tests. WireMock runs an HTTP server inside the test JVM (or in a container) and answers from declared stubs, including delays and fault injection. `@AutoConfigureWireMock(port = 0)` from Spring Cloud Contract WireMock starts it on a random port and exposes it as the property `wiremock.server.port`.

PaymentClientWireMockIT.java JAVA

```java
package shop.payment;

import static com.github.tomakehurst.wiremock.client.WireMock.aResponse;
import static com.github.tomakehurst.wiremock.client.WireMock.post;
import static com.github.tomakehurst.wiremock.client.WireMock.stubFor;
import static com.github.tomakehurst.wiremock.client.WireMock.urlEqualTo;
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
```

PaymentClientWireMockIT.java (continued) JAVA

```java
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.cloud.contract.wiremock.AutoConfigureWireMock;
import org.springframework.test.context.TestPropertySource;

@SpringBootTest
@AutoConfigureWireMock(port = 0)
@TestPropertySource(properties = {
        "payment.base-url=http://localhost:${wiremock.server.port}",
        "payment.read-timeout=300ms"})
class PaymentClientWireMockIT {

    @Autowired PaymentClient client;

    @Test
    void mapsSuccessfulResponse() {
        stubFor(post(urlEqualTo("/payments")).willReturn(aResponse()
                .withStatus(200).withHeader("Content-Type", "application/json")
                .withBody("{\"status\":\"OK\"}")));

        assertThat(client.charge("acc-1", 1000).status()).isEqualTo("OK");
    }

    @Test
    void slowUpstreamTriggersTimeout() {
        stubFor(post(urlEqualTo("/payments")).willReturn(aResponse()
                .withStatus(200).withFixedDelay(2000)));

        assertThatThrownBy(() -> client.charge("acc-1", 1000))
                .hasRootCauseInstanceOf(java.net.SocketTimeoutException.class);
    }
}
```

The mechanism of such a stub is small enough to reproduce with the JDK's built-in `com.sun.net.httpserver.HttpServer`. The program below serves three scripted responses (slow, then 503, then 200) and calls them with a client whose per-request timeout is 150 ms; it demonstrates what a timeout test and a fault-injection test verify. The server uses a four-thread executor: with the default single-threaded dispatcher the slow first response blocks the later ones, which is a frequent source of misleading timeouts in hand-made stubs.

StubServerDemo.java JAVA

```java
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.*;
import java.time.Duration;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class StubServerDemo {
    public static void main(String[] args) throws Exception {
        AtomicInteger calls = new AtomicInteger();
        HttpServer stub = HttpServer.create(new InetSocketAddress("127.0.0.1", 21501), 0);
        stub.createContext("/payments", ex -> {
            int n = calls.incrementAndGet();
            try {
                if (n == 1) Thread.sleep(400);                // first call: slow upstream
                byte[] body = (n == 2 ? "{\"error\":\"unavailable\"}"
                                      : "{\"status\":\"OK\"}").getBytes();
                ex.getResponseHeaders().add("Content-Type", "application/json");
                ex.sendResponseHeaders(n == 2 ? 503 : 200, body.length);
                ex.getResponseBody().write(body);
            } catch (InterruptedException | IOException ignored) {
            } finally { ex.close(); }
        });
        stub.setExecutor(Executors.newFixedThreadPool(4));
        stub.start();

        HttpClient client = HttpClient.newBuilder().connectTimeout(Duration.ofMillis(200)).build();
        for (int i = 1; i <= 3; i++) {
```

StubServerDemo.java (continued) JAVA

```
            HttpRequest req = HttpRequest.newBuilder(URI.create("http://127.0.0.1:21501/payments"))
                    .timeout(Duration.ofMillis(150))
                    .POST(HttpRequest.BodyPublishers.noBody()).build();
            long t0 = System.nanoTime();
            String outcome;
            try {
                HttpResponse<String> r = client.send(req, HttpResponse.BodyHandlers.ofString());
                outcome = r.statusCode() + " " + r.body();
            } catch (HttpTimeoutException e) {
                outcome = "timeout after ~150 ms (" + e.getClass().getSimpleName() + ")";
            }
            System.out.printf("call %d -> %s [%d ms]%n", i, outcome,
                    (System.nanoTime() - t0) / 1_000_000);
        }
        stub.stop(0);
        System.exit(0);
    }
}
```

Output (real run; durations vary with JIT warm-up) TERMINAL

```
$ java StubServerDemo.java
call 1 -> timeout after ~150 ms (HttpTimeoutException) [172 ms]
call 2 -> 503 {"error":"unavailable"} [106 ms]
call 3 -> 200 {"status":"OK"} [51 ms]
```

Contract testing and security tests

When two services are developed by different teams, a stub written by the consumer may drift from the real provider. **Consumer-driven contract testing** records the interactions the consumer relies on as a contract; the provider build verifies that it satisfies every contract, and the consumer build receives stubs generated from the same contract. Two toolchains are common: Spring Cloud Contract (contracts in a Groovy, YAML or Kotlin DSL, provider tests generated automatically, stubs published as a `-stubs` jar and consumed through `@AutoConfigureStubRunner`) and Pact (language-neutral JSON pacts exchanged through a broker). Both detect a breaking change in a provider (a renamed field, a removed status) in the provider's build, before deployment. They complement, and do not replace, a small number of end-to-end checks.

Security rules are verified by requests with a security context. `spring-security-test` supplies `@WithMockUser(roles = "ADMIN")`, `SecurityMockMvcRequestPostProcessors.jwt()` for resource servers and `csrf()` for form-based applications. A test suite for protected endpoints asserts three outcomes per endpoint: anonymous is rejected (`401`), authenticated without the right authority is forbidden (`403`), authorised succeeds.

AdminEndpointSecurityTest.java JAVA

```
package shop.admin;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.delete;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import shop.config.SecurityConfig;

@WebMvcTest(AdminController.class)
@Import(SecurityConfig.class)
class AdminEndpointSecurityTest {

    @Autowired MockMvc mvc;
    @MockitoBean AdminService admin;
    @MockitoBean org.springframework.security.oauth2.jwt.JwtDecoder jwtDecoder;

    @Test
    void anonymousIsUnauthorized() throws Exception {
        mvc.perform(delete("/admin/users/5")).andExpect(status().isUnauthorized());
    }
```

AdminEndpointSecurityTest.java (continued) JAVA

```java
    @Test
    void userWithoutAuthorityIsForbidden() throws Exception {
        mvc.perform(delete("/admin/users/5")
                .with(SecurityMockMvcRequestPostProcessors.jwt()
                        .authorities(() -> "SCOPE_orders:read")))
            .andExpect(status().isForbidden());
    }

    @Test
    void adminSucceeds() throws Exception {
        mvc.perform(delete("/admin/users/5")
                .with(SecurityMockMvcRequestPostProcessors.jwt()
                        .authorities(() -> "ROLE_ADMIN")))
            .andExpect(status().isNoContent());
    }
}
```

5.3.3 Hands-on

Container-based tests need a reachable Docker daemon. On a developer machine, any Docker-compatible runtime works; on CI the choice affects speed and configuration (see the problems below). The command line does not change:

Typical output (not run on the build machine) TERMINAL

```
$ ./mvnw verify -Dit.test=OrderApiIT
INFO  o.t.d.DockerClientProviderStrategy : Found Docker environment with local Unix socket
INFO  tc.postgres:16-alpine : Creating container for image: postgres:16-alpine
INFO  tc.postgres:16-alpine : Container postgres:16-alpine is starting: 3f9c1d...
INFO  tc.postgres:16-alpine : Container postgres:16-alpine started in PT2.7S
INFO  o.f.c.i.s.JdbcTableSchemaHistory   : Creating Schema History table "flyway_schema_history"
INFO  o.f.core.internal.command.DbMigrate : Successfully applied 4 migrations to schema "public"
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.9 s
```

Container reuse speeds up the local edit-run loop. It requires both `withReuse(true)` on the container and `testcontainers.reuse.enable=true` in `~/.testcontainers.properties`; reused containers are not removed by Ryuk and carry state between runs, so it is a developer convenience and is not enabled in CI. Spring Boot's development-time support (`SpringApplication.from(Application::main).with(Containers.class).run(args)`, with `@RestartScope` on container beans) shares the same container declarations for local runs.

5.3.4 Real-world problems and solutions

Problem 1: Tests pass against H2 and fail against PostgreSQL in production

Scenario. Repository tests run on an in-memory H2 database (the default replacement of `@DataJpaTest`) and are green. After deployment, a query using a PostgreSQL operator fails, and a duplicate insert leaves a request in a broken state.

Symptoms. `PSQLException: ERROR: current transaction is aborted, commands ignored until end of transaction block` in production logs; a native query with `->>` or `ILIKE` that cannot even be prepared on H2; `jsonb` columns that the H2-based schema generation silently maps differently.

Diagnosis. Compare the dialects. The statements below were executed on a real PostgreSQL 16 server and show behaviour that an embedded database must emulate.

dialect_check.sql SQL

```sql
CREATE TABLE product (id serial PRIMARY KEY, name text NOT NULL UNIQUE, attrs jsonb);
INSERT INTO product(name, attrs) VALUES ('Keyboard', '{"color":"black","keys":104}');
INSERT INTO product(name, attrs) VALUES ('Mouse', '{"color":"white"}');
SELECT name FROM product WHERE attrs->>'color' = 'black';
SELECT name FROM product WHERE name ILIKE 'key%';
BEGIN;
INSERT INTO product(name) VALUES ('Mouse');
SELECT count(*) FROM product;
ROLLBACK;
SELECT name FROM product WHERE name LIKE 'key%';
```

Real output from PostgreSQL 16 TERMINAL

```
$ psql -f dialect_check.sql
CREATE TABLE
INSERT 0 1
INSERT 0 1
   name
----------
 Keyboard
(1 row)

   name
----------
 Keyboard
(1 row)

BEGIN
ERROR:  duplicate key value violates unique constraint "product_name_key"
DETAIL:  Key (name)=(Mouse) already exists.
ERROR:  current transaction is aborted, commands ignored until end of transaction block
ROLLBACK
 name
------
(0 rows)
```

Three facts follow. The `jsonb` type, the `->>` operator and `ILIKE` are PostgreSQL features; an embedded engine supports them only partially or through a compatibility mode. The `LIKE` comparison is case-sensitive in PostgreSQL (zero rows for `'key%'`), whereas other engines or collations may be case-insensitive. After a failed statement inside a transaction, PostgreSQL rejects every further statement until rollback; code that catches a constraint violation and continues inside the same transaction works on engines that do not abort the transaction and fails on PostgreSQL. (The H2-side behaviour was not executed on the build machine; it is stated from the H2 documentation, and the version in use should be checked.)

Root cause. The test database is not the production database; the compatibility mode (`MODE=PostgreSQL`) emulates a subset only.

Fix. Use the production engine in tests: Testcontainers with the same major version as production (`postgres:16`). Run the real Flyway migrations rather than `spring.jpa.hibernate.ddl-auto=create`, so that migration scripts are tested too. Catch constraint violations in a separate transaction (`REQUIRES_NEW`) or avoid catching them.

Prevention. Ban H2 in the test classpath by a dependency rule; keep the container image version in one place (a constant or the `Containers` configuration) and update it together with production.

Behaviour	Embedded engine (H2)	PostgreSQL 16 (production)
jsonb type, ->> operator	partial / mode-dependent	native
ILIKE, DISTINCT ON	not standard	supported
error inside a transaction	see H2 docs (not run)	transaction aborted
LIKE case sensitivity	depends on setting	case-sensitive
SKIP LOCKED, advisory locks	limited	supported
DDL generated by ddl-auto	H2 dialect types	Flyway scripts

Figure 5.8. Where an embedded database and the production database diverge: type system, operators, transaction abort behaviour and locking features are the usual sources of tests that pass for the wrong reason.

Problem 2: The pipeline fails randomly with "Connection refused" while starting containers

Scenario. On a shared CI runner, one build in ten fails with `Connection to localhost:49153 refused` or `Could not connect to Ryuk`, and with an authentication error against the database in the first test.

Symptoms. The failure occurs in the first test that touches the database; re-running the job succeeds. Slow runners fail more often.

Diagnosis. (1) Read the container log in the build output (`Container postgres:16-alpine started in PT...`): a start time near the wait timeout indicates an overloaded runner. (2) Check whether the wait strategy is satisfied by the wrong event. PostgreSQL logs "database system is ready to accept connections" twice during initialisation; a wait strategy that matches only one occurrence is satisfied while the init script is still running. The official `PostgreSQLContainer` waits for two occurrences. A hand-written `GenericContainer` for a database needs the same care. (3) On Docker-in-Docker runners, check that the host name used to reach mapped ports is correct (`TESTCONTAINERS_HOST_OVERRIDE`).

Root cause. The test starts before the service is ready, because the wait strategy is missing, wrong, or its timeout is shorter than the container start time under load.

Fix. Use the specialised container class, or define an explicit strategy and timeout:

Waits.java — JAVA

```java
package shop.support;

import java.time.Duration;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.containers.wait.strategy.Wait;

final class Waits {
    static GenericContainer<?> redis() {
        return new GenericContainer<>("redis:7-alpine")
                .withExposedPorts(6379)
                .waitingFor(Wait.forLogMessage(".*Ready to accept connections.*\\n", 1)
                        .withStartupTimeout(Duration.ofSeconds(60)));
    }
}
```

Prevention. Pre-pull images in the runner image or a registry mirror; cap parallel jobs per runner; use a CI service that offers a Docker daemon with enough memory (databases and brokers need 1 GB or more each); keep one container per JVM rather than per class.

Problem 3: Integration tests leave data behind and fail in a different order

Scenario. Two integration tests pass individually. Run together, the second fails with a unique-key violation or an unexpected row count.

Symptoms. Failures depend on class order; a test that asserts `count == 1` returns 2.

Diagnosis. Check whether the failing test is `@Transactional`: on `RANDOM_PORT` tests the server transaction commits, so rollback does not apply. Query the table after the first test to see the leftovers.

Root cause. The data written through the HTTP layer commits; the test's own transaction cannot undo it.

Fix. Clean explicitly before each test (`TRUNCATE ... RESTART IDENTITY CASCADE`), or give each test unique keys (random SKU, unique e-mail) and assert on those only. For `MOCK` web environment tests, in-process calls share the test transaction and rollback works.

Prevention. Decide on one isolation strategy per suite and document it in the base class; do not mix transactional rollback and explicit cleanup in the same suite.

Problem 4: Outgoing HTTP calls reach a real third-party service from CI

Scenario. A test of the checkout flow calls the real payment sandbox and fails whenever the sandbox is slow or rate-limited.

Symptoms. Failures coincide with external outages; the log shows real host names; builds fail for network-restricted runners.

Diagnosis. Check the effective property `payment.base-url` in the test context (`/actuator/env` in a running test, or a debug assertion on the `Environment`). Search the test classpath for production URLs in `application.yml`.

Root cause. The test profile does not override the client base URL, so the production-default URL is used.

Fix. Point the URL to a WireMock instance (`@AutoConfigureWireMock`) and simulate faults (delay, 503, malformed body) as shown above. Block accidental outbound traffic by setting the default URL in `application-test.properties` to an unroutable address (`http://localhost:1`), which converts a missing override into an immediate failure.

Prevention. Keep third-party base URLs out of defaults; add a contract test so that the stubs track the real provider.

sreeram

5.3.5 Questions to expect

Why use Testcontainers instead of H2?

The embedded database differs from the production database in types, operators, SQL functions, locking and transaction semantics, so tests can pass for wrong reasons. A container runs the real engine and version, and also verifies the migration scripts.

What does @ServiceConnection do?

It creates a `ConnectionDetails` bean from the running container so that the auto-configuration connects to it, replacing connection properties. The test needs no property names and stays correct when the container port is random.

How is container start-up cost controlled?

One container per JVM (singleton or bean-scoped container imported by a shared base class), a shared cached context, images pre-pulled on the runner, and optional reuse in local development.

Why does @Transactional not roll back in a RANDOM_PORT test?

The client and the server handle the request on different threads and connections, so the server's transaction commits independently of the test's transaction.

What is Ryuk?

A Testcontainers sidecar that removes containers, networks and volumes created by a test JVM when the JVM terminates, including after a crash.

What is the benefit of consumer-driven contracts?

The provider build fails when it breaks an expectation of any consumer, and consumers test against stubs generated from the same contract, so stubs cannot drift from reality.

When is WireMock preferable to MockRestServiceServer?

WireMock exercises the complete HTTP stack, including timeouts, connection handling, serialisation and fault injection. `MockRestServiceServer` replaces the request factory, so it verifies mapping logic quickly but not network behaviour.

How are authorisation rules tested?

With request post-processors or annotations from `spring-security-test`, asserting `401` for anonymous, `403` for insufficient authority and success for an authorised principal.

KEY POINTS

Key points

- Embedded databases hide dialect, type and transaction differences; use the production engine through Testcontainers and run the real migrations.
- `@ServiceConnection` wires a container into auto-configuration without property names; container beans share the lifetime of the cached context.
- Keep suites fast: one container per JVM, one cached context, images pre-pulled, explicit data cleanup for server-side commits.
- Wait strategies define readiness; a container that has started is not necessarily ready.
- Stub HTTP dependencies with WireMock, including delays and faults, and protect the build against accidental real calls.
- Contract tests keep stubs and providers in agreement; security tests assert 401, 403 and success per endpoint.

sreeram

5.4 Actuator and Micrometer

An application that passes its tests can still fail in production, and the first question during an incident is always the same: what is the system doing now? Spring Boot Actuator answers it with operational endpoints (health, metrics, thread dumps, environment), and Micrometer provides the vendor-neutral metrics API on which those answers are built. Together they implement the first two pillars of observability, health and metrics. (Logs and traces are treated in the chapter on logging and in the Microservices part.) This chapter explains the internals of both, shows the instrumentation of application code, and develops the operational methods (RED, USE, alerting) that turn raw numbers into decisions.

5.4.1 Concept and mechanism

Endpoints, exposure and access

An Actuator endpoint is a bean annotated `@Endpoint` whose methods (`@ReadOperation`, `@WriteOperation`, `@DeleteOperation`) are technology-neutral. Adapters expose the same operation over JMX and over HTTP (`WebMvcEndpointHandlerMapping` under the base path `/actuator`). Two independent controls decide whether an endpoint is reachable: **exposure** selects the endpoints published on a technology, and **access** (Boot 3.4 and later: `management.endpoint.<id>.access`, previously `enabled`) selects what operations are permitted at all. By default only `health` is exposed over HTTP and JMX. Exposing everything is a one-line change that must never reach production unreviewed, because `env`, `heapdump`, `threaddump` and `beans` disclose secrets, memory contents and internal structure.

Endpoint	Purpose	Sensitivity
`health`	Aggregated status of dependencies; probe groups for Kubernetes	Low (details: medium)
`info`	Build, git and custom information	Low
`metrics`	Names and values of recorded meters (`/actuator/metrics/jvm.memory.used`)	Medium
`prometheus`	All meters in Prometheus text format (needs `micrometer-registry-prometheus`)	Medium
`loggers`	Read and change log levels at run time	High (write)
`env`, `configprops`	Properties of the environment; values sanitised by default	High
`threaddump`	Stack of every thread (same content as `jcmd <pid> Thread.print`)	High
`heapdump`	HPROF heap dump; access disabled by default	Critical
`httpexchanges`	Last 100 HTTP exchanges; needs an `HttpExchangeRepository` bean (name changed from `httptrace` in Boot 3)	High
`scheduledtasks`, `caches`, `mappings`, `conditions`, `startup`, `flyway`	Introspection	Medium

Frequently used endpoints (Boot 3.5).

Production practice has three parts: expose the minimum (`health`, `info`, `prometheus`, optionally `loggers`), move the management endpoints to a separate port that is not routed by the public ingress (`management.server.port=9090`), and protect what remains with Spring Security (`EndpointRequest.toAnyEndpoint()` in a dedicated `SecurityFilterChain`).

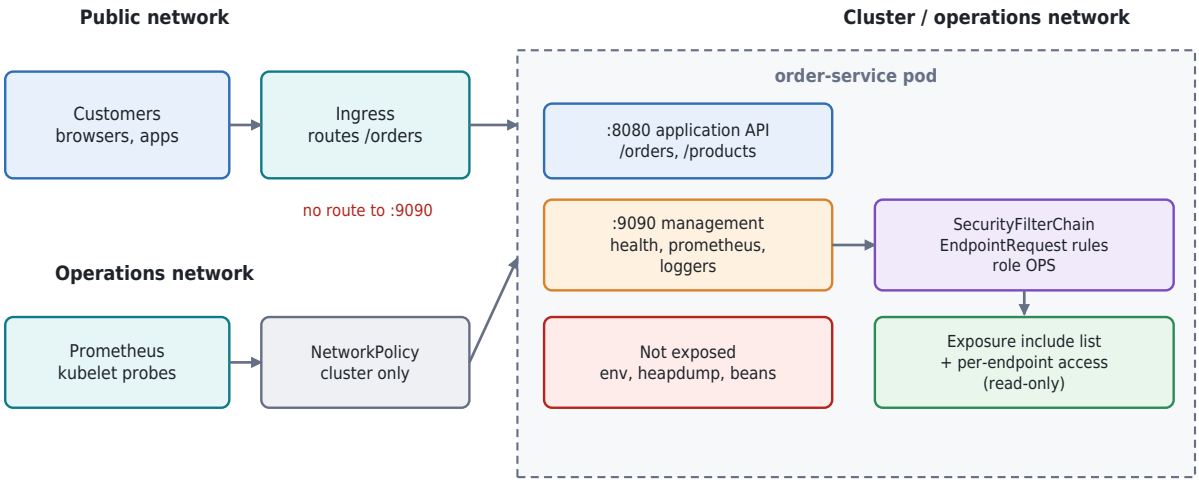


Figure 5.9. Exposure layers for management endpoints: a separate management port keeps the operational surface off the public network, and security rules plus per-endpoint access settings narrow it further.

Health: indicators, groups and probes

`HealthEndpoint` aggregates `HealthIndicator` beans (database, disk space, Redis, Kafka and custom ones) through a `StatusAggregator`. The default order is `DOWN`, `OUT_OF_SERVICE`, `UP`, `UNKNOWN`: any `DOWN` indicator makes the aggregate `DOWN`, which the HTTP adapter maps to status code 503. Details (`components`) are shown according to `management.endpoint.health.show-details` (`never` by default; `when-authorized` or `always`).

Health groups select subsets of indicators under their own path (`management.endpoint.health.group.<name>.include`). Boot defines two groups for Kubernetes probes, available as `/actuator/health/liveness` and `/actuator/health/readiness` when `management.endpoint.health.probes.enabled=true` (enabled automatically when Boot detects a Kubernetes environment). The groups are backed by `ApplicationAvailability`, which holds two states: `LivenessState` (`CORRECT` or `BROKEN`) and `ReadinessState` (`ACCEPTING_TRAFFIC` or `REFUSING_TRAFFIC`). The application changes them by publishing an `AvailabilityChangeEvent`; Boot itself switches readiness on after startup and off when graceful shutdown begins.

The two probes answer different questions, and confusing them is the most common probe defect:

- **Liveness** asks whether the process is irrecoverably broken and must be restarted. A failing liveness probe causes the kubelet to kill the container. It must therefore depend on nothing outside the process: a database outage is not cured by restarting the application, and restarting every replica at once converts a partial outage into a total one.
- **Readiness** asks whether the instance can serve requests now. A failing readiness probe removes the pod from the Service endpoints without restarting it. It may include essential dependencies, and it is the correct place to express "warming up" or "draining".
- **Startup probe** gives slow-starting applications a separate, generous budget before liveness checking begins.

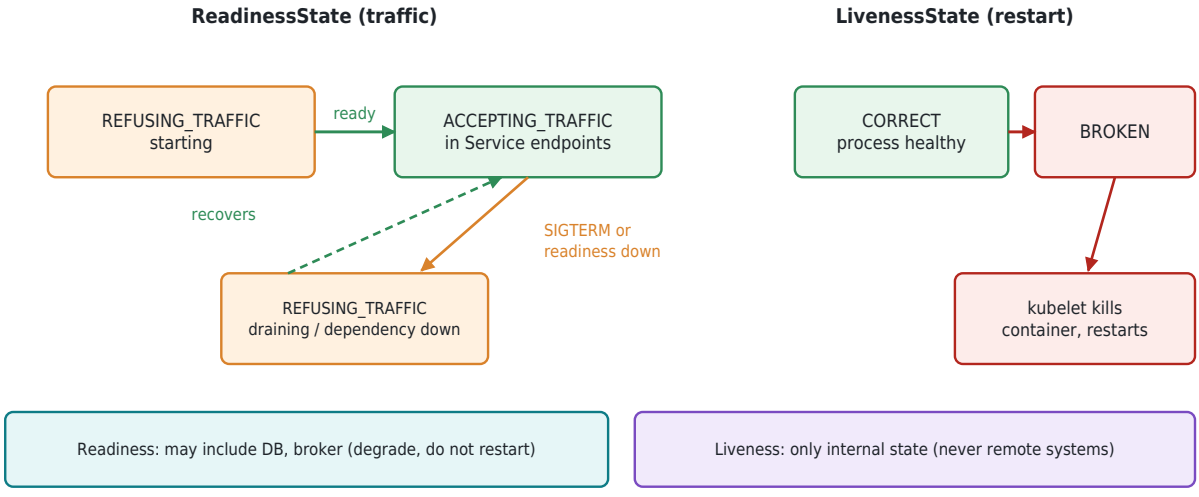


Figure 5.10. Readiness and liveness states of a pod: readiness toggles traffic, liveness triggers restarts, and graceful shutdown moves the instance to REFUSING_TRAFFIC before the process exits.

Micrometer: meters, registries and tags

Micrometer is to metrics what SLF4J is to logging: an API in front of many monitoring systems. A `MeterRegistry` creates and stores **meters**; each meter is identified by a name and a set of key-value **tags**. Boot configures a `CompositeMeterRegistry` containing one registry per dependency present on the classpath (`PrometheusMeterRegistry`, `OtlpMeterRegistry`, and so on) and binds a large set of `MeterBinder` beans: JVM memory and GC, threads, class loading, process CPU, Tomcat, HikariCP, logback event counts, `http.server.requests`, `http.client.requests`, cache statistics.

Type	Records	Typical use
`Counter`	A monotonically increasing count	Orders placed, errors, retries
`Gauge`	A current value read from a function	Queue size, active connections, cache entries
`Timer`	Count, total time and (optionally) a distribution	Latency of a method or request
`DistributionSummary`	Count, total and distribution of a non-time value	Payload size, batch size
`LongTaskTimer`	Duration of in-flight long operations	Batch jobs, exports

Meter types.

In Boot 3, instrumentation is unified under the **Observation API** (`ObservationRegistry`, `Observation`, `@Observed`): one instrumented block yields a timer metric and, when a tracer is present, a span with the same name and tags. `http.server.requests`, `http.client.requests`, `spring.data.repository.invocations` and `spring.security.*` are observations.

Tags and cardinality

Each distinct combination of tag values is a separate **time series** that the monitoring system stores and indexes. The series count of a metric is the product of the numbers of values of its tags, multiplied by the number of histogram buckets. Tags with unbounded values (user ID, order ID, raw URL, exception message, e-mail address) therefore create an unbounded number of series, exhaust memory in the metrics backend and in the application (the registry keeps every meter), and are the standard cause of "Prometheus out of memory" incidents. Spring MVC avoids the problem for URLs by tagging with the **template** (`/orders/{id}`) rather than the actual path; custom code must follow the same rule.

The Prometheus format and the pull model

Prometheus **pulls**: it requests `/actuator/prometheus` every scrape interval (commonly 15 s) from every instance, stores each sample with a timestamp and evaluates queries and alert rules on the stored series. Counters are cumulative; rates are computed at query time (`rate(x[5m])`). A latency histogram is exposed as cumulative bucket counters (`..._bucket{le="0.1"}`) plus `_sum` and `_count`; quantiles are estimated at query time with `histogram_quantile`, which allows aggregation across instances (percentiles pre-computed inside the application cannot be averaged correctly across instances).

The program below implements the core of a histogram and its exposition in plain Java. It records 2000 synthetic latencies (96 percent between 15 and 60 ms, 4 percent between 800 and 1200 ms), prints the mean and the quantiles computed exactly and from buckets, writes the result in Prometheus text format, and computes series counts for templated and raw URIs.

MiniMetrics.java JAVA

```java
import java.util.*;
import java.util.concurrent.atomic.LongAdder;

public class MiniMetrics {

    /** Cumulative histogram with fixed upper bounds in ms (Prometheus "le" semantics). */
    static class Histogram {
        final double[] bounds;
        final long[] counts;
        double sum; long count;
        Histogram(double... bounds) { this.bounds = bounds; this.counts = new long[bounds.length]; }
        void record(double ms) {
            sum += ms; count++;
            for (int i = 0; i < bounds.length; i++) if (ms <= bounds[i]) counts[i]++;
        }
        /** Estimate a quantile by linear interpolation inside the bucket (as Prometheus does). */
```

MiniMetrics.java (continued) JAVA

```
        double quantile(double q) {
            double rank = q * count;
            double lowBound = 0; long lowCount = 0;
            for (int i = 0; i < bounds.length; i++) {
                if (counts[i] >= rank) {
                    return lowBound + (bounds[i] - lowBound) * (rank - lowCount)
                            / (counts[i] - lowCount);
                }
                lowBound = bounds[i]; lowCount = counts[i];
            }
            return bounds[bounds.length - 1];
        }
    }

    public static void main(String[] args) {
        Random rnd = new Random(42);
        Histogram h = new Histogram(10, 25, 50, 100, 250, 500, 1000, 2500);
        List<Double> raw = new ArrayList<>();
        for (int i = 0; i < 2000; i++) {
            double ms = rnd.nextDouble() < 0.96 ? 15 + rnd.nextDouble() * 45     // fast path
                                                : 800 + rnd.nextDouble() * 400;  // slow path
            h.record(ms); raw.add(ms);
        }
        Collections.sort(raw);
        System.out.printf("mean        = %.1f ms%n", h.sum / h.count);
        System.out.printf("p50 (exact) = %.1f ms, p99 (exact) = %.1f ms%n",
                raw.get(1000), raw.get(1980));
        System.out.printf("p50 (histo) = %.1f ms, p99 (histo) = %.1f ms%n",
                h.quantile(0.50), h.quantile(0.99));

        String labels = "method=\"GET\",uri=\"/orders/{id}\",status=\"200\"";
        System.out.println();
        System.out.println("# HELP http_server_requests_seconds Request latency");
        System.out.println("# TYPE http_server_requests_seconds histogram");
        for (int i = 0; i < h.bounds.length; i++) {
            System.out.printf("http_server_requests_seconds_bucket{%s,le=\"%s\"} %d%n",
                    labels, fmt(h.bounds[i] / 1000.0), h.counts[i]);
        }
        System.out.printf("http_server_requests_seconds_bucket{%s,le=\"+Inf\"} %d%n",
                labels, h.count);
        System.out.printf("http_server_requests_seconds_count{%s} %d%n", labels, h.count);
        System.out.printf("http_server_requests_seconds_sum{%s} %.3f%n", labels, h.sum / 1000.0);

        // cardinality: series = product of distinct tag values
        System.out.println();
        int methods = 4, statuses = 6, instances = 10, buckets = h.bounds.length + 3;
        long templated = (long) 25 * methods * statuses * instances * buckets;
        long perUserId = (long) 100_000 * methods * statuses * instances * buckets;
        System.out.println("series with 25 templated URIs : " + templated);
        System.out.println("series with 100000 raw URIs   : " + perUserId);
    }

    static String fmt(double d) {
        return d == Math.rint(d) ? String.valueOf((long) d) : String.valueOf(d);
    }
}
```

Output (real run; seed 42, so the numbers are reproducible) TERMINAL

```
$ java MiniMetrics.java
mean        = 78.0 ms
p50 (exact) = 39.3 ms, p99 (exact) = 1118.9 ms
p50 (histo) = 39.0 ms, p99 (histo) = 1802.3 ms

# HELP http_server_requests_seconds Request latency
# TYPE http_server_requests_seconds histogram
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="0.01"} 0
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="0.025"} 403
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="0.05"} 1471
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="0.1"} 1918
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="0.25"} 1918
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="0.5"} 1918
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="1"} 1957
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="2.5"} 2000
```

Output (real run; seed 42, so the numbers are reproducible) (continued) TERMINAL

```
http_server_requests_seconds_bucket{method="GET",uri="/orders/{id}",status="200",le="+Inf"} 2000
http_server_requests_seconds_count{method="GET",uri="/orders/{id}",status="200"} 2000
http_server_requests_seconds_sum{method="GET",uri="/orders/{id}",status="200"} 156.036

series with 25 templated URIs : 66000
series with 100000 raw URIs   : 264000000
```

What to notice: the mean (78.0 ms) hides that 4 percent of requests take about a second; the median is 39 ms. The bucket-based p99 estimate (1802 ms) differs from the exact value (1119 ms) because the only bucket boundaries around the answer are 1000 ms and 2500 ms, and interpolation assumes a uniform spread inside the bucket: **the accuracy of a histogram quantile is bounded by the bucket layout**, so boundaries must be placed around the service-level objective (for example 50, 100, 200, 500, 1000 ms). Finally, replacing a templated URI by raw URIs multiplies the series count by 4000.

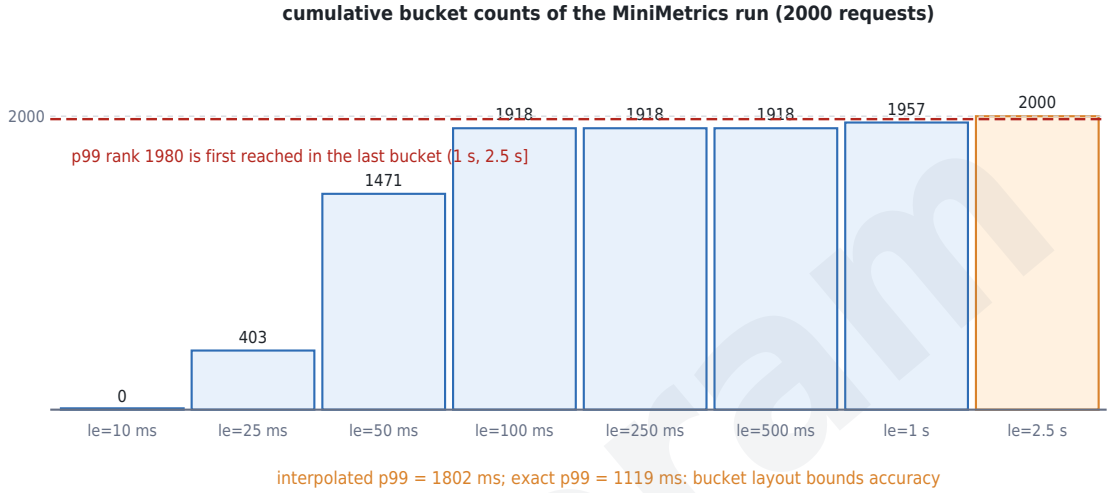


Figure 5.11. Cumulative histogram buckets from the program above: each bucket counts all observations up to its bound, and a quantile is interpolated inside the first bucket that reaches the requested rank.

Operational methods: RED, USE and alerting

Dashboards and alerts are organised around two checklists. The **RED method** applies to every request-serving component: **R**ate (requests per second), **E**rrors (failed requests per second or as a ratio) and **D**uration (latency distribution). The **USE method** applies to resources (CPU, memory, connection pool, thread pool, disk): **U**tilisation (fraction of capacity in use), **S**aturation (queued or waiting work) and **E**rrors. In a Spring Boot service, RED comes from `http.server.requests`; USE comes from `hikaricp.connections.*` (active, pending, timeouts), `tomcat.threads.*` (busy against configured), `jvm.memory.*`, `jvm.gc.pause` and `executor.*` for `@Async` pools.

Alerts should fire on symptoms users experience (error ratio, latency of the slow tail, saturation that precedes failure), not on every internal cause (CPU at 80 percent). A rule has an expression, a duration (`for`) that filters brief spikes, a severity label and a link to a runbook.

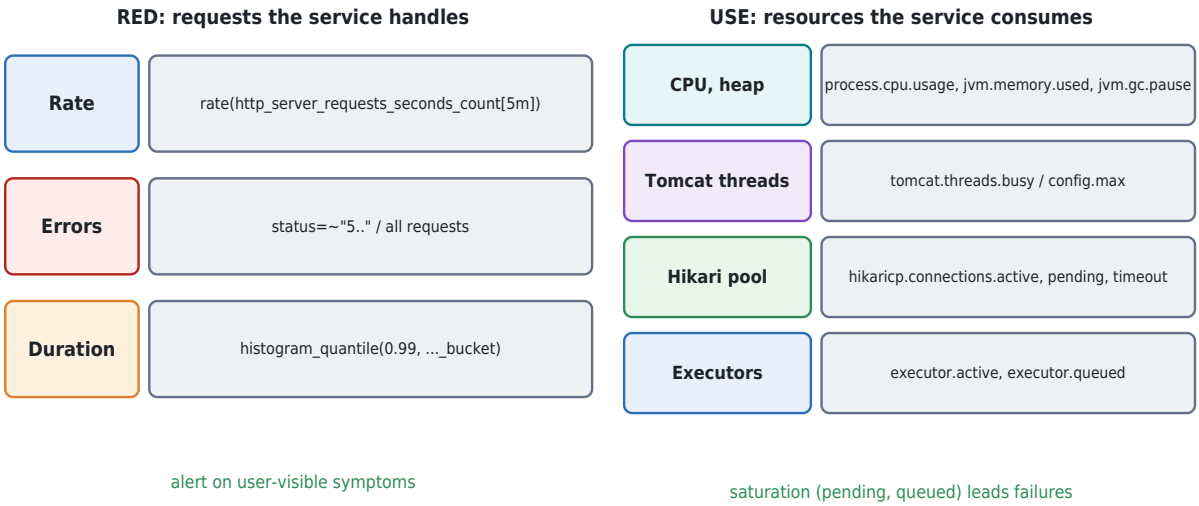


Figure 5.12. Metrics map for a Spring Boot service: RED signals measure the requests a service handles, USE signals measure the resources it consumes, and saturation signals (pending connections, busy threads) lead failures.

5.4.2 Worked example

The service from the earlier chapters exposes an order API. The requirements: Kubernetes probes that behave correctly, business and latency metrics, a health indicator for a non-critical dependency, management endpoints on a separate port with restricted access, and a Prometheus scrape plus two alert rules.

Configuration

application.yml — YAML

```yaml
management:
  server:
    port: 9090                      # not published by the public Ingress
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,loggers
  endpoint:
    health:
      probes:
        enabled: true               # /actuator/health/liveness and /readiness
      show-details: when-authorized
      group:
        readiness:
          include: readinessState,db,inventory
        liveness:
          include: livenessState    # nothing external in liveness
    loggers:
      access: read-only             # Boot 3.4+: no level changes from outside
  metrics:
    tags:
      application: ${spring.application.name}
    distribution:
      percentiles-histogram:
        http.server.requests: true
      slo:
        http.server.requests: 50ms,100ms,200ms,500ms,1s
spring:
  application:
    name: order-service
server:
  shutdown: graceful
  tomcat:
    mbeanregistry:
      enabled: true                 # exposes tomcat.threads.* meters
```

The `slo` entry makes the histogram buckets coincide with the latency objectives, which fixes the quantile-accuracy problem shown above. The Tomcat setting is required because Tomcat meters are disabled by default.

Custom metrics

OrderMetrics.java JAVA

```java
package shop.observability;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.Gauge;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Supplier;
import org.springframework.stereotype.Component;

@Component
public class OrderMetrics {

    private final Counter placed;
    private final Counter rejected;
    private final Timer paymentTimer;
    private final AtomicInteger backlog = new AtomicInteger();

    public OrderMetrics(MeterRegistry registry) {
        this.placed = Counter.builder("orders.placed")
                .description("Orders accepted").register(registry);
        this.rejected = Counter.builder("orders.rejected")
                .tag("reason", "out_of_stock")               // bounded set of values only
                .register(registry);
        this.paymentTimer = Timer.builder("orders.payment")
                .description("Payment gateway latency")
                .publishPercentileHistogram()
                .register(registry);
        Gauge.builder("orders.backlog", backlog, AtomicInteger::get).register(registry);
    }

    public void orderPlaced() { placed.increment(); }
    public void orderRejected() { rejected.increment(); }
    public void backlogChanged(int delta) { backlog.addAndGet(delta); }

    public <T> T timePayment(Supplier<T> call) {
        return paymentTimer.record(call);
    }
}
```

Counters and timers are created once in the constructor and reused. Creating a meter by name on every call (`registry.counter("orders.rejected", "reason", reason)`) is permitted and cheap enough for bounded tags, but a dynamic tag value must come from a closed set.

sieram

Custom health indicator

InventoryHealthIndicator.java JAVA

```java
package shop.observability;

import java.time.Duration;
import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;
import shop.inventory.InventoryClient;

@Component("inventory")
public class InventoryHealthIndicator implements HealthIndicator {

    private final InventoryClient client;

    public InventoryHealthIndicator(InventoryClient client) {
        this.client = client;
    }

    @Override
    public Health health() {
        long start = System.nanoTime();
        try {
            client.ping(Duration.ofMillis(300));              // short timeout: a probe must be fast
            long ms = (System.nanoTime() - start) / 1_000_000;
            return Health.up().withDetail("latencyMs", ms).build();
        } catch (RuntimeException ex) {
            return Health.down().withDetail("error", ex.getClass().getSimpleName()).build();
        }
    }
}
```

The bean name `inventory` becomes the component key (the suffix `HealthIndicator` is stripped from the default name). It is listed in the readiness group above, so an unreachable inventory service removes the pod from rotation without restarting it.

Securing the management port

ManagementSecurityConfig.java JAVA

```java
package shop.config;

import org.springframework.boot.actuate.autoconfigure.security.servlet.EndpointRequest;
import org.springframework.boot.actuate.health.HealthEndpoint;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
class ManagementSecurityConfig {

    @Bean
    @Order(1)
    SecurityFilterChain actuator(HttpSecurity http) throws Exception {
        http.securityMatcher(EndpointRequest.toAnyEndpoint())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(EndpointRequest.to(HealthEndpoint.class)).permitAll()
                .anyRequest().hasRole("OPS"))
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

Probes and the Prometheus scrape must authenticate or be network-restricted. Where the management port is reachable only inside the cluster, a network policy replaces the credentials for `health` and `prometheus`; the `OPS` role then guards `loggers` and the rest.

Prometheus scrape and alert rules

prometheus.yml YAML

```yaml
global:
  scrape_interval: 15s
rule_files:
  - alerts.yml
scrape_configs:
  - job_name: order-service
    metrics_path: /actuator/prometheus
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        regex: order-service
        action: keep
      - source_labels: [__address__]
        regex: '([^:]+)(?::\d+)?'
        replacement: '$1:9090'
        target_label: __address__
```

alerts.yml YAML

```yaml
groups:
  - name: order-service
    rules:
      - alert: HighErrorRatio
        expr: |
          sum(rate(http_server_requests_seconds_count{status=~"5.."}[5m]))
          / sum(rate(http_server_requests_seconds_count[5m])) > 0.02
        for: 10m
        labels: {severity: page}
        annotations:
          summary: "More than 2% of requests fail for 10 minutes"
      - alert: SlowTail
        expr: |
          histogram_quantile(0.99,
            sum by (le) (rate(http_server_requests_seconds_bucket[5m])))
          > 0.5
        for: 10m
        labels: {severity: ticket}
        annotations:
          summary: "p99 latency above 500 ms for 10 minutes"
      - alert: HikariPoolSaturated
        expr: max(hikaricp_connections_pending{application="order-service"}) > 0
        for: 5m
        labels: {severity: ticket}
```

5.4.3 Hands-on

With the configuration above, the endpoints behave as follows. The output is typical of Boot 3.4 and was not produced on the build machine.

Typical output (not run on the build machine) TERMINAL

```
$ curl -s localhost:9090/actuator/health/readiness
{"status":"UP","components":{"db":{"status":"UP"},"inventory":{"status":"UP"},
"readinessState":{"status":"UP"}}}
$ curl -s localhost:9090/actuator/metrics/http.server.requests | head -c 300
{"name":"http.server.requests","baseUnit":"seconds","measurements":[
{"statistic":"COUNT","value":1842.0},{"statistic":"TOTAL_TIME","value":96.4},
{"statistic":"MAX","value":1.21}],"availableTags":[{"tag":"uri","values":["/orders/{id}"]},...
$ curl -s localhost:9090/actuator/prometheus \
    | grep -E '^(orders_placed|tomcat_threads_busy|hikaricp_connections_active)'
orders_placed_total{application="order-service"} 311.0
tomcat_threads_busy_threads{application="order-service",name="http-nio-8080"} 4.0
hikaricp_connections_active{application="order-service",pool="HikariPool-1"} 2.0
```

Two naming details matter when writing queries. Micrometer names such as `orders.placed` become `orders_placed_total` for counters and `http_server_requests_seconds_*` for timers in Prometheus, because the exporter converts dots to underscores and appends base units. A query written against the Micrometer name returns no data.

5.4.4 Real-world problems and solutions

Problem 1: The readiness probe fails during start-up and the rollout stalls

Scenario. A new version is deployed with a rolling update. The new pods stay `0/1 Ready`, the old pods are not replaced, and the deployment reports `ProgressDeadlineExceeded`.

Symptoms. `kubectl get pods` shows `Running 0/1`; `kubectl describe pod` lists `Readiness probe failed: HTTP probe failed with statuscode: 503`. The application log shows a normal start.

Diagnosis. (1) Query the probe from inside the pod: `kubectl exec <pod> -- wget -qO- localhost:9090/actuator/health/readiness`. (2) Show details by temporarily setting `management.endpoint.health.show-details=always` or reading the log of the health aggregation. (3) Identify the failing component; with the configuration above it is typically `db` (wrong credentials from the new secret) or `inventory` (network policy blocks the call). (4) Check whether the probe port matches `management.server.port` (a probe sent to 8080 receives 404 or 401 when the management port moved).

Root cause. Readiness includes a dependency that is unreachable from the new pod, or the probe is aimed at the wrong port or path. The platform behaves correctly: it does not route traffic to an instance that reports itself unable to serve, and the rolling update waits for readiness.

Fix. Correct the dependency (secret, network policy) or the probe definition:

deployment.yaml — YAML

```yaml
readinessProbe:
  httpGet: {path: /actuator/health/readiness, port: 9090}
  periodSeconds: 5
  failureThreshold: 3
  timeoutSeconds: 2
```

Prevention. Run the readiness endpoint in the integration test suite against the Testcontainers database, so a broken connection configuration fails the build rather than the rollout. Keep non-essential dependencies out of the readiness group so that their outage degrades a feature rather than removing every pod.

Problem 2: The liveness probe kills a healthy but slow pod, repeatedly

Scenario. After a feature adds cache warming, start-up grows from 20 s to 45 s. The pods enter a restart loop in Kubernetes while the same image runs normally on a developer machine.

Symptoms. `RESTARTS` increases every 20 to 30 seconds, `kubectl describe pod` shows `Liveness probe failed` and `Container ... failed liveness probe, will be restarted`. The application log ends abruptly during start-up, repeated each time.

Diagnosis. Compare the time to first successful response with the probe schedule. The following model reproduces the kubelet logic in virtual time (probes start after `initialDelay`, every `period`, restart after `failureThreshold` consecutive failures; the real kubelet also applies restart back-off, which is omitted).

ProbeSim.java — JAVA

```java
import java.util.*;

/** Virtual-time model of kubelet probe handling for one container (seconds). */
public class ProbeSim {
    record Probe(int initialDelay, int period, int failureThreshold) { }

    static void run(String title, int appStart, Probe startup, Probe liveness, int horizon) {
        System.out.println("== " + title);
        int restarts = 0, containerStart = 0, failures = 0;
        boolean started = startup == null, ready = false;
        for (int t = 1; t <= horizon; t++) {
            int age = t - containerStart;
            boolean appUp = age >= appStart;                          // HTTP endpoint answers
            if (appUp && !ready && started) {
                ready = true;
                System.out.printf("t=%3d s  READY (restart %d)%n", t, restarts);
            }
            if (!started) {
                int since = age - startup.initialDelay();
                if (since >= 0 && since % startup.period() == 0) {
                    if (appUp) {
                        started = true; failures = 0;
                        System.out.printf("t=%3d s  startup probe OK%n", t);
```

ProbeSim.java (continued) JAVA

```
                    } else if (++failures >= startup.failureThreshold()) {
                        restarts++; containerStart = t; failures = 0; ready = false;
                        System.out.printf("t=%3d s  startup probe failed %d times: KILLED%n",
                                t, startup.failureThreshold());
                    }
                }
                continue;
            }
            int lsince = age - liveness.initialDelay();
            if (lsince >= 0 && lsince % liveness.period() == 0) {
                if (appUp) failures = 0;
                else if (++failures >= liveness.failureThreshold()) {
                    restarts++; containerStart = t; failures = 0; ready = false;
                    System.out.printf("t=%3d s  liveness failed %d times: container KILLED%n",
                            t, liveness.failureThreshold());
                }
            }
        }
        System.out.println("restarts in " + horizon + " s: " + restarts);
    }

    public static void main(String[] args) {
        Probe liveness = new Probe(10, 5, 3);
        run("A: 45 s start-up, liveness only (initialDelay 10, period 5, failures 3)",
                45, null, liveness, 120);
        run("B: 45 s start-up, startup probe (period 5, failures 12)",
                45, new Probe(0, 5, 12), liveness, 120);
    }
}
```

Output (real run of the model) TERMINAL

```
$ java ProbeSim.java
== A: 45 s start-up, liveness only (initialDelay 10, period 5, failures 3)
t= 20 s  liveness failed 3 times: container KILLED
t= 40 s  liveness failed 3 times: container KILLED
t= 60 s  liveness failed 3 times: container KILLED
t= 80 s  liveness failed 3 times: container KILLED
t=100 s  liveness failed 3 times: container KILLED
t=120 s  liveness failed 3 times: container KILLED
restarts in 120 s: 6
== B: 45 s start-up, startup probe (period 5, failures 12)
t= 45 s  startup probe OK
t= 46 s  READY (restart 0)
restarts in 120 s: 0
```



Figure 5.13. Probe timelines for a 45 s start-up: with liveness only, the third failed probe at 20 s kills the container before it can start; a startup probe with a 60 s budget lets the first start finish.

Root cause. The liveness probe begins at 10 s and tolerates 3 failures at a 5 s period, so the container is killed at 20 s, before the 45 s start-up completes. Every restart repeats the same sequence.

Fix. Add a startup probe whose budget (`periodSeconds` x `failureThreshold`) exceeds the worst-case start-up time, and keep the liveness probe strict afterwards:

deployment.yaml YAML

```yaml
startupProbe:
  httpGet: {path: /actuator/health/liveness, port: 9090}
  periodSeconds: 5
  failureThreshold: 24          # up to 120 s to start
livenessProbe:
  httpGet: {path: /actuator/health/liveness, port: 9090}
  periodSeconds: 10
  failureThreshold: 3
```

Prevention. Measure start-up in the cluster (the `application.started.time` and `application.ready.time` metrics) rather than locally, and size the startup budget from the 99th percentile. Never include the database or other remote services in liveness: a shared outage would restart every replica simultaneously.

Problem 3: The metrics backend runs out of memory after a release (cardinality explosion)

Scenario. A developer adds a timer tagged with the customer identifier to analyse slow customers. Within hours the Prometheus server is OOM-killed and dashboards are empty.

Symptoms. `prometheus_tsdb_head_series` grows from 60,000 to several million; scrape duration climbs; the response of `/actuator/prometheus` grows from kilobytes to hundreds of megabytes; the application heap grows because the registry retains every meter.

Diagnosis. (1) Rank metrics by series count in Prometheus: `topk(10, count by (__name__)({__name__=~".+"}))`. (2) Inspect the label values of the top metric: `count(count by (customerId) (orders_payment_seconds_count))`. (3) Compare with the application's metric list at `/actuator/metrics` and read the tag values at `/actuator/metrics/orders.payment`.

Root cause. The tag has an unbounded value set. The series count is the product of tag value counts times the bucket count; the arithmetic computed by the program above shows 66,000 series for 25 templated URIs and 264 million for 100,000 raw URIs.

Fix. Remove the tag; record the high-cardinality identifier in logs or trace attributes, where cost scales with events and not with distinct values. As a protection, add a registry filter that caps tag values and denies offenders:

MeterGuardConfig.java JAVA

```java
package shop.observability;

import io.micrometer.core.instrument.Meter;
import io.micrometer.core.instrument.config.MeterFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
class MeterGuardConfig {

    @Bean
    MeterFilter dropCustomerTag() {
        return MeterFilter.ignoreTags("customerId");
    }

    @Bean
    MeterFilter capUriCardinality() {
        return MeterFilter.maximumAllowableTags("http.server.requests", "uri", 100,
                MeterFilter.deny());
    }

    @Bean
    MeterFilter noJvmBufferPools() {
        return MeterFilter.denyNameStartsWith("jvm.buffer");
    }
}
```

Prevention. Review every new tag during code review with the question "how many distinct values can this have in a year?". Alert on `prometheus_tsdb_head_series` growth and on scrape sample counts (`scrape_samples_scraped`) per job.

sreram

Problem 4: Latency percentiles are missing or wrong in the dashboard

Scenario. A dashboard panel shows "average latency 80 ms" while customers complain about slow checkouts. The query for p99 returns "no data".

Symptoms. `http_server_requests_seconds_bucket` does not exist in Prometheus; only `_count`, `_sum` and `_max` exist. A p99 panel based on `histogram_quantile` is empty.

Diagnosis. `curl localhost:9090/actuator/prometheus | grep http_server_requests_seconds_bucket | head`. If nothing is printed, histogram publishing is off, which is the default for `http.server.requests`.

Root cause. Boot publishes only count, total time and max for server requests unless a distribution is requested. An average computed from `_sum / _count` hides the slow tail (78.0 ms mean against a 1.12 s p99 in the program above).

Fix. Enable the histogram and align the buckets with the objectives (configuration shown in the worked example), then query `histogram_quantile(0.99, sum by (le) (rate(http_server_requests_seconds_bucket[5m])))`. Do not use client-side percentiles (`management.metrics.distribution.percentiles`) for dashboards across several instances, because pre-computed quantiles cannot be aggregated.

Prevention. Define latency objectives per endpoint group and verify during review that the bucket boundaries bracket them. Show p50, p95 and p99 together with the request rate.

5.4.5 Questions to expect

What is the difference between liveness and readiness probes?

Liveness reports whether the process must be restarted; a failure makes the kubelet kill the container. Readiness reports whether the instance can accept traffic; a failure removes it from the Service endpoints without a restart. Liveness must not depend on external systems; readiness may.

How does Spring Boot compute the aggregate health status?

`HealthEndpoint` collects the status of every `HealthIndicator` in the group and a `StatusAggregator` chooses the most severe by the order `DOWN`, `OUT_OF_SERVICE`, `UP`, `UNKNOWN`. `DOWN` is returned as HTTP 503.

How are Actuator endpoints secured in production?

Expose the minimum set, move them to a separate management port that is not routed publicly, restrict access with a `SecurityFilterChain` using `EndpointRequest`, keep sanitised values (`show-values=never`), and disable `heapdump` and `env` unless operationally required.

What is metric cardinality and why does it matter?

It is the number of distinct time series, the product of tag value counts times the number of buckets. Unbounded tag values (identifiers, raw URLs) multiply storage and memory in the application and in the backend until it fails.

Counter, gauge, timer: which is used for what?

A counter counts events and only increases; a gauge samples a current value that can rise and fall; a timer records the count and duration of events, optionally with a histogram.

Why are histograms preferred to pre-computed percentiles?

Bucket counts can be summed across instances and time windows, and the quantile is computed afterwards; pre-computed percentiles cannot be combined correctly.

What do RED and USE stand for?

RED (rate, errors, duration) describes the requests a service handles; USE (utilisation, saturation, errors) describes the resources it consumes.

Which metrics reveal thread pool or connection pool saturation?

`tomcat.threads.busy` approaching `tomcat.threads.config.max`, `hikaricp.connections.pending` above zero, `hikaricp.connections.timeout` increasing, and `executor.queued` for `@Async` pools.

sreeram

Why does a Micrometer name differ from the name in Prometheus?

The Prometheus registry converts names to snake case, appends `_total` to counters and the base unit to timers (`_seconds`), so `orders.placed` becomes `orders_placed_total`.

KEY POINTS

Key points

- Expose only the endpoints needed, on a separate management port, with Spring Security and per-endpoint access settings; `env`, `heapdump` and `threaddump` disclose sensitive data.
- Liveness answers "restart me?" and must not depend on external systems; readiness answers "send traffic?"; a startup probe covers slow start-up.
- Micrometer meters are identified by name and tags; use counters, gauges and timers from a `MeterRegistry` and keep tag values bounded.
- Series count equals the product of tag cardinalities and bucket counts; never tag by identifier or raw URL.
- Publish histograms with buckets around the latency objectives; averages hide the tail and client-side percentiles cannot be aggregated.
- Organise dashboards and alerts by RED for requests and USE for resources; alert on user-visible symptoms with a `for` duration.

5.5 Logging in Spring Boot Applications

Logs are the record of discrete events, and during an incident they are the only source that explains why a particular request failed. Their value depends on three properties: each line can be found (consistent fields, a correlation identifier), each line is trustworthy (no forged entries, no sensitive data) and logging does not harm the service (no flooding, no blocking). Spring Boot configures logging by default; this chapter explains what is configured, how to move from text to structured JSON, how context travels in the MDC, and how to keep logging from becoming the incident.

5.5.1 Concept and mechanism

SLF4J, Logback and what Boot sets up

Application code logs against the **SLF4J** facade (`org.slf4j.Logger`). The default backend in `spring-boot-starter-logging` is **Logback**, reached through SLF4J; bridges (`jul-to-slf4j`, `log4j-to-slf4j`, `jcl-over-slf4j`) route libraries that use other APIs into the same pipeline. A logger is named hierarchically (`shop.orders.OrderService`) and inherits the level of its nearest configured ancestor; the event then flows to the appenders of the logger and, because of additivity, of its ancestors up to the root logger.

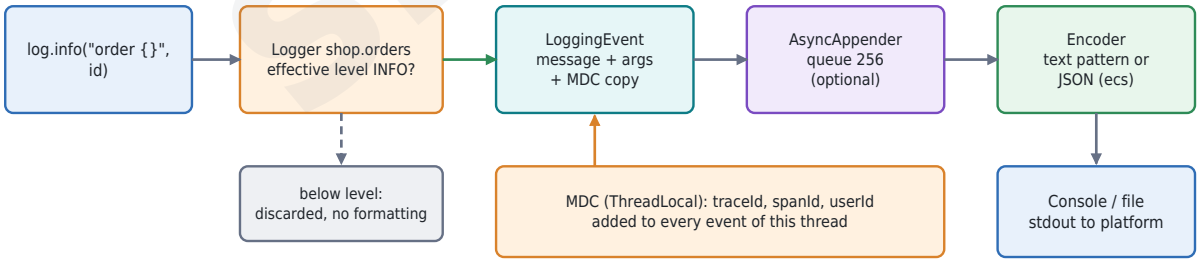


Figure 5.14. The logging pipeline: the level check happens before any formatting work; accepted events pass to appenders, whose encoders choose the text or JSON layout, and the MDC contributes context fields.

Boot reads `logging.*` properties and configures Logback before the application context exists: `logging.level.<logger>=<LEVEL>` (levels `TRACE`, `DEBUG`, `INFO`, `WARN`, `ERROR`, `FATAL`, `OFF`), `logging.group.<name>` for sets of loggers (predefined groups `web` and `sql`), `logging.file.name` or `logging.file.path` to add file output (by default Boot logs to the console only), and `logging.logback.rollingpolicy.*` for rotation. A custom `logback-spring.xml` (the `-spring` suffix enables `<springProfile>` and property access) replaces the defaults when the properties are insufficient. The levels can be changed at run time without restart through the `loggers` Actuator endpoint:

Changing a log level at run time (typical; not run on the build machine) TERMINAL

```
$ curl -s -X POST localhost:9090/actuator/loggers/shop.orders \
    -H 'Content-Type: application/json' -d '{"configuredLevel":"DEBUG"}'
$ curl -s localhost:9090/actuator/loggers/shop.orders
{"configuredLevel":"DEBUG","effectiveLevel":"DEBUG"}
```

Writing log statements

Three rules cover most defects found in review.

1. **Use parameterised messages**: `log.info("order {} accepted for {}", id, customer)`. Formatting is deferred until the level check passes, whereas string concatenation always allocates the message. For arguments that are expensive to compute, the SLF4J 2 fluent API defers evaluation: `log.atDebug().setMessage("state {}").addArgument(() -> dump()).log()`.
2. **Pass the exception as the last argument**: `log.error("payment failed for order {}", id, ex)`. The stack trace is printed once, at the place where the exception is handled; logging and rethrowing the same exception at each layer produces duplicate stack traces and inflates log volume.
3. **Choose levels by audience.** `ERROR`: an operation failed and a person may need to act. `WARN`: unexpected but handled (retry succeeded, fallback used). `INFO`: business milestones and lifecycle (order placed, job finished), at a rate a person can read. `DEBUG`: diagnostic detail, off in production by default. `TRACE`: payload-level detail.

Structured logging

Plain text lines require regular expressions to extract fields, and the expressions break when a message changes. **Structured logging** emits each event as one JSON object whose fields (`@timestamp`, `log.level`, `message`, `service.name`, `traceId`, MDC entries, exception fields) are indexed by the log platform. Spring Boot 3.4 added built-in support: set `logging.structured.format.console` (or `.file`) to `ecs` (Elastic Common Schema), `logstash` or `gelf`, or to the fully qualified name of a class implementing `StructuredLogFormatter`. Service metadata comes from `logging.structured.ecs.service.name` (default: `spring.application.name`) and `.version`, `.environment`, `.node-name`. Boot 3.5 documents `logging.structured.json.include`, `.exclude`, `.rename` and `.add` for adjusting the members, and `logging.structured.json.stacktrace.*` for stack trace shape. Every MDC key-value pair is added to the JSON object automatically.

application-prod.yml YAML

```yaml
logging:
  structured:
    format:
      console: ecs
    json:
      add:
        deployment: blue
      exclude: log.logger
  level:
    root: INFO
    shop: INFO
    org.hibernate.SQL: WARN
```

Before Boot 3.4, JSON output required a third-party encoder (for example `logstash-logback-encoder`) configured in `logback-spring.xml`; applications on older versions continue to use it.

The MDC and correlation across threads

The **Mapped Diagnostic Context** (`org.slf4j.MDC`) is a per-thread map whose entries the encoder adds to every event. A servlet filter or the tracing instrumentation puts a correlation identifier into it at the start of a request, so that every line produced while handling the request carries the same `traceId`. With Micrometer Tracing on the classpath, Boot fills `traceId` and `spanId` automatically and shows them in the default console pattern.

The MDC is implemented with a `ThreadLocal`; therefore it does **not** cross a thread boundary by itself. A task handed to an executor runs on a pool thread whose MDC is empty or, worse, holds the entries of an earlier request. The program below models the mechanism with a `ThreadLocal` map and shows both the loss and the standard remedy, a wrapper that captures the submitting thread's map and installs it in the worker (in Spring, a `TaskDecorator` set on the `ThreadPoolTaskExecutor`, or the context-propagation library).

LogMdcDemo.java JAVA

```java
import java.util.*;
import java.util.concurrent.*;

public class LogMdcDemo {
    /** Minimal MDC: a per-thread map, as in SLF4J. */
    static final ThreadLocal<Map<String, String>> MDC = ThreadLocal.withInitial(HashMap::new);

    static void log(String msg) {
        System.out.printf("[%-14s] traceId=%s  %s%n", Thread.currentThread().getName(),
                MDC.get().getOrDefault("traceId", "-"), msg);
    }

    /** Wraps a task so the submitting thread's MDC is copied to the worker (TaskDecorator idea). */
    static Runnable withMdc(Runnable task) {
        Map<String, String> captured = new HashMap<>(MDC.get());
        return () -> {
            Map<String, String> previous = MDC.get();
            MDC.set(captured);
            try { task.run(); } finally { MDC.set(previous); }
        };
    }

    static String sanitise(String s) { return s.replaceAll("[\\r\\n]", "_"); }

    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(1, r -> new Thread(r, "async-1"));
        MDC.get().put("traceId", "4bf92f35");
        log("request received");

        pool.submit(() -> log("plain async task")).get();            // MDC is lost
        pool.submit(withMdc(() -> log("decorated async task"))).get(); // MDC propagated
        pool.shutdown();

        String userInput = "bob\n2026-10-19 10:00:00 INFO  admin logged in";
        System.out.println("--- log injection");
        System.out.println("raw      : login failed for " + userInput);
        System.out.println("sanitised: login failed for " + sanitise(userInput));
    }
}
```

Output (real run) TERMINAL

```
$ java LogMdcDemo.java
[main          ] traceId=4bf92f35  request received
[async-1       ] traceId=-  plain async task
[async-1       ] traceId=4bf92f35  decorated async task
--- log injection
raw      : login failed for bob
2026-10-19 10:00:00 INFO  admin logged in
sanitised: login failed for bob_2026-10-19 10:00:00 INFO  admin logged in
```

The second half of the output illustrates **log injection** (CWE-117): user-controlled text containing a line break forges a second, apparently legitimate log line (here a fake administrator login). Defences: never log raw user input in text-format logs; replace or escape control characters (as in `sanitise`); prefer structured JSON logging, where the encoder escapes line breaks inside a string value so that the event stays one record.

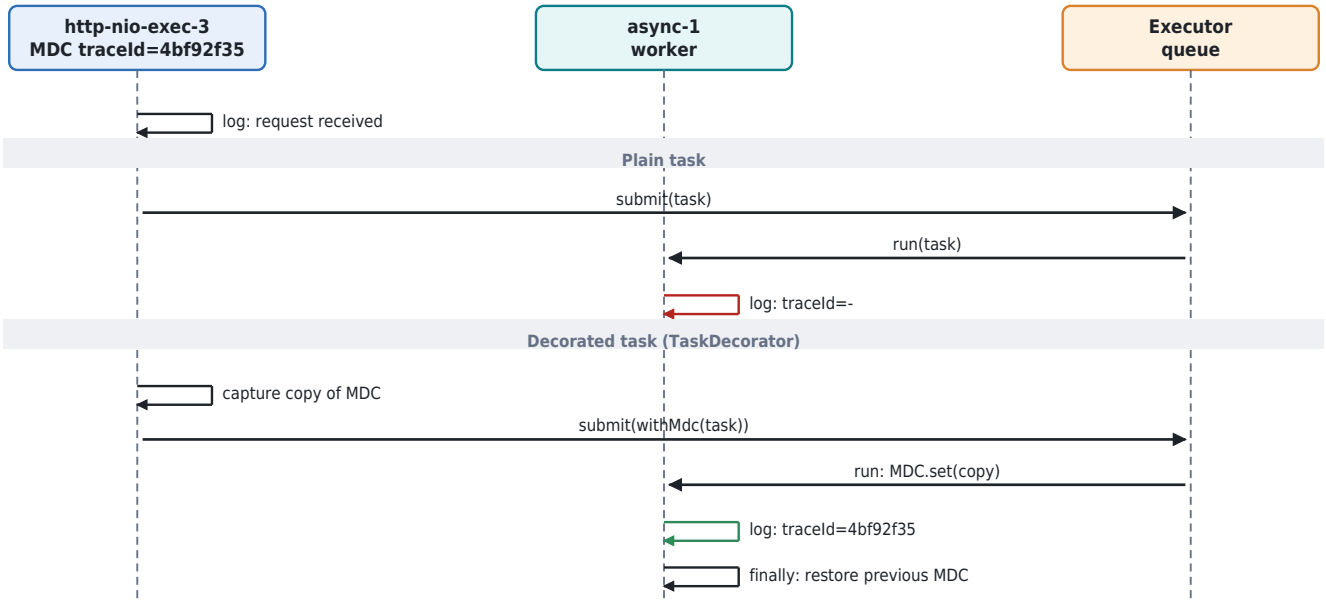


Figure 5.15. The MDC is bound to a thread: a plain executor task logs without the trace identifier, while a decorated task receives a copy of the submitting thread's context.

Asynchronous appenders, flooding and cost

Appending to a file or a socket takes time, and a synchronous appender makes the request thread wait for it. A Logback `AsyncAppender` decouples the two with a bounded queue (`queueSize`, default 256) and a worker thread. Its two key parameters define the behaviour under pressure: `discardingThreshold` (default: 20 percent of the queue) causes events of level `INFO` and below to be discarded once the remaining capacity falls under the threshold, and `neverBlock` (default `false`) decides whether the caller blocks when the queue is full. The program below measures the three behaviours with a sink that takes 1 ms per line and 2000 lines produced as fast as possible.

AsyncLogDemo.java JAVA

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class AsyncLogDemo {
    static void slowSink() { try { Thread.sleep(1); } catch (InterruptedException e) { } }

    public static void main(String[] args) throws Exception {
        int lines = 2000;

        long t0 = System.nanoTime();
        for (int i = 0; i < lines; i++) slowSink();
        System.out.printf("synchronous        : caller blocked %4d ms, 0 dropped%n", ms(t0));

        for (String policy : new String[]{"discard", "block"}) {
            BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(256);
            AtomicInteger written = new AtomicInteger(), dropped = new AtomicInteger();
            Thread writer = new Thread(() -> {
                try { while (true) { queue.take(); slowSink(); written.incrementAndGet(); } }
                catch (InterruptedException e) { }
            });
            writer.setDaemon(true); writer.start();
            t0 = System.nanoTime();
            for (int i = 0; i < lines; i++) {
                if (policy.equals("block")) queue.put(i);
                else if (!queue.offer(i)) dropped.incrementAndGet();
            }
            System.out.printf("async, queue 256, %-7s: caller blocked %4d ms, %d dropped%n",
                    policy, ms(t0), dropped.get());
            writer.interrupt();
        }
    }
    static long ms(long t0) { return (System.nanoTime() - t0) / 1_000_000; }
}
```

Output (real run on the build machine; timings vary) TERMINAL

```
$ java AsyncLogDemo.java
synchronous        : caller blocked 2263 ms, 0 dropped
async, queue 256, discard: caller blocked    3 ms, 1741 dropped
async, queue 256, block  : caller blocked 2266 ms, 0 dropped
```

The result is a trade-off, not an improvement: an asynchronous appender that blocks when full is as slow as a synchronous one once the sink cannot keep up, and the variant that never blocks keeps the request fast by losing 87 percent of the lines. Asynchronous logging protects latency against short bursts and slow sinks; it does not remove the need to limit volume. Container deployments avoid the sink problem by writing to standard output and leaving collection to the platform.

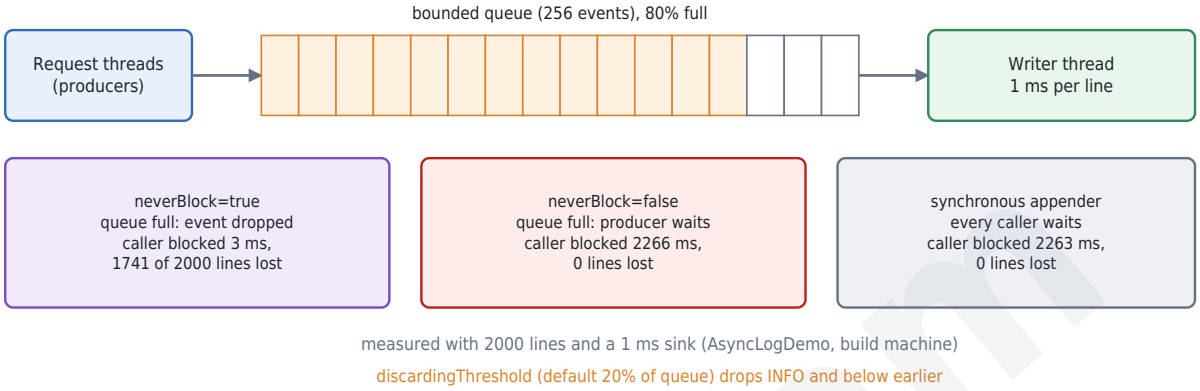


Figure 5.16. Behaviour of a bounded asynchronous appender when producers outrun the sink: the queue fills, and the configured policy decides between blocking the request thread and discarding events.

What to log, and what not to log

Log business events and decisions with their identifiers (`orderId`, `paymentId`), outcomes and durations; log at boundaries (incoming request summary, outgoing call failures, retries, fallbacks); log configuration at start-up (without secrets). Do not log passwords, tokens, API keys, full card numbers, session identifiers or request bodies that may contain them; mask or hash identifiers where the purpose allows. Under the GDPR, a log line containing a name, e-mail address, IP address or user identifier is **personal data**: collection needs a lawful basis and a defined retention period, access is restricted, and a deletion request applies to it. Practical consequences: log the internal identifier instead of the e-mail address, keep retention short (days to weeks) and configure it (`logging.logback.rollingpolicy.max-history`, platform-side retention), and ship logs only to systems that are covered by the data processing agreement. Sampling applies to high-volume, low-value events (successful health checks, per-item loop messages): log one in N, or summarise ("processed 10,000 items in 2.1 s").

5.5.2 Worked example

The order service must produce JSON logs with trace correlation, propagate the context into an `@Async` notification executor, protect against log injection and keep the application responsive when the log sink is slow.

AsyncConfig.java JAVA

```java
package shop.config;

import java.util.Map;
import org.slf4j.MDC;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.task.TaskDecorator;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

@Configuration
@EnableAsync
class AsyncConfig {

    @Bean
    TaskDecorator mdcTaskDecorator() {
        return runnable -> {
```

AsyncConfig.java (continued) JAVA

```
            Map<String, String> context = MDC.getCopyOfContextMap();      // captured on submit
            return () -> {
                Map<String, String> previous = MDC.getCopyOfContextMap();
                if (context != null) MDC.setContextMap(context); else MDC.clear();
                try {
                    runnable.run();
                } finally {
                    if (previous != null) MDC.setContextMap(previous); else MDC.clear();
                }
            };
        };
    }

    @Bean(name = "notificationExecutor")
    ThreadPoolTaskExecutor notificationExecutor(TaskDecorator decorator) {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(4);
        executor.setQueueCapacity(200);
        executor.setThreadNamePrefix("notify-");
        executor.setTaskDecorator(decorator);
        return executor;
    }
}
```

OrderService.java JAVA

```
package shop.orders;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    public void place(String orderId, String customerNote) {
        log.info("order {} placed, note length {}", orderId,
                customerNote.length());                                  // the raw note is not logged
        notifyCustomer(orderId);
    }

    @Async("notificationExecutor")
    public void notifyCustomer(String orderId) {
        log.info("sending confirmation for order {}", orderId);   // carries the traceId
    }
}
```

Note that `place` calls `notifyCustomer` on `this`; the `@Async` annotation is applied by a proxy, so the self-invocation would run synchronously. The method belongs on a separate bean in real code (the proxy mechanics are covered in the AOP chapter of the container part). The listing keeps both methods in one class only to keep the example short.

logback-spring.xml XML

```
<configuration>
  <include resource="org/springframework/boot/logging/logback/defaults.xml"/>
  <include resource="org/springframework/boot/logging/logback/console-appender.xml"/>

  <appender name="ASYNC_CONSOLE" class="ch.qos.logback.classic.AsyncAppender">
    <queueSize>1024</queueSize>
    <discardingThreshold>0</discardingThreshold>   <!-- never discard automatically -->
    <neverBlock>true</neverBlock>                   <!-- drop rather than block request threads -->
    <appender-ref ref="CONSOLE"/>
  </appender>

  <root level="INFO">
    <appender-ref ref="ASYNC_CONSOLE"/>
  </root>
</configuration>
```

sreeram

`discardingThreshold=0` with `neverBlock=true` means events are dropped only when the queue is completely full, never earlier; the application stays responsive and the loss is limited to overload situations. The alternative policy (`neverBlock=false`) suits batch jobs where losing lines is unacceptable and latency is not critical.

5.5.3 Hands-on

With the ECS format enabled, one line of the request log looks like the following. The fields are those documented for the ECS formatter; the values are illustrative and the output was not produced on the build machine.

Typical output (not run on the build machine) TERMINAL

```
$ java -jar order-service.jar --spring.profiles.active=prod
{"@timestamp":"2026-10-19T10:15:42.318Z","log":{"level":"INFO","logger":"shop.orders.OrderService"},
"process":{"pid":1,"thread":{"name":"http-nio-8080-exec-3"}},"service":{"name":"order-service"},
"message":"order 8812 placed, note length 24","traceId":"4bf92f3577b34da6a3ce929d0e0e4736",
"spanId":"00f067aa0ba902b7","ecs":{"version":"8.11"}}
```

(In the real output each event is a single line; the example is wrapped for the page.) A log platform can then answer "show every line of one request across services" with a single query on `traceId`.

5.5.4 Real-world problems and solutions

Problem 1: Log flooding fills the disk and slows the service

Scenario. A downstream service is unavailable. The retry loop of the order service logs a full stack trace at `ERROR` on every attempt, for every request. Within twenty minutes the log volume grows from 40 MB per hour to 25 GB per hour, the node's disk fills and the pod is evicted.

Symptoms. Latency rises while error rates are still low; disk usage alarms; `kubectl describe pod` shows `Evicted: The node was low on resource: ephemeral-storage`; the log platform bill spikes. A handful of distinct messages account for nearly all lines.

Diagnosis. (1) Count lines by logger and message template: `kubectl logs <pod> --since=10m | cut -c1-120 | sort | uniq -c | sort -rn | head`. (2) In the log platform, group by `log.logger` and `message`. (3) Check whether the repeating line is logged inside a loop or a retry. (4) Read `logback.events` counters (`logback_events_total{level="error"}`), which Micrometer exposes by default; a steep increase of the error counter confirms the flood and gives an alert signal.

Root cause. Log volume proportional to the failure rate: one statement executed per attempt, with a stack trace that is identical every time, and no rate limit or deduplication. Retries multiply the effect.

Fix. Short term: raise the level of the noisy logger without a restart (`POST /actuator/loggers/shop.payment` with `{"configuredLevel":"OFF"}` for a specific class). Structural: log the exception once at the point where the failure is finally handled (after retries), log intermediate attempts at `DEBUG` or summarise them ("3 attempts failed"), and limit repetition with a counter-based or time-based guard.

ThrottledLog.java JAVA

```java
package shop.support;

import java.util.concurrent.atomic.AtomicLong;
import org.slf4j.Logger;

public final class ThrottledLog {

    private final Logger log;
    private final long intervalNanos;
    private final AtomicLong lastLogged = new AtomicLong(System.nanoTime() - Long.MAX_VALUE / 2);
    private final AtomicLong suppressed = new AtomicLong();

    public ThrottledLog(Logger log, long intervalSeconds) {
        this.log = log;
        this.intervalNanos = intervalSeconds * 1_000_000_000L;
    }

    public void error(String message, Throwable cause) {
        long now = System.nanoTime();
        long last = lastLogged.get();
        if (now - last >= intervalNanos && lastLogged.compareAndSet(last, now)) {
            log.error("{} (suppressed {} similar events)", message, suppressed.getAndSet(0), cause);
        } else {
```

ThrottledLog.java (continued) JAVA

```
            suppressed.incrementAndGet();
        }
    }
}
```

Prevention. Alert on `rate(logback_events_total{level="error"}[5m])`; set a per-service log volume budget and review the top ten messages after each release; combine the circuit breaker (Microservices part) with throttled logging so that a failing dependency produces one line per interval rather than one per request.

Problem 2: Logs from @Async and scheduled work cannot be correlated with the request

Scenario. A support engineer searches for the `traceId` of a failed order and finds the request lines but none of the notification lines that explain why the customer received no e-mail.

Symptoms. Lines from threads named `notify-1`, `task-1` or `scheduling-1` show an empty or stale `traceId`.

Diagnosis. Look at the thread names and the `traceId` field of the missing lines (the demonstration above reproduces the pattern: `traceId=-` on the pool thread). Check whether the executor has a `TaskDecorator` or whether the context-propagation library is on the classpath and enabled.

Root cause. The MDC is thread-local; executors do not copy it.

Fix. Register a `TaskDecorator` on every `ThreadPoolTaskExecutor` (shown in the worked example), or use the `ContextPropagatingTaskDecorator` that Spring Framework 6.1 provides when `io.micrometer:context-propagation` is on the classpath (whether Boot applies it to its own executor automatically depends on the Boot version; verify in the release notes of the version in use). For `CompletableFuture` and virtual-thread executors, wrap the executor with the same capture logic.

Prevention. Add an integration test that triggers an async path, captures log output with a test appender and asserts that the `traceId` equals the request's.

Problem 3: Personal data and secrets appear in logs

Scenario. A security review finds e-mail addresses, bearer tokens and card numbers in the production log index, retained for 400 days.

Symptoms. Search for `Authorization`, `password=` or `@` in the log platform returns request dumps. Debug-level logging of request and response bodies was left enabled by a wrapper filter (`CommonsRequestLoggingFilter` with payload) or by `logging.level.org.springframework.web=DEBUG`.

Diagnosis. Search the indexed logs for token patterns (`eyJ` prefixes of JWTs, `Bearer ` , 16-digit sequences passing the Luhn check) and group by logger. Review configuration for `includePayload`, `includeHeaders` and the `web` group at `DEBUG`.

Root cause. Diagnostic logging enabled globally, no masking and no retention policy.

Fix. Disable payload and header logging; where request logging is needed, log method, path template, status and duration only. Add a masking layer for known fields (a `StructuredLogFormatter` customisation or an encoder rule) as a safety net, and shorten retention. Rotate any token or key that appeared in logs, and treat the log index as a data breach scope for assessment.

Prevention. Keep the `web` and `sql` groups at `INFO` or higher in production profiles, verify in a test that sensitive fields are absent from the log output of key flows, and apply data minimisation: log identifiers, not content.

5.5.5 Questions to expect

Why use SLF4J instead of calling Logback directly?

SLF4J is a facade: application and library code depend only on its API, and the backend can be changed or bridged without code changes. Boot uses Logback by default and supports Log4j2.

Why parameterised messages instead of string concatenation?

The message is formatted only if the level is enabled, so disabled statements cost almost nothing. Concatenation always builds the string.

What is the MDC and why can it be lost?

A per-thread map of context entries added to every log event. It is a `ThreadLocal`, so it is not visible to other threads; executors and reactive pipelines need a decorator or the context-propagation library to carry it.

What changed in Spring Boot 3.4 for logging?

Built-in structured logging: `logging.structured.format.console` and `.file` select `ecs`, `logstash`, `gelf` or a custom `StructuredLogFormatter`, removing the need for a separate JSON encoder dependency.

What is log injection and how is it prevented?

Attacker-controlled input containing line breaks creates forged log lines. Prevention: sanitise or escape control characters, avoid logging raw input, and use JSON output where line breaks are escaped.

What are the risks of asynchronous logging?

A bounded queue can discard events under load (the default discards `INFO` and below near capacity), and events in the queue can be lost on a crash. If the queue blocks when full, latency returns to that of synchronous logging.

How are log levels changed without a restart?

Through the `loggers` Actuator endpoint (`POST /actuator/loggers/{name}`), or through the logging system's own management interface. The endpoint must be protected.

How does logging relate to GDPR?

Log lines containing personal data are personal data: they need a lawful basis, minimisation, access control, bounded retention and the ability to honour deletion requests.

KEY POINTS

Key points

- Log through SLF4J with parameterised messages and pass exceptions as the last argument; log an exception once, where it is handled.
- Boot 3.4 and later emit ECS, Logstash or GELF JSON with `logging.structured.format.console`; MDC entries become JSON members.
- The MDC is thread-bound: copy it into async tasks with a `TaskDecorator` or context propagation, otherwise correlation breaks.
- Never log raw user input in text logs (log injection) or secrets and personal data; keep retention short and configured.
- Asynchronous appenders trade lost events or blocking against latency; neither removes the need to limit volume.
- Flooding is a volume problem: alert on `logback_events_total`, log retries once, and throttle repeated errors.

5.6 Performance Tuning

Performance work has a fixed order: define the objective (for example p99 latency under 300 ms at 200 requests per second), measure where time and resources go, change one thing, and measure again. Tuning without measurement optimises the wrong component; the most common outcome is a configuration copied from a blog post that moves the bottleneck or hides it until production load arrives. This chapter treats the levers of a Spring Boot service in the order in which they usually matter: the request-handling threads, the connection pool, the garbage collector and heap, caching, and start-up time, and then the tools that locate a bottleneck: JDK Flight Recorder, thread dumps, `jstat` and flame graphs. Operating-system and container-level JVM behaviour (cgroups, CPU throttling, memory accounting) is covered in the operating systems guide and only the Spring-relevant settings are repeated here. Measurements below were taken on the build machine (2 virtual CPUs, JDK 21) and are labelled; absolute numbers will differ elsewhere, the shapes do not.

5.6.1 Concept and mechanism

Capacity, latency and saturation

Three relations govern every service. **Little's law**: the average number of requests in the system equals the arrival rate multiplied by the average time in the system (`L = λ × W`). A pool of 10 workers with a service time of 100 ms can complete at most 10 / 0.1 = 100 requests per second. **Utilisation and queueing**: as the arrival rate approaches capacity, waiting time grows without bound; a system at 90 percent utilisation is already queueing noticeably, and at more than 100 percent the queue grows for as long as the excess lasts. **The bottleneck rule**: throughput is limited by the scarcest resource (CPU, a thread pool, a connection pool, a lock, a remote service), and improving anything else changes nothing.

The following program makes the second relation visible. A pool of 10 threads serves tasks of 100 ms each (capacity 100 per second). The offered load is 80 per second, then 130 per second for three seconds, then 80 per second again. Each line shows the pool state at the end of the second, in the form that the metrics `executor.active` and `executor.queued` would report.

Saturation.java JAVA

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class Saturation {
    public static void main(String[] args) throws Exception {
        int threads = 10, serviceMs = 100;       // capacity: 10 / 0.1 s = 100 requests per second
        ThreadPoolExecutor pool = new ThreadPoolExecutor(threads, threads, 0, TimeUnit.SECONDS,
                new LinkedBlockingQueue<>());
        AtomicLong totalLatency = new AtomicLong();
        AtomicInteger done = new AtomicInteger();
        int[] ratePerSecond = {80, 80, 130, 130, 130, 80, 80, 80};       // offered load per second
        System.out.println(" t(s) offered  active  queued  mean latency so far (ms)");
        for (int sec = 0; sec < ratePerSecond.length; sec++) {
            long start = System.nanoTime();
            for (int i = 0; i < ratePerSecond[sec]; i++) {
                long submitted = System.nanoTime();
                pool.execute(() -> {
                    try { Thread.sleep(serviceMs); } catch (InterruptedException e) { }
                    totalLatency.addAndGet((System.nanoTime() - submitted) / 1_000_000);
                    done.incrementAndGet();
                });
                long next = start + (long) ((i + 1) * 1e9 / ratePerSecond[sec]);
                while (System.nanoTime() < next) Thread.sleep(1);
            }
            int d = done.get();
            System.out.printf("%5d %7d %7d %7d  %d%n", sec + 1, ratePerSecond[sec],
                    pool.getActiveCount(), pool.getQueue().size(),
                    d == 0 ? 0 : totalLatency.get() / d);
        }
        pool.shutdownNow();
    }
}
```

Output (real run on the build machine; the figures vary slightly between runs) TERMINAL

```
$ java Saturation.java
 t(s) offered  active  queued  mean latency so far (ms)
    1      80       7       0  100
    2      80       8       0  100
    3     130      10      30  134
    4     130      10      60  215
    5     130      10      90  311
    6      80      10      70  414
    7      80      10      50  485
    8      80      10      30  503
```

What to notice: while the load stays below 100 per second, the queue is empty and latency equals the service time (100 ms). From second 3 the pool is full (`active = 10`) and the queue grows by 30 entries per second, the excess of 130 over 100. Even after the load returns to 80 per second, latency stays high because the backlog must drain at the spare capacity of only 20 per second. The alert-worthy signal is therefore the **queue** (saturation), not the error count, which is still zero. In a Spring Boot service the corresponding signals are `tomcat.threads.busy` against `tomcat.threads.config.max`, the connection accept queue, `hikaricp.connections.pending` and `executor.queued`.

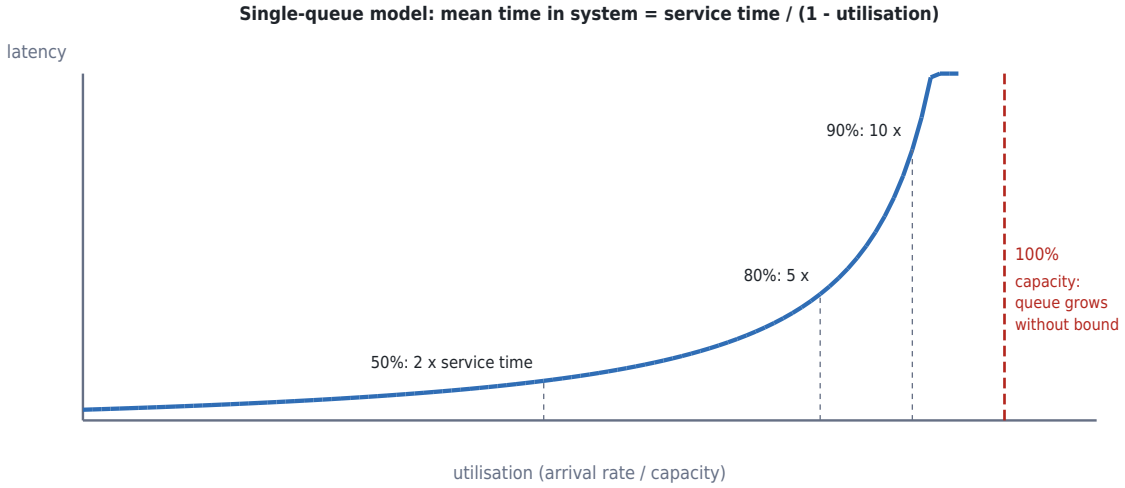


Figure 5.17. Latency against utilisation for a pool with a fixed service time: waiting time is negligible at low load, rises steeply near full utilisation and grows without limit when the arrival rate exceeds capacity.

Request threads: Tomcat pool and virtual threads

In the thread-per-request model, Tomcat takes each request from a bounded worker pool (`server.tomcat.threads.max`, default 200; `server.tomcat.threads.min-spare`, default 10). When all workers are busy, new connections wait in the accept queue (`server.tomcat.accept-count`, default 100) and then are refused; the number of simultaneously open connections is limited by `server.tomcat.max-connections` (default 8192). A worker is occupied for the entire request, including every blocking call to a database or remote service. For a service whose requests spend most of their time waiting (a typical CRUD service), throughput is therefore `threads / mean request time`, and the thread pool is the bottleneck long before the CPU is busy.

Virtual threads (final in Java 21) remove that limit for blocking code. A virtual thread is a lightweight thread managed by the JVM: when it blocks on I/O it is unmounted from its carrier (platform) thread, which immediately runs another virtual thread. Spring Boot 3.2 and later enable them with one property, `spring.threads.virtual.enabled=true` (Java 21 or later required): Tomcat then handles each request on a new virtual thread, and the auto-configured `applicationTaskExecutor` (used by `@Async`) and task scheduler also use virtual threads. Consequently the properties that configure these thread pools no longer have an effect, and, because virtual threads are daemon threads, `spring.main.keep-alive=true` is recommended when the JVM would otherwise exit.

The next program compares the two models for 2000 requests that each block for 200 ms (standing for a database call): a fixed pool of 200 platform threads (Tomcat's default) against one virtual thread per request.

ThreadsVsVirtual.java JAVA

```java
import java.util.*;
import java.util.concurrent.*;

public class ThreadsVsVirtual {
    static final int REQUESTS = 2000;
    static final int BLOCK_MS = 200;                        // stands for a database or HTTP call

    static long run(ExecutorService pool) throws Exception {
        long t0 = System.nanoTime();
        List<Future<?>> futures = new ArrayList<>();
        for (int i = 0; i < REQUESTS; i++) {
            futures.add(pool.submit(() -> { Thread.sleep(BLOCK_MS); return null; }));
        }
        for (Future<?> f : futures) f.get();
        pool.shutdown();
        return (System.nanoTime() - t0) / 1_000_000;
    }

    public static void main(String[] args) throws Exception {
        System.out.println("requests = " + REQUESTS + ", each blocks " + BLOCK_MS + " ms");
        long platform = run(Executors.newFixedThreadPool(200));
        System.out.printf("platform pool of 200 threads : %5d ms  (ideal %d ms)%n",
                platform, REQUESTS / 200 * BLOCK_MS);
        long virtual = run(Executors.newVirtualThreadPerTaskExecutor());
```

ThreadsVsVirtual.java (continued) JAVA

```
        System.out.printf("virtual thread per request   : %5d ms  (ideal %d ms)%n",
                virtual, BLOCK_MS);
    }
}
```

Output (real run on the build machine; typical) TERMINAL

```
$ java ThreadsVsVirtual.java
requests = 2000, each blocks 200 ms
platform pool of 200 threads :  2065 ms  (ideal 2000 ms)
virtual thread per request   :   248 ms  (ideal 200 ms)
```

The platform pool needs ten "waves" of 200 requests, so 2000 requests take about 2 s; the virtual-thread executor runs all 2000 blocked calls concurrently and finishes in about one blocking period. This is the maximal case, in which nothing else limits concurrency. Four conditions decide whether the gain appears in a real Spring Boot service.

- **Downstream limits remain.** The default HikariCP pool has 10 connections (`spring.datasource.hikari.maximum-pool-size`). With 2000 concurrent virtual threads, 1990 wait for a connection, and the pool, not the thread count, is the new bottleneck; the wait appears as `hikaricp.connections.pending` and, after 30 s, as connection timeout exceptions. Pool size must be set from what the database can sustain, and concurrency limited with a semaphore or bulkhead where necessary.
- **CPU-bound work is not accelerated.** Virtual threads help only when threads wait. A request that computes for 50 ms on a 2-core machine still limits throughput to 40 requests per second.
- **Pinning.** In JDK 21, a virtual thread that blocks inside a `synchronized` block or a native call stays pinned to its carrier and blocks it. JDK 24 removes the `synchronized` case (JEP 491). Detect pinning with `-Djdk.tracePinnedThreads=full` or the JFR event `jdk.VirtualThreadPinned`, and prefer `ReentrantLock` in application code; JDBC drivers and libraries must be recent enough.
- `ThreadLocal` **and context.** Per-request thread-local data works unchanged but is no longer pooled; large thread-local caches multiply by the number of concurrent requests.

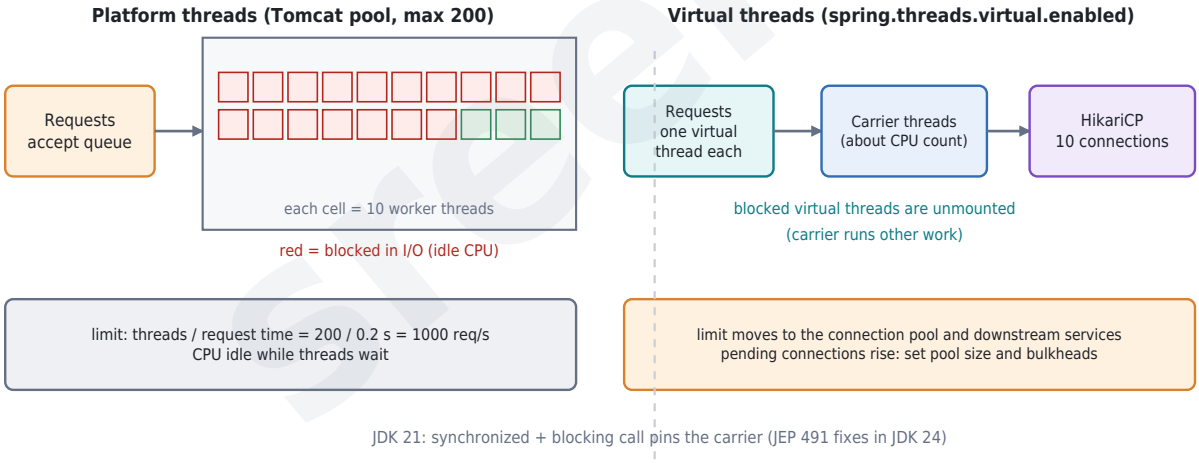


Figure 5.18. Thread-per-request with a bounded pool against virtual threads: the pool queues requests while workers wait on I/O, whereas virtual threads release their carrier thread during the wait and shift the limit to the connection pool.

Garbage collector and heap

The JVM selects the collector ergonomically. On a machine with at least two CPUs and 1792 MB of memory it is **G1** (`UseG1GC = true` was reported by `-XX:+PrintFlagsFinal` on the build machine), with a pause-time goal of 200 ms (`MaxGCPauseMillis`). G1 is the correct default for services with heaps from about 1 GB upward. Alternatives: **Serial** (`-XX:+UseSerialGC`) for very small containers (below about 1 CPU and 1 GB), where its low overhead and footprint matter; **Parallel** (`-XX:+UseParallelGC`) for batch jobs where throughput matters and pauses do not; **ZGC** (`-XX:+UseZGC`; generational mode via `-XX:+ZGenerational` on JDK 21, the default mode of ZGC from JDK 23) for large heaps or strict pause goals, at some cost in CPU and memory overhead. A collector change is justified by GC logs showing pauses that violate the latency objective, not by preference.

Heap sizing in containers uses percentages of the memory limit rather than absolute values. On the build machine the JVM reported `MaxRAMPercentage = 25` and `InitialRAMPercentage = 1.5625`; the default maximum heap is therefore a

quarter of the available memory, which is conservative for a container dedicated to one JVM. Setting `-XX:MaxRAMPercentage=75` raised the reported `MaxHeapSize` from 1,568,669,696 to 4,706,009,088 bytes on the same machine. The remainder must accommodate metaspace, thread stacks (about 1 MB per platform thread), code cache, direct buffers and the JVM's own structures; a container limit that equals the heap size ends in `OOMKilled` (see the chapters on containers). A common service configuration is:

JVM flags for a 1 GB container (illustrative) BASH

```
JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=70 -XX:+ExitOnOutOfMemoryError \
  -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/dumps \
  -Xlog:gc*:file=/tmp/gc.log:time,uptime:filecount=5,filesize=10m"
```

`-XX:+ExitOnOutOfMemoryError` makes the JVM exit so that the orchestrator restarts a healthy process instead of keeping a degraded one; the heap dump flag preserves the evidence. GC logging at low overhead is safe to leave on in production.

Caching with Spring Cache and Caffeine

Caching trades memory and staleness for latency and load reduction. Spring's cache abstraction (`@EnableCaching`, `@Cacheable`, `@CacheEvict`, `@CachePut`) is implemented by proxies, so it shares the self-invocation limitation of `@Transactional`. A cache manager is selected by `spring.cache.type` or by the library on the classpath. **Caffeine** is the standard in-process cache: it bounds size (`maximumSize`), expires entries (`expireAfterWrite`, `expireAfterAccess`) and records statistics (`recordStats`). Boot binds those statistics to Micrometer (`cache.gets` with tags `result=hit` and `result=miss`, `cache.size`, `cache.evictions`) when statistics are enabled.

A cache must always be **bounded** (by size, by time, or both): an unbounded map used as a cache is a memory leak with a delay, as demonstrated in the problem section below. Distributed caches (Redis) share entries between instances and survive restarts, at the cost of a network round trip; they are treated in the part on messaging and delivery. Typical settings:

application.yml YAML

```yaml
spring:
  cache:
    type: caffeine
    cache-names: products,prices
    caffeine:
      spec: maximumSize=1000,expireAfterWrite=10m,recordStats
```

Start-up time

Start-up time matters for scaling out (a new replica is useful only after it is ready), for rollouts and for serverless-style workloads. A Spring Boot start-up consists of JVM start, class loading and linking, classpath scanning and bean definition processing, bean instantiation with auto-configuration, and, for the first requests, JIT warm-up. The levers, in increasing order of effort:

1. **Reduce work.** Remove unused starters and auto-configurations (`spring.autoconfigure.exclude`), narrow component scanning, avoid heavy work in `@PostConstruct` and `ApplicationRunner` beans before readiness, and run migrations outside the application where possible.
2. **Lazy initialisation** (`spring.main.lazy-initialization=true`) creates beans when first needed. It reduces start-up time but moves failures from start-up to the first request, defers the cost to the first caller (a latency spike) and hides configuration errors from readiness; the documentation advises tuning the heap first and excluding critical beans with `@Lazy(false)`.
3. **Class Data Sharing (CDS).** The JVM maps a pre-parsed archive of classes instead of loading them from jars. Since Boot 3.3 the documented procedure is: extract the jar (`java -Djarmode=tools -jar app.jar extract --destination application`), perform a training run that writes the archive (`java -XX:ArchiveClassesAtExit=application.jsa -Dspring.context.exit=onRefresh -jar app.jar`), and start with `-XX:SharedArchiveFile=application.jsa`. On Java 25 and later the JEP 483 AOT cache succeeds CDS, and Boot documents it as the preferred option.
4. **Ahead-of-time processing and GraalVM native image.** `spring-boot:process-aot` generates bean definitions as code at build time, avoiding reflection-heavy scanning. A GraalVM native image compiles the application to a native executable with start-up in tens of milliseconds and a smaller memory footprint, at the cost of long builds (minutes), the closed-world assumption (reflection, proxies and resources must be declared as hints, which Spring generates for its own code but not for every library), no JIT (peak throughput can be lower) and a different

debugging and profiling toolset. Native image suits scale-to-zero and short-lived processes; for long-running services on a warm JVM, CDS or the AOT cache usually delivers enough.

The effect of CDS can be measured on the JDK alone. The program below uses a representative selection of JDK classes (streams, `java.net.http`, regex, `java.time`, `CompletableFuture`); the three measurements are the median of seven runs of the same compiled jar on the build machine.

Start-up with and without class data sharing (real run, median of 7) TERMINAL

```
$ java -Xshare:off -cp boot.jar Boot              # no CDS at all
212 ms
$ java -cp boot.jar Boot                          # default: JDK classes from the built-in archive
146 ms
$ java -XX:ArchiveClassesAtExit=app.jsa -cp boot.jar Boot
$ java -XX:SharedArchiveFile=app.jsa -cp boot.jar Boot    # JDK and application classes archived
113 ms
```

A Spring Boot application loads several thousand classes instead of the few hundred here, so absolute savings are larger (typically a noticeable fraction of the start-up time) but must be measured on the application itself.

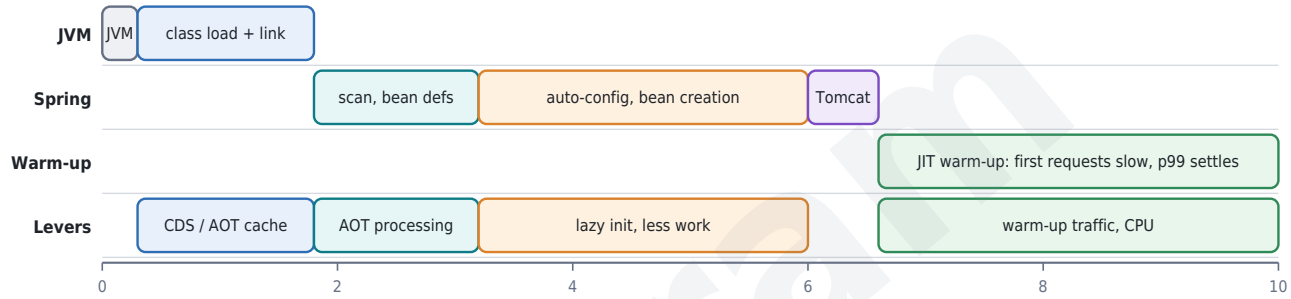


Figure 5.19. Illustrative phases of a Spring Boot start-up (durations not measured) and the lever that shortens each: CDS and the AOT cache act on class loading, AOT processing and lazy initialisation on bean creation, and warm-up requires load before p99 latency stabilises.

Finding the bottleneck: the diagnosis path

A slow endpoint is diagnosed from the outside in. First the metrics identify the symptom and its scope (which endpoint, which instances, since when): `http.server.requests` by `uri`, error ratio, and saturation of the thread and connection pools. Then the resource that is exhausted is identified: CPU (profile), waiting threads (thread dump), GC pressure (`jvm.gc.pause`, GC log), connection pool (`hikaricp.connections.pending`) or a downstream call (`http.client.requests`). Finally a profile or trace shows the code path.

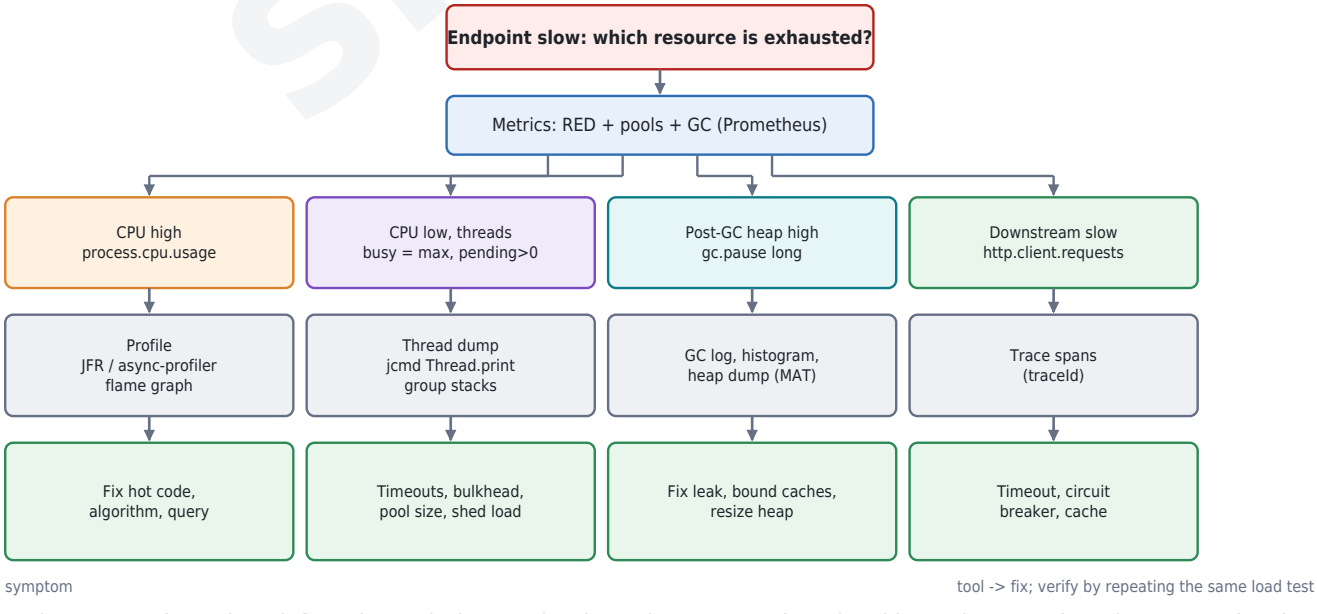


Figure 5.20. Diagnosis path for a slow endpoint: metrics choose between CPU-bound, waiting and memory-bound causes, and each branch has its own tool (profiler, thread dump, GC log) and its own typical fix.

Profiling with JDK Flight Recorder and flame graphs

JDK Flight Recorder (JFR) is built into HotSpot, samples running Java threads about every 10 to 20 ms and records GC, allocation, lock and I/O events at an overhead usually below two percent, which makes it suitable for production. A recording is started at launch (`-XX:StartFlightRecording=filename=rec.jfr,settings=profile`) or on a running process (`jcmd <pid> JFR.start name=r settings=profile duration=60s filename=rec.jfr`), and read with the `jfr` tool or JDK Mission Control. **async-profiler** (an external agent, `asprof -d 30 -f flame.html <pid>`) samples through operating-system signals, avoids the safepoint bias of some profilers, and can profile CPU, allocation and lock time and render flame graphs.

The program below contains two deliberate hot spots: a string built by repeated concatenation in a loop and a regular expression applied to many inputs. It runs for six seconds under JFR.

HotSpots.java — JAVA

```java
import java.util.regex.Pattern;

public class HotSpots {
    static final Pattern EMAIL = Pattern.compile("^[a-z0-9._%+-]+@[a-z0-9.-]+\\.[a-z]{2,}$");

    static boolean validate(String s) { return EMAIL.matcher(s).matches(); }

    static String report(int n) {
        String out = "";
        for (int i = 0; i < n; i++) out += i + ",";          // copies the growing string each time
        return out;
    }

    public static void main(String[] args) {
        long end = System.nanoTime() + 6_000_000_000L;
        long sink = 0;
        while (System.nanoTime() < end) {
            sink += report(2500).length();
            for (int i = 0; i < 2000; i++) sink += validate("user" + i + "@example.com") ? 1 : 0;
        }
        System.out.println("done " + sink);
    }
}
```

Recording and reading the samples (real run on the build machine) — TERMINAL

```
$ java -XX:StartFlightRecording=filename=hot.jfr,settings=profile HotSpots.java
done 27636960
$ jfr summary hot.jfr | grep -E "jdk.ExecutionSample|jdk.GarbageCollection "
 jdk.ExecutionSample                      135          1482
 jdk.GarbageCollection                     79          1817
$ jfr print --events jdk.ExecutionSample hot.jfr | grep -A1 "stackTrace = \[" \
    | grep -v "stackTrace\|--" | sed 's/line:.*//' | sort | uniq -c | sort -rn | head -4
     30     java.lang.StringConcatHelper.newString(byte[], long)
     25     java.util.regex.Pattern$BmpCharProperty.match(Matcher, int, CharSequence)
     18     java.util.regex.Pattern.lambda$union$3(Pattern$CharPredicate, ...)
     17     java.util.regex.Pattern$Begin.match(Matcher, int, CharSequence)
```

The listing of top frames ranks the methods that were executing when a sample was taken (`jfr print` shows only the five topmost frames of each stack unless `--stack-depth` is given). Attributing each sample to the application method on its full stack (`jfr print --stack-depth 64`) gives the inclusive view: of 135 samples, 56 (41 percent) had `HotSpots.report` on the stack, 64 (47 percent) had `HotSpots.validate`, and the remaining 15 (11 percent) were in the loop of `main` and in the Java source launcher that compiled the file. The profile therefore contradicts the natural assumption that the quadratic string concatenation is the main cost: the regular expression, which is called 2000 times per iteration, costs slightly more. A profile replaces such assumptions with data. The 79 young collections are a side effect of the concatenation: each iteration allocates a new string of growing length, so the allocation rate, and with it GC activity, follows from that hot spot. The fixes are a `StringBuilder` for the report and a reduction of the regex work (for example a cheaper pre-check or fewer validations), and a second recording confirms the effect.

A **flame graph** renders many stack samples as one picture: the x-axis is the share of samples (not time order), the y-axis is stack depth, each box is a method and its width is the number of samples whose stack contains it. Reading rules: look for the **widest boxes near the top** (the code that actually burns CPU); a wide box with narrow children is self time in that method; a wide plateau in framework code (`DispatcherServlet`, `HikariPool.getConnection`) shows where requests wait; compare two graphs (before and after) rather than judging one in isolation.

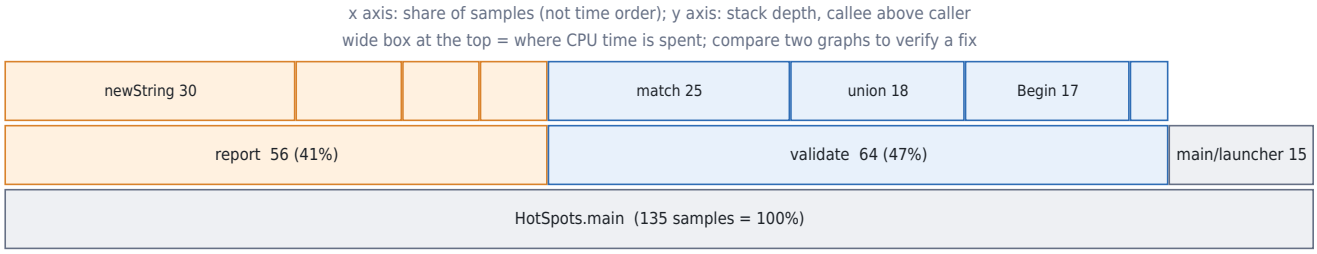


Figure 5.21. A flame graph of the HotSpots recording, drawn from the sample counts above: width is the share of samples, so the regex matcher and StringConcatHelper dominate, and the widths of children never exceed their parent.

For a process that is already running, three JDK commands cover most first-response needs, without any agent: `jcmd <pid> Thread.print` (stack of every thread, equal to the Actuator `threaddump` endpoint), `jcmd <pid> GC.class_histogram` (live objects by class) and `jstat -gcutil <pid> 1000` (generation occupancy and GC counts and times every second). Their use is shown in the memory-leak problem below.

5.6.2 Worked example

The order service must meet a p99 of 300 ms at 150 requests per second on two replicas with 1 GB of memory each. The relevant configuration brings together the levers above.

application.yml YAML

```yaml
spring:
  threads:
    virtual:
      enabled: true                 # Boot 3.2+, Java 21+: request threads become virtual threads
  main:
    keep-alive: true
  datasource:
    hikari:
      maximum-pool-size: 20         # sized from database capacity, not from request concurrency
      minimum-idle: 20
      connection-timeout: 2000      # fail fast under saturation instead of 30 s default
      leak-detection-threshold: 20000
  cache:
    type: caffeine
    cache-names: products
    caffeine:
      spec: maximumSize=2000,expireAfterWrite=5m,recordStats
server:
  shutdown: graceful
management:
  endpoints:
    web:
      exposure:
        include: health,prometheus,threaddump
  metrics:
    distribution:
      slo:
        http.server.requests: 50ms,100ms,200ms,300ms,500ms
```

ProductService.java JAVA

```java
package shop.catalog;

import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    private final ProductRepository products;

    public ProductService(ProductRepository products) {
```

sreeram

ProductService.java (continued) JAVA

```java
        this.products = products;
    }

    @Cacheable(cacheNames = "products", key = "#id")
    public ProductView find(long id) {
        return products.findViewById(id).orElseThrow(() -> new ProductNotFoundException(id));
    }

    @CacheEvict(cacheNames = "products", key = "#id")
    public void update(long id, ProductUpdate update) {
        products.updateFields(id, update);
    }
}
```

Three decisions are visible. The connection timeout is lowered to 2 s so that saturation surfaces as a quick, countable error rather than as a 30 s hang; the cache is bounded and records statistics so that its effectiveness (`cache.gets` hit ratio) is observable; and the SLO buckets surround the 300 ms objective so that the p99 alert is accurate. The cached value is a read-only view object (`ProductView`), never a managed entity, so cache hits cannot trigger lazy loading outside a session.

The load test that validates the configuration uses an open-loop generator (a fixed arrival rate, for example `k6` with `constant-arrival-rate`, or `wrk2`) rather than a closed loop of N users that wait for each response: a closed loop slows its own arrival rate when the service slows down and hides queueing (the "coordinated omission" problem). The test warms the JVM for several minutes before measurement, and compares p50, p95, p99, error rate and the saturation metrics between runs.

5.6.3 Hands-on

The memory-related commands operate on a running JVM. The program `LeakSlow` stores a 20 KB array per iteration in a static map that is never trimmed and allocates short-lived garbage; it runs with a 300 MB heap. The commands were executed against it a few seconds after start.

LeakSlow.java JAVA

```java
import java.util.*;

public class LeakSlow {
    static final Map<String, byte[]> CACHE = new HashMap<>();          // unbounded, never evicted

    public static void main(String[] args) throws Exception {
        for (int i = 1; i <= 100_000; i++) {
            CACHE.put("order-" + i, new byte[20 * 1024]);
            byte[] garbage = new byte[64 * 1024];                      // short-lived allocation
            if (i % 10 == 0) Thread.sleep(40);
        }
    }
}
```

Inspecting the running JVM (real run on the build machine) TERMINAL

```
$ java -Xmx300m -Xlog:gc:file=gc.log -cp . LeakSlow &
$ jstat -gcutil 6529 1000 4
  S0     S1     E      O      M     CCS    YGC     YGCT     FGC    FGCT     CGC    CGCT       GCT
     - 100.00   5.13  39.80  47.47   7.00      6     0.039     0     0.000     0     0.000     0.039
     - 100.00  58.97  39.80  47.47   7.00      6     0.039     0     0.000     0     0.000     0.039
     - 100.00  18.92  53.84  47.47   7.00      7     0.056     0     0.000     0     0.000     0.056
     - 100.00  72.97  53.84  47.47   7.00      7     0.056     0     0.000     0     0.000     0.056
$ jcmd 6529 GC.class_histogram | head -4
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:         12250       43321208  [B (java.base@21.0.12.1)
$ jcmd 6529 Thread.print | grep -A2 '"main"'
"main" #1 [6531] prio=5 os_prio=0 cpu=131.11ms elapsed=8.92s ... waiting on condition
   java.lang.Thread.State: TIMED_WAITING (sleeping)
        at java.lang.Thread.sleep0(java.base@21.0.12.1/Native Method)
```

Reading the output: column `O` (old generation occupancy, percent) steps from 39.8 to 53.8 and never returns to a lower value, `YGC` counts young collections, and the class histogram attributes 43 MB to `[B` (byte arrays), the type of the cached values. A few seconds later the same process shows `O` at 83 percent and its first full collection (`FGC = 1`).

5.6.4 Real-world problems and solutions

Problem 1: Memory grows until the pod is restarted (unbounded in-process cache)

Scenario. A product-price lookup is cached in a `ConcurrentHashMap` keyed by `orderId` to "avoid repeated database calls". After each deployment the service runs for two to three days and then the pod restarts with `OutOfMemoryError: Java heap space`, or is `OOMKilled`.

Symptoms. `jvm.memory.used` for the old generation rises in a saw-tooth whose lower envelope climbs steadily; `jvm.gc.pause` grows in frequency and duration; latency degrades before the restart; after the restart everything is normal. The p99 worsens as the heap fills because collections run back to back.

Diagnosis. (1) Confirm the shape: plot `jvm_memory_used_bytes{area="heap"}` and, more importantly, the heap **after** collection (`jvm_gc_live_data_size_bytes`); growth of the post-GC value over hours is a leak, growth of the peak alone is not. (2) Take a class histogram twice, ten minutes apart, and compare: `jcmd <pid> GC.class_histogram | head -20`. The earlier program's output shows the pattern, with `[B` (byte arrays) at the top. (3) Capture a heap dump (`jcmd <pid> GC.heap_dump /tmp/heap.hprof`, or the Actuator `heapdump` endpoint if enabled) and open it in Eclipse MAT or VisualVM: the **dominator tree** shows one `HashMap` retaining most of the heap, and the path to GC roots names the static field. (4) Run with `-XX:+HeapDumpOnOutOfMemoryError` so that a dump exists after a crash.

The following program reproduces the failure in seconds: each request stores 20 KB in a map and the heap is limited to 64 MB. The bounded variant evicts the least recently used entry above 500 entries (the mechanism behind Caffeine's `maximumSize`).

CacheLeak.java — JAVA

```java
import java.util.*;

public class CacheLeak {
    /** Bounded cache: evicts the least recently used entry (idea of Caffeine maximumSize). */
    static class Lru<K, V> extends LinkedHashMap<K, V> {
        final int max;
        Lru(int max) { super(16, 0.75f, true); this.max = max; }
        @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > max; }
    }

    public static void main(String[] args) {
        boolean bounded = args.length > 0 && args[0].equals("bounded");
        Map<String, byte[]> cache = bounded ? new Lru<>(500) : new HashMap<>();
        Runtime rt = Runtime.getRuntime();
        System.out.println("mode=" + (bounded ? "bounded(500)" : "unbounded")
                + ", max heap " + rt.maxMemory() / (1 << 20) + " MB");
        try {
            for (int i = 1; i <= 20_000; i++) {
                cache.put("order-" + i, new byte[20 * 1024]);            // 20 KB per entry
                if (i % 500 == 0) {
                    System.gc();
                    System.out.printf("requests=%5d  entries=%5d  heap after GC=%3d MB%n", i,
                            cache.size(), (rt.totalMemory() - rt.freeMemory()) >> 20);
                }
            }
            System.out.println("finished without error");
        } catch (OutOfMemoryError e) {
            int entries = cache.size();
            cache.clear();                                               // free memory before printing
            System.out.println("OutOfMemoryError after " + entries + " entries: " + e.getMessage());
        }
    }
}
```

Output (real run on the build machine; the bounded run is trimmed) — TERMINAL

```
$ java -Xmx64m CacheLeak.java
mode=unbounded, max heap 64 MB
requests=  500  entries=  500  heap after GC= 14 MB
requests= 1000  entries= 1000  heap after GC= 24 MB
requests= 1500  entries= 1500  heap after GC= 33 MB
requests= 2000  entries= 2000  heap after GC= 43 MB
requests= 2500  entries= 2500  heap after GC= 53 MB
requests= 3000  entries= 3000  heap after GC= 62 MB
OutOfMemoryError after 3050 entries: Java heap space
$ java -Xmx64m CacheLeak.java bounded
mode=bounded(500), max heap 64 MB
```

Output (real run on the build machine; the bounded run is trimmed) (continued) TERMINAL

```
requests=  500  entries=  500  heap after GC= 14 MB
requests= 1000  entries=  500  heap after GC= 14 MB
requests= 1500  entries=  500  heap after GC= 14 MB
...
requests=20000  entries=  500  heap after GC= 14 MB
finished without error
```

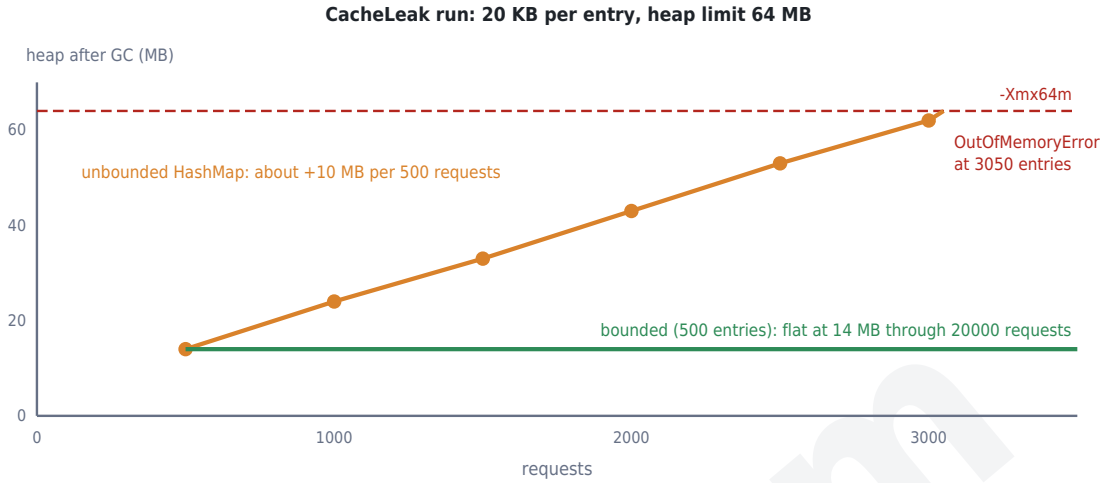


Figure 5.22. Heap after garbage collection for an unbounded and a bounded cache: the unbounded curve climbs linearly until the heap limit and ends in OutOfMemoryError, while the bounded curve flattens at the size of the working set.

Root cause. A collection used as a cache without an eviction policy. Entries are strongly reachable from a static or singleton field, so the garbage collector cannot reclaim them; every distinct key adds permanent memory.

Fix. Replace the map by a bounded, expiring cache: Spring's `@Cacheable` with Caffeine (`maximumSize`, `expireAfterWrite`) as configured in the worked example, or `Caffeine.newBuilder().maximumSize(10_000).expireAfterWrite(Duration.ofMinutes(10)).recordStats().build()` for a manual cache. Choose keys with a bounded domain; keying by `orderId` for data that is rarely read twice yields a hit ratio near zero and only costs memory.

Prevention. Alert on the post-GC heap trend, not the instantaneous usage. Record cache statistics and review the hit ratio (`cache.gets{result="hit"}` against misses) and `cache.evictions`; a cache with a low hit ratio is removed. In code review, treat every `static` collection and every `Map` field of a singleton bean as a potential leak.

Problem 2: Latency spikes caused by garbage collection

Scenario. Requests normally complete in 40 ms. Every few minutes, or constantly under load, a share of requests takes one to several seconds. CPU is not saturated and the database is idle.

Symptoms. The p99 line on the latency dashboard shows plateaus that coincide with `jvm.gc.pause` maxima; `jstat -gcutil` shows `FGC` (full collections) increasing; the GC log contains many consecutive full collections that free almost nothing.

Diagnosis. Enable or read the GC log (`-Xlog:gc*:file=gc.log`) and look at three things: the duration of pauses against the latency objective, the frequency of collections, and the heap occupancy after each collection. The log of the leak program above, run with a 300 MB heap, shows the terminal state of a heap that is full of live data:

Excerpt of a real GC log (build machine, -Xmx300m, LeakSlow running without eviction) TERMINAL

```
[9.361s][info][gc] GC(101) Pause Full (G1 Compaction Pause) 294M->294M(300M) 10.288ms
[9.363s][info][gc] GC(102) Pause Young (Concurrent Start) (G1 Evacuation Pause) 294M->294M
```

The two lines show the signature of an exhausted heap: a full collection that frees nothing (`294M->294M`) immediately followed by another collection. Short pauses of this size are harmless on their own; the harm is that the application spends its time collecting, and a real service with a 2 GB live set sees pauses of seconds. Further diagnostic steps: `jcmd <pid> GC.heap_info` for the generation sizes; the JFR events `jdk.GarbageCollection` and `jdk.GCPhasePause`; the Micrometer meters `jvm.gc.pause`, `jvm.gc.memory.allocated` and `jvm.gc.live.data.size`.

Root cause. Three distinct causes produce the same symptom and are told apart by the post-collection occupancy: (a) a leak or an oversized cache, where occupancy after collection climbs toward the maximum (fix as in problem 1); (b) a heap that is too small for the legitimate live set, where occupancy after collection is stable but near the maximum

(increase `MaxRAMPercentage` or the container limit); (c) a high allocation rate with large short-lived objects, where occupancy after collection is low but young collections are frequent, or humongous allocations (objects of at least half a G1 region) force expensive collections.

Fix. For (a), fix the retention. For (b), raise the heap while keeping room for non-heap memory. For (c), reduce allocation (stream large results instead of materialising them, avoid per-request copies of large buffers, paginate), and only then consider tuning: a larger heap lowers collection frequency; `-XX:MaxGCPauseMillis` expresses a goal to G1; a switch to ZGC is justified when pauses must be in the low-millisecond range on large heaps.

Prevention. Keep GC logging on, alert on `jvm.gc.pause` maxima against the latency objective, and load-test with production-sized data before release, because a GC problem that depends on the live-set size does not appear with test-sized data.

Problem 3: Slow start-up in Kubernetes, though local start-up is fast

Scenario. The application starts in 9 s on a developer laptop but needs 70 s in the cluster, where it sometimes fails its probes (see the readiness and liveness problems in the chapter on Actuator) and slows rollouts and autoscaling.

Symptoms. The log shows `Started OrderApplication in 68.4 seconds`; CPU usage of the container sits at its limit during start-up; `container_cpu_cfs_throttled_seconds_total` rises; the same image started with more CPU is much faster.

Diagnosis. (1) Compare resources: `kubectl describe pod` shows `limits.cpu: 500m`. The JVM sizes its internal thread pools and the JIT compiler from the CPU count it detects, and start-up is CPU-intensive (class loading, parsing, JIT compilation, bean creation). (2) Check throttling: the metric above, or `/sys/fs/cgroup/cpu.stat` inside the container (`nr_throttled`, `throttled_usec`). (3) Obtain the start-up breakdown with Boot's `startup` endpoint (`BufferingApplicationStartup`, `spring.main.lazy-initialization` comparisons) or `-Xlog:class+load` counts; look for beans with long initialisation (remote calls, migrations, cache warming) as opposed to uniformly slow progress.

A small measurement on the build machine shows the direction of the effect when the process is restricted to one CPU with `taskset` (median of 11 runs of the class-loading program used earlier; the machine was shared with other jobs, so the figures are indicative only):

Effect of CPU availability on a short start-up (real run; shared machine, indicative) TERMINAL

```
$ java -cp boot.jar Boot                 # 2 CPUs available
136 ms
$ taskset -c 0 java -cp boot.jar Boot    # restricted to 1 CPU
189 ms
```

A CFS quota (`limits.cpu` in Kubernetes) is harsher than this affinity restriction: when the quota for a 100 ms period is used up, **all** threads of the container are paused until the next period, so a 500m limit interrupts a start-up that needs several cores' worth of work in bursts.

Root cause. CPU limits below what start-up needs, so that the work is spread over many throttled periods; additional contributors are expensive start-up work (component scanning, migrations, cache warming) and a cold JIT.

Fix. Give the container a higher CPU request and limit, or a high limit with a modest request (a pattern that permits bursts at start-up); remove the limit for latency-sensitive services where cluster policy allows. Reduce work: CDS archive or AOT cache, `spring.main.lazy-initialization` for rarely used beans, no warm-up in `@PostConstruct` before readiness (use readiness gating instead), migrations as a separate job. Protect against the interaction with probes by adding a startup probe sized for the worst case.

Prevention. Track `application.started.time` and `application.ready.time` as metrics, alert when they exceed a budget, and measure in the cluster with realistic limits rather than on a laptop.

Problem 4: Requests time out although CPU and database are idle (thread pool saturation)

Scenario. A service calls a payment provider whose latency rises from 100 ms to 3 s during an incident. The service's own CPU stays at 15 percent and the database is idle, yet its endpoints begin to time out, including those that never call the provider (for example `/health` and `/products`).

Symptoms. `tomcat.threads.busy` equals `tomcat.threads.config.max` (200) and stays there; `http.server.requests` shows latency rising for every endpoint; connections queue in the accept queue and clients receive timeouts or `Connection refused`; thread dumps show nearly all threads waiting in the HTTP client.

Diagnosis. (1) Confirm saturation in metrics: `tomcat_threads_busy_threads / tomcat_threads_config_max_threads` near 1 (this requires `server.tomcat.mbeanregistry.enabled=true`). (2) Group the stacks of a thread dump: `jcmd <pid> Thread.print | grep -A8 "http-nio" | grep "at " | sort | uniq -c | sort -rn | head`, or the Actuator

`threaddump` endpoint. Many identical stacks ending in `SocketInputStream.read`, `HttpClient` or `HikariPool.getConnection` identify the resource that holds the threads. (3) Check whether timeouts are configured on the outgoing client: no read timeout means a request can occupy a worker indefinitely.

The earlier `Saturation` program shows the quantitative picture: with 10 workers and 100 ms per task, the capacity is 100 per second, and a sustained 130 per second makes the queue grow by 30 entries per second with latency rising from 100 ms to over 500 ms even though no task fails. A slow dependency has the same effect by lengthening the service time: 200 workers and a 3 s service time give a capacity of only 67 requests per second.

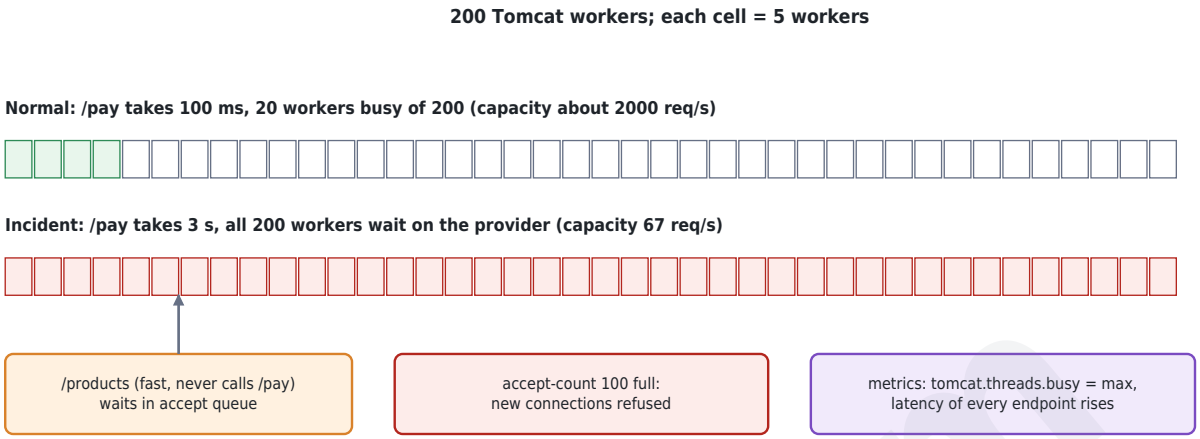


Figure 5.23. Worker threads occupied by a slow dependency: when each request holds a thread for 3 s instead of 100 ms, 200 workers sustain 67 requests per second instead of 2000, and unrelated endpoints queue behind the blocked ones.

Root cause. A shared, bounded pool whose service time was extended by a slow dependency, with no timeout, no isolation and no load shedding. Because the pool is shared by all endpoints, one slow dependency degrades all of them.

Fix. Set connect and read timeouts on every outgoing client (see the HTTP clients chapter); add a bulkhead or semaphore limiting concurrent calls to the slow dependency so that the remaining capacity serves other endpoints (Resilience4j `@Bulkhead`, covered in the Microservices part); use a circuit breaker with a fallback; consider virtual threads (`spring.threads.virtual.enabled=true`) so that blocked calls do not occupy platform threads, while limiting concurrency toward the dependency explicitly and sizing the connection pool; shed load early (return 503 with `Retry-After`) instead of queueing without bound. Increasing `server.tomcat.threads.max` alone only postpones the failure and raises memory use by about 1 MB per thread stack.

Prevention. Alert on saturation (busy/max above 0.8 for five minutes, pending connections above zero), not only on errors; test the failure mode with a stub that delays responses (the WireMock fixed delay shown in the integration testing chapter) and verify that unrelated endpoints stay fast.

5.6.5 Questions to expect

How is a performance problem diagnosed in a running Spring Boot service?

From the outside in: metrics (RED for the endpoint, USE for the resources) locate the symptom and the exhausted resource; a thread dump shows where threads wait; JFR or async-profiler shows where CPU time goes; the GC log and `jstat` show memory behaviour; and a change is verified by measuring again under the same load.

When do virtual threads improve throughput, and when not?

When requests spend most of their time blocked on I/O and the limit is the number of request threads. They do not help CPU-bound work, and the limits of downstream resources (connection pool, remote services) remain. Pinning in `synchronized` blocks (JDK 21) can negate the benefit.

What happens when spring.threads.virtual.enabled=true is set?

On Java 21 or later, Tomcat or Jetty handles each request on a virtual thread, and the auto-configured async task executor and scheduler also use virtual threads; settings that configure these thread pools no longer apply.

Which garbage collector does a Spring Boot application use by default, and when is another chosen?

G1 on machines with at least two CPUs and about 1.8 GB of memory (Serial otherwise). Parallel suits throughput-oriented batch jobs, ZGC very large heaps or strict pause goals, Serial very small containers.

How is the heap sized in a container?

With `-XX:MaxRAMPercentage` (default 25), leaving room for metaspace, thread stacks, code cache and direct memory; a container limit equal to the heap size results in `OOMKilled`.

How is a memory leak found?

Observe the post-GC heap trend, compare two class histograms (`jcmd GC.class_histogram`), take a heap dump (`jcmd GC.heap_dump`) and inspect the dominator tree and GC root paths in MAT or VisualVM.

How can start-up time be reduced?

Remove unused auto-configuration and heavy start-up work, use CDS or the AOT cache, AOT processing, selective lazy initialisation, and, for scale-to-zero workloads, a GraalVM native image; measure start-up in the target cluster with its CPU limits.

What are the trade-offs of lazy initialisation?

Faster start and less memory at start-up, but errors surface at first use, the first request pays the cost, and the memory needed in steady state is not visible at start-up.

How do you read a flame graph?

Width is the share of samples containing a frame; height is stack depth. The widest boxes near the top mark where time is spent; comparing graphs before and after a change verifies a fix.

Why is a closed-loop load test misleading?

Virtual users wait for each response, so when the service slows down the arrival rate drops and queueing is hidden. An open-loop test with a fixed arrival rate shows latency under real offered load.

KEY POINTS

Key points

- Measure first: define the objective, find the exhausted resource with metrics, thread dumps, GC logs and profiles, change one thing, measure again.
- Throughput is limited by the scarcest resource; queues and pending counters (saturation) warn before errors appear.
- Virtual threads (`spring.threads.virtual.enabled=true`, Java 21+) remove the request-thread limit for blocking code, but the connection pool and downstream services become the limit; mind pinning on JDK 21.
- G1 is the default and sufficient for most services; size the heap by percentage, leave room for non-heap memory, and log GC permanently.
- Every cache must be bounded and observable; an unbounded map is a leak that ends in OutOfMemoryError.
- JFR, `jcmd`, `jstat` and flame graphs work in production; shorten start-up with less work, CDS or the AOT cache, and sensible CPU limits.

Sriram

PART 6

Messaging, Delivery and Interview Preparation

Asynchronous messaging, caching, packaging, deployment and the interview material that ties the whole guide together.

In this part

- **6.1** Messaging with Spring and Kafka
- **6.2** RabbitMQ, Scheduling, Async Work and Events
- **6.3** Caching with Redis in Spring
- **6.4** Packaging and Containers
- **6.5** Kubernetes for Spring Developers
- **6.6** CI/CD, Maven and Release Strategies
- **6.7** Interview Question Bank
- **6.8** System Design Scenarios for Spring Services
- **6.9** Cheat Sheets
- **6.10** Six-Week Study Plan
- **6.11** Putting it Together: One Request through a Spring Boot Microservice

6.1 Messaging with Spring and Kafka

A synchronous call couples the caller to the availability, latency and capacity of the callee. When an order service calls a payment service, a notification service and an inventory service in sequence, the response time of the order request is the sum of the three, and a failure in any one of them becomes a failure of the order. Asynchronous messaging removes that coupling: the order service records the fact that an order was created and returns, and each interested service consumes the fact at its own pace. The price is a different consistency model (eventual consistency), a new class of failure (duplicates, reordering, lag) and new operational surface (the broker).

This chapter covers the Spring side of that design: `KafkaTemplate`, `@KafkaListener`, serialisation, error handling, retry topics, dead-letter topics, transactions, rebalancing and the outbox pattern. The Kafka concepts themselves (log structure, replication, broker internals) are summarised only to the depth needed to reason about Spring behaviour; a separate Kafka guide covers the broker in depth.

6.1.1 Concept and mechanism

Queue versus log

A classic message queue (RabbitMQ, ActiveMQ) holds messages until a consumer acknowledges them; the broker then deletes the message. Delivery state lives in the broker, routing is flexible (exchanges, bindings), and each message is processed by one consumer of a queue. A log (Kafka) is an append-only, partitioned, replicated sequence of records. The broker never tracks per-message delivery. It retains records for a configured time or size, and each consumer group stores only an offset per partition. Consequences: records can be replayed, any number of independent groups can read the same topic, and the throughput per partition is sequential-disk fast. The cost is that ordering is per partition only, and that individual messages cannot be acknowledged out of order without extra machinery.

Property	Queue (RabbitMQ)	Log (Kafka)
Message after consumption	deleted on ack	retained until retention expires
Replay of old data	not possible	seek to any offset
Consumer state	broker tracks per message	consumer group tracks offset per partition
Ordering	per queue, weakened by requeue	per partition, strict
Typical use	task distribution, routing, RPC style	event streams, integration, audit, analytics

Queue broker compared with a log broker.

Kafka concepts used by Spring

A **topic** is split into **partitions**. Each partition is an ordered log in which every record has an **offset**. A producer chooses the partition: with a non-null key, the default partitioner hashes the key (murmur2) modulo the partition count, so all records of one key land in one partition and keep their relative order. A **consumer group** is a set of consumers sharing a `group.id`; the group coordinator on the broker assigns each partition to exactly one member. Parallelism of a group is therefore bounded by the partition count. The group commits the **offset** of the next record to read, per partition, into the internal topic `__consumer_offsets`. After a restart or a rebalance, consumption resumes at the committed offset.

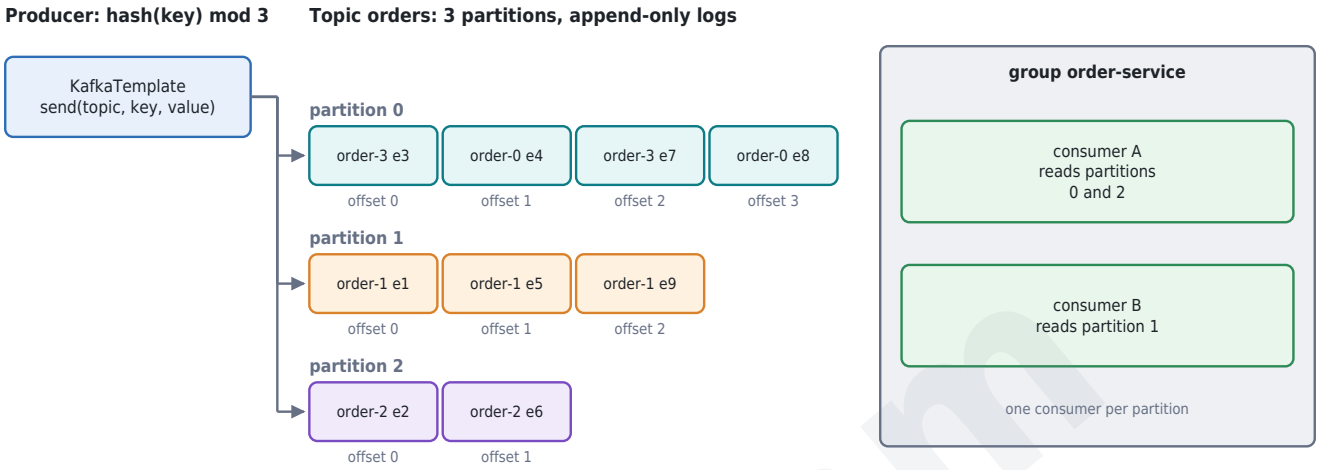


Figure 6.1. The key decides the partition (order-1 always lands in partition 1), offsets grow per partition, and the two consumers of the group split the partitions between them.

The Spring layers

Spring Boot auto-configures the following beans when `spring-kafka` is on the classpath and `spring.kafka.*` properties exist: a `KafkaAdmin` (creates `NewTopic` beans at startup), a `ProducerFactory` and `KafkaTemplate`, a `ConsumerFactory` and a `ConcurrentKafkaListenerContainerFactory`. The annotation `@KafkaListener` is processed by `KafkaListenerAnnotationBeanPostProcessor`, which registers a `KafkaListenerEndpoint` with the `KafkaListenerEndpointRegistry`. For each endpoint the registry creates a `ConcurrentMessageListenerContainer`, which in turn starts `concurrency` instances of `KafkaMessageListenerContainer`. Each of those owns one `KafkaConsumer` and one thread that runs the poll loop: `poll()`, invoke the listener for each record, commit offsets according to the ack mode, repeat. The listener method therefore runs on the consumer thread, and a slow listener delays the next `poll()`.

The producer path

`KafkaTemplate.send()` serialises the key and value, selects a partition, and appends the record to a per-partition batch in the `RecordAccumulator`. A dedicated sender thread ships the batch when it is full (`batch.size`, default 16 KiB) or when `linger.ms` expires (default 0, meaning send as soon as possible). The call returns a `CompletableFuture<SendResult>` (Spring for Apache Kafka 3.0 and later; earlier versions returned `ListenableFuture`). Completion of that future, not return of `send()`, is the moment at which the broker acknowledged the record.

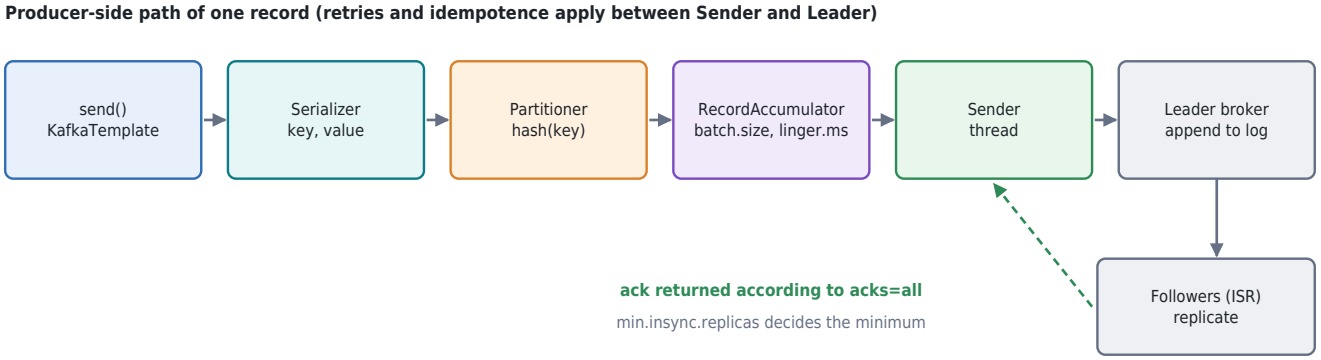


Figure 6.2. A record passes through serialiser, partitioner and accumulator before the sender thread delivers it to the partition leader, which acknowledges according to the acks setting.

With `acks=all` the leader replies only after all in-sync replicas have the record; `min.insync.replicas` (for example 2 with replication factor 3) sets the lower bound of replicas that must be in sync for a write to succeed at all. Since Kafka clients 3.0 the defaults are `acks=all` and `enable.idempotence=true`; the idempotent producer attaches a producer id and a per-partition sequence number so that the broker discards duplicates caused by internal retries. Idempotence protects against duplicates inside one producer session only; it does not make an application-level retry (a second `send()` of the same business event) idempotent.

6.1.2 Worked example

The worked example is an `order-service` that publishes an `OrderCreated` event and a `payment-service` that consumes it. All files of the producer side are shown; the consumer repeats the dependency block.

pom.xml (fragment) XML

```xml
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.kafka</groupId>
        <artifactId>spring-kafka</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.kafka</groupId>
        <artifactId>spring-kafka-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

The versions of `spring-kafka` and the Kafka client are managed by the Spring Boot BOM; no version element is written.

application.yml (order-service) YAML

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      properties:
        enable.idempotence: true
        delivery.timeout.ms: 30000
        linger.ms: 5
        spring.json.add.type.headers: false
```

The property `spring.json.add.type.headers=false` stops `JsonSerializer` from writing a `__TypeId__` header that contains the fully qualified producer class name. Consumers in other services do not have that class, and a type header couples the two code bases.

sreeram

OrderCreated.java JAVA

```java
package com.example.orders.events;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.UUID;

public record OrderCreated(
        UUID eventId,
        UUID orderId,
        UUID customerId,
        BigDecimal total,
        Instant occurredAt) {
}
```

The event carries its own `eventId`. That identifier is what makes an idempotent consumer possible later in this chapter.

KafkaTopics.java JAVA

```java
package com.example.orders.config;

import org.apache.kafka.clients.admin.NewTopic;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.TopicBuilder;

@Configuration
public class KafkaTopics {

    public static final String ORDER_EVENTS = "orders.events";

    @Bean
    NewTopic orderEvents() {
        return TopicBuilder.name(ORDER_EVENTS)
                .partitions(6)
                .replicas(3)
                .config("min.insync.replicas", "2")
                .config("retention.ms", String.valueOf(7L * 24 * 3600 * 1000))
                .build();
    }
}
```

`KafkaAdmin` creates the topic at startup if it is missing. In production, topics are normally created by infrastructure code, and the bean is kept for local development and tests.

OrderEventPublisher.java JAVA

```java
package com.example.orders.messaging;

import com.example.orders.config.KafkaTopics;
import com.example.orders.events.OrderCreated;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.stereotype.Component;

import java.util.concurrent.CompletableFuture;

@Component
public class OrderEventPublisher {

    private static final Logger log = LoggerFactory.getLogger(OrderEventPublisher.class);

    private final KafkaTemplate<String, OrderCreated> template;

    public OrderEventPublisher(KafkaTemplate<String, OrderCreated> template) {
        this.template = template;
    }

    public CompletableFuture<SendResult<String, OrderCreated>> publish(OrderCreated event) {
        String key = event.orderId().toString();       // same order, same partition
        return template.send(KafkaTopics.ORDER_EVENTS, key, event)
                .whenComplete((result, ex) -> {
                    if (ex != null) {
                        log.error("publish failed orderId={}", key, ex);
```

OrderEventPublisher.java (continued) JAVA

```
                    } else {
                        var md = result.getRecordMetadata();
                        log.info("published orderId={} partition={} offset={}",
                                key, md.partition(), md.offset());
                    }
                });
    }
}
```

What to notice: the key is the order identifier, so all events of one order are ordered relative to each other; the returned future is not ignored, because an ignored future silently discards failures; and the publisher is called after the database commit in this first version, which has a failure window that the outbox section closes.

The consumer side uses `ErrorHandlingDeserializer` so that a malformed payload is routed to the error handler instead of blocking the partition in an endless deserialisation loop.

application.yml (payment-service) YAML

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: payment-service
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
      max-poll-records: 100
      properties:
        spring.deserializer.value.delegate.class:
          org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.value.default.type: com.example.payments.events.OrderCreated
        spring.json.use.type.headers: false
        spring.json.trusted.packages: com.example.payments.events
        max.poll.interval.ms: 300000
    listener:
      ack-mode: record
      concurrency: 3
```

OrderCreatedListener.java JAVA

```
package com.example.payments.messaging;

import com.example.payments.events.OrderCreated;
import com.example.payments.service.PaymentService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.support.KafkaHeaders;
import org.springframework.messaging.handler.annotation.Header;
import org.springframework.stereotype.Component;

@Component
public class OrderCreatedListener {

    private static final Logger log = LoggerFactory.getLogger(OrderCreatedListener.class);

    private final PaymentService payments;

    public OrderCreatedListener(PaymentService payments) {
        this.payments = payments;
    }

    @KafkaListener(topics = "orders.events", groupId = "payment-service")
    public void onOrderCreated(OrderCreated event,
                               @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
                               @Header(KafkaHeaders.OFFSET) long offset) {
        log.info("received orderId={} partition={} offset={}",
                event.orderId(), partition, offset);
        payments.authorise(event);
    }
}
```

The consumer declares its own copy of `OrderCreated` in its own package. Sharing a jar of event classes across services is convenient but creates build-time coupling; a schema (JSON Schema or Avro, see below) is the contract, not the class.

6.1.3 Hands-on

The mechanism behind ordering, group assignment and duplicate delivery can be demonstrated without a broker. The following plain-JDK program implements a partitioned log, a key-based partitioner, a consumer group with range-style assignment and committed offsets. It then simulates a consumer that processes records and crashes before committing, which is the situation behind every duplicate in Kafka consumers. The program is deterministic because `String.hashCode()` is specified.

MiniLog.java JAVA

```
import java.util.*;
import java.util.stream.*;

public class MiniLog {

    record Rec(String key, String value, int partition, long offset) { }

    static class Topic {
        final List<List<Rec>> partitions = new ArrayList<>();
        Topic(int n) { for (int i = 0; i < n; i++) partitions.add(new ArrayList<>()); }
        Rec append(String key, String value) {
            int p = Math.floorMod(key.hashCode(), partitions.size());
            List<Rec> log = partitions.get(p);
            Rec r = new Rec(key, value, p, log.size());
            log.add(r);
            return r;
        }
    }

    static class Group {
        final Topic topic;
        final Map<Integer, Long> committed = new HashMap<>();
        final Map<String, List<Integer>> assignment = new TreeMap<>();
        Group(Topic t) {
            topic = t;
            for (int p = 0; p < t.partitions.size(); p++) committed.put(p, 0L);
        }

        void rebalance(Collection<String> members) {
            assignment.clear();
            List<String> ms = new ArrayList<>(members); Collections.sort(ms);
            for (String m : ms) assignment.put(m, new ArrayList<>());
            for (int p = 0; p < topic.partitions.size(); p++)
                assignment.get(ms.get(p % ms.size())).add(p);
            System.out.println("rebalance -> " + assignment);
        }

        // Poll up to max records; process; commit only every commitEvery records.
        List<Rec> poll(String member, int max) {
            List<Rec> out = new ArrayList<>();
            for (int p : assignment.getOrDefault(member, List.of())) {
                long off = pos.getOrDefault(member + p, committed.get(p));
                List<Rec> log = topic.partitions.get(p);
                while (off < log.size() && out.size() < max) out.add(log.get((int) off++));
                pos.put(member + p, off);
            }
            return out;
        }
        final Map<String, Long> pos = new HashMap<>();
        void commit(String member) {
            for (int p : assignment.getOrDefault(member, List.of()))
                if (pos.containsKey(member + p)) committed.put(p, pos.get(member + p));
        }
        void forgetPositions() { pos.clear(); }
    }

    public static void main(String[] args) {
        Topic orders = new Topic(3);
        for (int i = 1; i <= 9; i++) {
            Rec r = orders.append("order-" + (i % 4), "event-" + i);
```

MiniLog.java (continued) JAVA

```
            System.out.printf("produced %-8s -> partition %d offset %d%n",
                r.key(), r.partition(), r.offset());
        }
        System.out.println("-- per-key ordering check");
        for (int p = 0; p < 3; p++)
            System.out.println("partition " + p + ": " + orders.partitions.get(p).stream()
                .map(r -> r.key() + "/" + r.value())
                .collect(Collectors.joining(", ")));

        Group g = new Group(orders);
        g.rebalance(List.of("A"));
        List<Rec> batch = g.poll("A", 4);
        System.out.println("A polled " + batch.stream().map(Rec::value).toList());
        g.commit("A");
        System.out.println("A committed " + g.committed);

        g.rebalance(List.of("A", "B"));
        g.forgetPositions();
        System.out.println("-- A polls 3 more, processes them, crashes BEFORE commit");
        List<Rec> more = g.poll("A", 3);
        System.out.println("A processed " + more.stream().map(Rec::value).toList()
            + " (not committed)");

        g.rebalance(List.of("B"));
        g.forgetPositions();
        List<Rec> redo = g.poll("B", 100);
        System.out.println("B polled " + redo.stream().map(Rec::value).toList());
        Set<String> seen = new HashSet<>();
        Stream.concat(batch.stream(), Stream.concat(more.stream(), redo.stream()))
            .forEach(r -> {
                if (!seen.add(r.value()))
                    System.out.println("DUPLICATE delivery: " + r.value());
            });
        System.out.println("distinct events delivered: " + seen.size() + " of 9");
    }
}
```

Output of the in-memory log simulation (run on the build machine) TERMINAL

```
$ java MiniLog.java
produced order-1  -> partition 1 offset 0
produced order-2  -> partition 2 offset 0
produced order-3  -> partition 0 offset 0
produced order-0  -> partition 0 offset 1
produced order-1  -> partition 1 offset 1
produced order-2  -> partition 2 offset 1
produced order-3  -> partition 0 offset 2
produced order-0  -> partition 0 offset 3
produced order-1  -> partition 1 offset 2
-- per-key ordering check
partition 0: order-3/event-3, order-0/event-4, order-3/event-7, order-0/event-8
partition 1: order-1/event-1, order-1/event-5, order-1/event-9
partition 2: order-2/event-2, order-2/event-6
rebalance -> {A=[0, 1, 2]}
A polled [event-3, event-4, event-7, event-8]
A committed {0=4, 1=0, 2=0}
rebalance -> {A=[0, 2], B=[1]}
-- A polls 3 more, processes them, crashes BEFORE commit
A processed [event-2, event-6] (not committed)
rebalance -> {B=[0, 1, 2]}
B polled [event-1, event-5, event-9, event-2, event-6]
DUPLICATE delivery: event-2
DUPLICATE delivery: event-6
distinct events delivered: 9 of 9
```

Three observations follow from the output. First, all records with key `order-1` are in partition 1 in production order (`event-1`, `event-5`, `event-9`), which is the ordering guarantee. Second, the committed offsets after the first poll are `{0=4, 1=0, 2=0}`: only partition 0 was read and committed. Third, consumer A processed `event-2` and `event-6` but crashed before committing; the new owner resumed from committed offset 0 on partition 2 and delivered both again. Nothing was lost, and two records were processed twice. This is **at-least-once** delivery.

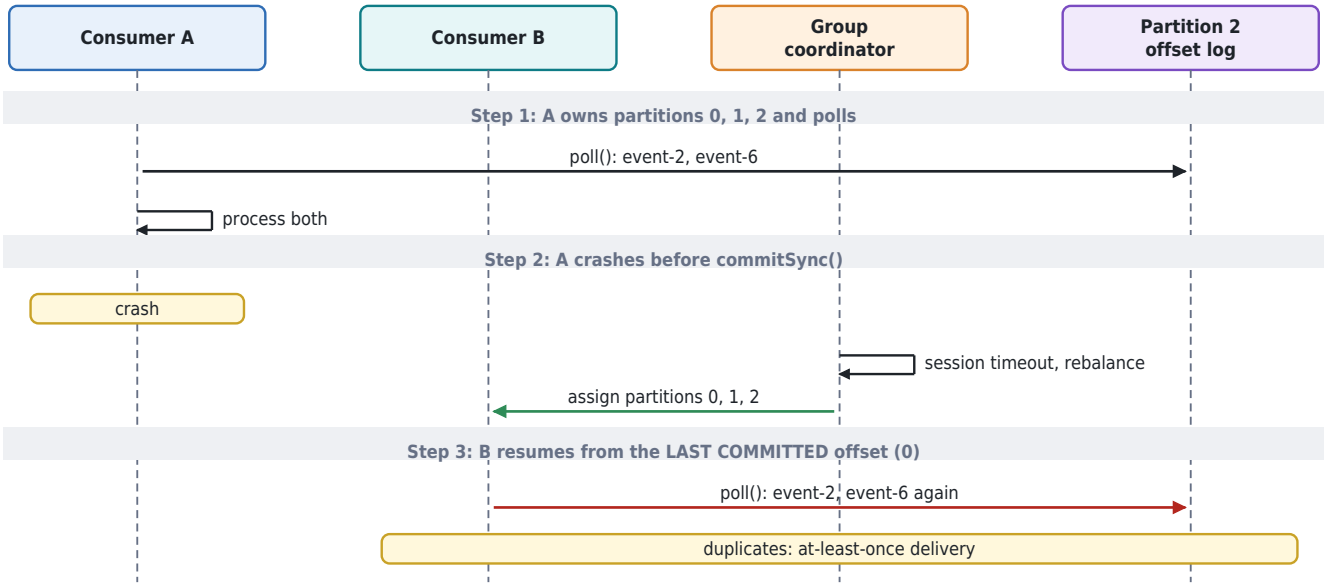


Figure 6.3. Processing happens before the offset commit, so a crash between the two causes redelivery to the next owner of the partition.

A real `kafka-consumer-groups.sh` inspection shows the same state per partition. The following session is the form of the output and is not from the build machine.

typical output (not run on the build machine) — TERMINAL

```
$ kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
    --describe --group payment-service
GROUP            TOPIC          PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
payment-service  orders.events  0          1520            1523            3
payment-service  orders.events  1          1498            1498            0
payment-service  orders.events  2          1311            1330            19
```

The column `LAG` is `LOG-END-OFFSET` minus `CURRENT-OFFSET` (the committed offset). Lag per partition, not the total, shows whether a single partition is stuck.

6.1.4 Offsets, ack modes and delivery semantics

When the offset is committed relative to the processing defines the delivery guarantee.

- Commit before processing gives **at-most-once**: a crash loses the record.
- Commit after processing gives **at-least-once**: a crash duplicates the record.
- **Exactly-once** across arbitrary side effects does not exist. Kafka provides exactly-once semantics only for the read-process-write cycle inside Kafka (transactions); a side effect such as a database write or an HTTP call is outside that boundary and must be made idempotent.

With `enable.auto.commit=true` the Kafka consumer itself commits the offsets returned by the last `poll()` every `auto.commit.interval.ms` (5 s by default) on the next poll. Those offsets may include records that are still in progress in the listener, and a crash after such a commit loses them. Spring therefore sets auto-commit to false in the container and commits through the `AckMode`:

AckMode	Behaviour
`RECORD`	commit after each record has been processed by the listener
`BATCH`	default; commit after all records returned by one poll() were processed
`TIME, COUNT, COUNT_TIME`	commit after a time or record threshold
`MANUAL`	listener calls Acknowledgment.acknowledge(); the container commits at the end of the batch
`MANUAL_IMMEDIATE`	the commit is executed immediately on acknowledge()

Spring Kafka listener ack modes (ContainerProperties.AckMode).

`RECORD` costs one commit round trip per record and bounds the replay window after a crash to a single record. `BATCH` is cheaper and replays up to `max.poll.records` records. A rule of thumb for a business service: use `RECORD` or `BATCH`, keep the listener idempotent, and never use broker auto-commit.

6.1.5 Consumer groups and rebalancing

A rebalance moves partitions between group members. It is triggered by a member joining or leaving, a missed heartbeat (`session.timeout.ms`, default 45 s in Kafka 3.0 and later), a poll loop that does not call `poll()` within `max.poll.interval.ms` (default 5 minutes), or a change in the partition count. During an eager rebalance (the behaviour of `RangeAssignor` and `RoundRobinAssignor`) every member revokes all partitions and processing stops group-wide. The `CooperativeStickyAssignor` moves only the partitions that must move; Kafka clients since 3.0 list `RangeAssignor` followed by `CooperativeStickyAssignor` as the default, which still allows an eager protocol in practice until `RangeAssignor` is removed from the list. Setting `partition.assignment.strategy` to `org.apache.kafka.clients.consumer.CooperativeStickyAssignor` once all members are upgraded gives incremental rebalancing. Kafka 4.0 additionally introduces a broker-driven consumer group protocol (KIP-848, enabled client-side by `group.protocol=consumer`) that removes the group-wide synchronisation barrier; its adoption in Spring Boot applications should be checked against the Spring for Apache Kafka version in use.

Two settings reduce unnecessary rebalances in Kubernetes, where pods restart often: **static membership** (`group.instance.id` set to the stable pod name of a StatefulSet; a restart within `session.timeout.ms` does not trigger a rebalance) and a `max.poll.interval.ms` that exceeds the worst-case processing time of one `poll()` batch.

Spring exposes rebalance events through `ConsumerAwareRebalanceListener` and publishes `ConsumerStartedEvent`, `ListenerContainerIdleEvent` and similar application events. Offsets for in-flight records should be committed in `onPartitionsRevoked` when `MANUAL` acknowledgement is used.

INTERVIEW ANGLE

How many consumers should a group have, and what happens if there are more consumers than partitions? At most one consumer per partition receives data. With six partitions, a group of eight consumers has two idle members. The partition count is therefore the upper bound of parallelism, and it must be chosen with expected peak throughput in mind because increasing it later changes the key-to-partition mapping and breaks per-key ordering across the change.

6.1.6 Error handling, retry topics and dead-letter topics

The container invokes the listener inside a try block and passes any exception to a `CommonErrorHandler`. Since Spring for Apache Kafka 2.8 the default is `DefaultErrorHandler`, which replaces the older `SeekToCurrentErrorHandler`. On failure it **seeks** the consumer back to the failed record (so the failed record and all unprocessed records of the poll are redelivered by the next `poll()`), waits according to its `BackOff`, and after the back-off is exhausted calls a **recoverer**. The default back-off is `FixedBackOff(0L, 9)`: no delay and nine retries, that is ten delivery attempts in total, after which the record is logged at ERROR level and skipped. Because the retries run on the consumer thread with the partition blocked, a long back-off delays every record behind the failing one and counts towards `max.poll.interval.ms`.

Some exceptions are classified as not retryable and go straight to the recoverer: `DeserializationException`, `MessageConversionException`, `ConversionException`, `MethodArgumentResolutionException`, `NoSuchMethodException` and `ClassCastException`. Further types are added with `addNotRetryableExceptions(...)`. Only `RuntimeException` subclasses are handled; an `Error` terminates the consumer thread.

The recommended recoverer is `DeadLetterPublishingRecoverer`, which publishes the failed record to `<topic>-dlt` on the same partition number, with headers that carry the exception class, message, stack trace and the original topic, partition and offset. The dead-letter topic therefore needs at least as many partitions as the source topic unless a custom destination resolver is supplied.

KafkaConsumerConfig.java JAVA

```java
package com.example.payments.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.KafkaOperations;
import org.springframework.kafka.listener.DeadLetterPublishingRecoverer;
import org.springframework.kafka.listener.DefaultErrorHandler;
import org.springframework.util.backoff.ExponentialBackOff;

@Configuration
public class KafkaConsumerConfig {

    @Bean
    DefaultErrorHandler errorHandler(KafkaOperations<Object, Object> template) {
        var recoverer = new DeadLetterPublishingRecoverer(template);
```

KafkaConsumerConfig.java (continued) JAVA

```
        var backOff = new ExponentialBackOff(500L, 2.0);   // 0.5 s, 1 s, 2 s, ...
        backOff.setMaxAttempts(4);                         // 4 retries, 5 deliveries
        backOff.setMaxInterval(5_000L);
        var handler = new DefaultErrorHandler(recoverer, backOff);
        handler.addNotRetryableExceptions(IllegalArgumentException.class);
        return handler;
    }
}
```

Spring Boot detects a single `CommonErrorHandler` bean and applies it to the auto-configured `ConcurrentKafkaListenerContainerFactory`; no explicit factory is needed. `ExponentialBackOff.setMaxAttempts` is available from Spring Framework 6.1; earlier versions bound the sequence with `setMaxElapsedTime`.

Non-blocking retries with @RetryableTopic

Blocking retries preserve order but stall the partition. For failures that are transient and independent per record (a downstream outage of two minutes), **retry topics** move the failing record away from the main topic and continue with the next record. Spring creates a chain of topics; the consumer of each retry topic pauses the partition until the due time stored in a header has passed.

PaymentRequestListener.java JAVA

```
package com.example.payments.messaging;

import com.example.payments.events.OrderCreated;
import com.example.payments.service.PaymentService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.DltHandler;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.annotation.RetryableTopic;
import org.springframework.kafka.retrytopic.DltStrategy;
import org.springframework.retry.annotation.Backoff;
import org.springframework.stereotype.Component;

@Component
public class PaymentRequestListener {

    private static final Logger log = LoggerFactory.getLogger(PaymentRequestListener.class);

    private final PaymentService payments;

    public PaymentRequestListener(PaymentService payments) {
        this.payments = payments;
    }

    @RetryableTopic(
            attempts = "4",
            backoff = @Backoff(delay = 1000, multiplier = 2.0),
            dltStrategy = DltStrategy.FAIL_ON_ERROR,
            include = { PaymentGatewayUnavailableException.class })
    @KafkaListener(topics = "orders.events", groupId = "payment-service")
    public void onOrderCreated(OrderCreated event) {
        payments.authorise(event);          // throws on a transient gateway failure
    }

    @DltHandler
    public void onDeadLetter(OrderCreated event) {
        log.error("payment permanently failed orderId={}", event.orderId());
        payments.markFailed(event);
    }
}
```

With four attempts and exponential back-off, the framework creates `orders.events-retry-1000`, `orders.events-retry-2000`, `orders.events-retry-4000` and `orders.events-dlt`. The annotation class `@Backoff` is the one from `org.springframework.retry.annotation`, so `spring-retry` must be on the classpath. The name of the exception class `PaymentGatewayUnavailableException` stands for any application exception marking a transient failure.

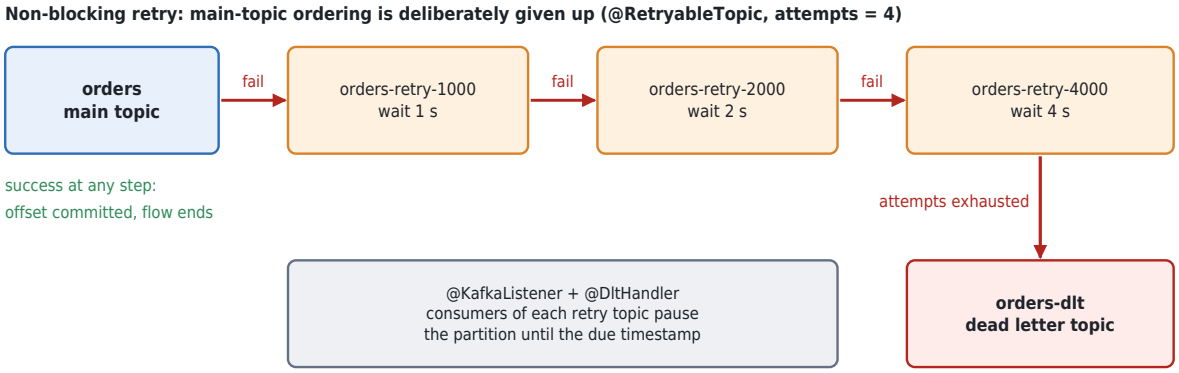


Figure 6.4. A failing record travels through retry topics with growing delays and ends in the dead-letter topic; the main topic keeps flowing, so ordering within a key is deliberately given up.

GOTCHA

Retry topics break per-key ordering. A second event for the same order may be processed while the first waits in a retry topic. Domain logic must then tolerate out-of-order arrival, for example by rejecting transitions that are illegal in the current state of the aggregate or by comparing a version number carried in the event.

Poison messages and the dead-letter topic

A **poison message** is a record that fails deterministically (a malformed payload, a violated business rule). Without a dead-letter path and a bounded retry count, the partition makes no progress; consumer lag on exactly one partition grows without bound. A dead-letter topic is useful only if it is monitored (an alert on records per minute), if records can be replayed after the cause is fixed (a small admin endpoint or a script that republishes from `-dlt` to the source topic), and if the headers allow root-cause analysis.

6.1.7 Producer reliability, idempotence and transactions

The producer settings that decide durability are `acks`, `enable.idempotence`, `retries` and `delivery.timeout.ms` (default 120 s, an upper bound on the total time including retries). With the defaults of Kafka clients 3.0 and later (`acks=all`, idempotence on, `retries=Integer.MAX_VALUE` bounded by the delivery timeout) a record is neither lost nor duplicated by transient network errors within one producer session. Order within a partition is retained for up to five in-flight requests when idempotence is enabled.

Kafka transactions let a producer write to several partitions atomically and, combined with `sendOffsetsToTransaction`, commit consumer offsets in the same transaction. This yields exactly-once for the consume-transform-produce pattern between topics. Spring enables it with a transaction id prefix; the `KafkaTransactionManager` or the container participates.

application.yml (transactional processor) YAML

```yaml
spring:
  kafka:
    producer:
      transaction-id-prefix: tx-enrich-
    consumer:
      isolation-level: read_committed
      enable-auto-commit: false
```

When `transaction-id-prefix` is set, Boot creates a `KafkaTransactionManager`, and the listener container starts a Kafka transaction for each record or batch; offsets and the records the listener sends via `KafkaTemplate` commit or abort together. Downstream consumers must use `isolation.level=read_committed` or they also read records of aborted transactions.

Limits that interview answers should state explicitly. Exactly-once applies only while all reads and writes are in Kafka; a database update or an e-mail sent inside the listener is not part of the transaction and repeats if the transaction aborts and the record is redelivered. Mixing a JPA transaction and a Kafka transaction gives at best best-effort one-phase commit (`ChainedTransactionManager` is deprecated and removed in recent releases); the supported answer for database plus Kafka is the outbox pattern, not a distributed transaction.

6.1.8 The outbox pattern with Kafka

The dual-write problem: a service must update its database and publish an event, and no transaction spans both. If the service commits and then crashes before `send()`, the event is lost; if it sends first and the commit fails, the event describes a state that never existed. The transactional outbox writes the event into an `outbox` table in the same database transaction as the business change. A separate relay reads unsent rows and publishes them to Kafka, then marks them as sent.

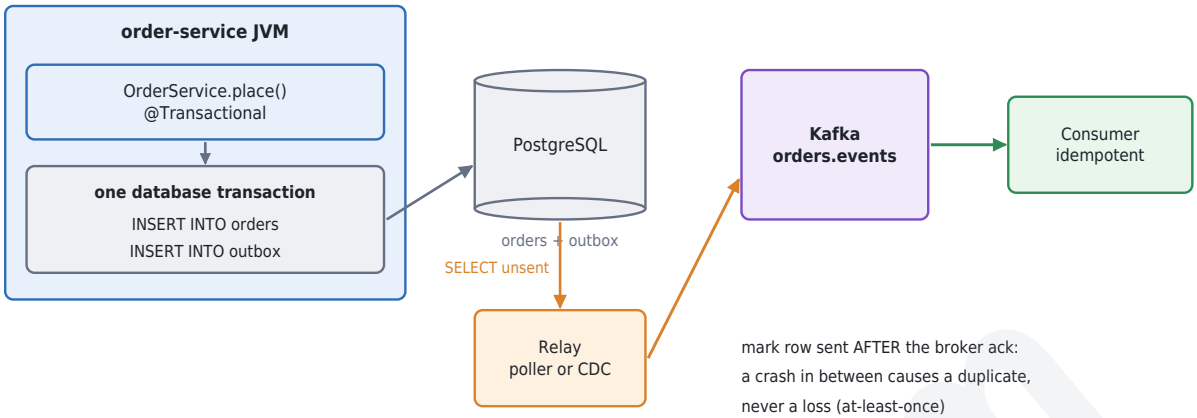


Figure 6.5. The business row and the outbox row commit atomically; the relay publishes and marks rows afterwards, so a crash between publish and mark produces a duplicate, never a loss.

The following plain-JDK program contrasts a dual write with an outbox. The in-memory `Db` rolls back all tables when a transaction body throws; crashes are injected after the commit, and a relay crash is injected after publishing but before marking the row as sent.

Outbox.java JAVA

```
import java.util.*;

public class Outbox {
    /** Tiny atomic store: tx() rolls back every table if the body throws. */
    static class Db {
        List<String> orders = new ArrayList<>();
        List<String[]> outbox = new ArrayList<>();      // {id, payload, sent}

        void tx(Runnable body) {
            List<String> o = new ArrayList<>(orders);
            List<String[]> ob = new ArrayList<>();
            for (String[] r : outbox) ob.add(r.clone());
            try {
                body.run();
            } catch (RuntimeException e) {
                orders = o;
                outbox = ob;
                throw e;
            }
        }
    }

    static final List<String> broker = new ArrayList<>();

    static void dualWrite(Db db, String id, boolean crashAfterCommit) {
        db.tx(() -> db.orders.add(id));                 // 1. commit to the database
        if (crashAfterCommit) throw new IllegalStateException("process killed");
        broker.add("OrderCreated:" + id);               // 2. publish (skipped on crash)
    }

    static void outboxWrite(Db db, String id, boolean crashAfterCommit) {
        db.tx(() -> {                                   // one atomic transaction
            db.orders.add(id);
            db.outbox.add(new String[] {id, "OrderCreated:" + id, "false"});
        });
        if (crashAfterCommit) throw new IllegalStateException("process killed");
    }
```

Outbox.java (continued) JAVA

```
    static void relay(Db db, boolean crashBeforeMarkSent) {
        for (String[] row : db.outbox) {
            if (row[2].equals("true")) continue;
            broker.add(row[1]);
            if (crashBeforeMarkSent) throw new IllegalStateException("relay died");
            row[2] = "true";
        }
    }

    static void attempt(Runnable r) {
        try {
            r.run();
        } catch (IllegalStateException e) {
            System.out.println("  !! " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        Db a = new Db();
        attempt(() -> dualWrite(a, "o-1", true));
        System.out.println("dual write   -> orders=" + a.orders + " broker=" + broker);

        broker.clear();
        Db b = new Db();
        attempt(() -> outboxWrite(b, "o-1", true));
        System.out.println("outbox write -> orders=" + b.orders + " outbox rows="
            + b.outbox.size() + " broker=" + broker);
        attempt(() -> relay(b, true));
        relay(b, false);
        System.out.println("relay restarted -> broker=" + broker);
    }
}
```

Output of the outbox simulation (run on the build machine) TERMINAL

```
$ java Outbox.java
  !! process killed
dual write   -> orders=[o-1] broker=[]
  !! process killed
outbox write -> orders=[o-1] outbox rows=1 broker=[]
  !! relay died
relay restarted -> broker=[OrderCreated:o-1, OrderCreated:o-1]
```

The dual write ends with the order in the database and nothing on the broker: a lost event. With the outbox, the order and the pending event survive the crash; after the relay restarts the event is published, and because the relay died before marking the row, it is published twice. The consumer must therefore deduplicate on the event identifier.

The Spring implementation uses a JPA entity, a repository with a locking query for multi-instance relays, and a scheduled relay.

OutboxEvent.java JAVA

```
package com.example.orders.outbox;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

import java.time.Instant;
import java.util.UUID;

@Entity
@Table(name = "outbox_event")
public class OutboxEvent {

    @Id
    private UUID id;

    @Column(nullable = false)
    private String aggregateId;          // used as the Kafka key

    @Column(nullable = false)
    private String topic;
```

OutboxEvent.java (continued) JAVA

```java
    @Column(nullable = false, columnDefinition = "text")
    private String payload;              // JSON

    @Column(nullable = false)
    private Instant createdAt;

    private Instant sentAt;

    protected OutboxEvent() { }

    public OutboxEvent(UUID id, String aggregateId, String topic, String payload) {
        this.id = id;
        this.aggregateId = aggregateId;
        this.topic = topic;
        this.payload = payload;
        this.createdAt = Instant.now();
    }

    public UUID getId() { return id; }
    public String getAggregateId() { return aggregateId; }
    public String getTopic() { return topic; }
    public String getPayload() { return payload; }
    public void markSent() { this.sentAt = Instant.now(); }
}
```

OutboxRepository.java JAVA

```java
package com.example.orders.outbox;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;
import java.util.UUID;

public interface OutboxRepository extends JpaRepository<OutboxEvent, UUID> {

    @Query(value = """
            SELECT * FROM outbox_event
            WHERE sent_at IS NULL
            ORDER BY created_at
            LIMIT :batch
            FOR UPDATE SKIP LOCKED
            """, nativeQuery = true)
    List<OutboxEvent> lockNextBatch(int batch);
}
```

`FOR UPDATE SKIP LOCKED` lets several relay instances poll the same table without blocking each other and without publishing the same row concurrently; each row is locked by exactly one transaction. Named parameters in native queries require `-parameters` compilation, which the Spring Boot Maven plugin parent enables.

OutboxRelay.java JAVA

```java
package com.example.orders.outbox;

import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.util.concurrent.TimeUnit;

@Component
public class OutboxRelay {

    private final OutboxRepository repository;
    private final KafkaTemplate<String, String> template;

    public OutboxRelay(OutboxRepository repository, KafkaTemplate<String, String> template) {
        this.repository = repository;
        this.template = template;
    }
```

OutboxRelay.java (continued) JAVA

```
    @Scheduled(fixedDelay = 500)
    @Transactional
    public void publishPending() throws Exception {
        for (OutboxEvent e : repository.lockNextBatch(100)) {
            template.send(e.getTopic(), e.getAggregateId(), e.getPayload())
                    .get(10, TimeUnit.SECONDS);      // wait for the broker ack
            e.markSent();                            // flushed at commit
        }
    }
}
```

What to notice: the relay waits for the broker acknowledgement before marking the row, so success of `get()` implies durability on the broker; a failure rolls back the transaction and the rows stay pending; and the `String` template means the payload is already JSON, so the relay needs no knowledge of event classes. Production relays often replace polling with change data capture (Debezium reading the write-ahead log), which removes the polling delay and the load on the table.

The producer in the service layer writes the business row and the outbox row inside one `@Transactional` method:

OrderService.java JAVA

```
package com.example.orders.service;

import com.example.orders.domain.Order;
import com.example.orders.domain.OrderRepository;
import com.example.orders.events.OrderCreated;
import com.example.orders.outbox.OutboxEvent;
import com.example.orders.outbox.OutboxRepository;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.UUID;

@Service
public class OrderService {

    private final OrderRepository orders;
    private final OutboxRepository outbox;
    private final ObjectMapper mapper;

    public OrderService(OrderRepository orders, OutboxRepository outbox, ObjectMapper mapper) {
        this.orders = orders;
        this.outbox = outbox;
        this.mapper = mapper;
    }

    @Transactional
    public Order place(Order order) throws JsonProcessingException {
        Order saved = orders.save(order);
        var event = new OrderCreated(UUID.randomUUID(), saved.getId(), saved.getCustomerId(),
                saved.getTotal(), java.time.Instant.now());
        outbox.save(new OutboxEvent(event.eventId(), saved.getId().toString(),
                "orders.events", mapper.writeValueAsString(event)));
        return saved;
    }
}
```

The idempotent consumer

A consumer is idempotent when processing the same event twice has the same effect as processing it once. The standard implementation stores processed event identifiers in a table with a unique constraint, inside the same database transaction as the business effect.

PaymentService.java JAVA

```
package com.example.payments.service;

import com.example.payments.events.OrderCreated;
import com.example.payments.domain.Payment;
import com.example.payments.domain.PaymentRepository;
import com.example.payments.domain.ProcessedEvent;
```

sreeram

PaymentService.java (continued) JAVA

```java
import com.example.payments.domain.ProcessedEventRepository;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class PaymentService {

    private final PaymentRepository payments;
    private final ProcessedEventRepository processed;

    public PaymentService(PaymentRepository payments, ProcessedEventRepository processed) {
        this.payments = payments;
        this.processed = processed;
    }

    @Transactional
    public void authorise(OrderCreated event) {
        try {
            processed.saveAndFlush(new ProcessedEvent(event.eventId()));   // unique id
        } catch (DataIntegrityViolationException duplicate) {
            return;                                                         // already handled
        }
        payments.save(Payment.authorised(event.orderId(), event.total()));
    }
}
```

WATCH OUT

The catch block above is only safe if the exception does not poison the transaction. With some JPA providers and databases, a unique violation marks the transaction rollback-only or aborts the PostgreSQL transaction (SQLSTATE 25P02). The robust variant uses `INSERT ... ON CONFLICT DO NOTHING` through a native query and inspects the affected row count; zero affected rows means duplicate.

6.1.9 Schema evolution and the event contract

An event in a topic outlives the code that produced it, and consumers deploy independently. The rules for compatible evolution are the same as for REST payloads, with the difference that old events remain in the log.

- Adding an optional field is compatible in both directions when consumers ignore unknown properties (`spring.jackson.deserialization.fail-on-unknown-properties=false`, the Boot default).
- Removing or renaming a field, or changing its type, is breaking. The two-step procedure is: add the new field and publish both; migrate all consumers; stop publishing the old field.
- A schema registry (Confluent Schema Registry, Apicurio) with Avro, Protobuf or JSON Schema enforces a compatibility mode (BACKWARD, FORWARD, FULL) at publish time and stores the schema id inside the record, which removes the need for type headers.
- Include a `type` and a `version` in the payload or headers so that one topic can carry several event kinds and a consumer can route on them.
- Never reuse a topic for a different meaning; create `orders.events.v2` and run both until consumers have migrated.

6.1.10 Real-world problems and solutions

Problem 1: consumer lag keeps growing

Scenario. The `payment-service` consumes `orders.events` with three instances. After a marketing campaign the lag on partitions 2 and 4 grows steadily while partitions 0, 1, 3 and 5 stay at zero. **Symptoms.** Alerts on `records-lag-max`; payments delayed by minutes; no errors in the logs. **Diagnosis.** Run `kafka-consumer-groups.sh --describe --group payment-service` and read `LAG` per partition. Skewed lag on two partitions points to hot keys or a slow instance, not to total capacity. Take a thread dump of the instance that owns those partitions with `jcmd <pid> Thread.print`; the consumer threads (`org.springframework.kafka.KafkaListenerEndpointContainer#0-2-C-1`) show the blocking call. Check the Actuator metric `kafka.consumer.fetch.manager.records.lag.max` and the listener timer `spring.kafka.listener`. **Root cause.** The listener performed a synchronous HTTP call to the payment gateway without a timeout; two keys with large order volumes (a corporate customer) hash to partitions 2 and 4, and each call took four seconds under load. **Fix.** Set connect and read timeouts on the client, move the call behind a circuit breaker with a bulkhead, raise `concurrency` to the partition count, and if one key is genuinely hot, key by `orderId` instead of `customerId`. **Prevention.** Alert on per-partition lag growth rate rather than a fixed threshold, load-test with a skewed key distribution, and require timeouts on every outbound call in code review.

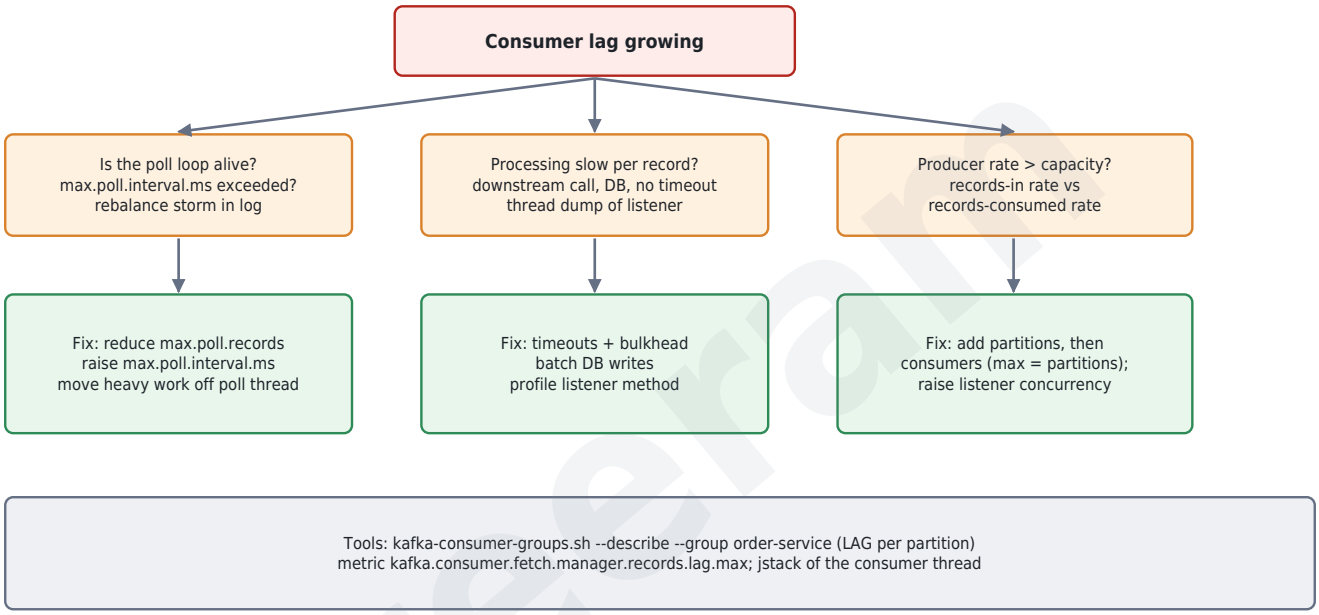


Figure 6.6. Lag diagnosis starts at the poll loop, then the per-record processing time, then the production rate; each branch has a distinct fix.

Problem 2: a poison message blocks one partition

Scenario. A producer bug publishes one record with a null `total`. The consumer throws `NullPointerException` on it. **Symptoms.** Lag on a single partition grows without bound; the same stack trace repeats in the log every few seconds; other partitions are healthy. **Diagnosis.** Grep the log for the repeated offset (`partition=3 offset=88123`); inspect the record with `kafka-console-consumer.sh --topic orders.events --partition 3 --offset 88123 --max-messages 1`. **Root cause.** A custom error handler with `FixedBackOff.UNLIMITED_ATTEMPTS` retried the record forever, and no dead-letter topic existed. **Fix.** Use `DefaultErrorHandler` with a bounded `BackOff` and a `DeadLetterPublishingRecoverer`; register deterministic exceptions as not retryable; validate the payload on the producer side. **Prevention.** Alert on dead-letter topic traffic, test a malformed record in an integration test with Testcontainers, and keep unbounded retries out of listeners.

Problem 3: duplicate processing after a rebalance

Scenario. During a rolling update the payment service charged some customers twice. **Symptoms.** Two `Payment` rows for one `orderId`; the logs show the same offset processed by two different pods within a minute. **Diagnosis.** Compare `partition` and `offset` in both log lines; check the logs for `Revoking previously assigned partitions` and `Rebalance` entries between them; look at `max.poll.interval.ms` versus the processing time of one batch. **Root cause.** The listener was at-least-once by design, but the write was not idempotent. A pod was terminated after charging and before the offset commit, and the new owner replayed the record. **Fix.** The idempotent consumer shown earlier: a unique `event_id` constraint written in the same transaction, plus a unique index on `payment(order_id)`. **Prevention.** Treat duplicates as normal. Use `RECORD` ack mode and graceful shutdown to shrink the window, but rely on idempotence for correctness.

Problem 4: message loss with auto-commit

Scenario. After an out-of-memory crash of a consumer, about 300 events never appear in the downstream database, although the consumer group shows zero lag. **Symptoms.** Missing rows with no error trace; offsets are ahead of the data. **Diagnosis.** Compare the committed offset of the group with the highest offset successfully written to the database. Inspect the effective configuration with `--debug` logging of `ConsumerConfig values` at startup: `enable.auto.commit = true`. **Root cause.** The property `spring.kafka.consumer.enable-auto-commit=true` had been copied from a tutorial; the Kafka client committed the offsets of the polled batch every five seconds while the listener handed the records to an in-memory executor that had not yet finished. **Fix.** Set `enable-auto-commit: false`, process on the consumer thread or acknowledge manually after the asynchronous work completes, and size `max.poll.records` to the processing capacity. **Prevention.** Keep the Kafka consumer defaults of the container, and review any configuration that hands records to another thread.

6.1.11 Questions to expect

1. **What does a Kafka consumer group guarantee about partition ownership?** Within one group each partition is assigned to exactly one member at a time. Different groups are independent and each reads all records.
2. **How is ordering guaranteed?** Only within a partition. A key maps to one partition by hashing, so records with the same key are consumed in the order they were written, provided the partition count does not change.
3. **Why can the same message be delivered twice?** The offset commit is separate from the processing. A crash, rebalance or timeout between them replays the uncommitted records. Idempotent consumers absorb this.
4. **What does `acks=all` do, and what is `min.insync.replicas` for?** The leader acknowledges after all in-sync replicas have the record. `min.insync.replicas` sets the least number of in-sync replicas that must exist for the write to be accepted; with replication factor 3 and a minimum of 2, one broker may fail without data loss or unavailability of writes.
5. **What is the difference between blocking retries and retry topics?** Blocking retries re-run the record on the consumer thread and keep order but stall the partition. Retry topics publish the record to delayed topics and continue; order is lost.
6. **What is a dead-letter topic?** A topic that receives records that exhausted their retries or are not retryable, with the failure reason in headers, so that the main flow continues and the records can be analysed and replayed.
7. **Explain exactly-once in Kafka.** Idempotent producers plus transactions give atomic read-process-write between Kafka topics. External side effects are outside the guarantee and need idempotence or the outbox pattern.
8. **What is the outbox pattern and why is it needed?** It writes the event into a table in the same database transaction as the state change and publishes it asynchronously, which avoids the lost update or phantom event of a dual write. Delivery is at-least-once.
9. **What triggers a consumer group rebalance?** Members joining or leaving, missed heartbeats, exceeding `max.poll.interval.ms`, and partition count changes. Cooperative assignors and static membership reduce the disruption.
10. **How is a poison message handled?** Classify the exception as not retryable or bound the retries, send the record to the dead-letter topic, alert, fix the cause and replay.

KEY POINTS

Kafka guarantees order only within a partition, so the record key is a design decision tied to the aggregate that needs ordering.

- Offsets commit after processing gives at-least-once; every consumer must be idempotent, typically with a unique event id in the same transaction as the effect.
- `DefaultErrorHandler` with a bounded back-off and a `DeadLetterPublishingRecoverer` prevents poison messages from blocking a partition; `@RetryableTopic` trades ordering for availability.
- Exactly-once holds only inside Kafka; database plus Kafka consistency is achieved with the transactional outbox.
- Lag per partition, rebalance frequency and dead-letter traffic are the three operational signals to monitor.

6.2 RabbitMQ, Scheduling, Async Work and Events

Kafka is one answer to asynchronous work and not always the most appropriate. Task distribution with flexible routing, per-message acknowledgement and delayed redelivery is the strength of an AMQP broker such as RabbitMQ. Work that stays inside one JVM needs no broker at all: `@Async`, `@Scheduled` and Spring application events cover it. This chapter explains each mechanism, the thread and failure model behind it, and the criteria for choosing between them.

6.2.1 Concept and mechanism

RabbitMQ and Spring AMQP

In AMQP 0-9-1 a producer publishes to an **exchange**, never directly to a queue. The exchange routes the message to zero or more **queues** according to **bindings**; consumers read from queues. Exchange types: `direct` (exact routing key match), `topic` (pattern match with `*` for one word and `#` for zero or more words), `fanout` (all bound queues) and `headers`. A message is removed from a queue when the consumer acknowledges it. A message that is rejected without requeue, expires (TTL) or overflows a queue length limit is republished to the queue's **dead-letter exchange** (DLX).

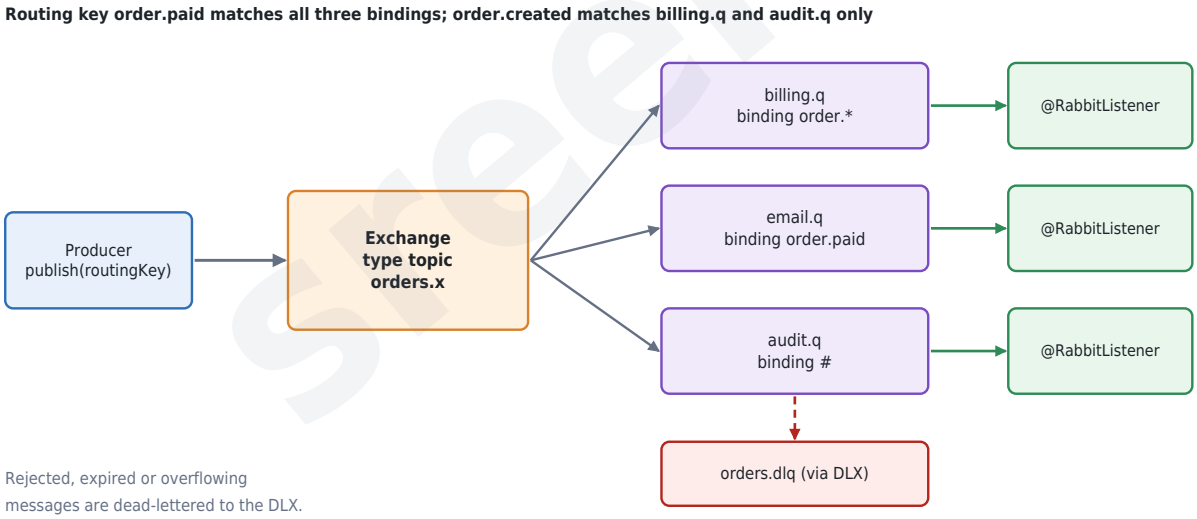


Figure 6.7. A topic exchange routes by pattern: the key order.paid reaches all three queues, the key order.created only two; rejected messages leave through the dead-letter exchange.

Spring AMQP provides `RabbitTemplate` for publishing, `@RabbitListener` for consuming (executed by a `SimpleMessageListenerContainer` or `DirectMessageListenerContainer`), and `RabbitAdmin`, which declares exchanges, queues and bindings from `@Bean` definitions at connection time. Boot auto-configures `CachingConnectionFactory`, `RabbitTemplate`, `RabbitAdmin` and the listener container factory from `spring.rabbitmq.*`.

The default acknowledge mode of the Spring listener container is `AUTO`: the container acknowledges when the listener returns normally and rejects when it throws. The default for a rejected message is to **requeue** it (`spring.rabbitmq.listener.simple.default-requeue-rejected=true`), which for a deterministic failure produces an endless hot loop. Setting it to `false`, or throwing `AmqpRejectAndDontRequeueException`, sends the message to the DLX.

Scheduling

`@Scheduled` methods run on a `TaskScheduler`. Boot auto-configures a `ThreadPoolTaskScheduler` with a pool size of one (`spring.task.scheduling.pool.size=1`), so two long-running scheduled methods delay each other. `fixedRate` starts runs at fixed intervals measured start to start (overlapping runs are not started by the single-threaded scheduler; the next run starts late), `fixedDelay` waits the given time after the previous run finished, and `cron` follows a calendar expression with six fields in Spring (seconds first). Scheduling runs in every instance of the application; with three replicas, a nightly job executes three times unless coordinated. The options are a distributed lock (ShedLock, which uses a row in a shared table or a Redis key), a leader election, or a clustered scheduler such as Quartz with a JDBC job store, in which triggers are persisted and claimed by exactly one node.

Asynchronous execution with @Async

`@Async` is implemented by `AsyncAnnotationBeanPostProcessor`, which wraps the bean in an AOP proxy whose `AsyncExecutionInterceptor` submits the invocation to a `TaskExecutor` and returns immediately. It is therefore subject to the same limits as every proxy-based feature: self-invocation and `private` or `final` methods bypass it. Boot auto-configures `applicationTaskExecutor` (a `ThreadPoolTaskExecutor` with core size 8, an unbounded queue and an effectively unbounded maximum size, which means only eight threads ever run because the queue never fills). With `spring.threads.virtual.enabled=true` (Spring Boot 3.2 and later, JDK 21) the executor is a `SimpleAsyncTaskExecutor` that starts a virtual thread per task. Neither the `SecurityContext` nor the SLF4J MDC is propagated to the executor thread by default; a `TaskDecorator` copies them.

Application events

`ApplicationEventPublisher.publishEvent()` calls every matching `@EventListener` synchronously on the publishing thread, by default inside the publisher's transaction. Methods annotated `@TransactionalEventListener` (phase `AFTER_COMMIT` by default) are deferred by `TransactionSynchronization` until the transaction completes; they are skipped if the transaction rolls back, and if no transaction is active the event is dropped unless `fallbackExecution = true`.

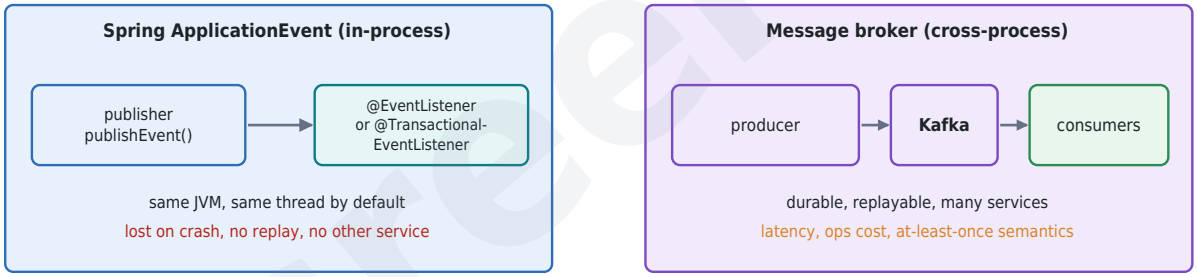


Figure 6.8. In-process events share the JVM and its failure; a broker adds durability, replay and cross-service delivery at the cost of latency and operations.

Criterion	Spring events / @Async	Broker (RabbitMQ, Kafka)
Survives a crash	no	yes
Consumers in other services	no	yes
Replay of history	no	Kafka only
Per-message routing and priorities	no	RabbitMQ
Operational cost	none	broker cluster, monitoring
Typical use	cache eviction, audit entry after commit, local decoupling	integration events between services

Choosing between in-process events, a queue and a log.

6.2.2 Worked example

The `billing` and `email` queues of the example diagram, declared as beans, with a dead-letter exchange and JSON conversion.

RabbitConfig.java JAVA

```java
package com.example.orders.amqp;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitConfig {

    public static final String EXCHANGE = "orders.x";
    public static final String DLX = "orders.dlx";

    @Bean
    TopicExchange ordersExchange() {
        return new TopicExchange(EXCHANGE);
    }

    @Bean
    DirectExchange deadLetterExchange() {
        return new DirectExchange(DLX);
    }

    @Bean
    Queue billingQueue() {
        return QueueBuilder.durable("billing.q")
                .deadLetterExchange(DLX)
                .deadLetterRoutingKey("billing.dead")
                .build();
    }

    @Bean
    Queue billingDeadLetterQueue() {
        return QueueBuilder.durable("billing.dlq").build();
    }

    @Bean
    Binding billingBinding(Queue billingQueue, TopicExchange ordersExchange) {
        return BindingBuilder.bind(billingQueue).to(ordersExchange).with("order.*");
    }

    @Bean
    Binding billingDeadBinding(Queue billingDeadLetterQueue, DirectExchange deadLetterExchange) {
        return BindingBuilder.bind(billingDeadLetterQueue).to(deadLetterExchange)
                .with("billing.dead");
    }

    @Bean
    MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}
```

Boot detects a `MessageConverter` bean and applies it to the `RabbitTemplate` and to the listener container factory. Without it, the template serialises payloads with Java serialisation, which is both insecure and unreadable for other consumers.

BillingListener.java JAVA

```java
package com.example.orders.amqp;

import com.example.orders.events.OrderCreated;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.AmqpRejectAndDontRequeueException;
import org.springframework.amqp.core.AmqpTemplate;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

@Component
public class BillingListener {

    private static final Logger log = LoggerFactory.getLogger(BillingListener.class);

    @RabbitListener(queues = "billing.q")
    public void onOrder(OrderCreated event) {
        if (event.total() == null) {
            throw new AmqpRejectAndDontRequeueException("order without total " + event.orderId());
        }
        log.info("invoice for order {}", event.orderId());
    }
}

@Component
class OrderMessagePublisher {

    private final AmqpTemplate template;

    OrderMessagePublisher(AmqpTemplate template) {
        this.template = template;
    }

    void orderCreated(OrderCreated event) {
        template.convertAndSend(RabbitConfig.EXCHANGE, "order.created", event);
    }
}
```

application.yml (fragment) YAML

```yaml
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: ${RABBIT_USER}
    password: ${RABBIT_PASSWORD}
    publisher-confirm-type: correlated
    publisher-returns: true
    listener:
      simple:
        acknowledge-mode: auto
        default-requeue-rejected: false
        prefetch: 20
        concurrency: 2
        max-concurrency: 8
        retry:
          enabled: true
          max-attempts: 3
          initial-interval: 1s
          multiplier: 2
```

`publisher-confirm-type: correlated` makes the broker confirm each publish; without confirms, a publish can be lost silently when the broker crashes. `publisher-returns` reports messages that matched no queue (mandatory flag). The `retry` block configures stateless in-memory retries inside the container; after the attempts are exhausted the message is rejected without requeue and reaches the DLX.

Scheduling and asynchronous work with explicit executors:

sreeram

AsyncConfig.java JAVA

```java
package com.example.orders.config;

import org.slf4j.MDC;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.task.TaskDecorator;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.Map;
import java.util.concurrent.ThreadPoolExecutor;

@Configuration
@EnableAsync
@EnableScheduling
public class AsyncConfig {

    @Bean
    TaskDecorator mdcTaskDecorator() {
        return runnable -> {
            Map<String, String> context = MDC.getCopyOfContextMap();
            return () -> {
                try {
                    if (context != null) MDC.setContextMap(context);
                    runnable.run();
                } finally {
                    MDC.clear();
                }
            };
        };
    }

    @Bean("mailExecutor")
    ThreadPoolTaskExecutor mailExecutor(TaskDecorator decorator) {
        var executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(4);
        executor.setMaxPoolSize(8);
        executor.setQueueCapacity(200);                    // bounded: back-pressure
        executor.setThreadNamePrefix("mail-");
        executor.setTaskDecorator(decorator);
        executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
        executor.setWaitForTasksToCompleteOnShutdown(true);
        executor.setAwaitTerminationSeconds(20);
        return executor;
    }
}
```

MailService.java JAVA

```java
package com.example.orders.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Async;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;

import java.util.concurrent.CompletableFuture;

@Service
public class MailService {

    private static final Logger log = LoggerFactory.getLogger(MailService.class);

    @Async("mailExecutor")
    public CompletableFuture<Void> sendConfirmation(String address) {
        log.info("sending mail to {}", address);       // runs on a mail-N thread
        return CompletableFuture.completedFuture(null);
    }

    @Scheduled(cron = "0 0 2 * * *", zone = "UTC")      // every day at 02:00 UTC
```

sreeram

MailService.java (continued) JAVA

```
    public void purgeExpiredTokens() {
        log.info("purge job started");
    }
}
```

The executor is bounded and uses `CallerRunsPolicy`: when eight threads are busy and 200 tasks queue up, the submitting thread executes the task itself, which slows the producer down instead of growing memory. An exception thrown from a `void @Async` method is not visible to the caller; it goes to the `AsyncUncaughtExceptionHandler` (a logging handler by default), so a method that returns `CompletableFuture` is preferable.

A transactional event listener that keeps a cache consistent without coupling the service to the cache:

ProductEvents.java JAVA

```
package com.example.catalog.events;

import org.springframework.cache.CacheManager;
import org.springframework.stereotype.Component;
import org.springframework.transaction.event.TransactionPhase;
import org.springframework.transaction.event.TransactionalEventListener;

public record ProductChanged(long productId) { }

@Component
class ProductCacheEvictor {

    private final CacheManager cacheManager;

    ProductCacheEvictor(CacheManager cacheManager) {
        this.cacheManager = cacheManager;
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void evict(ProductChanged event) {
        cacheManager.getCache("products").evict(event.productId());
    }
}
```

The service calls `publisher.publishEvent(new ProductChanged(id))` inside its `@Transactional` method. The eviction runs only after a successful commit, which removes the window in which a concurrent reader repopulates the cache from the not yet committed old row.

6.2.3 Hands-on

Run RabbitMQ locally and observe routing:

typical output (not run on the build machine) TERMINAL

```
$ docker run -d --name rabbit -p 5672:5672 -p 15672:15672 rabbitmq:3-management
$ ./mvnw spring-boot:run
$ curl -X POST localhost:8080/orders -H 'Content-Type: application/json' \
    -d '{"customerId":"c-1","total":12.50}'
$ docker exec rabbit rabbitmqctl list_queues name messages consumers
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name        messages  consumers
billing.q   0         2
billing.dlq 0         0
```

A malformed message published from the management UI (`total` missing) moves to `billing.dlq` after the three configured attempts, and `rabbitmqctl list_queues` then shows `billing.dlq 1 0`. The headers `x-death` and `x-first-death-reason` on that message name the queue and the reason (`rejected`).

6.2.4 Real-world problems and solutions

Problem 1: a scheduled job runs three times

Scenario. The reporting job `@Scheduled(cron = "0 0 2 * * *")` sends a daily summary mail. After scaling to three replicas, every recipient gets three mails. **Symptoms.** Identical log lines at 02:00:00 from three pods; duplicated side effects. **Diagnosis.** `kubectl logs` of each pod filtered by the job's logger shows concurrent execution; the scheduler is local to each JVM. **Root cause.** `@Scheduled` has no cluster awareness. **Fix.** Guard the method with ShedLock (`@SchedulerLock(name = "dailySummary", lockAtMostFor = "10m", lockAtLeastFor = "1m")` with a JDBC or Redis lock provider), or move the job to a Kubernetes `CronJob` that runs one pod, or use Quartz with `spring.quartz.job-store-type=jdbc` and `org.quartz.jobStore.isClustered=true`. **Prevention.** Treat every scheduled method as a distributed-systems concern; also make the job idempotent (a `summary_sent` flag per day) so that an overlapping run is harmless.

Problem 2: @Async silently runs synchronously

Scenario. `OrderService.place()` calls `this.sendConfirmation(...)`, annotated `@Async`, and the HTTP response still takes two seconds. **Symptoms.** The log line of the mail method shows the `http-nio-8080-exec-3` thread instead of `mail-1`. **Diagnosis.** Log `Thread.currentThread().getName()` in the async method; inspect the bean class with `AopUtils.isAopProxy(bean)`. **Root cause.** Self-invocation: the call goes through `this`, not through the proxy, so `AsyncExecutionInterceptor` never runs. Also verify that `@EnableAsync` is present; without it the annotation is ignored. **Fix.** Move the method into a separate bean and inject it, or inject the proxy of the class itself with `@Lazy`. **Prevention.** Keep proxy-based annotations at the boundary between beans; the chapter on AOP and proxies in the first part of this guide describes the mechanism.

Problem 3: RabbitMQ message redelivered in an endless loop

Scenario. A listener throws `IllegalStateException` for a particular message; the CPU of the consumer is at 100 per cent and the broker shows the message with `redelivered: true` thousands of times per minute. **Symptoms.** Log flooding with the same stack trace; queue depth does not drop. **Diagnosis.** In the management UI, the queue shows an `unacked` count of 1 and a high deliver rate with a zero ack rate. **Root cause.** The default `default-requeue-rejected=true` requeues a failed message at the head of the queue. **Fix.** Set `spring.rabbitmq.listener.simple.default-requeue-rejected=false`, enable bounded retry, and configure a DLX on the queue. **Prevention.** Declare a dead-letter exchange for every queue as part of the topology and alert on the dead-letter queue depth.

6.2.5 Questions to expect

1. **When RabbitMQ and when Kafka?** RabbitMQ for task queues, routing flexibility, priorities and per-message acknowledgement with moderate volume; Kafka for high-throughput event streams, replay and multiple independent consumer groups.
2. **What are the AMQP exchange types?** `direct`, `topic`, `fanout` and `headers`. Producers publish to exchanges; bindings connect exchanges to queues.
3. **What does `@Async` need in order to work?** `@EnableAsync`, a call through the Spring proxy (not `this`), a non-final public method, and ideally a configured bounded executor.
4. **Why does `@Scheduled` run more than once in a cluster?** Each instance has its own scheduler. A distributed lock, a leader election or a clustered Quartz job store is required for single execution.
5. **What is the difference between `@EventListener` and `@TransactionalEventListener`?** The first runs synchronously at publication, inside the transaction; the second runs in a transaction phase, by default after commit, and is skipped on rollback.
6. **Why are unbounded executor queues dangerous?** The pool never grows beyond the core size and a backlog accumulates in memory until an `OutOfMemoryError`. A bounded queue with a rejection policy applies back-pressure.
7. **How is the MDC propagated into `@Async` threads?** With a `TaskDecorator` that copies `MDC.getCopyOfContextMap()` to the worker thread; the `SecurityContext` needs `DelegatingSecurityContextAsyncTaskExecutor` or the equivalent.

KEY POINTS

RabbitMQ routes through exchanges and bindings; a dead-letter exchange and `default-requeue-rejected=false` prevent endless redelivery.

- `@Scheduled` and `@Async` are local to one JVM; clustered execution needs ShedLock, Quartz JDBC or a Kubernetes CronJob.
- `@Async`, like every proxy-based annotation, is bypassed by self-invocation; executors must be bounded and decorated to carry the MDC.
- Application events are in-process and non-durable; `@TransactionalEventListener(AFTER_COMMIT)` ties side effects to a successful commit.

6.3 Caching with Redis in Spring

A cache trades consistency for latency and load. A product page that is read a thousand times for each update does not need a database query per read; a cache in front of the database returns the previous answer in microseconds and removes load from the connection pool. The costs are staleness, a second system to operate, and a set of failure modes (stampede, penetration, inconsistency) that only appear under load. The Redis data structures and persistence model are covered in a separate Redis guide; this chapter covers the Spring Cache abstraction, its Redis integration and the failure modes.

6.3.1 Concept and mechanism

The Spring Cache abstraction

`@EnableCaching` (applied automatically by Boot when a cache starter is present) registers a `BeanFactoryCacheOperationSourceAdvisor`. The advisor wraps beans that have caching annotations in a proxy whose `CacheInterceptor` reads the annotation metadata through `AnnotationCacheOperationSource`, computes a key with a `KeyGenerator` or the SpEL `key` expression, and calls the `Cache` obtained from the `CacheManager`. The business method is a black box: on a hit it is not executed. As with `@Transactional`, the interceptor sits in the proxy, so self-invocation and private methods bypass the cache.

Annotation	Behaviour
`@Cacheable`	return the cached value if the key exists; otherwise invoke the method and store the result
`@CachePut`	always invoke the method and store the result (refresh without reading)
`@CacheEvict`	remove one key, or all keys with allEntries = true; beforeInvocation = true evicts even if the method throws
`@Caching`	combine several operations on one method
`@CacheConfig`	class-level defaults such as the cache names

Cache annotations.

The default key generator (`SimpleKeyGenerator`) builds the key from all method parameters: no parameter gives `SimpleKey.EMPTY`, one parameter uses the value itself. Explicit keys with SpEL (`key = "#id"`, `key = "#root.methodName + #a0"`) are clearer and avoid collisions between methods that share a cache name.

Cache-aside with Redis

With `spring.cache.type=redis` Boot creates a `RedisCacheManager` over the `RedisConnectionFactory`. Each cache name is a key prefix in Redis; an entry for cache `products` and key `42` is stored under the Redis key `products::42`. Values are serialised with the configured `RedisSerializer` (JDK serialisation by default, which requires `Serializable` and is unreadable in `redis-cli`; JSON is preferred). Every entry should have a TTL; without one, entries live until evicted by memory policy.

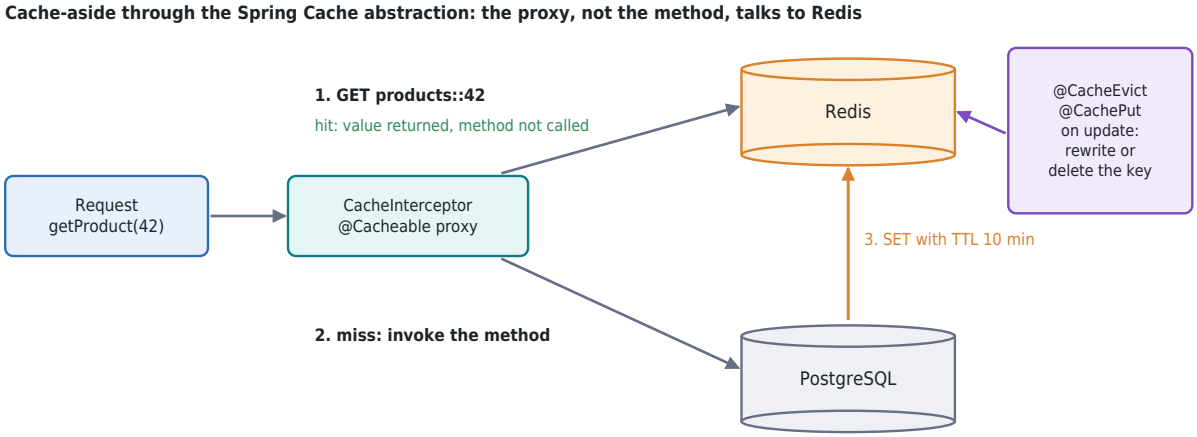


Figure 6.9. The caching proxy first asks Redis; only a miss reaches the method and the database, and the result is written back with a time to live.

6.3.2 Worked example

pom.xml (fragment) XML

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

application.yml (fragment) YAML

```yaml
spring:
  data:
    redis:
      host: localhost
      port: 6379
      timeout: 500ms
      lettuce:
        pool:
          enabled: true
          max-active: 16
  cache:
    type: redis
    redis:
      time-to-live: 10m
      cache-null-values: false
      key-prefix: "catalog:"
      use-key-prefix: true
```

The prefix `spring.data.redis.*` applies to Spring Boot 3.0 and later (it was `spring.redis.*` in 2.x). The timeout of 500 ms bounds how long a request waits for Redis; an unreachable cache must not become an outage of the service.

CacheConfig.java JAVA

```java
package com.example.catalog.config;

import org.springframework.boot.autoconfigure.cache.RedisCacheManagerBuilderCustomizer;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializationContext.SerializationPair;

import java.time.Duration;

@Configuration
@EnableCaching
```

sreeram

CacheConfig.java (continued) JAVA

```java
public class CacheConfig {

    @Bean
    RedisCacheConfiguration defaultCacheConfiguration() {
        return RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10))
                .disableCachingNullValues()
                .serializeValuesWith(SerializationPair.fromSerializer(
                        new GenericJackson2JsonRedisSerializer()));
    }

    @Bean
    RedisCacheManagerBuilderCustomizer perCacheTtl(RedisCacheConfiguration defaults) {
        return builder -> builder
                .withCacheConfiguration("products", defaults.entryTtl(Duration.ofMinutes(30)))
                .withCacheConfiguration("categories", defaults.entryTtl(Duration.ofHours(6)));
    }
}
```

ProductService.java JAVA

```java
package com.example.catalog.service;

import com.example.catalog.domain.Product;
import com.example.catalog.domain.ProductRepository;
import com.example.catalog.events.ProductChanged;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class ProductService {

    private final ProductRepository repository;
    private final ApplicationEventPublisher events;

    public ProductService(ProductRepository repository, ApplicationEventPublisher events) {
        this.repository = repository;
        this.events = events;
    }

    @Cacheable(cacheNames = "products", key = "#id", unless = "#result == null", sync = true)
    @Transactional(readOnly = true)
    public Product get(long id) {
        return repository.findById(id).orElse(null);
    }

    @Transactional
    public Product update(long id, String name, java.math.BigDecimal price) {
        Product p = repository.findById(id).orElseThrow();
        p.setName(name);
        p.setPrice(price);
        events.publishEvent(new ProductChanged(id));       // evicts after commit
        return p;
    }
}
```

What to notice: `sync = true` makes concurrent callers for the same key in one JVM wait for a single load instead of all executing the method (`RedisCache` serialises the loader per instance; it does not coordinate between application instances); the return type `Product` must be serialisable to JSON and deserialisable without type information problems, which is why entities should not be cached directly (use a DTO or record); and the eviction is event-driven and happens after commit, so a concurrent reader cannot re-cache the pre-update row.

A record is the safest cached value:

ProductView.java JAVA

```java
package com.example.catalog.api;

import java.math.BigDecimal;

public record ProductView(long id, String name, BigDecimal price, int version) { }
```

sieeram

6.3.3 Key design, TTL and consistency

Keys. A key should identify the value and nothing else: include the entity identifier and, where the value depends on them, the tenant, the locale or a representation version (`products::v2::42`). A change of the cached class layout requires a new prefix or a cache flush, otherwise deserialisation of old entries fails. Parameters that are objects need a stable `toString`/`hashCode` or an explicit SpEL key.

TTL. The TTL is the upper bound on staleness, and it protects against missed evictions. Entries that expire at the same moment cause synchronised misses; adding random jitter to TTLs (for example 30 minutes plus or minus 10 per cent) spreads the reload load.

Consistency with the database. Cache-aside gives eventual consistency. Two races matter:

- A reader loads the old row, a writer updates the row and evicts, then the reader stores its old value. The cached value is now stale until the TTL expires. Evicting after commit and using short TTLs narrows the window; versioned values (`version` field compared on write, or Redis `SET` with a check script) close it.
- Evicting before the commit lets a concurrent reader re-populate the cache from the old row. For this reason the eviction in the example above is bound to `AFTER_COMMIT`.

Failure of Redis. By default a Redis exception propagates from the `CacheInterceptor` and fails the request. A `CacheErrorHandler` bean (via `CachingConfigurer`) can log and continue, which turns a cache outage into a slower service instead of an unavailable one.

6.3.4 Stampede, penetration and avalanche

A **stampede** (thundering herd, dog-pile) occurs when a hot key expires and many concurrent requests miss at once; every one of them executes the loader against the database. **Penetration** is repeated lookups for keys that do not exist (nothing is ever cached, each request reaches the database). An **avalanche** is the simultaneous expiry of many keys or the loss of the whole cache. The mitigation for a stampede is single flight: one request loads while the others wait or serve the stale value.

The following plain-JDK program fires 200 virtual-thread clients at a cold key with a 200 ms loader, first without protection and then with a lock and a second cache check (double-checked loading), which is what `@Cacheable(sync = true)` does within one JVM.

Stampede.java — JAVA

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class Stampede {
    static final AtomicInteger dbCalls = new AtomicInteger();

    static String loadFromDb(String key) {
        dbCalls.incrementAndGet();
        try { Thread.sleep(200); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        return "value-of-" + key;
    }

    static long run(String title, boolean singleFlight) throws Exception {
        dbCalls.set(0);
        ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();
        ConcurrentHashMap<String, Object> locks = new ConcurrentHashMap<>();
        int clients = 200;
        CountDownLatch go = new CountDownLatch(1);
        long t0 = System.nanoTime();
        try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < clients; i++) {
                pool.submit(() -> {
                    go.await();
                    String v = cache.get("product:42");
                    if (v == null) {
                        if (singleFlight) {
                            Object lock = locks.computeIfAbsent("product:42", k -> new Object());
                            synchronized (lock) {
                                v = cache.get("product:42");
                                if (v == null) {
                                    v = loadFromDb("product:42");
                                    cache.put("product:42", v);
                                }
```

Stampede.java (continued) JAVA

```
                        }
                    } else {
                        v = loadFromDb("product:42");
                        cache.put("product:42", v);
                    }
                }
                return v;
            });
        }
        go.countDown();
    }
    long ms = (System.nanoTime() - t0) / 1_000_000;
    System.out.printf("%-34s clients=%d  db calls=%d%n",
        title, clients, dbCalls.get());
    return ms;
  }

  public static void main(String[] a) throws Exception {
    run("no protection (cold key)", false);
    run("single flight (lock + re-check)", true);
  }
}
```

Output of the stampede simulation (run on the build machine) TERMINAL

```
$ java Stampede.java
no protection (cold key)           clients=200  db calls=200
single flight (lock + re-check)    clients=200  db calls=1
```

The unprotected run executes the loader 200 times; the single-flight run executes it once. In a service, those 199 extra executions are 199 extra database connections occupied for the duration of the query, and the connection pool is exhausted first.

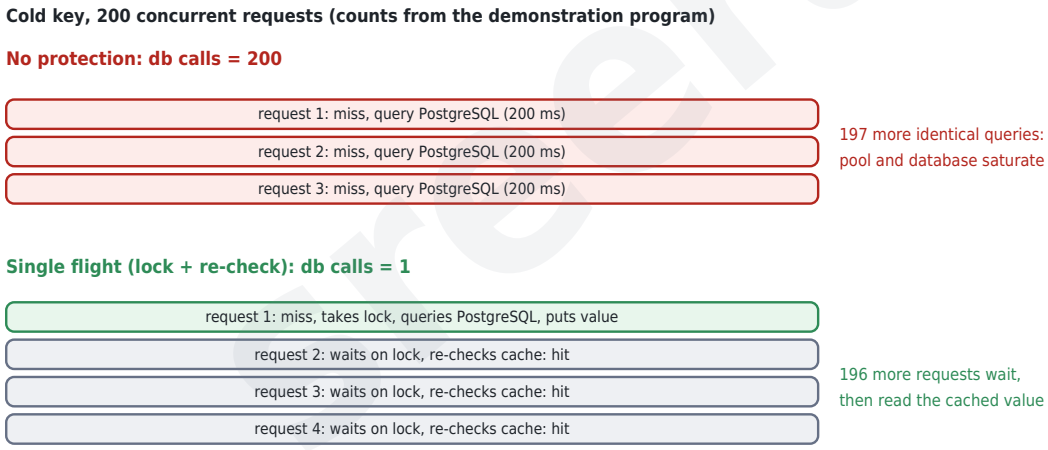


Figure 6.10. Without single flight every concurrent miss queries the database; with a lock and a re-check one request loads and the rest read the stored value.

Mitigations in order of preference: single flight (`sync = true`, or a distributed lock with a short lease for multi-instance protection), TTL jitter, serving stale data while one request refreshes (soft TTL), pre-warming hot keys after a deployment, and caching negative results for a short time (a few seconds) against penetration.

6.3.5 Hands-on

typical output (not run on the build machine) TERMINAL

```
$ docker run -d --name redis -p 6379:6379 redis:7
$ curl -s localhost:8080/products/42            # miss: loads from PostgreSQL
{"id":42,"name":"Keyboard","price":49.90,"version":3}
$ docker exec redis redis-cli KEYS 'catalog:*'
1) "catalog:products::42"
$ docker exec redis redis-cli TTL catalog:products::42
(integer) 1794
$ curl -s -X PUT localhost:8080/products/42 -H 'Content-Type: application/json' \
    -d '{"name":"Keyboard Pro","price":59.90}'
$ docker exec redis redis-cli EXISTS catalog:products::42
(integer) 0
```

The sequence shows the key naming, the TTL counting down from the configured 30 minutes, and the key disappearing after the committed update.

Actuator exposes cache statistics through the metric `cache.gets` with tags `result=hit|miss` and `cache=products` for caches that report statistics; for Redis caches enable them with `spring.cache.redis.enable-statistics=true`. The hit ratio is `hit / (hit + miss)`.

6.3.6 Real-world problems and solutions

Problem 1: cache stampede after expiry

Scenario. A product list endpoint caches a heavy aggregate query with a TTL of five minutes. Every five minutes the 95th percentile latency spikes from 20 ms to 4 seconds. **Symptoms.** Periodic latency spikes; `hikaricp.connections.pending` rises at the same moments; PostgreSQL shows many identical queries in `pg_stat_activity`. **Diagnosis.** Correlate the spike timestamps with the TTL; run `SELECT query, count(*) FROM pg_stat_activity WHERE state = 'active' GROUP BY query` during a spike and observe the same statement dozens of times. **Root cause.** The key expires, 150 concurrent requests miss, and each runs the 3-second aggregate query. **Fix.** `@Cacheable(sync = true)` for single flight per instance; TTL jitter; for several instances, a Redis lock (`SET key value NX PX 5000`) around the loader, or a background refresh job that rewrites the entry before it expires. **Prevention.** Load tests that start from a cold cache, and a dashboard for hit ratio and loader concurrency.

Problem 2: stale data after an update

Scenario. After a price change the website shows the old price for up to 30 minutes, and occasionally the old price reappears after the new one was visible. **Symptoms.** Support tickets; the database has the new value, `redis-cli GET` returns the old one. **Diagnosis.** Compare the DB row and the cached value; check whether `@CacheEvict` is on a method reached through self-invocation; inspect the order of eviction and commit (log both with timestamps). **Root cause.** Two defects: the update method called `evict` through `this` (no proxy), and a second code path (a batch import using JDBC directly) changed the table without touching the cache. The re-appearance came from a reader that had loaded the old row before the commit and stored it after the eviction. **Fix.** Evict through a published event with `AFTER_COMMIT`; route every write path through the service; shorten the TTL of volatile data; store a version in the value and reject writes of an older version. **Prevention.** Define the invariant ("a price is at most 60 seconds stale") per cache, and test the write path with an integration test against a real Redis (Testcontainers).

Problem 3: the cache makes things worse when Redis is slow

Scenario. A Redis failover takes 20 seconds. During that time all requests of the service time out. **Symptoms.** Tomcat threads blocked in `LettuceConnection`; 5xx rate rises; the database is idle. **Diagnosis.** `jcmd <pid> Thread.print` shows threads waiting in `io.lettuce.core.protocol.AsyncCommand`; the command timeout was the default one minute. **Root cause.** No bounded timeout, and the cache error propagated to the caller. **Fix.** `spring.data.redis.timeout=300ms`, a `CacheErrorHandler` that logs and falls through to the database, and a circuit breaker around cache access for sustained outages. **Prevention.** Treat the cache as optional in every code path, and rehearse its failure.

6.3.7 Questions to expect

1. **How does `@Cacheable` work internally?** A proxy with `CacheInterceptor` checks the `Cache` for the computed key before invoking the method and stores the result on a miss. It is bypassed by self-invocation.
2. **What is the difference between `@CachePut` and `@Cacheable`?** `@CachePut` always executes the method and updates the cache; `@Cacheable` skips the method on a hit.
3. **What is cache-aside, and what are its alternatives?** The application reads the cache first and loads and stores on a miss. Read-through puts that logic in the cache layer; write-through and write-behind write to the cache and the store together or asynchronously.
4. **What is a cache stampede and how is it prevented?** Many concurrent misses of one key all hit the backend. Single flight (`sync = true` or a lock), TTL jitter and background refresh prevent it.
5. **How can the cache and the database become inconsistent?** A missed or early eviction, a write path that bypasses the cache, and the read-old, write-new, store-old race. Evict after commit, use TTLs and version values.
6. **Why should entities not be cached directly?** They carry lazy proxies and a persistence context association; serialisation fails or triggers lazy loading, and the class layout becomes the cache schema. DTOs or records are stable and small.
7. **What is cache penetration?** Repeated requests for non-existent keys bypass the cache. Cache a negative marker for a short time or use a Bloom filter in front.
8. **What happens when Redis is down?** Without a `CacheErrorHandler`, cache exceptions fail the request. Bounded timeouts and an error handler degrade to database reads.

KEY POINTS

`@Cacheable` is a proxy-based, key-driven cache-aside; the key, the TTL and the serialisation format are design decisions, not defaults.

- Evict after commit (`@TransactionalEventListener(AFTER_COMMIT)`) and keep a TTL as the safety net.
- A cold hot key creates a stampede; single flight, TTL jitter and soft refresh bound the load on the database.
- Cache DTOs, not entities, and bound Redis timeouts so that a cache failure degrades latency instead of availability.

6.4 Packaging and Containers

A Spring Boot application is delivered as an executable archive and, in most organisations, as a container image. The choices made at this stage (layer structure, base image, JVM memory flags, user, shutdown behaviour) decide build time, image size, start-up time, resource consumption and whether a deployment drops requests. This chapter explains the structure of the executable jar, builds an efficient image in two ways, sets JVM flags correctly for containers, and covers graceful shutdown and local development with Docker Compose. The JVM behaviour inside containers (cgroup limits, ergonomics) is covered in the operating-systems guide and is referenced here only by name.

6.4.1 Concept and mechanism

The executable jar and layers

`mvn package` with the `spring-boot-maven-plugin` runs the `repackage` goal, which rewrites the plain jar into an executable archive: application classes under `BOOT-INF/classes`, dependency jars under `BOOT-INF/lib`, the launcher classes under `org/springframework/boot/loader`, and a manifest whose `Main-Class` is `JarLauncher` and whose `Start-Class` is the application class. Nested jars are not supported by the standard class loader, which is why `JarLauncher` installs its own `LaunchedURLClassLoader`.

The archive also contains `BOOT-INF/layers.idx`, which assigns every file to a layer: `dependencies` (released dependencies), `spring-boot-loader`, `snapshot-dependencies` and `application`. Docker caches an image layer as long as the instruction and its input are unchanged. If the entire fat jar is one `COPY` layer, a one-line code change invalidates the full 40 MB (typical size) layer; with extracted layers only the `application` layer, a few hundred kilobytes, changes.

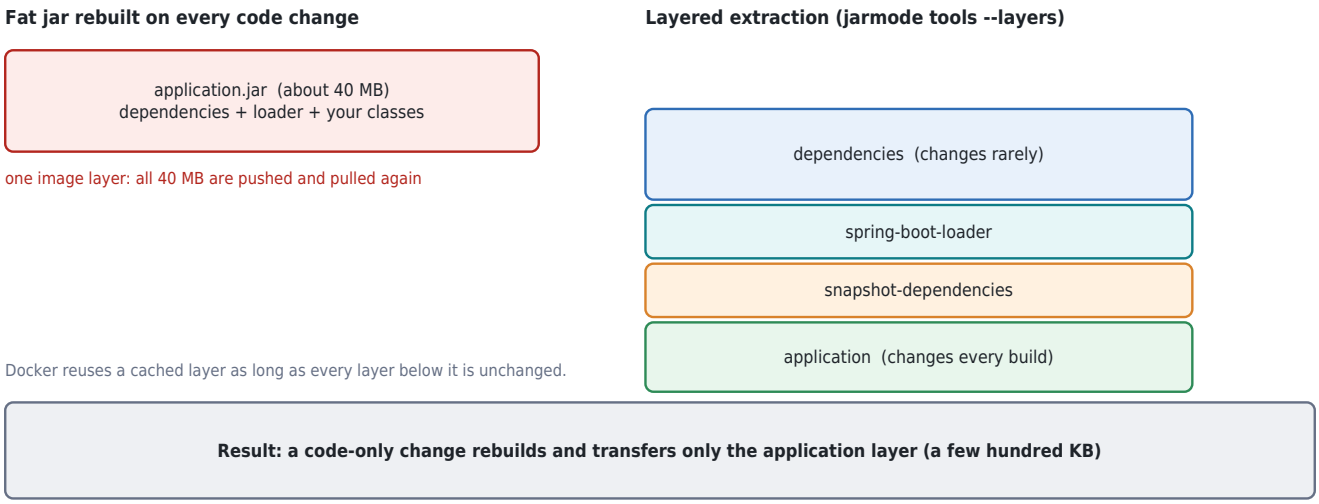


Figure 6.11. Extracting the jar into dependency and application layers confines a code-only change to the smallest layer, so builds and pulls transfer a fraction of the data.

Spring Boot 3.3 introduced the `tools` jar mode (`java -Djarmode=tools -jar app.jar extract --layers --destination extracted`) as the replacement of the older `layertools` mode. The extracted application starts with `java -jar application.jar` from the extraction directory; the jar then contains only application classes and references its dependencies through the manifest class path, which is also the layout recommended for class-data sharing and the ahead-of-time cache.

Two ways to build an image

Dockerfile. Full control and the most transparent option. A multi-stage build uses a JDK image to extract the layers and a JRE image to run them.

Cloud Native Buildpacks. `mvn spring-boot:build-image` (or `gradle bootBuildImage`) runs the Paketo Java buildpack through the Docker daemon. It selects a JRE, creates layers, configures the memory calculator, and sets a non-root user. No Dockerfile is written. The cost is less control and a requirement for a Docker daemon at build time.

6.4.2 Worked example

Dockerfile DOCKERFILE

```
# ---------- stage 1: extract layers ----------
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /builder
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} application.jar
RUN java -Djarmode=tools -jar application.jar extract --layers --destination extracted

# ---------- stage 2: runtime ----------
FROM eclipse-temurin:21-jre
WORKDIR /application

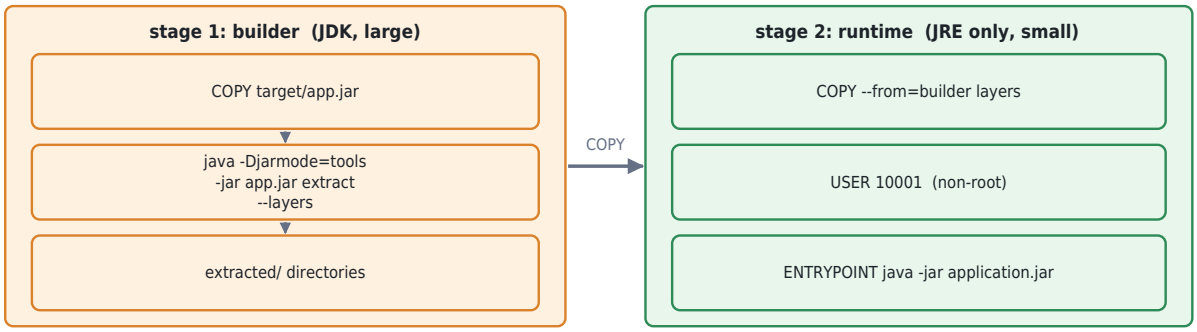
RUN groupadd --system --gid 10001 app \
 && useradd --system --uid 10001 --gid app --no-create-home app

COPY --from=builder /builder/extracted/dependencies/ ./
COPY --from=builder /builder/extracted/spring-boot-loader/ ./
COPY --from=builder /builder/extracted/snapshot-dependencies/ ./
COPY --from=builder /builder/extracted/application/ ./

USER 10001:10001
EXPOSE 8080
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
ENTRYPOINT ["java", "-jar", "application.jar"]
```

The instruction order is the layer order: dependencies first because they change least. `USER 10001:10001` makes the process run as a numeric non-root user, which Kubernetes `runAsNonRoot` verifies without needing to resolve a name. The base image tags should be pinned further than `21-jre` in production (for example by digest) so that rebuilds are

reproducible. The structure follows the Spring Boot reference Dockerfile; the base image there is a Liberica JRE and the one here is Eclipse Temurin.



Only stage 2 becomes the pushed image; the JDK, Maven cache and source never ship.

Figure 6.12. Only the runtime stage becomes the pushed image; the JDK, the build cache and the extraction step stay in the discarded builder stage.

The same result with buildpacks, configured in the `pom.xml`:

pom.xml (fragment) XML

```xml
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <image>
            <name>registry.example.com/shop/orders:${project.version}</name>
            <env>
                <BP_JVM_VERSION>21</BP_JVM_VERSION>
            </env>
        </image>
    </configuration>
</plugin>
```

.dockerignore

```
target/
.git/
.idea/
*.iml
node_modules/
```

6.4.3 JVM flags and memory in containers

Since JDK 10 the JVM reads the cgroup limits of the container. Without flags, the maximum heap is 25 per cent of the container limit (`MaxRAMPercentage` default 25), which wastes memory in a container that holds only one JVM. `-XX:MaxRAMPercentage=75` is a common starting point for containers of 1 GiB or more; for smaller limits a lower percentage is needed because non-heap memory is a larger fraction.

Heap is only part of the footprint. The process also needs metaspace, the code cache, thread stacks (default 1 MiB each on 64-bit Linux, and a Tomcat of 200 threads can reserve 200 MiB), direct buffers (NIO, Netty, Kafka client), GC data structures and native allocator overhead. The Paketo memory calculator computes `-Xmx` from the container limit after subtracting estimates for those areas (thread count default 250, class count from the jar) and writes the flags into `JAVA_TOOL_OPTIONS`; with a hand-written Dockerfile the sizing is the operator's task.

sreeram

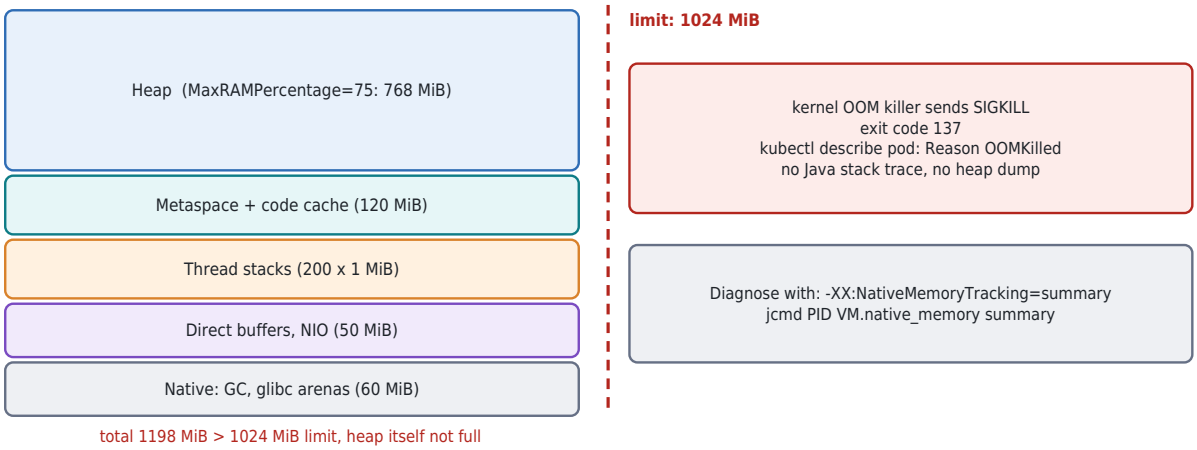


Figure 6.13. A container with a 1 GiB limit can be killed while the heap is below its maximum, because heap, metaspace, thread stacks, direct buffers and native memory together exceed the cgroup limit.

Useful flags:

Flag	Purpose
`-XX:MaxRAMPercentage=75`	maximum heap as a fraction of the container memory limit
`-XX:+ExitOnOutOfMemoryError`	terminate on OutOfMemoryError so that the orchestrator restarts a clean process
`-XX:+HeapDumpOnOutOfMemoryError`	write a heap dump; needs a writable volume and enough disk
`-XX:NativeMemoryTracking=summary`	enable native memory accounting for jcmd VM.native_memory
`-Xss512k`	smaller thread stacks when thread counts are high
`-XX:+UseG1GC / -XX:+UseSerialGC`	the JVM selects Serial GC under 2 CPUs or 1792 MiB of memory; G1 otherwise

JVM flags for containerised Spring Boot services (JDK 21).

The last row reflects the JVM ergonomics rule that a machine with fewer than two available processors or less than about 1792 MiB of memory is classified as not a server-class machine and gets the serial collector; the choice should be verified with `java -XX:+PrintFlagsFinal -version` inside the actual container.

6.4.4 Graceful shutdown

When Kubernetes (or `docker stop`) terminates a container, it sends `SIGTERM`, waits for the grace period, then sends `SIGKILL`. Without special handling, Tomcat stops immediately and in-flight requests receive connection resets. With `server.shutdown=graceful` the embedded web server stops accepting new connections, lets active requests finish, and the application context closes afterwards; the wait is bounded by `spring.lifecycle.timeout-per-shutdown-phase` (default 30 s). The behaviour is implemented by `SmartLifecycle` beans that are stopped in phases: the web server's graceful shutdown phase runs before the closing of beans such as data sources.

application.yml (fragment) YAML

```yaml
server:
  shutdown: graceful
spring:
  lifecycle:
    timeout-per-shutdown-phase: 25s
```

The principle can be demonstrated with the JDK HTTP server: a handler takes one second, the server is stopped 300 ms after the request starts, once with `stop(0)` (immediate) and once with `stop(5)` (wait up to five seconds for exchanges to finish).

Graceful.java JAVA

```java
import com.sun.net.httpserver.HttpServer;
import java.net.*;
import java.net.http.*;
import java.nio.charset.StandardCharsets;
import java.time.Duration;

public class Graceful {
    static String attempt(int stopDelaySeconds, int port) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", port), 0);
        server.createContext("/slow", ex -> {
            try { Thread.sleep(1000); } catch (InterruptedException e) { }
            byte[] body = "done".getBytes(StandardCharsets.UTF_8);
            ex.sendResponseHeaders(200, body.length);
            ex.getResponseBody().write(body);
            ex.close();
        });
        server.start();
        HttpClient client = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(2)).build();
        var req = HttpRequest.newBuilder(URI.create("http://127.0.0.1:" + port + "/slow")).build();
        var future = client.sendAsync(req, HttpResponse.BodyHandlers.ofString());
        Thread.sleep(300);                      // request is now in flight
        server.stop(stopDelaySeconds);          // SIGTERM equivalent
        String result;
        try { result = "HTTP " + future.get().statusCode() + " " + future.get().body(); }
        catch (Exception e) { result = "client error: " + e.getCause().getClass().getSimpleName(); }
        return String.format("stop(%d): in-flight request -> %s", stopDelaySeconds, result);
    }
    public static void main(String[] a) throws Exception {
        System.out.println(attempt(0, 21699));
        System.out.println(attempt(5, 21698));
    }
}
```

Output of the shutdown simulation (run on the build machine) TERMINAL

```
$ java Graceful.java
stop(0): in-flight request -> client error: IOException
stop(5): in-flight request -> HTTP 200 done
```

An immediate stop cuts the in-flight exchange and the client sees an `IOException`; a stop with a delay lets the request complete with status 200. Spring Boot applies the same idea at the framework level.

Kubernetes adds a second problem: removing the pod from the Service endpoints and delivering `SIGTERM` happen concurrently, so a few requests can still be routed to a pod that has begun shutting down. A `preStop` hook with a short sleep (5 s) delays the signal until endpoint propagation has completed. The grace period must exceed the preStop sleep plus the shutdown timeout, or the container receives `SIGKILL` first.

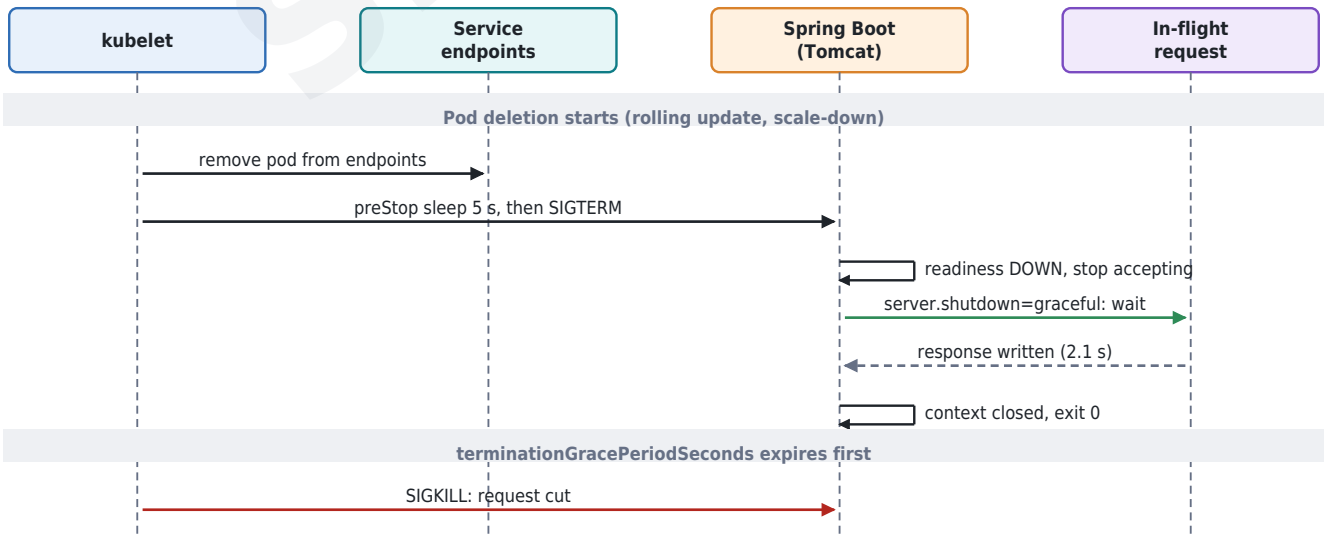


Figure 6.14. Endpoint removal, the preStop delay and the application drain run in sequence; when the grace period expires first, the kubelet kills the process and the in-flight request is lost.

6.4.5 Local development with Docker Compose

A compose file defines the backing services of the application for local work.

compose.yaml YAML

```yaml
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: orders
      POSTGRES_USER: orders
      POSTGRES_PASSWORD: orders
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U orders -d orders"]
      interval: 5s
      timeout: 3s
      retries: 10
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7
    ports:
      - "6379:6379"

  kafka:
    image: apache/kafka:3.8.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1

volumes:
  pgdata:
```

The single-node KRaft configuration is for development only. With the dependency `spring-boot-docker-compose` (Maven `optional`, Gradle `developmentOnly`), Spring Boot 3.1 and later finds `compose.yaml` in the working directory, runs `docker compose up` at startup, and creates connection details for recognised images, so that `spring.datasource.url` and `spring.data.redis.port` need not be configured. Matching is by image name (`postgres`, `redis`, and a list of other databases and brokers; the reference documentation lists which Kafka images are recognised for the Boot version in use). The relevant properties are `spring.docker.compose.file`, `spring.docker.compose.lifecycle-management` (`none`, `start-only`, `start-and-stop`) and `spring.docker.compose.skip.in-tests` (Compose support is off in tests by default). Services that are already running are reused.

For tests, Testcontainers with `@ServiceConnection` (Boot 3.1 and later) fills the same role; the testing part of this guide covers it.

6.4.6 Hands-on

typical output (not run on the build machine) TERMINAL

```
$ ./mvnw -q package -DskipTests
$ docker build -t orders:dev .
$ docker images orders:dev --format '{{.Repository}}:{{.Tag}}  {{.Size}}'
orders:dev  318MB
$ docker history orders:dev --format '{{.Size}}\t{{.CreatedBy}}' | head -6
2.1MB      COPY /builder/extracted/application/ ./
0B         COPY /builder/extracted/snapshot-dependencies/ ./
255kB      COPY /builder/extracted/spring-boot-loader/ ./
38.4MB     COPY /builder/extracted/dependencies/ ./
2.3kB      RUN /bin/sh -c groupadd --system ...
$ docker run --rm -p 8080:8080 -m 768m orders:dev
```

After editing one controller and rebuilding, `docker build` reports `CACHED` for the layers up to `dependencies`, and `docker push` uploads only the changed `application` layer.

6.4.7 Real-world problems and solutions

Problem 1: container killed with OOMKilled while the heap is healthy

Scenario. A service with a 1 GiB memory limit restarts every few hours; Grafana shows the JVM heap at 450 MiB of 768 MiB just before each restart. **Symptoms.** `kubectl get pod` shows `RESTARTS` increasing; `kubectl describe pod` shows `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`; the application log ends without an exception. **Diagnosis.** The kernel killed the process, so no Java diagnostics exist. Check `container_memory_working_set_bytes` against the limit. Start the container with `-XX:NativeMemoryTracking=summary` and run `jcmd <pid> VM.native_memory summary` after some hours; compare `Thread`, `Class`, `Code`, `Internal` and `Other` against the plan. Count threads with `jcmd <pid> Thread.print | grep -c '"'`. **Root cause.** `MaxRAMPercentage=75` assumed 25 per cent for everything else, but 400 request threads (400 MiB reserved stack), 120 MiB metaspace and 80 MiB of direct buffers from the Kafka client exceeded it. **Fix.** Lower the heap fraction (60 per cent), cap Tomcat threads (`server.tomcat.threads.max=100`) or use virtual threads, set `-Xss512k`, limit direct memory (`-XX:MaxDirectMemorySize`), or raise the container limit. **Prevention.** Budget memory as the sum of its parts, load-test at the container limit, and alert on `working_set / limit` above 85 per cent.

Problem 2: image is large and builds are slow

Scenario. A pipeline takes nine minutes, of which six are image build and push; the image is 780 MB. **Symptoms.** Every push uploads the full image; the registry is slow; pulls on new nodes take a minute. **Diagnosis.** `docker history <image>` shows one 560 MB layer for the jar copy and a JDK base image; `docker image ls` shows the size; `dive` can show layer content. **Root cause.** The Dockerfile copies the fat jar in one layer and uses a full JDK image as the runtime stage. **Fix.** Multi-stage build with layer extraction and a JRE base image; `.dockerignore` for `target/` history and `.git`; remove unused starters and dependencies (`mvn dependency:tree`, `mvn dependency:analyze`); for fast start-up consider the AOT cache or CDS. **Prevention.** Track image size in the pipeline and fail on a growth threshold.

Problem 3: requests fail during a deployment

Scenario. Every rolling update produces a burst of 502 responses at the ingress for about three seconds, plus a few client-visible connection resets. **Symptoms.** 502/503 spikes aligned with deployments; no errors in the application log. **Diagnosis.** Compare the timestamps of the 502s with pod termination events (`kubectl get events --sort-by=.lastTimestamp`); check `server.shutdown` in the effective configuration (`/actuator/configprops` or `/actuator/env`); check `terminationGracePeriodSeconds`. **Root cause.** `server.shutdown` was `immediate` (the default), and there was no `preStop` delay, so endpoints still pointed at a terminating pod. **Fix.** `server.shutdown=graceful`, a `preStop` sleep of 5 seconds, `terminationGracePeriodSeconds` larger than the sum of that sleep and the shutdown timeout, and a readiness probe that turns `OUT_OF_SERVICE` during shutdown (Boot does this when probes are enabled). **Prevention.** A deployment test that runs a constant request stream during a rollout and asserts zero errors.

6.4.8 Questions to expect

1. **Why layered jars?** Layers separate rarely changing dependencies from frequently changing application classes so that Docker caches and registries transfer only what changed.
2. **Dockerfile or buildpacks?** Dockerfiles give full control and transparency; buildpacks give secure, consistent defaults without maintaining a Dockerfile but need a Docker daemon and offer less tailoring.
3. **What does `MaxRAMPercentage` do and why is 75 not always safe?** It sets the maximum heap as a percentage of the container limit. Non-heap memory (stacks, metaspace, direct buffers) can exceed the remaining 25 per cent in thread-heavy services.
4. **What is exit code 137?** The process received `SIGKILL` (128 + 9); in Kubernetes with `OOMKilled` it was the kernel out-of-memory killer.
5. **How does graceful shutdown work in Spring Boot?** `server.shutdown=graceful` stops accepting new requests and waits for active ones for up to `spring.lifecycle.timeout-per-shutdown-phase` before the context closes.
6. **Why run the container as non-root?** A compromise of the application then has no root privileges inside the container, which limits escape and file system damage; Kubernetes can enforce it with `runAsNonRoot`.
7. **Why a multi-stage build?** The build tools and the JDK stay in the first stage; only runtime files are in the final image, which is smaller and has a smaller attack surface.
8. **How is configuration supplied to a container?** Through environment variables (relaxed binding maps `SPRING_DATASOURCE_URL` to `spring.datasource.url`), mounted files and `spring.config.import`; never baked into the image.

KEY POINTS

Layered extraction and a JRE runtime stage give small, cache-friendly images; a non-root numeric user is the default.

- Heap is only part of container memory; budget stacks, metaspace, direct buffers and native memory, or the kernel kills the process with exit code 137.
- `server.shutdown=graceful`, a `preStop` delay and a sufficient `terminationGracePeriodSeconds` are all required for zero-error rollouts.
- Docker Compose with Spring Boot's Compose support, or Testcontainers with `@ServiceConnection`, replaces manual configuration of local backing services.

6.5 Kubernetes for Spring Developers

Kubernetes runs containers, restarts them when they fail, routes traffic to the healthy ones and replaces them gradually during a release. A Spring developer does not need to administer a cluster, but must understand the objects that determine whether an application starts, receives traffic, scales and shuts down correctly. Most production incidents involving Spring Boot on Kubernetes are interactions between application behaviour (start-up time, memory use, shutdown) and platform behaviour (probes, limits, rolling updates). The YAML in this chapter is typical configuration and was not applied to a cluster on the build machine.

6.5.1 Concept and mechanism

The objects

A **Pod** is one or more containers sharing a network namespace; it is the unit of scheduling and is disposable. A **Deployment** declares the desired number of identical pods and an update strategy; it manages **ReplicaSets**, one per pod template version. A **Service** gives a set of pods (selected by labels) a stable virtual IP and DNS name (`orders.shop.svc.cluster.local`) and load-balances over the ready ones. An **Ingress** routes external HTTP(S) requests by host and path to services, through an ingress controller such as ingress-nginx. A **ConfigMap** holds non-sensitive configuration and a **Secret** holds sensitive values (stored base64-encoded, which is an encoding and not encryption; encryption at rest and RBAC must be configured on the cluster). Both are injected as environment variables or mounted as files.

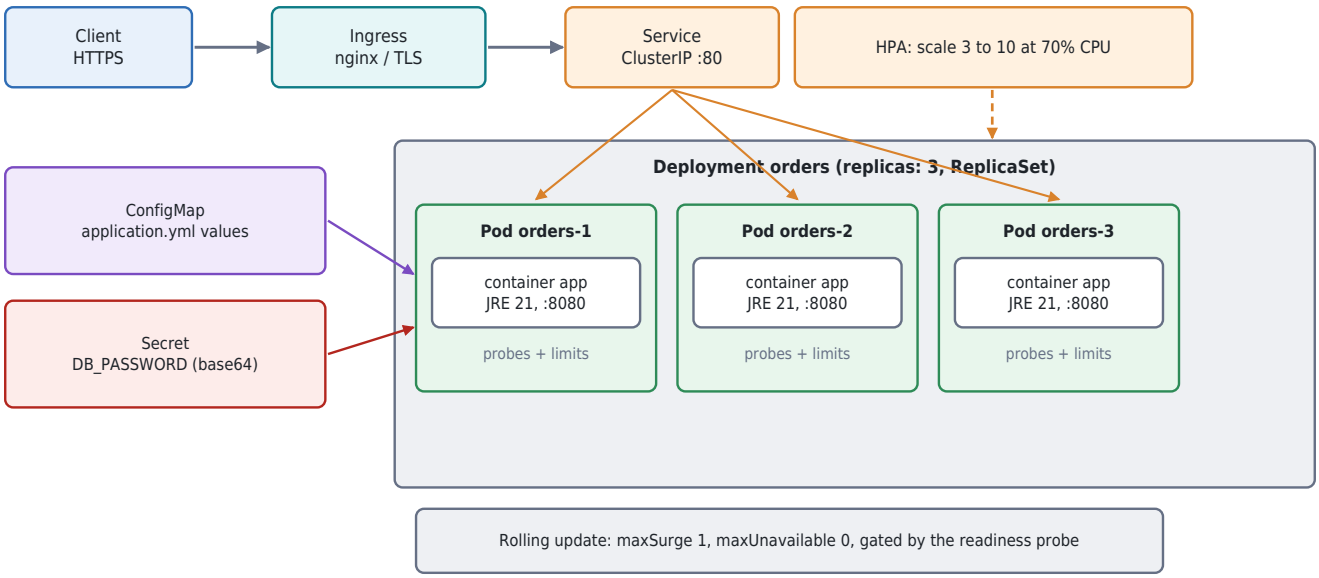


Figure 6.15. Traffic enters through the ingress and the service, reaches only ready pods of the Deployment, and the pods receive configuration from a ConfigMap and a Secret.

Probes

Three probes tell the kubelet what to do with a container:

- The **startup probe** runs until it first succeeds; liveness and readiness probes are disabled until then. It covers slow start-up without making the liveness probe lenient forever.
- The **liveness probe** answers "is the process unrecoverably broken?". Failure makes the kubelet restart the container.
- The **readiness probe** answers "can this pod serve traffic now?". Failure removes the pod from the Service endpoints without restarting it.

Spring Boot Actuator provides two health groups for this purpose: `/actuator/health/liveness` (state `LivenessState`, `CORRECT` or `BROKEN`) and `/actuator/health/readiness` (state `ReadinessState`, `ACCEPTING_TRAFFIC` or `REFUSING_TRAFFIC`). They are enabled automatically when Boot detects Kubernetes through the `*_SERVICE_HOST` and `*_SERVICE_PORT` environment variables, and explicitly with `management.endpoint.health.probes.enabled=true`. During graceful shutdown the readiness state changes to `REFUSING_TRAFFIC` first. The liveness group should not include external dependencies such as the database: if the database is down, restarting every pod does not help and turns an incident into a restart storm. The readiness group may include them.

Resources

Each container has `requests` (the amount the scheduler reserves, and the CPU share under contention) and `limits` (the cap). A memory limit is enforced by the kernel: exceeding it ends in `OOMKilled`. A CPU limit is enforced by throttling via the CFS quota; a JVM that starts with a CPU limit of 500m is throttled during class loading and JIT compilation, so start-up takes several times longer than with a larger limit, which can fail the probes. The JVM sizes its default thread pools and GC threads from the CPU limit it sees.

Rolling updates

A Deployment with `strategy.type: RollingUpdate` replaces pods gradually. `maxSurge` is the number of extra pods allowed above the desired count and `maxUnavailable` the number of pods that may be unavailable. With `maxSurge: 1` and `maxUnavailable: 0`, a new pod is created, and only when its readiness probe succeeds is an old pod terminated. The readiness probe is therefore the gate of the whole update.

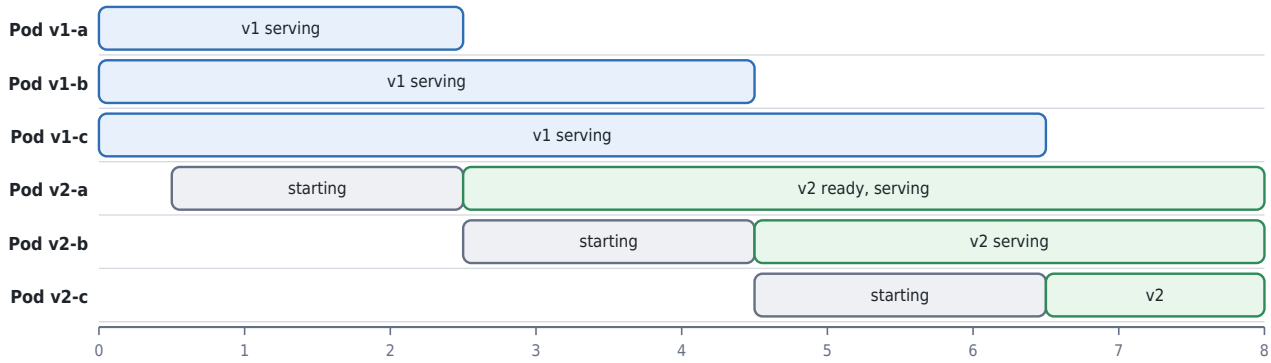


Figure 6.16. With maxSurge 1 and maxUnavailable 0, each new pod must become ready before one old pod is terminated, so serving capacity never drops below the desired replica count.

6.5.2 Worked example

k8s/configmap.yaml YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: orders-config
  namespace: shop
data:
  SPRING_PROFILES_ACTIVE: "prod"
  SPRING_DATASOURCE_URL: "jdbc:postgresql://postgres.shop.svc.cluster.local:5432/orders"
  SPRING_DATASOURCE_USERNAME: "orders"
  SPRING_KAFKA_BOOTSTRAP_SERVERS: "kafka.shop.svc.cluster.local:9092"
  SERVER_SHUTDOWN: "graceful"
  MANAGEMENT_ENDPOINT_HEALTH_PROBES_ENABLED: "true"
```

Relaxed binding maps the upper-case environment variable `SPRING_DATASOURCE_URL` to the property `spring.datasource.url`; underscores replace dots and hyphens.

k8s/secret.yaml YAML

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: orders-secret
  namespace: shop
type: Opaque
stringData:
  SPRING_DATASOURCE_PASSWORD: "change-me"
```

This file is a template. A real value is never committed; it is created by a secret manager integration (External Secrets Operator, Sealed Secrets, a CSI driver) or by the pipeline from a protected store.

k8s/deployment.yaml YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders
  namespace: shop
  labels:
    app: orders
spec:
  replicas: 3
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app: orders
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
```

sreeram

k8s/deployment.yaml (continued) YAML

```yaml
    metadata:
      labels:
        app: orders
    spec:
      terminationGracePeriodSeconds: 45
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
      containers:
        - name: orders
          image: registry.example.com/shop/orders:1.4.2
          ports:
            - containerPort: 8080
              name: http
          envFrom:
            - configMapRef:
                name: orders-config
            - secretRef:
                name: orders-secret
          env:
            - name: JAVA_TOOL_OPTIONS
              value: "-XX:MaxRAMPercentage=70 -XX:+ExitOnOutOfMemoryError"
          resources:
            requests:
              cpu: "500m"
              memory: "768Mi"
            limits:
              memory: "1Gi"
          startupProbe:
            httpGet:
              path: /actuator/health/liveness
              port: http
            periodSeconds: 5
            failureThreshold: 30
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: http
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: http
            periodSeconds: 5
            failureThreshold: 3
          lifecycle:
            preStop:
              exec:
                command: ["sleep", "5"]
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
          volumeMounts:
            - name: tmp
              mountPath: /tmp
      volumes:
        - name: tmp
          emptyDir: {}
```

What to notice: the memory request is below the limit but the CPU limit is omitted on purpose (a request without a limit avoids throttling while still reserving capacity; the policy is a team decision); the startup probe allows 30 x 5 s = 150 s for start-up before liveness takes over; the `preStop` sleep of 5 s plus a shutdown timeout of 25 s is below `terminationGracePeriodSeconds: 45`; `readOnlyRootFilesystem` requires a writable `/tmp`, which Tomcat and the JVM use for working files; and the heap fraction (70 per cent of 1 GiB) leaves 300 MiB for non-heap memory.

k8s/service-ingress.yaml YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: orders
  namespace: shop
```

sreeram

k8s/service-ingress.yaml (continued) YAML

```yaml
spec:
  selector:
    app: orders
  ports:
    - name: http
      port: 80
      targetPort: http
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: orders
  namespace: shop
  annotations:
    nginx.ingress.kubernetes.io/proxy-read-timeout: "30"
spec:
  ingressClassName: nginx
  tls:
    - hosts: ["api.example.com"]
      secretName: api-example-com-tls
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /orders
            pathType: Prefix
            backend:
              service:
                name: orders
                port:
                  name: http
```

k8s/hpa-pdb.yaml YAML

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: orders
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: orders
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: orders
  namespace: shop
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: orders
```

The HPA computes utilisation relative to the CPU **request**, so the request must be realistic: a request of 500m and an average usage of 350m is 70 per cent. The `PodDisruptionBudget` limits voluntary evictions (node drains) so that at least two replicas stay available.

sreeram

Helm basics

Helm packages the manifests as a chart: `Chart.yaml` (name, version), `values.yaml` (defaults) and `templates/` (manifests with Go templating). A release is created or upgraded with one command, and the previous revision can be restored.

values.yaml (fragment) YAML

```yaml
replicaCount: 3
image:
  repository: registry.example.com/shop/orders
  tag: "1.4.2"
resources:
  requests:
    cpu: 500m
    memory: 768Mi
  limits:
    memory: 1Gi
```

templates/deployment.yaml (fragment)

```
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

typical commands (not run on the build machine) TERMINAL

```
$ helm upgrade --install orders ./chart -n shop --set image.tag=1.4.3 --atomic --timeout 5m
$ helm history orders -n shop
$ helm rollback orders 3 -n shop
```

`--atomic` rolls the release back automatically if the pods do not become ready within the timeout.

6.5.3 Hands-on

typical output (not run on the build machine) TERMINAL

```
$ kubectl apply -f k8s/ && kubectl -n shop rollout status deployment/orders
Waiting for deployment "orders" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "orders" rollout to finish: 2 old replicas are pending termination...
deployment "orders" successfully rolled out
$ kubectl -n shop get pods -l app=orders
NAME                      READY   STATUS    RESTARTS   AGE
orders-6d9c7b5f64-4kq2x   1/1     Running   0          2m
orders-6d9c7b5f64-9tz7p   1/1     Running   0          98s
orders-6d9c7b5f64-vw8m5   1/1     Running   0          54s
$ kubectl -n shop describe pod orders-6d9c7b5f64-4kq2x | grep -A4 'Last State'
$ kubectl -n shop logs deploy/orders --tail=20
$ kubectl -n shop rollout undo deployment/orders
```

6.5.4 Real-world problems and solutions

Problem 1: rolling update stalls, new pods never become ready

Scenario. A new version is deployed; `kubectl rollout status` waits indefinitely, while the old pods keep serving. **Symptoms.** New pods show `0/1 Running`; `READY` never turns to `1/1`; the progress deadline (600 s by default) eventually marks the rollout `ProgressDeadlineExceeded`. **Diagnosis.** `kubectl describe pod <new>` lists `Readiness probe failed: HTTP probe failed with statuscode: 503` or `connection refused` in Events. `kubectl logs <new>` shows whether the application started. `kubectl exec <new> -- curl -s localhost:8080/actuator/health/readiness` (or `wget`) shows the group state and, with `management.endpoint.health.show-details=always`, the failing component. **Root cause.** Three typical causes. The readiness group included `db`, and the new configuration pointed to an unreachable host. The probe port was 8080 while the management server had been moved to `management.server.port=9090`. The probe period was too short for a pod that had only 200m of CPU, so the start-up took 70 seconds and `failureThreshold` was reached. **Fix.** Correct the configuration; align the probe port with `management.server.port`; add a `startupProbe` with a generous `failureThreshold`; raise the CPU request. The old pods keep serving because `maxUnavailable: 0`, which is the reason the readiness gate exists. Roll back with `kubectl rollout undo` if the cause is not immediate. **Prevention.** Run the same image with the production profile in a staging namespace before production; use `helm --atomic`; keep the liveness group free of external dependencies.

Problem 2: liveness probe kills a pod that is merely slow

Scenario. During a traffic peak pods restart in sequence, which reduces capacity and amplifies the load on the others. **Symptoms.** `RESTARTS` increases on all pods within minutes; the log ends at an arbitrary point; `describe` shows `Liveness probe failed: context deadline exceeded`. **Diagnosis.** The Tomcat thread pool is saturated (metric `tomcat.threads.busy` equals `tomcat.threads.config.max`), so the health request waits in the accept queue and exceeds the probe `timeoutSeconds` (1 s default). **Root cause.** The probe shares the thread pool with business traffic, and the liveness probe tests responsiveness instead of broken state. **Fix.** Serve Actuator on a separate management port (`management.server.port`), which uses its own connector and thread pool; raise `timeoutSeconds` and `failureThreshold` of liveness; leave overload to readiness and load shedding. **Prevention.** Treat liveness as "restart will help" only; a slow application should shed load, not restart.

Problem 3: very slow start-up under a CPU limit

Scenario. A service that starts in 12 seconds on a developer laptop takes 95 seconds in the cluster with a CPU limit of 500m. **Symptoms.** Restart loops caused by failing probes; the log timestamps show class loading and bean creation spread over minutes. **Diagnosis.** `container_cpu_cfs_throttled_periods_total` is high during start-up; `kubectl top pod` shows usage pinned at the limit. **Root cause.** The JIT compiler and class loading are CPU-bound and are throttled by the CFS quota. **Fix.** Raise the CPU limit or remove it while keeping a request; add a startup probe; reduce start-up work (lazy initialisation, fewer auto-configurations); use the AOT cache or CDS where the JDK supports it. **Prevention.** Measure start-up in a container with the production limits.

6.5.5 Questions to expect

1. **What is the difference between liveness and readiness?** Liveness failure restarts the container; readiness failure removes the pod from Service endpoints without a restart. Liveness should detect an unrecoverable state, readiness an inability to serve.
2. **What does the startup probe add?** It suspends the other probes until the application has started, which allows slow starts without weakening liveness.
3. **How does Spring Boot support the probes?** The Actuator health groups `liveness` and `readiness`, backed by `ApplicationAvailability` states, enabled automatically on Kubernetes.
4. **What are requests and limits?** Requests are reserved for scheduling; limits are enforced caps. Exceeding the memory limit kills the container; exceeding the CPU limit throttles it.
5. **Explain a rolling update.** The Deployment creates a new ReplicaSet and scales it up while scaling the old one down within `maxSurge` and `maxUnavailable`, gated by readiness.
6. **How does the HPA decide to scale?** It compares the observed average metric (for example CPU utilisation relative to the request) with the target and adjusts the replica count between the minimum and maximum.
7. **How are secrets handled?** As Secret objects injected as environment variables or files, with encryption at rest, RBAC and preferably an external secret store; never in the image or the repository.
8. **Why is a preStop hook used?** Endpoint removal and `SIGTERM` are concurrent; a short sleep lets traffic drain before the application stops accepting connections.

9. **ClusterIP, NodePort, LoadBalancer?** ClusterIP exposes inside the cluster, NodePort on each node's port, LoadBalancer provisions a cloud load balancer; Ingress adds HTTP routing on top.

KEY POINTS

The readiness probe gates both traffic and rolling updates; the liveness probe must not depend on external systems.

- Memory limits kill (OOMKilled), CPU limits throttle (slow start-up); requests drive scheduling and HPA arithmetic.
- Graceful shutdown needs `server.shutdown=graceful`, a preStop delay and a grace period longer than both.
- Configuration comes from ConfigMaps and Secrets through relaxed binding of environment variables; no secrets in images or repositories.

6.6 CI/CD, Maven and Release Strategies

Continuous integration builds and tests every change automatically; continuous delivery keeps the main branch always releasable and automates the path to production. For a Spring Boot service the pipeline compiles with Maven, runs unit and integration tests (the latter with Testcontainers), scans dependencies and the image, builds and pushes the image, and deploys it with a strategy that limits the damage of a faulty release. The pipeline is part of the product: a flaky pipeline stops delivery, and a leaked credential in a pipeline is an incident. The YAML in this chapter is typical GitHub Actions configuration and was not executed on the build machine.

6.6.1 Concept and mechanism

Maven essentials

Maven builds by walking a **lifecycle** of phases. The default lifecycle contains, in order, `validate`, `compile`, `test`, `package`, `verify`, `install` and `deploy`; invoking a phase runs all earlier phases. Plugins bind **goals** to phases: `maven-compiler-plugin:compile` to `compile`, `maven-surefire-plugin:test` to `test`, `maven-failsafe-plugin` to `integration-test` and `verify`, and the `spring-boot-maven-plugin:repackage` goal to `package`. CI normally runs `mvn -B clean verify`: batch mode without colour or interactive prompts, and no `install` or `deploy` of artefacts into repositories.

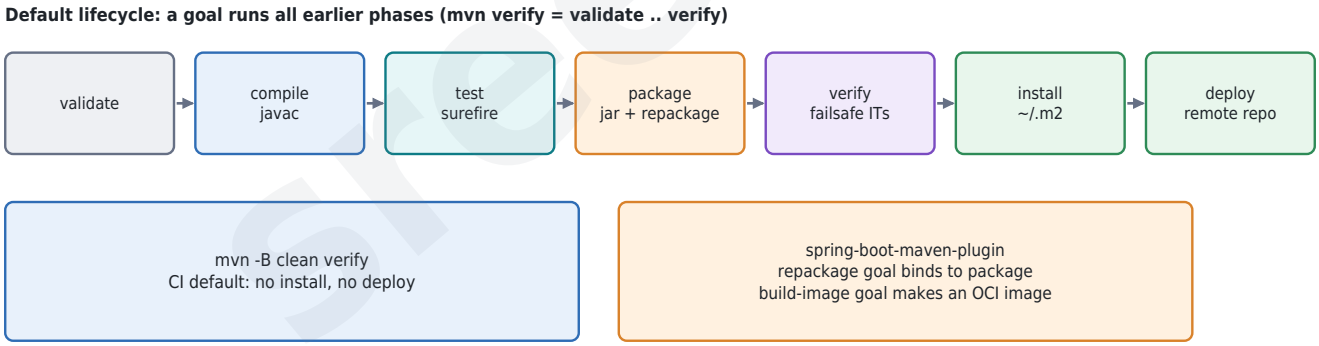


Figure 6.17. Running a phase executes every earlier phase; the Spring Boot plugin attaches repackaging to the package phase and image building is a separate goal.

Dependency management. `spring-boot-starter-parent` (or the `spring-boot-dependencies` BOM imported in `dependencyManagement`) fixes the versions of several hundred libraries to a mutually tested set; dependencies declared without a `version` take the managed version. An override of a managed version is done with a property such as `<jackson-bom.version>` in the properties of the project and requires care, because the set was tested as a whole. Scopes: `compile` (default), `provided` (needed at compile time only, for example servlet API when deploying to a container), `runtime`, `test`. Transitive dependencies are resolved by nearest definition; `mvn dependency:tree` shows the resolved tree and `-Dverbose` shows conflicts.

Surefire and Failsafe. Surefire runs unit tests (classes named `*Test`, `Test*`, `*Tests`, `*TestCase`) in the `test` phase. Failsafe runs integration tests (`*IT`, `IT*`, `*ITCase`) in `integration-test` and checks the results in `verify`, so that cleanup in `post-integration-test` runs even when a test fails. Separating them gives a fast first feedback stage.

Profiles activate sets of configuration by command line (`-Pnative`), property, JDK or environment. They are appropriate for build variants; runtime configuration belongs to Spring profiles (`spring.profiles.active`), not Maven

profiles, because a Maven profile bakes the choice into the artefact and the same artefact should run in every environment.

pom.xml (fragment) XML

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.1</version>
    <relativePath/>
</parent>

<properties>
    <java.version>21</java.version>
</properties>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-failsafe-plugin</artifactId>
            <executions>
                <execution>
                    <goals>
                        <goal>integration-test</goal>
                        <goal>verify</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

The version numbers of Maven plugins that appear without a version in this fragment are managed by the parent. The Boot version in the example is illustrative; the project pins the current patch release of the supported 3.x line.

Versioning and artefacts

The pipeline builds the artefact once and promotes the same image through environments; rebuilding per environment defeats the purpose of testing it. The image tag is immutable and traceable: the Git commit SHA (`orders:3f9c2d1`) for every build and a semantic version (`1.4.2`) for releases. The mutable tag `latest` is avoided in deployments because it cannot be rolled back deterministically and gives no information about what runs. Semantic versioning (MAJOR.MINOR.PATCH) applies to libraries and to public API contracts; for deployed services the commit SHA is usually the real identity.

Sheram

6.6.2 Worked example

.github/workflows/ci.yml YAML

```yaml
name: ci

on:
  pull_request:
  push:
    branches: [main]
    tags: ["v*"]

permissions:
  contents: read

env:
  REGISTRY: registry.example.com
  IMAGE: registry.example.com/shop/orders

jobs:
  build-test:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Build, unit tests, integration tests (Testcontainers)
        run: ./mvnw -B clean verify
      - name: Upload test reports
        if: failure()
        uses: actions/upload-artifact@v4
        with:
          name: surefire-reports
          path: |
            target/surefire-reports
            target/failsafe-reports
      - uses: actions/upload-artifact@v4
        with:
          name: app-jar
          path: target/*.jar

  image:
    needs: build-test
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
      id-token: write
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with:
          name: app-jar
          path: target
      - uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ secrets.REGISTRY_USER }}
          password: ${{ secrets.REGISTRY_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ env.IMAGE }}:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - name: Scan image for known vulnerabilities
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.IMAGE }}:${{ github.sha }}
          severity: CRITICAL,HIGH
          exit-code: "1"
```

sreeram

.github/workflows/ci.yml (continued) YAML

```
          ignore-unfixed: true

  deploy-staging:
    needs: image
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - name: Deploy with Helm
        run: |
          helm upgrade --install orders ./chart -n shop-staging \
            --set image.tag=${{ github.sha }} --atomic --timeout 5m
```

What to notice. The `permissions` block starts at read-only and widens per job. Pull requests run only the `build-test` job; images are built and pushed only on pushes to `main` or tags. The same jar produced by the verified build is reused for the image. The environment `staging` can hold its own secrets and protection rules, and a production job would use an environment with required reviewers (manual approval). The cluster credentials are supplied through the environment's secrets or, better, through short-lived OIDC federation (`id-token: write`) instead of a long-lived kubeconfig. Third-party actions should be pinned to a specific version or commit SHA; the versions in the example are illustrative and must be checked when the workflow is written.

GitHub Actions workflow: one job per stage, fail fast, publish only from main

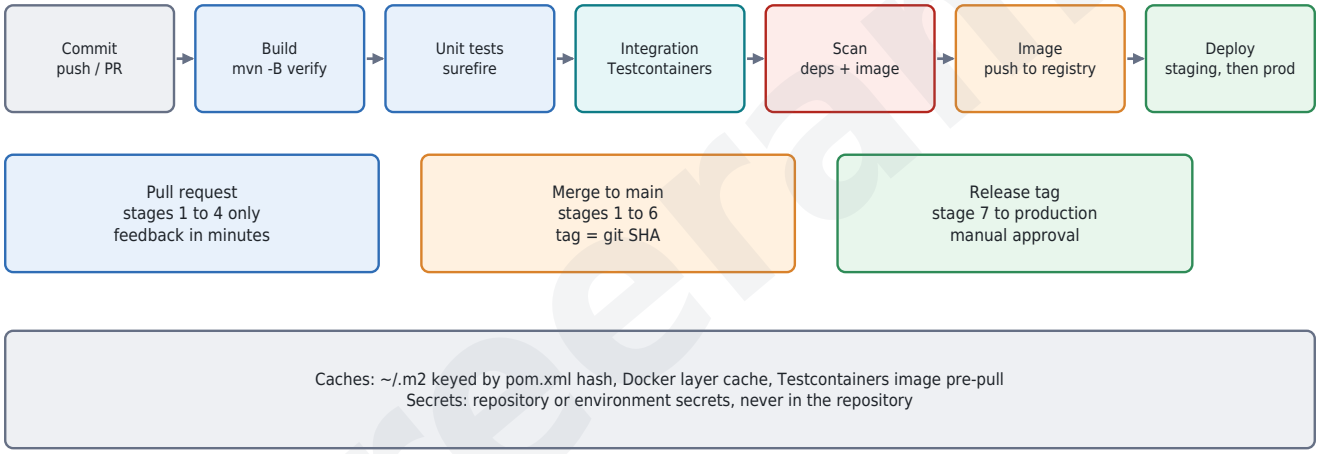


Figure 6.18. Pull requests run only build and test stages for fast feedback; merges to main add the scan and the image; production deployment is gated by a manual approval.

6.6.3 Release strategies

Rolling update. The default in Kubernetes: replace pods gradually. Two versions serve traffic concurrently, so every change must be backward compatible with the previous version, especially the database schema and message formats.

Blue-green. Two complete environments exist; traffic is switched from the live one (blue) to the new one (green) at the router or service selector. Rollback is switching back. The cost is double capacity, and the database must be compatible with both versions.

Canary. A small fraction of traffic (5 per cent) goes to the new version while error rate and latency are compared with the stable version; the share grows step by step or the canary is rolled back. It needs traffic splitting (ingress annotations, a service mesh, Argo Rollouts) and metrics-based analysis, and gives the smallest blast radius.

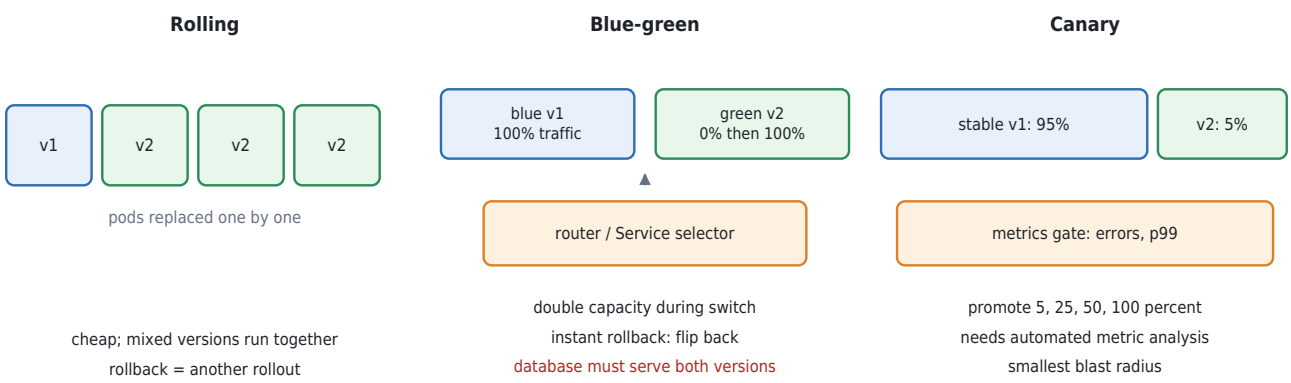


Figure 6.19. Rolling replaces pods in place, blue-green swaps whole environments with an instant rollback, and canary exposes a small share of users first.

Feature flags. A flag separates deployment from release: the code ships switched off and is enabled per user, tenant or percentage at runtime (a library such as FF4j or Unleash, or a simple `@ConfigurationProperties` toggle backed by a refreshable config source). Flags need an owner and a removal date; stale flags are untested code paths.

Database migrations in the pipeline. Schema changes follow the expand-and-contract pattern so that old and new application versions work against the same schema. Step one adds a nullable column or a new table (expand) with Flyway or Liquibase running at application start or as a separate migration job before the rollout; step two deploys code that writes both and reads the new structure; step three, one release later, drops the old column (contract). A destructive migration in the same release as the code that stops using the column breaks a rolling update, because old pods still use it. A migration that locks a large table (an `ALTER TABLE` with a rewrite, an index created without `CONCURRENTLY` in PostgreSQL) can block the whole service; it requires a dedicated procedure.

Secrets management. Credentials enter the pipeline and the runtime through a secret store (GitHub environment secrets, HashiCorp Vault, a cloud secret manager) and are rotated. They never appear in the repository, image layers, build logs or `docker history`. Secret scanning (GitHub secret scanning, gitleaks) runs on every push, and any committed secret is treated as compromised and rotated regardless of removal from history.

6.6.4 Hands-on

A local reproduction of the CI build, the commands that the workflow runs:

typical output (not run on the build machine) TERMINAL

```
$ ./mvnw -B clean verify
[INFO] --- surefire:3.2.5:test (default-test) @ orders ---
[INFO] Tests run: 142, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- spring-boot:3.4.1:repackage (repackage) @ orders ---
[INFO] --- failsafe:3.2.5:integration-test (default) @ orders ---
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- failsafe:3.2.5:verify (default) @ orders ---
[INFO] BUILD SUCCESS
$ ./mvnw dependency:tree -Dincludes=com.fasterxml.jackson.core
$ ./mvnw -B spring-boot:build-image -DskipTests
```

6.6.5 Real-world problems and solutions

Problem 1: the pipeline is flaky because of Testcontainers

Scenario. About one build in ten fails in the integration stage with `Could not start container` or `Connection refused`, and passes on re-run. **Symptoms.** Failures are not tied to code changes; the logs show timeouts waiting for PostgreSQL or Kafka; sometimes `no space left on device`. **Diagnosis.** Read the Testcontainers log lines (`Waiting for database connection to become available`) and the elapsed time per test class; run `docker system df` on the runner; check whether several test classes each start their own container; check the registry for rate-limit errors (`toomanyrequests`) in the image pull. **Root cause.** Each test class started its own PostgreSQL container, producing 25 containers per build on a shared two-core runner; image pulls from a public registry were rate limited; the default startup timeout was exceeded under load. **Fix.** Share one container per JVM with a singleton or `@ServiceConnection` bean in a shared `@TestConfiguration`; enable the Spring test context cache by using identical configuration; pre-pull images from a mirror or the organisation's registry; raise the startup timeout (`withStartupTimeout`) and use wait strategies instead of sleeps; prune images on the runner. **Prevention.** Track test duration and failure rate per test class; quarantine and fix flaky tests, not retry them silently.

Problem 2: a secret is committed to the repository

Scenario. A developer commits `application-prod.yml` containing the database password and a cloud access key; the push succeeds and the repository is visible to the whole organisation (and, in the worst case, public). **Symptoms.** A secret scanning alert, or unexpected usage of the key (billing anomaly, access from unknown addresses). **Diagnosis.** `git log -p --all -S'AKIA'` or a scanner (`gitleaks detect`) locates the commit; the provider's audit log shows use of the credential since the commit time. **Root cause.** Secrets were kept in a file in the repository, `.gitignore` did not exclude it, and no push protection existed. **Fix.** Revoke and rotate the credential first, since history rewriting does not undo exposure; then remove the file and, if required by policy, rewrite history (`git filter-repo`) and invalidate forks and caches. Move values to environment variables resolved from a secret manager (`spring.config.import=optional:configtree:/etc/secrets/` for mounted secrets, or `${DB_PASSWORD}` placeholders). **Prevention.** Enable push protection and secret scanning, add a pre-commit hook (gitleaks), keep only `application.yml` with placeholders in the repository, and grant cloud credentials short lifetimes through OIDC.

Problem 3: the release breaks the running version during the rolling update

Scenario. A migration renames a column in the same release as the code that reads the new name. During the rollout, old pods throw `column "customer_name" does not exist`. **Symptoms.** A burst of 500 responses from old pods during the update; the new pods are healthy. **Diagnosis.** The error log of the old pods shows the SQL error with the time of the Flyway migration; the `flyway_schema_history` table shows the version applied at that time. **Root cause.** The migration was not backward compatible with the version still serving traffic. **Fix.** Roll back the application and apply expand-and-contract: add `full_name`, write both columns, backfill, switch reads in the next release, drop the old column in a later one. **Prevention.** A pipeline test that runs the previous version's integration tests against the migrated schema.

6.6.6 Questions to expect

1. **What are the Maven lifecycle phases?** `validate`, `compile`, `test`, `package`, `verify`, `install`, `deploy` in the default lifecycle; running a phase runs the earlier ones. Plugins bind goals to phases.
2. **What is a BOM?** A POM whose `dependencyManagement` section declares compatible versions; importing it (or inheriting `spring-boot-starter-parent`) lets dependencies omit versions.
3. **Surefire versus Failsafe?** Surefire runs unit tests in `test`; Failsafe runs integration tests in `integration-test` and fails the build in `verify`, after cleanup.
4. **How should images be tagged?** By immutable identifiers (commit SHA, release version) and never deployed by `latest`; one artefact is promoted through the environments.
5. **Blue-green versus canary?** Blue-green switches all traffic between two full environments with instant rollback; canary shifts a fraction progressively with metric analysis.
6. **What is expand-and-contract?** A migration approach in which the schema is first extended compatibly, the code is migrated, and the old structure is removed in a later release, so that two application versions can run concurrently.
7. **How are secrets kept out of a pipeline?** In a secret store, injected at runtime as environment secrets or via OIDC federation, scanned for leaks, rotated, and never echoed into logs.
8. **How is a flaky test handled?** Reproduce and fix the cause (shared state, time, ordering, resources); quarantine with a ticket; never rely on blind retries.

Sperian

9. **Why build the artefact once?** The tested binary is the one deployed; rebuilding can include different dependencies or timestamps.

KEY POINTS

The pipeline builds once, tests at several levels, scans, publishes an immutable image and promotes it; pull requests run the fast stages only.

- `mvn -B clean verify` runs unit tests with Surefire and integration tests with Failsafe; the Spring Boot BOM manages dependency versions.
- Rolling, blue-green and canary releases all require backward-compatible schemas and messages; expand-and-contract migrations make that possible.
- A committed secret is compromised and must be rotated; prevention is push protection, scanning and short-lived credentials.

6.7 Interview Question Bank

The questions below cover the topics of the whole guide, grouped by area. The answers are model answers of two to five lines: a complete spoken answer starts with the mechanism, names the real classes or properties, and ends with a trade-off or a pitfall. A candidate should be able to produce the first sentence of each answer from memory and extend it with the mechanism when the interviewer probes. The chapter on the study plan maps these areas to a six-week schedule.

6.7.1 Container, AOP and auto-configuration

1. **What is the difference between `BeanFactory` and `ApplicationContext`?** `BeanFactory` is the minimal container (`DefaultListableBeanFactory` is the implementation) that creates beans lazily on request. `ApplicationContext` adds eager singleton initialisation, event publication, message resources, environment and profiles, and automatic registration of `BeanPostProcessor` and `BeanFactoryPostProcessor` beans.
2. **Describe the bean creation sequence.** Instantiate (constructor), populate properties (injection), `Aware` callbacks, `BeanPostProcessor.postProcessBeforeInitialization`, `@PostConstruct` and `InitializingBean.afterPropertiesSet`, custom init method, `postProcessAfterInitialization` (where AOP proxies are created), then use; on shutdown `@PreDestroy`, `DisposableBean`, destroy method.
3. `BeanFactoryPostProcessor` **versus** `BeanPostProcessor`**?** The first modifies bean definitions before any bean is instantiated (for example `PropertySourcesPlaceholderConfigurer`, `ConfigurationClassPostProcessor`). The second wraps or modifies bean instances during creation (for example `AutowiredAnnotationBeanPostProcessor`, the auto-proxy creators).
4. **How are circular dependencies handled?** Singleton setter or field injection is resolved with the three-level cache (`singletonObjects`, `earlySingletonObjects`, `singletonFactories`), exposing an early reference. Constructor injection cannot be resolved and fails with `BeanCurrentlyInCreationException`. Since Spring Boot 2.6 circular references are prohibited by default (`spring.main.allow-circular-references`), and the correct fix is to remove the cycle.
5. **Constructor, setter or field injection?** Constructor injection makes dependencies explicit and final, allows plain unit tests without a container and detects cycles at start-up. Setter injection fits optional dependencies. Field injection hides dependencies and prevents immutability.
6. **How does Spring resolve two beans of the same type?** By `@Primary`, then `@Qualifier`, then by matching the parameter name with the bean name; otherwise `NoUniqueBeanDefinitionException`.
7. **What are the bean scopes?** `singleton` (default, one per container), `prototype`, and in web applications `request`, `session`, `application`. A narrower scope injected into a singleton needs a scoped proxy (`proxyMode`) or `ObjectProvider`.
8. **JDK dynamic proxy versus CGLIB?** JDK proxies implement the interfaces of the target and use `InvocationHandler`. CGLIB (repackaged inside Spring, with Objenesis for instantiation) creates a subclass of the target class, so the class and the advised methods must not be `final`. Spring Boot defaults to CGLIB (`spring.aop.proxy-target-class=true`).
9. **Why does `@Transactional` not work on a self-invoked method?** The advice is applied by the proxy. A call through `this` goes directly to the target object and bypasses `TransactionInterceptor`. The same holds for `@Async`, `@Cacheable` and `@PreAuthorize`. Fix by moving the method to another bean or by injecting the proxy.

10. **What does `@Configuration` do differently from `@Component`?** A `@Configuration` class is CGLIB-proxied so that calls between its `@Bean` methods return the container's singleton instead of creating a new instance (full mode). With `proxyBeanMethods = false` (lite mode) the proxy is skipped, which speeds start-up but changes inter-bean method call behaviour.
11. **What is in `@SpringBootApplication`?** `@SpringBootConfiguration` (a `@Configuration`), `@EnableAutoConfiguration` (imports the candidates listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`), and `@ComponentScan` of the package of the annotated class and its sub-packages.
12. **How does auto-configuration decide what to create?** Each auto-configuration class carries conditions: `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty` and others. A user-defined bean of the same type makes `@ConditionalOnMissingBean` back off. `--debug` prints the condition evaluation report.
13. **How is externalised configuration ordered?** Later sources win: default properties, `application.properties|yml` (profile-specific files over general ones), OS environment variables, Java system properties, command-line arguments (highest of the common ones). `@ConfigurationProperties` binds with relaxed rules (kebab, camel and upper-snake forms).
14. **How is a custom starter built?** An auto-configuration module with `@AutoConfiguration` classes, conditions and `@ConfigurationProperties`, registered in the `.imports` file, plus a starter module that depends on it.

6.7.2 Spring MVC and HTTP clients

1. **Describe the lifecycle of a request in Spring MVC.** The servlet container passes the request through filters to `DispatcherServlet`. `HandlerMapping` (`RequestMappingHandlerMapping`) selects a handler, `HandlerAdapter` invokes it after argument resolution and `HttpMessageConverter` reading, the return value is written by a converter or rendered as a view, and interceptors run before and after.
2. **Filter versus `HandlerInterceptor`?** A filter belongs to the servlet container, wraps the whole chain including static resources and is independent of Spring MVC. An interceptor is part of MVC, runs around handler execution, and knows the matched handler.
3. **`@Controller` versus `@RestController`?** `@RestController` is `@Controller` plus `@ResponseBody`: return values are written through message converters instead of being resolved as view names.
4. **What causes 415, 406 and 400?** 415: no converter for the request `Content-Type`. 406: no converter satisfies the `Accept` header. 400: malformed JSON (`HttpMessageNotReadableException`), failed validation (`MethodArgumentNotValidException`) or a missing parameter.
5. **How is global error handling done?** `@RestControllerAdvice` with `@ExceptionHandler` methods; Spring 6 supports `ProblemDetail` (RFC 9457) responses, enabled for built-in exceptions with `spring.mvc.problemdetails.enabled=true`.
6. **What does `@Valid` do and what is `@Validated`?** `@Valid` triggers Bean Validation of the argument, producing `MethodArgumentNotValidException`. `@Validated` on a class enables method-level validation of parameters (constraint violations raise `ConstraintViolationException`) and supports validation groups.
7. **How many requests can a Boot web application serve concurrently?** With Tomcat the pool is `server.tomcat.threads.max` (default 200), one thread per request; beyond that requests wait in `accept-count` (100) and `max-connections` (8192 for NIO). With `spring.threads.virtual.enabled=true` on JDK 21 (Boot 3.2 and later), request handling uses virtual threads and the pool is no longer the limit; downstream pools become the limit.
8. **How do CORS preflights work?** A browser sends `OPTIONS` with `Origin` and `Access-Control-Request-Method` before a non-simple request; the server must answer with `Access-Control-Allow-*` headers. With Spring Security the CORS configuration must be applied before authorisation (`http.cors(...)`), otherwise the preflight is rejected with 401 or 403.
9. **How is pagination done in Spring Data and MVC?** A controller parameter of type `Pageable` is resolved from `page`, `size` and `sort`; the repository returns `Page<T>` (with a count query) or `Slice<T>` (without).
10. **`RestTemplate`, `WebClient`, `RestClient` and HTTP interfaces?** `RestTemplate` is blocking and in maintenance mode. `WebClient` is reactive and non-blocking. `RestClient` (Spring 6.1 and Boot 3.2) is a blocking fluent client that replaces `RestTemplate` in new code. `@HttpExchange` interfaces generate a client from an annotated interface on top of `RestClient` or `WebClient`.
11. **Which timeouts must every HTTP client set?** Connect timeout and read timeout (and a pool acquisition timeout). Without them a slow dependency holds threads indefinitely.
12. **What is `ETag` support in Spring MVC?** `ShallowEtagHeaderFilter` computes a hash of the response body and returns 304 for a matching `If-None-Match`; it saves bandwidth, not server work.

6.7.3 JPA and Hibernate

1. **What is the persistence context?** The first-level cache and unit of work of an `EntityManager`: it tracks managed entities by identity, detects changes (dirty checking) at flush time and guarantees one instance per row per context.
2. **What are the entity states?** Transient (new), managed (attached), detached (was managed, context closed) and removed. `persist`, `merge`, `remove` and `detach` move between them.
3. **When does Hibernate flush?** Before a query that could be affected (`FlushModeType.AUTO`), at transaction commit, and on explicit `flush()`.
4. **What is the N+1 problem and how is it fixed?** One query loads N parents and a further N queries load each lazy association. Fix with `JOIN FETCH`, `@EntityGraph`, `@BatchSize`/`default_batch_fetch_size`, or a DTO projection.
5. `LAZY` **versus** `EAGER`**?** `LAZY` loads on first access through a proxy or collection wrapper; `EAGER` loads immediately and is a fixed global plan that is usually wrong. Associations should be `LAZY` and fetched per use case.
6. **What causes** `LazyInitializationException`**?** Access to an uninitialised proxy or collection after the persistence context was closed. Fetch what the view needs inside the transaction, or map to DTOs there; open-in-view only hides it.
7. **What is open-in-view?** `OpenEntityManagerInViewInterceptor` keeps the `EntityManager` open for the whole web request (default `spring.jpa.open-in-view=true`, with a warning at start-up). It allows lazy loading in controllers and serialisation, and holds a database connection for the entire request.
8. `MultipleBagFetchException`**?** Fetching two `List` collections (bags) in one query is rejected because of the Cartesian product. Use `Set`, fetch one collection per query, or batch fetching.
9. **Why is paging with a collection fetch dangerous?** Hibernate cannot apply `LIMIT` to a joined collection and paginates in memory (warning HHH000104). Page the root ids first, then fetch the associations for those ids.
10. **How is optimistic locking implemented?** A `@Version` attribute is included in the `UPDATE ... WHERE id = ? AND version = ?`; zero updated rows raises `ObjectOptimisticLockingFailureException`.
11. `IDENTITY` **or** `SEQUENCE` **id generation?** `IDENTITY` forces an immediate insert to obtain the key, which disables JDBC insert batching. `SEQUENCE` with an allocation size permits batching.
12. **What must** `equals` **and** `hashCode` **of an entity respect?** They must be stable across the entity lifecycle: use a business key or a database-independent identifier assigned at construction; an id generated at persist time changes `hashCode` of an object already stored in a `HashSet`.
13. `save` **in** `CrudRepository`**: persist or merge?** `SimpleJpaRepository.save` calls `persist` for a new entity (`isNew`, determined by a null id or version) and `merge` otherwise; `merge` issues a `SELECT` first and copies state onto a managed instance.
14. **What is a derived query versus** `@Query`**?** Derived queries are parsed from the method name (`findByStatusAndCreatedAtAfter`); `@Query` takes JPQL or native SQL and is clearer for anything beyond two predicates.

6.7.4 Transactions

1. **What does** `@Transactional` **do?** A proxy with `TransactionInterceptor` obtains a `PlatformTransactionManager`, begins a transaction according to the propagation, invokes the method, commits, or rolls back on a `RuntimeException` or `Error`.
2. **Which exceptions roll back by default?** Unchecked exceptions and errors. Checked exceptions commit unless `rollbackFor` is specified.
3. **Explain the propagation types.** `REQUIRED` (join or create, default), `REQUIRES_NEW` (suspend and start independent), `NESTED` (savepoint), `SUPPORTS`, `NOT_SUPPORTED`, `MANDATORY`, `NEVER`.
4. **What does** `readOnly = true` **do?** It is a hint: Hibernate skips dirty checking (flush mode `MANUAL`) and some drivers or routing data sources choose a read replica. It does not prevent writes at database level on its own.
5. **What are the isolation levels and anomalies?** `READ_UNCOMMITTED`, `READ_COMMITTED` (PostgreSQL default), `REPEATABLE_READ`, `SERIALIZABLE`. They prevent, progressively, dirty reads, non-repeatable reads and phantoms; PostgreSQL implements snapshot isolation at `REPEATABLE_READ` and may abort with a serialisation failure that must be retried.
6. **How is a lost update prevented?** Optimistic locking with `@Version`, pessimistic locking (`SELECT ... FOR UPDATE`, `LockModeType.PESSIMISTIC_WRITE`), or an atomic update statement.
7. **Why does a long transaction exhaust the Hikari pool?** The connection is held from the first statement to commit; a transaction that waits on an HTTP call keeps the connection idle-in-transaction. Keep remote calls outside transactions.

8. **How is work run after a successful commit?** `@TransactionalEventListener(phase = AFTER_COMMIT)` or `TransactionSynchronizationManager.registerSynchronization`.
9. **How should deadlocks be handled?** Acquire locks in a consistent order, keep transactions short, and retry the transaction on `CannotAcquireLockException` or `DeadlockLoserDataAccessException`.
10. **What is `TransactionTemplate` for?** Programmatic transaction boundaries inside a method, where a declarative annotation does not fit (self-invocation, several transactions in one flow).

6.7.5 Security, JWT and OAuth 2.0

1. **How does Spring Security plug into the servlet stack?** `DelegatingFilterProxy` (registered by Boot under the bean name `springSecurityFilterChain`) delegates to `FilterChainProxy`, which selects the first matching `SecurityFilterChain` and runs its ordered filters.
2. **What is `SecurityContextHolder`?** A holder, thread-local by default, of the `SecurityContext` that contains the `Authentication` of the current request; it is cleared at the end of each request.
3. **How does authentication proceed?** A filter creates an unauthenticated `Authentication`; `AuthenticationManager` (`ProviderManager`) asks `AuthenticationProvider`s, for instance `DaoAuthenticationProvider` with a `UserDetailsService` and `PasswordEncoder`; the result is stored in the context.
4. **401 versus 403?** 401 means the request is not authenticated (or the credentials are invalid) and is produced by the `AuthenticationEntryPoint`. 403 means the authenticated principal lacks permission and is produced by the `AccessDeniedHandler`.
5. **`hasRole("ADMIN")` versus `hasAuthority("ROLE_ADMIN")`?** They are equivalent: `hasRole` adds the `ROLE_` prefix. Mismatches arise when the stored authorities lack the prefix.
6. **Why `BCrypt`/`Argon2` and not `SHA-256` for passwords?** Password hashing must be slow and salted so that brute force is expensive; fast hashes are cheap to attack on GPUs. `DelegatingPasswordEncoder` stores an `{id}` prefix and allows migration of algorithms.
7. **When is CSRF protection needed?** When authentication is carried automatically by the browser (session cookies). A stateless API with an `Authorization` header token is not exposed in the same way, and CSRF can be disabled for it.
8. **Session or token authentication?** Sessions are stateful and revocable, and need shared storage (Spring Session with Redis) when scaled. Tokens are stateless and scale easily, but are hard to revoke before expiry.
9. **Structure of a JWT?** `base64url(header).base64url(payload).base64url(signature)`. HS256 signs with a shared secret (HMAC), RS256 and ES256 with a private key and verify with a public key.
10. **What must a JWT verifier check?** Signature with the expected algorithm (never trust the `alg` header: `none` and algorithm confusion attacks), `exp`, `nbf`, `iss`, `aud`, and a small clock skew allowance.
11. **How are refresh tokens secured?** Long-lived, stored hashed server-side, rotated on every use with reuse detection (reuse of an old token revokes the family), delivered in an `HttpOnly`, `Secure`, `SameSite` cookie.
12. **Where should the access token live in a browser?** An `HttpOnly` cookie is not readable by scripts (XSS cannot steal it, but CSRF must be handled); `localStorage` is readable by any script on the page.
13. **Explain the authorisation code flow with PKCE.** The client sends the user to the authorisation server with a `code_challenge`; after login it receives a code at the redirect URI and exchanges it together with the `code_verifier` for tokens. PKCE binds the code to the client instance and protects public clients.
14. **Resource server versus client?** A resource server validates access tokens and serves an API (`oauth2ResourceServer().jwt()`); a client obtains tokens on behalf of a user (`oauth2Login`, `oauth2Client`).
15. **Why does `@PreAuthorize` not apply?** `@EnableMethodSecurity` is missing, or the call is a self-invocation, or the class is not a proxied bean.

6.7.6 Microservices and resilience

1. **When should a system not be split into microservices?** When the team is small, the domain boundaries are unclear, or operational maturity (CI/CD, observability, on-call) is missing. A modular monolith keeps boundaries without network failure modes.
2. **Why database per service?** Services own their data to deploy and evolve independently; sharing a schema couples them through the database. Cross-service consistency then needs sagas and events instead of joins and transactions.

3. **What is a saga?** A sequence of local transactions where each step publishes an event or command, and failures trigger compensating transactions. Choreography has services react to events; orchestration has a coordinator drive the steps.
4. **Why not two-phase commit?** It blocks resources while the coordinator decides, reduces availability, and is not supported across heterogeneous systems such as a database and Kafka.
5. **Explain the circuit breaker states.** Closed (calls pass, failures counted), open (calls fail fast for a wait duration), half-open (a limited number of trial calls decide whether to close or reopen). Resilience4j's `CircuitBreaker` opens when the failure rate exceeds a threshold over a sliding window.
6. **Why must retries use backoff and jitter?** Immediate retries multiply the load on a struggling dependency (a retry storm). Exponential backoff spaces attempts and jitter prevents synchronised retry waves.
7. **When is a retry safe?** When the operation is idempotent or carries an idempotency key, and the error is transient. A non-idempotent `POST` retried without a key can duplicate side effects.
8. **What is a bulkhead?** Isolation of resource pools (threads or semaphore permits) per dependency, so that one slow dependency cannot consume all capacity.
9. **What is the order of Resilience4j decorators?** By default, from outside to inside: `Retry`, `CircuitBreaker`, `RateLimiter`, `TimeLimiter`, `Bulkhead` (as aspect order `Retry ( CircuitBreaker ( RateLimiter ( TimeLimiter ( Bulkhead ( Function ) ) ) ) )`); each can be reordered through aspect order properties.
10. **What does an API gateway do?** Single entry point: routing, authentication, rate limiting, TLS termination, request and response transformation, aggregated cross-cutting concerns. Spring Cloud Gateway is reactive, based on routes, predicates and filters.
11. **Client-side versus server-side load balancing?** The client holds the instance list and chooses (Spring Cloud LoadBalancer); a server-side balancer (Kubernetes Service, a cloud load balancer) chooses for the client. In Kubernetes the Service DNS name is usually sufficient.
12. **How is a trace followed across services?** A trace id and span id are propagated in the W3C `traceparent` header; Micrometer Tracing adds them to the MDC for log correlation and exports spans to a backend. `@Async` and thread pools need context propagation.
13. **What is the transactional outbox and when is it used?** An outbox table written in the business transaction and relayed to the broker; it solves the dual-write problem with at-least-once delivery.
14. **What is CQRS?** Separate models for commands (writes) and queries (reads), possibly with separate stores updated from events; it adds eventual consistency and complexity and fits read-heavy or complex-reporting cases only.

6.7.7 Kafka, messaging and caching

1. **Queue versus log?** A queue deletes messages after acknowledgement and tracks delivery per message; a log retains records, and consumers track offsets, which allows replay and multiple groups.
2. **How is parallelism achieved in Kafka?** By partitions: each partition is consumed by one member of a group, so the number of partitions bounds consumer parallelism.
3. **Why can messages be processed twice, and what is the remedy?** Offset commit and processing are separate steps. Idempotent consumers (unique event id, upsert) make duplicates harmless.
4. **What guarantees does `acks=all` give?** The leader acknowledges after all in-sync replicas have the record; with `min.insync.replicas=2` and replication factor 3, an acknowledged write survives the loss of one broker.
5. **What is a rebalance and why is it costly?** Reassignment of partitions after membership change; with the eager protocol all consumers stop. Cooperative assignors, static membership and tuned timeouts reduce the impact.
6. **How does `DefaultErrorHandler` retry?** It seeks back to the failed offset and redelivers according to a `BackOff` (default ten attempts without delay), then calls a recoverer such as `DeadLetterPublishingRecoverer`.
7. **When are retry topics preferable to blocking retries?** When failures are transient, order is not required, and stalling a partition is unacceptable; the price is lost ordering and more topics.
8. **What does Kafka exactly-once cover?** Idempotent producer plus transactions with offsets committed in the transaction (consume-transform-produce within Kafka). External effects need idempotence.
9. **RabbitMQ: what are exchanges for?** Routing: producers publish to exchanges, which distribute messages to queues by binding rules (`direct`, `topic`, `fanout`, `headers`).
10. **Why is `@Scheduled` unsafe in a cluster?** Every instance runs it. Use ShedLock, a Kubernetes `CronJob`, or Quartz with a clustered JDBC store.

11. **What is a cache stampede?** Many concurrent misses of one expired key reaching the backend. Single flight (`sync = true`), TTL jitter and background refresh mitigate it.
12. **How is cache consistency kept with the database?** Evict after commit, use TTLs as a safety net, route all writes through one component, and accept bounded staleness explicitly.

6.7.8 Testing, Actuator and performance

1. **Test pyramid in a Spring project?** Many fast unit tests without a context, fewer slice tests (`@WebMvcTest`, `@DataJpaTest`), few full `@SpringBootTest` integration tests with real infrastructure through Testcontainers.
2. `@WebMvcTest` **versus** `@SpringBootTest`**?** `@WebMvcTest` loads only MVC infrastructure and the named controllers, with collaborators replaced by `@MockitoBean` (Boot 3.4; `@MockBean` before). `@SpringBootTest` loads the full context.
3. **Why are Spring tests slow?** Each distinct configuration (different mocks, properties, `@DirtiesContext`) creates a new application context that the test context cache cannot share. Standardise the configuration and avoid `@DirtiesContext`.
4. **Why Testcontainers instead of H2?** H2 differs in SQL dialect, locking, types and isolation, so tests pass on H2 and fail on PostgreSQL. Testcontainers runs the real database; `@ServiceConnection` wires its properties automatically.
5. **What are Actuator health groups?** Named subsets of health indicators, including `liveness` and `readiness` used by Kubernetes probes; groups are configured under `management.endpoint.health.group.*`.
6. **How are Actuator endpoints secured?** Expose only the required endpoints (`management.endpoints.web.exposure.include`), put them on a separate management port or behind security rules, and never expose `env`, `heapdump` or `threaddump` publicly.
7. **What is metric cardinality and why does it matter?** The number of distinct tag combinations. A tag with unbounded values (user id, URL with path variables) creates a time series per value and exhausts the monitoring system.
8. **RED and USE methods?** RED (rate, errors, duration) for services; USE (utilisation, saturation, errors) for resources such as thread pools and connection pools.
9. **How is a slow endpoint investigated?** Check metrics (`http.server.requests` percentiles), traces to locate the slow span, thread dumps (`jcmd <pid> Thread.print`) for blocking, a JFR or async-profiler recording for CPU, and SQL logs for the database.
10. **How does Hikari pool sizing work?** A small pool (often `cores * 2 + effective spindles` as a starting point) with a queue is faster than a large one; `maximumPoolSize` and `connectionTimeout` are tuned against database capacity and transaction length.
11. **Platform threads versus virtual threads for MVC?** Virtual threads (JDK 21, `spring.threads.virtual.enabled=true`) remove the thread-per-request limit for blocking I/O; CPU-bound work, pinned `synchronized` sections and limited downstream pools remain constraints.
12. **How is start-up time reduced?** Lazy initialisation, fewer auto-configurations and classpath scanning, Class Data Sharing or the AOT cache, and GraalVM native image for the extreme case at the cost of build time and reflection configuration.

6.7.9 Docker, Kubernetes and CI/CD

1. **What is a layered jar?** An executable jar with a `layers.idx` that groups files into dependency and application layers, extracted into separate Docker layers for cache efficiency.
2. **How is memory sized for a JVM in a container?** `-XX:MaxRAMPercentage` for the heap, with headroom for metaspace, thread stacks, direct memory and native memory; exceeding the container limit is `OOMKilled` (exit code 137).
3. **How does graceful shutdown work end to end?** Endpoint removal and a `preStop` sleep, `SIGTERM`, readiness down, `server.shutdown=graceful` drains active requests within the timeout, the context closes, and the grace period must exceed all of it.
4. **Liveness versus readiness?** Liveness restarts a broken container; readiness controls traffic. External dependencies belong in readiness at most.
5. **What gates a rolling update?** The readiness probe, together with `maxSurge` and `maxUnavailable`.
6. **How does the Horizontal Pod Autoscaler work?** It scales replicas to keep an average metric (CPU utilisation relative to requests, or custom metrics) at the target, between minimum and maximum.

7. **ConfigMap versus Secret?** Both inject configuration; Secrets are for sensitive values, base64-encoded in the API and protected by RBAC and optional encryption at rest.
8. **How are environment variables mapped to Spring properties?** By relaxed binding: `SPRING_DATASOURCE_URL` maps to `spring.datasource.url`.
9. **What does a CI pipeline for a Spring Boot service contain?** Checkout, JDK setup with dependency cache, `mvn verify` (unit and Testcontainers integration tests), static and dependency scanning, image build and push with an immutable tag, deployment to staging, approval, production.
10. **Surefire versus Failsafe?** Unit tests in `test` versus integration tests in `integration-test` with verification in `verify`.
11. **Blue-green versus canary versus rolling?** Full environment switch with instant rollback; progressive traffic shift with metric analysis; gradual pod replacement in place.
12. **What does a secret leak response consist of?** Revoke and rotate immediately, assess usage in audit logs, remove the secret from the repository and history, and add preventive scanning.
13. **How are database migrations kept compatible with rolling updates?** Expand-and-contract: additive changes first, code migration second, removal in a later release.
14. **How are Docker Compose and Testcontainers used by Spring Boot?** `spring-boot-docker-compose` starts the compose services for development and derives connection details; Testcontainers with `@ServiceConnection` does the same for tests.

KEY POINTS

Strong answers start from the mechanism: the real class, the order of steps, and the property that controls it.

- Group questions by failure mode: proxy bypass, lazy loading, transaction boundaries, token verification, retries and idempotency are asked in many forms.
- Every answer should end with the trade-off or the pitfall, which is what distinguishes experience from recitation.
- Practise answering aloud against the clock: two minutes for the mechanism, one minute for the pitfall.

6.8 System Design Scenarios for Spring Services

A system design interview for a Java role asks for a service that works, with numbers, a data model, an API, failure handling and the Spring components that implement it. The method is the same for each scenario: state functional and non-functional requirements, estimate load, define the API and the data model, draw the components, then examine bottlenecks and failure modes. The three scenarios below use the techniques of the whole guide: caching, idempotency, the outbox, rate limiting and resilience. Estimates are order-of-magnitude assumptions, stated as such, and not measurements.

6.8.1 Scenario 1: a URL shortener

Requirements. Create a short code for a long URL (`POST /urls`), redirect from `GET /{code}`. Redirects must be fast (p99 under 20 ms) and highly available; creation may be slower. Codes are unique; optional custom aliases and expiry.

Estimates (assumed). 100 million new URLs per year, a read to write ratio of 100:1, so about 10 billion redirects per year, an average of 320 per second and a peak of about 10 000 per second. A record of code, long URL and metadata takes about 500 bytes, which gives roughly 50 GB per year.

Code generation. A 7-character code over a 62-symbol alphabet (digits, upper and lower case letters) gives 62^7 = 3 521 614 606 208 codes, enough for 35 000 years at the assumed rate. The code is the base-62 encoding of a unique number. The following program shows the encoding and decoding, run on the build machine.

Base62.java JAVA

```java
public class Base62 {
    private static final String ALPHABET =
        "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";

    static String encode(long id) {
        if (id == 0) return "0";
        StringBuilder sb = new StringBuilder();
        while (id > 0) {
            sb.append(ALPHABET.charAt((int) (id % 62)));
            id /= 62;
        }
```

Base62.java (continued) JAVA

```java
        return sb.reverse().toString();
    }

    static long decode(String code) {
        long id = 0;
        for (char c : code.toCharArray()) id = id * 62 + ALPHABET.indexOf(c);
        return id;
    }

    public static void main(String[] args) {
        long[] ids = {1, 61, 62, 125, 1_000_000L, 3_521_614_606_207L};
        for (long id : ids) {
            String code = encode(id);
            System.out.printf("id=%-16d code=%-8s decoded=%d%n", id, code, decode(code));
        }
        System.out.println("62^7 = " + (long) Math.pow(62, 7) + " codes of length 7");
    }
}
```

Output of the base-62 encoder (run on the build machine) TERMINAL

```
$ java Base62.java
id=1                code=1        decoded=1
id=61               code=z        decoded=61
id=62               code=10       decoded=62
id=125              code=21       decoded=125
id=1000000          code=4C92     decoded=1000000
id=3521614606207    code=zzzzzzz  decoded=3521614606207
62^7 = 3521614606208 codes of length 7
```

A database sequence that is incremented per URL yields unique numbers without coordination, but sequential codes are guessable, which leaks the creation rate and allows enumeration. Remedies: apply a bijective scrambling of the number (a multiplication modulo a prime near 62^7, or a block cipher on the id), or draw ids from a generator that mixes time and random bits, with a unique index as the final arbiter and a retry on conflict. A hash of the URL (truncated) is not unique and needs collision handling.

Components.

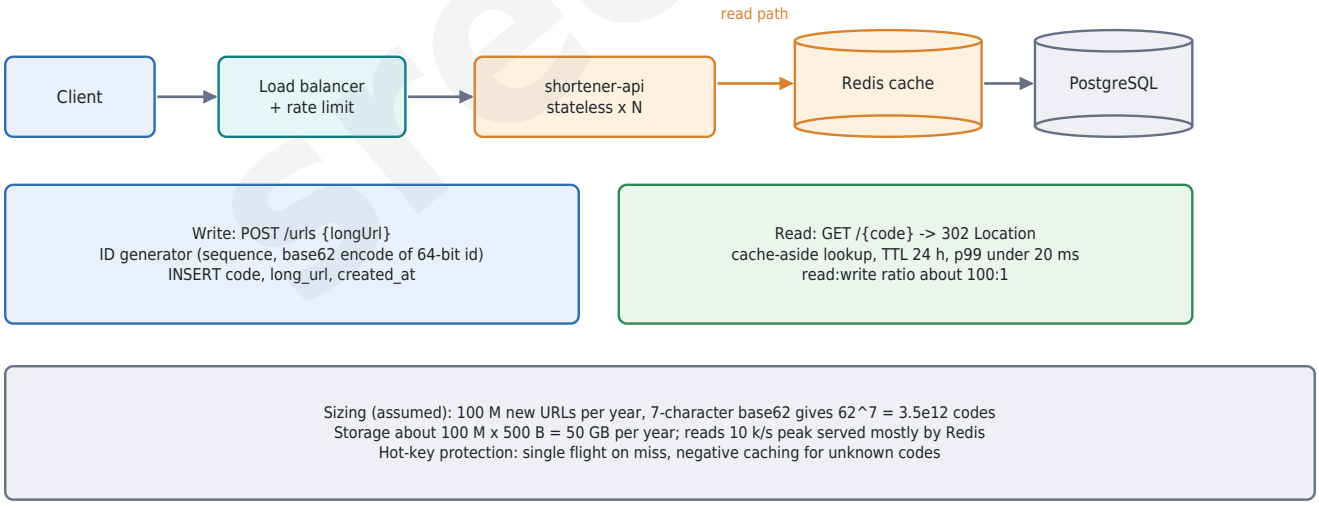


Figure 6.20. Stateless API instances read through Redis with the database as the system of record; writes obtain an id, encode it and insert one row.

Data model. One table `short_url(code varchar(10) primary key, long_url text not null, created_at timestamptz, expires_at timestamptz, owner_id bigint)`. The primary key lookup is the only hot query, so a relational database suffices at this scale; sharding by `code` hash becomes relevant above a few terabytes or write rates the primary cannot absorb.

Redirect path. `GET /{code}` is cache-aside: Redis lookup, on a miss a primary-key read and a write-back with a TTL of 24 hours. The response is `302 Found` with `Location` when click analytics matter (a `301` is cached by browsers and hides

later requests from the service). Unknown codes are cached negatively for a short time to protect the database from scans.

RedirectController.java JAVA

```java
package com.example.shortener.api;

import com.example.shortener.service.UrlService;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import java.net.URI;

@RestController
public class RedirectController {

    private final UrlService service;

    public RedirectController(UrlService service) {
        this.service = service;
    }

    @GetMapping("/{code:[0-9A-Za-z]{1,10}}")
    public ResponseEntity<Void> redirect(@PathVariable String code) {
        return service.resolve(code)
                .map(url -> ResponseEntity.status(HttpStatus.FOUND)
                        .header(HttpHeaders.CACHE_CONTROL, "no-store")
                        .location(URI.create(url)).<Void>build())
                .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

The regular expression in the mapping rejects malformed codes before any lookup. `UrlService.resolve` is annotated `@Cacheable(cacheNames = "urls", sync = true)` and returns an `Optional<String>` (Spring caches the unwrapped value and handles `Optional` specially, so a missing value is not stored unless null caching is enabled).

Bottlenecks and failure modes. A viral link is one hot key: the cache absorbs it, and a local in-process cache (Caffeine, a few seconds) in front of Redis removes the network hop. Redis loss means a cold start with a stampede on the database: single flight and a connection-pool limit protect it. Analytics (click counts) must not be a synchronous database write per redirect; the service publishes a click event to Kafka or increments a Redis counter and aggregates asynchronously. Expired links are removed by a scheduled job that is made cluster-safe (see the scheduling chapter).

6.8.2 Scenario 2: an order service

Requirements. Customers place orders for several items; payment and stock reservation must both succeed or the order is cancelled; no double charge when a client retries; order status is visible; peak 500 orders per second during sales.

Decomposition. `order-service` (owns orders), `payment-service`, `inventory-service` and `notification-service`, each with its own database. Communication for the order flow is event-driven with a saga.

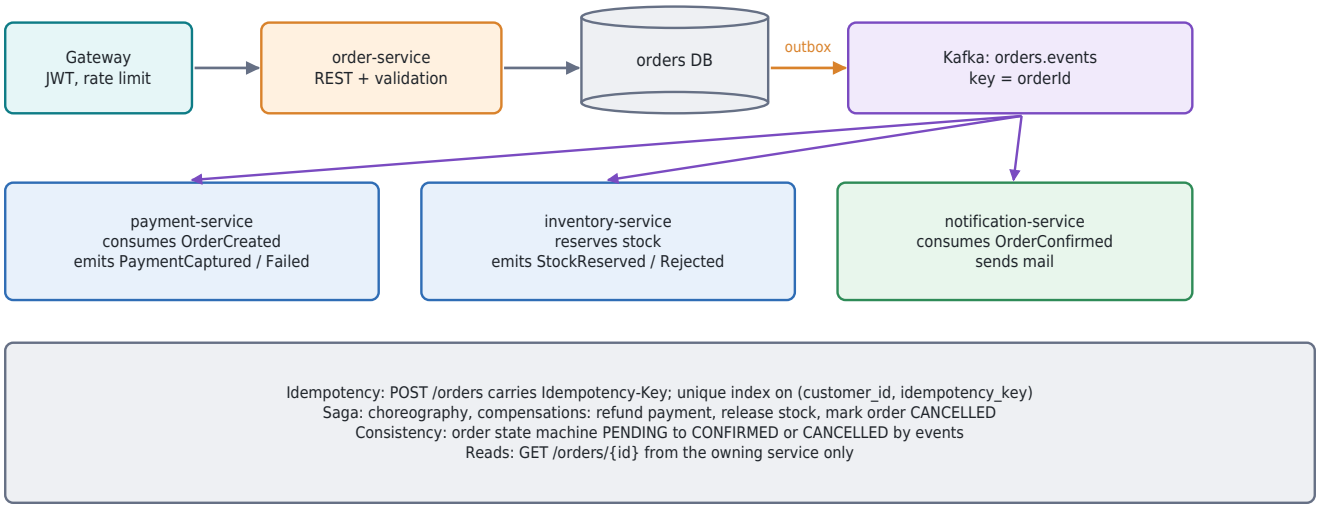


Figure 6.21. The order service persists the order and an outbox row atomically; payment and inventory react to the event, and their outcomes drive the order state machine.

API and idempotency. `POST /orders` requires an `Idempotency-Key` header chosen by the client (a UUID). The service stores the key with the order under a unique constraint on `(customer_id, idempotency_key)`. A retry with the same key returns the original response instead of creating a second order. This turns client and gateway retries, which are inevitable, into a safe operation.

OrderStatus.java JAVA

```java
package com.example.orders.domain;

import java.util.EnumSet;
import java.util.Set;

public enum OrderStatus {
    PENDING, CONFIRMED, CANCELLED, SHIPPED;

    private static final java.util.Map<OrderStatus, Set<OrderStatus>> ALLOWED = java.util.Map.of(
            PENDING, EnumSet.of(CONFIRMED, CANCELLED),
            CONFIRMED, EnumSet.of(SHIPPED, CANCELLED),
            CANCELLED, EnumSet.noneOf(OrderStatus.class),
            SHIPPED, EnumSet.noneOf(OrderStatus.class));

    public boolean canMoveTo(OrderStatus next) {
        return ALLOWED.get(this).contains(next);
    }
}
```

The transition table is what makes event handling safe against reordering and duplicates: an event that asks for an illegal transition (`PaymentFailed` after `CONFIRMED`) is ignored or routed to review. The aggregate carries a `@Version` field so that two events processed concurrently cannot both succeed.

Saga. `OrderCreated` is published through the outbox. `inventory-service` reserves stock and emits `StockReserved` or `StockRejected`; `payment-service` authorises payment on `StockReserved` and emits `PaymentCaptured` or `PaymentFailed`. The order service moves to `CONFIRMED` on `PaymentCaptured`. On `PaymentFailed` it moves to `CANCELLED` and publishes `OrderCancelled`, which makes inventory release the reservation (the compensation). Every consumer is idempotent by event id. This is choreography; with more steps an orchestrator (a state machine in `order-service`) is easier to reason about and to monitor.

Resilience. The gateway applies a timeout and a rate limit. Calls to the payment provider (the one synchronous dependency) use a timeout, a circuit breaker and a bounded retry with backoff and an idempotency key at the provider. A stuck saga (no event after five minutes) is detected by a scheduled query and escalated or compensated.

Reads. `GET /orders/{id}` is answered by `order-service` from its own database. A customer-facing view that joins order, payment and shipment data is built from events into a read model, so that no request fans out to three services.

Bottlenecks. At 500 orders per second the order table insert rate is modest; the limiting factors are the payment provider, Kafka partitioning (key by `orderId` for per-order ordering, enough partitions for consumer parallelism) and database connection pools. Outbox polling is replaced by change data capture if latency or load matters.

6.8.3 Scenario 3: a rate-limited public API

Requirements. Limit each API key to 10 requests per second with a burst of 20; reject excess with `429 Too Many Requests`; the limit must hold across all instances; the limiter must not become a single point of failure; limits differ by plan.

Algorithms. Fixed window counters are simple but allow double the rate at window boundaries. Sliding windows are accurate and cost more state. The token bucket permits bursts up to its capacity and refills at a constant rate, which matches API semantics: capacity 20, refill 10 tokens per second. In a distributed setting the bucket state (tokens and last refill time) lives in Redis and is updated atomically by a Lua script, since a read-modify-write from several instances would race.

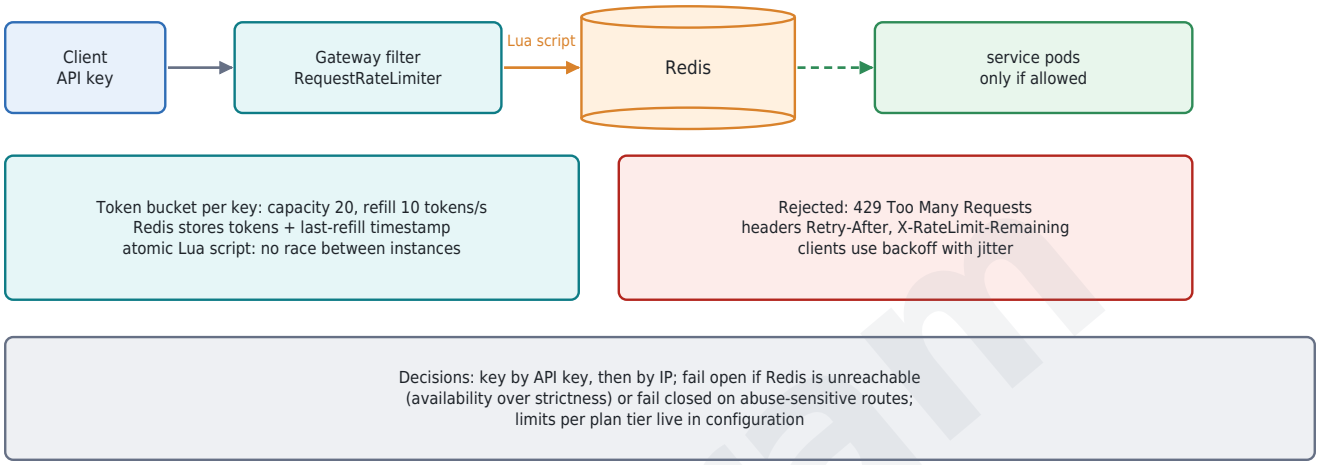


Figure 6.22. The gateway filter consults an atomic token bucket in Redis before a request reaches the service, and rejected requests receive 429 with a retry hint.

Spring Cloud Gateway implements this with the `RequestRateLimiter` filter and `RedisRateLimiter`:

application.yml (gateway) YAML

```yaml
spring:
  data:
    redis:
      host: redis
      timeout: 200ms
  cloud:
    gateway:
      routes:
        - id: public-api
          uri: lb://order-service
          predicates:
            - Path=/api/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
                redis-rate-limiter.requestedTokens: 1
                key-resolver: "#{@apiKeyResolver}"
                deny-empty-key: true
                empty-key-status-code: 401
```

RateLimitConfig.java JAVA

```java
package com.example.gateway.config;

import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import reactor.core.publisher.Mono;

@Configuration
public class RateLimitConfig {

    @Bean
```

sreeram

RateLimitConfig.java (continued) JAVA

```
    KeyResolver apiKeyResolver() {
        return exchange -> Mono.justOrEmpty(
                exchange.getRequest().getHeaders().getFirst("X-API-Key"));
    }
}
```

What to notice: the key resolver decides what is limited (here the API key; an unauthenticated route would use the client address, which is unreliable behind proxies unless `X-Forwarded-For` is configured and trusted); `RedisRateLimiter` rejects with status 429 and adds `X-RateLimit-Remaining`, `X-RateLimit-Burst-Capacity`, `X-RateLimit-Replenish-Rate` and `X-RateLimit-Requested-Tokens` headers; and per-plan limits require a custom `RateLimiter` bean or one route per plan, because the `RedisRateLimiter` configuration is per route.

Failure behaviour. When Redis is unreachable the Spring Cloud Gateway Redis limiter logs and allows the request (fail open), which favours availability; abuse-sensitive routes may need fail-closed behaviour through a custom limiter. Clients are expected to honour `Retry-After` and use exponential backoff with jitter; otherwise rejected requests are retried immediately and the limiter amplifies the load it is protecting against.

Layers. The edge (CDN, load balancer) applies coarse per-IP limits against volumetric abuse; the gateway applies per-key quotas; the services use bulkheads for protection of their own resources. Each layer protects a different resource.

INTERVIEW ANGLE

How would the design change by a factor of ten in load? State the first component that saturates and the lever for it: stateless instances scale horizontally behind the load balancer; the cache absorbs reads, with replicas or local caches for hot keys; the database needs read replicas, then partitioning by a key chosen from the access pattern; Kafka needs more partitions and consumers; synchronous dependencies need bulkheads and degradation. Name the metric that shows saturation for each (CPU, pool wait time, cache hit ratio, consumer lag).

KEY POINTS

A design answer states requirements, estimates, API, data model, components and then bottlenecks and failure modes, in that order.

- Unique ids come from a sequence encoded in base 62; the code space is sized from the creation rate.
- Idempotency keys, a state machine with versioned aggregates and idempotent consumers make order flows safe under retries and reordering.
- A distributed token bucket needs atomic state in Redis; the failure policy (open or closed) is an explicit decision.

6.9 Cheat Sheets

The tables below condense the guide into reference form. Each row names the exact annotation, property or command; the mechanism behind a row is explained in the corresponding chapter. Values marked as defaults are those of Spring Boot 3.x and the Kafka clients at the time of writing and should be verified against the version in use.

6.9.1 Annotations

Annotation	Meaning
`@SpringBootApplication`	@SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan
`@Configuration, @Bean`	declare beans in Java; @Configuration classes are CGLIB-proxied unless proxyBeanMethods = false
`@Component, @Service, @Repository`	stereotypes picked up by component scanning; @Repository adds exception translation
`@Autowired, @Qualifier, @Primary`	injection point, disambiguation by name, preferred candidate
`@Value, @ConfigurationProperties`	single property placeholder, typed property binding with validation
`@Profile, @Conditional*`	register beans only for profiles or conditions
`@Scope, @Lazy`	bean scope; deferred creation or lazy resolution proxy
`@Transactional`	transaction boundary through a proxy (propagation, isolation, readOnly, rollbackFor)
`@Async, @Scheduled`	background execution; periodic execution (both per JVM)
`@Cacheable, @CachePut, @CacheEvict`	cache-aside through a proxy
`@RestController, @RequestMapping, @GetMapping`	web endpoints; @ResponseBody is implied by @RestController
`@PathVariable, @RequestParam, @RequestBody, @RequestHeader`	argument binding
`@Valid, @Validated`	Bean Validation of arguments; method validation and groups
`@RestControllerAdvice, @ExceptionHandler`	global error handling, ProblemDetail responses
`@Entity, @Id, @GeneratedValue, @Version`	JPA mapping, key generation, optimistic locking
`@OneToMany, @ManyToOne, @EntityGraph`	relationships and fetch plans
`@PreAuthorize, @EnableMethodSecurity`	method-level authorisation through a proxy
`@KafkaListener, @RetryableTopic, @DltHandler`	consumers, retry topics, dead-letter handler
`@RabbitListener`	AMQP consumer
`@SpringBootTest, @WebMvcTest, @DataJpaTest`	full context, MVC slice, JPA slice
`@MockitoBean, @ServiceConnection`	replace a bean in a test (Boot 3.4+); wire a Testcontainers service

Core Spring and Spring Boot annotations.

sreeram

6.9.2 application.yml essentials

application.yml YAML

```yaml
server:
  port: 8080
  shutdown: graceful
  tomcat:
    threads:
      max: 200
    accept-count: 100
spring:
  application:
    name: orders
  lifecycle:
    timeout-per-shutdown-phase: 25s
  threads:
    virtual:
      enabled: false              # true on JDK 21+ (Boot 3.2+) for virtual threads
  datasource:
    url: jdbc:postgresql://localhost:5432/orders
    username: orders
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 10
      connection-timeout: 3000
      max-lifetime: 1500000
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate
    properties:
      hibernate.default_batch_fetch_size: 32
      hibernate.jdbc.batch_size: 50
  flyway:
    enabled: true
  jackson:
    default-property-inclusion: non_null
  mvc:
    problemdetails:
      enabled: true
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      probes:
        enabled: true
      show-details: when-authorized
logging:
  level:
    root: INFO
    com.example: DEBUG
```

sreeram

6.9.3 Actuator endpoints

Endpoint	Purpose
`/actuator/health, /health/liveness, /health/readiness`	aggregate health and the Kubernetes probe groups
`/actuator/info`	build, git and custom information
`/actuator/metrics, /metrics/{name}?tag=k:v`	Micrometer metrics by name with tag filters
`/actuator/prometheus`	scrape endpoint in Prometheus text format
`/actuator/env, /configprops, /beans, /conditions`	effective configuration, bound properties, beans, auto-configuration report
`/actuator/mappings`	request mappings of all handlers
`/actuator/loggers`	read and change log levels at runtime (POST)
`/actuator/threaddump`	thread dump as JSON or text
`/actuator/heapdump`	heap dump download; never exposed publicly
`/actuator/httpexchanges`	recent HTTP exchanges (needs an HttpExchangeRepository bean)
`/actuator/scheduledtasks, /caches, /flyway`	scheduled tasks, caches (DELETE evicts), migrations

Frequently used Actuator endpoints.

6.9.4 Spring Security configuration skeleton

SecurityConfig.java JAVA

```java
package com.example.orders.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())                    // stateless token API
            .cors(Customizer.withDefaults())                 // before authorisation
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**", "/auth/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**")
                    .hasAuthority("SCOPE_catalog.read")
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()));
        return http.build();
    }
}
```

sreram

6.9.5 JPA fetching decision table

Situation	Technique	Caution
One entity with one needed association	JOIN FETCH or @EntityGraph	one row per child, duplicates need distinct
List of parents with a collection, no paging	JOIN FETCH on one collection	two List collections cause MultipleBagFetchException
List of parents with a collection, paged	page ids first, then fetch by id list	collection fetch with paging is done in memory
Many parents, associations used occasionally	@BatchSize or default_batch_fetch_size	N/batch size queries instead of N
Read-only view or report	DTO projection (interface, record, constructor expression)	no managed state, no dirty checking
Very large result	keyset pagination or streaming with a cursor	offset paging slows down with depth
Association always loaded	review the model	EAGER is a global plan and rarely correct

Choosing a fetching technique.

6.9.6 Resilience settings

Setting	Typical value	Effect
`circuitbreaker.instances.x.slidingWindowSize`	20	number of calls in the failure-rate window
`circuitbreaker.instances.x.failureRateThreshold`	50	percent of failures that opens the breaker
`circuitbreaker.instances.x.waitDurationInOpenState`	10s	time open before half-open trial calls
`circuitbreaker.instances.x.permittedNumberOfCallsInHalfOpenState`	3	trial calls deciding the next state
`circuitbreaker.instances.x.slowCallDurationThreshold`	2s	calls slower than this count as slow
`retry.instances.x.maxAttempts`	3	total attempts including the first
`retry.instances.x.waitDuration`	200ms	base wait; with enableExponentialBackoff and jitter
`bulkhead.instances.x.maxConcurrentCalls`	20	semaphore permits for concurrent calls
`timelimiter.instances.x.timeoutDuration`	2s	upper bound for asynchronous calls
`ratelimiter.instances.x.limitForPeriod`	50	permits per refresh period

Resilience4j settings that matter (Spring Boot property prefix resilience4j.).*

sreeram

6.9.7 Kafka settings

Setting	Default	Notes
acks	all (clients 3.0+)	durability of an acknowledged write
enable.idempotence	true (clients 3.0+)	per-session duplicate suppression
linger.ms, batch.size	0, 16384	batching versus latency
delivery.timeout.ms	120000	total time budget including retries
enable.auto.commit	true	set to false with Spring; the container commits
auto.offset.reset	latest	earliest for a new group that needs history
max.poll.records	500	records per poll; bounds work between polls
max.poll.interval.ms	300000	maximum time between polls before the member is removed
session.timeout.ms	45000	heartbeat loss detection (clients 3.0+)
isolation.level	read_uncommitted	read_committed to skip aborted transactions
partition.assignment.strategy	Range, CooperativeSticky	cooperative rebalancing
group.instance.id	none	static membership

Producer and consumer settings.

6.9.8 kubectl and Docker crib

Frequently used commands TERMINAL

```
$ kubectl get pods -n shop -o wide
$ kubectl describe pod orders-abc -n shop
$ kubectl logs deploy/orders -n shop --tail=100 -f
$ kubectl logs orders-abc -n shop --previous
$ kubectl exec -it orders-abc -n shop -- sh
$ kubectl rollout status deployment/orders -n shop
$ kubectl rollout undo deployment/orders -n shop
$ kubectl top pod -n shop
$ kubectl get events -n shop --sort-by=.lastTimestamp
$ kubectl port-forward svc/orders 8080:80 -n shop
$ docker build -t orders:dev .
$ docker run --rm -p 8080:8080 -m 768m orders:dev
$ docker logs -f orders
$ docker exec -it orders sh
$ docker compose up -d
$ docker history orders:dev
$ docker system df
```

6.10 Six-Week Study Plan

The plan maps the chapters of this guide to six weeks of about three and a half hours per working day. Each week ends with a build task that exercises the material and with the corresponding questions from the question bank, answered aloud. The schedule assumes familiarity with the basic Spring Boot guide; when a topic is shaky, the week is extended rather than skipped, because later weeks depend on the container, proxy and transaction mechanisms.

Six-week plan: each week ends with the matching interview questions and a build task

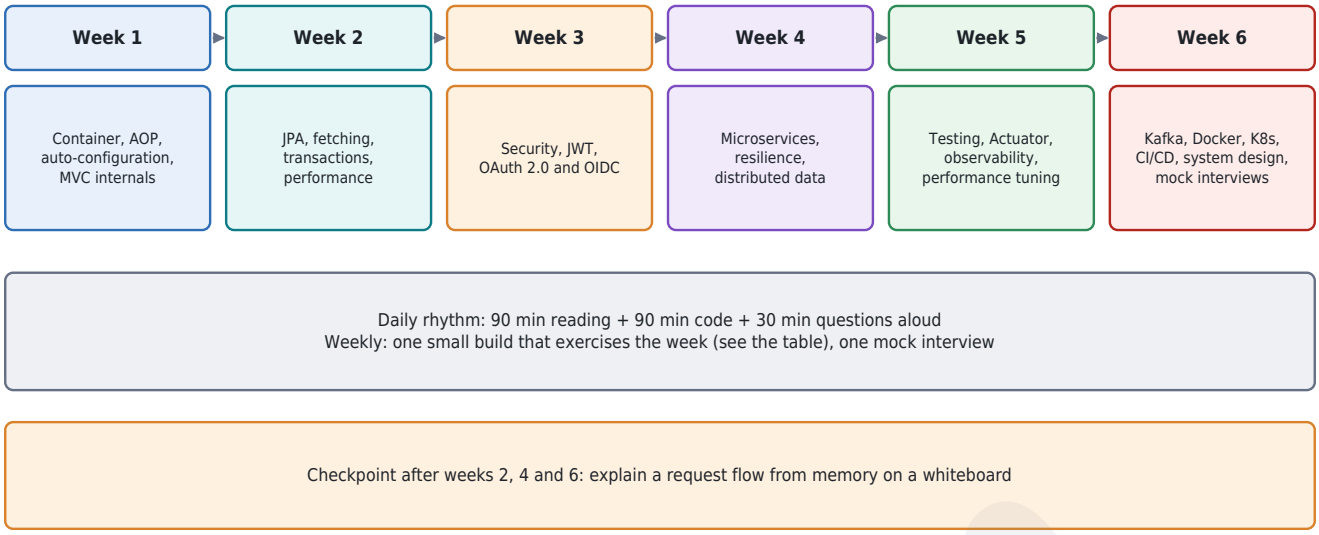


Figure 6.23. Each week combines reading, a coding task and spoken answers; checkpoints after weeks two, four and six test the ability to explain a request flow from memory.

Week	Chapters	Build task	Question bank sections
1	container internals, AOP and proxies, auto-configuration, MVC request lifecycle, HTTP clients	write a custom starter with auto-configuration and properties; reproduce the self-invocation problem and fix it	Container, AOP and auto-configuration; Spring MVC and HTTP clients
2	JPA architecture, mapping, fetching and N+1, transactions, performance, queries	build a list endpoint, show the N+1 in the SQL log, fix it with an entity graph and a projection, add @Version	JPA and Hibernate; Transactions
3	security architecture, authorisation, web attacks, JWT, OAuth 2.0 and OIDC, securing services	implement login, refresh with rotation and a resource server with method security	Security, JWT and OAuth 2.0
4	microservices, synchronous communication, discovery and gateway, resilience, distributed data, tracing	two services with a gateway, a circuit breaker, an outbox and a saga	Microservices and resilience
5	unit and slice tests, Testcontainers, Actuator and Micrometer, logging, performance tuning	full test suite with Testcontainers; a dashboard with RED metrics and one alert; a JFR recording	Testing, Actuator and performance
6	Kafka, RabbitMQ and caching, packaging, Kubernetes, CI/CD, system design, capstone	containerise, deploy with Helm, pipeline in GitHub Actions; two mock design interviews	Kafka, messaging and caching; Docker, Kubernetes and CI/CD

Weekly schedule.

Method for each study session. Read the concept section with the diagram, close the guide and redraw the diagram from memory, then run the worked example and change one thing to break it (remove a timeout, add a self-invocation, disable a unique constraint) and observe the failure named in the problems section. A mechanism that has been broken deliberately is remembered better than one that has only been read.

Interview rehearsal. From week 3 onward, answer five random questions from the bank every day with a timer, aloud. Record the answer, and check that it contains a class or property name, the order of steps, and a pitfall. For the design scenarios, use the template: requirements, estimates, API, data model, diagram, bottlenecks, failure modes.

Portfolio. The final project demonstrates the guide better than any answer: a multi-service application with CI, containers, tests with Testcontainers, security with JWT, a Kafka flow with an outbox, and a README that documents the decisions and the measured behaviour. A hiring engineer reads the repository for tests, commit history and the handling of failure cases before reading the code in detail.

KEY POINTS

The cheat sheets are indexes into the guide, not substitutes for it; a row is useful only when the mechanism behind it is understood.

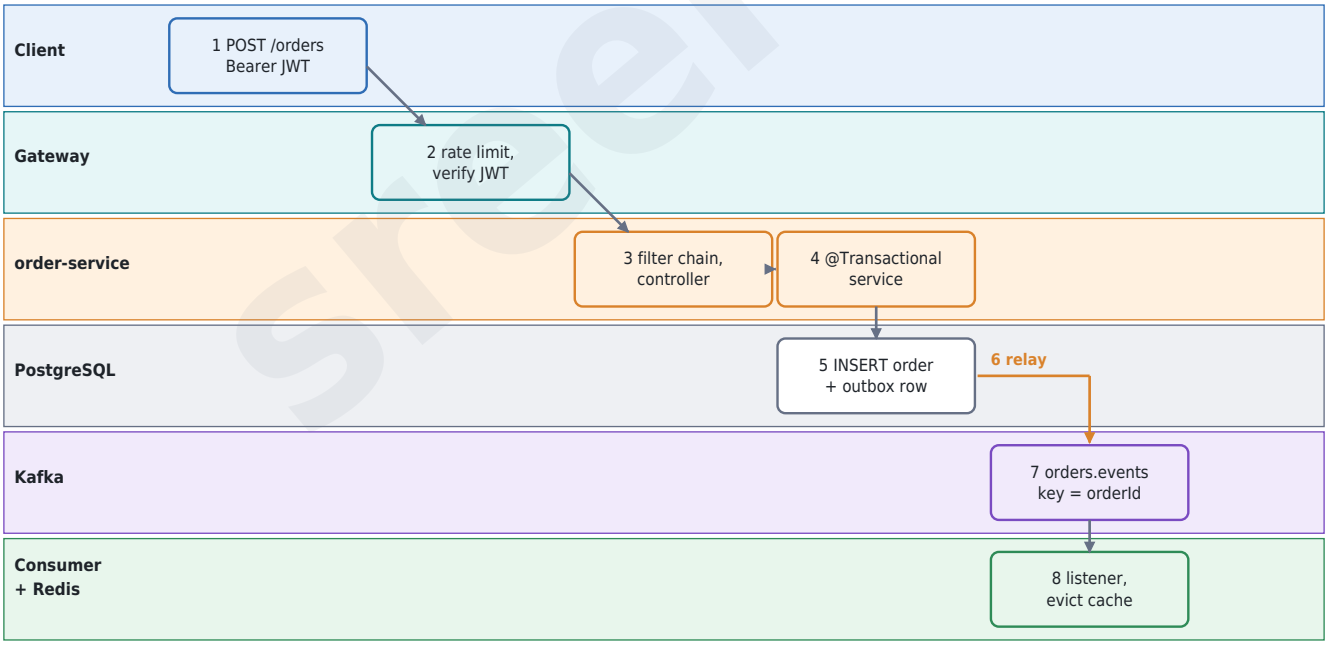
- Study in the order container, data, security, distribution, quality, delivery; later topics depend on proxies and transactions.
- Break every worked example once; the failure modes are the interview material.
- Rehearse aloud with a timer; structure beats volume.

6.11 Putting it Together: One Request through a Spring Boot Microservice

Every chapter of this guide describes one mechanism in isolation. A production request passes through all of them in a few milliseconds, and a defect at any point appears to the user as the same symptom: a slow or failed response. This chapter follows a single `POST /api/orders` from the client to the consumer that reacts to the resulting event, naming at each step the component involved, the Spring class that implements it, the failure that can occur there, and the place to look when it does. It is the integration exercise for the whole guide and also the structure of a strong answer to the interview question "what happens when a request arrives at your service?".

6.11.1 Concept and mechanism

The scenario: a client submits an order with a bearer token. The request is rate limited and authenticated at the gateway, routed to `order-service`, processed through the servlet filter chain and `DispatcherServlet`, validated, executed in a database transaction that writes the order and an outbox row, and answered with `201 Created`. Afterwards the outbox relay publishes `OrderCreated` to Kafka, a consumer in another service processes it idempotently and evicts a cache entry. Metrics and a trace cover the whole path.



One traceId (Micrometer Tracing, MDC) is carried through every lane, including the Kafka headers.

Figure 6.24. The numbered steps cross six lanes; the synchronous part ends with the commit and the response, and the Kafka and cache steps are asynchronous consequences of the same transaction.

Step	Component	What happens (Spring class or mechanism)	Typical failure and first place to look
1	client	HTTPS POST with Authorization Bearer token and Idempotency-Key	DNS, TLS, client timeout: client logs, gateway access log

Step	Component	What happens (Spring class or mechanism)	Typical failure and first place to look
2	gateway	RequestRateLimiter (Redis token bucket), JWT validation, route to lb://order-service	429 or 401: gateway metrics, filter order, clock skew
3	Tomcat	acceptor thread, worker thread from the pool, or a virtual thread (Boot 3.2+)	503 or timeouts: tomcat.threads.busy, accept queue
4	filter chain	DelegatingFilterProxy, FilterChainProxy, SecurityFilterChain: BearerTokenAuthenticationFilter, AuthorizationFilter	401 or 403: debug logging of org.springframework.security
5	DispatcherServlet	RequestMappingHandlerMapping, argument resolvers, Jackson read, Bean Validation	400, 415: exception handler and ProblemDetail body
6	controller	calls the service, maps the entity to a DTO	LazyInitializationException on serialisation: fetch plan
7	transaction	proxy, TransactionInterceptor, PlatformTransactionManager	self-invocation, rollback rules: SQL log for BEGIN and COMMIT
8	Hikari	getConnection from the pool, bounded by connectionTimeout	pool exhausted: hikaricp.connections.pending, long transactions
9	Hibernate	persist, flush at commit, INSERT order and INSERT outbox_event	extra statements, batching: SQL log, statistics
10	PostgreSQL	commit, WAL flush	lock waits, slow commit: pg_stat_activity
11	response	201 Created with Location, Jackson write	slow serialisation: trace span for the handler
12	outbox relay	polls with FOR UPDATE SKIP LOCKED, send to Kafka, mark sent	growing outbox table: count of unsent rows
13	Kafka	partition by key orderId, replicate, acknowledge	produce timeouts: producer metrics, broker logs
14	consumer	@KafkaListener, idempotent processing, cache evict	consumer lag, duplicates, DLT traffic
15	observability	Micrometer metrics, trace id in MDC and Kafka headers	missing trace across threads: context propagation

The path of one request and where each step can fail.

6.11.2 Worked example

The controller and service of `order-service` show steps 5 to 9 in code. The entity and repository follow the earlier chapters; the outbox classes are those of the Kafka chapter.

OrderController.java JAVA

```java
package com.example.orders.api;

import com.example.orders.service.OrderService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import java.util.UUID;

@RestController
@RequestMapping("/api/orders")
public class OrderController {

    private final OrderService service;

    public OrderController(OrderService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<OrderView> create(
            @RequestHeader("Idempotency-Key") UUID idempotencyKey,
```

OrderController.java (continued) JAVA

```
            @Valid @RequestBody CreateOrderRequest request) {
        OrderView view = service.place(idempotencyKey, request);
        var location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(view.id()).toUri();
        return ResponseEntity.created(location).body(view);
    }
}
```

CreateOrderRequest.java JAVA

```
package com.example.orders.api;

import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.NotNull;

import java.util.List;
import java.util.UUID;

public record CreateOrderRequest(
        @NotNull UUID customerId,
        @NotEmpty List<@Valid Line> lines) {

    public record Line(@NotNull String sku, @Min(1) int quantity) { }
}
```

OrderPlacementService.java JAVA

```
package com.example.orders.service;

import com.example.orders.api.CreateOrderRequest;
import com.example.orders.api.OrderView;
import com.example.orders.domain.Order;
import com.example.orders.domain.OrderRepository;
import com.example.orders.outbox.OutboxEvent;
import com.example.orders.outbox.OutboxRepository;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.UUID;

@Service
public class OrderService {

    private final OrderRepository orders;
    private final OutboxRepository outbox;
    private final ObjectMapper mapper;
    private final Counter placed;

    public OrderService(OrderRepository orders, OutboxRepository outbox,
                        ObjectMapper mapper, MeterRegistry registry) {
        this.orders = orders;
        this.outbox = outbox;
        this.mapper = mapper;
        this.placed = Counter.builder("orders.placed").register(registry);
    }

    @Transactional
    public OrderView place(UUID idempotencyKey, CreateOrderRequest request) {
        return orders.findByCustomerIdAndIdempotencyKey(request.customerId(), idempotencyKey)
                .map(OrderView::from)                                  // retry: same answer
                .orElseGet(() -> create(idempotencyKey, request));
    }

    private OrderView create(UUID idempotencyKey, CreateOrderRequest request) {
        Order order = orders.save(Order.pending(idempotencyKey, request));
        try {
            outbox.save(new OutboxEvent(UUID.randomUUID(), order.getId().toString(),
                    "orders.events", mapper.writeValueAsString(OrderView.from(order))));
```

OrderPlacementService.java (continued) JAVA

```
        } catch (JsonProcessingException e) {
            throw new IllegalStateException("cannot serialise event", e);   // rolls back
        }
        placed.increment();
        return OrderView.from(order);
    }
}
```

What to notice: the idempotency check and the insert are in one transaction, and a unique constraint on `(customer_id, idempotency_key)` closes the race between two concurrent retries (the loser receives `DataIntegrityViolationException` and the controller advice maps it to a lookup of the existing order); `create` is a private method called within the transactional `place`, which is correct because the transaction is already open; the unchecked `IllegalStateException` rolls back the order and the outbox row together; and the counter is a Micrometer meter without unbounded tags.

6.11.3 Hands-on

A trace of the request in the logs of the two services, with the trace id carried by Micrometer Tracing in the MDC (logging pattern `%5p [${spring.application.name},%X{traceId},%X{spanId}]`). The lines are the typical form and were not produced on the build machine.

typical output (not run on the build machine) TERMINAL

```
INFO [gateway,6f1c2a9d4b7e83150a2c9d7e41b8f003,5a1c0d2e9b7f4412] rate limit ok key=k-117
INFO [order-service,6f1c2a9d4b7e83150a2c9d7e41b8f003,0b7e83150a2c9d7e] POST /api/orders 201 in 38 ms
DEBUG [order-service,6f1c2a9d4b7e83150a2c9d7e41b8f003,0b7e83150a2c9d7e] insert into orders
DEBUG [order-service,6f1c2a9d4b7e83150a2c9d7e41b8f003,0b7e83150a2c9d7e] insert outbox_event
INFO [order-service,6f1c2a9d4b7e83150a2c9d7e41b8f003,3d9a41c07be25f88] relay sent p=4 off=90211
INFO [payment-service,6f1c2a9d4b7e83150a2c9d7e41b8f003,91e0a7d63c5b2284] received p=4 off=90211
```

The same trace id appears in the gateway, in the service, in the relay (the span of the scheduled run links to the stored trace context when the outbox row keeps the `traceparent`) and in the consumer, because Micrometer Tracing propagates the W3C `traceparent` in HTTP and Kafka headers when observation of the Kafka template and listener containers is enabled (`spring.kafka.template.observation-enabled` and `spring.kafka.listener.observation-enabled` in Boot 3.2 and later). A scheduled relay has no incoming request, so it must store the trace context in the outbox row to continue the trace, or it starts a new trace and the link is lost.

6.11.4 Real-world problems and solutions

Problem 1: the order endpoint is slow and the cause is unknown

Scenario. The 99th percentile of `POST /api/orders` rises from 80 ms to 1.8 s after a release; no errors are reported. **Symptoms.** Dashboards show higher `http.server.requests` percentiles; CPU is normal. **Diagnosis.** Follow the path. Open a slow trace and read the span durations: gateway, controller, database. If the database span dominates, check `hikaricp.connections.pending` and `hikaricp.connections.acquire`; if acquisition is slow, the pool is saturated, and `pg_stat_activity` shows connections `idle in transaction`. If the handler span dominates without database time, take two thread dumps ten seconds apart (`jcmd <pid> Thread.print`) and compare the stacks of the worker threads. Enable `spring.jpa.properties.hibernate.generate_statistics=true` temporarily to count statements per request. **Root cause.** The release added a synchronous call to a tax service inside the `@Transactional` method; each request held a connection for the duration of that call, and the pool of 10 connections became the limit. **Fix.** Move the call before the transaction or make the data available through the event flow; add timeouts to the client; keep the transaction to the two inserts. **Prevention.** Load test with realistic dependencies; alert on pool wait time, not only on pool usage.

Problem 2: the order exists but no downstream service reacted

Scenario. Support reports orders that never reach payment. **Symptoms.** Rows in `orders` without a matching payment; no errors in `order-service`. **Diagnosis.** Check the outbox: `SELECT count(*), min(created_at) FROM outbox_event WHERE sent_at IS NULL`. A growing count and an old minimum mean the relay is not publishing: look at the relay logs for `TimeoutException` from the Kafka send, check `@EnableScheduling`, and check whether the relay transaction is blocked by a long-held lock. If the outbox is empty, check consumer lag and the dead-letter topic of the payment service. **Root cause.** The relay method threw on a record larger than `max.request.size`; the transaction rolled back and the same record was retried at the head of the batch every 500 ms, blocking all later records. **Fix.** Handle per-record failures (catch, mark the row as failed with an attempt counter, continue), and publish oversized payloads by reference. **Prevention.** Alert on the age of the oldest unsent outbox row and on the dead-letter topic rate.

6.11.5 Questions to expect

1. **What happens when a request reaches a Spring Boot application?** The connector hands it to a worker thread; the filter chain runs (including the security filters); `DispatcherServlet` finds the handler, resolves arguments, validates, invokes the controller, converts the result; interceptors and exception resolvers wrap the process; the response is written and the thread returns to the pool.
2. **Where are transactions started in that flow, and why not in the controller?** At the service boundary through the proxy; a transaction in the controller would include serialisation and filter time, hold a connection longer and mix web and persistence concerns.
3. **How are duplicate orders from client retries prevented?** An idempotency key stored under a unique constraint, with the original response returned on repetition.
4. **How is the event published without losing it?** Through the transactional outbox, relayed to Kafka with at-least-once delivery.
5. **How is one request traced across services?** Propagation of the `traceparent` header through HTTP and Kafka, correlation in the MDC, and a trace backend; asynchronous hand-offs need context propagation.
6. **What is checked first when latency rises?** Percentiles by endpoint, then traces to locate the slow span, then pool and thread metrics, then thread dumps and SQL statistics.

KEY POINTS

A request is a chain of mechanisms; naming them in order, with the class and the failure at each step, is the strongest demonstration of understanding.

- The synchronous path ends at the database commit; everything after it is asynchronous and at-least-once.
- Idempotency keys, the outbox and idempotent consumers make retries and duplicates harmless at each hop.
- Metrics locate the symptom, traces locate the step, thread dumps and SQL logs locate the cause.

sreram